AGC 046 解説

DEGwer

2020/06/20

For International Readers: English editorial starts on page 6.

A: Takahashikun, The Strider

高橋君のいる初期地点を A とし、1 回の移動後にいる地点を B としましょう。$|AO| = |BO|$ かつ角 AOB の大きさが X 度となるような点 O を、高橋君の初期地点から見て左側にあるように取ります。このとき、各移動後に高橋君のいる位置は、すべて O を中心とする半径 $|AO|$ の円周上に載ることがわかります。結局、高橋君の移動は円周上を X 度ぶんだけ移動するものと見做せるので、kX が 360 の倍数となる最小の k が答えになります。

B: Extension

各盤面の塗られ方を、ひとつの操作列に対応させることを考えましょう。一番上の行に黒く塗られたマスが複数ある (もしくは存在しない) 場合、最後の操作は右に列を追加する操作です。逆に、一番上の行に黒く塗られたマスが 1 つだけある場合、一番上の行を取り除いてできる盤面も、初期状態から作ることができることが分かります。よって、初期状態から作ることができる各盤面から、以下のようなアルゴリズムで一意な操作列を復元することができます。

- 一番上の行に黒く塗られたマスが複数ある場合、一番右の列を取り除いてできる盤面を作る操作列を求めたあと、右に列を追加する操作を付け加える
- そうでない場合、一番上の行を取り除いてできる盤面を作る操作列を求めた後、上に行を追加する操作を付け加える

このような操作列は、i 行 j 列のマス目であって一番上の行に黒く塗られたマスが複数あるような/そうでないような盤面の個数をそれぞれ $\mathtt{DP}[i][j][0], \mathtt{DP}[i][j][1]$ としたような動的計画法を用いて $O(CD)$ 時間で数え上げることができます。

C: Shifts

各 0 を文字列の区切りと考えましょう。0 が A 個あるとすれば、0 によって文字列は両端を含めた $A+1$ 個の部分に分割できます。このとき、問題で与えられている操作は、ある区間に存在する 1 を 1 個減らし、それより左の区間に 1 を 1 個増やすものととらえることができます。

$\mathtt{DP}[i][j][k]$ を、後ろから i 個目の区間まで見て、j 回の操作を行い、それより後ろの区間に合計で 1 を k 個増やしてできるような列の個数とすれば、この DP は $O(N^4)$ 時間で更新可能です。各区間に対してまとめて操作を行うのではなく、1 回ずつ分けて行うように気を付けて実装すれば、この DP は $O(N^3)$ 時間に高速化できます。

D: Secret Passage

S の文字の中で、S の先頭の文字として操作を行われたか、あるいは 2 文字目として操作を行われた結果取り除かれたかした文字を「動いた文字」と呼ぶことにします。動いた文字の中で取り除かれなかったものを「挿入された文字」と呼びます。S の動いた文字全体は、S の接頭辞をなします。作れる文字列は、挿入された文字を、動かなかった文字たちの間 (または端) に自由に挿入することで得られるものです。

できる列を T とします。S の接尾辞であって T の部分列になっているようなものの中で最長のものを S' とします。まず、定義より、S' に属していない文字はすべて動いた文字であることが分かります。逆に、S' の文字が動いた文字とならないような操作列であって T をつくるものが存在することが証明できます。実際、S' の先頭 k 文字を追加で動かすことで T を作ることができたとします。このときに挿入された文字の多重集合から S' の先頭 k 文字を除いたものを X とします。 X が挿入された文字の多重集合となるような、S' の文字を動かさない操作列が存在することを証明すればよいです。これは、S' の先頭 k 文字を追加で動かして T を作る場合 S' 以外の挿入される文字の多重集合は X を含まなければならず、X を含む多重集合を挿入される文字の多重集合とするならば、適切に操作を追加することで X を挿入される文字の多重集合ともできることから分かります。

以上より、S' を全通り試し、それぞれの場合の数を足し合わせることで答えを求めることができます。先頭 k 文字を動かすことで挿入される文字の多重集合を 0 を i 個、1 を j 個にできるかどうかを $\mathtt{DP}[k][i][j]$ とする動的計画法によって挿入される文字の多重集合としてありうるものを列挙し、0 を i 個と 1 を j 個挿入することで S の末尾 k 文字から得られる相異なる文字列の個数を $\mathtt{DP}'[k][i][j]$ とする動的計画法によって作れる文字列の個数を数えます。これらを組合わせれば、この問題を解くことができます。どちらの動的計画法も適切な実装によって $O(N^3)$ 時間で動くように実装できるので、時間計算量は $O(N^3)$ 時間となります。

E: Permutation Cover

以下、$[n]$ で集合 $\{1, \ldots, n\}$ を表します。

■ステップ 1. まず、条件を満たす列が存在するための必要十分条件は、A_i たちの中の最大値が A_i たちの中の最小値の 2 倍以下であることです。まずはこれを証明しましょう。まず、A_i たちの中の最大値/最小値を取るような i を 1 つずつとって x, y とします。$A_x > 2A_y$ なら、できる列の x と y だけを取り出してできる列には、x が 3 個 (または端に 2 個) 連続するような箇所が存在します。このような箇所があれば、それらの中の真ん中 (端) の要素はどの順列をなす連続部分列にも含まれず、この条件は必要です。逆に、任意の集合 $S \subseteq [K]$ に対し、S の要素を 2 個ずつ、$[K] \setminus S$ の要素を 1 個ずつ含む列であって、どの項も順列をなすある連続部分列に含まれるようなものが作れます。$A_x \leq 2A_y$ なら、このような列を並べたものとして条件を満たす列を作れるので、この条件は十分条件でもあります。

■ステップ 2.　ステップ 1. の条件が満たされているとして、条件を満たす辞書順最小の列を構成する方法を考えましょう。どの要素も順列を成すある連続部分列に含まれるような列 P を考えます。特に、P の最後 K 項は順列をなします (この順列を Q とします)。また、B_i でこれから追加する必要のある整数 i の個数を表すことにします。 P にまだ追加していない要素を追加することで全体を条件を満たす列にできる必要十分条件は、以下であることが示せます。

- B_i たちの中の最大値を x とし、最小値を y として、$x \leq 2y+1$ である
- $x = 2y+1$ なら、Q の中では、$B_i = x$ なるどの i も $B_i = y$ なる任意の i よりも前に現れる

1 つ目の条件の必要性は、これから追加すべき項からなる列に対してステップ 1. と同様の議論を行うことによって証明できます。2 つ目の条件の必要性も、$B_i = x$ なる i が P で最後に現れる位置以降の列にこれから追加すべき項を追加したものに対して同様の議論を行うことで証明できます。十分性は、$B_i = x$ なる i が Q の先頭 s 文字にすべて現れるような最小の s を取れば、P に Q の先頭 s 項を追加したような列 R を考えれば、R のどの項も順列をなるある連続部分列に含まれ、かつこれから追加すべき項たちの個数はステップ 1. の条件を満たすことから分かります。

■アルゴリズム　現在の列がステップ 2. の条件をみたすとして、これをどの要素も順列をなすある連続部分列に含まれるようなより長い列に拡張することを考えましょう。現在の列を何文字伸ばせばはじめてそのような列になるかを全通り試し、各候補に対してステップ 2. の条件を満たすような伸ばし方が存在するか判定し、存在する場合は辞書順最小のものを求めます (これは簡単な貪欲法で可能です)。こうして求めた伸ばし方のうち、辞書順で最小のものを取って列を拡張すればよいです。この操作を追加すべき項がなくなるまで繰り返すことで、この問題を解くことができます。時間計算量は、A_i たちの合計を S として、$O(SK^2)$ 時間です。

F: Forbidden Tournament

以下、問題文で与えられている禁止部分グラフを H とします。すなわち、H は 4 頂点のグラフであって、そのうち 3 点が三角形をなし、その 3 点すべてから残りの 1 点に向かって辺が出ているようなトーナメントグラフです。また、G が H を誘導部分グラフに持つとき、G が H を **含む** と呼びます。以下、G の構造を、ステップに分けて眺めていきましょう。適宜、グラフの頂点集合とそれが誘導するグラフを同一視します。

■ステップ 1.　トーナメントグラフ G を固定します。G に入次数 0 の頂点 x があるとします。このとき、G が H を含まず入次数の最大値が K 以下であることは、G から x を取り除いたグラフが H を含まず入次数の最大値が $K-1$ 以下であることと同値です。このようなケースは N が 1 小さい場合の問題を解いて求めることにして、以下、G のどの頂点の入次数も 1 以上であると仮定します。

■ステップ 2.　G の頂点 v をひとつ任意に取って固定します。Y を $v \to w$ 辺があるような頂点 w の集合とし、$X = V(G) \setminus Y$ とします ($v \in X$ に注意)。もし X がサイクルを含めば、それは三角形を含み (トーナメントグラフの性質)、その三角形と v を併せて H を得ます。よって、X の頂点は、任意の $i < j$ に対して $x_i \to x_j$ 辺があるように $x_1, \ldots, x_k = v$ と番号づけられます。

■ステップ 3. 入次数 0 の頂点がないので、Y の頂点から X の頂点に出る辺は必ず存在します。このような辺を任意に取り、$w \to u \quad (w \in Y, u \in X)$ とします。$w \to w'$ 辺の存在する頂点 $w' \in Y$ が存在したとします。このとき、$u \to v, v \to w, w \to u, w \to w', v \to w'$ の 5 辺が存在するので、G が H を含まないためには $w' \to u$ 辺が存在する必要があります。この議論を繰り返し行えば、Y 内で w から到達できるすべての頂点 w' について $w' \to u$ 辺が存在する必要があることが分かります。Y 内で w から到達できる頂点たちの集合がサイクルを含めば、それは三角形を含み、その三角形と u を併せて H を得ます。よって、Y 内で w から到達できる頂点たちの集合はサイクルを含みません。逆に、Y 内に w から到達できないようなサイクルがあれば、その中の三角形と w を併せて H を得ます。すなわち、Y もサイクルを含まず、Y の頂点を任意の $i < j$ に対して $y_i \to y_j$ 辺があるように $y_1, \ldots, y_l$ と番号づけることができます。

■ステップ 4. X 内および Y 内の辺の向きはすべて定まったので、以下 X と Y の間の辺の向きを考えていきましょう。ステップ 3. での考察により、$y_j \to x_i$ 辺があるとき、任意の $j' = j, \ldots, l$ に対し、$y_{j'} \to x_i$ 辺が存在することが分かります。このことより、仮に $x_1 \to y_l$ 辺が存在すれば、任意の $j' = 1, \ldots, l$ に対し $x_1 \to y_{j'}$ 辺が存在することになり、x_1 の入次数が 0 になって矛盾します。よって、$y_l \to x_1$ 辺が存在します。

■ステップ 5. $2 \leq i \leq k$ として、$x_i \to y_l$ 辺が存在すると仮定します。このとき、もし $i < i'$ に対して $y_l \to x_{i'}$ 辺が存在すれば、$x_i, x_{i'}, y_l, x_1$ の 4 点が H を誘導します。よって、$x_i \to y_l$ 辺が存在すれば、任意の $i' = i, \ldots, k$ に対して $x_{i'} \to y_l$ 辺が存在します。t を $y_l \to x_t$ 辺が存在するような最大の整数としてとり、$1 \leq i \leq t$ とします。$1 \leq j \leq l-1$ とし、$x_i \to y_j$ 辺が存在するとします。このとき、x_i, y_j, y_l は三角形をなします。特に、$i < i' \leq k$ に対し、G が H を含まないためには $x_{i'} \to y_j$ 辺か $x_{i'} \to y_l$ 辺のどちらかが必要です。ステップ 3. での考察により、$x_{i'} \to y_l$ 辺が存在するなら $x_{i'} \to y_j$ 辺も存在するため、結局 $x_{i'} \to y_j$ 辺が必要なことがわかります。

■ステップ 6. $k \times l$ のグリッドであって、$y_j \to x_i$ 辺があるような (i, j) に対して i 行 j 列目のマスを黒く塗ったものを考えます。ここまでの考察により、このグリッドは以下を満たす必要があることが分かります。

- (i, j) が黒く塗られているなら、$i' < i$ に対して (i', j) も黒く塗られている
- (i, j) が黒く塗られているなら、$j < j'$ に対して (i, j') も黒く塗られている
- $(1, l)$ は黒く塗られている
- 任意の j に対し、(k, j) は黒く塗られていない

逆に、上の 2 つの条件を満たせば G は H を誘導部分グラフに持たないことは、H のサイクルがどこに現れるかで場合分けをすることで示すことができます。よって、黒く塗られているマスと塗られていないマスの境界は、グリッドの対角線上の 2 頂点を (マンハッタン距離最小で) 結ぶパスとして表すことができます。入次数の条件は、このパスがある 2 つの階段状領域の点を含まないという条件で記述することができます。

■アルゴリズム. 結局、k, l の値を決め打ちすれば、この問題はグリッド上に与えられた 2 点間をマンハッタン距離最小のパスで結ぶ方法の個数を数える問題に帰着されることが分かりました。頂点 $x_1, \ldots, x_k$ と $y_1, \ldots, y_l$ の取り方が $N!$ 通りあり、条件を満たす各グラフ G について v の選び方は N 通り存在するので、このようなパスの個数を $(N-1)!$ 倍すると求める場合の数が得られます。また、このようなパスの個数は動的計画法で $O(kl)$ 時間で求めることができます。k, l の定め方は $O(N)$ 通り、最初に入次数 0 の頂点を取り

除く回数の候補が $O(N)$ 通り存在するので、時間計算量は合計で $O(N^4)$ となります。

A: Takahashikun, The Strider

Let A be Takahashi's initial position, and B be his position after one move. Let O be the point to the left side of Takahashi's initial position such that $|AO| = |BO|$ and the angle AOB is X degrees. Then, we can see that Takahashi's position after every move lies on the circle of radius $|AO|$ centered at O. We can now regard Takahashi's move as moving X degrees on the circle, so the answer is the minimum k such that kX is a multiple of 360.

B: Extension

Let us correspond each way of painting the grid to one sequence of operations. If there are multiple (or no) black squares in the top row, the last operation is the addition of a column to the right end. On the other hand, if there is just one black square in the top row, we can see that the grid without the top row is also reachable from the initial state. Thus, from each state of the grid that is reachable from the initial state, we can restore a unique sequence of operation with the following algorithm:

- If there are multiple black squares in the top row, find the sequence of operations resulting in the grid without the rightmost column and add a column to the right end.
- Otherwise, find the sequence of operations resulting in the grid without the top row and add a row to the top.

With Dynamic Programming where $\texttt{DP}[i][j][0], \texttt{DP}[i][j][1]$ are the number of grids with i rows and j columns where there (are / are not) multiple black squares at the top row, we can count such sequence of operations in $O(CD)$ time.

C: Shifts

Consider each 0 as a separator of the string. If the string has A 0s, they separate the string into $A+1$ parts, including both ends. Then, the operation given in the problem can be seen as removing one 1 from a segment and adding one 1 to a segment to the left.

Let $\texttt{DP}[i][j][k]$ be the number of sequence that can be obtained considering the 1-st, ..., i-th segment from the back, having done j operations, and adding k 1s to the later segments. We can calculate this table in $O(N^4)$ time. We can improve it to $O(N^3)$ time by careful implementation: instead of collectively applying the operation to the segments, we can handle them separately.

D: Secret Passage

We call a character in S a "moved" character if we did the operation when that character was the first character of S, or the second character and got removed. We call a moved but unremoved character an "inserted" character. The set of all the moved characters in S forms a prefix of S. A string we can make

is a string we can obtain by freely inserting the "inserted" characters between unmoved characters (or to the ends of the string).

Let T be an obtainable string. Let S' be the longest suffix of S that is a subsequence of T. First, from the definition, we can see that every character not belonging to S' is a moved character. On the other hand, we can prove that there exists a sequence of operations resulting in T such that no character in S' is a moved character. Assume that we were actually able to make T by additionally moving the first k characters in S'. Let X be the multiset of the inserted characters here except the first k characters in S'. Now we have to prove that there is a sequence of operations without moving the characters in S' such that X will be the multiset of the inserted characters. This can be seen from the following fact: when we additionally move the first k characters in S' to make T, the multiset of the inserted characters not in S' must contain X, and if the multiset of the inserted characters contains X, we can make X the multiset of the inserted characters by properly adding operations.

Therefore, we can try all possible S' and sum the count for each case to find the answer. We will use Dynamic Programming. Let $\mathtt{DP}[k][i][j]$ be whether the multiset of the inserted character can be i 0s and j 1s by moving the first k characters. We will use it to enumerate the possible multisets of inserted characters. Also, let $\mathtt{DP}'[k][i][j]$ be the number of different strings that can be obtained from the last k characters of S by inserting i 0s and j 1s. We use it to count the strings obtained. The problem can be solved by the combination of these two. Both tables can be computed in $O(N^3)$ time when properly implemented, so the time complexity of our solution is $O(N^3)$.

E: Permutation Cover

Below, let $[n]$ denote the set $\{1, \ldots, n\}$.

■Step 1. First, there exists a sequence satisfying the condition if and only if the maximum element among A_i is at most twice the minimum element among A_i. Let us prove it now. Let x and y be the indices i such that A_i is maximum and minimum. If $A_x > 2A_y$, the sequence obtained by extracting only xs and ys from the resulting sequence contains a part where x appears three times in a row (or twice at one end). If such a part exists, the middle one among them (or the one at the end) is not contained in any contiguous subsequence that is a permutation, so this condition is necessary. On the other hand, for any set $S \subseteq [K]$, we can form a sequence containing each element in S twice and each element in $[K] \setminus S$ once such that every term is contained in some contiguous subsequence that is a permutation. If $A_x \leq 2A_y$, we can satisfy the condition by forming such a sequence, so this condition is also sufficient.

■Step 2. Assuming that the condition in Step 1. is satisfied, let us consider the way to construct the lexicographically smallest such sequence. Consider a sequence P where every element is contained in some contiguous subsequence that is a permutation. Notably, the last K terms in P form a permutation (let Q be this permutation). Also, let B_i be the number of times the integer i currently needs to be added. We can show that we can satisfy the condition by adding P the elements that are not yet added if and only if:

- Let x and y be the maximum and minimum among B_i. Then, $x \leq 2y + 1$ holds.
- If $x = 2y + 1$, every i such that $B_i = x$ appears earlier in Q than every i such that $B_i = y$ does.

We can prove the necessity of the first condition by making an argument similar to that in Step 1. for the sequence of terms to be added. We can also prove the necessity of the second condition by making a similar argument for the part of the sequence after the last occurrence of i such that $B_i = x$ in P, with the terms to be added appended at the end. The sufficiency can be seen as follows. Let us take the minimum s such that the first s characters of S contains every i such that $B_i = x$ appear. Then, consider the sequence R obtained by appending the first s terms of Q to P. Now, every term in R is contained in some contiguous subsequence that is a permutation, and the numbers of terms that should be added satisfy the condition in Step 1., so the necessity is shown.

■Algorithm Assuming that the current sequence satisfies the condition in Step 2., let us consider making it a longer sequence such that every term is contained in some contiguous subsequence that forms a permutation. We will try all candidates of the minimum number of characters we need to add to the current sequence so that we obtain a desirable sequence. Then, for each candidate, we will determine whether there is a way to extend the sequence satisfying the condition in Step 2., and find the lexicographically smallest such one if it exists (we can do it with a simple greedy algorithm). Among these ways to extend the sequence, we will use the lexicographically smallest one and extend the sequence. The problem can be solved by repeating this procedure until there are no more terms to be added. The time complexity of our solution is $O(SK^2)$, where S is the sum of A_i.

F: Forbidden Tournament

F: Forbidden Tournament

Below, let H be the forbidden subgraph given in the statement. That is, H is a tournament with four vertices, three vertices forming a triangle, and an edge going from each of them to the remaining vertex. We also say G **contains** H when H is inducible in G. Let us observe the structure of G in steps. Below, when appropriate, we will treat a vertex set of the graph as the graph induced by that set.

■Step 1. Let us consider a fixed tournament G. Assume that G has a vertex x whose in-degree is 0. Then, the condition "G does not contain H, and the maximum in-degree of a vertex in G is at most K" is equivalent to the following: "removing x from G results in a graph that does not contain H, and the maximum in-degree of a vertex in that graph is at most $K - 1$." Let us solve such a case by solving the problem for one less N, and assume below that the in-degree of every vertex in G is at least one.

■Step 2. Let us choose a vertex in G arbitrarily and fix it. Let Y be the set of vertices w such that the edge $v \to w$ exists, and let $X = V(G) \setminus Y$ (note that $v \in X$). If X contains a cycle, it contains a triangle (a property of a tournament), and that triangle plus v form H. Thus, we can number the vertices in X so that the edge $x_i \to x_j$ exists for every $i < j$.

■Step 3. Since there is no vertex whose in-degree is 0, there always exists an edge going from a vertex in Y to a vertex in X. Let us choose such an edge arbitrarily and let it be $w \to u \quad (w \in Y, u \in X)$. Assume that there exists a vertex $w' \in Y$ such that the edge $w \to w'$ exists. Then, the five edges $u \to v$, $v \to w$, $w \to u$, $w \to w'$, and $v \to w'$ exist. Thus, for G to not contain H, the edge $w' \to u$ has to exist. By repeating this argument, for every vertex w' in Y that are reachable from w, the edge $w' \to u$ has to exist. If the set of vertices in Y that are reachable from w contains a cycle, it contains a triangle, and that triangle plus u form H. Thus, the set of vertices in Y that are reachable from w contains no cycle. On the other hand, if there is a cycle in Y that is unreachable from w, a triangle in it plus w forms H. That is, Y also contains no cycle, and we can number the vertices in Y $y_1, \ldots, y_l$ so that the edge $y_i \to y_j$ exists for every $i < j$.

■Step 4. The directions of the edges in X and Y are determined, so let us consider those between X and Y. From the observation in Step 3., when the edge $y_{j'} \to x_i$ exists, the edge $y_{j'} \to x_i$ exists for all $j' = j, \ldots, l$. From this fact, if the edge $x_1 \to y_l$ existed, the edge $x_1 \to y_{j'}$ would exist for all $j' = 1, \ldots, l$, resulting in the in-degree of x_1 being 0, which is a contradiction. Thus, the edge $y_l \to x_1$ exists.

■Step 5. Let $2 \leq i \leq k$, and assume that the edge $x_i \to y_l$ exists. If the edge $y_l \to x_{i'}$ exists for $i < i'$, the four vertices x_i, $x_{i'}$, y_l, and x_1 induces H. Thus, if the edge $x_i \to y_l$ exists, the edge $x_{i'} \to y_l$ exists. Let t be the largest integer such that the edge $y_l \to x_t$ exists, and let $1 \leq i \leq t$. Also, let $1 \leq j \leq l-1$ and the edge $x_i \to y_j$ exists. Then, x_i, y_j, and y_l form a triangle. Particularly, for $i < i' \leq k$, either the edge $x_{i'} \to y_j$ or the edge $x_{i'} \to y_l$ has to exist for G to not contain H. From the observation in Step 3., when the edge $x_{i'} \to y_l$ exists, the edge $x_{i'} \to y_j$ also exists. Thus, eventually, the edge $x_{i'} \to y_j$ has to exist.

■Step 6. Consider a $k \times l$ grid where the square at the i-th row and j-th column is painted black for (i, j) such that the edge $y_j \to x_i$ exists. From the observations above, we can see that the grid has to satisfy the following:

- If (i, j) is black, also (i', j) is black for $i' < i$.
- If (i, j) is black, also (i, j') is black for $j < j'$.
- $(1, l)$ is black.
- (k, j) is unblackened for all j.

On the other hand, we can show that H is not inducible in G if the above conditions are satisfied, by a case-by-case analysis on where the cycle is formed in H. Thus, the boundary between black squares and unblackened squares can be represented as a path connecting two vertices on the diagonal of the grid (with the minimum Manhattan distance). The condition on the in-degrees can be described as a condition that the path does not contain a point in some two stair-like regions.

■Algorithm. Eventually, we have seen that the problem can be reduced to counting the number of ways to connect two given points in a grid by a path with the minimum Manhattan distance. There are $N!$ ways to choose the vertices $x_1, \ldots, x_k$ and $y_1, \ldots, y_l$, and there are N choices of v for each graph G satisfying the condition, so we can find the answer by multiplying the number of paths by $(N-1)!$. We can count such paths in $O(kl)$ time with Dynamic Programming. There are $O(N)$ ways in which k and l are determined, and $O(N)$ candidates for the number of times we remove a vertex whose in-degree is 0 in the beginning, so the total time complexity is $O(N^4)$.

AGC045 解説

writer : maroonrk

For International Readers: English editorial starts on page 8.

A : Xor Battle

この解説では，0-based index を用います．次のような DP を考えます．

- $DP[i]$ = 次の条件を満たす整数 t の集合．
  - ラウンド i(0-based) の直前で $x = t$ であるとき，ここからゲームを続けて，最終的に $x = 0$ になる．

この DP を後ろから埋めていきます．まず，$DP[N] = \{0\}$ です．
次に $DP[i]$ の遷移を考えます．$S_i = 0$ の場合は簡単で，

- $DP[i] = DP[i+1] \cup \{v \oplus A_i | v \in DP[i+1]\}$

です．
つぎに $S_i = 1$ の場合です．まず $A_i \in DP[i+1]$ の場合を考えます．ラウンド i の時点で $x \notin DP[i+1]$ なら，何もしなければよいです．逆に，$x \in DP[i+1]$ の場合，$x \oplus A_i \in DP[i+1]$ なので，操作をしても意味がありません．よって $DP[i] = DP[i+1]$ となります．
$A_i \notin DP[i+1]$ の場合，どんな x に対しても，x または $x \oplus A_i$ の少なくとも一方は $DP[i+1]$ に含まれません．よって，$DP[i] = \emptyset$ です．
この DP は，xor の基底を管理することで，効率的に計算することができます．
計算量は 1 ケースあたり $O(N \log(\max(A_i)))$ になります．
解答例

B : 01 Unbalanced

この問題には 2 つの解法があります．1 つは二分探索を用いた素朴な解法です（writer の提案時の解法です）．ここでは，もう 1 つの賢い方法を解説します (この解法は tester の latte さんとの議論で生まれました)．

$0, 1$ をそれぞれ $-1, +1$ に対応させてみます．すると問題は，累積和の最大と最小の差を最小化しろ，となります．累積和の最大が M を超えないようにした際の，累積和の最小としてありうる値の最大値を $f(M)$ とします．累積和の最大としてありうる値の最小値を Z とすると，この問題は，$\min\{M - f(M) : Z \leq M\}$ を求める問題になります．

まず Z を求めます．これは簡単で，全ての '?' を -1 にすればよいです．

次に，ある M が与えられたときに $f(M)$ を求めることを考えます．これは，先頭から貪欲を行えば良いです．具体的には，まずすべての '?' を -1 だとします．つぎに，'?' を先頭から見ていって，そこを $+1$ にしても累積和の最大が M を超えないかをチェックし，超えないなら $+1$ に変えます．この貪欲の正当性は，あえて $+1$ にしない解に比べたときに，解が悪化しないことから従います．

また，この貪欲アルゴリズムの様子を眺めると，M が 2 増えた時，-1 から $+1$ に変わる '?' は高々一個です．よって，$f(M+2) \leq f(M)+2$ であることがわかります．これより，$M - f(M) \leq (M+2) - f(M+2)$ とわかります．よって，$\min\{M - f(M) : Z \leq M\} = \min(Z - f(Z), (Z+1) - f(Z+1))$ です．

$f(M)$ は上述の貪欲で求められます．計算量は $O(N)$ です．

解答例

解答例 (二分探索)

C : Range Set

明らかに，全部 1 にする，という操作が可能なので，01 を反転させても答えは変わりません．そこで，$A \leq B$ を仮定します．

ある 01 列が，操作後の x としてあり得るための必要十分条件は，次のようになります．

- A 個以上連続する 0 を全て 1 に変える．その後，1 が B 個以上連続した部分が存在する．

十分性は，次のように示せます．まず，1 が B 個並ぶ部分を固定すると，それ以外の部分は，自由に変更できます（端の方から埋めていくイメージです）．次に，選んだ B 個を 1 に変更します．最後に，A 個連続した 0 に変える，という操作を適宜行えばよいです．

必要性は，操作の逆（連続する A, B 個の $0, 1$ を選んで自由に変える）を考えるとわかります．

操作後の x としてありえないものを数えます．まず，0 が A 個以上連続せず，1 が B 個以上連続しないような列の個数は，DP を簡単に求まります．あとは，1 が連続する部分に A 個以上連続する 0 をどのように入れるかを考えれば良いです．

これは，予め各 n について，1 が n 個連続する部分に 0 を A 個以上連続させるように入れる方法が何通りあるかを数えておけば良いです．そうすれば，上記の DP の遷移で n 個の 1 を付け足すときに，今求めた値をかければ良いです．

これらの計算は全て $O(N^2)$ で行えます．

解答例

D : Lamps and Buttons

ここでは，順列 p で i が含まれるサイクル，と言った場合，$i \to p_i \to p_{p_i} \to \cdots \to i$ というサイクルを指すことにします．

すぬけくんの最適な戦術は，以下のとおりです．

- ランプ i が点灯しており，p_i の値が不明な最小の i を探し，ボタン i を押す．
- そのような i がない場合は負け．
- $p_i = i$ とわかったら負け，
- そうでない場合，i が含まれるサイクル上のランプを全部点灯させることができる．
- サイクルを一周したあと，全ランプが点灯していたら勝ち．
- そうでなければ，上の操作を再び繰り返す．

操作の最初の段階で押すボタンには本来自由度がありますが，p がランダムに生成されていることから，最小の i を使うとしてよいです．

この戦略ですぬけくんが勝つ条件を，順列の条件で言い換えると，以下のようになります．

> $t \leq A$, $p_t = t$ なる最小の t をとる．そのような t が存在しない場合，$t = A + 1$ とする．このとき，全ての $A + 1 \leq i \leq N$ について，i の含まれるサイクル上に，$t - 1$ 以下の値が存在する．

t の値を決め打ったとします．するとまず，$i < t$ を満たす全ての i について $i \neq p_i$ という条件が発生します．この条件は，包除原理を使って処理することにします．すると結局，次の問題が解ければ良いです．

> 整数 a, b, c が与えられる．長さ $a + b + c$ の順列であって，次の条件を満たすものは何個あるか．
>
> - 全ての $a + 1 \leq i \leq a + b$ について，i の含まれるサイクル上に a 以下の値が存在する．

これは次のように数えられます．まず，長さ a の順列は $a!$ 通りあります．つぎに，この順列のサイクルに，b 個の要素を追加することを考えます．このうち i 個目の要素を入れる場所は，$a + i - 1$ 通りあります．最後に，c 個の要素を追加します．追加する際は，既存のサイクルに挿入する場合と新しいサイクルを作る場合があり，合わせると，i 個目の要素を入れる場所は $a + b + i$ 個あります．これらの積をとると，$(a + b + c)! \times a/(a + b)$ になります．この値は，$O(N)$ 時間の前処理をしておくと，$O(1)$ で求めることができます．

よって，この問題は全体で $O(A^2 + N)$ 時間で解けます．

解答例

E : Fragile Balls

$A_i \to B_i$ と辺を貼った有向グラフを考えます．以降，”連結成分” と言った場合は，辺を無向だと考えた場合のグラフの連結成分を指すことにします．”強連結成分” と言った場合は，元の有向グラフの強連結成分を指します．

$C_i = 1$ のケースを考えます．この時，目標が達成可能であることと，グラフに長さ 2 以上のサイクルであるような連結成分が存在しないことは同値です．まずこれを示します．

長さ 2 以上のサイクルが存在したとします．このサイクル上で操作を行うためには，このサイクル上の頂点へ，別の連結成分からボールを持ってくる操作が 1 回必要です．しかし，$C_i = 1$ なので，この操作を行った場合，サイクル上へ移動してきたボールは元の連結成分に戻ることができません．よって目標達成は不可能です．

長さ 2 以上のサイクルが存在しないとします．まず，長さ 1 のサイクルについては，何もしなくて良いです．サイクルでない連結成分について考えます．この連結成分を強連結成分分解してみます．トポロジカル順序最小の強連結成分を任意に 1 つ取ります．全ての頂点の入次数が 1 以上であることに注意すると，この強連結成分内に出次数 2 以上の頂点が必ず存在することが示せます．この頂点を任意に 1 つとって v と呼びます．強連結成分の定義より，この強連結成分内で v を通る（辺）単純サイクルが存在します．このサイクルに沿って v から始めてボールを移動させたとします．すると以降，このサイクル上の頂点からは，自由にボールを動かせるとしてよいです．あとは，このサイクルから到達できる全ての頂点について，芋づる式にボールを動かしていくことができます．

トポロジカル順序が最小になりうる全ての強連結成分について同じことを行うと，結局，この連結成分内ですべてのボールを動かすことができます．よって示されました．

今，$C_i = 1$ のケースを解きました．一般の C_i について考えます．長さ 2 以上のサイクルの個数を X とします．また，長さ 1 のサイクルのうち，ちょうど Y 個を”使う”とします．ここで，”使う”とは，そのサイクルのボールを移動させることを意味します．

各ボールについて，そのボールを目標でない場所に運べる回数は $C_i - 1$ 回です．よって，目標を達成するためには，$\sum C_i - 1 \geq X + Y$ が成り立つ必要があります（ここで $\sum$ の範囲は，使わなかった長さ 1 のサイクル以外の全てのボールです）．また，$X + Y \geq 1$ の場合，サイクル以外の連結成分内に，$C_i - 1 \geq 1$ を満たす辺が存在する必要があります．

逆に，これらの必要条件を満たせば，目標を達成することは可能です．実際に，次のような手順で操作手順を構築できます．

- $C_i = 1$ のときと同様に，サイクルでない連結成分について，操作手順をもとめる．
- $C_t \geq 2$ なるボール t を見つけたら，そのボールを B_t へ動かす前に，まだ訪れていないサイクルへと動かす．（ただし，$C_i \geq 2$ なる辺を持つサイクルがあるなら，それを優先して選ぶ）
- そして，サイクル上でボールを一周させる．（一周させる過程で $C_i \geq 2$ なるボールがあったら，それを更に別のサイクル上へ動かして・・・と再帰的に行う）
- ボール t を動かす．C_t に余裕があり，かつまだ訪れていないサイクルが残っているなら，そのサイクルへと動かし，再び上記の操作を行う．そうでないなら B_t に動かす．

このときの操作回数を考えます．$A_i \neq B_i$ であるボールの個数を P とします．サイクルでない連結成分に

含まれており，$A_i = B_i$ であり，他のサイクルへと一度以上移動したボールの個数を Z とします．すると，全体の操作回数は $P + Z + X + 2Y$ になります．結局，Y と Z を適切に選び，操作が達成可能であるという条件の元で，$2Y + Z$ を最小化する問題が解ければ良いです．これは条件を言い換えると，コスト 1 とコスト 2 の品物があるナップザック問題に帰着されます．この問題は，各コストごとに価値で品物をソートすることで解けます．

よって，この問題は $O(MlogM)$ で解けます．

解答例

F : Division into Multiples

適当な前処理によって，$\gcd(A,C)=1, \gcd(B,C)=1$ の問題に帰着できます．$C=1$ の場合は自明です．非負整数の組 (x,y) を，$Ax+By \equiv 0 \mod C$ である場合に，よい組と呼ぶことにします．明らかに，極小なよい組だけを考えればよいです．

$D = A/B \mod C$ とします．$\gcd(B,C)=1$ なので D は一意に定まります．極小な組を列挙していきます．まず，$(0,C)$ があります．つぎに，$1 \le x \le C$ の範囲では，$(x, (C - Dx) \mod C)$ が極小な組の候補として考えられます．結局，$(D \times j \mod C) < (D \times i \mod C)$ for all $j < i$ を満たす i を列挙できれば良いです．

これは，次のようにして求められます．まず，四隅が $(0,0),(C,0),(C,D),(0,D)$ の長方形を考えます．そして，原点から $(1,1)$ ベクトルの方向へ，光線を発射します．この光線は，$x=C$ に到達した場合は $x=0$ に戻り，$y=D$ に到達した場合は $y=0$ に戻るものとします．この光線が $y=0$ に到達した瞬間の x 座標を記録することにします．すると求めたいのは，記録した x 座標が，今までの x 座標よりも大きくなったタイミングです．

これは再帰的に解くことができます．一般に，幅 W，高さ H の長方形について問題を解いているとします．まず，$H \le W$ の場合，まず，H が記録されます．そして，その後は，四隅が $(H,0),(W,0),(W,H),(H,H)$ の長方形の内部だけを光線が動くと考えて良いです．なぜなら，$x=0$ に到達した光線は，そのときと全く同じ y 座標で $x=H$ に到達するからです．これを繰り返し，$H > W$ になったとします．この場合も，上と同様に，幅 W，高さ $H-W$ の問題に帰着できます．

これで，よい組を列挙できることになりました．もちろん，よい組を全て列挙していては間に合いませんが，等間隔で並ぶ部分をまとめて管理すると，$O(\log C)$ 個の値を管理するだけで済みます．$O(\log C)$ であることは，ユークリッドの互除法と同じ要領でわかります．

今得た良い組を，$(0,C) = (x_1,y_1),(x_2,x_2),\cdots,(x_k,y_k) = (C,0)$ と置きます．ここで，$x_i - x_{i-1} \le x_{i+1} - x_i$, $y_{i-1} - y_i \ge y_i - y_{i+1}$ が全ての i $(2 \le i \le k-1)$ について成り立ちます．これは，よい組を列挙するアルゴリズムの動きを見ればわかります．

さて，元の問題を解きましょう．よい組の多重集合 S を選んで $\sum_{i \in S} x_i \le X$, $\sum_{i \in S} y_i \le Y$ の元で，$|S|$ を最大化する問題でした．

ここで，x,y の凸性から，S に含まれる元の最大値と最小値は，高々 1 しか違わないことが示せます．ここまで分かれば残りは簡単です．答えで二分探索を行います．O(良い組の個数) の判定関数は自明です．これを少し工夫すれば，等間隔に並んだ良い組をまとめて処理することができ，$O(\log C)$ の判定関数が得られます．

最終的に，入力される値の最大値を V とすると，1 テストケースあたり $O(\log^2 V)$ 時間で解けることになり，十分高速です．

解答例

A: Xor Battle

We will use the 0-based index. Consider the following DP:

- $DP[i]$ = the set of integers t with the following property.
  - If $x = t$ before the i-th (0-based) round, the game ends with $x = 0$.

Let's calculate this DP from N to 0. First, we have $DP[N] = \{0\}$.
When $S_i = 0$, $DP[i]$ is calculated by the following formula:

- $DP[i] = DP[i+1] \ \cup \ \{v \oplus A_i | v \in DP[i+1]\}$

When $S_i = 1$, there are two cases.

- The case when $A_i \in DP[i+1]$: If $x \notin DP[i+1]$, the person 1 does nothing and wins. Otherwise, $x \in DP[i+1]$ and $x \oplus A_i \in DP[i+1]$ hold, so the person 1 loses. Therefore, $DP[i] = DP[i+1]$.
- The case when $A_i \notin DP[i+1]$: For all x, at least one of x and $x \oplus A_i$ is not contained in $DP[i+1]$, so the person 1 can always win. Therefore, $DP[i] = \emptyset$.

We can calculate the set $DP[i]$ efficiently by maintaining the basis of it.
The time complexity is $O(N \log(\max(A_i)))$ per test.
author's solution

B: 01 Unbalanced

We have two solutions to this problem. One of them is a naive one with binary search, which the writer had when proposing the problem. Below, we explain the other, more clever one, which was born from an argument between the writer and latte, the tester.

Let 0 and 1 correspond to -1 and $+1$, respectively. Then, the problem becomes the following: minimize the difference between the maximum and minimum prefix sums. Let $f(M)$ be the maximum value that can be the minimum of the prefix sums when the maximum prefix sum is at most M. Let Z be the minimum value that can be the maximum of the prefix sums, and then the problem is to find $\min\{M - f(M) : Z \leq M\}$.

First, let us find Z. We can easily do so by replacing every '?' with -1.

Then, let us consider how to find $f(M)$ for given M. We can use a greedy algorithm from front to back. More specifically, let us first assume every '?' to be -1. Then, for each '?' from front to back, check whether the maximum prefix sum will not exceed M if we change that '?' to $+1$. If the condition holds, change the '?' to $+1$. The validity of this greedy algorithm follows from the fact that the solution does not get worse when changing '?' to $+1$ than not changing it.

We can observe that, when M increases by 2, at most '?' changes from -1 to $+1$. Thus, we can see that $f(M+2) \leq f(M) + 2$, from which it follows that $M - f(M) \leq (M+2) - f(M+2)$. Therefore, $\min\{M - f(M) : Z \leq M\} = \min(Z - f(Z), (Z+1) - f(Z+1))$.

Since $f(M)$ can be found with the greedy algorithm above, the time complexity of this solution is $O(N)$.

Author's solution

Author's solution (binary search)

C: Range Set

We can obviously make every bit 1, so the answer does not change if 0 and 1 are inverted. Thus, we assume $A \leq B$.

A bit sequence can be the string x after operations if and only if the following condition holds:

- Change all A or more consecutive 0s to 1s. Now we have B or more consecutive 1s somewhere.

The sufficiency can be shown as follows. First, if we fix the part with B consecutive 1s, we can freely change the other parts (by, intuitively, filling them from the end). Then, we change the chosen B bits to 1. Finally, we properly do the operation of changing bits to A consecutive 0s.

The necessity can be seen from considering the inverted version of the operations (choose A/B consecutive 0s/1s and change them freely).

Let us count the strings that cannot be the string x after operations. First, we can easily count the sequences without A or more consecutive 0s or B or more consecutive 1s by DP. We now have to consider how we can insert A or more consecutive 0s into the part with consecutive 1s.

We can do that by counting beforehand, for each n, the number of ways to insert A or more consecutive 0s into the part with n consecutive 1s. We can then use that value as a multiplier when adding n 1s in the transition in the DP above.

We can do all of these computations in $O(N^2)$ time.

Author's solution

D: Lamps and Buttons

For a permutation p, we use the phrase "the cycle containing i" to represent the cycle $i \to p_i \to p_{p_i} \to \cdots \to i$.

The optimal strategy for Snuke is as follows:

- Find the minimum i such that Lamp i is on and p_i is unknown, and press Button i.
- If there is no such i, he loses;
- if it turns out that $p_i = i$, he loses;
- otherwise, he can turn on all the lamps in the cycle containing i.
- After going round the cycle, if all the lamps are on, he wins;
- otherwise, repeat the above procedure.

We actually have freedom in choosing the button to press in the first step, but we can assume we use the minimum i since p is randomly generated.

We can rephrase the winning condition for Snuke in this strategy to the following condition for the permutation:

> Take the minimum t such that $t \leq A$ and $p_t = t$. If there is no such t, let $t = A+1$. Then, for every $A+1 \leq i \leq N$, there is a value less than or equal to $t-1$ in the cycle containing i.

Assume that we fixed the value of t. Then, we first have the condition $i \neq p_i$ for every i such that $i < t$. We will deal with it using the inclusion-exclusion principle. Then, we have to solve the following problem:

> Given are integers a, b, and c. Count the permutations of $\{1, \ldots, a+b+c\}$ that satisfy the following conditions:
>
> - For every $a+1 \leq i \leq a+b$, there is a value less than or equal to a in the cycle containing i.

We can count them as follows. There are $a!$ permutations of $\{1, \ldots, a\}$. Consider how we can add b elements into the cycles of these permutations. For the i-th element to add, there are $a+i-1$ positions available. Finally, we add the c elements. We can either insert them to existing cycles or make a new cycle, and there are $a+b+i$ positions available in total for the i-th element to add. The product of these numbers is $(a+b+c)! \times a/(a+b)$, which can be found in $O(1)$ time after pre-processing in $O(N)$ time.

Therefore, we can solve the problem in a total of $O(A^2 + N)$ time.

Author's solution

E: Fragile Balls

Consider a directed graph with the edges $A_i \to B_i$. Below, when we say "connected component", it means a connected component when the edges are seen as undirected. However, when we say "strongly connected component", it means a strongly connected component in the directed graph itself.

Consider the case $C_i = 1$. In this case, the objective is achievable if and only if the graph does not contain a connected component that is a cycle of length 2 or more. We will first show this.

Assume that there is a cycle of length 2 or more. To do the operation in this cycle, we need one operation to carry a ball from another connected component to a vertex in this cycle. Since $C_i = 1$, however, after this operation, the carried ball cannot go back to the connected component it comes from. Thus, the objective is unachievable.

Assume that there is no cycle of length 2 or more. First, for a cycle of length 1, we need to do nothing. Consider connected components that are not cycles. Let us decompose this component into strongly connected components. We take one arbitrary strongly connected component that can be the topologically smallest. By noticing that the in-degree of every vertex is at least 1, we can show that there is always a vertex whose out-degree is at least 2. We take one arbitrary such vertex and call it v. From the definition of a strongly connected component, there is an (edge-)simple cycle that passes v within this strongly connected component. Assume that we moved a ball along this cycle from v. From then on, we can assume that we can freely move balls from the vertices in this cycle. We can now carry balls to all the vertices reachable from this cycle one after another.

By considering the above for every strongly connected component that can be the topologically smallest, we can see that we can move all the balls in this connected component. Thus, the necessity and sufficiency of the condition are shown.

Now that we have solved the case $C_i = 1$, let us consider the general case. Let X be the number of cycles of length 2 or more. Also, assume that we "use" Y of the cycles of length 1. Here, we say we use a cycle when we move the ball on the cycle.

We can move each ball to a position that is not the destination of the ball at most $C_i - 1$ times. Thus, to achieve the objective, $\sum C_i - 1 \geq X + Y$ must hold (Here the range of $\sum$ is all the balls that are not in unused cycles of length 1). Additionally, when $X + Y \geq 1$, there must be an edge such that $C_i - 1 \geq 1$ in a connected component that is not a cycle.

On the other hand, when these necessary conditions are met, the objective is achievable. We can construct the sequence of operations as follows:

- Similarly to the case $C_i = 1$, construct the sequence of operations for each non-cycle connected component.
- When we find a ball t such that $C_t \geq 2$, move it to an unvisited cycle before moving it to B_t. (If there is a cycle with an edge such that $C_i \geq 2$, prioritize that cycle.)
- Then, move the ball around the cycle. (If there is a ball such that $C_i \geq 2$ on the way, move that ball to another cycle, and continue recursively.)

- Move the ball t. If there is still an unvisited cycle, and the remaining value of C_t is sufficient, move the ball to that cycle and do the above step again. Otherwise, move the ball to B_t.

Consider the number of operations in this case. Let P be the number of balls such that $A_i \neq B_i$. Also, let Z be the number of balls such that $A_i = B_i$ contained in a non-cycle connected component and moved to a cycle at least once. Then, we have a total of $P + Z + X + 2Y$ operations. In the end, we have to solve the problem of choosing Y and Z so that the objective is achievable to minimize $2Y + Z$. After rephrasing the conditions, we can reduce it to a knapsack problem where the items have the cost of 1 or 2. We can solve this problem by sorting the items for each cost according to their values.

Therefore, the problem can be solved in $O(M \log M)$ time.

Author's solution

F: Division into Multiples

With proper pre-processing, the problem can be reduced to the case $\gcd(A, C) = 1, \gcd(B, C) = 1$. If $C = 1$, we can easily solve it. We say that a pair of non-negative integers (x, y) is good when $Ax + By \equiv 0 \mod C$. Obviously, we only have to consider the minimal pairs.

Let $D = A/B \mod C$. Since $\gcd(B, C) = 1$, D is uniquely determined. Let us enumerate the minimal pairs. First, we have $(0, C)$. Then, for the range $1 \leq x \leq C$, we have the candidates $(x, (C - Dx) \mod C)$. Eventually, we have to enumerate all i such that $(D \times j \mod C) < (D \times i \mod C)$ for all $j < i$.

This can be done as follows. Consider a rectangle with the corners $(0, 0), (C, 0), (C, D)$, and $(0, D)$. Let us emit a beam from the origin in the direction of the vector $(1, 1)$. Assume the following: when the beam reaches $x = C$, it goes back to $x = 0$; when the beam reaches $y = D$, it goes back to $y = 0$. Let us record the x-coordinates at the moments when the beam reaches $y = 0$. Then, we want to find the moment when we record an x-coordinate that is greater than the previous coordinates.

This can be found recursively. Assume that we are dealing with a general rectangle of width W and height H. If $H \leq W$, we will first record H. Then, we can regard that the beam moves within a rectangle with the corners $(H, 0), (W, 0), (W, H)$, and (H, H). This is because the beam reaches $x = H$ having the same y-coordinate as its y-coordinate when it reaches $x = 0$. Let us apply this repeatedly and assume that we now have $H > W$. Similarly to the above, we can also reduce this case to the case of width W and height $H - W$.

We can now enumerate the good pairs. Of course, we will run out of time if we enumerate all of them, but we only have to maintain $O(\log C)$ values when we put together equidistant pairs. The fact that we have $O(\log C)$ values can be seen similarly to the analysis of the Euclidean algorithm.

Let $(0, C) = (x_1, y_1), (x_2, x_2), \cdots, (x_k, y_k) = (C, 0)$ be the good pairs we obtained. Here, $x_i - x_{i-1} \leq x_{i+1} - x_i$, $y_{i-1} - y_i \geq y_i - y_{i+1}$ holds for all i $(2 \leq i \leq k - 1)$. This can be seen from observing the behavior of the algorithm enumerating the good pairs.

Now, let us solve the original problem. It asks us to choose a multiset S of good pairs such that $\sum_{i \in S} x_i \leq X$ and $\sum_{i \in S} y_i \leq Y$ to maximize $|S|$.

From the convexity of x, y, we can show that the maximum and minimum values of the elements in S differ by at most 1. Now, the rest is easy. Let us do binary search on the answer. We can naively construct a determination function that works in $O(\text{The number of good pairs})$ time. It can be improved a little by processing equidistant good pairs together, into a determination function that works in $O(\log C)$ time.

In the end, we have a solution that solves each test case in $O(\log^2 V)$ time where V is the maximum possible value in input, which is fast enough.

Author's solution

AtCoder Grand Contest 044
Editorial

dario2994

May 23, 2020



A: Pay to Win

We consider the problem *backward*. Hence, we start with the number $x = N$ and we have to reach $x = 0$. We can perform the following moves:

2. If $2 \mid x$, replace x with $\frac{x}{2}$; paying A.
3. If $3 \mid x$, replace x with $\frac{x}{3}$; paying B.
5. If $5 \mid x$, replace x with $\frac{x}{3}$; paying C.
$\pm$. Replace x with $x - 1$ or $x + 1$; paying D.

What is the structure of an optimal strategy? It might be that we apply only move 4., in that case the cost is ND. Otherwise, the strategy begins with

$$N \overset{\pm.}{\longmapsto} \cdots \overset{\pm.}{\longmapsto} ky \overset{k.}{\longmapsto} y\,,$$

where $k \in \{2, 3, 5\}$. We claim that $y \in \{\lfloor \frac{N}{k} \rfloor, \lceil \frac{N}{k} \rceil\}$. In fact, if $y < \lfloor \frac{N}{k} \rfloor$ (and the same reasoning works if $y > \lceil \frac{N}{k} \rceil$), the following sequence of moves is strictly cheaper than the above one:

$$N \overset{\pm.}{\longmapsto} \cdots \overset{\pm.}{\longmapsto} k\left\lfloor \frac{N}{k} \right\rfloor \overset{k.}{\longmapsto} \left\lfloor \frac{N}{k} \right\rfloor \overset{\pm.}{\longmapsto} \cdots \overset{\pm.}{\longmapsto} y\,.$$

Hence, we have shown that an optimal strategy either goes directly to 0 using only the move $\pm$. or goes to one of $\{\lfloor \frac{N}{k} \rfloor, \lceil \frac{N}{k} \rceil\}$ for $k \in \{2, 3, 5\}$. In particular, if $f(n)$ is the minimum cost of reaching 0 starting from n, we have shown that $f(n)$ can be easily computed if we know $f(\lfloor \frac{n}{k} \rfloor)$ and $f(\lceil \frac{n}{k} \rceil)$ for $k \in \{2, 3, 5\}$.

The above observation strongly suggests a dynamic programming approach. But it remains to understand how many states we are going to reach starting from N. It turns out that the reachable numbers are those of the two forms[1]

$$\left\lfloor \frac{N}{2^a 3^b 5^c} \right\rfloor \quad \text{and} \quad \left\lceil \frac{N}{2^a 3^b 5^c} \right\rceil.$$

Since $0 \le a \le 60$, $0 \le b \le 40$ and $0 \le c \le 30$ (otherwise the denominator is larger than the numerator), the reachable numbers are less[2] than $2 \cdot 61 \cdot 41 \cdot 31 = 155.062$ (to get accepted, a map should be used to keep the values $f(n)$ for the reachable numbers n).

Implementation: https://atcoder.jp/contests/AGC044/submissions/13531498.

[1] Can be shown applying the two identities

$$\left\lfloor \frac{\lfloor \frac{n}{u} \rfloor}{v} \right\rfloor = \left\lfloor \frac{n}{uv} \right\rfloor \quad \text{and} \quad \left\lceil \frac{\lceil \frac{n}{u} \rceil}{v} \right\rceil = \left\lceil \frac{n}{uv} \right\rceil.$$

[2] With a more careful estimate, one can see that the reachable numbers are ≤ 20.000.

B: Joker

For each $1 \leq i \leq N^2$, we compute the minimum number of viewers that will hate viewer i forever (the answer is the sum of these values). This number coincides with the minimum cost of a path from the seat of viewer i to the sides of the square, considering that going through an empty seat has cost 0 and going through an occupied seat has cost 1. Let $H_k(i)$ be the minimum cost (as defined above) of a path from the seat of viewer i to the outside after the first k viewers have left the cinema.

Key observation. The values $H_k(i)$ are decreasing (for k going from 0 to N^2) and at the beginning we have

$$H_0(1) + H_0(2) + \cdots + H_0(N^2) \approx \frac{N^3}{6}.$$

Our strategy is to keep all values $H_k(i)$ updated at all times. Initializing $H_0(1), \ldots, H_0(N^2)$ in $O(N^2)$ is straightforward. Let us show how to update $H_{k-1}(1), \ldots, H_{k-1}(N^2)$ to get $H_k(1), \ldots, H_k(N^2)$. When the viewer P_k goes away, we perform a breadth-first search (or a depth-first search) starting from the seats of P_k and updating the values. During the k-th breadth-first search, we will visit only the seats i such that $H_k(i) < H_{k-1}(i)$, hence the total number of seats visited in the N^2 steps (for $1 \leq k \leq N^2$) is $O(N^3)$ (see the *key observation*).

The time complexity of this solution is $O(N^3)$ with a small constant which is sufficient to get accepted (some optimization might be required in slow languages such as `python`).

Implementation: https://atcoder.jp/contests/AGC044/submissions/13531558.

C: Strange Dance

Let us denote with i the person starting in position i. For each $n = 0, 1, \ldots, N$ we are going to compute the final permutation $P_0^n, \ldots, P_{3^n-1}^n$ (after all the songs have been played) if there are 3^n people and the sequence $W_0^n, \ldots, W_{|T|-1}^n$, where W_k^n denotes the person in the last position (that is $3^n - 1$) after k songs have been played.

It is straightforward to compute P^0 and W^0. We show how to compute P^{n+1} and W^{n+1} from P^n and W^n.

Key observation. For any $0 \le i < 3^n$ and any $0 \le k < |T|$, it holds

$$P_i^{n+1} \equiv P_{i+3^n}^{n+1} \equiv P_{i+2\cdot 3^n}^{n+1} \equiv P_i^n \pmod{3^n}$$

and

$$W_k^{n+1} \equiv W_k^n \pmod{3^n}.$$

Hence, we have to determine only the most significant digit in base 3 of P_i^{n+1}, as the other digits are the same of P_i^n (the same holds for W_k^{n+1}). Let $D_0, \ldots, D_{3^{n+1}-1}$ be the most significant digit of the positions of the people. It is straightforward to initialize these values before any song is played. How do these values change when a song is played?

- If the $(k+1)$-th song is a Salsa, all values 2 become 1 and all values 1 become 2.
- If the $(k+1)$-th song is a Rumba, all the values stay the same apart from the entries of D with indexes W_k^n, $W_k^n + 3^n$, $W_k^n + 2 \cdot 3^n$. The corresponding 3 values of D change according to the transformation:

$$0 \mapsto 1,\ 1 \mapsto 2,\ 2 \mapsto 0.$$

If we keep D updated at all times, it is easy to see that we are able to compute W_k^{n+1} for all values of k and P_i^{n+1} for all i (thanks to the *key observation*).

Sadly, we cannot naïvely keep D updated, because when a Salsa is played a lot of values change. This issue can be solved handling Salsas lazily: if the $(k+1)$-th song is a Salsa, we update only the only the entries of D with indexes W_k^n, $W_k^n + 3^n$, $W_k^n + 2 \cdot 3^n$ (which can be done efficiently keeping track of the last time we have updated a certain entry).

The time complexity of this solution is $O(3^N + N \cdot |T|)$.

Implementation: https://atcoder.jp/contests/AGC044/submissions/13531969.

D: Guess the Password

We describe two possible solutions.

Let $A = 62$ be the size of the alphabet. Both solutions begin asking A queries, one for each character in the alphabet. The query corresponding to character c is a string containing only c with length L. It is easy to see that after these A queries, we know the frequencies of all the characters (and, of course, we know also the length of the hidden password $|S|$).

Solution 1: Bisecting the alphabet. Given a subset E of the alphabet, let $f(A)$ be the maximal subsequence of the hidden password S that contains only characters in E. We already know $f(\{c\})$ for any single character c in the alphabet.

> **Key observation.** Given two strings S, T with $|T| \leq |S|$, the edit-distance between the two is $|S| - |T|$ if and only if T is a subsequence of S.

Thanks to the *key observation*, if we know $f(A)$ and $f(B)$ for two disjoint subsets of the alphabet, we can merge them and obtain $f(A \cup B)$ with $|f(A)|+|f(B)|-1$ queries. Hence we can find $S = f(\{a, b, \ldots, z, A, \ldots, Z, 0, \ldots, 9\})$ with no more than $L\lceil \log_2(A) \rceil - A + 1$ queries. This solution, considering the initial A queries, asks at most $L\lceil \log_2(A) \rceil + 1 = 769$ queries.

Implementation: https://atcoder.jp/contests/AGC044/submissions/13532026.

Solution 2: Bisecting the string. Given $0 \leq l \leq r < |S|$ and a character c, let $q(c, l, r)$ be the number of occurrences of c in the substring $S[l \ldots r]$.

> **Key observation.** Given two characters a and b, let us define the string T as
>
> $$T := \overbrace{aa\cdots a}^{n}\overbrace{bb\cdots b}^{|S|-n}.$$
>
> The edit-distance between the hidden password S and T is $|S| - q(a, 0, n-1) - q(b, n, |S|-1)$.

Fix an interval $[l, r]$ and let $m = \lfloor \frac{l+r}{2} \rfloor$ be the midpoint. Assume that we know $q(c, 0, l-1)$, $q(c, l, r)$, $q(c, r, |S|-1)$ for all characters c and we want to find $q(c, l, m)$ and $q(c, m+1, r)$. Notice that if we have $q(c, l, m)$, the value of $q(c, m+1, r)$ can be obtained as $q(c, l, r) - q(c, l, m)$; hence we focus on computing $q(c, l, m)$.

Thanks to the *key observation*, with one query we can find $q(c, l, m)+q(z, m+1, r)$, where z is a *fixed character*. Hence, with $\min(A, r - l + 1) - 1$ queries (we perform the queries only for the characters c present in $S[l \ldots r]$ and we do not perform the query for the character z), we obtain $q(c, l, m) + q(z, m+1, r)$ for all characters c appearing at least once in the $S[l \ldots r]$. From this information, it easy to deduce the value of $q(z, m+1, r)$ and therefore to obtain the sought values $q(c, l, m)$.

Since we learnt how to compute the frequencies in the two halves of a substring (given the frequencies in the whole substring), and at the beginning we know the frequencies of all the characters in S, we can repeat this argument bisecting the string obtaining a solution that performs $61+2\cdot61+4\cdot31+8\cdot15+16\cdot7+32\cdot3+64\cdot1 = 699$ queries. This solution, considering the initial A queries, asks at most 761 queries.

Implementation: https://atcoder.jp/contests/AGC044/submissions/13463830.

E: Random Pawn

First of all, without loss of generality (up to rotating and duplicating a position) we can assume that $A_0 = A_{N-1} = \max A$. This assumption is useful as it implies that whenever we arrive at $p = 0$ or $p = N-1$ it is convenient to end the game (hence we have simplified the game from a circle to a segment!).

Let E_p be the expected gain of an optimal strategy if the pawn is initially at position p (if we are able to compute the array E, then the answer to the problem is its average).

The values E_p are characterized by $E_0 = A_0$, $E_{N-1} = A_{N-1}$ and

$$E_p = \max\left(A_p, \frac{E_{p-1} + E_{p+1}}{2} - B_p\right) \quad \text{for any } 1 \le p < N-1.$$

Given another array $C_0, C_1, \ldots, C_{N-1}$ (that we are going to fix later), let us rewrite this identity as

$$E_p - C_p = \max\left(A_p - C_p, \frac{E_{p-1} - C_{p-1} + E_{p+1} - C_{p+1}}{2} + \frac{C_{p-1} + C_{p+1}}{2} - C_p - B_p\right).$$

If we choose C in such a way that $\frac{C_{p-1}+C_{p+1}}{2} - C_p - B_p = 0$ for any $1 \le p < N-1$ (it is easy to construct such a C with integer entries), then the above identity simplifies to

$$F_p = \max\left(A_p - C_p, \frac{F_{p-1} + F_{p+1}}{2}\right),$$

where $F_p := E_p - C_p$. This condition is exactly equivalent to saying that (p, F_p) belongs to the boundary of the upper convex-hull of the points $(p, A_p - C_p)$.

Since the upper convex-hull can be computed in linear time, the time complexity is $O(N)$.

How to find the solution. Let us briefly describe how one can obtain the described solution. Since the game is *memory-less*, the optimal strategy is characterized by the subset of positions where we choose to end the game. Let $l < r$ be two consecutive positions where we want to end the game. We have $E_l = A_l$, $E_r = A_r$ and, for all positions $l < i < r$, it holds

$$E_i = -B_i + \frac{E_{i-1} + E_{i+1}}{2} \iff E_i - E_{i-1} + 2B_i = E_{i+1} - E_i\,.$$

In order to make this formula homogeneous, it is natural to consider the array C such that $C_i - C_{i-1} + 2B_i = C_{i+1} - C_i$. Doing so, we quickly deduce (with the same definition of F given above)

$$F_i = \frac{(r-i)F_l + (i-l)F_r}{r-l}$$

that shows clearly that the solution is given by a suitable convex-hull.

The continuous version. What follows requires a bit of not-so-elementary math; we give only a sketch.

The problem has a natural continuous version: the pawn moves according to a Brownian motion X_t (on $\mathbb{R}^d$ or on a closed manifold, as in this problem) and if we decide to end the game at time T the payoff is

$$A(X_T) - \int_0^T B(X_t)\,dt\,.$$

Let $E(x)$ be the expected payoff of an optimal strategy if we start with $X_0 = x$. One can prove that the function $E(x)$ is characterized by the two properties:

- Either $E(x) = A(x)$ and $\Delta E(x) \le 2B(x)$,
- Or $E(x) > A(x)$ and $\Delta E(x) = 2B(x)$.

If C is the function such that $\Delta C = 2B$ and $F := E - C$, the two properties become

- Either $F(x) = A(x) - C(x)$ and $\Delta F(x) \le 0$,
- Or $F(x) > A(x) - C(x)$ and $\Delta F(x) = 0$.

This is the classical formulation of the *obstacle problem* (the function $A - C$ being the obstacle) and its solution is the smallest super-harmonic function above $A - C$. In 1-d being super-harmonic is equivalent to being concave and thus this problem admits an efficient solution. Note that the problem, even the discrete version, would have been much harder if instead of moving on a segment the pawn was moving on a square.

Implementation: https://atcoder.jp/contests/AGC044/submissions/13532052.

F: Name-Preserving Clubs

Let us first consider the problem under the assumption that *two clubs cannot have the exact same members*. In the end, we will explain how to handle equal clubs.

Let us define a (k, n)-board as a matrix with k rows and n columns such that each cell is either 0 or 1. A (k, n)-board is good if the rows are distinct and any permutation (different from the identity) of the columns generates a (k, n)-board that cannot be obtained from the original one permuting the rows.

It is easy to see that good (k, n)-boards are in bijection with name-preserving configurations of clubs (rows represent clubs, columns represent people). Thus, we want to count the number of good (k, n)-boards with k minimum.

Given a good (k, n)-board T (notice that a good board has distinct columns):

- let T^t be its transpose, i.e., the (n, k)-board with $T^t[j][i] = T[i][j]$;
- let T^c be its complement, i.e., the $(k, 2^k - n)$-board that has all the columns not present in T (in an arbitrary order).

Key observation. If T is a good board, then also T^t and T^c are good boards. Moreover, the number of good (k, n)-boards (up to permutation of rows/columns) is the same as the number of good (n, k)-boards and the number of good $(k, 2^k - n)$-boards.

Let $g(n)$ be the minimum k such that there is a good (k, n)-board. Applying the *key observation* we get immediately $2^{g(n)} - n \geq g(g(n))$ (because we obtain the existence of a good $(2^{g(n)} - n, g(n))$-board). It turns out that this inequality is sharp.

Let $G(n)$ be the minimum function such that $G(1) = 0$ and for any $n \geq 2$ it holds $2^{G(n)} - n \geq G(G(n))$ (notice that this function coincides with $\lceil \log_2(n) \rceil$ for almost all n, but not for all n).

Lemma 1. *There exists a valid (k, n)-board if and only if $G(n) \leq k \leq 2^n - G(n)$ (in particular $g(n) = G(n)$).*

Proof. Let us prove the statement by induction on n.

If $G(n) \leq k < n$, applying the *key observation*, it is sufficient to construct a valid (n, k)-board and such a board exists by inductive hypothesis.

Moreover, again thanks to the *key observation*, it is sufficient to prove the existence of a valid (k, n)-board for $k \leq 2^{n-1}$ (indeed, if $k > 2^{n-1}$ we can consider the equivalent problem of finding a valid $(2^n - k, n)$-board). In the range $n \leq k \leq 2^{n-1}$, we construct explicitly a good (k, n)-board. Let T be the good $(n - 1, n)$-board induced by the name-preserving configuration of clubs $\{\{1\}, \{1, 2\}, \{2, 3\}, \ldots, \{n-2, n-1\}\}$. We fill the remaining $k - n + 1$ rows with at least 3 *ones* per row (this way, the first $n - 1$ rows do not get confused with the others). It is always possible to do so if $k \leq 2^{n-1}$. □

Now, let us focus on *counting* the number of good (k, n)-boards. Let $c(k, n)$ be the number of good (k, n)-boards (the problem asks for $c(G(n), n)$)). The *key observation* provides us with the two identities $c(k, n) = c(n, k)$ and $c(k, n) = c(k, 2^k - n)$. Applying these identities, we reduce to the case $k \leq n \leq 2^{k-1}$.

Lemma 2. *If $6 \leq k \leq n \leq 2^{k-1}$, then $c(k, n) > 1000$.*

Proof. Consider the two name-preserving configurations of clubs (if $A \subseteq \{1, \ldots, k\}$, we denote $A^{\mathsf{c}} = \{1, \ldots, k\} \setminus A$)

$$\{\{1, 2\}, \{2, 3\}, \ldots, \{k - 4, k - 3\}, \{k - 3, k - 2\}, \{k - 2, k - 1\}^{\mathsf{c}}\};$$
$$\{\{1, 2\}^{\mathsf{c}}, \{2, 3\}^{\mathsf{c}}, \ldots, \{k - 4, k - 3\}^{\mathsf{c}}, \{k - 3, k - 2\}^{\mathsf{c}}, \{k - 2, k - 1\}\}.$$

They have $k - 2$ clubs each, and all of their clubs have either 2 or $k - 2$ members. Hence, if we add, to any of the two configurations, $n - k + 2$ clubs with a number of members different from 2 or $k - 2$, we still have a name-preserving family (and all the families generated in this way are different). Hence we deduce

$$c(k, n) \geq 2 \binom{2^k - \binom{k}{2} - \binom{k}{k-2}}{n - k + 2}.$$

It is easy to see that, under the constraint $6 \leq k \leq n \leq 2^{k-1}$, this quantity is minimized at $k = n = 6$ and therefore

$$c(k, n) \geq 2 \binom{2^6 - 15 - 15}{2} = 34 \cdot 33 > 1000.$$

□

Given Lemma 2, it remains only to compute $c(k,n)$ for $k \leq n \leq 2^{k-1}$ and $k \leq 5$. We can iterate over all (k,n)-boards checking (in the naïve way) whether the board is good (stopping as soon as $1000+1$ good boards are found). Note that there are 22 possible (k,n) pairs and only 11 of them are such that there are ≤ 1000 good boards.

Equal clubs. Let us explain how to handle the fact that the clubs may be non-distinct. If there is a name-preserving configuration of k clubs with two equal clubs, then[3] $2^{k-1} \geq N$ and therefore $k \geq G(N)$ (so the minimum number of clubs computed above remains correct). If $k = G(N)$, then $2^{G(N)-1} \geq N$ and this inequality holds only for the numbers N of the form $2^M - G(M) + 1, 2^M - G(M) + 2, \ldots, 2^M$ for some natural number M. Considering what we have said above, it is straightforward to see that the number of good $(G(N), N)$-boards is larger than 1000 for all those numbers apart from 4 and 7 and 8. If $N = 4$, there is 1 minimal name-preserving configuration with non-distinct clubs; if $N = 7$ there are 2 minimal name-preserving configurations with non-distinct clubs; if $N = 8$ there are 0 minimal name-preserving configurations with non-distinct clubs (these facts can be obtained either with pen and paper or generating all possible configurations with a computer).

Implementation: https://atcoder.jp/contests/AGC044/submissions/13532081.

[3] Indeed, if the number of unique clubs is h (two clubs having the same members are counted only once) then it holds $2^h \geq n$, because two people cannot belong to the exact same clubs (since the configuration is name-preserving).

AGC 043 解説

yosupo, maroonrk, sigma425

2020 年 3 月 21 日

For International Readers: English editorial starts on page 8.

A: Range Flip Find Route

経路を一つ決めたときにその経路を通れるようにするには何回操作する必要があるかを考えます。

経路において、白いマスから黒いマスに移動する回数に、s_{11} が黒の時のみ 1 を加えた値を Z とします。実はこの経路を通れるようにするための最小の操作回数は Z です。

操作を行ったときに、経路のマスの色がどのように変化するかを考えます。これは、全く変化しないか、もしくは経路のある区間の色が反転することがわかります。また、経路の全ての区間について、そこを反転させるような操作が可能であることもわかります。

以上の観察により、経路の色の列の区間を好きに反転させられるとき、全てのマスを白色にするには何回操作を行う必要があるか、を考えればいいことがわかります。そして、どのような操作をしても Z が必ず 1 しか減らせないこと、また、Z を 1 減らすような操作が可能であることに気づくと、最小回数が Z であることがわかります。

後は Z が最小の経路を探せばよく、DP を行うことでこの問題の答えを得ることができます。

B: 123 Triangle

まず入力から 1 を引いて $a_i = 0, 1, 2$ としてもかまいません。$x_{i,j}$ はすべて $0, 1, 2$ のいずれかになります。まず答えの偶奇を考えてみます。すると $x_{i,j}$ は $x_{i-1,j}$ と $x_{i-1,j+1}$ のちょうどどちらか一方が奇数のときに奇数になることがわかります。よって次の問題が解ければ答えの偶奇がわかります。

> 各要素が $0, 1$ である $x_{1,1}, \ldots, x_{1,N}$ が与えられます。$x_{i,j}$ を次のように定義します。
> $x_{i,j} = x_{i-1,j} \hat{} x_{i-1,j+1}$
> $x_{N,1}$ を求めてください。ただし $\hat{}$ は排他的論理和を表します。

これは次のようにして解けます。まずパスカルの三角形のようなものを考えると、a_i が答えへ寄与する回数は $\binom{N-1}{i}$ であることがわかるので、この偶奇が全て求まればよいです。これは Lucas の定理から求めたり、二項係数の式に従って 2 で何回割れるかを求めたりすることで可能です。

これで答えが 1 のケースは特定できました。0 と 2 を区別するために次のことを考えます。

- 入力に ($0, 1, 2$ のうち) 1 が存在するか？

存在する場合、答えは 2 にならないことが示せます。よって、次のような手順で問題を解くことができます。

1. 答えが奇数かどうか判定する。 奇数なら答えは 1。
2. 入力に 1 が含まれるか判定する。含まれるなら答えは 0
3. そうでない場合、入力は 0 か 2 なので、全体を 2 で割ると前のケースに帰着できる。

計算量は $O(N)$ です。

C: Cubic Graph

価値の決め方から、価値の大きい頂点から順番に見ていき、独立点集合に加えられるならば加える、という貪欲が回ることがわかります。つまり、

$$\mathrm{dp0}[i,j,k] := (x_i, y_j, z_k) \text{ を独立点集合に加えられるか？}$$

という DP で $O((N+M)^3)$ で解けます。なお、$M = \max(M_1, M_2, M_3)$ とします。

次に、以下の二人ゲームを考えます

> X, Y, Z のそれぞれどこかの頂点に、コマが 1 つずつ置いてある。2 人が交互に操作行い、操作を行えなくなった方が負けである。最善を尽くしたら先手と後手、どちらが勝つか？
>
> - 3 つのコマから一つ選ぶ。そのコマを隣接している、より頂点番号の大きい頂点に動かす

この問題は

$$\mathrm{dp1}[i,j,k] := \text{コマが } x_i, y_j, z_k \text{にある状態でスタートしたら後手が勝つか？}$$

という DP で $O((N+M)^3)$ で解けます。

ここで、$\mathrm{dp0}, \mathrm{dp1}$ に注目すると、漸化式と初期状態が全く同じであることに気づくと思います。なので当然内容も全く同じです。

各頂点に grundy 数を割り振ります。つまり各頂点に、その頂点と隣接している、より頂点番号の大きい頂点に書かれていないような最小の正整数を書きます。すると $\mathrm{dp1}[i,j,k] = \mathrm{true}$ と $\mathrm{grundy}(x_i) \oplus \mathrm{grundy}(y_j) \oplus \mathrm{grundy}(z_k) = 0$ が同値となることが知られています。

grundy の性質より、grundy 数は高々 $O(\sqrt{M})$ です。よって、$\mathrm{grundy}(x_i), \mathrm{grundy}(y_i), \mathrm{grundy}(z_k)$ を決め打つことで $O(N+M)$ でこの問題は解けました。

実装例

D: Merge Triplets

まず数列たち $A_1, A_2, \ldots, A_N$ が与えられたとき、最終的な順列 P がどのようになるか考えます。ある一つの数列 $x = (x_1, x_2, x_3)$ に着目したとき、もし $x_i > x_{i+1}$ なら、 x_i が選ばれた直後に x_{i+1} も選ばれることがわかります。したがって、x を前から順番に見ていき、これまでで最も大きい値が出てきたらその手前に区切りを入れる、という操作を行うと、区切られた各ブロックは連続して選ばれます。

全体の中で選ばれるタイミングはブロックの先頭の値によって決まり、ひとつの数列の中のブロックの先頭の値は前から単調に増加するので、最終的にできる数列はブロックの先頭の値でブロックをソートして並べたものになります。

逆に最終的な列 P からも同様にブロックは一意に定まり、それらを $A_1, \ldots, A_N$ にどう割り振るかを考えると、順列 P が作れる条件は次のようになります。

> P を上述の方法でブロックに分割する。これらのブロックを $A_1, \ldots, A_N$ に割り振って、それぞれの数列の長さを 3 にできる。

これは次と同値です。

> P を上述の方法でブロックに分割する。これらのブロックのサイズを考えたとき、
>
> - すべて 3 以下
> - 2 の個数 が 1 の個数以下

ブロックのサイズが前から順番に $a_1, \ldots, a_K$ だったとすると、これから作れる順列の個数は $N!/(a_1 * (a_1 + a_2) * \cdots * (a_1 + \cdots + a_K))$ となるので、うまく遷移することで次のような状態の DP が可能です。

> $dp[s][d]$ = 前から見てサイズの和が s で (1 の個数 - 2 の個数) が d であるようなすべての $a_1, \ldots, a_k$ に対する、$N!/(a_1 * (a_1 + a_2) * \cdots * (a_1 + \cdots + a_k))$ の総和。

s は 0 から $3N$, d は $-3N$ から $3N$ を渡り、遷移は $O(1)$ なので、全体の計算量は $O(N^2)$ です。

E: Topology

説明のため、 $x = \sum_{i \in S} 2^i$ のとき、 A_x を A_S とも書くことにします。まず明らかな必要条件として、$A_S = 1$ かつ $T \subset S$ なら、 $A_T = 1$ である必要があります。逆に、これを満たせば構成可能であることを示します。

まずジャッジ方法について説明します。本質的に同じなので $S = \{0, 1, \ldots, N-1\}$ の場合を説明します。$(i + 0.5, 0.5)$ のことを点 i と呼ぶことにします。

閉曲線 C から、次のように文字列 f_C を生成します。

- 閉曲線をなぞっていって、点 i から上向きに伸ばした半直線を横切ったら文字 u_i を , 下向きに伸ばした半直線を横切ったら文字 d_i を f_C の末尾に追加。

例えばサンプル 2 の出力例に対しては、 $d_1u_1u_0d_0$ という文字列ができます。

すると、この文字列が次の条件を満たすことと、$A_S = 1$ であることが同値であることが示せます。

> 次の操作を繰り返して、文字列を空にできる。
>
> - 隣り合う二つの文字が同じなら、その二つを同時に消す。

以降この文字列の形で閉曲線を表します。

次に、入力が $11\ldots110$ すなわち、すべての点が通れないときのみ閉曲線をうまく $y < 0$ に持っていけないケースを構成します。これは再帰的に構成できます。この文字列を a_N とおきます。

$N = 1$ の場合は $a_1 = u_0d_0$ でよいです。

$N \geq 2$ の場合は、まず a_{N-1} のすべての文字のインデックスを 1 増やした文字列を s として、次のように構成できます。

- $a_N = u_0su_0d_0s^{-1}d_0$

ただし s^{-1} は s を文字列として反転させたものとします。

例えばはじめのいくつかの a_N は次のようになります。

- $a_1 = u_0d_0$
- $a_2 = u_0(u_1d_1)u_0d_0(d_1u_1)d_0$
- $a_3 = u_0(u_1u_2d_2u_1d_1d_2u_2d_1)u_0d_0(d_1u_2d_2d_1u_1d_2u_2u_1)d_0$

a_N が条件を満たすことを証明します。

> 帰納法で示す。 $N = 1$ は明らか。$N \geq 2$ とする。まず $S = \{0, 1, \ldots, N-1\}$ のときは、$A_S = 0$ となる。なぜなら、a_N に同じ文字が連続することがないため。次に S がそれ以外の場合に $A_S = 1$ となることを示す。$0 \notin S$ の場合、文字 u_0, d_0 を無視することになるので、残った s と s^{-1} が打ち消し合って消えるのでよい。そうでない場合、ある $i (1 \leq i < N)$ が存在して $i \notin S$ となるので、帰納法の仮定から s (と s^{-1}) は空文字列にできる。従って残る文字列は $u_0u_0d_0d_0$ となり、これは明らかに空文字列にできる。よって示された。

最後に一般のケースについて話します。$A_S = 0$ となる S のうち極小なものをすべて列挙します。そしてそれぞれの S について上述のように閉曲線を作り、それらをつなげばよいです。(例えば、常に $(0, 0)$ からスタートする閉曲線にしておけば簡単につなげます)

正当性は次のように示せます。

まず $A_S = 1$ の場合はつないだすべての閉曲線が空文字列にできるので、全体も空文字列にできます。$A_S = 0$ の場合は、まずこのうち極小な S について空文字列にできないことを示せば十分です。実際、S が極小なら S に対応して作った閉曲線以外は空文字列になって、S に対応して作った閉曲線のみが残るので、これは空文字列にはなりません。

サイクルの長さの制限はかなり緩めに設定しています。構成方法にもよりますが、かなり雑に見積もっても $S \subseteq \{0, 1, \ldots, N-1\}$ に対応する閉曲線の長さが $4N \cdot 2^{|S|}$ で抑えられて、これをすべての S について足し合わせても $4N \cdot 3^N$ となり、これは十分小さいです。

F: Jewelry Box

クエリが 1 つで，ちょうど A 個の宝石箱を準備する問題だとします．一般性を失わず，$S_{i,1} \leq S_{i,2} \leq \cdots \leq S_{i,K_i}$ と仮定します．

宝石 $(i,1),(i,2),\cdots,(i,j-1)$ を買った個数の合計を $x(i,j)$ $(1 \leq i \leq N,\ 1 \leq j \leq K_i+1)$ と表すことを考えます．ここで，$x(i,j)$ は，以下の条件を満たす必要があります．

- $x(i,j)$ は整数
- $0 = x(i,1) \leq x(i,2) \leq \cdots \leq x(i,K_i+1) = A$
- $x(i,j+1) - x(i,j) \leq C_{i,j}$
- 全ての制約 U_i, V_i, W_i と，任意の宝石 (V_i, j) について考える．ここで，$S_{U_i,k} \geq S_{V_i,j} - W_i$ を満たす最小の k を考える．そのような k が存在しない場合は $k = K_{U_i}+1$ とする．このとき，宝石 $(V_i,j),(V_i,j+1),\cdots,(V_i,K_{V_i})$ と同時に使うことができる宝石店 U_i の宝石は，宝石 $(U_i,k),(U_i,k+1),\cdots,(U_i,K_{U_i})$ である．よって，$x(U_i,k) \leq x(V_i,j)$.

実は，これらの条件は十分条件でもあります．この事実をまず証明します．

まず，使う宝石を決めた場合，それらを箱に分ける際には，各宝石店の宝石を昇順に並べ，最もサイズの小さい宝石を入れた箱，2 番めに小さい宝石を入れた箱…と作っていくのが最適です．これは次のように証明できます．もしこの方法で構成できなかったとすると，ある制約 (U_i, V_i, W_i) があって，これがどこかで満たされていないということです．しかし，宝石店 U_i, V_i の宝石とこの制約だけが存在するとした場合の最適戦略は，宝石をサイズでソートすることなので，結局，この制約は絶対に満たすことができないことがわかります．

以上の手順で z 番目に作られた宝石箱について考えます．各 i について，$x(i,j(i)) < z$ になる最大の $j(i)$ を考えると，この宝石箱には，宝石 $(1,j(1)),(2,j(2)),\cdots,(N,j(N))$ が入ることになります．ここで，ある制約 (U_i, V_i, W_i) が満たされていなかったと仮定します．すると，$S_{U_i,j(U_i)} < S_{V_i,j(V_i)} - W_i$ となります．ここで，上記のうち 4 番目の条件を考えると，ある k が存在し，$j(U_i) < k$, $x(U_i,k) \leq x(V_i,j(V_i))$ となります．$x(V_i,j(V_i)) < z$ より，$x(U_i,k) < z$ となります．しかしこれは，$j(U_i)$ の取り方に矛盾します．よって，満たされない制約は存在しないことがわかります．

$x(i,j)$ を定めてしまえば，その宝石を買うためのコストは，$\sum P_{i,j} \times (x(i,j+1) - x(i,j))$ と計算できます．

ここで，今まで出てきた $x(i,j)$ の制約と目的関数が全て一次式であることに注目すると，この問題を整数計画問題として定式化できます．そしてこの問題は，緩和して LP にしても，最適解が変わりません．詳細は省略しますが，このことはいくつかの方法で証明できます．例えば，後述するアルゴリズムを拡張して実際に LP の解を復元すると，整数になっていることが確認できます．また，LP の行列が totally unimodular であることを直接示すこともできます．

LP を解きます．まず，$foo = bar$ の形の制約は，$foo \leq bar$, $-foo \leq -bar$ として不等式に分解します．また，$x(i,j) \leq baz$ という形の制約は，ダミー変数 $D = 0$ を用い，$x(i,j) - D \leq baz$ と変形します．すると，今までの制約は全て，$p - q \leq r$ という形の不等式になります．

この LP の双対問題を考えます．するとこれは，**最大**費用流になっていることがわかります．目的関数の符号を反転させて，最小費用流として考えます．

この双対問題では，主問題の変数がグラフの頂点に対応しています．そこで，変数 $x(i,j), D$ に対応する頂点をそれぞれ，頂点 $x(i,j), D$ と呼ぶことにします．この最小費用流では，各 $x(i,j)$ に対して，そこから流

れ出る流量が $P_{i,j-1} - P_{i,j}$ として定まっています．(なお，$P_{i,0} = P_{i,K_i+1} = 0$ です) ここで，今考えているグラフに対し，頂点 $x(i,j)$ から $x(i,j+1)$ へ向かう容量 $P_{i,j}$，コスト 0 の辺を追加したとします．追加後のグラフにおいて，各頂点から流れ出る流量が 0 の最小費用循環流を考えると，これは元の問題と同じ答えになることがわかります．以降はこのグラフに対して，各点から流れ出る流量が 0 の問題を解いていきます．

各点から流れ出る流量が全て 0 なので，最終的な循環流は，負閉路の集まりになっているはずです．ここで，グラフの辺のコストに注目すると，そもそもコストが負の辺は，頂点 $x(i,K_i+1)$ から D へ向かうコスト $-A$ の辺しかありません．D から $x(i,K_i+1)$ へ向かうコスト A の辺は絶対に使う必要がないことに注意すると，この最小費用循環流は，次のようなアルゴリズムで解けることになります．

> 各 $f = 0,1,2,\cdots$ について，D からいずれかの $x(i,K_i+1)$ へ向かう容量 f の最小費用流を求める．このコストを $mcf(f)$ としたとき，$mcf(f) - Af$ の最小値が答えになる．

$mcf(f)$ の値は，primal-dual 法で求めることができます．またここで，$mcf(f+1) - mcf(f) \leq mcf(f+2) - mcf(f+1)$ が成り立っています．よって，$mcf(f+1) - mcf(f) \geq A$ が成立する最小の f が求まればよいです．なお，$f \to \infty$ でも $mcf(f+1) - mcf(f) < A$ となるときは，いくらでも解が小さくなるということになり，これは主問題における解が存在しないことを意味します．

以上で，クエリが 1 つのときの問題は解けました．$mcf(f)$ の値がクエリに寄らないことに注意すると，この問題を複数のクエリについて答えるのも容易です．

計算量について考えます．K_i の最大値を $Kmax$，$P_{i,j}$ の最大値を $Pmax$ とおきます．まず最小費用流の計算量を考えます．グラフの形から，$mcf(f+1) - mcf(f) < mcf(f+2) - mcf(f+1)$ が成立する最大の f は高々 $\sum P_{i,j} = O(NKmaxPmax)$ であることがわかるので，primal-dual 法で最短路を求める回数は，最大でもこの値以下です．グラフの頂点数は $O(NKmax)$，辺数は $O(MKmax)$ なので，最小費用流の計算量は $O(NKmaxPmax \times SP(NKmax, MKmax))$ になります．ただしここで，$SP(V,E)$ は，頂点数 V，辺数 E のグラフの最短路を求めるのにかかる計算量です．普通の dijkstra 法を使えば，$SP(V,E) = O(V + ElogE)$ となり，十分高速です．クエリに関しては，二分探索をするだけなので，1 クエリあたり $O(log(NKmaxPmax))$ で処理できます．

writer は，最小費用流の流量が $O(NKmaxPmax)$ になるケースを用意しましたが，実際に最短路を求める回数が $O(NKmaxPmax)$ になるケースは生成できていません．これについてアイディアのある方がいれば，コメント等で教えてください．

解答例 (グラフの形状が解説とは少し異なります)

A: Range Flip Find Route

Let's fix a certain path from the top-left corner to the bottom-right corner, and compute the minimum number of operations required to make all squares on this path white.

Let's define the value Z as follows: Consider a sequence of 'black' and 'white' obtained by concatenating the colors of all squares on the path (call it S). The value Z is defined as the number of black runs in this sequence. It turns out that the minimum number of required operations is Z.

This is because:

- In each operation, we flip a continuous subsequence of S. Thus, Z decreases by at most 1.
- For any non-empty continuous subsequence of S, we can flip the subsequence by choosing an operation appropriately. Thus, if $Z > 0$, we can always decrease it by 1.

Thus, the answer is the minimum possible value of Z when we choose the path optimally, and this can be computed by a simple DP.

B: 123 Triangle

First, we can assume that the inputs are $0, 1, 2$, not $1, 2, 3$ by subtracting one. The answer will also be $0, 1, 2$. Let's consider the parity of the answer. $x_{i,j}$ is odd if and only if exactly one of $x_{i-1,j}$ and $x_{i-1,j+1}$ is odd, so we consider the following problem.

> Given are $x_{1,1}, \ldots, x_{1,N}$, where each element is either 0 or 1. Let $x_{i,j}$ be defined as follows:
> $x_{i,j} = x_{i-1,j} \hat{} x_{i-1,j+1}$
> Find $x_{N,1}$. Where $\hat{}$ denotes exclusive-or.

This problem can be solved as follows. By analogy with Pascal's triangle, the number of times a_i contributes to the answer is $\binom{N-1}{i}$. So what we have to do is find the parity of $\binom{n-1}{i}$ for all i, this can be done by Lucas's theorem or just computing how many times the binomial coefficients can be divided by 2 according to the definition.

Now we have identified the case with the answer 1. To distinguish between 0 and 2, consider the following:

- Is there 1 (of $0, 1, 2$) in the input?

It can be proved that if 1 exists, then the answer cannot be 2. Therefore, you can solve the problem using the following procedure.

1. Determine if the answer is odd. If so, the answer is 1.
2. Determine if the input contains 1. If so, the answer is 0
3. Otherwise, the input is either 0 or 2, so dividing all a_i by 2 can reduce the problem to the previous case.

The time complexity is $O(N)$.

C: Cubic Graph

By the definition of the weights of vertices, we can get the following greedy algorithm easily: we check vertices one by one in the decreasing order of weights and add it to the independent set if we can add it. In other words, we can solve this problem in $O((N+M)^3)$ time ($M = \max(M_1, M_2, M_3)$) by the following DP:

$$\mathrm{dp0}[i,j,k] := \text{Is } (x_i, y_j, z_k) \text{ contained in the independent set?}$$

Next, we consider the following two-player game.

> There are three graphs X, Y, Z. Initially each graph contains one token (on some vertex). Alice and Bob take turns alternately and performs the following move. The (current) player loses when he cannot move.
>
> - Choose one of the three tokens arbitrary, and move it to an adjacent vertex that has an index higher than that of the current vertex.

This problem can be solved in $O((N+M)^3)$ time by the following DP:

$$\mathrm{dp1}[i,j,k] := \text{Does the current player lose when tokens are on the vertices } x_i, y_j, \text{ and } z_k?$$

We can notice that the recurrence formula and initial values of dp0 and dp1 are completely the same. Therefore we can focus on dp1 instead of dp0.

We write grundy numbers on each vertex. In other words, we write the minimum positive integer that is not written on any adjacent vertices.

It is known that $\mathrm{dp1}[i,j,k] = \text{true}$ iff $\mathrm{grundy}(x_i) \oplus \mathrm{grundy}(y_j) \oplus \mathrm{grundy}(z_k) = 0$.

The upper bound of grundy numbers is $O(\sqrt{M})$, therefore we can solve this problem in $O(N+M)$ time by fixing $\mathrm{grundy}(x_i)$, $\mathrm{grundy}(y_i)$, and $\mathrm{grundy}(z_k)$.

Implementation Example

D: Merge Triplets

First, let's assume the sequences $A_1, A_2, \ldots, A_N$ are given and consider what the final permutation P looks like. Focus on one sequence $x = (x_1, x_2, x_3)$. If $x_i > x_{i+1}$, we know that x_{i+1} is selected immediately after x_i is selected. Therefore, look at x in order from the front, and put a break before x_i if x_i is largest among before it. Then each block is selected all at once.

When a block is chosen is determined by the value of the beginning of the block, and the value of the beginning of the block in one sequence increases monotonously from the front. Thus, the resulting sequence P is a sequence of blocks sorted by the first value.

Conversely, the blocks are uniquely determined from P. Considering how to allocate them to $A_1, \ldots, A_N$, we can show that the condition P can be created is as follows.

> Divide P into blocks as described above. We can allocate these blocks to $A_1, \ldots, A_N$ and make each sequence length to 3.

This is equivalent to the following:

> Divide P into blocks as described above. The size of these blocks satisfy the following conditions.
>
> - Each of them is not more than 3.
> - the number of 2 is less than or equal to the number of 1

Assume that the size of the blocks are $a_1, \ldots, a_K$ from the beginning. Then the number of permutations that can be created from this is $N!/(a_1 * (a_1 + a_2) * \cdots * (a_1 + \cdots + a_K))$. So the following DP is possible.

> $dp[s][d]$: Consider all $a_1, \ldots, a_k$ with $\sum a_i$ is s and (# of 1 - # of 2) is d. $dp[s][d]$ is the sum of $N!/(a_1 * (a_1 + a_2) * \cdots * (a_1 + \cdots + a_k))$ for all such a.

s can be between 0 and $3N$, d can be between $-3N$ and $3N$, and the transition is $O(1)$ time, so the whole complexity is $O(N^2)$.

E: Topology

For simplicity, we will write A_x as A_S under $x = \sum_{i \in S} 2^i$. The first obvious requirement is that if $A_S = 1$ and $T \subset S$, then $A_T = 1$ should hold. Conversely, this is the sufficient condition: we show how to construct the answer from now.

First, we will explain how to judge whether the output is correct. Because it is essentially the same, we explain the case with $S = \{0, 1, \ldots, N-1\}$.

We call $(i + 0.5, 0.5)$ the point i.

From the closed curve C, generate the string f_C as follows:

- Follow the closed curve. When crossing the half line extending upward from the point i, the character u_i is added to the end of f_C. Similarly, the character d_i is added to the end of f_C when crossing the half line extending downward from the point i.

For example, for the output of sample 2, f_C is $d_1u_1u_0d_0$.

Then we can prove the equivalence between that this string satisfies the following condition and that $A_S = 1$

> You can make the string empty by repeating the following:
>
> - If two adjacent characters are the same, erase the two at the same time.

Hereafter, we represent a closed curve in the form of this string.

Next, we construct a case where the input is $11\ldots110$: Only when all points cannot pass, the curve cannot be moved to $y < 0$ completely. This can be constructed recursively. Let's call this string a_N.

For $N = 1$, $a_1 = u_0d_0$.

For the case with $N \geq 2$, let s be the string a_{N-1} with all characters indexed up by 1. Then,

- $a_N = u_0su_0d_0s^{-1}d_0$

Where s^{-1} is the inverse of s as a string.

For example, the first few a_N are:

- $a_1 = u_0d_0$
- $a_2 = u_0(u_1d_1)u_0d_0(d_1u_1)d_0$
- $a_3 = u_0(u_1u_2d_2u_1d_1d_2u_2d_1)u_0d_0(d_1u_2d_2d_1u_1d_2u_2u_1)d_0$

Let's prove that a_N satisfies the conditions.

Prove by induction. $N = 1$ is obvious. Assume $N \geq 2$.
First, for $S = \{0, 1, \ldots, N-1\}$, $A_S = 0$. This is because a_N does not have the same character in a row. Next we prove that $A_S = 1$ otherwise.
In the case of $0 \notin S$, the characters u_0, d_0 should be ignored, so the remaining parts (s and s^{-1}) are canceled out. Otherwise, there exists some $i (1 \leq i < N)$ such that $i \notin S$. Then by the induction hypothesis s (and s^{-1}) can become an empty string. The remaining string is $u_0 u_0 d_0 d_0$, which can obviously be an empty string. QED.

Finally, let's consider about general cases. Enumerate all minimal S among $A_S = 0$. Then create a closed curve for each S as described above and "connect" them. For example, a closed curve that always starts from $(0, 0)$ can be easily connected as string.

The validity can be shown as follows.

First, for cases with $A_S = 1$, all closed curves can be empty strings, so the whole curve can also be an empty string. Next, for cases with $A_S = 0$, it is sufficient to show that S cannot be an empty string for minimal Ss among them. Indeed, if S is minimal, it will not be an empty string bccause only the closed curve created for S remains and other curves becomes empty strings.

The limit for cycle length is not so tight. It depends on the construction method, but the length of the closed curve corresponding to $S \subseteq \{0, 1, \ldots, N-1\}$ is less than $4N \cdot 2^{|S|}$. Summing up this for all S, we get the upper bound $4N \cdot 3^N$, which is small enough.

F: Jewelry Box

Assume there is only one query, and we want to prepare A jewelry boxes. WLOG $S_{i,1} \leq S_{i,2} \leq \cdots \leq S_{i,K_i}$.

Let $x(i,j)$ $(1 \leq i \leq N, 1 \leq j \leq K_i+1)$ be the total number of purchased jewelries $(i,1),(i,2),\cdots,(i,j-1)$. Here, $x(i,j)$ must satisfy the following conditions:

- $x(i,j)$ is an integer
- $0 = x(i,1) \leq x(i,2) \leq \cdots \leq x(i,K_i+1) = A$
- $x(i,j+1) - x(i,j) \leq C_{i,j}$
- Let's consider a constraint U_i, V_i, W_i and a jewel (V_i, j). Then let k be the smallest number such that $S_{U_i,k} \geq S_{V_i,j} - W_i$. If no such number exists, let k be $K_{U_i}+1$. What jewelry from Shop U_i can be purchased with one of jewelries $(V_i,j),(V_i,j+1),\cdots,(V_i,K_{V_i})$? It's one of jewelries $(U_i,k),(U_i,k+1),\cdots,(U_i,K_{U_i})$. This implies that $x(U_i,k) \leq x(V_i,j)$.

These are necessary conditions, but they are also sufficient! We will prove this fact.

First, if you decide a set of jewels to purchase, the optimal strategy to distribute them into boxes is to combine the smallest jewels from each shop, combine the second smallest, third, and so on. It's easy to prove this.

Consider the z-th jewelry box made in the above procedure. For each i, let $j(i)$ be the largest number such that $x(i,j(i)) < z$. Then, the z-th jewelry box contains jewelries $(1,j(1)),(2,j(2)),\cdots,(N,j(N))$. Now, suppose a constraint (U_i,V_i,W_i) is violated. This means $S_{U_i,j(U_i)} < S_{V_i,j(V_i)} - W_i$. However, considering the 4-th condition, there exists k such that $j(U_i) < k$ and $x(U_i,k) \leq x(V_i,j(V_i))$. Since $x(V_i,j(V_i)) < z$ we get $x(U_i,k) < z$, but this contradicts the maximality of $j(U_i)$. So there is no violated constraint.

If you decide $x(i,j)$, the total cost can be calculated as $\sum P_{i,j} \times (x(i,j+1) - x(i,j))$.

Now, all the constraints and the cost is linear to $x(i,j)$, so we can formulate the problem as integer programming. And we can prove that the relaxation to LP doesn't change the answer. We will omit the proof, but there are a few approaches. For example, you can indeed restore an integral answer to the LP by modifying the algorithm below. Also, we can directly prove that the constraint matrix is totally unimodular.

Let's solve the LP. First, equality constrains like $foo = bar$ can be reduced to $foo \leq bar$ and $-foo \leq -bar$. Also, we prepare dummy variable D and transform constraints like $x(i,j) \leq baz$ to $x(i,j) - D \leq baz$. Now, all the constraints are of the form $p - q \leq r$.

Take the dual of the LP. One can see that this dual problem is **maximum** cost flow. Let's negate the goal value and solve the minimum cost flow.

In this dual problem, variables in the primal problem correspond to the vertices in the graph. So we will call the vertices $x(i,j)$ or D according to the correspondant variables. In this minimum cost flow problem, each vertex $x(i,j)$ has a fixed flux of $P_{i,j-1} - P_{i,j}$. Consider adding edge from $x(i,j)$ to $x(i,j+1)$, with capacity $P_{i,j}$ and cost 0. After this addition, we need to solve the minimum cost

circulation (with each vertex having the flux of 0). You can check this doesn't change the answer.

What we want to calculate is a set of negative cycles. However, you can note that edges with negative costs are those edges from one of $x(i, K_i + 1)$ to D. Since the edges from D to $x(i, K_i + 1)$ with cost A will never be used in a negative cycle, we can solve the problem by the following algorithm:

> For each $f = 0, 1, 2, \cdots$, let $mcf(f)$ be the minimum cost of flow of capacity A from D to one of $x(i, K_i + 1)$. Then, the answer is the minimum value of $mcf(f) - Af$.

We can calculate $mcf(f)$ by primal-dual method. Because of the convexity of $mcf(f)$, we just need to know the minimum f such that $mcf(f+1)-mcf(f) \geq A$. Note that if $f \to \infty$ and $mcf(f+1)-mcf(f) < A$, this means A can go $-\infty$ and the primal problem has no feasible solution.

Now we solved the problem with one query. Since $mcf(f)$ doesn't change between queries, it's easy to answer many queries.

What is the time complexity? Let $Kmax$ be the maximum of K_i and $Pmax$ be the maximum of $P_{i,j}$. We can notice that maximum f such that $mcf(f + 1) - mcf(f) < mcf(f + 2) - mcf(f + 1)$ is at most $\sum P_{i,j} = O(NKmaxPmax)$, so the number of shortest path algorithms is $O(NKmaxPmax)$. The graph has $O(NKmax)$ vertices and $O(MKmax)$ edges, so the complexity of the (ordinary) mincost flow algiorithm is $O(NKmaxPmax \times MinDist(NKmax, MKmax))$. Here, $SP(V, E)$ is the time complexity of the shortest path algorithm. If you run dijkstra, $SP = O(V + ElogE)$ and it is fast enough. Each query requires a simple binary search and takes only $O(log(NKmaxPmax))$ time.

The author of this tasks developed a test case where the capacity of the min-cost-flow is $O(NKmaxPmax)$, but couldn't generate a test where one needs to run the shortest path algorithm $O(NKmaxPmax)$ times. If you have any ideas, please share!

authors solution(the graph is a bit different from what explained in this editorial)

AtCoder Grand Contest 041 解説

writer: tourist

2019 年 12 月 28 日

For International Readers: English editorial starts on page 9.

A: Table Tennis Training

A, B の偶奇が同じであれば、1 人目はすべての試合で敗北し続け、2 人目は勝利し続けるべきです。この場合、2 人は $\frac{B-A}{2}$ ラウンド後に同じ卓で出会えます。

A, B の偶奇が異なるとしましょう。2 人がただ近づき続けるだけでは、隣り合う卓までは近づけますが、同じ卓で出会えません。

偶奇を調整するには、卓 1 で勝利するか卓 N で敗北するしかありません。次の 2 通りの場合のみ考えれば十分です。

- 1 人目が卓 1 まで移動し、そこでもう 1 勝してから 2 人目がいる方に向かう。2 人目は、1 人目と出会うまで卓 1 の方に移動し続ける。
- 2 人目が卓 N まで移動し、そこでもう 1 敗してから 1 人目がいる方に向かう。1 人目は、2 人目と出会うまで卓 N の方に移動し続ける。

よって、必要な最小のラウンド数は

$$\min(A-1, N-B) + 1 + \frac{B-A-1}{2}$$

です。ここで、$\min(A-1, N-B)$ は 2 人のどちらかが卓 $1, N$ のいずれかに到達するのに必要な最小のラウンド数、1 は偶奇を調整するラウンド、$\frac{B-A-1}{2}$ は偶奇の調整後に 2 人が出会うまでのラウンド数 (2 人の間の距離の半分) を表します。

B: Voting Judges

一般性を失うことなく $A_1 \geq A_2 \geq \ldots \geq A_N$ と仮定します。初期スコアが A_i である問題が採用される可能性がある場合、それより初期スコアが高い問題も採用される可能性があることに注目して、二分探索を用いましょう。X 番目に初期スコアが高い問題に採用される可能性があるかを判定することになります。

$X \leq P$ の場合、すべてのジャッジが問題 $1, 2, \ldots, P$ に投票すれば問題 X は採用されます。

$A_X + M < A_P$ の場合、問題 X に採用される可能性はありません。

上記以外の場合、問題 X が採用されるためには問題 $1, 2, \ldots, P-1, X$ が採用されるように動くのが最適です。なぜでしょうか。「問題 X に $P-1$ 問を残して全問題に勝ってもらう必要がある。どの問題を残そうか」と考えてみましょう。答えが「初期スコアが最も高い $P-1$ 問」であることは明らかです。

当然ながら、全ジャッジが問題 X には投票すると仮定します。問題 $1, 2, \ldots, P-1$ にも全ジャッジが投票して構いません。問題 $X+1, X+2, \ldots, N$ についても、これらの問題の最終スコアは問題 X の最終スコアより高くはならないため、やはり全ジャッジが投票して構いません。$P \leq i < X$ であるような問題 i については、最大で $A_X + M - A_i$ 人が投票しても構いません。

以上で述べたような「入れられても構わない票」の総数が MV 未満であれば、問題 X に採用される可能性がないことがわかります。

そうでない場合、実は採用される可能性があると言い切れます。これは以下のように示せます。問題 i に入れられても構わない票の数を $B_i (\leq M)$ とし、$\sum B_i \geq MV$ と仮定します。$1, 1, \ldots, 1, 2, 2, \ldots, 2, 3, \ldots$ のように、問題 i が B_i 回連続で現れるような問題の列を考え、この列の最初の MV 問にジャッジ $1, 2, \ldots, M, 1, 2, \ldots, M, \ldots$ の票をこの順に割り当てます。このとき、どの問題に入る票数も B_i 以下であり、どのジャッジもちょうど V 票を投じ、どのジャッジも同じ問題に二度投票していないことが容易にわかります。

この解法の時間計算量は $O(N \log N)$ です。

C: Domino Quality

S を、$N \times N$ マスの盤面に牌を何枚か置くことですべての行とすべての列のクオリティを Q にすることが可能であるような組 (N, Q) すべての集合とします。

このとき、$(A, Q) \in S$ かつ $(B, Q) \in S$ であれば $(A + B, Q) \in S$ であることがわかります。実際に、$A \times A$ マスのクオリティ Q の盤面を左上隅に、$B \times B$ マスの盤面を右下隅に配置することで、$(A + B) \times (A + B)$ マスのクオリティ Q の盤面を得ることができます。

また、$\{(3, 1), (4, 3), (5, 3), (6, 3), (7, 3)\} \subset S$ であることもわかります。これらは、全探索か乱択アルゴリズムを実装して見つけるのが最も簡単ですが、以下のような整った配置も存在します。

```
(3, 1)      (4, 3)      (5, 3)      (6, 3)      (7, 3)

 aa.         aabc        aabba       aabc..      aabbcc.
 ..a         ddbc        bcc.a       ddbc..      dd.dd.a
 ..a         bcaa        b..cb       ..aabc      ..d..da
             bcdd        a..cb       ..ddbc      ..d..db
                         abbaa       bc..aa      dd.dd.b
                                     bc..dd      ..d..dc
                                                 ..d..dc
```

最後に、$N = 2$ の場合には構成が不可能であること、$N = 3$ の場合には $(3, 1)$ の盤面を出力すればよいこと、そして $N \geq 4$ の任意の場合には $x \geq 0, 4 \leq y \leq 7$ なる整数 x, y を用いて $N = 4x + y$ と表せ、4×4 マスの盤面 x 個と $y \times y$ マスの盤面 1 個を使って $N \times N$ マスのクオリティ 3 の盤面を得られることがわかります。

D: Problem Scores

各 k に対して以下の条件があり、合計 $N-1$ 個の条件を満たす必要があります。

$$\sum_{i=1}^{k+1} A_i > \sum_{i=N-k+1}^{N} A_i.$$

実際にはこのうち 1 個のみ、すなわち $k = \lfloor \frac{n}{2} \rfloor$ に対する条件のみ満たせばよいことがわかります。$k > \lfloor \frac{n}{2} \rfloor$ のときは、両辺の同じ変数が打ち消し合い、何らかの $k' < \lfloor \frac{n}{2} \rfloor$ に対する条件と同じ条件になります。$k < \lfloor \frac{n}{2} \rfloor$ のときは、k が 1 大きい式では左辺に A_{k+2} が、右辺に A_{N-k} が増えており、この式が元の式より「緩い」ということはありません。

$A_k = 1 + x_1 + x_2 + \ldots + x_k$ とします (任意の i に対し $x_i \geq 0$)。これを $\lfloor \frac{n}{2} \rfloor$ 番目の条件式に代入すると、問題は次のようになります。

・以下の条件を満たす非負整数の組 $(x_1, x_2, \ldots, x_N)$ を数えよ。

- $x_1 + x_2 + \ldots + x_N \leq N - 1$
- $x_1 \geq [x_2, x_3, \ldots, x_N] \cdot [0, 1, 2, 3, \ldots, 3, 2, 1]$ (“$\cdot$” は内積を表す)

$x_2, x_3, \ldots, x_N$ を固定し、以下が成り立っているとします。

- $x_2 + x_3 + \ldots + x_N = a$
- $[x_2, x_3, \ldots, x_N] \cdot [0, 1, 2, 3, \ldots, 3, 2, 1] = b$

このとき、第一の制約から $x_1 \leq N - 1 - a$ であり、第二の制約から $x_1 \geq b$ です。よって、x_1 の値の選択肢はちょうど $\max(N - a - b, 0)$ 通りです。

ここから、$a + b = [x_2, x_3, \ldots, x_N] \cdot [1, 2, 3, 4, \ldots, 4, 3, 2]$ の値のみが本質的であることがわかり、問題は次のようになります。

- 非負整数の組 $(x_2, x_3, \ldots, x_N)$ すべてに対する $\max(N - [x_2, x_3, \ldots, x_N] \cdot [1, 2, 3, 4, \ldots, 4, 3, 2], 0)$ の和を求めよ。

これは単純な $O(N^2)$ の DP により解けます。

E: Balancing Network

均一的な状態の発見

何らかのケーブル w を選びましょう。どのケーブルから出発しても w に行き着くような均一的な状態があるか判定するにはどうすればよいでしょうか。

長さ N のブール値の列 A を作り、ケーブル i からケーブル w に行き着くことが可能である場合に A_i を真とします。はじめネットワークは空で A_w のみが真であり、右から左へと平衡器を加えていくとします。

現状のネットワークの左端に平衡器 (x, y) を加えたとき、A はどのように変化するでしょうか？起こるのは、A_x, A_y がともに $(A_x|A_y)$ へと変わる、という変化のみです。実際に、A_x, A_y がともに真であるかともに偽である場合、この平衡器は何の影響ももたらず、一方のみが真である場合、平衡器を適切に設定することで双方を真にすることができます (例えば A_x が真である場合、平衡器を y から x へと向けるとケーブル y からでもケーブル w に行き着けます)。

すべての平衡器を加えた後で A_i が偽であるような i が存在するなら、条件を満たす状態は存在しません。そうでない場合には存在するのでしょうか。すなわち、どの i に対しても、i から w へと行き着くような状態が個別に存在することはわかりますが、どのケーブルからでも w に行き着くような単一の状態は存在するのでしょうか。

実は存在します。上記と似たような、平衡器を右から左へと加えていく過程を考えます。

- A_x が真、A_y が偽であるなら、平衡器を y から x へと向ける。
- A_x が偽、A_y が真であるなら、平衡器を y から x へと向ける。
- このいずれでもなければ、平衡器の方向を任意に設定する。

任意の i に対して A_i が真であるなら、このような設定で任意の i から w へと行き着くような状態が得られることがわかります。

この解法の時間計算量は $O(nm)$ です。これを高速化するには、すべてのケーブル w に対する処理はほぼ同じであり、A の初期状態のみが異なることに着目します。各 A_i を単一のブール値とするのではなく、各 w に対して 1 ビット、合計 N ビットの bitset としましょう。この他はすべて上記と同一ですが、時間計算量は $O(\frac{nm}{wordsize})$ となり、$wordsize = 64$ のとき演算の回数は 10^8 未満です。

非均一的な状態の発見

実は、$N \geq 3$ に対しては必ず非均一的な状態が存在します。

各 i に対し、ケーブル i から出発したときにどのケーブルに行き着くかを表す整数列 B_i を管理しましょう。はじめ、ネットワークは空で、任意の i に対して $B_i = i$ であるとします。ここに右から左へと平衡器を加えていきましょう。

現状のネットワークの左端に平衡器 (x, y) を加えたとき、B はどのように変化するでしょうか。平

衡器を x から y へと向けた場合、起こるのは B_x が B_y と等しくなるという変化のみです。y から x へと向けた場合は、起こるのは B_y が B_x と等しくなるという変化のみです。目標は、すべての平衡器を加えた後で B に 2 個以上の異なる値が存在するようにすることです。

そして、実はこの条件を常に保つことができます。実際に、$N \geq 3$ の場合は以下のいずれかが成り立ちます。

- B_x が B に 2 回以上現れる: この場合、$B_x \leftarrow B_y$ と設定して問題ありません。
- B_y が B に 2 回以上現れる: この場合、$B_y \leftarrow B_x$ と設定して問題ありません。
- 別の何らかの値 z ($z \neq B_x, B_y$) が B に 1 回以上現れる: この場合、平衡器の向きは任意に設定して問題ありません。

時間計算量は $O(M)$ です。

F: Histogram Rooks

$f(L, R, B)$ を、残された盤面の列 L から列 R までの部分について、$y > B$ のマスのみを考慮して (すなわち、列 i の高さを $h_i - B$ として) 何らかの DP テーブルを返す関数とします。$a_i = B$ であるような $L \leq i \leq R$ が存在する場合、この区間を $a_i > B$ なる列のみからなる何個かの区間に分割し、返された DP テーブルを併合します。そうでない場合、$f(L, R, B + 1)$ を呼んで返された DP テーブルを書き換えます。

下から B 行目より上の行にルークを詰める方法を固定したとき、すべての列は次の 3 つのカテゴリーに分類されます。

1. カテゴリー R: ルークを含む。
2. カテゴリー A: ルークを含まないが、この列のどのマスも横方向からのいずれかのルークの支配下にある。
3. カテゴリー U: ルークを含まず、ルークの支配下にないマスが 1 つ以上存在する (すなわち、この列にルークを 1 個以上置く必要がある)。

f から返されるテーブルには 2 つの引数 A, U を持たせます。これらは、対応するカテゴリーの列の数を表します (R は (区間の長さ) $-$ $(A + U)$ です)。

するとテーブルの併合は簡単で、左側と右側の A, U の組を全列挙するのみです。$dp(A, U) = \sum_{i=0}^{A} \sum_{j=0}^{U} dpLeft(i, j) \cdot dpRight(A - i, U - j)$ となります。

テーブルの書き換えについては、何個かの場合を考えます。

1. この行にルークがない: カテゴリー A の列がすべて U の列に変わる。$dp(0, A + U)$ に $dpOld(A, U)$ を加える。
2. R の列にのみルークがある: $dp(A, U)$ に $dpOld(A, U) \cdot (2^R - 1)$ を加える。
3. ルークが A の列に X 個、U の列に Y 個 (そして R の列に任意個) ある: $dp(A - X, U - Y)$ に $dpOld(A, U) \cdot \binom{A}{X} \binom{U}{Y} 2^R$ を加える。

$f(1, N, 0)$ が返したテーブルの $dp(A, 0)$ の和が答えです。

この解法を高速化するには、f が返すテーブルの状態数を $O(N^2)$ から $O(N)$ にする必要があります。次の 2 つの場合を考えます。

1. 残された盤面のこの部分より下では、どの「行」(正確には、行のうちこの列区間と繋がっている部分) もルークを 1 個以上含む。
2. 残された盤面のこの部分より下に、ルークを含まない「行」がある。

ケース 1 では A の列をすべて R の列とみなしても差し支えず、ケース 2 では A の列を (いずれ U の列になるのでこの時点で) すべて U の列とみなしても差し支えないことに注目します。

よって、C を上記のケース番号 (1 または 2) として、$dp(C,U)$ というテーブルで計算を行えます。遷移はいずれも元のテーブルの場合と類似したものになります。当然ながら、$C=1$ のとき「この行にルークがない」の遷移は生じません。また、$dpOld(2,U)$ から $dp(1,U)$ に新たな遷移が生じます。空の「行」があったときに、これ以降に空行がない可能性を想定しての遷移です。

時間計算量は $O(N^3)$ です。

AtCoder Grand Contest 041 Editorial

writer: tourist

December 29, 2019

A: Table Tennis Training

If A and B have the same parity, the first player should lose all matches and the second player should win all matches. In this case, they will meet after $\frac{B-A}{2}$ rounds.

Suppose that A and B have different parity. If the players just move towards each other, they can get very close (to neighboring tables), but they can't meet.

The only way to change parity is winning at table 1 or losing at table N. We need to consider only two cases:

- The first player moves to table 1, wins an additional game there, and moves towards the second player, who just moves towards table 1 for the whole time, until they meet.
- Similarly, the second player moves to table N, loses an additional game there, and moves towards the first player, who just moves towards table N for the whole time, until they meet.

The smallest number of rounds is then

$$\min(A-1, N-B) + 1 + \tfrac{B-A-1}{2}.$$

Here, $\min(A-1, N-B)$ stands for the number of rounds needed to reach table 1 or table N, 1 stands for a single round to change parity, and $\frac{B-A-1}{2}$ stands for the number of rounds before the friends meet after changing parity (which is half the distance between them).

B: Voting Judges

Without loss of generality, let $A_1 \geq A_2 \geq \ldots \geq A_N$. Note that if a problem with an initial score of A_i can be chosen for the problemset, a problem with a higher initial score can also be chosen. Let's use binary search to find the answer. We need to check if problem with the X-th highest initial score has a chance to be chosen.

If $X \leq P$, problem X can be chosen if all judges vote for problems $1, 2, \ldots, P$.

If $A_X + M < A_P$, problem X can't be chosen.

Otherwise, it's best to try to form a problemset with problems $1, 2, \ldots, P-1, X$. Why? If you ask yourself a question "we need problem X to beat all problems except $P-1$ of them, which problems should be left unbeaten?", it's obvious that the answer is "$P-1$ problems with the highest score".

Clearly, we assume that all judges vote for problem X. It's fine if all judges vote for problems $1, 2, \ldots, P-1$. It's also fine if all judges vote for problems $X+1, X+2, \ldots, N$, as these problems can't have a higher final score than problem X anyway. For problem i such that $P \leq i < X$, at most $A_X + M - A_i$ judges can vote.

If the total number of votes that can be cast (as described above) is less than MV, we can see that problem X can't be chosen.

Otherwise, it can! This can be shown as follows. Let $B_i \leq M$ be the number of votes that can be cast for problem i, and $\sum B_i \geq MV$. Write down a sequence where problem i is repeated B_i times consecutively. Now, assign votes by judges $1, 2, \ldots, M, 1, 2, \ldots, M, \ldots$ to the first MV problems in the sequence, in this order. It's easy to see that every problem gets at most B_i votes, every judge votes exactly V times, and no judge votes for the same problem twice.

Time complexity of this solution is $O(N \log N)$.

C: Domino Quality

Let S be the set of all pairs (N, Q) such that it's possible to place some dominoes on an $N \times N$ grid so that the quality of all rows and columns is Q.

Observe that if $(A, Q) \in S$ and $(B, Q) \in S$, then $(A + B, Q) \in S$. Indeed, we can form an $(A + B) \times (A + B)$ matrix of quality Q by putting an $A \times A$ matrix of quality Q into the top-left corner, and a $B \times B$ matrix of quality Q into the bottom-right corner.

Observe that $\{(3, 1), (4, 3), (5, 3), (6, 3), (7, 3)\} \subset S$. The easiest way to find the corresponding matrices is to implement a brute force or a randomized approach, but there are some nice regular pictures as well:

```
(3, 1)      (4, 3)       (5, 3)       (6, 3)       (7, 3)

aa.         aabc         aabba        aabc..       aabbcc.
..a         ddbc         bcc.a        ddbc..       dd.dd.a
..a         bcaa         b..cb        ..aabc       ..d..da
            bcdd         a..cb        ..ddbc       ..d..db
                         abbaa        bc..aa       dd.dd.b
                                      bc..dd       ..d..dc
                                                   ..d..dc
```

Finally, observe that the $N = 2$ case is impossible, for $N = 3$ we can output the $(3, 1)$ matrix, and any $N \geq 4$ can be represented as $N = 4x + y$, where x and y are integers such that $x \geq 0$ and $4 \leq y \leq 7$, and thus we can form an $N \times N$ matrix of quality 3 using the 4×4 matrix x times along with a single $y \times y$ matrix.

D: Problem Scores

We have $N-1$ conditions that must be satisfied, one condition for each k:

$$\sum_{i=1}^{k+1} A_i > \sum_{i=N-k+1}^{N} A_i.$$

Note that we only need to satisfy one condition, namely, when $k = \lfloor \frac{n}{2} \rfloor$. When $k > \lfloor \frac{n}{2} \rfloor$, equal variables in both sides cancel each other, and we end up with the same condition for some $k' < \lfloor \frac{n}{2} \rfloor$. When $k < \lfloor \frac{n}{2} \rfloor$, increasing k by 1 adds A_{k+2} to the left side and A_{N-k} to the right side, and such an inequality is never less restricting.

Let $A_k = 1 + x_1 + x_2 + \ldots + x_k$, where $x_i \geq 0$ for any i. If we substitute A_i in the $\lfloor \frac{n}{2} \rfloor$-th condition using this equality, our problem becomes the following:

Count the number of tuples of non-negative integers $(x_1, x_2, \ldots, x_N)$ such that:

- $x_1 + x_2 + \ldots + x_N \leq N - 1$;
- $x_1 \geq [x_2, x_3, \ldots, x_N] \cdot [0, 1, 2, 3, \ldots, 3, 2, 1]$, where "$\cdot$" denotes dot product.

Suppose that we have fixed $x_2, x_3, \ldots, x_N$, and:

- $x_2 + x_3 + \ldots + x_N = a$;
- $[x_2, x_3, \ldots, x_N] \cdot [0, 1, 2, 3, \ldots, 3, 2, 1] = b$.

Then we have $x_1 \leq N - 1 - a$ from the first constraint, and $x_1 \geq b$ from the second constraint. Thus, we have exactly $\max(N - a - b, 0)$ choices for x_1.

It follows that only $a+b = [x_2, x_3, \ldots, x_N] \cdot [1, 2, 3, 4, \ldots, 4, 3, 2]$ is important. The problem becomes:

- Sum up $\max(N - [x_2, x_3, \ldots, x_N] \cdot [1, 2, 3, 4, \ldots, 4, 3, 2], 0)$ over all tuples of non-negative integers $(x_2, x_3, \ldots, x_N)$.

This leads to a simple $O(N^2)$ DP solution.

E: Balancing Network

Finding a uniforming state

Let's choose some wire w. How can we check if a uniforming state exists that leads to w from any starting wire?

Create a boolean array A of length N. Let A_i be true if we can finish at wire w starting from wire i. Initially, the network is empty, and only A_w is true. We'll add balancers from right to left.

How does A change when we add a balancer (x, y) on the left of the network? The only change is that A_x and A_y are changed to $(A_x|A_y)$. Indeed, if A_x and A_y are both true or both false, this balancer doesn't change anything, and if one of them is true (say, A_x), we can direct the balancer correspondingly (say, from y to x), and now we can finish at wire w starting from wire y as well.

If after adding all balancers A_i is false for some i, the required state doesn't exist. Does it exist otherwise? In other words, we know that for any i individually a state that leads from i to w exists, but does a single state that leads all wires to w exists?

It turns out that it does. Just consider the same process of adding balancers from right to left:

- whenever A_x is true and A_y is false, direct the balancer from y to x;
- whenever A_x is false and A_y is true, direct the balancer from x to y;
- otherwise, direct the balancer arbitrarily.

Observe that if A_i is true for any i, there exists a state that leads from i to w in this setting.

Time complexity of this solution is $O(nm)$. To optimize it, notice that the process for all wires w is almost the same, the only difference is the initial state of A. Instead of keeping a boolean value in each A_i, let's keep a bitset of N values, one bit per each w. Everything else stays the same, but time complexity becomes $O(\frac{nm}{wordsize})$ (with $wordsize = 64$ it's less than 10^8 operations).

Finding a non-uniforming state

It turns out that a non-uniforming state always exists for $N \geq 3$.

Let's maintain an integer array B_i, denoting the wire we finish at if we start at wire i, for each i. Initially, the network is empty, and $B_i = i$ for any i. We'll add balancers from right to left.

How does B change when we add a balancer (x, y) on the left of the network? If we direct the balancer from x to y, the only change is B_x becoming equal to B_y, and if we direct it from y to x, the only change is B_y becoming equal to B_x. Our goal is to have at least two distinct values in B after adding all balancers.

It turns out we can keep this condition true at any time. Indeed, if $N \geq 3$, at least one of the following is true:

- B_x appears at least two times in B: then we can safely set $B_x \leftarrow B_y$;
- B_y appears at least two times in B: then we can safely set $B_y \leftarrow B_x$;
- some other value z ($z \neq B_x, B_y$) appears at least once in B: then we can direct the balancer arbitrarily.

Time complexity is $O(M)$.

F: Histogram Rooks

Let $f(L, R, B)$ be a function that returns some DP on a part of the histogram from column L to column R, only considering cells at $y > B$ (that is, assuming that the height of column i is $h_i - B$). If some $a_i = B$ for $L \le i \le R$, we can divide this segment into several segments containing columns with $a_i > B$, and merge DPs. Otherwise, we can call $f(L, R, B + 1)$ and modify DP.

If we have fixed the way of filling all rows above B-th with rooks, all columns can be divided into three categories:

1. category R: contains a rook;
2. category A: doesn't contain a rook, but all cells in this column are under horizontal attack of some rook;
3. category U: doesn't contain a rook, and at least one cell is not under attack of any rook (thus, this column needs at least one rook).

Our DP can have two arguments: A and U, the number of columns of the corresponding categories (R is segment width minus $A + U$).

Merging DPs is easy then, just loop over A and U in the left part and in the right part: $dp(A, U) = \sum_{i=0}^{A} \sum_{j=0}^{U} dpLeft(i, j) \cdot dpRight(A - i, U - j)$.

In modifying DP, there are a couple of cases:

1. no rook in this row: all A-columns transform into U-columns, increase $dp(0, A + U)$ by $dpOld(A, U)$;
2. only rooks in R-columns: increase $dp(A, U)$ by $dpOld(A, U) \cdot (2^R - 1)$;
3. X rooks in A-columns and Y rooks in U-columns (and maybe rooks in R-columns): increase $dp(A - X, U - Y)$ by $dpOld(A, U) \cdot \binom{A}{X}\binom{U}{Y}2^R$.

The answer is the sum of $dp(A, 0)$ returned from $f(1, N, 0)$.

To optimize this solution, we need to move from $O(N^2)$ DP states returned from f to $O(N)$ DP states. Consider two cases:

1. below this part of histogram, every "row" (actually, part of the row connected to this column segment) will contain at least one rook;
2. below this part of histogram, at least one "row" will contain no rooks.

In case 1, note that we can assume that all A-columns are just R-columns, and it doesn't change anything.

In case 2, note that we can assume that all A-columns are just U-columns (they will become U-columns in the future, but we can just assume they already are).

Hence, we can have $dp(C, U)$ where C is 1 or 2, denoting case number from above. All transitions work in a similar way. Of course, the "no rook in this row" transition can not happen when $C = 1$. An additional transition happens

from $dpOld(2, U)$ to $dp(1, U)$: any time we encounter an empty "row", we can make this transition as an assumption that we won't have empty rows in the future.

Time complexity is $O(N^3)$.

AGC040 解説

writer : maroonrk

For International Readers: English editorial starts on page 7.

A : ><

$a_{i-x} < a_{i-x+1} < \cdots < a_i$ のとなっていれば，a_i は x 以上でなくてはなりません．また同様に，$a_i > a_{i+1} > \cdots > a_{i+x}$ のときも，a_i は x 以上でなくてはいけません．

すると，a の各項について，その値は，左に連続する $<$ の個数と右の連続する $>$ の個数の max 以上である必要があるとわかります．逆に，左に連続する $<$ の個数と右の連続する $>$ の個数の max とすれば，条件をみたす数列が得られます．

よって，各項の最小値がわかり，その総和が答えになります．

解答例

B : Two Contests

問題は，区間の集合を 2 つの集合に分割して，それぞれの共通部分の長さの和を最大化することです．以下，区間は全て半開区間だと思って扱います．（つまり，R_i の値を入力 +1 だとします）

L_i の最大値を達成する i を p，R_i の最小値を達成する i を q とします．

区間 p と区間 q を同じ集合に含める場合を考えます．区間 p と q を含む集合に，他のどの区間を入れても，この集合の区間の共通部分は減りません．よってこの場合，区間 p,q を含まない方の集合は，区間 p,q 以外でもっとも長い区間 1 個にするのが最適です．（実は，区間 p, q 以外で，という部分を考えなくても正しい答えが出るので，writer 解はそのように実装しています）

区間 p と区間 q を違う集合に含めることを考えます．区間 p と同じ集合に入った区間が，区間 $[l_1, r_1), [l_2, r_2) \cdots$ だとすると，その集合の区間の共通部分の長さは，$min_{(1 \leq i)}\{max(r_i - L_p, 0)\}$ となります．

同様に，区間 q と同じ集合に入った区間が，区間 $[l_1, r_1), [l_2, r_2) \cdots$ だとすると，その集合の区間の共通部分の長さは，$min_{(1 \leq i)}\{max(R_q - l_i, 0)\}$ となります．

よって，次のような問題が解ければ良いです．

> $a_i = max(r_i - L_p, 0), b_i = max(R_q - l_i, 0)$ とする．$\{1, 2, \cdots, N\}$ を 2 つの集合 s, t に分割し，$min_{(i \in s)}\{a_i\} + min_{(i \in t)}\{b_i\}$ を最大化せよ．ただし，$p \in s, q \in t$ とする．（実は $p \in s, q \in t$ を無視しても正しい答えが出るので，writer 解はそのように実装しています）

この問題の解は，i を a_i の昇順（タイブレークは b_i の降順）に並べたあと，先頭の何個かを t に，残りを s に入れるという形です．よって，ソートして前から見ていくことで解くことが出来ます．

以上より，この問題は $O(NlogN)$ 時間で解くことが出来ます．

解答例

C : Neither AB nor BA

奇数番目の文字の ‘A‘,‘B‘ を入れ替えると，行える操作は，‘AA‘，‘BB‘ 以外を選んで消すことになります．この時，s を空にできる必要十分条件は，‘A‘,‘B‘ の個数がどちらも全体の半分以下であることです．必要性は明らかです．十分性は，‘A‘,‘B‘ のうち個数の多い方を優先的に消すことを考えればわかります．

よって，答えは，$3^N -$ ‘A‘ が過半数になる文字列の個数 $-$ ‘B‘ が過半数になる文字列の個数 になります．‘A‘ が過半数になる文字列の個数は，‘A‘ の個数 k $(k > N/2)$ を全通り試せば，あとは簡単なコンビネーションの計算でわかります．

よって，この問題は $O(N)$ で解けます．

解答例

D : Balance Beam

$S = \sum_{i=1}^{N} A_i$ とします.

平均台の並べ方を固定して考えます．明らかに，ある距離 p であって，りんごさんのスタート位置が左端から p 以下ならすぬけくんが勝ち，そうでないならりんごさんが勝つ，というものが存在します.

x 軸に平均台の左端からの距離，y 軸に時間をとって，すぬけくんの移動を 2 次元平面上に折れ線として書いてみます．同様に，りんごさんの動きも折れ線として書くことにします．このとき，りんごさんの折れ線を上下に自由に動して，すぬけくんの折れ線に下側からちょうど触れるように書いてみます．このときのりんごさんの折れ線と，x 軸の交点が，座標 $(p, 0)$ の点になっています.

座標 $(p, 0)$ を含む平均台 k を固定したとします．この時，p の値を最大化することを考えます．そこで，次のような経路 C を考えます.

- 座標 $(p, 0)$ から出発し
- りんごさんの折れ線上を (右上方向へ) すすみ
- すぬけくんの折れ線と触れた地点からすぬけくんの折れ線上をすすみ
- 座標 (N, S) に至る経路

平均台 k より後ろに置かれた平均台 i について，経路 C が平均台 i 上に対応する部分でどれだけ上方向へ移動するか考えます．この値は明らかに $max(A_i, B_i)$ 以下です．そしてこの上界は，達成することができます．具体的には，$A_i \leq B_i$ となる平均台を前に，$A_i > B_i$ となる平均台を後ろに並べれば良いです.

よって，平均台 k を固定したあとは，$max(A_i, B_i)$ の大きいものから貪欲に使うことを考えれば良いです．具体的には，$max(A_i, B_i)$ の大きい順に足していって（ただし $i = k$ は除外），総和が $S - B_k$ 以上になるところを見つければ良いです．ここで，平均台 k に対応する部分では，経路 C は上方向に高々 B_k しか移動しないことに注意する必要があります.

上記のアルゴリズムをすべての k について行うようにするのも容易にできます．よってこの問題は $O(NlogN)$ で解けます.

解答例

E : Prefix Suffix Addition

便利のため，$A_0 = 0, A_{N+1} = 0$ とします．操作 1 だけで数列を作るときの最小の操作回数は，$A_i > A_{i+1}$ となる i の個数になります．必要性は明らかです．十分性は，$A_i > A_{i+1}$ をみたす最小の i について，$c = (A_1, A_2, \cdots, A_i)$ で操作することを考えれば分かります．

同様に，操作 2 だけで数列を作るときの最小の操作回数は，$A_i < A_{i+1}$ となる i の個数になります．

よってこの問題は，次のように言い換えられます．

> 非負整数列 x, y を $A_i = x_i + y_i$ をみたすように作る．「$x_i > x_{i+1}$ なる i の個数」＋「$y_i < y_{i+1}$ なる i の個数」を最小化せよ．

よって，次のような DP が考えられます．

- $dp[i][j]$ = 先頭 i 項を既に決定していて，$x_i = j$ としたときのコスト．

この dp を愚直に行ってはとても間に合いません．しかし，この dp の状態は圧縮できます．まず，どの i についても，$dp[i][j]$ は j に対して広義単調減少であることがわかります．さらに，$dp[i][0] \leq dp[i][A_i] + 2$ であることもわかります．この正当性は，$dp[i][A_i]$ を達成する解を考え，x_i だけ 0 に変更することを考えれば示せます．

よって，$dp[i]$ の情報は，$dp[i][0]$ の値と，$dp[i][j-1] > dp[i][j]$ となる j の値（これは 高々 2 つしかない）さえ覚えておけばよいということになります．

この値の遷移は $O(1)$ で行えます．よってこの問題は全体で $O(N)$ で解けます．

解答例

F : Two Pieces

2 つの駒の座標が 2 以上離れているときのみ，座標の小さい駒を 1 動かすという操作が行えることにします．すると，単に異なる操作列が何個あるか数える問題になります．

駒の状態を，座標の大きい駒の座標 x，2 つの駒の距離 d の組 (x, d) で表すことにします．各操作は，以下のように言い換えられます．

- 操作 1: x, d を 1 増やす．
- 操作 2: d を 1 減らす．$d \geq 2$ の時のみ行える．
- 操作 3: d を 0 にする．

操作 1 の回数は合計でちょうど B 回です．操作 2 の回数 k を全通り試すことにします．

操作 1,2 の順番を固定したとします．このとき，1,2 の順番は，d の値が（初期状態を除いて）常に 1 以上であるようにします．

ここに，$N - B - k$ 個の操作 3 を挿入することを考えます．なお，$N = B + k$ の場合は明らかなので，$N - B - k > 0$ を仮定します．すると，「d の値がこれ以降，現在以下の値にならない」タイミングでのみ操作 3 を行えるとわかります．例えば，操作 1,2 を $(1, 1, 2, 1, 1)$ と行った場合，操作 3 を行えるのは，$0, 3, 4, 5$ 回目の操作のあととなります．それ以外のタイミングで操作 3 を行うと，操作 2 を行うタイミングで $d < 2$ となるためだめです．逆に，上記の条件をみたすタイミングなら，好きな場所に好きな個数操作 3 を挿入できます．操作 3 を挿入し終えたあと，最終的な d の値が $B - A$ になる必要があるので，(操作 1,2 だけを考えたときの)d の値が $A - k$ になったタイミングでは，必ず操作 3 を行う必要があります．そして，残りの $N - B - k - 1$ 個の操作 3 は，d の値が最後に $0, 1, ...A - k$ になったタイミングのうちどこかで行えば良いです．

以上より，k を固定すると，操作 1,2 の順番と，操作 3 をどう挿入するかは独立に考えることが出来ます．操作 1,2 の順番の決め方は，カタラン数と同じように求められます．操作 3 の挿入の仕方は，（区別のつかない）$N - B - k - 1$ このボールを（区別のつく）$A - k + 1$ 個の箱に入れる方法は何通りか，という問題なので，これもコンビネーションの計算で求められます．

よって，この問題は全体で $O(N)$ で解けます．

解答例

AGC040 Editorial

by maroonrk

A : ><

If $a_{i-x} < a_{i-x+1} < \cdots < a_i$, a_i must be at least x. Similarly, if $a_i > a_{i+1} > \cdots > a_{i+x}$, a_i must be at least x.

Thus, for each element of a, its value must be at least max(the number of consecutive '<' to the left, the number of consecutive '>' to the right). On the other hand, if each element of a is max(the number of consecutive '<' to the left, the number of consecutive '>' to the right), the condition is satisfied.

Thus, the answer is the sum of these lower limits.

Sample Code

B : Two Contests

The problem asks us to separate segments into two sets, maximizing the sum of the lengths of the intersections of those individual sets. Below, we consider intervals to be half-open (we add 1 to the given value of R_i).

Let p be the value of i that maximizes L_i, and q be the value of i that minimizes R_i.

Consider when Segment p and Segment q belong to the same set. Adding any other segment to this set will not shorten the intersection of this set. Thus, in this case, it is optimal to make the other set contain just the longest segment except Segment p and q. (Actually, we can omit the part "except Segment p and q" and still obtain the correct answer, so the writer's code does so.)

Now, consider when Segment p and Segment q belong to different sets. When the segments belonging to the set of Segment p are $[l_1, r_1), [l_2, r_2) \cdots$, the length of the intersection of that set is $\min_{(1 \leq i)}\{\max(r_i - L_p, 0)\}$. Similarly, when the segments belonging to the set of Segment q are $[l_1, r_1), [l_2, r_2) \cdots$, the length of the intersection of that set is $\min_{(1 \leq i)}\{\max(R_q - l_i, 0)\}$.

Thus, we want to solve the following problem:

> Let $a_i = \max(r_i - L_p, 0), b_i = \max(R_q - l_i, 0)$. Separate $\{1, 2, \cdots, N\}$ into two sets s and t, maximizing $\min_{(i \in s)}\{a_i\} + \min_{(i \in t)}\{b_i\}$. Here, $p \in s, q \in t$ must hold. (Actually, we can omit the part "$p \in s, q \in t$" and still obtain the correct answer, so the writer's code does so.)

The solution to this problem has the following form. First, sort the indices i in ascending order of a_i (ties are broken in descending order of b_i), then put some number of indices from the beginning to t, and put the others to s. Thus, we can solve it by actually sorting the indices in this way and try all candidates of the solution in this form one by one.

Therefore, the problem can be solved in $O(N \log N)$ time.

Sample Code

C : Neither AB nor BA

For odd positions in s ($s_1, s_3, s_5, \ldots$), let us replace each 'A' with 'B' and vice versa. Now we can erase any two characters except 'AA' and 'BB'. Here, s can be emptied if and only if the numbers of 'A's and 'B's are both at most $N/2$. The necessity is obvious. To see the sufficiency, consider prioritizing erasing 'A' over erasing 'B' when there are more 'A's than 'B's, and vice versa.

Thus, the answer is $3^N -$(the number of strings with more than $N/2$ 'A's)$-$(the number of strings with more than $N/2$ 'B's). We can find the number of strings with more than $N/2$ 'A's by trying all possible counts k ($k > N/2$) of 'A's and simple calculations with binomial coefficients.

Therefore, the problem can be solved in $O(N)$ time.

Sample Code

D : Balance Beam

Let $S = \sum_{i=1}^{N} A_i$.

Let us fix the order in which beams are arranged. Obviously, there exists a distance p such that Snuke wins if and only if the initial position of Ringo is at most p meters to the right of the left end of the long beam.

Let us draw Snuke's motion as a polyline in a two-dimensional coordinate plane. The x- and y-coordinate correspond to the distance from the left end of the long beam and time, respectively. We will also draw a polyline for Ringo's motion. Let us set the vertical position of Ringo's polyline so that it touches Snuke's polyline from the lower side. Now, Ringo's polyline crosses the x-axis at coordinates $(p, 0)$.

Let Beam k be the beam that contains the point $(p, 0)$, and assume that k is fixed. Let us maximize p here. Consider the path C that:

- starts at $(p, 0)$,
- goes along Ringo's polyline (towards upper-left),
- after the intersection of the two polylines, goes along Snuke's polyline,
- and reach (N, S).

For Beam i placed somewhere to the right of Beam k, let us consider how much does the path C go up in the part corresponding to Beam i. This value is obviously at most $\max(A_i, B_i)$, and we can achieve this upper limit, by putting beams such that $A_i \leq B_i$ to the left and beams such that $A_i > B_i$ to the right.

Thus, after fixing Beam k, we can greedy use beams in descending order of $\max(A_i, B_i)$ from left to right. More specifically, take the beams except Beam k one by one in descending order of $\max(A_i, B_i)$, and find out when the sum gets $S - B_k$ or greater. Here, note that the path C only goes up by B_k in the part corresponding to Beam k.

We can easily modify the algorithm above to do the computation for all k. Therefore, the problem can be solved in $O(N \log N)$ time.

Sample Code

E : Prefix Suffix Addition

For convenience, let $A_0 = 0, A_{N+1} = 0$. If we only use Operation 1, the number of operations required to make the sequence is the number of i such that $A_i > A_{i+1}$. The necessity is obvious. To see the sufficiency, consider doing the operation choosing $c = (A_1, A_2, \cdots, A_i)$ for the minimum i such that $A_i > A_{i+1}$.

Similarly, if we only use Operation 2, the number of operations required to make the sequence is the number of i such that $A_i < A_{i+1}$.

Thus, the problem can be rephrased to the following:

> Make two sequences x and y of non-negative integers such that $A_i = x_i + y_i$, minimizing (the number of i such that $x_i > x_{i+1}$) + (the number of i such that $y_i < y_{i+1}$).

This can be solved by the following DP:

- $dp[i][j]$ = The minimum cost when setting the first i elements in x and y so that $x_i = j$.

It is too slow to do it naively, but we can compress the states in this DP. First, for each i, $dp[i][j]$ is non-increasing for increasing j. Additionally, we can see that $dp[i][0] \leq dp[i][A_i] + 2$, which can be validated by modifying a solution achieving $dp[i][A_i]$ so that only x_i is changed to 0.

Thus, the only information we only have to memorize for $dp[i]$ is $dp[i][0]$ and the values of j such that $dp[i][j-1] > dp[i][j]$ (there are at most two such values).

The transition from $dp[i]$ to $dp[i+1]$ can be done in $O(1)$ time, so the whole problem can be solved in $O(N)$ time.

Sample Code

F : Two Pieces

Let us assume that the operation of moving the piece with the smaller coordinate by 1 is only possible when the distance between the two pieces is 2 or greater. Then, we just have to count the number of different sequences of operations.

Let us represent the state of the pieces as (x, d), where x is the greater of the pieces' coordinates, and d is the distance between the two pieces. The possible operations are now rephrased as follows:

- Operation 1: decrement both x and d by 1.
- Operation 2: decrement d by 1. Only possible when $d \geq 2$.
- Operation 3: set d to 0.

We have to do Operation 1 exactly B times. Let us try all candidates for the number of times we do Operation 2 (let this number be k).

Assume that the order in which we do Operation 1 and 2 is fixed. This order should be chosen so that, considering only those operations, d will be always 1 or greater (except at the beginning).

Now, let us insert Operation 3 here $N - B - k$ times. Below, we will assume $N - B - k > 0$, since the case $N = B + k$ is obvious. We can see that Operation 3 can only be inserted when the following holds: "the value of d will never be equal to or less than its current value again." For example, if we do Operation 1 and 2 in the order $(1, 1, 2, 1, 1)$, we can only insert Operation 3 just after the 0-th, 3-rd, 4-th, and 5-th of those operations. If we did Operation 3 at other times, we would have $d < 2$ when we do Operation 2, so it is not allowed. On the other hand, we can insert Operation 3 any number of times to any positions satisfying the condition above. After we finish inserting Operation 3, the final value of d needs to be $B - A$, so we must always insert Operation 3 where the value of d becomes $A - k$ for the last time (considering only Operation 1 and 2). The remaining $N - B - k - 1$ insertions of Operation 3 can be done to any of the positions where the value of d becomes $0, 1, ...A - k$ for the last time.

Thus, when we fix k, we can separately deal with the order in which we do Operation 1 and 2 and the positions to which we insert Operation 3. The number of possible orders in which we do Operation 1 and 2 can be found similarly to how the Catalan numbers are found. The number of possible choices to insert Operation 3 can be found by calculations with binomial coefficients, since it is equivalent to the problem of finding the number of ways in which we put $N - B - k - 1$ indistinguishable balls into $A - k + 1$ distinguishable boxes.

Therefore, the whole problem can be solved in $O(N)$ time.

Sample Code

AGC 039 解説

DEGwer

2019/10/05

For International Readers: English editorial starts on page 4.

A: Connection and Disconnection

同じ文字が k 個連続する場合、そのうちの $\lfloor k/2 \rfloor$ 文字を変更する必要があります。逆に、それだけ変更すれば十分なことも分かります。

S の全文字が同じ場合、答えは $\lfloor |S|K/2 \rfloor$ です。そうでない場合、同じ文字の連続は S の内部または 2 つの S の接続部に現れます。S の先頭と末尾の文字が異なる場合、答えは S に対する答えの K 倍です。同じ場合、S の先頭・末尾に続く同じ文字の個数を a, b として、S に対する答えの K 倍から $\lfloor a/2 \rfloor + \lfloor b/2 \rfloor - \lfloor (a+b)/2 \rfloor$ の $K-1$ 倍を引いたものになります。時間計算量は $O(|S|)$ です。

B: Graph Partition

条件を満たす分割が可能だとすると、$V_1 \cup V_3 \cup \ldots$ と $V_2 \cup V_4 \cup \ldots$ は頂点集合の二分割であって同じ側の頂点どうしを結ぶ辺のないものになります。よって、与えられるグラフが二部グラフでないなら、分割は不可能です。

そうでない場合、与えられるグラフの直径を d とし、距離 d の 2 点を s, t とします。V_i を s からの距離が $i-1$ であるような頂点集合とすれば、条件を満たす $d+1$ 分割が得られます。逆に、$d+2$ 個以上の分割ができた場合、V_1 の頂点と V_{d+2} の頂点の間の距離は $d+1$ 以上必要であり、矛盾します。よって、分割の最大数は $d+1$ です。時間計算量は、Warshall-Floyd のアルゴリズムなどを用いれば $O(N^3)$ です。

C: Division by Two with Something

X の二進表記を考えれば、この操作は $2N$ 回行うと元に戻ることが分かります。各 $2N$ の約数 k に対し、k 回の操作で (初めてとは限らず) 元にもどるような整数の個数を求められれば、あとは包除原理を用いて答えを求められます。以下、X と言った時は X の N 桁での 二進表記を表すことにします。

X の後ろに X のビットをすべて反転したものを連結した長さ $2N$ の文字列を S とします。k 回の操作で元に戻ることは、S が周期 k を持つことと同値です。S の作り方より、$2N/k$ が奇数となる必要があることも分かります (偶数の場合、X と X のビット反転が一致することになるため)。よって、S が周期 k を持つのは、k が偶数かつ、ある $k/2$ 文字の列 T が存在して、S が T と T のビット反転を交互に並べてできる場合のみであることが分かります。

あとはこのようにして作った S の先頭 N 文字が辞書順で X 以下であるような T の個数を求めればよいです。X の先頭 $k/2$ 文字を T' として、T が辞書順で T' より小さい場合は条件を満たします。大きい場合は条件を満たしません。等しい場合は、$T = T'$ から実際に S を構成し、X と比較すればよいです。

時間計算量は、N の奇数の約数の個数を d として、$O(Nd)$ です。問題の制約下で $d \leq 72$ なので、実行時間制限に間に合います。

D: Incenters

単位円周上の 3 点 A, B, C に対し、点 A' を B, C を結ぶ弧であって A を含まないものの中点とします。点 B', C' も同様に定義します。このとき、三角形 ABC の内心 I は三角形 $A'B'C'$ の垂心に一致することが証明できます (円周角の定理より A, I, A' が同一直線上にあることに注意しながら、円周角の定理を繰り返し使って角度計算を行う)。一般に三角形の垂心 H 、重心 G 、外心 O はこの順に直線上にのっており、$|HG| : |GO| = 2 : 1$ であることが知られています (オイラー線)。三角形 $A'B'C'$ の外心の座標は $(0, 0)$ であり、重心の座標は A', B', C' の座標の和の $1/3$ 倍であるので、I の座標は A', B', C' の座標の和となります。

A' の座標は B, C の選び方と A, B, C の位置関係だけに依存し、(A が 直線 BC に対して A' と異なる側にある限り) A の具体的な位置には依存しません。よって、A' のそれぞれの選び方 $O(N^2)$ 通りに対し、A' の座標の最終的な答えへの寄与分を求め、すべて足し合わせることで答えを求めることができます。時間計算量は $O(N^2)$ です。

E: Pairing Points

頂点 $2N, 1$ の間で円環を切り開き、直線として考えます。頂点 1 のマッチング相手を頂点 i とします。

もし $j < k < i < j' < k'$ であって (j, j') と (k, k') がともにマッチされているものが存在すれば、$(1, i)$ をあわせてサイクルができ、矛盾します。よって、($N \geq 2$ なら) $(1, i)$ と交わるマッチングの辺 (a, b) $(a < b)$ であって、i から見て一番外側にあるものを取ることができます。$(1, i)$ を取り除いたときの頂点 a, b を含む連結成分に属する頂点であって添字が i 未満のものの集合を A とし、添字が i より大きいものの集合を B とし、$1, A, B$ 以外の頂点集合を C とします。このとき、A, C, B はこの順に $[2, \ldots, 2N]$ の 3 つの区間への分割になっていることが証明できます (そうでない場合、サイクルができます)。A, C, B それぞれに対し、マッチングの辺 $(1, i)$ に対応するものをそれぞれ $(b, a), (1, i), (a, b)$ とすることで、再帰的に同様の議論を行うことができます。

$\mathrm{DP}[s][t][u]$ を、区間 $[s, u]$ の頂点のうち t 以外からなる完全マッチングであって、マッチングの辺からなるサイクルが発生しないものの個数とします。上の考察より、$\mathrm{DP}[s][t][u]$ は、$(r_1, r_2) \in S$ となるような $s \leq r_1 \leq p_1 < t < p_2 \leq r_2 \leq u$ すべてに対して、$\mathrm{DP}[s][r_1][p_1] \times \mathrm{DP}[p_1 + 1][t][p_2 - 1] \times \mathrm{DP}[p_2][r_2][u]$ を足し合わせたものになります。よって動的計画法が動作し、時間計算量 $O(N^7)$ でこの問題を解くことができます。定数倍を評価すれば、適当な実装でも十分高速に動作することが分かります。

F: Min Product Sum

この問題は以下の問題と等価です。

問題 1　$N \times M$ の 2 つのグリッド A, B に 1 以上 K 以下の整数を書き込む方法であって、以下を満たすも

のの個数を　mod 998244353 で求めよ。

- 任意の $1 \leq i, k \leq N, 1 \leq j \leq M$ に対し、$A_{i,j} \leq B_{k,j}$
- 任意の $1 \leq i \leq N, 1 \leq j, k \leq M$ に対し、$A_{i,j} \leq B_{i,k}$

これは、もとの問題のグリッドを B としたときの A としてありうるものの個数が求めて足し合わせるべき値に一致することから分かります。

X_i を A の i 行目の整数の最大値、Y_j を B の j 列目の整数の最小値とします。$X_i = 1, Y_j = 1, X_i = 2, Y_j = 2, \ldots, X_i = K, Y_j = K$ の順に X_i, Y_j の値を決めていく DP を考えます。具体的には、DP[p][q][k][0] は以下のように定まります。

- $X_i \leq k$ なる i に対応する行集合を X とする
- $Y_j \leq k$ なる j に対応する列集合を Y とする
- X は p 行からなり、Y は q 行からなる
- X, Y の和集合に属する A のマスと X, Y の共通部分に属する B のマスに、条件を満たすように値を書き込む方法の個数が DP[p][q][k][0] である

同様に、DP[p][q][k][1] は以下のように定まります。

- $X_i \leq k+1$ なる i に対応する行集合を X とする
- $Y_j \leq k$ なる j に対応する列集合を Y とする
- X は p 行からなり、Y は q 行からなる
- X, Y の和集合に属する A のマスと X, Y の共通部分に属する B のマスに、条件を満たすように値を書き込む方法の個数が DP[p][q][k][1] である

このように定めれば、DP[p][q][k][0] たちの値から DP[p'][q'][k][1] たちの値を、DP[p][q][k][1] たちの値から DP[p'][q'][$k+1$][0] たちの値を、それぞれ $O(NM(N+M))$ 時間で求めることができます。直感的には、前者の遷移は $X_i = k+1$ なる行を、後者の遷移は $Y_j = k+1$ なる列を、それぞれ決めていることになります。合計の時間計算量は $O(NM(N+M)K)$ となり、制限実行時間に収まります。

AGC 039 Editorial

DEGwer

2019/10/05

A: Connection and Disconnection

When the same character occurs k times in a row, we have to change $\lfloor k/2 \rfloor$ of them. We can also see that no more change is necessary.

When all the characters in S are the same, the answer is $\lfloor |S|K/2 \rfloor$. Otherwise, contiguous segments of the same character appear within a single copy of S or straddle between two copies of S. If the first and last characters of S are different, the answer is the answer for a single copy of S multiplied by K. If these characters are the same, let a and b be the number of the same characters at the beginning and the end of S, respectively. The answer for this case is the answer for a single copy of S multiplied by K, minus $\lfloor a/2 \rfloor + \lfloor b/2 \rfloor - \lfloor (a+b)/2 \rfloor$ multiplied by $K-1$. Thus, the problem can be solved in $O(|S|)$ time.

B: Graph Partition

Suppose that the division is possible. Then, $V_1 \cup V_3 \cup \ldots$ and $V_2 \cup V_4 \cup \ldots$ is a bisection of the vertex set such that no edge connects two vertices on the same side. Thus, if the given graph is not bipartite, the division is impossible.

Otherwise, let d be the diameter of the given graph, and let s and t be two vertices with the distance of d. By letting V_i be the set of vertices whose distances from s are $i-1$, we can divide the vertices into $d+1$ sets and satisfy the condition. On the other hand, if we could divide the vertices into $d+2$ or more sets, the distance between a vertex in V_1 and a vertex in V_{d+2} has to be at least $d+1$, and we have a contradiction. Thus, the maximum possible number of sets is $d+1$. Using Floyd-Warshall algorithm, for example, the problem can be solved in $O(N^3)$ time.

C: Division by Two with Something

Considering the binary representation of X, we can see that after $2N$ operations the integer returns to its original value. Thus, if we find the number of integers that return to the original value (not necessarily for the first time) after k operations for each divisor k of $2N$, we can find the answer using inclusion-exclusion principle. Below, we denote by X the binary representation of X with N digits.

Let S be the string of length $2N$ obtained by concatenating X and a copy of X where each bit is

inverted in this order. X returns to the original value after k operations if and only if S has a period of k. From the construction of S, we can also see that $2N/k$ needs to be odd (otherwise X and its inversion would be equal). Thus, S has a period of k only if k is even and there exists a sequence T of length $k/2$ such that S can be made by alternating T and its inversion.

Now, we only have to find the number of T such that the first N characters of S, which is made as above, is lexicographically less than or equal to X. Let T' be the first $k/2$ characters of X. If T is lexicographically smaller than T', this condition is satisfied. If T is lexicographically greater than T', the condition is not satisfied. If T is equal to T', we can actually construct S from $T = T'$, then compare it with X.

Thus, the problem can be solved in $O(Nd)$ time where d is the number of odd divisors of N. Under the constraints of the problem, $d \leq 72$, so the solution will run in time.

D: Incenters

For points A, B and C on the circumference, let A' be the midpoint of the arc connecting B and C not passing through A. B' and C' are also defined similarly. It can be proved here that the incenter I of triangle ABC coincides with the orthocenter of triangle $A'B'C'$ (by noticing that, from the theorem of the angle of circumference, A, I, A' lie on the same line, and repeatedly use this theorem to calculate angles). Also, it is known that, for a general triangle, the orthocenter H, the centroid G, and the circumcenter O, in this order, lie on the same line (Euler line) and $|HG| : |GO| = 2 : 1$. The circumcenter of triangle $A'B'C'$ has the coordinates $(0, 0)$, and the centroid has the coordinates equal to the average of the coordinates of A', B', and C', so the coordinates of I is the sum of those of A', B', and C'.

The coordinates of A' only depend on the choices of B and C and positional relation of A, B, and C, and do not depend on the specific position of A (as long as A is on the different side from A' across line BC). Thus, we can find the answer by, for each of the $O(N^2)$ choices of A', finding the contribution of the coordinates of A' to the answer and summing them up, in $O(N^2)$ time.

E: Pairing Points

Let point a be the opponent of point 1. Let m be the number of segments that intersect with the segment between 1 and a (call it the main segment). Then, there exist $1 < b_1 < ... < b_m < a < c_m < ... < c_1$ such that for each i, the segment between b_i and c_i intersect with the main segment.

Let's remove the main segment. Now the remaining segments are divided into m components. For some (p, q), the points $[2, p] \cup [q, 2N]$ are in the first component (i.e., the component with a_1 and b_1), and the points $[p+1, a-1] \cup [a+1, q-1]$ are in the other components. We can repeat the same observation recursively and get a DP solution.

- Let $DP[a][b][c][d]$ be the number of ways to connect points $[a, b] \cup [c, d]$ such that they form a tree and there is exactly one segment between the left part ($[a, b]$) and the right part ($[c, d]$).
- Let $DP2[a][b][c][d]$ be the number of ways to connect points $[a, b] \cup [c, d]$ such that they form a forest (i.e., possibly multiple trees).

The recurrence is easy:

- $DP[a][b][c][d] = \sum_{a \le p \le b, c \le q \le d} DP2[a][p-1][p+1][b] \dot{D}P2[c][q-1][q+1][d] \dot{A}[p][q]$
- $DP2[a][b][c][d] = \sum_{a \le p \le b, c \le q \le d} DP[a][p][q][d] \dot{D}P2[p+1][b][c][q-1]$

This leads to an $O(N^6)$ solution, but if you count the number of iterations more carefully it's about $(2N)^6/6!$ and it works in time.

F: Min Product Sum

The problem is equivalent to the following:

Problem 1 Let A, B be two grids of dimensions $N \times M$. Count the number of ways to fill the grids with integers between 1 and K, such that for each cell of A, the number written on the cell is less than or equal to the minimum of the $N + M - 1$ integers written on the corresponding row or column of grid B.

We can see B as the original grid in the statement; then it's clear that the number of ways to fill A is equal to the product in the statement.

This is also equivalent to:

Problem 2 Let A, B be two grids of dimensions $N \times M$. Count the number of ways to fill the grids with integers between 1 and K, such that

- For each i, (the maximum of row i in grid A) $\leq$ (the minimum of row i in grid B).
- For each i, (the maximum of column j in grid A) $\leq$ (the minimum of column j in grid B).

Here's the tricky part: let's fix (the **maximum** of **row** i in grid A) for each i and (the **minimum** of **column** j in grid B) for each j. Once we fix these values, for example, we want to count the number of ways to fill the grid in the following way:

- See picture 1 (in the next page). We want to count the number of ways to fill these two grids.
- See picture 2. We assigned candidate of integers that can be written on each cell.
- See picture 3. There are actually more constraints: each row of the red subrectangle must contain at least one 1 (though this says nothing), each row of the orange subrectangle must contain at least one 2, and each row of the yellow subrectangle must contain at least one 3. Similar conditions for columns in grid B exist.
- See picture 4. Now the number of ways to fill the grids when rowmax/columnmin are fixed can be rephrased as the number of paths along the bold line. For example, when we move from P to Q, we multiply the coefficient (the number of ways to fill the orange rectangle) times (the number of ways to fill the purple rectangle).

By counting the total number of paths from the top-left corner to the bottom-right corner (possible along non-bold ways), we can count the number of ways to fill the grid when rowmax/columnmin are not fixed. This leads to an $O(NM(N + M)K)$ DP.

Picture 1

A	Max<=1	Max<=1	Max<=2	Max<=2	Max<=3	Max<=3
Max=1						
Max=1						
Max=2						
Max=2						
Max=3						
Max=3						

B	Min=1	Min=1	Min=2	Min=2	Min=3	Min=3
Min>=1						
Min>=1						
Min>=2						
Min>=2						
Min>=3						
Min>=3						

Picture 2

A	Max<=1	Max<=1	Max<=2	Max<=2	Max<=3	Max<=3
Max=1	1	1	1	1	1	1
Max=1	1	1	1	1	1	1
Max=2	1	1	12	12	12	12
Max=2	1	1	12	12	12	12
Max=3	1	1	12	12	123	123
Max=3	1	1	12	12	123	123

B	Min=1	Min=1	Min=2	Min=2	Min=3	Min=3
Min>=1	123	123	23	23	3	3
Min>=1	123	123	23	23	3	3
Min>=2	23	23	23	23	3	3
Min>=2	23	23	23	23	3	3
Min>=3	3	3	3	3	3	3
Min>=3	3	3	3	3	3	3

Picture 3

A	Max<=1	Max<=1	Max<=2	Max<=2	Max<=3	Max<=3
Max=1	1	1	1	1	1	1
Max=1	1	1	1	1	1	1
Max=2	1	1	12	12	12	12
Max=2	1	1	12	12	12	12
Max=3	1	1	12	12	123	123
Max=3	1	1	12	12	123	123

B	Min=1	Min=1	Min=2	Min=2	Min=3	Min=3
Min>=1	123	123	23	23	3	3
Min>=1	123	123	23	23	3	3
Min>=2	23	23	23	23	3	3
Min>=2	23	23	23	23	3	3
Min>=3	3	3	3	3	3	3
Min>=3	3	3	3	3	3	3

Picture 4

A	Max<=1	Max<=1	Max<=2	Max<=2	Max<=3	Max<=3
Max=1						
Max=1			P			
Max=2						
Max=2						
Max=3		Q				
Max=3						

図 1　ガザニア

AGC038 解説

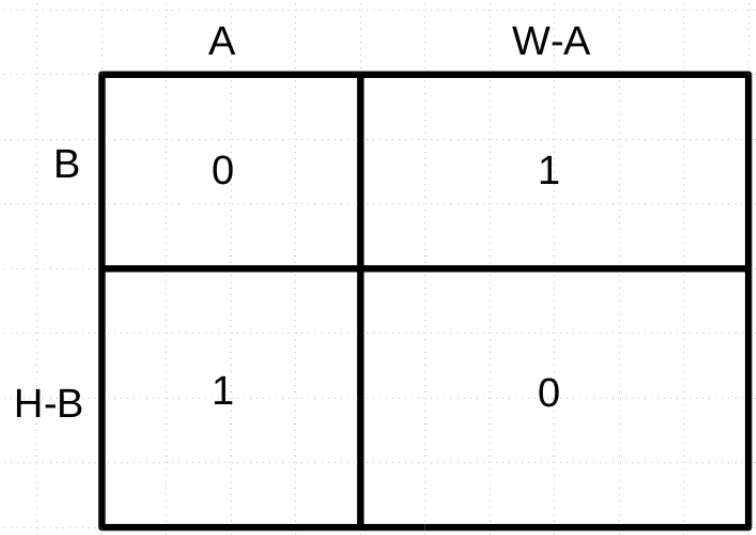
writer : maroonrk

令和元年 9 月 21 日

For International Readers: English editorial starts on page 7.

A : 01 Matrix

以下の図のようなマス目を構築することで、必ず条件をみたすことができます。



解答例

B : Sorting a Segment

区間 $[i, i+K)$ と区間 $[j, j+K)$ $(i<j)$ をソートした結果が同じになることを、i, j が equivalent であると呼ぶことにします。以下、i, j が equivalent になる条件について考えます。

i, j が equivalent である一つの場合は、区間 $[i, i+K)$ と $[j, j+K)$ がどちらももともと昇順に並んでいる場合です。

そうでない場合を考えます。まず、$[i, i+K)$ と $[j, j+K)$ が共通部分を持つ必要があることは明らかです。よって、$j < i+K$ は必要条件です。また、区間 $[i, j)$ は、区間 $[i, i+k)$ を昇順に並び替えたあとでも変化しないことがわかります。これは、区間 $[i, j)$ の値がもともと昇順に並んでおり、かつどの値も $[j, i+k)$ の中にあるどの値よりも小さいことを意味しています。同様にして、区間 $[i+K, j+K)$ の値は、もともと昇順に並んでおり、かつどの値も $[j, i+K)$ の中にあるどの値よりも大きいことになります。

すると、i, j が equivalent なら、任意の x $(i<x<j)$ について、x は i, j と equivalent であることがわかります。特に、i と $i+1$ は equivalent です。

この問題で求めたい答えは、頂点 $0, 1, \cdots, N-K$ のグラフで、i, j が equivalent である場合にのみ i, j 間に辺を張ったものの、連結成分の数となります。先程の考察より、$[i, i+K)$ がもともと昇順に並んでいないものに関しては、$i, i+1$ 間の辺についてのみ考えれば良くなります。$i, i+1$ が equivalent であるかどうかは、$[i, i+K+1)$ の中の最小、最大がそれぞれ P_i, P_{i+K} であるかどうかで判定できます。

スライド最小値を用いれば $O(N)$ 時間で、全ての i について $i, i+1$ が equivalent であるか判定できます。その他の処理も $O(N)$ でできるので、この問題は全体で $O(N)$ で解けます。(なお、std::set を用いれば実装を簡略化でき、計算量は $O(NlogN)$ になりますがこれでも十分高速です)

解答例

C : LCMs

入力される整数の最大値を $V(=1000000)$ とします。次の条件をみたす有理数列 $w_1, w_2, \cdots, w_V$ を考えます。

- 全ての $1 \leq i \leq V$ について、$\sum_{d|i} w_d = 1/i$

このような数列は、i の昇順に定めていくことができます。つまり、$w_1, w_2, \cdots, w_{k-1}$ までの値が定まっている時、$w_k = 1/k - \sum_{d|i, d<i} w_d$ とすれば良いです。この計算量は、$1, 2, \cdots, V$ の約数の個数の総和と同じオーダーであり、$O(VlogV)$ です。

$lcm(x, y) = xy/gcd(x, y)$ です。これはさらに、次のように言い換えられます。

$$xy/gcd(x, y) = xy \times \left(\sum_{d|gcd(x,y)} w_d \right) = xy \times \left(\sum_{d|x, d|y} w_d \right)$$

よって、この問題で求める式は、

$$\sum_{i=0}^{N-2} \sum_{j=i+1}^{N-1} lcm(A_i, A_j) = \sum_{i=0}^{N-2} \sum_{j=i+1}^{N-1} A_i A_j \times \left(\sum_{d|A_i, d|A_j} w_d \right)$$

となります。ここで、d に注目してこの式を書き直してみると、以下のようになります。

$$\sum_{1 \leq d \leq V} w_d \left(\sum_{d|A_i, d|A_j, i<j} A_i A_j \right)$$

$\sum_{d|A_i, d|A_j, i<j} A_i A_j$ の部分に注目すると、これは、A の要素のうち d の倍数から 2 つ選んでかけたものの総和となっています。これは、各 x $(1 \leq x \leq V)$ について、$A_i = x$ となる i が何個あるかを予め数えておけば、$O(V$ 以下の d の倍数の個数 $)$ で求められます。これを全ての d について行うと、全体で計算量は $O(VlogV)$ になります。

よって、この問題は $O(N + VlogV)$ で解けました。

解答例

D : Unique Path

$M = N - 1$ の場合、すぬけくんがもらったグラフは木です。その場合、どの頂点ついについても単純パスは 1 個です。以下、$M \geq N$ の場合について考えます。

すぬけくんがもらったグラフを G と呼ぶことにします。G の橋について考えてみます。

すぬけくんがもらったグラフの橋だけを残したグラフを H と呼びます。まず、単純パスが 1 つしか存在しない頂点対 a, b は、H において同じ連結成分に属していなければなりません。(そうでない場合は、G において橋以外を通る a, b パスが存在する。ここで、そのパス上の橋以外の辺を一個消しても G が連結であることから、もう一つの a, b パスが存在することになり、矛盾する) また、単純パスが 2 個以上ある頂点対 a, b は、H において異なる連結成分に属していなければなりません。(そうでない場合は、a, b の属する連結成分の辺を上手く選んで 1 つ削除すると a, b が連結であるようにでき、矛盾する。)

よって、単純パスが 1 つしかない頂点間に辺を張ったグラフ I を考えると、単純パスが 2 つ以上存在する頂点対は、I において異なる連結成分に属する必要があるとわかります。

H の連結成分の個数 k を固定してみます。G において、H の異なる連結成分 x, y 間に張られているような辺を考えます。まず、同じ x, y 間に 2 つ以上の辺が張られていてはいけません。よって、G の辺数の最大値は H の辺数 $+ k(k-1)/2 = N - k + k(k-1)/2 = N + k(k-3)/2$ です。また、G の辺数の最小値を考えると、これは N です。(H の各連結成分から代表点を選び、それらを結ぶサイクルを作れば良いです。) また明らかに、すべての $N \leq i \leq N + k(k-3)/2$ について、G の辺をちょうど i 本にすることができます。

よって、H の連結成分数 k はできるだけ大きい方がよく、これは I の連結成分数そのものであることがわかります。あとは、$M \leq N + k(K-3)/2$ であるかどうかを判定すればよいです。

解答例

E : Gachapon

$A'_i = A_i/S$ とします。

包除原理を使うと、以下の問題が解ければよいことになります。

> $\{0, 1, \cdots, N-1\}$ の空でない部分集合 $s = \{i_0, i_1, \cdots, i_{|s|-1}\}$ すべてについて、次の問題を解き、その答えを $|s|$ の偶奇によってそれぞれ $-1, 1$ の重みをかけて足し合わせよ。
>
> - すべての j $(0 \leq j \leq |s|-1)$ について i_j の登場回数が B_{i_j} 回未満であるような状態が、何回続くか期待値を求めよ。

部分集合 s が固定されている場合を考えます。

整数列 $x_0, x_1, \cdots, x_{|s|-1}$ $(0 \leq x_j < B_{i_j})$ について、i_j の登場回数がちょうど x_j 回であるような状態を、状態 x と呼ぶことにします。状態 x になる回数の期待値を考えます。乱数生成器が $|s|$ の中に含まれる数を生成する確率を P とすると、初めて状態 x になった後、それが維持される回数の期待値は $1/P$ です。よって、一度でも状態 x になる確率を数えれば良いです。これは、次式で求めることができます。ただし、$X = \sum_{i=0}^{|s|-1} x_i$ です。

$$X! \prod_{j=0}^{|s|-1} \left(\frac{A'_{i_j}}{P} \right)^{x_j} \frac{1}{x_j!}$$

求めたいのは、整数列 x としてあり得る全てのものについて上式の値を足し合わせたものです。そしてこれは、DP で求めることができます。(DP のキーとして、今 x の先頭何項目まで決めたか、今まで決めた x の項の総和を持てば良いです)

最後に、ありうる全ての $|s|$ について、上の DP を計算することを考えます。すると、上記の DP のキーの他に、今まで s に追加することに決めた i の A_i の総和をキーとして持てば、計算できることがわかります。

DP の計算量は全体で $O\left(\left(\sum_{i=0}^{N-1} A_i\right)\left(\sum_{i=0}^{N-1} B_i\right)^2\right)$ となり、十分高速です。

解答例

F : Two Permutations

$i \to P_i$ の辺を張ったグラフを考えます。このグラフはいくつかのサイクルからなるグラフです。また、A を作る際には、各サイクルについて、そのサイクルに含まれるすべての i について $A_i = i$ にする、またはすべての i について $A_i = P_i$ とする、のどちらかを選ぶことになります。ここで、前者を選ぶ場合を、そのサイクルに 1 を割り当てることに、後者を選ぶ場合を 0 を割り当てることに対応させます。頂点 i の含まれるサイクルを $x(i)$ と書くことにします。

また、$i \to Q_i$ の辺を張ったグラフを考えます。先ほどと同様に、B を作ることと、各サイクルに 0,1 の値を割り振ることを対応させます。ただし今回は、$B_i = i$ とすることに 0 を、$B_i = Q_i$ とすることに 1 を対応させることにします。また、頂点 i の含まれるサイクルを $y(i)$ と書くことにします。

ある i について、$A_i = B_i$ となるかどうかは、$x(i)$ と $y(i)$ に割り当てられた値によって決定されます。具体的には以下のようになります。ここで、サイクル c に割り当てる値を $v(c)$ と書いています。

- $P_i = Q_i = i$ の時: どのようにしても、$A_i = B_i$
- 上記の条件に当てはまらず、$P_i = i$ の時: $A_i = B_i \Leftrightarrow v(y(i)) = 0$
- 上記の条件に当てはまらず、$Q_i = i$ の時: $A_i = B_i \Leftrightarrow v(x(i)) = 1$
- 上記の条件に当てはまらず、$P_i \neq Q_i$ の時: $A_i = B_i \Leftrightarrow v(x(i)) = 1 \ and \ v(y(i)) = 0$
- 上記の条件に当てはまらない時: $A_i = B_i \Leftrightarrow (v(x(i)) = 1 \ and \ v(y(i)) = 0) \ or \ (v(x(i)) = 0 \ and \ v(y(i)) = 1)$

これは、Project selection problem として定式化できます。すると、必要なのは、$O(N)$ 頂点、$O(N)$ 辺、各辺の容量 1 のグラフの最大流になります。これは、dinic 法を使うと $O(N\sqrt{N})$ で解くことができ、十分高速です。

解答例

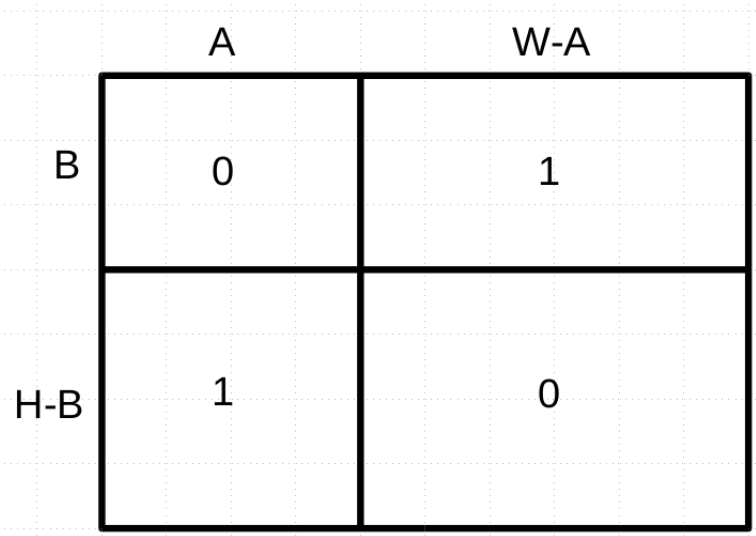
AGC038 Editorial

written by maroonrk

September 21, 2019

A : 01 Matrix

We can always satisfy the conditions by constructing a grid as follows:



Sample Code

B : Sorting a Segment

Let us say i and j are *equivalent* when sorting the interval $[i, i+K)$ and sorting the interval $[j, j+K)$ $(i < j)$ give the same result. Let us consider when i and j will be equivalent.

One case where i and j will be equivalent is the case where the intervals $[i, i+K)$ and $[j, j+K)$ are both already arranged in ascending order.

Let us consider the other cases. First, it is obviously necessary that $[i, i+K)$ and $[j, j+K)$ intersect. Thus, $j < i+K$ is a necessary condition. We also see that sorting the interval $[i, i+k)$ does not change the interval $[i, j)$. This means that the interval $[i, j)$ is already arranged in ascending order, and every element in this interval is smaller than every element in $[j, i+k)$. Similarly, the interval $[i+K, j+K)$ is also already arranged in ascending order, and every element in this interval is larger than every element in $[j, i+k)$.

We then see that, if i and j are equivalent, for any x $(i < x < j)$, x is equivalent to i and j. Particularly, i and $i+1$ are equivalent.

The answer to this problem is the number of connected segments in the graph with vertices $0, 1, \cdots, N-K$ where there is an edge between i and j if and only if i and j are equivalent. From the observation above, we only need to consider edges between i and $i+1$ for the case where $[i, i+K)$ are not initially arranged in ascending order. We can determine whether i and $i+1$ are equivalent by checking if the minimum and maximum elements in $[i, i+K+1)$ are P_i and P_i, P_{i+K}, respectively.

Thus, by applying sliding window minimum algorithm, we can determine whether i and $i+1$ are equivalent for all i in $O(N)$ time. The other computations can also be done in $O(N)$, so the problem can be solved in a total of $O(N)$ time. (With std::set, the implementation will be easier but the complexity becomes $O(NlogN)$, though this is still fast enough.)

Sample Code

C : LCMs

Let $V(= 1000000)$ be the maximum possible value of a given integer. Consider a sequence of rational numbers $w_1, w_2, \cdots, w_V$ that satisfies the following condition:

- For every $1 \leq i \leq V$, $\sum_{d|i} w_d = 1/i$.

We can determine these numbers in ascending order of i, that is, when we have already determined $w_1, w_2, \cdots, w_{k-1}$, we have $w_k = 1/k - \sum_{d|i, d<i} w_d$. The complexity of finding these numbers has the same order as that of the total number of divisors of $1, 2, \cdots, V$, that is, $O(VlogV)$.

We know that $lcm(x, y) = xy/gcd(x, y)$, which can be transformed as follows:

$$xy/gcd(x,y) = xy \times \left(\sum_{d|gcd(x,y)} w_d \right) = xy \times \left(\sum_{d|x, d|y} w_d \right)$$

Thus, the sum in question is equal to:

$$\sum_{i=0}^{N-2} \sum_{j=i+1}^{N-1} lcm(A_i, A_j) = \sum_{i=0}^{N-2} \sum_{j=i+1}^{N-1} A_i A_j \times \left(\sum_{d|A_i, d|A_j} w_d \right)$$

Let us focus on d here and rewrite the formula. Then, we have:

$$\sum_{1 \leq d \leq V} w_d \left(\sum_{d|A_i, d|A_j, i<j} A_i A_j \right)$$

Now, focus on the part $\sum_{d|A_i, d|A_j, i<j} A_i A_j$. This is the sum of the products of all possible pairs of two distinct divisors of d. We can compute it in O(The number of divisors of d not greater than V) time by pre-calculating the number of i such that $A_i = x$ for each x $(1 \leq x \leq V)$. By doing this for all d, the total complexity will be $O(VlogV)$.

Therefore, the problem is solved in $O(N + VlogV)$ time.

Sample Code

D : Unique Path

If $M = N - 1$, Snuke got a tree from his mother. In that case, there is one simple path between every pair of vertices. Below, we will consider the case where $M \geq N$.

Let us call the graph Snuke received G, and consider the bridges in G.

Let H be the graph obtained from G by removing all the edges except the bridges. First, a pair of vertices a, b with only one simple path between them must belong to the same connected component in H. (If not, there exists a a, b path in G that traverses non-bridge edges. Here, if we remove a non-bridge edge on that path, G will be still connected, so there exists another a, b path, and contradiction is reached.) Also, a pair of vertices a, b with two or more simple paths between them must belong to different connected components in H. (If not, we can choose an edge in the connected component containing a and b so that a and b will be connected, and contradiction is reached.)

Therefore, we see the following: Let I be the graph where an edge is only spun between pairs of vertices with only one simple path between them. Then, a pair of vertices with two or more simple paths between them must belong to different connected components in I.

Let us fix the number of connected components in H and call it k. In G, consider edges spun between different connected components x and y of H. First, a pair of x and y must not have two or more edges spun between them. Thus, the maximum possible number of edges in G is (the number of edges in H) $+ k(k-1)/2 = N - k + k(k-1)/2 = N + k(k-3)/2$. Also, we can see that the minimum possible number of edges in G is N. (We can choose one vertex from each connected component of H and make a cycle connecting these vertices.) Additionally, for every $N \leq i \leq N + k(k-3)/2$, we can obviously have exactly i edges in G.

Thus, k, that is, the number of connected components in H should be as large as possible, and we can see that it should be the number of connected components of I itself. Now, we only need to determine if $M \leq N + k(K-3)/2$.

Sample Code

E : Gachapon

Let $A'_i = A_i/S$.

After applying inclusion-exclusion principle, we need to solve the following problem:

> For all non-empty subset $s = \{i_0, i_1, \cdots, i_{|s|-1}\}$ of $\{0, 1, \cdots, N-1\}$, solve the following problem, and find the sum of those answered multiplied by -1 or 1 depending on the parity of $|s|$.
>
> - Find the expected number of turns in which, for every j $(0 \leq j \leq |s|-1)$, i_j has appeared less than B_{i_j} times.

Let us consider the case where the subset s is fixed.

For an integer sequence $x_0, x_1, \cdots, x_{|s|-1}$ $(0 \leq x_j < B_{i_j})$, we will represent the state where i_j has appeared exactly x_j times by state x. Consider the expected number of turns until state x is reached. Let P be the probability that the generator generates a number contained in $|s|$. After state x is reached for the first time, the expected number of turns it will last is $1/P$. Thus, we just need to compute the probability that state x is reached, which can be found by the following formula (here, $X = \sum_{i=0}^{|s|-1} x_i$):

$$X! \prod_{j=0}^{|s|-1} \left(\frac{A'_{i_j}}{P}\right)^{x_j} \frac{1}{x_j!}$$

We want to find the sum of the value above over all possible integer sequence x. This can be done by DP. (The "keys" in DP should be the number of elements we have already decided from the beginning, and the sum of those elements).

Finally, consider doing the above computation for all possible $|s|$. This can be done by adding a new key to the above DP, which is the sum of A_i for all i that we have decided to add to s.

The total complexity of the DP is $O\left(\left(\sum_{i=0}^{N-1} A_i\right)\left(\sum_{i=0}^{N-1} B_i\right)^2\right)$, which is fast enough.

Sample Code

F : Two Permutations

Consider a graph with edges $i \to P_i$. This graph consists of some number of cycles. When we make A, for each cycle, we have two options: have $A_i = i$ for all i contained in the cycle, or have $A_i = P_i$ for all i. Let us make the first option correspond to allocating 1 to the cycle, and the second option correspond to allocating 0 to the cycle. Also, let $x(i)$ denote the cycle that contains Vertex i in this graph.

Consider another graph with edges $i \to Q_i$. Similarly to the above, we will make the process of making B correspond to allocating 0 and 1 to the cycle. This time, however, 0 corresponds to $B_i = i$ and 1 corresponds to $B_i = Q_i$. Also, let $y(i)$ denote the cycle that contains Vertex i in this graph.

For a index i, whether $A_i = B_i$ is determined by the value allocated to $x(i)$ and $y(i)$, more specifically, determined as follows ($v(c)$ means the value allocated to cycle c):

- If $P_i = Q_i = i$: always $A_i = B_i$
- If none of the above applies and $P_i = i$: $A_i = B_i \Leftrightarrow v(y(i)) = 0$
- If none of the above applies and $Q_i = i$: $A_i = B_i \Leftrightarrow v(x(i)) = 1$
- If none of the above applies and $P_i \neq Q_i$: $A_i = B_i \Leftrightarrow v(x(i)) = 1$ *and* $v(y(i)) = 0$
- If none of the above applies: $A_i = B_i \Leftrightarrow (v(x(i)) = 1$ *and* $v(y(i)) = 0)$ *or* $(v(x(i)) = 0$ *and* $v(y(i)) = 1)$

This can be formulated as project selection problem. Now, we need to find the maximum flow in a graph with $O(N)$ vertices and $O(N)$ edges, each with capacity 1. With Dinic's algorithm, this can be done in $O(N\sqrt{N})$ time, which is fast enough.

Sample Code

AGC 037 解説

yutaka1999

2019 年 08 月 17 日

A: Dividing a String

解法 1

条件をみたす分割のうち分割個数が最大である分割を考えると、各 S_i の長さは高々定数（実際には 2）となります。よって、動的計画法により $O(N)$ で答えを求めることができます。

解法 2

文字列 S に対して、求める答えを $f(S)$ とします。さらに与えられる文字列を $s_1s_2\dots s_N$ として $g(i)=f(s_1\dots s_i)$ とします。$i\geq 3$ のとき実は以下の漸化式が成り立ちます。

$$g(i)=\begin{cases}g(i-1)+1 & (s_i\neq s_{i-1})\\ g(i-3)+2 & (s_i=s_{i-1})\end{cases}$$

この漸化式を用いれば先ほどより簡単に $O(N)$ で答えを求めることができます。以下この漸化式を示します。

帰納法により証明します。帰納法の仮定を用いるとまず g が単調増加であることが分かるので、$s_i\neq s_{i-1}$ の場合は $g(i)=g(i-1)+1$ となります。そうでない場合について考えます。$s_1s_2\dots s_i$ の条件をみたす分割を $S_1S_2...S_I$ とおくと $S_{I-1}=S_I=s_i$ となることはないので、$|S_{I-1}|+|S_I|\geq 3$ となります。これより $S_1S_2...S_{I-2}=s_1s_2\dots s_j$ $(j\leq i-3)$ となるので $I-2\leq g(j)\leq g(i-3)$ が成り立ちます。よって $I\leq g(i-3)+2$ であるから $g(i)\leq g(i-3)+2$ です。また $g(i)\geq g(i-3)+2$ であることは、$s_1s_2\dots s_{i-3}$ の条件をみたす $f(i-3)$ 個への分割を取り、$s_{i-2}s_{i-1}s_i$ をうまく 2 分割して先の分割と連結すると、$s_1s_2\dots s_i$ の $g(i-3)+2$ 個への分割が取れることから分かります。よって、この場合も $g(i)=g(i-3)+2$ となるので、漸化式が示されました。

B: RGB Balls

$\sum_j (c_j - a_j)$ の最小値をまず求めます。R, G, B それぞれの色について、その色を持つボールの番号を小さい順に $r_1 < r_2 < \ldots < r_N, g_1 < g_2 < \ldots < g_N, b_1 < b_2 < \ldots < b_N$ として、$m_i = \min(r_i, g_i, b_i), M_i = \max(r_i, g_i, b_i)$ とします。このとき $\min \sum_j (c_j - a_j) = \sum_i (M_i - m_i)$ となります。これを示します。

人に適当な順序を与えて、$a_1 < a_2 < \ldots < a_N$ とします。$i \leq j$ のとき $a_i \leq a_j < b_j < c_j$ であるので、各 i に対し、番号が a_i より小さいボールの割り当てられている人は人 1 から人 $(i-1)$ のいずれかです。特に番号が a_i より小さいボールには各色のボールは高々 $i-1$ 個しかないため、$a_i \leq r_i, g_i, b_i$ つまり $a_i \leq m_i$ となります。よって $\sum_j -a_j \geq \sum_i -m_i$ が成り立ちます。同様にして、人に適当な順序を再び与え $c_1 < c_2 < \ldots < c_N$ となるようにすれば、$c_i \geq M_i$ となり $\sum_j c_j \geq \sum_i M_i$ が成り立ちます。よって、$\sum_j (c_j - a_j) \geq \sum_i (M_i - m_i)$ となります。この等号が成立するようにボールを分配することができるのは明らかです。（人 i にはボール r_i, g_i, b_i を配ればよいです。）よって $\sum_j (c_j - a_j)$ の最小値が求まりました。

次に $\sum_j (c_j - a_j)$ を最小にする配り方の個数を求めます。$\sum_j (c_j - a_j) \geq \sum_i (M_i - m_i)$ を示す過程で、$a_i \leq m_i$ や $c_i \leq M_i$（この 2 つの式における人の順序は異なることに注意して下さい）といった不等式が成り立ちましたが、これらの等号成立が $\sum_j (c_j - a_j) \geq \sum_i (M_i - m_i)$ の等号成立条件です。つまり、$A = \{m_i | 1 \leq i \leq N\}, C = \{M_i | 1 \leq i \leq N\}, B = \{1, 2, \ldots, 3N\} \backslash (A \cup C)$ としたときに、以下の条件をみたすボールの配り方を数えればよいです。

- j 番目の人が受け取ったボールの番号を小さい順に $a_j < b_j < c_j$ とする。
- このとき $a_j \in A, b_j \in B, c_j \in C$

B に属する各ボールに対して、それより番号の小さい A に属するボールとそれより番号の大きい C に属するボールを割り当てる方法を求めればよいので、簡単に数えることができます。以上により、この問題を $O(N)$ の計算量で解くことができます。ただし、人の区別をするので、答えに $N!$ をかけるのを忘れないよう気を付けてください。

C: Numbers on a Circle

操作を逆方向にして考えると、以下のようになります。

言い換え

i 番目の数は B_i です。以下の操作を繰り返して i 番目の数を A_i にします。必要な操作の最小回数を求めてください。

- $1 \leqq i \leqq N$ なる整数 i を一つ選び、$i-1, i, i+1$ 番目の数をそれぞれ a, b, c とする。
- i 番目の数を $b-a-c$ に置き換える。

上と同様、$1 \leqq i \leqq N$ なる整数 i を一つ固定し、$i-1, i, i+1$ 番目の数をそれぞれ a, b, c とします。与えられる数がすべて正なので、常に各数が正である必要があります。このとき次のことが分かります。

- i 番目の数を変更するためには $b > a+c$ が必要。
- その条件が成立しているとき、i 番目以外の数に対してどのように操作を行ったとしても a, c の値は変化しない。
- よって、i 番目の数を変更する必要があり、かつ $b > a+c$ をみたしているならば、他の操作を行う前に i 番目の数を変更する操作を行ってよい。
- つまり、$b > a+c$ かつ変更する必要のある i を探し、その i に対して操作を行うことを繰り返してよい。

特に条件をみたすように操作を繰り返したとき、かかる操作回数は常に一定であることが分かります。ただし、操作回数自体はとても大きくなる可能性があるので、愚直にシミュレーションするのでは間に合いません。そこで、以下のような工夫が必要です。

- $b > a+c$ かつ変更する必要のある i を探し、その i に対して b が A_i と一致するまでもしくは $b \leq a+c$ となるまで i **に対して繰り返し**操作を行う。

このように行うことにするとシミュレーションにかかるステップ数が十分少なくなります。というのも、b が A_i と一致するように操作を行うのは合計で高々 N 回であり、$b \leq a+c$ となるまで操作を行うのは、行うごとに b の値が半分以下になるため、合計で高々 $N \log M_B$ 回（M_B は B_i の最大値）であるからです。（ただし、$x \geq y$ に対して x を y で割ったあまりが $\frac{x}{2}$ 以下であることを用いました。）よって、後は先程記した操作をシミュレーションする方法を考えればよいです。一例をあげると、以下のように解くことができます。

- 優先度付きキュー等を用いて、変更する必要がある数の内、最大の数を取る。
- その数に対して先程の操作を行う。

計算量は $O(N \log N \log M_B)$ です。

D: Sorting a Grid

可能な手順が3段階あるので、このような問題ではまず条件を整理することが重要です。手順3を終えた時点でみたしているべき条件が与えられているので、手順$k(k \geq 1)$を終えた時点でみたしているべき条件というのをkが大きい方から順に整理すると以下のようになります。

- 手順2を終えた時点で、上からi行目は$M \times (i-1)+1$〜$M \times (i-1)+M$からなる。
- 手順1を終えた時点で、各列は以下のように構成されている。
  - 1以上M以下の数が1つ
  - $M+1$以上$2M$以下の数が1つ
  - ...
  - $MN-M+1$以上MN以下の数が1つ

ここで以下のような二部グラフを考えます。

- $2N$頂点NM辺からなり、各頂点は$S_1, S_2, \ldots, S_N, T_1, T_2, \ldots, T_N$とラベル付けされている。
- 1からNMの各数jについて、jが属する行をi行目とする。
- このとき、jに対応する辺e_jが頂点S_iと頂点$T_{[\frac{j+M-1}{M}]}$を繋いでいる。

つまり、手順1を終えた時に、各列に属する数に対応する辺が完全マッチングをなすことが条件であるので、先の二部グラフをM個の完全マッチングに分割することができればよいです。ここで、先の二部グラフの次数がすべて等しくMであることに注目します。実は、ホールの定理から各頂点の次数が等しい二部グラフは常に完全マッチングを持つことが分かるので、適当な完全マッチングを見つけ、それを除いた二部グラフで再び完全マッチングを見つける、ということを繰り返すことで、条件をみたすように手順1を行うことができます。計算量は一つのマッチングを見つけるのに$O(NM\sqrt{N})$ですので$O(NM^2\sqrt{N})$となります。

E: Reversing and Concatenating

S に現れる文字の中で辞書順最小のものを a とします。最終的に S の先頭に a を何文字連続させることができるかを考えます。最初の操作での U において a が最大 L 文字連続しているとすると、実は S の先頭に並べることができる a の個数は最大で $2^{K-1}L$ 個であり、実際に $2^{K-1}L$ 個並ぶように操作をする方法はとても少ないです。

まず高々 $2^{K-1}L$ 個しか a を並べられないことを示します。これは操作を一回行った後の S において a が連続する個数は高々 L であり、操作をもう一度行っても S において a が連続する個数は高々2倍にしかならないことから分かります。次に実際に $\min(N, 2^{K-1}L)$ 個 a を連続させられることを示します。i 回目の操作において、以下のように操作を行うことで a をその個数分先頭に並べることができまｓｙ。

- $i < K$ のとき U において a が $2^{i-1}L$ 個連続する部分が末尾となるように次の S を選ぶ。
- $i = K$ のとき U において a が $2^{i-1}L$ 個連続する部分が先頭となるように次の S を選ぶ。

よって、$2^{K-1}L \geq N$ の場合は a を N 個並べた文字列が答えとなります。そうでない場合は、最終的に a が $2^{K-1}L$ 個先頭に並ぶ必要がありますが、その条件を満たすように操作をする方法は、上に述べた方法しかないです。よって、最初の操作でどの S を選ぶかを決めると、上の条件を満たすように操作をする方法が一意に定まるので、最終的な解の候補が定まります。高々 N 個の解の候補の中で辞書順最小のものを求めるのは $O(N^2)$ で可能であるので、全体として $O(N^2)$ の計算量でこの問題を解くことができます。

F: Counting of Subarrays

ある正整数 K に対してレベル (K, L) に属する数列を**良い数列**と呼ぶことにします。まず数列 $A_1, A_2, \ldots, A_N$ が良い数列であるかどうかを判定する方法を考えましょう。まず $N = 1$ の場合は明らかに良い数列です。$N \geq 2$ の場合は以下が必要十分であることが分かります。

- 数列が一つの値のみからなる場合 $N \geq L$ が必要十分。
- そうでない場合、最小の値 m をとるインデックスの集合を S とする。
- S が $[l_1, r_1], \ldots, [l_k, r_k] (l_i + 1 < r_{i+1})$ の和集合となるように k 個の区間を選ぶ。
- $[l_i, r_i]$ に含まれる要素数が全て L 以上であることが必要。
- そのうえで $[l_i, r_i]$ に含まれる要素を A から削除して、対応する部分に値 $m + 1$ の項を $[\frac{r_i - l_i + 1}{L}]$ 個加えた数列が良い数列であることが必要十分。

では良い部分列の個数はどのようにして数えればよいでしょうか。まず問題を少し拡張して、以下のような問題を解くことにします。

- 数列 A に加えて、数列 $L_1, \ldots, L_N, R_1, \ldots, R_N$ が与えられる。
- このとき長さ 2 以上の良い部分列 $A_i, \ldots, A_j$ に対して $L_i \times R_j$ を足し合わせた値を求める。

実はこのように問題を拡張すると、先ほどの判定法と同様にして最小値を削除し一つ大きな値を適当な個数加える、という操作を行うことでより短い数列に帰着することができます。具体的には以下のようにすればよいです。

- 数列が一つの値のみからなる場合は、よい部分列は長さ L 以上の部分列すべてなので簡単に計算できる。
- そうでない場合、先ほどと同様に区間 $[l_1, r_1], \ldots [l_k, r_k]$ をとる。
- $[l_i, r_i]$ に含まれる長さ L 以上の部分列すべてに対する $L_i \times R_j$ の値の合計値を求める。
- $[l_i, r_i]$ に含まれる要素を A から削除して、対応する部分に値 $m + 1$ の項を $[\frac{r_i - l_i + 1}{L}]$ 個加えた数列を考える。

ここで、新たに追加する $[\frac{r_i - l_i + 1}{L}]$ 個の要素について L_j, R_j の値をどのように定めればよいか考える必要があります。ここで辻褄を合わせるには、以下のように定めればよいです。

- $[l_i, r_i]$ の区間を削除して、そこに新たな項を加える場合を考える。
- 新たに加える項の中で左から j 個目の項の R_j の値を考える。
- $[l_i, r_i]$ の中で左から jX 項目から $jX + X - 1$ 項目までの R の値の合計を R_j とする。

このように定めればよいことは、良い数列かどうかの判定法から分かります。ただし、少しスコアに余分があるので適当な値を引かなければいけません。具体的には以下の値を引けば求めたい値が求まります。

- $[l_i, r_i]$ の区間を削除して新たに追加してできる区間を $[l'_i, r'_i]$ とする。
- $[l'_i, r'_i]$ の長さ L 以上の部分列に対する $L_i \times R_j$ の合計を求める。

以上により求めたい値が求まることは、判定法のアルゴリズムから従います。計算量は解析すると $O(N \log N)$ であることが分かります。これはアルゴリズムに対応する X 分木のようなものを考えると、その頂点数が $O(N)$ であることから従います。以上により、この問題を $O(N \log N)$ で解くことができました。

AGC 037 Editorial

yutaka1999

A: Dividing a String

We can prove that in the optimal solution, the length of each part is at most 4.

For example, suppose that the length of some part is 5. There are four ways to divide it into two parts: "1+4", "2+3", "3+2", and "4+1". At most two of them may be invalid. For example, "2+3" is invalid if the part immediately to the left have length 2 and the string coincides with the first two letters of the current part, or a similar thing for the part immediately to the right. Since there are four divisions and at most two of them are invalid, we can always find a valid division.

(With more careful analysis we can show that the length is at most 2, but this is not necessary to solve the problem).

Using this fact, a straightforward DP works in $O(N)$.

B: RGB Balls

Let $r_1 < r_2 < \ldots < r_N, g_1 < g_2 < \ldots < g_N, b_1 < b_2 < \ldots < b_N$ be the positions of R, G, B balls, respectively. Let $m_i = \min(r_i, g_i, b_i), M_i = \max(r_i, g_i, b_i)$. Then, we claim that the minimum possible value of

Let k be an arbitrary integer. Since k people get one of the red balls $r_1, ..., r_k$, the value of a_* for them is less than or equal to r_k. The same holds for green and blue balls, so at least k people have the value of a_* less than or equal to m_k. This proves that $\sum_j a_j \leq \sum_i m_i$. By addind a similar inequality for c and M, we get $\sum_j (c_j - a_j) \geq \sum_i (M_i - m_i)$. It's obvious that we can satisfy this equation (for example person i gets r_i, g_i, b_i), so we want to count the number of ways to satisfy the equation.

Let A be the set of balls that appear in m, C be the set of balls that appear in M, and B be the set of all other balls. Then, the equation holds if and only if each person gets one ball each from A, B, C, and the three balls satisfy the relative order "$A < B < C$".

We handle the $3N$ balls from left to right. When we handle a ball, we count the number of people who can receive the ball without breaking the rule above, and multiply it to the answer. It turns out that this way the intermediate state is always (essentially) unique, so the answer is simply the product of those coefficients.

C: Numbers on a Circle

Let's see the operations in the reverse order:

> You want to convert the sequence B into A with minimum number of the following operations:
>
> - Choose i and let a, b, c be the $i-1, i, i+1$-th terms.
> - Replace the i-th term with $b - a - c$.

Let i be an arbitrary indice and a, b, c be the $i-1, i, i+1$-th terms. We get some observations:

- $b > a + c$ must hold to perform an operation at i.
- If $b > a + c$ holds, no matter how we perform operations at other places, the values a, b, c never change.
- Thus, if $B_i > A_i$ and $b > a + c$, we can assume that we perform the next operation at i.

Thus, a (slow) solution is the following: Find i that satisfies $B_i > A_i$ and $b > a + c$, and perform an operation at i. Repeat it until B coincides with A.

To make it faster, do the following:

- With a priority queue, keep the set of indices that satisfy $B_i > A_i$, in the decreasing order of B_i.
- While the queue is non-empty, pop the first element and see the index (call it i). If we can't perform the operation at i, the answer is -1 (because we can never satisfy $B_i = A_i$). Otherwise, replace B_i with $B_i \% (B_{i-1} + B_{i+1})$ with $[B_i / (B_{i-1} + B_{i+1})]$ operations.

It's known that if $x > y$, $x \% y \leq x/2$. Thus, each time we perform the operation above on i, B_i is halved, and the total number of opeartions is $O(N \log MAX_B)$.

This solution works in $O(N \log N \log MAX_B)$ time.

D: Sorting a Grid

In the decreasing order of k, let's determine the desired state of the grid after Phase k.

- After Phase 2, the i-th row must be a permutation of $(i-1)M+1, \cdots, iM$. Let's "color" these integers with Color i. Then, after Phase 2, the colors of all squares in the i-th row must be i.
- In Phase 2, we can shuffle each column, and as a result we want to have Color 1 at the top square, Color 2 at the next square, and so on. Thus, after Phase 1, we want each column to contain exactly one square of each color.
- At the beginning, the grid contains exactly M occurrences of each of Color 1, $\cdots$, Color N.

Thus, the problem can be restated as follows:

> You are given an $N \times M$ grid consisting of exactly M squares of each of Color 1, $\cdots$, Color N. You can freely swap two cells in the same row. Find a way to swap the cells such that each column contains each color exactly once.

We decide the colors column by column. First, let's decide the colors of the leftmost column.

Consider the following bipartite graph:

- There are N vertices corresponding to Row 1, ..., Row N.
- There are N more vertices corresponding to Color 1, ..., Color N.
- If Row i contains a square with Color j, add an edge between two vertices corresponding to Row i and Color j.

Let's find a perfect matching of this bipartite graph (this is always possible as described below). If Row i is matched with Color j in the matching, we decide the color of the leftmost square of Row i to be Color j.

Why can we always find a complete matching? This comes from the fact that the degrees of all vertices are the same (M), and all such bipartite graphs have perfect matcihngs. (We can prove this by, for example, Hall's marriage theorem).

Now we fix the leftmost column and consider the remaining $N \times (M-1)$ grid; in this grid each color appears exactly $M-1$ times, so the same

condition holds and we can find a perfect matching again. Thus, we can repeat the same process M times, and achieve the goal.

This solution works in $O(NM^2\sqrt{N})$ time.

E: Reversing and Concatenating

Let a be the lexicographically smallest letter that appears in S.

We want to mazimize the number of consecutive occurrences of a at the beginning of the resulting string (call it M). Let L be the maximum length of consecutive occurrences of a in U (in the beginning). Then, we can prove that $M = 2^{K-1}L$ (unless $2^{K-1}L > N$, in this case $M = N$). This is because, in each step, we can always double the length of consecutive occurrences of a in U (unless it exceeds N), and this is the optimum.

Thus, in case $2^{K-1}L \geq N$, the answer is the concatenation of N as. Otherwise, in order to achieve $M = 2^{K-1}L$, we have limited options at each step. In the i-th step,

- If $i < K$, we must choose S' in U such that $2^{i-1}L$ consecutive occurrences of a comes at the end of S'.
- If $i = K$, we must choose S' in U such that $2^{i-1}L$ consecutive occurrences of a comes at the beginning of S'.

Notice that, once we decide the choice of S' when $i = 1$, the choice of S' in the future can be uniquely determined. There are $O(N)$ choices for the case $i = 1$, and we can get the resulting string for a fixed choice in $O(N)$, so this solution works in $O(N^2)$ time.

F: Counting of Subarrays

Let's call a sequence **good** if it belongs to level (K, L) for some K.

First, let's check if the entire sequence $(A_1, A_2, \ldots, A_N)$ is good. In case $N = 1$, it's good. Otherwise, we can check in the following way:

- If all elements of the sequence are the same, the sequence is good iff $N \geq L$.
- Otherwise, let m be the minimum of the sequence, and let $[l_1, r_1] \cup \cdots \cup [l_k, r_k]$ be the set of indices where the elements are m (here, the intervals are not adjacent, i.e., $r_i + 1 < l_{i+1}$)
- If the length of some interval $[l_i, r_i]$ is less than L, the sequence is not good.
- Otherwise, for each i, replace the interval $[l_i, r_i]$ (that is currently $(r_i - l_i + 1)$ repetitions of m) with $[\frac{r_i - l_i + 1}{L}]$ repetitions of $m + 1$; then recursively check if the new sequence is good.

What's the time complexity of this solution? Each time we handle L elements, the length of the sequence decreases by $L - 1$, so we need to handle $O(N)$ elements in total. Thus, if we keep the elements with sets, it's $O(N \log N)$.

How to count the number of good (continuous) subsequnces? Notice that if Y is a subsequence of X, the sequences we get in intermediate steps when we run the algorithm for Y is also a subsequence of the intermediate sequences of X. Thus, basically, we run the same process, but with "coefficients" associated with elements.

Let's extend the problem a bit. You are additionally given parameters $L_1, \ldots, L_N, R_1, \ldots, R_N$, and your task is to compute the sum of $L_i R_j$ for all pairs (i, j) such that the sequence $A_i, \ldots, A_j$ is good. Then, you can solve the problem in a similar way as the observation above.

- If all elements of the sequence are the same, the answer is the sum of $L_i R_j$ for all pairs (i, j) such that $i = j$ or $j = i + 1 \geq L$, and it can be computed in linear time.
- Otherwise, define m and $l_1, r_1, \cdots, l_k, r_k$ in the same way as above.
- For each i, consider the sequence corresponding to the interval $[l_i, r_i]$, and compute the sum of $L_p R_q$ of good intervals $[p, q]$ within this interval.

- For each i, replace the interval $[l_i, r_i]$ with $[\frac{r_i - l_i + 1}{L}]$ repetitions of $m+1$, with proper values of Ls and Rs associated with those new elements.
- Recursively compute the answer for the new sequence.

How should we define Ls and Rs for the new elements?

We describe it with an example (but works for general cases in the same way). Suppose that $m = 1$, $L = 3$, and the sequence starts with ten 1s, followed by some greater terms.

1, 1, 1, 1, 1, 1, 1, 1, 1, 1, a, b, c, d, e

If we "compress" this sequence, we get the following:

2, 2, 2, a, b, c, d, e

Here, the bold part in the former sequence will be compressed to the bold part in the latter sequence:

1, 1, **1, 1, 1, 1, 1, 1, 1, 1, a, b, c**, d, e

2, **2, 2, a, b, c**, d, e

This means that the value L_3 in the former sequence should be added to the L of the second 2 in the latter sequence.

In this manner, we get the values of L of the three 2s of the compressed sequence should be $L_1 + L_2$, $L_3 + L_4 + L_5$, and $L_6 + L_7 + L_8$, respectively.

Don't forget to avoid extra terms to be added to the answer. Intervals that are entirely contained in the newly added part doesn't correspond to anything in the original sequence, so we should subtract the sum of $L_p R_q$ of good intervals $[p, q]$ within the newly added interval from the answer.

This solution works in $O(N \log N)$ time for the same reason as above.

AGC036 解説

writer : maroonrk

2019 年 7 月 21 日

For International Readers: English editorial starts on page 8.

A : Triangle

$X_1 = 0, Y_1 = 0$ と固定して考えてみます。すると三角形の面積は $|X_2 \times Y_3 - Y_2 \times X_3|/2$ になります。よって、$X_2 \times Y_3 - Y_2 \times X_3 = S$ を満たす X_2, Y_2, X_3, Y_3 を見つけられれば十分です。

ここでさらに、$X_2 = 10^9, Y_2 = 1$ と固定してみます。あとは、$10^9 \times Y_3 - X_3 = S$ を満たす X_3, Y_3 を見つければ良いです。これは S を 10^9 で割った商と余りから求めることができます。詳しくは回答例を参照してください。

回答例

B : Do Not Duplicate

$X_0 = X_i$ を満たす最小の $i > 0$ を考えます。(そのような i がないこともありますが、とりあえずあると仮定します) すると、$X_0, X_1, \cdots, X_i$ を処理した段階で、s は空になっています。次に、$X_{i+1} = X_j$ を満たす最小の $j > i+1$ を考えると、X_j を処理した段階でも s は空です。

一般に、各 i $(0 \leq i \leq N \times K - 1)$ について、次に $X_j = X_i$ なる最小の $j > i$ が存在する時、X_i を処理する直前で s が空なら、X_j を処理した直後、つまり X_{j+1} を処理する直前も s が空です。ここで、$y(i) = (X_j = X_i$ なる最小の $j > i($ そのような j が存在しないなら $\infty)) + 1 - i$ と定義します。すると問題は、次のように言い換えられます。

- 最初、整数 $a = 0$ を持っている。
- ステップ 1: $a + y(a) \leq K$ である限り、$a := a + y(a)$ と置き換え続ける。
- ステップ 2: ステップ 1 が終わったら、$i = a, a+1, \cdots, N \times K - 1$ について、問題文中の操作を行う。

ステップ 1 が終了した時点で、$N \times K - 1 \leq a$ が成り立つことは容易に確認できます。なので、ステップ 2 の実行は簡単です。

ステップ 1 を高速に処理します。X の性質から、$y(i) = y(i \mod N)$ であることがわかります。そこで、ダブリングをします。各 $k(0 \leq k \leq log_2(N \times K)), i(0 \leq i \leq N-1)$ について、$g_k(i) = (i := g(i) + i$ と置き換える操作を 2^k 回行ったあとの $i) - i$ と定義します。あとはこうして得た g_k を使えば、ステップ 1 が終了する段階の a がわかります。

ステップ 1 の実行に $O(Nlog(NK))$ かかります。ステップ 2 は $O(NlogN)$ で実行可能です。よってこの問題は $O(Nlog(NK))$ で解けます。

回答例

C : GP 2

非負整数列 $p_0, p_1, \cdots, p_{N-1}$ が、M 回の操作後の x としてありうることの必要十分条件は、以下の条件を全て満たすことです。

- $p_0 + p_1 + \cdots + p_{N-1} = 3M$
- $\max(p_0, p_1, \cdots, p_{N-1}) \leq 2M$
- p_i が奇数となるような i の個数が M 個以下。

この条件の必要性は明らかです。十分性は、M に関する帰納法で示します。まず、$M = 1$ のとき、明らかに正しいです。次に、$M = k$ でこの命題が正しいとします。$M = k+1$ において条件を満たす p があったとします。このとき、適切に 2 つの添字 i, j を選び、p_i を $p_i - 2$ で置き換え、p_j を $p_j - 1$ で置き換えると、$M = k$ において条件を満たす p が得られます。これは場合分けによって示すことができます。ここでは詳細は省略しますが、$\max(p_0, p_1, \cdots, p_{N-1})$ と、p_i が奇数となるような i の個数に注目して場合分けをすればいいです。

上で示した 3 つの条件を満たす p の個数を数えることにします。まず、2 番目の条件を無視して考えてみます。ここで、p_i が奇数となる i の個数を固定し、a 個だとします。するとまず、p_i が奇数になる i の選び方が ${}_N C_a$ 通り考えられます。次に、総和が $3M - a$ になる長さ N の偶数列が何通りか数える必要がありますが、これは、総和が $(3M - a)/2$ になる長さ N の整数列の個数と同じです。そしてこれは、${}_{(3M-a)/2+N-1} C_{N-1}$ 個あります。コンビネーションの計算は、適切に前計算をしておけば 1 回あたり $O(1)$ で求められます。前計算に必要な計算量は $O(N + M)$　です。a が $O(min(N, M))$ 通りあるので、全体で $O(N + M)$ で答えが求められます。

2 番目の条件を無視した場合が求まりました。あとは、2 番目の条件に違反する p の個数を数えればよいです。p の総和が $3M$ なので、$p_i > 2M$ となる i は高々 1 つです。$p_0 > 2M$ となる p の個数が求まれば、これを N 倍すれば答えが得られます。よって、次のような問題が解ければ良いです。

> 長さ N の非負整数列 q であって、次の条件を全て満たすものの個数を数えよ。
>
> - $q_0 + q_1 + \cdots + q_{N-1} = M$
> - q_i が奇数となるような i の個数が M 個以下。
> - $q_0 > 0$

ここで、3 番目の条件を無視すると、この問題は先程解いた問題と同じ形をしているので、すぐに解けます。3 番目の条件に違反するものを考えると、それは、先程の問題で $N := N - 1$ とした場合に対応しています。

よってこの問題は全体で $O(N + M)$ で解けます。

回答例

D : Negative Cycle

辺をいくつか削除して、負閉路がなくなったという状態を考えます。この時、次の条件を満たすように、各頂点に整数 p_i を設定できます。

- 各辺 $e = (i \to j)$ について、$p_j \leq p_i + weight(e)$ が成り立つ。

これは例えば $p_i =$ 頂点 0 から頂点 i までの最短距離、と定義すればよいです。逆に、この条件を満たすように p_i を設定することができるなら、グラフに負閉路が存在しないことがわかります。(負閉路が存在すると仮定した場合、そこに沿って $p_j - p_i \leq +weight(e)$ を足し合わせると矛盾)

p_i が満たすべき条件を確認します。まず、各 i $(0 \leq i \leq N-2)$ について、$p_i \geq p_{i+1}$ が成り立つ必要があります。これは、もともとある辺 $(i \to i+1)$ (重み 0) に対応しています。ここで、$q_i = p_i - p_{i+1}$ と定義します。$p_i \geq p_{i+1}$ より、q_i は非負整数です。

次に、辺 $(i \to j)$ (重み 1) について考えると、$p_j \leq p_i + 1$ が成り立つ必要があります。この不等式を q を使って表すと、$q_j + q_{j+1} + \cdots + q_{i-1} \leq 1$ となります。

同様に、辺 $(i \to j)$ (重み -1) について考えると、$q_i + q_{i+1} + \cdots + q_{j-1} \geq 1$ が成り立っている必要があります。

使う辺集合を決めた時、条件を満たすように q の値を設定できるか、という問題を考えます。すると、q の要素としては $0, 1$ だけを用いればよいことがわかります。

よって、次のような問題を解けば良いです。

> $0, 1$ からなる数列 q を設定する。この q で条件が満たされないような辺を削除する。このとき、削除する辺のコストの総和を最小化する。

これは、次のような DP で解くことができます。

- $dp[i][j] = q_0$ から q_i までの $0, 1$ を決定していて、そのうち最後の 1 の位置が i、最後から 2 番目の 1 の位置が j になっているときの、既に削除することが決定した辺のコストの総和の最小値。

$dp[i][j] \to dp[x][i]$ の遷移を考えます。まず、新たに削除する必要のある重み 1 の辺は、$j < a \leq i,\ x < b$ を満たす辺 $b \to a$ です。新たに削除する必要のある重み 1 の辺は、$i < a < b \leq x$ を満たす辺 $a \to b$ です。この遷移は累積和を用いると高速化できます。するとこの DP は $O(N^3)$ 時間で計算でき、この問題を解くのに十分高速です。

回答例

E : ABC string

もとの文字列で同じ文字が隣接している場合は、まとめて 1 文字と考えて良いです。よって、S では隣接する 2 文字が全て異なると仮定します。

S の中で'A','B','C' の個数をそれぞれ a, b, c とします。ここで、$a \leq b, a \leq c$ と仮定しても一般性を失いません。

今、使う 'A' の集合が決まっているとしましょう。このとき、残りの 'B','C' を上手く消すことで、条件を満たす部分列を得られるかどうかを判定することを考えます。

まず、使わない 'A' を削除しておきます。これによって、同じ文字が隣合うことがあるので、それを 1 つにまとめる処理をします。こうして得られた文字列を t とします。また、t に含まれる 'A','B','C' の個数をそれぞれ、p, q, r とします。'A' を 1 文字消すごとに、'B' または 'C' は高々 1 文字しか減らないので、$p \leq q, p \leq r$ が成り立っています。

もしここで $q = r$ なら、'B','C' を上手く消すことで、条件を満たす部分列が得られます。具体的には、$p < q$ を満たす限り、'BC' または 'CB' と 2 文字が隣り合っている部分を選んで消す（このとき 'A' が新たに隣合うことがない）という操作が可能です。これは、操作が不可能な状態（2 つの 'A' に挟まれている部分の長さは 2 以下であり、先頭、末尾にある 'B','C' の個数はそれぞれ 1 以下）を考えると、その段階で既に $2p \geq q + r$ が満たされていることからわかります。

$q \neq r$ の場合について考えます。$q < r$ と仮定しても一般性を失いません。t は、'A' によって $p + 1$ 個の（空かもかもしれない）部分文字列に分割されます。ここで、分割された文字列のうち、長さが 1 であって、2 つの 'A' に挟まれていたもの（つまり、t の先頭や末尾にあったものではないもの）を、単体文字列と呼ぶことにします。この $p + 1$ 個の文字列のうち、'B' を含む文字列の個数を x、'C' からなる単体文字列の個数を y とおきます。ここで、$x < y$ の場合、t からどう 'B','C' を削除しても、'B' と 'C' の個数は同じになりません。逆に、$x \geq y$ の場合、適切に 'C' を削除し続けることで、'B' と 'C' の個数を同じにすることができます。これは、'B' と 'C' の個数の差について注目したとき、'B' を含む文字列なら 'B' が 1 つ多い状態にすることができ、'C' からなる単体文字列の場合は 'C' が 1 つ多い状態にしかできないということからわかります。

もとの問題に戻ります。以上の考察から、この問題は、使う 'A' を上手く選んで、'A' によって分離された部分文字列が次の条件を満たすようにする問題だとわかります。

- 'B' からなる単体文字列の個数が 'C' を含む文字列の個数以下である。
- 'C' からなる単体文字列の個数が 'B' を含む文字列の個数以下である。

まず、'A' をすべて使うと考えてみます。そして、$b1, b2, c1, c2$ を以下のように定義します。

- $b1$ = 'B' を含む文字列の個数
- $b2$ = 'B' からなる単体文字列の個数
- $c1$ = 'C' を含む文字列の個数
- $c2$ = 'C' からなる単体文字列の個数

もし、$b2 \leq c1$ かつ $c2 \leq b1$ ならば、すべての 'A' を使うとしてよいです。以下、一般性を失わずに $b2 > c1$ と仮定します。ある 'A' を使わないことにすると、その両隣の部分文字列を連結することになります。このとき、$b1, b2, c1, c2$ の値はそれぞれ高々 1 減ることになります。また、'B' からなる単体文字列に隣接する 'A'

を消した場合は、$b2$ は 1 減少し、$c1$ は変化しません。よって、'A' を削除する必要のある最小回数は $b2 - c1$ です。'A' の削除が終わった段階では、$b2 = c1$ なので、明らかに $c2 \leq b1$ は成立しています。よって、$b2 - c1$ 回 'A' を削除すれば条件を満たすことができ、またこれが条件を満たす最長の部分列を生成します。

あとは、以上の構築をそのまま実装すれば良いです。全体で $O(N)$ の時間でこの問題は解けます。

回答例

F : Square Constraints

包除原理を使います。各 k $(0 \leq k \leq N)$ について、次の問題が解ければいいです。

> k 個の添字 $0 \leq x_0 < x_1 < \cdots < x_{k-1} \leq N-1$ を選ぶ。それぞれの選び方について、$x_i^2 + P_{x_i}^2 < N^2$ (for all $0 \leq i \leq k-1$), $i^2 + P_i^2 \leq (2N)^2$ (for all $0 \leq i \leq 2N-1$) を満たす順列を数え、その総和を求める。

各 i $(0 \leq i \leq N-1)$ について、$f(i) = (a^2 + i^2 < N^2$を満たす最大の a $(a \leq 2N-1))+1$ と定義します。また、各 i $(0 \leq i \leq 2N-1)$ について、$g(i) = (a^2 + i^2 \leq (2N)^2$を満たす最大の a $(a \leq 2N-1))+1$ と定義します。

k 個の添字 x の選び方が決まっているとき、この問題は簡単です。各 i について、制約が $P_i < h(i) = f(i)$ or $g(i)$ とかけるとします。$h(i)$ を昇順にソートしたものを $h'_0, h'_1, \cdots, h'_{2N-1}$ とすると、条件を満たす順列の個数は $(h'_0 - 0) \times (h'_1 - 1) \times \cdots \times (h'_{2N-1} - (2N-1))$ です。

x が固定されていない場合について考えます。まず、$f(i)$ $(0 \leq i \leq N-1)$ と $g(i)$ $(0 \leq i \leq 2N-1)$ を全てまとめて昇順ソートした列を $A_0, A_1, \cdots, A_{3N-1}$ とします。A のそれぞれの要素に対し、それが $f(i)$ $(0 \leq i \leq N-1)$ 由来なら 'a'、$g(i)$ $(0 \leq i \leq N-1)$ 由来なら 'b'、$g(i)$ $(N \leq i \leq 2N-1)$ 由来なら 'c' というラベルをつけておきます。これらのラベルをならべてみると、'aaccacac...bbbb...' という風になっています（※この文字列は適当です)。正確に言えば、全ての 'b' の前に、'a' と 'c' のラベルが並ぶことになります。

各 i について、$f(i), g(i)$ が A の中で登場する位置を $fpos(i), gpos(i)$ とします。ここで、$f(i), g(i)$ がともに広義単調減少なので、$fpos(0) > fpos(1) > \cdots > fpos(N-1)$, $gpos(0) > gpos(1) > \cdots > gpos(N-1)$ となります。正確に言えば、A をソートする際にタイブレークを適切に行うとこのようになります。

よって、あとは以下のような問題が解ければよいです。

> 各 i $(0 \leq i \leq N-1)$ について、$A_{fpos(i)}$ もしくは $A_{gpos(i)}$ のどちらか一方のみを残し、長さ $2N$ の数列 $h'_0, h'_1, \cdots, h'_{2N-1}$ を作る。$A_{fpos(i)}$ を残した i の個数が k のもの全てについて、$(h'_0 - 0) \times (h'_1 - 1) \times \cdots \times (h'_{2N-1} - (2N-1))$ の値を計算し、その総和を求める。

これは、次のような DP で解けます。

- $dp[i][j] = A$ の先頭 i 項について残すかどうかを決定していて、j 項について、$A_{fpos(i)}$ を選ぶ選択をした場合の、$(h'_0 - 0) \times (h'_1 - 1) \times \cdots \times (h'_{2N-1} - (2N-1))$ の既に決定している部分の値の総和

$A_{fpos(i)}$ を残す個数が k であることから、$A_{gpos(i)}$ を選んだ場合に、その値が h' で先頭から何番目かを決定することができます。よって、この DP の遷移は $O(1)$ で行えることとなり、DP は全体で $O(N^2)$ でできます。

すべての k について同じ DP を行うので、この問題は全体で $O(N^3)$ で解けます。

回答例

AGC036 Editorial

writer : maroonrk

July 21, 2019

A : Triangle

Let us fix (X_1, Y_1) to $(0, 0)$. Then, the area of the triangle will be $|X_2 \times Y_3 - Y_2 \times X_3|/2$. Thus, we can solve the problem if we find X_2, Y_2, X_3, Y_3 such that $X_2 \times Y_3 - Y_2 \times X_3 = S$.

Here, let us also fix (X_2, Y_2) to $(10^9, 1)$. Now we just need to find X_3, Y_3 such that $10^9 \times Y_3 - X_3 = S$, which can be found from the quotient and remainder when dividing S by 10^9. For more detail, see the sample code below:

Link to sample code

B : Do Not Duplicate

Let us consider the minimum $i > 0$ such that $X_0 = X_i$. (There may be no such i, but for now we assume there is.) When we have processed $X_0, X_1, \cdots, X_i$, s will be empty. Then, let us consider the minimum $j > i + 1$ such that $X_{i+1} = X_j$. Again, when we have processed X_j, s will be empty.

Generally, for each i ($0 \leq i \leq N \times K - 1$), when the minimum $j > i$ such that $X_j = X_i$ exists, if s is empty just before X_i is processed, s will also be empty just after X_j is processed, or, just before X_{j+1} is processed. Let us define $y(i) =$ (the minimum $j > i$ such that $X_j = X_i$, or ∞ if such j does not exist)$+ 1 - i$. Then, the problem can be rephrased as follows:

- Initially, we have an integer $a = 0$.
- Step 1: As long as $a + y(a) \leq K$, we repeatedly assign $a := a + y(a)$.
- Step 2: Then, for each $i = a, a + 1, \cdots, N \times K - 1$, perform the operation in the statement.

We can easily verify that, when Step 1 is done, $N \times (K - 1) \leq a$ holds, so it is easy to execute Step 2.

Now, let us think of a way to run Step 1 fast. We can see that $y(i) = y(i \mod N)$ from the properties of X. Let us use the binary lifting method. For each pair of $k(0 \leq k \leq \log_2(N \times K))$ and $i(0 \leq i \leq N-1)$, let us define $g_k(i) =$ (The value of i when the operation of assigning $i := g(i) + i$ is performed 2^k times)$-i$, which will allow us to know the value of a when Step 1 is finished.

Step 1 takes $\mathrm{O}(N \log(NK))$ time to execute, and Step 2 takes $\mathrm{O}(N \log N))$ time, so we have solved the problem in a total of $\mathrm{O}(N \log(NK))$ time.

Link to sample code

C : GP 2

A sequence of non-negative integers $p_0, p_1, \cdots, p_{N-1}$ can occur as the result of M operations if and only if all the following conditions are satisfied:

- $p_0 + p_1 + \cdots + p_{N-1} = 3M$
- $\max(p_0, p_1, \cdots, p_{N-1}) \leq 2M$
- There are at most M values of i such that p_i is odd.

The necessity of these conditions is obvious, and we can show the sufficiency by induction. When $M = 1$, the conditions are obviously sufficient. Now, let us assume that the conditions are sufficient when $M = k$, and there is a sequence p satisfying the condition for the case $M = k + 1$. Then, if we choose two indices i, j properly and replace p_i with $p_i - 2$ and p_j with $p_j - 1$, we can obtain a sequence p satisfying the condition for the case $M = k$, which can be proved by cases. We will omit the detail, but we can prove it by paying attention to $\max(p_0, p_1, \cdots, p_{N-1})$ and the number of i such that p_i is odd.

Let us count the number of p satisfying the three conditions. For now, let us ignore the second condition and count it. Assume that there are a values of i such that p_i is odd. There are ${}_NC_a$ choices for such i. Then, we need to count the number of sequences of N even numbers totaling to $3M - a$, which is equal to the number of sequences of N integers totaling to $(3M - a)/2$. There are ${}_{(3M-a)/2+N-1}C_{N-1}$ such sequences. We can compute each binomial coefficient in O(1) time with proper pre-calculation. The pre-calculation takes $O(N + M)$ time, and there are $O(min(N, M))$ values of a, so we can find the answer in a total of $O(N + M)$ time.

We have solved the problem without the second condition, so what remains is to count the number of p violating the second condition. p totals to $3M$, so there is at most one value of i such that $p_i > 2M$. If we find the number of p such that $p_0 > 2M$, we can multiply it by N and find the answer. Thus, we want to solve the following problem:

> Count the number of sequences of N non-negative integers, q, that satisfy all of the following conditions:
>
> - $q_0 + q_1 + \cdots + q_{N-1} = M$
> - There are at most M values of i such that q_i is odd.
> - $q_0 > 0$

If we ignore the third condition here, the problem will be the same as the one we solved above and we can immediately solve it. Now, let us count the number of q violating the third condition. This is also equivalent to the problem above after replacing N with $N - 1$.

Therefore, the problem can be solved in a total of $O(N + M)$ time.

Link to sample code

D : Negative Cycle

Let us consider the situation where we have deleted some edges and have no negative cycle. Now, we can assign an integer p_i to each vertex so that the following condition is satisfied:

- For each edge $e = (i \to j)$, $p_j \leq p_i + weight(e)$ holds.

This can be achieved by, for example, letting $p_i =$ (the shortest distance from Vertex 0 to Vertex i). On the other hand, we can see that, if we can assign p_i to the vertices so that this condition is satisfied, the graph contains no negative cycle. (If we assume there is a negative cycle, adding $p_j - p_i \leq +weight(e)$ along it would lead to contradiction.)

Let us confirm what conditions p_i must satisfy. First, for each i $(0 \leq i \leq N-2)$, $p_i \geq p_{i+1}$ must hold, which corresponds to the originally present edge $(i \to i+1)$ of weight 0. Here, let us define $q_i = p_i - p_{i+1}$, which is a non-negative integer since $p_i \geq p_{i+1}$.

Then, considering the edge $(i \to j)$ of weight 1, $p_j \leq p_i + 1$ must hold, which can be represented as $q_j + q_{j+1} + \cdots + q_{i-1} \leq 1$ with q.

Similarly, considering the edge $(i \to j)$ of weight -1, $q_i + q_{i+1} + \cdots + q_{j-1} \geq 1$ must hold.

Let us consider the problem of whether we can set the values of q so that the condition is satisfied. Then, we can see that we only need to consider 0 and 1 as the elements of q.

Thus, what we have to do is to solve the following problem:

> We will set up a sequence q consisting of 0 and 1, and delete the edges whose conditions are not satisfied for this q. Minimize the total cost of deleting these edges.

We can solve it with the DP as follows:

- $dp[i][j]$ = the minimum total cost of the edges that are confirmed to be deleted when we have chosen the values from q_0 through q_i and the positions of the last 1 and the second last 1 are i and j, respectively.

Let us consider the transition $dp[i][j] \to dp[x][i]$. The additional edges of weight 1 we additionally need to delete are the edges $b \to a$ such that $j < a \leq i,\ x < b$, and the additional edges of weight -1 we additionally need to delete are the edges $a \to b$ such that $i < a < b \leq x$. With prefix sums, this transition can be calculated faster in $O(N^3)$ time, which is fast enough.

Link to sample code

E : ABC string

If S has adjacent equal characters, we can treat them as just one character. Thus, below we assume that any two adjacent characters in S are different.

Let a, b, c be the number of occurrences of `A`, `B`, `C` in S, respectively. Without loss of generality, we assume $a \leq b, a \leq c$.

Now, assuming that the set of `A`s to use is already fixed, let us determine if we can choose `B`s and `C`s to delete so that we can obtain a subsequence satisfying the condition.

First, let us delete all the unused `A`s. After this, the string may have adjacent equal characters, but we will combine each series as one character. Let t be the string obtained this way, and p, q, r be the number of occurrences of `A`, `B`, `C` in t, respectively. Each deletion of `A` reduces the number of `B`s or `C`s by at most one, so $p \leq q$ and $p \leq r$ hold.

If $q = r$ at this point, we can obtain a subsequence satisfying the condition by properly choosing `B`s and `C`s to delete. Specifically, we can repeatedly delete a substring `BC` or `CB` from the string as long as $p < q$ without making a pair of adjacent `A`s. This can be seen from the fact that: Consider the situation where the deletion of `BC` or `CB` is no longer possible. That is, the situation where any substring sandwiched between two `A`s has a length of at most 2, and there is at most one `B` or `C` at the beginning and the end of the string. In this situation, $2p \geq q + r$ already holds.

Let us consider the case $q \neq r$. We assume $q < r$ without loss of generality. t is divided into $p + 1$ substrings (some of them may be empty) by `A`s in it. Let us call such a substring of length 1 sandwiched between two `A`s (that is, not at the beginning or the end of t) a *unit string*. Among those $p+1$ substrings, let x be the number of strings containing `B`, and y be the number of unit strings consisting of `C`. If $x < y$, the number of `B`s and `C`s will never be equal after deleting `B`s and `C`s any number of times. On the other hand, if $x \geq y$, we can make the number of `B`s and `C`s equal by properly choosing the `C`s to delete. This can be seen from the fact that, for a string containing 'B' we can make the number of `B`s greater than the number of `C`s by 1, and for a unit string consisting of `C` we can only make the number of `C`s greater than the number of `B`s by 1.

Now, let us get back to the original problem. From the observations above, we can see that the problem is to choose which `A`s to use so that the substrings separated by `A`s satisfy the following conditions:

- the number of unit strings consisting of `B` is at most the number of strings containing `C`s.
- the number of unit strings consisting of `C` is at most the number of strings containing `B`s.

First, assume that we use all the `A`s, and define $b1, b2, c1, c2$ as follows:

- $b1$ = the number of strings containing `B`s
- $b2$ = the number of unit strings consisting of `B`
- $c1$ = the number of strings containing `C`s
- $c2$ = the number of unit strings consisting of `C`

If $b2 \leq c1$ and $c2 \leq b1$, we can go with all the `A`s used. Below, we assume $b2 > c1$ without loss of

generality. If we decide not to use a certain `A`, the two substrings around it will be concatenated into one string, reduing each of $b1, b2, c1, c2$ by at most 1. Also, when we delete an `A` adjacent to a unit string consisting of `B`, $b2$ decreases by 1 and $c1$ does not change. Thus, we have to delete an `A` at least $b2 - c1$ times, and after deleting an `A` as above $b2 - c1$ times, we have $b2 = c1$, from which $c2 \leq b1$ obviously holds. Therefore, we can satisfy the conditions by deleting an `A` $b2 - c1$ times, and this generates one longest subsequence that satisfies the condition.

By directly implementing these methods, we can solve the whole problem in $\mathrm{O}(N)$ time.

Link to sample code

F : Square Constraints

Let us use the inclusion-exclusion principle. For each k ($0 \leq k \leq N$), we want to solve the following problem:

> We will choose k indices $0 \leq x_0 < x_1 < \cdots < x_{k-1} \leq N-1$. For each possible choice, count the number of permutations satisfying $x_i^2 + P_{x_i}^2 < N^2$ for all $0 \leq i \leq k-1$ and $i^2 + P_i^2 \leq (2N)^2$ for all $0 \leq i \leq 2N-1$, and find the sum of these counts for all possible choices.

For each i ($0 \leq i \leq N-1$), let us define $f(i) =$ (the maximum a ($a \leq 2N-1$) such that $a^2+i^2 < N^2$)$+1$. Also, for each i ($0 \leq i \leq 2N-1$), let us define $g(i) =$ (the maximum a ($a \leq 2N-1$) such that $a^2+i^2 < (2N)^2$) $+1$.

This problem would be easily solved if the choice of x, the k indices, is fixed. For each i, assume that the condition can be written as $P_i < h(i) = f(i)$ or $g(i)$. Let $h'_0, h'_1, \cdots, h'_{2N-1}$ be the sorted list of $h(i)$ in ascending order. Then, the number of permutations satisfying the condition is $(h'_0 - 0) \times (h'_1 - 1) \times \cdots \times (h'_{2N-1} - (2N-1))$.

Now, let us consider the problem when x is not fixed. First, let $A_0, A_1, \cdots, A_{3N-1}$ be the sorted list of $f(i)$ ($0 \leq i \leq N-1$) and $g(i)$ ($0 \leq i \leq 2N-1$) all together in ascending order. We assign a label to each element of A as follows: `a` if it derives from $f(i)$ ($0 \leq i \leq N-1$), `b` if it derives from $g(i)$ ($0 \leq i \leq N-1$), and `c` if it derives from $g(i)$ ($N \leq i \leq 2N-1$). As a whole, the labels assigned to the elements will look as this: `aaccacac...bbbb...` (this is just an illustration). More formally, all `a`s and `c`s will come before all `b`s. For each i, let $fpos(i), gpos(i)$ be the positions of $f(i), g(i)$ in A, respectively. Since $f(i), g(i)$ are both non-increasing, we have $fpos(0) > fpos(1) > \cdots > fpos(N-1)$, $gpos(0) > gpos(1) > \cdots > gpos(N-1)$ (more strictly, these will hold if we properly do tie-breaks).

What remains is to solve the following problem:

> For each i ($0 \leq i \leq N-1$), we keep either $A_{fpos(i)}$ or $A_{gpos(i)}$ to make a sequence $h'_0, h'_1, \cdots, h'_{2N-1}$ of length $2N$. For each sequence that can be obtained this way when we choose to keep $A_{fpos(i)}$ for k of the indices i, find the value $(h'_0 - 0) \times (h'_1 - 1) \times \cdots \times (h'_{2N-1} - (2N-1))$, and compute the sum of these values for all such sequences.

We can solve it with the DP as follows:

- $dp[i][j]$ = the sum of the already fixed parts in $(h'_0 - 0) \times (h'_1 - 1) \times \cdots \times (h'_{2N-1} - (2N-1))$ for all possible cases when we have decided whether we keep each of the first i elements in A and we have decided to keep $A_{fpos(i)}$ for j indices.

Since we will keep k of the elements $A_{fpos(i)}$, when we choose to keep $A_{gpos(i)}$, we can determine its position in h'. Thus, we can calculate each transition in this DP in $O(1)$ time, for a total of $O(N^2)$ time.

The whole problem can be solved in $O(N^3)$ time by performing this DP for each value of k.

Link to sample code

AGC 035 解説

camypaper, chokudai, DEGwer

2019 年 7 月 14 日

For International Readers: English editorial starts on page 7.

A: XOR Circle

両隣のラクダが被っている帽子に書かれた数のビットごとの排他的論理和が自身の被っている帽子に書かれた数と等しいというのは、連続する 3 つの帽子に書かれた数のビットごとの排他的論理和が 0 ということと同値です。ここから円環上で連続する 4 つの数 $x_i, x_{i+1}, x_{i+2}, x_{i+3}$ に着目すると、$x_i \oplus x_{i+1} \oplus x_{i+2} = x_{i+1} \oplus x_{i+2} \oplus x_{i+3} = 0$ より、$x_i = x_{i+3}$ が成立することがわかります。

以上の事実から整理して考えると、条件を満たす被せ方が存在するのは以下の場合に限られます。これは $O(N \log N)$ で判定可能です。

- 全ての帽子に書かれた数が 0
- x が書かれた帽子が $\frac{2N}{3}$ 枚、0 が書かれた数が $\frac{N}{3}$ 枚
- $x \oplus y \oplus z = 0$ が成立するような 3 つの相異なる整数 x, y, z が書かれた帽子がそれぞれ $\frac{N}{3}$ 枚

B: Even Degrees

全ての辺はいずれかの頂点から出るように向き付けられるので、辺の数が奇数なら、全頂点の出次数を偶数にすることは不可能です。

逆に、辺の数が偶数ならそのような向き付けを以下のように構成することができます。まず、適当な根付き全域木を取り、残りの辺を適当に向き付けます。全域木の辺をボトムアップに決めていきます。頂点 v とその親を結ぶ辺の向きを決める際には、v の出次数が偶数になるようにします。このようにすれば、根以外の頂点の出次数は構成のルールより、根の出次数は全体の出次数の合計値が偶数であることより、それぞれ偶数であることが保証されます。

C: Skolem XOR Tree

頂点の番号ではなく頂点に割り当てられた整数のみを考えることにします。

N が 2 の冪の場合、明らかに答えは `No` となります。

それ以外の場合、答えは `Yes` です。例えば N が 3 以上の奇数の場合は以下の図 **??** のようにして構成可能です。

N が偶数の場合もほぼ同様です。先程の構成方法において、1 が書かれた紫色の頂点は $2, 3, \ldots, N-1$ が書かれた頂点と隣接することを利用します。例えば $N = 2018$ の場合、995 と書かれた頂点と 1024 と書かれた頂点につなぐことで $995 \oplus 1 \oplus 1024 = 2018$ と条件を満たすように構成できます。

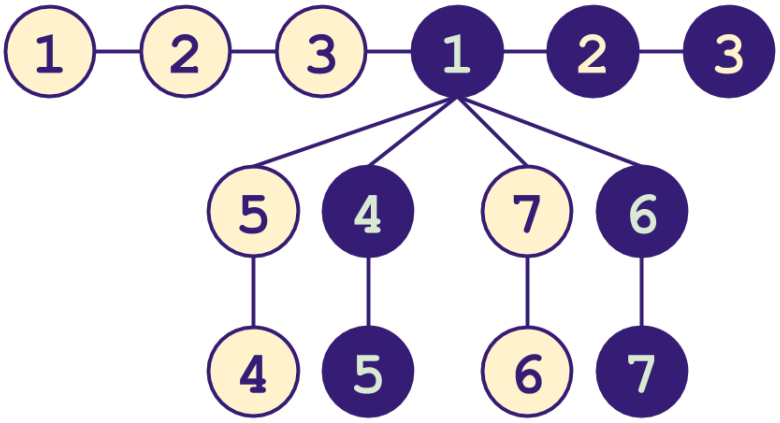


図 1　N=7 の場合

D: Add and Remove

操作を逆順に見ていきましょう。以下、カードを食べる操作の逆操作を「吐き出す」と呼ぶことにします。最初 2 枚のカードがあり、これらに書かれた整数は目的関数値に 1 倍されて足されます。

さて、現在 k 枚のカードがあるとして、これらに書かれた整数は順に目的関数値に $x_1, \ldots, x_k$ 倍され足されるとします。i 番目のカードと $i+1$ 番目のカードの間に吐き出され山に加えられるカードに書かれた整数は $x_i + x_{i+1}$ 倍されて足されることがわかります。こうして、山のどの位置にカードを吐き出して戻したかの情報から、最終的な列の何番目のカードに書かれた整数が何倍されて目的関数値に足されるかを復元することができます。

さて、$\mathtt{DP}[l][x_l][r][x_r]$ を、入力で与えられたカード列の l, r 番目のカードが隣接していたタイミングが存在し、順に x_l, x_r 倍されて足されるときの、l 番目から r 番目までのカードに書かれていた整数が目的関数値に足される量の合計の最小値とします。この DP は、$\mathtt{DP}[l][x_l][r][x_r] = \min_{l<m<r} \mathtt{DP}[l][x_l][m][x_l + x_r] + \mathtt{DP}[m][x_l + x_r][r][x_r] + (x_l + x_r)A_m$ として更新していくことができます。

この DP の状態数を見積もりましょう。組 (l, r) の個数は高々 $O(N^2)$ 個です。組 (x_l, x_r) としてありうるものの個数は、以下のように抑えることができます。

l, r 番目のカードの両方が現れるまでカードを吐き出していく操作を考えます。x_l, x_r の値に影響を与えるのは、$l' \leq l < r \leq r'$ を満たすような l', r' 番目のカードの間にカードが吐き出されるような場合だけです。この操作で吐き出されたカードは $l' < t \leq l$ または $r \leq t < r'$ を満たす t 番目のカードであり、x_l, x_r の値に関係するのは l 以下または r 以上のどちらの t が選ばれたかのみです。よって、カードを 1 枚吐き出す操作によって x_l, x_r の値の組の候補は高々 2 倍にしかならず、DP の状態数が全体として $O(2^n poly(n))$ で抑えられることがわかるので、適切な実装によりこの問題を解くことができます。

E: Develop

黒板に現れない整数の集合 S を特徴づけましょう。S の要素を頂点とし、$x, x-2$ の両方が S に含まれるなら x から $x-2$ に、また $x, x+K$ の両方が S に含まれるなら x から $x+K$ にそれぞれ辺を張ったグラフを考えます。S は、こうして作ったグラフにサイクルがないとき、またその時に限り、黒板に現れない整数の集合としてありうるものとなります。これは以下のようにして証明できます。グラフにサイクルが含まれるなら、操作の定義より、サイクルを構成する要素のうちどれかひとつは黒板に残ります。逆に、グラフにサイクルが含まれないなら、そのトポロジカル順に黒板から整数を消していくことができます。

さて、適切な考察により、S から構成されるグラフにサイクルが含まれないことは、S に $a, a-2, ..., b-K, b, b-2, ..., a-K, a$ という形のサイクルが含まれないことと同値であることが証明できます。以下、この特別な形のサイクルを含まないような S を数え上げることを考えます。

整数を小さい順に S に追加していく DP を考えます。整数 a を S に追加できない状況は、ある $b < a, a \neq b \pmod 2$ が存在し、$a-2, a-4, ..., b-K, b, b-2, ..., a-K$ が S に含まれている状況のみです。このとき、a と偶奇の等しく S に含まれない最後の整数を x、a と偶奇の異なり S に含まれない最後の整数を y とすれば、$x < y$ が成り立ちます (そうでない場合、すでにサイクルが存在します)。

$x < y, a = t = x \neq y \pmod 2, a \leq t$ に対し $\mathtt{DP}[a][x][y][t]$ を、$a-1$ までの整数を S に入れるかどうかを決定し、x, y が上記のように設定され、a と偶奇の等しい整数を連続して t まで選ぶことができるような場合の数とします。$x > y$ に対しても $\mathtt{DP}[a][x][y][t]$ を同様に設定することで、DP を更新していくことができます。時間計算量は $O(N^4)$ ですが、$1/100$ オーダーの定数がつくので高速に動作します。

F: Two Histograms

$k_i+1=j, l_j=i$ なる (i,j) の組が存在すれば、(k_i,l_j) を (k_i+1,l_j-1) で置き換えることにより同じマス目を得ることができます。この操作によって k_i たちの合計は増加するので、できるマス目を変えることなしに、有限回の操作によってそのような組が存在しないように出来ます。こうしてできる $k_1,...,k_N,l_1,...,l_M$ の組を**標準表現**と呼ぶことにしましょう。最初の $k_1,...,k_N,l_1,...,l_M$ によって標準表現は一意に定まることが証明できます。

逆に、標準表現が異なるならばできるマス目も異なることが以下のように証明できます。

異なる標準表現を 2 つ取ってきて $k_1,...,k_N,l_1,...,l_M$ と $k'_1,...,k'_N,l'_1,...,l'_M$ としましょう。これらからできるマス目 A が等しいとして矛盾を導きます。$l_j \neq l'_j$ なる最小の j をとります。$l_j < l'_j$ として一般性を失いません。$j=1$ のとき、$k'_{l'_1}+1=1$ となり矛盾します。$j>1$ のとき、$A_{l'_j j}=1$ より $k_{l'_j} \geq j$ かつ $k'_{l'_j} < j$ ですが、これは $k'_{l'_j} \neq j-1$ と $l_{j-1}=l'_j$ に矛盾します。

さて、あとは標準表現の個数を数えればよいですが、これは $k_i+1=j, l_j=i$ なる (i,j) の組の集合に関する包除原理で線形時間で求めることができ、この問題を解くことができました。

AGC 035 Editorial

camypaper, chokudai, DEGwer

July 14, 2019

A: XOR Circle

The condition "the bitwise XOR of the numbers written on the hats on both adjacent camels is equal to the number on the hat on itself" is equivalent to the following: "the bitwise XOR of the numbers written on the three consecutive hats is 0." Now, let us focus on four consecutive numbers in the circle x_i, x_{i+1}, x_{i+2}, and x_{i+3}. From $x_i \oplus x_{i+1} \oplus x_{i+2} = x_{i+1} \oplus x_{i+2} \oplus x_{i+3} = 0$, it follows that $x_i = x_{i+3}$.

Further analysis based on these facts reveals that the condition can be satisfied only in the following cases, which can be determined in $\mathrm{O}(N \log N)$ time:

- Every hat has 0 written on it.
- There are $\frac{2N}{3}$ hats with x, and $\frac{N}{3}$ hats with 0.
- There are $\frac{N}{3}$ hats with x, $\frac{N}{3}$ hats with y, and $\frac{N}{3}$ hats with z, where x, y and z are distinct integers such that $x \oplus y \oplus z = 0$.

B: Even Degrees

Since every edge will be directed so that it goes out from one of the vertices, it is impossible for every vertex to have an even degree if there are an odd number of edges.

On the other hand, if there are an even number of edges, we can direct the edges so that every vertex has an even degree, as follows. First, we take some rooted spanning tree of the graph, and direct the remaining edges arbitrarily. Now, we will decide the direction for the edges in the spanning tree from bottom to top. When deciding the direction for the edge connecting a vertex v and its parent, we will make sure that the out-degree of v becomes even. This guarantees that every vertex other than the root will have an even degree, which in turn guarantees that the root will have an even degree, since the sum of the out-degrees of all the vertices is even.

C: Skolem XOR Tree

Below, we will refer to the vertices as their weights.

When N is a power of 2, the answer is obviously `No`.

In the other cases, it turns out that the answer is `Yes`.

When N is odd and not less than 3, we can, for example, construct a solution in the following manner:

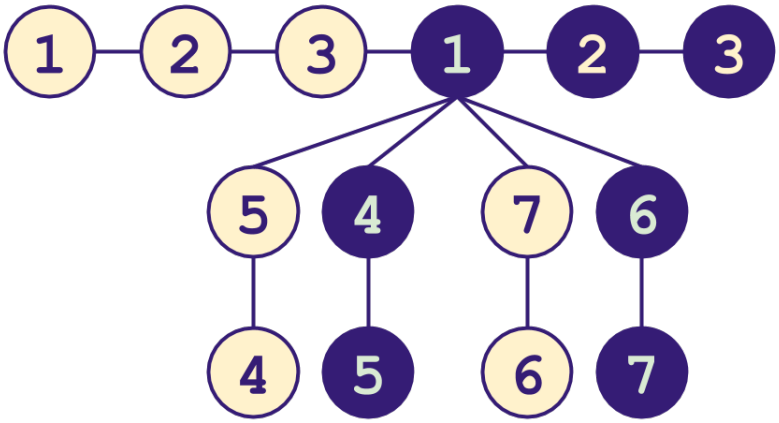


Fig.1 A solution for N=7

We can slightly modify this to handle the case where N is even. We only need to add two vertices with weight N. Let us take advantage of the fact that, in the graph above, the violet vertex with weight 1 is adjacent to the vertices with weight $2, 3, \ldots, N-1$. We can choose two of these vertices and connect them to the vertices with weight N. For example, if $N = 2018$, connect the vertices with weight N to the vertices with weight 995 and 1024 and we have $995 \oplus 1 \oplus 1024 = 2018$.

D: Add and Remove

Let us observe the operations in the reverse order. We say we *vomit* a card when we perform the reverse operation of eating a card. We have initially two cards, whose numbers will be added to the objective function after multiplied by 1.

Assume that now there are k cards remaining, whose numbers will be added to the objective function after multiplied by $x_1, \ldots, x_k$, respectively. We can see that, if we vomit a card between the i-th and $(i+1)$-th cards from the left, the number on it will be added to the objective function after multiplied by $x_i + x_{i+1}$. In this manner, from the information of the positions where we vomit the cards, we can restore the coefficients multiplied to the number written on each card in the objective function.

Let $\texttt{DP}[l][x_l][r][x_r]$ be the minimum possible sum of the contributions of the l-th to r-th cards in the input sequence to the objective function if there is a moment where these two cards are adjacent and they are multiplied by x_l and x_r before added to the objective function. We can update the table as follows: $\texttt{DP}[l][x_l][r][x_r] = \min_{l<m<r} \texttt{DP}[l][x_l][m][x_l + x_r] + \texttt{DP}[m][x_l + x_r][r][x_r] + (x_l + x_r)A_m$.

Let us estimate the number of states in this DP. The number of pairs (l, r) is at most $\mathrm{O}(N^2)$. Now we will estimate the upper bound of the number of pairs (x_l, x_r).

Consider vomiting cards until both the l-th and r-th cards appear. The values of x_l and x_r will be affected only when we vomit a card between the l'-th and r'-th cards such that $l' \leq l < r \leq r'$. Let such a card vomited be the t-th card. Then, $l' < t \leq l$ or $r \leq t < r'$ holds, and which of these two holds is the only factor that matters to the values of x_l and x_r. Thus, the number of possible pairs of the values of x_l and x_r is at most doubled when a card is vomited, so there are at most $\mathrm{O}(2^n poly(n))$ states in this DP, which can solve the problem if properly implemented.

E: Develop

Let us characterize the set S of integers that are not present on the blackboard. Consider a graph where the elements of S are the vertices and an edge is drawn from vertex x to $x-2$ when both x and $x-2$ are contained in S, and an edge is drawn from vertex x to $x+K$ when both x and $x+K$ are contained in S. S can be a set of the integers not present on the blackboard if and only if this graph does not contain a cycle. We can prove it as follows. If the graph contains a cycle, from the definition of the operation, it follows that one of the elements that is a part of the cycle will remain on the blackboard. On the other hand, if the graph does not contain a cycle, we can erase the integers from the blackboard in the topological order of the elements.

Then, we can also prove that the graph generated by S does not contain a cycle if and only if S does not contain a cycle of the form $a, a-2, ..., b-K, b, b-2, ..., a-K, a$, by appropriate consideration. Now, let us count the sets S which does not contain a cycle of this form.

Let us consider an approach by DP where we add integers to S in ascending order. The only situation where we cannot add an integer a to S is the situation where there exists an integer b such that $b < a, a \neq b (\mod 2)$ and $a-2, a-4, ..., b-K, b, b-2, ..., a-K$ are contained in S. Let x be the last integer with the same parity as a and not contained in S, and y be the last integer with the parity different from a and not contained in S. Then, $x < y$ holds (or there is already a cycle).

For integers x, y, a, t such that $x < y, a = t = x \neq y (\mod 2), a \leq t$, let $\texttt{DP}[a][x][y][t]$ be the number of choices for the integers up to $a-1$ whether they are contained in S such that the values of x and y are defined as above and we can choose at most t consecutive integers with the same parity as a. We will also define $\texttt{DP}[a][x][y][t]$ for the case $x > y$ similarly, and now we can update the DP tables. The time complexity of this approach is $\mathrm{O}(N^4)$, with a small constant factor that enables it to work fast enough.

F: Two Histograms

If there exists a pair (i, j) such that $k_i + 1 = j, l_j = i$, we can replace (k_i, l_j) with $(k_i + 1, l_j - 1)$ and still obtain the same grid. This operation increases the sum of k_i, so we can repeat this operation a finite number of times so that there will be no such pair, without changing the resulting grid. Let us call such a sequence obtained, $k_1, ..., k_N, l_1, ..., l_M$, the **regular expression**. We can prove that the regular expression for a fixed initial sequence $k_1, ..., k_N, l_1, ..., l_M$ is unique.

On the other hand, if two initial sequences have different regular expressions, the resulting grid will also be different, which we can prove as follows.

Let the two different regular expressions be $k_1, ..., k_N, l_1, ..., l_M$ and $k'_1, ..., k'_N, l'_1, ..., l'_M$. Assume that we obtain the same grid A from both of them and reach a contradiction. Let j be the minimum index such that $l_j \neq l'_j$. We assume $l_j < l'_j$ without loss of generosity. If $j = 1$, $k'_{l'_1} + 1 = 1$ and a contradiction is reached. If $j > 1$, from $A_{l'_j j} = 1$ it follows that $k_{l'_j} \geq j$ and $k'_{l'_j} < j$, which contradicts with $k'_{l'_j} \neq j - 1$ and $l_{j-1} = l'_j$.

Now, we just have to count the number of regular expressions. We can use the inclusion-exclusion principle on the number of pairs (i, j) such that $k_i + 1 = j, l_j = i$ and count them in linear time, so the problem is solved.

AGC 034 解説

yosupo, sigma425

2019 年 6 月 2 日

For International Readers: English editorial starts on page 8.

A: Kenken Race

まず、すぬけ君とふぬけ君は、少なくとも他方がいない場合に目的のマスへ移動できる必要があります。これは途中に 2 マス連続で岩があると不可能です。

- $C < D$ の場合はこれだけでよいです。実際に、まずはふぬけ君を目的地に動かし、次にすぬけ君を動かせば目的が達成できます。
- $C > D$ の場合は、すぬけ君がふぬけ君をどこかで追い抜く必要があります。追い抜くためには 3 マス連続の空きが必要です。これが存在するか、つまり B から D のどこかに、そのマスを中心として 3 連続の空きがあるかを判定すればいいです。

サンプルコード (C++): https://atcoder.jp/contests/agc034/submissions/5747486

B: ABC

まず、文字列中にある 'BC' は、操作によって分断されたり、また新しく増えたりしないことがわかります。そこではじめに文字列 s 中の 'BC' をすべて 'D' に置き換えてみます。すると、可能な操作は 'AD' を 'DA' にすることと同じです。あまった 'B' や 'C' はそこを超えて操作できないというブロックになるので、'A','D' からなる連続する極大な文字列全てに対して操作回数を求め、その和を計算すれば答えが得られます。これは転倒数なので、これまでに出てきた 'A' の個数 などを持ちながら左から文字列を舐めることで線形で計算可能です。

C: Tests

高橋くんがテスト i で取る点数を a_i とします (変数です)。

$D := A - B$ とします。 $D \geq 0$ にするのが目標です。

また、テスト i の D への寄与 (すなわち、 $c_i \times (a_i - b_i)$) を D_i とおきます。

テスト i に対して a_i を固定したときに、重要度 c_i をどう設定すればいいかは簡単に判定できます: $a_i \leq b_i$ なら重要度を l_i に、$a_i > b_i$ なら重要度を u_i にすればよいです。このことから、D_i を a_i の関数としてみると、以下の形になっていることがわかります。

$$D_i(a_i) = \begin{cases} -l_i \times (b_i - a_i) & (a_i \leq b_i) \\ u_i \times (a_i - b_i) & (a_i > b_i) \end{cases}$$

a_i を 0 から 1 ずつ増やしていくことを考えると、問題は次のように言い換えられます。

> はじめ、$D = \sum_{i=1}^{N} D_i(0)$ である。長さ X の整数列が N 個与えられる。i 個目の数列は、はじめの b_i 項が l_i で、残りの $X - b_i$ 項が u_i である。あなたは数列の要素をいくつか選んで D に足していき、$D \geq 0$ となるようにしたい。ただし各数列内では要素を前から順番に選んでいく必要がある。最小で何個取れば条件を達成できますか。

これは次のようにして解くことができます。まず答えで二分探索をし、 k 個取ることで総和を最大化する問題を考えます。実はこのとき、次の条件を満たす最適解が存在することがわかります:

- 1 個以上 $X - 1$ 個以下の要素を選ぶ数列が高々 1 つ

これは、中途半端に選んでいる 2 つの数列があったとすると片方を 1 つ増やし、もう片方を 1 つ減らす という操作を限界まで続けることで悪くはならないようにできること (これには $l_i \leq u_i$ が効いています) からわかります。

なので、$q := \lfloor k/X \rfloor$, $r := k - qX$ とおくと、q 個の数列から要素をすべて (X 個) 選び、($r > 0$ なら) 1 個の数列から要素を r 個 選ぶことになります。r 個選ぶ数列 i を決め打つと、選ぶべき残り q 個の数列は、数列 i を除いたもののうち要素の和が大きい q 個なので、まず事前に (判定問題にする前に) 数列を要素の和の大きい順にソートしておけば、各 i ごとに $O(1)$ の足し引きをすることで求まります。よって判定問題が $O(N)$ で解けました。したがってもとの問題も解けて、全体の計算量は $O(N(logN + logX))$ です。

D: Manhattan Max Matching

おまけ： $O(S \log N)$ でもこの問題は解くことが可能です

ペアのスコアを $|rx - bx| + |ry - by|$ とする代わりに、ペアごとにスコアを

- $(rx - bx) + (ry - by) = (rx + ry) + (-bx - by)$
- $-(rx - bx) + (ry - by) = (-rx + ry) + (bx - by)$
- $(rx - bx) - (ry - by) = (rx - ry) + (-bx + by)$
- $-(rx - bx) - (ry - by) = (-rx - ry) + (bx + by)$

の 4 つから好きなものを選ぶ、としても答えは変わりません。

ボールごとに、この 4 つのうちどれを選ぶか、というのを先に決めておくことにします。

つまり、赤いボールを 4 つの以下のタイプに分類します。

- タイプ 0: スコアに $(rx + ry)$ を加算する
- タイプ 1: スコアに $(-rx + ry)$ を加算する
- タイプ 2: スコアに $(rx - ry)$ を加算する
- タイプ 3: スコアに $(-rx - ry)$ を加算する

同様に青いボールも以下のように分類します。

- タイプ 0: スコアに $(-bx - by)$ を加算する
- タイプ 1: スコアに $(bx - by)$ を加算する
- タイプ 2: スコアに $(-bx + by)$ を加算する
- タイプ 3: スコアに $(bx + by)$ を加算する

そして、各タイプごとに赤と青の個数が等しければ実際にペアを構成することができます。このような分類のうち、スコアの最大を求めれば良いです。

これは最小費用流で解けます。1(始点) + N(赤いボールの操作) + 4(タイプ) + N(青いボールの操作) + 1(終点) 頂点のグラフを作り、

- 始点から (赤いボールの操作) へ、それぞれ容量 RC_i、コスト 0
- (赤いボールの操作) からタイプへ、それぞれ容量 ∞、コストは $-$(先述のスコア)
- タイプから (青いボールの操作) へ、それぞれ容量 ∞、コストは $-$(先述のスコア)
- (青いボールの操作) から終点へ、それぞれ容量 BC_i、コスト 0

の辺を張り、S(ボールの個数) 流せば良いです。負の重みは、適切なオフセットを各辺に足すことで解決できます。

辺数が $O(N)$、流量が $O(S)$ なので、計算量は $O(SN \log N)$ です。

E: Complete Compress

コマを集める頂点を固定した時に，$O(N)$ で解く方法を解説します。これが可能ならば全ての頂点に対し固定してみることで合計で $O(N^2)$ で解け，十分高速です。以後，木を頂点 1 を根とする根付き木として考えることにし、すべてのコマを頂点 1 に集めたいものとします。また、頂点 v の深さを $\mathrm{dep}(v)$ とします。

コマの深さの和を S とすると、S が奇数だと不可能、また偶数のときは最低 $S/2$ 回は操作が必要です。

実は以下の 2 つが成立します。

- もしコマを集められるならば、必ず $S/2$ 回の操作でコマを集められる (lemma A)。
- i 回目の操作を頂点 a_i, b_i のコマに行うとするとき、$\mathrm{dep}(\mathrm{lca}(a_i, b_i))$ が広義単調減少の最短手数の手順が存在する (lemma B)。

とりあえずこれらを正しいものとすると、$\mathrm{dp}(v) =$ (自分の部分木の中で完結する操作のみを行う時，操作回数の最大値) を木 DP で計算し、$\mathrm{dp}(1) = S/2$ を判定すれば解けます。

lemma A の証明

無駄な操作 (=深さの和を変えない操作) をする必要がないことを示します。最初の 1 回のみが無駄な操作で，その後は無駄な操作を行わない操作列があった時，無駄な操作を行わない操作列へ変換出来ることを示します。

最初の無駄な操作で頂点 $u, v(\mathrm{dps}(u) < \mathrm{dps}(v))$ から u', v' にコマを動かしたとします (前者をコマ A, 後者をコマ B とします)。次にコマ A に対し操作を行うタイミング (*) で，コマ B が u' より上にいるかどうかに注目し，場合分けをします。

- 上にいない: 無駄な操作を行わず，その後 (*) の直前まで同じ操作列を適用します。そして (*) でコマ A の代わりにコマ B を動かします。すると，元の操作列を (*) まで適用した後と同じ状態になります。
- 上にいる: (*) より前のタイミングで，B が A の真上に移動するタイミングが必ず存在するはず (**) です。仮に無駄な操作を行わず，その後同じ操作列を適用すると，実は (**) のタイミングで盤面は全く同じになります。

lemma B の証明

ある i について，$\mathrm{dep}(\mathrm{lca}(a_i, b_i)) < \mathrm{dep}(\mathrm{lca}(a_{i+1}, b_{i+1}))$ であったとします。この 2 回の操作を swap する，つまり $i+1$ 回目で操作するコマたちを i 回目に操作し，i 回目で操作するコマたちを $i+1$ 回目に操作することを考えます。すると，swap しても $i+1$ 回目の終了後の盤面が変化しないこと，そして swap 後の操作列での $a_i, b_i, a_{i+1}, b_{i+1}$ を $a'_i, b'_i, a'_{i+1}, b'_{i+1}$ とすると

- $\mathrm{dep}(\mathrm{lca}(a'_i, b'_i)) = \mathrm{dep}(\mathrm{lca}(a_{i+1}, b_{i+1}))$
- $\mathrm{dep}(\mathrm{lca}(a'_{i+1}, b'_{i+1})) = \mathrm{dep}(\mathrm{lca}(a_i, b_i))$

であることが示せます。よってこの swap を繰り返し適用することで，有限回の操作で求める操作列が得られます。

F: RNG and XOR

i が生成される確率 を $a_i(=A_i/S)$ とおきます。まず 0 が i になるまでの期待値は i が 0 になるまでの期待値と同じです。これを x_i とおきます。連立方程式を解けば x_i が求まるのは明らかですが、計算量は $O(2^{3N})$ となってしまい駄目です。詳しく連立方程式の形を見てみましょう。$2^N \times 2^N$ 行列で書くと次のようになります。

$Mx = c$ ただし、

$$M_{i,j} = \begin{cases} 1 & (i = 0 \wedge j = 0) \\ 0 & (i = 0 \wedge j \neq 0) \\ a_0 - 1 & (i \neq 0 \wedge i = j) \\ a_{i \hat{} j} & (otherwise) \end{cases}$$

$$x = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_{2^N-1} \end{pmatrix}$$

$$c = \begin{pmatrix} 0 \\ -1 \\ \vdots \\ -1 \end{pmatrix}$$

うまく式を変形して高速に x を求めるのが目標です。

$a_0 = a_0 - 1$ と置き直します。$\sum_{i=0}^{2^N-1} a_i = 0$ に注意してください。$i = 1, \cdots, 2^N - 1$ に対して、式 F_i をこの連立方程式の i 行目の式とします。すなわち、$\sum_{j=0}^{2^N-1} a_{i \hat{} j} x_j = -1$ のことです。$i = 0$ に対しても同様の式が求まるでしょうか? 答えは YES です。実際、 $\sum_i a_i = 0$ より、$-(F_1 + F_2 + \cdots + F_{2^N-1})$ という式を考えると、$\sum_{j=0}^{2^N-1} a_{0 \hat{} j} x_j = 2^N - 1$ を得られます。これを F_0 としましょう。

実はこの 2^N 個の式に対してアダマール変換のようなことをすることで x が高速に求まります。$i = 1, \cdots, 2^N - 1$ に対して、式 G_i を $\sum_j (-1)^{\mathrm{popcnt(i\&j)}} F_j$ とします。$i = 0$ については自明な式が得られるので無視します。すると G_i は次のような簡潔な形になることがわかります。まず $b_i := \sum_j (-1)^{\mathrm{popcnt(i\&j)}} a_j$ とおきます。G_i は $b_i \times \sum_k (-1)^{\mathrm{popcnt(i\&k)}} x_k = 2^N$ という形になります。

右辺は $i \neq 0$ からすぐ示せます。左辺については、任意の i, k に対し G_i の x_k に関する等式 $\sum_j (-1)^{\mathrm{popcnt(i\&j)}} a_{j \hat{} k} = (-1)^{\mathrm{popcnt(i\&k)}} b_i$ を示せばよいですが、これは a_j の係数を比較すると $(-1)^{\mathrm{popcnt(i\&(j\hat{}k))}} = (-1)^{\mathrm{popcnt(i\&k)}} \times (-1)^{\mathrm{popcnt(i\&j)}}$ が各 bit ごとにみて正しいことからわかります。

よって $i = 1, \cdots, 2^N - 1$ に対して $y_i := \sum_k (-1)^{\mathrm{popcnt(i\&k)}} x_k$ が求まります。y_0 は、 $\sum_i y_i = x_0 \times 2^N$ と $x_0 = 0$ から求まります。最後にアダマール変換で y から x を復元することでこの問題は解けました。計算量は $O(N2^N)$ です。

AGC 034 Editorial

yosupo, sigma425

June 2, 2019

A: Kenken Race

First, each of Snuke and Fnuke needs at least to be able to move to the destination square if the other person does not exist. They cannot do this if there are two consecutive rock squares on the way.

- If $C < D$, that's it. We can achieve the objective by, for example, moving Fnuke to his destination first and then moving Snuke to his.
- If $C > D$, Snuke has to overtake Fnuke somewhere on the way, which requires three consecutive empty squares. Thus, we additionally need to check if there are three consecutive empty squares centered at somewhere between B and D.

B: ABC

First, we can see that the operation does not separate existing `BC`s or produce new `BC`s in the string, so let us replace each occurrence of `BC` in the string s with `D`. Then, the operation is now equivalent to change `AD` to `DA`. The remaining `B`s and `C`s are now obstacles that block the operation, so we can obtain the answer by finding the number of possible operations for each maximal contiguous substring consisting of `A` and `D` and summing them up. This count is equivalent to the inversion number, which we can compute in linear time by scanning the string from the left, maintaining the count of `A`s appeared so far, for example.

C: Tests

Let a_i be the variable representing Takahashi's score on Test i, and $D := A - B$. Our objective is to have $D \geq 0$.

Then, let D_i be the contribution of Test i in D, that is, $c_i \times (a_i - b_i)$.

When we fix a_i for Test i, we can easily determine the optimal choice of the importance c_i: we should set c_i to l_i if $a_i \leq b_i$, and set c_i to u_i if $a_i > b_i$. Thus, if we see D_i as a function of a_i, it looks as follows:

$$D_i(a_i) = \begin{cases} -l_i \times (b_i - a_i) & (a_i \leq b_i) \\ u_i \times (a_i - b_i) & (a_i > b_i) \end{cases}$$

Consider incrementing a_i by 1 at a time from 0, and we can rephrase the problem as follows:

> Initially, $D = \sum_{i=1}^{N} D_i(0)$. You are given N integer sequences, each of length X. The first b_i terms in the i-th sequence are l_i, and the remaining $X - b_i$ terms are u_i. You want to choose some terms in the sequences and add them to D so that $D \geq 0$. In each sequence, you have to choose terms in order from front to back. At least how many terms do you need to choose to achieve the objective?

We can solve it as follows. First, let us do a binary search on the answer, and consider the problem to maximize the sum by choosing k terms. Then, we can see that there exists an optimal solution that satisfies the following condition:

- There is at most one sequence in which 1 between $X - 1$ terms are chosen.

This is because, if two sequences are partially used, we can repeatedly choose one more term in one of them and choose one less term in the other as many times as possible, and the result does not get worse, because $l_i \leq u_i$.

Thus, let $q := \lfloor k/X \rfloor$, $r := k - qX$, and we will choose all the terms in q of the sequences, and choose r terms in one of the sequences if $r > 0$. If we decide to choose r terms in Sequence i, the other q sequences that should be chosen are the q sequences with the largest sums of terms excluding Sequence i, which we can find in $O(1)$ time for each i by sorting the sequences in descending order of the sum of their sums of terms in advance. Now we have solved the decision problem in $O(N)$ time, and also the original problem with the total time complexity $O(N(\log N + \log X))$.

D: Manhattan Max Matching

Bonus: This problem can also be solved in $O(S \log N)$ time.

Without affecting the score, we can assume that instead of getting the score of $|rx - bx| + |ry - by|$ for a pair, we can choose one of the following scores for a pair:

- $(rx - bx) + (ry - by) = (rx + ry) + (-bx - by)$
- $-(rx - bx) + (ry - by) = (-rx + ry) + (bx - by)$
- $(rx - bx) - (ry - by) = (rx - ry) + (-bx + by)$
- $-(rx - bx) - (ry - by) = (-rx - ry) + (bx + by)$

For each ball, let us decide in advance which of these four to use.

That is, we will classify the red balls into the following four types:

- Type 0: adds $(rx + ry)$ to the score.
- Type 1: adds $(-rx + ry)$ to the score.
- Type 2: adds $(rx - ry)$ to the score.
- Type 3: adds $(-rx - ry)$ to the score.

We will also classify the blue balls into the following four types:

- Type 0: adds $(-bx - by)$ to the score.
- Type 1: adds $(bx - by)$ to the score.
- Type 2: adds $(-bx + by)$ to the score.
- Type 3: adds $(bx + by)$ to the score.

Then, we can form the pairs if the number of red balls and that of blue balls are equal for each type. We want to find the maximum score of such a classification.

We can solve it as a minimum-cost flow problem. Let us build a graph with 1 (source) $+N$ (operations with red balls) +4 (types) $+N$ (operations with red balls) +1 (sink) vertices and the following edges:

- from the source to each "operations with red balls" vertex: an edge of capacity RC_i and cost 0
- from each "operations with red balls" vertex to each "type" vertex: an edge of capacity ∞ and cost $-$(the score above)
- from each "type" vertex to each "operations with blue balls" vertex: an edge of capacity ∞ and cost $-$(the score above)
- from each "operations with blue balls" vertex to the sink: an edge of capacity BC_i and cost 0

Then send the flow of S (the number of balls). We can resolve negative costs by adding some offset to each edge.

There are $O(N)$ edges, and the amount of flow is $O(S)$, so the time complexity is $O(SN \log N)$.

E: Complete Compress

Let's fix certain vertex as the root, and check if we can move all tokens to the root. We want to do this in $O(N)$ (then the entire solution will be $O(N^2)$).

Suppose that there is a sequence of operations that moves all tokens to the root. Then, we can prove that we can do that even with the following additional restrictions:

> Lemma A. Let's call an operation "bad" if this operation moves one of the tokens downward. (In other words, one of the two chosen tokens is the descendant of the other.) There is a valid sequence of operations without any bad moves.

Proof. Let's take a sequence of operations with at least one bad move. We show that we can alwats reduce the number of operations after the last bad move; then, by repeating this process, we can prove the lemma.

Suppose that in the last bad move, we choose token A at vertex u and token B at vertex v (and u is an ancestor of v). There are several cases depending on the next move.

- If the next move doesn't involve A and B, obviously, we can swap those two moves.
- If the next move is between A and C for some C, we can replace $(A, B) \to (A, C)$ with (B, C).
- If the next move is between B and C for some C, we can swao the two moves unless the distance between u and v is two; if the distance is two, we can simply drop the move between A and B and the multiset of positions of the tokens won't change.

> Lemma B. Let's call an operation "special" if the LCA of two chosen tokens is the root. There is a valid sequence of operations that starts with zero or more non-special moves, followed by zero or more special moves.

Proof. This is easier to prove; if there is a non-special move directly after a special move, we can always swap those two moves.

Now, let's return to the original problem. For each vertex x (in the order from leaves to the root), we compute the following values when we consider the subtree rooted at x:

- The number of tokens in the subtree.
- $high$: the sum of distances from each token to the root (i.e., x) in the initial configuration.
- low: the minimum possible value of the sum of distances from each token to the root, when we are allowed to perform arbitrary operations within the subtree.

Then, the set of possible values of "the sum of distances" is $low, low+2, ..., high$. Suppose that $y_1, ..., y_k$ are children of x. To compute low for x, we do the following:

- First, for each y_i, we choose the value s_i: it should be a value between low and $high$ of y_i, with the correct parity.

- Replace s_i by $s_i + cnt_{y_i}$.
- $low = max(\sum s_i mod 2, \max s_i * 2 - \sum s_i)$. (This corresponds to moves whose LCA are x. We should choose s_i that minimizes this value.)

The answer is "Yes" if the value low for the root is zero.

F: RNG and XOR

Let $p_i(= A_i/S)$ be the probability that i is generated. Let x_i be the i-th answer. This is the same as the expected number of moves to reach zero when we start from i, so we get the following for each nonzero i:

$$x_i = \sum_j p_j x_{i \hat{} j} + 1 \tag{1}$$

Let $\bigoplus$ denote XOR convolution. Then, these equations can be written as follows:

$$(x_0, x_1, \ldots, x_{2^N-1}) \bigoplus (p_0, p_1, \ldots, p_{2^N-1}) = (?, x_1 - 1, \ldots, x_{2^N-1} - 1) \tag{2}$$

Here, in the XOR convolution, the product of the i-th term of the first sequence and the j-th term of the second sequence is added to the $i \hat{} j$-th term of the third sequence (all zero-based).

Since $\sum p_i = 1$, the sum of the first sequence and the sum of the third sequence must be equal. Thus, we get:

$$(x_0, x_1, \ldots, x_{2^N-1}) \bigoplus (p_0, p_1, \ldots, p_{2^N-1}) = (x_0 + 2^N - 1, x_1 - 1, \ldots, x_{2^N-1} - 1) \tag{3}$$

and thus

$$(x_0, x_1, \ldots, x_{2^N-1}) \bigoplus (p_0 - 1, p_1, \ldots, p_{2^N-1}) = (2^N - 1, -1, \ldots, -1) \tag{4}$$

Now we can use an algorithm similar to Hadamard Transform. (For simplicity, assume that $N = 3$.)

First we get the following two equations:

$$(x_0 + x_1, x_2 + x_3, x_4 + x_5, x_6 + x_7) \bigoplus (p_0 - 1 + p_1, p_2 + p_3, p_4 + p_5, p_6 + p_7) = (6, -2, -2, -2) \tag{5}$$

$$(x_0 - x_1, x_2 - x_3, x_4 - x_5, x_6 - x_7) \bigoplus (p_0 - 1 - p_1, p_2 - p_3, p_4 - p_5, p_6 - p_7) = (8, 0, 0, 0) \tag{6}$$

By repeating the same process recursively, we can (almost) get the values $x_0+x_1, x_2+x_3, x_4+x_5, x_6+x_7$ and $x_0 - x_1, x_2 - x_3, x_4 - x_5, x_6 - x_7$, and we can get the values x_i.

There is a small catch: in one of the deepest recursion, we get the following:

$$(x_0 + \ldots + x_7) \bigoplus (p_0 - 1 + p_1 + \ldots + p_7) = (0) \tag{7}$$

This gives no information for the first term because both the second and the third terms are zeroes.

However, it is easy to see that if we change the value for $x_0 + \ldots + x_7$ here, we add the same constant to all x_i. Thus, we can choose an arbitrary value here, and then modify the values we get (x_i) using $x_0 = 0$.

This solution works in $O(N2^N)$ time.

AGC 033 解説

yutaka1999

2019 年 05 月 05 日

For International Readers: English editorial starts on page 9.

A : Darker and Darker

一回操作を行ったときに、どのようなマスが黒色になるのかを考えてみましょう。問題文に書かれている操作を言い換えているだけですが、次のように考えると分かりやすいかもしれません。

- 各々の黒マスに対して、隣接するマスをすべて黒色にする。

このように言い換えることで、高速にシミュレーションを行うことができます。元の操作でシミュレーションを行おうとすると、毎回すべての白マスを見る必要があります。しかしこのように言い換えると、見る必要があるマスは黒マスに変わり、しかもある操作でその黒マスの周囲を黒色に変えたならば、それ以降の操作ではその黒マスを見る必要がありません。つまり、各マスを操作で見る回数というのは高々 1 回になります。

実装する際には、幅優先探索などと同様に queue を用いるのがよいでしょう。そもそも先の操作自体が、グリッドグラフ上で黒マスから幅優先探索をしているとみなすことができるので、これは自然な発想でしょう。計算量としては $O(HW)$、つまりマス目の大きさに関して線形な時間計算量で解くことができます。

B : LRUD Game

まず、縦と横が独立であることに注目します。つまり、一次元の問題を解けばよいことが分かります。問題を書きなおしてみると次のようになります。

- 長さ L の棒上に駒が置いてある。
- 先手と後手は N 回交互に駒を動かす。各ターンごとに以下のいずれかを行うことができる。
  - 動かす場合は左のみに動かせる。
  - 動かす場合は右のみに動かせる。
  - この回は動かせない。
- 先手は棒から落としたいが、後手は棒から駒を落としたくないです。
- どちらが勝つでしょう。

このようにすると考えやすくなるでしょう。あとは i 回目にどの位置に駒が置いてあれば後手が勝てるかを i が大きい方から計算していけばよいです。

実際は縦と横が独立であることに気づかなくとも、後手が勝てる駒の位置というのが長方形になるので、同様に後ろから計算していけばよいです。計算量は線形です。

C : Removing Coins

コインが残っている頂点がなす部分木を考えてみましょう。そうすると、操作を行うごとにその部分木は以下のようにして変化します。

- 選んだ頂点を根として木を根付き木にする。
- 葉の頂点をすべて消す。1 頂点しかない場合は根を消す。

ここで、この木の直径に注目してみましょう。操作を行ったあと、直径が 1 または 2 しか減少しないのは明らかです。ここで、直径の端点を選んで操作をすることで、直径はちょうど 1 減らすこともできます。さらに、直径が 2 以上の場合は、葉でない頂点が存在するので、その頂点を選んで操作をすることで、直径をちょうど 2 減らすこともできます。よって、これは以下のようなゲームと見なすことができます。

- 直径 L が与えられる。
- 先手と後手はそれぞれ L を負にならないように 1 または 2 減らすことができる。
- $L = 0$ とした人が負けのとき、どちらが勝つか求めよ。

これにより、よくあるゲームに帰着できました。L を 3 で割ったあまりが 1 のとき後手が勝ち、それ以外の場合は先手の勝ちとなります。木の直径は $O(N)$ で求めることができるので、この問題は $O(N)$ で解くことができます。

D : Complexity

1×1 のマス目は複雑さ 0 であることに注目しましょう。すると、だいたい $\log_2 H + \log_2 W$ 回ぐらいの分割で複雑さ 0 のマス目に分割できるので、マス目の複雑さは $O(\log H + \log W)$ であることが分かります。ここで、以下のようなデータを計算することを考えます。

- 複雑さ i の長方形のうち、左上のマスが (r, c)、左下のマスが (r', c) のものの横の長さの最大
- 複雑さ i の長方形のうち、左上のマスが (r, c)、右上のマスが (r, c') のものの縦の長さの最大

これはデータ量としては $O(HW(H + W) \log HW)$ なので、すべて $O(1)$ で計算できれば時間制限には間に合います。i が小さいほうから計算していくと、実際に $O(1)$ でこれは計算することができます。

E : Go around a circle

まず各円弧が赤と青の 2 色で塗られているとして、その塗り分けが条件を満たすかどうか判定する方法を考えましょう。

□ S が一色のみからなる場合

S が R のみからなるとします。このとき、円周上に青色の弧が連続していると明らかに条件を満たしません。逆に、すべての点が R に隣接していれば十分です。よって、この場合は青色の弧が連続しないことが必要十分条件になります。

□ S が R と B をともに含む場合

S の最初の文字が R だとします。先同様、円周上に青色の弧が連続していると明らかに条件を満たさないので、青色の弧が連続しないことが必要です。また、S に B が含まれるので、青色の弧は 1 つ以上なければいけません。しかしこの場合はそれでは十分ではありません。ある偶数個の連続する円弧が赤色で、かつその隣の円弧は青色であるとしましょう。S の先頭 k 文字が R でその次の文字が B だとします。このとき、その偶数長の円弧に含まれるどの点に最初駒を置いたとしても、k 回赤色の円弧を移動したのちに、青色の円弧で移動できる点、つまり円弧の端の点にいなければなりません。しかし、k が奇数ならば円弧の端の点からはじめ、k が偶数ならばその隣の点からはじめると、そのように駒を移動させることができないことが分かります。このことから、赤色の円弧が連続する個数も奇数でないといけないです。また、同様のことを考えると、赤色の円弧が連続する個数に上限があると分かります。簡単のため、S の最後の文字は B であるとします。(末尾に続く R は無視してよいです。) 先頭 k 文字が R だとしたとき, 赤色の円弧は、k が偶数のとき $k+1$ 個まで、k が奇数のとき k 個までしか連続することができないことが分かります。また、その他で R が k' 文字連続するときは、直前に B で移動していることから、駒がある位置は B に隣接する場所のみだと分かるので、k' が偶数のときは常に移動可能で、k' が奇数のときはどの赤色の円弧も k' までしか連続しないことが必要になります。以上をまとめると、以下が必要です。これが必要十分であることは簡単に確認できます。

- 青色は存在し、かつ連続しない
- 赤色は奇数個連続している
- 赤色が連続する個数は高々 L 個 (L は S から定まるある整数)

以上により条件の言い換えができました。では数え上げるためにはどうすればよいでしょう。S が一色の場合も明らかではないですが、R と B をともに含む場合と同様に解くことができるので、ここでは R と B をともに含む場合のみを扱います。

まず、いずれの色も連続して奇数個でしか現れないので、円弧は偶数個でないといけな

いです。また、青色の円弧の番号の偶奇は等しい必要があります。その偶奇を固定したときに条件を満たす個数は同じであるので、以降青色は偶数番目の円弧にのみ塗るとして構いません。すると奇数番目の円弧はすべて赤色で塗る必要があるので、考えなくてもよさそうです。実際、以下の問題を解けばよいことが分かります。

- $\frac{N}{2}$ 個の円周を青と赤で塗る。
- 青を一か所以上塗る必要がある。
- 赤は連続して L 個までしか塗ることはできない。
- 以上の条件を満たすものを数える。

いきなり円周上で考えるのは難しいので、直線の場合を考えてみましょう。つまり以下の問題を考えてみます。

- X 等分されている棒のそれぞれの部分を赤と青で塗る。
- 両端は青で塗る必要がある。
- 赤は連続して L 個までしか塗ることはできない。
- 以上の条件を満たすものを数える。

これは累積和を持ちながら dp を行うことで $O(X)$ で解くことができます。$X = i$ のときの答えを $dp[i]$ とすると、$dp[i] = \sum_{j=max(1,i-L-1)}^{i-1} dp[j]$ であるので、$rdp[i] = \sum_{j=1}^{i} dp[j]$ とすると $dp[i] = rdp[i-1] - rdp[\max(0, i-L-2)]$ となります。よって、$dp[i]$ と $rdp[i]$ を $i = 1, \ldots, X$ の順に更新していくことで、$O(X)$ の計算量でこの問題が解けます。

では元の問題を考えてみましょう。実は先の $dp[i]$ を用いることで簡単に答えは求まります。円周上の適当な位置で円周を切ってみます。すると数えるべき塗り分け方は、青を一か所以上塗り、どの赤も L 個までしか連続せず、さらに両端に連続する赤の個数の合計が L 個以下のものです。これは、一番左の一番右の青の間の距離を定めれば、後は適当な係数をかけるだけで求まります。さらに、その距離を j とすると、一番左の青と一番右の青の間を塗る方法は $dp[j+2]$ 通りです。よって、この問題が線形時間で解けました。

F : Adding Edges

G に辺を加えていくのは難しいので、辺を短くしていくことを考えます。つまり、G に対して以下の操作を行うことを考えます。

- G に辺 ab,ac があり、a, b, c がこの順にあるパス上にあるとき、辺 ac を消して辺 bc を加える。

この操作を行ったあとに G に対して辺を加えていっても、最終的に生成できる辺の集合は変わりません。そこで、G に対してこの操作を繰り返し行い、これ以上操作ができなくなった状態というのを考えることにします。このとき、そのような G に対して元の操作を繰り返して生成できる辺というのは、以下のようにかけることが示せます。

- 辺 xy が生成できることは、ある頂点の列 $x = a_0, a_1, \ldots, a_k = y$ があって、$a_i a_{i+1}$ が G に含まれており、かつ a_i, a_{i+1}, a_{i+2} がこの順にあるパス上にあることと同値である。

Proof. 上の条件が十分であることは明らかです。ここでは必要であることを示します。証明するためには、先の条件を満たすような 2 つの辺 ab, ac があったときに、b, a, c があるパス上にあるならば、それらから生成される辺 bc も先の条件を満たすことを示せばよいです。b, a, c がこの順にパス上にあるときはよいです。そうでない場合は、辺 ab, ac に対して条件を満たす頂点の列をとると、G に対して辺を短くする操作がこれ以上行えないことから、ある辺 ab, ac 上の a と異なる頂点 x があって、辺 xb, xc が条件を満たして生成される辺であることが分かります。これを繰り返し行うことで、x が b または c と一致するので、辺 bc がこの命題の条件を満たして生成されることが示されます。 □

以上より、G の辺をこれ以上短くできない場合、BFS 等を用いることにより $O(NM)$ でこの問題を解くことができます。ではどうすれば G の辺を短くしていけるかを考えてみましょう。一つの方法は、各頂点ごとにその頂点を根としたときの euler-tour を計算しておき、set などを用いて短くなる辺を効率よく探す方法です。しかし、これは計算量に log がつきますし、実装も少し大変です。ここでは、$O(NM)$ で辺を短くしていく方法を紹介します。

辺を一つずつ追加していき、徐々に短くしていくことを考えます。各頂点ごとにその点を端点に持つ辺は、もう一方の端点が○○の頂点ならば短くできる、という情報を持っておきます。辺を追加するとき、一方の端点について先の情報から短くできるならば、その辺を短くして、もう一回やり直します。どちらの端点を見ても短くできない場合は、それ

ぞれの端点に対して、辺を短くできる頂点の集合というのが増えます。よって、適当に短くできる頂点の集合を増やし、短くできる辺を見つけたら短くすることで、短縮を効率よく行えます。

上には概略を述べましたが、詳しくは writer のコードを確認してもらえると、分かりやすいと思います。全体で $O(N^2 + NM)$ の計算量で解くことができます。

AGC 033 Editorial

yutaka1999

May 5, 2019

A : Darker and Darker

Let us consider what squares will become black after one operation. In different words from the statement, our operation is as follows:

- For each black square, paint the squares adjacent to it black.

By rephrasing this way, we can quickly simulate the process. We needed to observe all white squares every time if we are to literally simulate the process, but now we only need to observe each square once: we only need to observe a square when it is being painted black, and after we paint the squares adjacent to it black, we never need to observe it again.

In the implementation, we recommend using a queue in the similar way to breadth-first search. Since the rephrased operation can be regarded as performing breadth-first search from the black squares in the grid graph, this approach is quite natural. The time complexity of this solution is $O(HW)$: linear on the size of the grid.

B : LRUD Game

First, notice that the two directions in this game, horizontal and vertical, are independent from each other. That is, we just need to solve the one-dimensional version of this problem, as follows:

- There is a piece on a bar of length L.
- Two players alternately performs N moves each. Each turn is in one of the following forms:
  - The player can move the piece to the left or do nothing.
  - The player can move the piece to the right or do nothing.
  - The player cannot move the piece this time.
- The player who goes first wants to drop the piece from the bar, while the player who goes second wants to prevent it.
- Which player will win?

Now, the problem should be easier. We just need to calculate the area such that the second player can win if the piece is in that area in the i-th turn, in the descending order of i.

If you missed the first observation that the two directions are independent, you can still solve the problem similarly, since the area from which the second player can win is a rectangle. The complexity of the solution is linear.

C : Removing Coins

Let us consider the subtree formed by the vertices that still contain coins. In each operation, it is changed as follows:

- Make the tree rooted by designating the chosen vertex as the root.
- Erase all the leaf vertices. Erase the root if it is the only vertex.

Now, let us pay attention to the diameter of this tree. Obviously, the diameter decreases by 1 or 2 in each operation. We can always choose one end of the diameter to reduce the diameter by 1. Also, when the diameter is at least 2, the tree has a non-leaf vertex and we can always choose it to reduce the diameter by 2. Thus, we can consider the game to be played as follows:

- Given is the diameter L.
- Each of the two players can reduce L by 1 or 2 without making it negative.
- The player who lets $L = 0$ loses. Find which player will win.

This is a quite common game. The second player will win if $L \equiv 1 \mod 3$, and the first player will win otherwise. Since the diameter of a tree can be found in $O(N)$ time, the problem can be solved in $O(N)$ time.

D : Complexity

Note that a 1×1 subgrid has a complexity of 0. The grid will be divided into such subgrids after approximately $\log_2 H + \log_2 W$ divisions, so we can see that the complexity of the grid is $O(\log H + \log W)$. Now, let us consider computing the following values:

- The maximum horizontal length of a subgrid with complexity i whose top-left and bottom-left squares are (r, c) and (r', c)
- The maximum vertical length of a subgrid with complexity i whose top-left and top-right squares are (r, c) and (r, c')

There are $O(HW(H+W)\log HW)$ values, so it will run in time if we can compute each of them in $O(1)$, which is actually possible by computing them in ascending order of i.

E : Go around a circle

First, assume that each arc is already painted red or blue, and let us think of the way to determine if that coloring satisfies the condition.

□ If S consists of a single color

We assume that S consists only of R. If there are two consecutive blue arcs along the perimeter, the condition is obviously violated. On the other hand, it is sufficient if every point is neighboring to a red arc. Thus, the necessary and sufficient condition here is that there is no two consecutive blue arcs.

□ If S contains both R and B

We assume that S begins with R. Similarly to the above, two consecutive blue arcs obviously violate the condition, so it is necessary that there is no two consecutive blue arcs. Additionally, as S contains a B, there must be at least one blue arc. It is not sufficient this time, though. Assume that some even number of consecutive arcs are red, and the arcs next to them are blue. Let the first k characters in S be Rs and the next character is B. Then, wherever along the even number of red arcs we place the checker in the beginning, after k moves along a red arc we must be at the end of the sequence of red arcs, where we can make a move along a blue arc. However, this cannot be achieved if k is odd and we begin from an end of the red sequence, or k is even and we begin from a point next to that end. Therefore, the number of the consecutive red arcs must be odd. Similarly, we see that there is an upper limit to the number of the consecutive red arcs. For simplicity, assume that S ends with B. (Trailing Rs can be ignored.) Assuming that the first k characters are Rs, there can be at most $k+1$ consecutive red arcs if k is even, and at most k consecutive red arcs if k is odd. If there are k' consecutive Rs at other than the beginning of S, the piece is guaranteed to be next to a blue arc when we make the moves corresponding to those Rs since we have just moved along a blue arc. Thus, if k' is even, the moves corresponding to those Rs are always possible. If k' is odd, it is necessary that there are at most k' consecutive red arcs. To sum it up, the necessary conditions, which can be easily verified to be necessary and sufficient, are as follows:

- There is a blue arc, but not two in a row.
- The length of every sequence of consecutive red arcs is odd.
- There is at most L consecutive red arcs, where L is some integer derived from

S.

We have now rephrased the condition, but how do we count the colorings? It is not trivial even if S consists of a single color, but the solution is similar to the case with two colors, so here we will only handle the latter.

First, since the length of every sequence of arcs of the same color is odd, the number of arcs must be even. Also, the parities of the indices of all blue arcs must be the same. We can safely assume that blue is only used on the arcs with even indices, since fixing it to odd indices results in the same number of colorings. Then, all arcs with odd indices must be painted red, and it seems that we do not explicitly consider them. Actually, the problem is equivalent to the following:

- We paint $\frac{N}{2}$ arcs blue or red.
- At least one arc must be painted blue.
- There can be at most L consecutive red arcs.
- Find the number of colorings under the above conditions.

It is difficult on a circle, so let us consider the linear version, that is, the following problem:

- There is a bar divided into X segments of equal length. We paint each segment red or blue.
- The both ends must be painted blue.
- There can be at most L consecutive red segments.
- Find the number of colorings under the above conditions.

This can be solved in $O(X)$ time by dynamic programming with prefix sums. Let $dp[i]$ be the answer for the case $X = i$. Then, $dp[i] = \sum_{j=max(1,i-L-1)}^{i-1} dp[j]$ holds, so if we let $rdp[i] = \sum_{j=1}^{i} dp[j]$, $dp[i] = rdp[i-1] - rdp[\max(0, i-L-2)]$. Thus, by updating $dp[i]$ and $rdp[i]$ in the order $i = 1, \ldots, X$, the problem can be solved with the time complexity $O(X)$.

Now, let us get back to the original problem. It turns out that we can easily found the answer by using $dp[i]$ above. Let us cut the perimeter at some point. Then, we want to count the number of colorings such that there is at least one blue arc, at most L consecutive red arcs and the total number of the consecutive red arcs at both ends

is at most L. If we fix the distance between the leftmost and rightmost blue arcs to be j, the number of ways to color the arcs between them is $dp[j+2]$. By multiplying it by some coefficient, the problem is now solved in linear time.

F : Adding Edges

Instead of adding edges, which is difficult, let us consider "shortening" edges, that is, performing the following operation on G:

- When G has edges ab and ac and the vertices a, b, c in T lie on some path in this order, erase the edge ac and add an edge bc.

After this operation, the final set of edges in G that can be generated by adding edges does not change, so let us consider the situation where we have performed this operation repeatedly until we can no longer do it. In this situation, it can be shown that we can represent an edge that can be generated as follows:

- An edge xy can be generated if and only if there exists a sequence of vertices $x = a_0, a_1, \ldots, a_k = y$ such that G contains the edges $a_i a_{i+1}$ and the vertices a_i, a_{i+1}, a_{i+2} in T lie on some path in this order.

Proof. The sufficiency of the above condition is obvious, so here we will only show the necessity. We can prove it by showing that, if there are two edges ab, ac satisfying the condition and the vertices b, a, c in T lie on some path, the edge bc generated from these two edges also satisfy the condition. If the vertices b, a, c lie on the path in this order, the condition is obviously satisfied. Otherwise, consider the sequences of vertices for the edges ab, ac that the condition requires. From the fact that we can no longer perform the operation of shortening an edge in G, we can see that there exists a vertex x ($x \neq a$) along the generated edges ab, ac such that the edges xb, xc satisfy the condition to be generated. By applying this procedure repeatedly, x will finally coincide with b or c, which shows that the edge bc satisfies the condition to be generated. □

Thus, when we cannot shorten any edge in G, we can solve this problem by BFS or similar methods in $O(NM)$ time. Now, let us consider how we can shorten the edges in G. One way to do this is: for each vertex, we compute the euler-tour when that vertex is considered as the root, and efficiently search the edges to be shortened with `set` or similar structures. However, we will have log in the complexity, and it is a little tough to implement. Here is an $O(NM)$ approach to shorten the edges:

We will add the edges from the input one by one to the graph and gradually shorten

them. For each vertex v, we maintain the set of vertices such that an edge from v can be shortened if the other endpoint is in that set. When we add an edge, if it can be shortened according to the information above, we will shorten it and do this again. If we cannot shorten the edge after considering both endpoints, the set above for each of the endpoints will have new elements. For more detail, see the writer's code (link). The total complexity of the solution is $O(N^2 + NM)$.

AGC032 解説

writer : many

2019 年 3 月 23 日

For International Readers: English editorial starts on page 9.

A: Limited Insertion

操作を逆順に見ると問題は以下のように言い換えられます。

> 数の書かれた N 個のボールが一列に並んでいる。すぬけ君は左から k 番目のボールに k と書いてあるとき、それを選んで取り除くことができる。全てのボールを取り除くことは可能か？

すぬけ君は取り除くことが可能なボールのうち、最も右側にあるものを選ぶしかありません。そうでないとすると二度と取り除くことができないボールが生じてしまい、全てのボールを取り除くことが不可能になることから容易に示されます。以上のことから操作手順は存在するならば実は 1 通りしかないことも分かります。

上記の戦略は $O(N^2)$ でシミュレート可能なため、実行時間にも十分余裕があります。

B: Balanced Neighbors

以下のような条件を満たすようなグラフを考えます。

- 非連結
- 任意の頂点について、自身の番号と隣接する頂点の番号の和が一定

この条件は補グラフが問題文中の条件を満たすための十分条件となっています。この条件を考えると、$\{1, \ldots, N\}$ を 2 つ以上のグループに分割する方法であって、グループに含まれる数の和がグループによらず一定となるような方法を 1 つ見つけられればいいことが分かります。

そのような方法が存在したなら、同じグループに属するような頂点間に辺を張ったグラフが上の条件を満たします。

分割方法の方法の例としては以下の方法があります。

- N が奇数のとき： $\{1, N-1\}, \{2, N-2\}, \ldots, \{i, N-i\}, \ldots, \{N\}$
- N が偶数のとき： $\{1, N\}, \{2, N-1\}, \ldots, \{i, N+1-i\}$

上の方法で構成されるグラフの補グラフは $O(N^2)$ で構成可能であり、実行時間にも十分余裕があります。

C: Three Circuits

与えられたグラフ G がオイラーグラフでないとき、明らかに答えは `No` です。以降、G はオイラーグラフであることを仮定します。結論から言えば、以下のいずれかの条件を満たすときのみ答えは `Yes` です。

- 次数 6 以上の頂点が存在する
- 次数 4 の頂点が 3 つ以上存在する
- 次数 4 の頂点が 2 つ存在し、その 2 つの頂点をつなぐパスはちょうど 2 つある

次数 6 以上の頂点が存在する場合

そのような頂点 A を始点とするオイラーサイクルを 1 つ考えます。また、A の次数を D とします。このオイラーサイクルは、明らかに A を始点とする $\frac{D}{2}$ 個のサーキットに分解可能です。また、2 つのサーキットを 1 つにまとめられることも明らかです。よって 3 個のサーキットを構成可能です。

次数 4 の頂点が 3 つ以上存在する場合

上記の場合を除くと、次数 4 の頂点と次数 2 の頂点のみ含まれるグラフを考えればいいことが分かります。以降は全ての頂点は次数 2 か 4 であることを仮定します。

次数 4 の頂点 A を始点とするオイラーサイクルを 1 つ考えます。上記の議論から A を始点とする 2 つのサーキットを作ることができます。図 1 のように、このサーキットの少なくとも一方において、2 回訪問される頂点 B が存在したなら、B を始点とするサーキットを追加で 1 つ作ることができ、3 つのサーキットを構成可能です。そうでない場合、それぞれのサーキットでちょうど 1 回ずつ訪問されるような頂点 B, C が存在します。図 2 のように $A \to B, B \to C, C \to A$ というパスを 2 つずつ取れるので、これらを組み合わせて 3 つのサーキットが構成可能です。

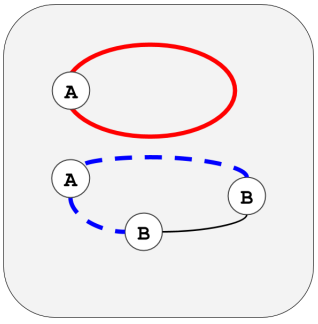


図 1　2 つのサーキットの少なくとも一方に 2 回訪問される頂点が存在する場合

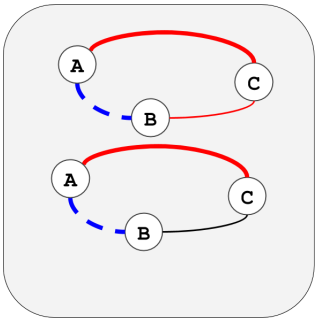


図 2　それ以外の場合

次数 4 の頂点が 2 つ存在する場合

ありうるグラフは以下の 2 通りです。A と B をつなぐパスが 2 つあるときのみ 3 つのサーキットが構成できることがわかります。例えば、A-B 最小カットを取るなどの方法で判定をすることができます。

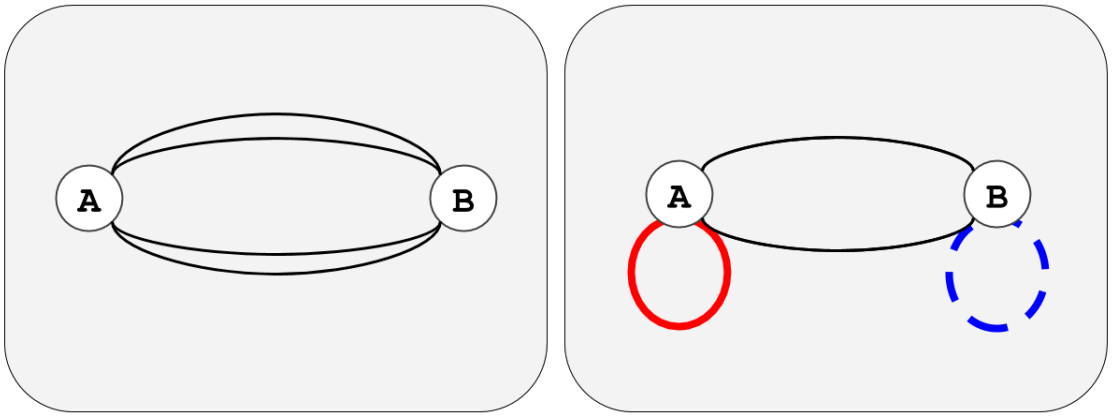


図 3　次数 4 の頂点が 2 つ存在するようなグラフ

次数 4 の頂点が 1 つ以下の場合は明らかです。いずれの場合も $O(N+M)$ で判定可能であり、十分高速です。

D - Rotation Sort

2 種類の操作はそれぞれ次のように言い換えられます.

- コスト A を支払い，ある要素を右側へ動かす.
- コスト B を支払い，ある要素を左側へ動かす.

見通しを良くするために，問題を次のように言い換えます.

> 最初，各 i $(1 \leq i \leq N)$ について要素 p_i が座標 i に置かれている．あなたは，次の 2 種類の操作を好きな順序で繰り返し行うことができる.
>
> - コスト A を支払い，ある要素を右側の (**整数とは限らない**) 座標へ動かす.
> - コスト B を支払い，ある要素を左側の (**整数とは限らない**) 座標へ動かす.
>
> 最終的に，N 個の要素が左から右へ昇順に並んでいるようにしたい．必要な総コストの最小値を求めよ.

明らかに，ある要素を 2 回以上動かすのは無駄です．よって，要素 k の最初の座標を x_k とし，最後の座標を x'_k とすると，この要素に関するコストは次のようになります.

$$\begin{cases} 0 & (x_k = x'_k) \\ A & (x_k < x'_k) \\ B & (x_k > x'_k) \end{cases}$$

このコストの総和を最小化するように，N 個の要素の最後の座標 $x'_1 < x'_2 < \cdots < x'_N$ を決めるのが目標です.

要素 $1, 2, \ldots, N$ の順に座標を決めて置いていくことにすると，次のような DP が可能です.

$\mathrm{dp}[k][x'] :=$ (要素 $1, 2, \ldots, k$ を既に置いており，要素 k の座標が x' であるときの，今までの総コストの最小値)

しかし，x' は連続値なので，このままでは計算ができません．よく考えると，$(-\infty, 1), (1, 2), \ldots, (N-1, N), (N, \infty)$ の各開区間については，x' の正確な値を覚えておく必要はなく，「x' がこの開区間に含まれる」ということだけ覚えておけば十分です．よって，x' の取りうる値は $1, 2, \ldots, N$ とこれらの開区間だけで十分であり，計算ができるようになりました.

愚直に実装すると時間計算量は $\mathrm{O}(N^3)$ で TLE しますが，適当な高速化によって $\mathrm{O}(N^2)$ へ改善できます.

E - Modulo Pairing

まず，a を昇順にソートしておきます．以降，ペア分けを図 1 のように表します．ここで，青は $x+y<M$ なるペアを表し，赤は $x+y\geq M$ なるペアを表します．

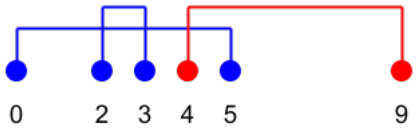


図 1

実は，図 2 のようなペア分けが最適解のひとつとして存在します (証明は後述)．

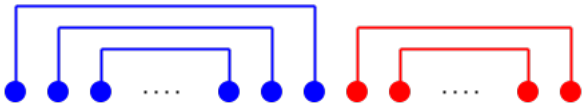


図 2

よって，青と赤の境界線の位置を全探索することで，最適値が求まります．境界線の位置をひとつ固定した後は，境界線の左側 (右側) で図 2 のようにペア分けし，各ペアが青 (赤) であることを確かめた後，全ペアの醜さの最大値を求めればよいです．しかし，この方法の時間計算量は $\mathrm{O}(N^2)$ で，TLE してしまいます．

この方法を高速化するために，次の事実に注目します．

- 境界線が左であるほど，境界線の左側の各ペアは青になりやすくなる．
- 境界線が左であるほど，境界線の右側の各ペアは赤になりにくくなる．
- 境界線が左であるほど，全ペアの醜さの最大値は小さくなる．

よって，境界線の位置を二分探索し，「境界線の右側の各ペアが赤である」という条件を満たすような最も左の境界線を求め，そのときの全ペアの醜さの最大値を求めれば，それが答えです．この方法の時間計算量は $\mathrm{O}(N\log N)$ であり，十分高速です．

図 2 のような最適解が存在することの証明

図 3 はペア分けの局所的な組み換えを表しています．ペア分けの中に矢印の左側のような 2 ペアが存在する場合，矢印の右側のような 2 ペアに組み換えることが常に可能であり，このとき，これら 2 ペアの醜さの最大値が大きくなることはありません．このような組み換えを繰り返すことで，任意の解を図 2 のような解に変形できることが示せます．さらに，このとき全ペアの醜さの最大値が大きくなることはありません．よって，図 2 のようなペア分けが最適解のひとつとして存在します．

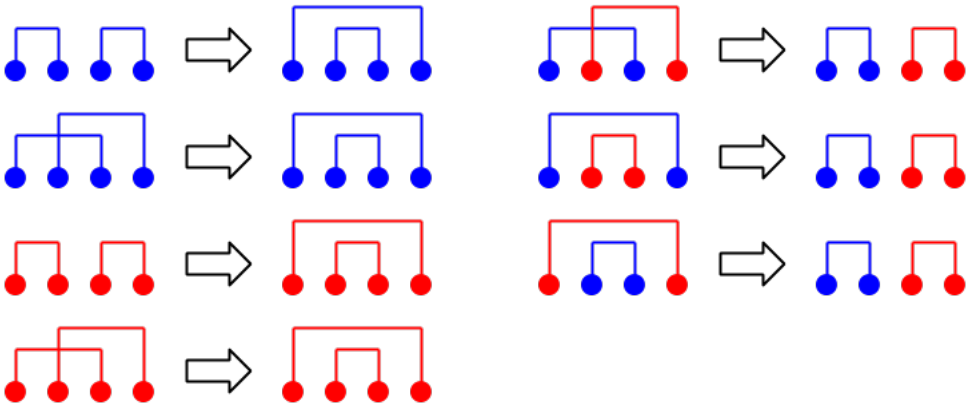


図 3

F - One Third

それぞれの切れ込みに沿って赤い線を引くことにします。また、各赤線に対し、それを時計回りに 120 度回転させた線を青で、反時計回りに 120 度回転させた線を緑で引きます。このとき、値 $|x - 1/3|$ は二本の異なる色の線分がなす最小の角度であることが容易にわかります。

よって、この問題は次と同値です。

> 数直線上の点 0 に赤い点を、点 1/3 に青い点を打ち、次の操作を $N - 1$ 回繰り返す:「0 以上 1/3 以下の座標を一つ、一様にランダムに選ぶ。また、赤青緑から一色をランダムに選ぶ。選んだ位置に選んだ色の点を打つ」。操作後、異なる色の二点間の最小距離の期待値はいくつか。

操作後、区間 $[0, 1/3]$ は数直線上にある $N + 1$ 個の点によって N 個の断片に分割されます。両端の点の色が異なるような断片を **カラフル** であると呼ぶことにすると、最短のカラフルな断片の長さを求めたいです。

次の値を計算しましょう。

- カラフルな断片の個数が k となる確率を $f(k)$ とします。これは、赤青緑の三色からなる長さ $N + 1$ の列であって、先頭が赤、末尾が青であり、異なる二色が隣接する箇所が k 個存在するようなものの個数を 3^{N-1} で割ったものです。単純な動的計画法と二項係数によって求められます。
- カラフルな断片の個数が k であるときの最短のカラフルな断片の長さの期待値を $g(k)$ とします。この場合の k 個の断片の長さの総和の期待値は $k/3n$ です。また、単純な計算により、線分を k 個の断片にランダムに分割するとき、最短の断片の長さの期待値は元の線分の長さの $1/k^2$ 倍であることがわかります。よって、$g(k) = 1/3nk$ です。

k が取りうるすべての値に対する $f(k)g(k)$ の和が答えです。

AGC032 Editorial

writer : many

March 23, 2019

A: Limited Insertion

By reversing the timeline, the problem can be rephrased into the following:

> We have N balls, each with an integer written on it. If the k-th ball from the left has the number k, Snuke can remove that ball. Can he remove all the balls?

Snuke has no choice but to remove the rightmost ball that can be removed. This can be easily shown from the fact that other moves will produce a ball that cannot be removed forever. From this, it can also be seen that the way to remove all the balls is unique if it exists.

The strategy above can be simulated in $O(N^2)$ time, which runs in time easily.

B: Balanced Neighbors

Consider a graph satisfying the following conditions:

- Not connected.
- For every vertex, the sum of the indices of the vertices adjacent to that vertex is constant.

These conditions are sufficient for its complement graph to satisfy the conditions in the statement. Thus, if we can find a way to divide $\{1, \ldots, N\}$ into two or more groups so that the sum of the integers in a group is constant, we have an instance of the above graph by connecting vertices belonging to the same group, and thus we have a solution.

One way to divide $\{1, \ldots, N\}$ is as follows:

- If N is odd: $\{1, N-1\}, \{2, N-2\}, \ldots, \{i, N-i\}, \ldots, \{N\}$
- If N is even : $\{1, N\}, \{2, N-1\}, \ldots, \{i, N+1-i\}$

The complement graph of the graph generated by the division above can be built in $\mathrm{O}(N^2)$ time, which is sufficiently fast.

C: Three Circuits

If the given graph G is not Eulerlian, the answer is obviously `No`. From now on, we assume that G is Eulerlian. The conclusion is: the answer is `Yes` if and only if the following condition is satisfied:

- There exists a vertex with degree 6 or greater.
- There exists three or more vertices with degree 4.
- There exists two vertices with degree 4, and exactly two paths connecting them.

If there is a vertex with degree 6 or greater

Consider a Eulerian cycle beginning with such a vertex A, and let D be the degree of A. This Eulerian cycle can obviously divided into $\frac{D}{2}$ circuits beginning with A. It is also obvious that two of these circuits can be merged into one. Thus, we can build three circuits.

If there are three or more vertex with degree 4

Excluding the above case, we only need to consider graphs that only contains vertices of degree 4 or 2. From now on, we assume that the degree of every vertex is 2 or 4.

Consider a Eulerian cycle beginning with a vertex A of degree 4. From the discussion similar to the above, we can build two circuits beginning with A. If there is a vertex B visited twice in at least one of them, as shown in Fig. 1, we can build one more circuit beginning with B, for a total of three. Otherwise, there exist vertices B and C that is visited exactly once in each of the two circuits. As shown in Fig. 2, we can take two paths in each of the forms $A \to B, B \to C, C \to A$, and we can combine them to form three circuits.

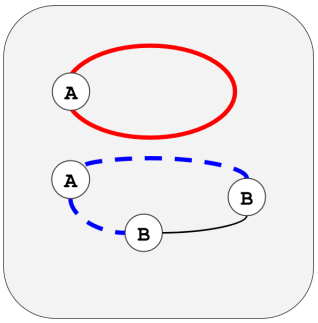


Fig.1 The case where there is a vertex visited twice in at least one of the circuits

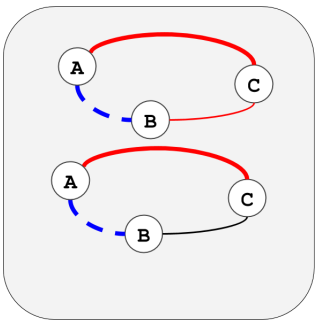


Fig.2 The other case

If there are two vertices with degree 4

The two possible forms of the graph are shown below. It can be seen that three circuits can be built if and only if there are two paths connecting A and B. One way to check this is to find an A-B minimum cut.

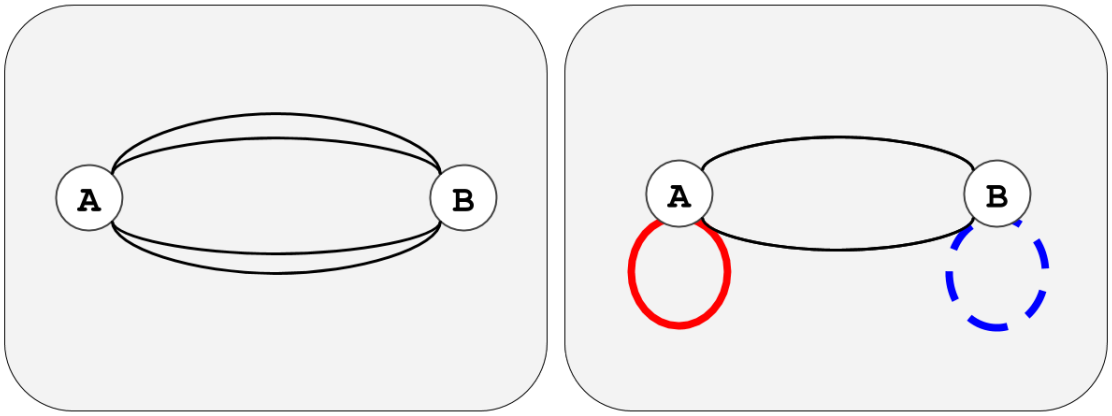


Fig.3 Graphs with two vertices with degree 4

The answer is obvious for a graph with one or less vertex with degree 4.
Therefore, in all the cases we can determine the answer in $\mathrm{O}(N + M)$ time, which is fast enough.

D - Rotation Sort

We can rephrase the two kinds operations as follows:

- Pay a cost A to move an element to some position to the right.
- Pay a cost A to move an element to some position to the left.

Now we rephrase the problem to make it more approachable, as follows:

> Initially, for each i ($1 \leq i \leq N$), an element p_i is placed at coordinate i. You can perform the following two kinds of operations any number of times in any order:
>
> - Pay a cost A and move an element to some coordinate (*not necessarily an integer*) to the right.
> - Pay a cost A and move an element to some coordinate (*not necessarily an integer*) to the right.
>
> In the end, the N elements should be sorted in ascending order from left to right. Find the minimum total cost required.

Obviously, it is unnecessary to move an element two or more times. Thus, the cost for element k is as follows, where x_k and x'_k are its initial and final coordinates:

$$\begin{cases} 0 & (x_k = x'_k) \\ A & (x_k < x'_k) \\ B & (x_k > x'_k) \end{cases}$$

Our objective is to determine the final coordinates of the N elements, $x'_1 < x'_2 < \cdots < x'_N$, so that the sum of these costs is minimized.

By deciding the final coordinates for the elements $1, 2, \ldots, N$ in this order, we have the following DP solution:

$$\mathrm{dp}[k][x'] := (\text{Minimum total cost to place elements } 1, 2, \ldots, k \text{ with } k \text{ placed at coordinate } x')$$

However, this does not work as is, since x' takes a continuous value. We can bypass this problem by not memorizing the exact value of x', and instead memorizing which of the open intervals $(-\infty, 1), (1, 2), \ldots, (N-1, N), (N, \infty)$ contains x', if it is contained in one of them. Thus, the only "values" that x' can take are $1, 2, \ldots, N$ and these open intervals, and now the solution actually works.

The time complexity is $\mathrm{O}(N^3)$ if naively implemented, which gets TLE, but it can be improved to $\mathrm{O}(N^2)$ by properly speeding it up.

E - Modulo Pairing

First, we sort a in ascending order. From now on, we use the visual representation of the pairing as shown in 4. Here, pairs in blue represent pairs such that $x + y < M$, and pairs in red represent pairs such that $x + y \geq M$.

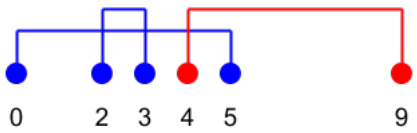


Fig.4

The conclusion is that one optimal pairing is the one such as shown in 5 (we will provide the proof later).

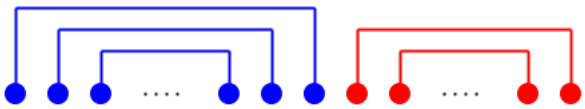


Fig.5

Thus, we can find the optimal solution by trying all possible positions of the border between blue and red. When we fix the border to one position, we make pairs in the left/right of the border as 5, verify that each pair is actually blue/red, and find the largest ugliness of all pairs. However, this method runs in $O(N^2)$ time and gets TLE.

In order to make it faster, we note the following facts:

- When the border is moved left, each pair to the left of the border has more tendency to be blue.
- When the border is moved left, each pair to the right of the border has less tendency to be red.
- When the border is moved left, the largest ugliness of all pairs gets smaller.

Thus, we can find the answer by finding the leftmost possible position of the border such that each pair to the right of the border is red, and finding the largest ugliness of all pairs in that case. This method now runs in $O(N \log N)$ time, which is fast enough.

Proof: there exists a optimal pairing in the form shown in Fig. 5

The following Fig. 6 shows the ways to reconstruct the local pairing. In a pairing, if there are two pairs in a form shown to the left of an arrow, we can always convert them to the corresponding two pairs shown to the right of the arrow, and this does not increase the larger ugliness of the pairs. By repeating this reconstruction, we can show that any pairing can be converted to a pairing in the form shown in

Fig. 5 without increasing the largest ugliness of all pairs. Thus, there exists a optimal pairing in the form shown in Fig. 5

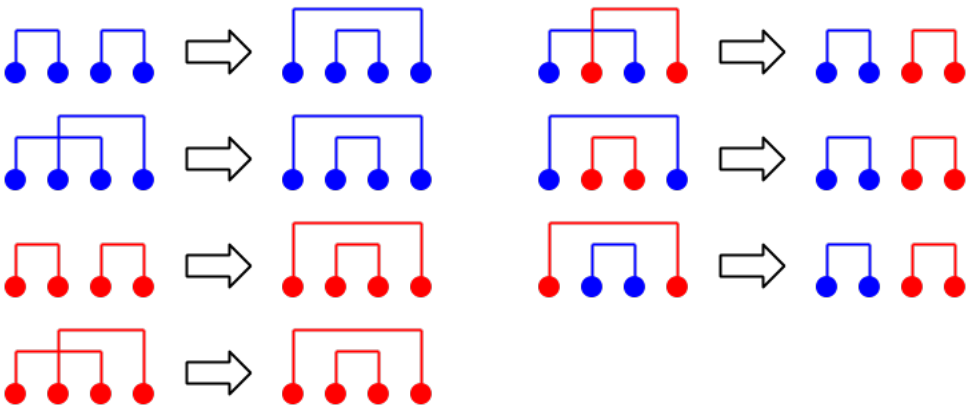


Fig.6

F - One Third

Let's draw a red segment along each cut. Also, for each red segment, draw a blue line 120 degrees clockwise to it, and draw a green line 120 degrees counter-clockwise to it. It's easy to see that the value $|x - 1/3|$ is the smallest angle formed by two segments of different colors.

Therefore, the problem is equivalent to the following:

> On the number line, we plot a red point at 0, and a blue point at 1/3. Then, we repeat the following operation $N - 1$ times: choose a coordinate between 0 and 1/3 uniformly at random, choose one of three colors at random, and draw a point of the chosen color at the chosen position. What is the shortest distance between two points of different colors?

After the operations, there will be $N + 1$ points, and these points will divide the interval $[0, 1/3]$ into N parts. Let's call a part *colorful* if the two points at its ends have different colors. We are interested in the length of the shortest colorful part.

Let's compute the following values:

- Let $f(k)$ be the probability that the number of colorful parts becomes k. This is the number of sequences of colors of length $k + 1$, such that the sequence starts with red, ends with blue, and a pair of adjacent same colors appear k times, divided by 3^{N-1}. This can be computed by a simple DP and binomial coefficients.
- Let $g(k)$ be the expected length of the shortest colorful part in case there are k colorful parts. In this case, the expected total length of the k colorful parts is $k/3n$. A simple calculation shows that when we divide a segment into k parts at random, the expected length of the shortest part is $1/k^2$ times the length of the original segment. Thus, $g(k) = 1/3nk$.

The answer is the sum of $f(k)g(k)$ over all possible k.

AGC032 解説

writer : many

2019 年 3 月 16 日

For International Readers: English editorial starts on page 8.

A : Colorful Subsequence

同じ文字列でも、異なる位置から作られたものは区別するため、$2^N - 1$ 通りのすべての部分列が区別されることになります。

条件より、同じ文字を 2 度使ってはいけないため、ある文字 c について colorful の条件を壊さないとり方は (c の出現回数 $+ 1$) となります (どれか 1 つを取るケース及び文字 c を一切取らないケース)

すべての文字 c についてのこのとり方の積を求め、空文字列の分の 1 を引いた数が答えとなります。

AGC032 解説

writer : many

平成 31 年 3 月 15 日

B : Reversi

まず、作れる列の連続する部分列であって、すべて同じ色の石からなるような極大なもの最初と最後の石の色は、最初の列から変化していないことがわかります (もし変化していたなら、その石の両側にそれと同じ色の石があるはずです)。逆に、この条件が満たされているような列は、同じ石の色の連続する部分列の両端についてそれぞれ操作を行うことで常に作ることができます。よって、この条件を満たす列を数えればよいです。

DP[i] を、最初から i 個の石たちの条件を満たすような塗り方の個数とします。DP[i] は、i 番目の石の色が $j+1$ 番目の石の色が等しく j 番目の石の色と異なるような j たちすべてに対する DP[j] の和として計算できます (ただし、便宜上 0 番目の石は他のどれとも異なる色で塗られていると考えます)。

このままでは時間計算量が $O(N^2)$ かかってしまい、間に合いません。そこで、各色に対し、$j+1$ 番目の石がその色で塗られており j 番目の石がその色で塗られていないような j に対する DP[j] の和を持っておき、DP 値が新しく計算されるたびにその都度足しこんでいけば、各遷移を $O(1)$ 時間で動くように実装でき、全体として $O(N)$ 時間でこの問題を解くことができます。

AGC032 解説

writer : many

平成 31 年 3 月 15 日

1 C : Differ by 1 Bit

P_i と P_{i+1} はちょうど 1bit だけ異なるという条件から、P_i の立っている bit の数の偶奇がわかります。特に、P_0 と P_{2^N-1} では、立っている bit の個数の偶奇は異なります。よって、A, B の立っている bit の個数の偶奇が等しい場合は、構成は不可能だとわかります。

以下、A, B の立っている bit の個数の偶奇が異なる場合に、必ず構成ができることを示します。帰納法で示します。まず、$N = 1$ のときは明らかに可能です。次に、ある K について、$N = K$ の場合に必ず構成ができると仮定します。ここで、$N = K + 1$ の問題を考えます。A と B で状態が異なる bit が少なくとも 1 つとれます。これが x bit 目だったとします。A, B の x bit 目を取り除いた、$(N-1)$ bit の数を考え、それぞれ A', B' とします。明らかに、A' と B' の立っている bit の個数の偶奇は等しいです。ここで、$(N-1)$ bit の数 C であって、A' 及び B' と立っている bit の数の偶奇が異なるものがとれます。（A' の 1 bit 目を反転させるなどすれば良いです）ここで、帰納法の仮定より、A' から始まって C で終わる長さ 2^{N-1} の順列 Q、及び C で始まって B' で終わる長さ 2^{N-1} の順列 R がとれます。ここで、条件を満たす順列 P を次のように構築できます。

- $i < 2^{N-1}$: P_i の x bit 目は A の x bit 目と同じにする。残りの $(N-1)$ bit は Q_i の対応する bit と同じにする。
- $2^{N-1} \leq i < 2^N$: P_i の x bit 目は B の x bit 目と同じにする。残りの $(N-1)$ bit は $R_{i-2^{N-1}}$ の対応する bit と同じにする。

こうしてできる順列 P が条件を満たすことは簡単に確認できます。

以上で構築が可能であることが示されました。上記の手順をそのまま実装すれば実際に順列を構築できます。実装にもよりますが、計算量は $O(N^2 2^N)$ や $O(N 2^N)$ です。

D : A Sequence of Permutations

順列 p, q に対して、pq でそれらの合成を表すことにします。つまり、pq は i 項目が p_{q_i} であるような順列を指します。この表記を用いると $f(p,q) = qp^{-1}$ となります。

そこで、最初のいくつかの項を書き出してみましょう。すると次のようになります。

- $a_1 = p$
- $a_2 = q$
- $a_3 = qp^{-1}$
- $a_4 = qp^{-1}q^{-1}$
- $a_5 = qp^{-1}q^{-1}pq^{-1}$
- $a_6 = qp^{-1}q^{-1}p^2q^{-1}$
- $a_7 = qp^{-1}q^{-1}pqpq^{-1}$
- $a_8 = qp^{-1}q^{-1}pqp^{-1}qpq^{-1}$
- ...

根気よく実験を続けることが重要です。書き下してみると、中央で同じ項がたくさん消えることが分かります。そこで、$a_i = A_iB_iA_i^{-1}$ という形で書き換えることを考えると、以下のようになります。

- $(A_1, B_1) = (id, p)$
- $(A_2, B_2) = (id, q)$
- $(A_3, B_3) = (id, qp^{-1})$
- $(A_4, B_4) = (q, p^{-1})$
- $(A_5, B_5) = (qp^{-1}, q^{-1})$
- $(A_6, B_6) = (qp^{-1}, q^{-1}p)$
- $(A_7, B_7) = (qp^{-1}q^{-1}p, p)$
- $(A_8, B_8) = (qp^{-1}q^{-1}p, q)$
- ...

$(A_7, B_7) = (qp^{-1}q^{-1}, p)$ とも書けますが、そうすると周期が見えづらくなってしまうので注意しましょう。これを見ると周期 6 を持つことが分かります。実際、(A, p), (A, q) から始めて 6 回進むと $(Aqp^{-1}q^{-1}p, p)$, $(Aqp^{-1}q^{-1}p, q)$ となるので、$qp^{-1}q^{-1}p$ を $\left[\frac{K-1}{6}\right]$ 回合成すれば A_K は求まります。$\{B_i\}$ は周期 6 であるので、適当にシミュレーションをすることで B_K は求まります。よって、この問題を $O(N)$ で解くことができました。

E: Snuke the Phantom Thief

「ちょうど K 個宝石を盗んでください」という条件を足すと一気に見通しがよくなるのがこの問題のキモです。

まずは 1 次元の場合、つまり $t_i = L, R$ の条件のみの場合を考えます。

「盗んだ宝石のうち、x 座標が a_i 以下の宝石が b_i 個以下」は、「盗んだ宝石を x 座標で sort した時、$b_i + 1$ 番目 (1-indexed) の宝石の x 座標が $a_i + 1$ 以上」と同値です。

そして盗む個数を決めると、「盗んだ宝石のうち、x 座標が a_i 以上の宝石が b_i 個以下」は、「盗んだ宝石を x 座標で sort した時、$K - b_i$ 番目の宝石の x 座標が $a_i - 1$ 未満」と同値となります。

要するに盗む宝石の条件は、ある整数 $L_1, L_2, ..., L_K$、$R_1, R_2, ..., R_K$ によって、「盗んだ宝石を x 座標で sort した時、i 番目の宝石の x 座標が L_i 以上 R_i 以下」と表せます。

明らかに $L_1 \leq L_2 \leq ... \leq L_K$, $R_1 \leq R_2 \leq ... \leq R_K$ として良いです。そうでないならば、つまり $L_i > L_{i+1}$ ならば $L_{i+1} = L_i$ として良いからです。

更に実はこの条件は、「各 i について、L_i 以上 R_i 以下から宝石を 1 個盗む」と言い換えられます。(ここの証明いりますか)

ここまで言い換えられれば、割り当て問題、つまり最小費用流に帰着することができます。1(始点) + K(x 座標制約) + N(宝石) + 1(終点) 頂点のグラフを作り、

- 始点から (x 座標制約) の頂点それぞれに重さ 0, 容量 1
- (x 座標制約) の i 番目の頂点からは、x 座標が $L_i \sim R_i$ の (宝石) それぞれに重さ 0, 容量 1
- (宝石) それぞれから終点に重さ=-(宝石の価値), 容量 1

の辺を貼り、このグラフで流量 K 流したものが求める答えです。

2 次元への対応も簡単です。グラフが 2 個できるので、片方をひっくり返して繋げれば良いです、つまり 1(始点) + K(x 座標制約) + N(宝石 x) + N(宝石 y) + K(y 座標制約) + 1(終点) のグラフを作り、

- 始点から (x 座標制約) にそれぞれ重さ 0, 容量 1
- (x 座標制約) の i 番目からは x 座標が $L_i \sim R_i$ の宝石に対応する (宝石 x) の頂点へ重さ 0, 容量 1
- (宝石 x) から (宝石 y) へは 重さ=-(宝石の価値), 容量 1
- 始点から (x 座標制約) にそれぞれ重さ 0, 容量 1
- (y 座標制約) の i 番目へ、 y 座標が $L_i \sim R_i$ の宝石に対応する (宝石 y) の頂点から重さ 0, 容量 1
- (y 座標制約) から終点へそれぞれ重さ 0, 容量 1

の辺を貼り、K 流せば良いです。負の重みの辺は、重みを 10^{15} 足しておき、最後に答えから $K \times 10^15$ 引けば非負の重みと見做すことができます。

計算量は V $= O(N)$, E $= O(N^2)$ のグラフへ容量 $O(N)$ 流すのを $O(N)$ 回行うので、$O(N^4)$ です。最短路を求める時に $O(ElogV)$ のダイクストラ法を使うと $O(N^4 logN)$ となりますが、これでも十分余裕を持って通るように調整したつもりです。

F: Walk on Graph

mod MOD における 2 の位数を ord と置きます。

今頂点 v に居て、ここまで通ってきた辺の本数の総数が mod ord で k であるような状態を (v,k) と書くことにします。

すると、与えられたグラフ上で頂点 s と頂点 t を結ぶ長さ L の辺があったとすると、(s,k) から $(t,k+1)$ へ長さ $2^k \times L$ で行くことが出来る、と思うことができます。

ここで、 (s,k) からスタートして、この辺を往復して頂点 s に戻ることを $ord \times MOD$ 回繰り返すと、再び (s,k) に戻り、かつ経路の長さの mod MOD における値も変化しないことが分かります。

よって、これは $(t,k+1)$ から (s,k) へ長さ $-2^k \times L$ で行くことができるということを示しています。

以下、ある頂点 s から長さ x と長さ y の辺が出ている場合に、ちょうど $3(x-y)$ だけ経路の長さを増やすことができることを示します。長さ x の辺は t 、長さ y の辺は u へのびていることにすると、

- (s,k) , $(t,k+1)$, $(s,k+2)$ と往復するときには $2^k \times x + 2^{k+1} \times x$ だけ経路の長さが増えます。
- (s,k) , $(u,k+1)$, $(s,k+2)$ と往復するときには $2^k \times y + 2^{k+1} \times y$ だけ経路の長さが増え、以上の議論より、これを逆にすると $(s,k+2)$ から (s,k) に $-2^k \times y - 2^{k+1} \times y$ だけ経路の長さを増やしていくことができます。

よって、結局 $3 \cdot 2^k \cdot (x-y)$ だけ経路の長さを増やして (s,k) に戻ってくることが出来ることがわかり、MOD は奇数なので、この動きを繰り返すことで $3(x-y)$ だけ経路の長さを増やすことができることが示せます。

さて、今は頂点 s に繋がる 2 辺について示しましたが、これを繰り返して適用することで、同じ連結成分に属する任意の 2 辺の差分に 3 を掛けた値だけ経路の長さを増やせることが分かります。すると、最終的に元の MOD と、与えられた全ての辺の値が mod M で等しいような最大の M (辺が 1 本しかない場合は $M = MOD$ とします) を取ってきて、mod $\gcd(MOD, 3M)$ で元の問題を解けば良い、ということがわかります。

よって、辺の長さを mod $3M$ で見ることができるので、 $aM + c$ と書くと、全ての辺に対して c は等しく、 a は $0,1,2$ のいずれかになります。以下、長さは全て $pM + q$ の形で表現します。 (p は常に $0,1,2$ のいずれかですが、q は M 以上になり得ます)

ここで、元の問題を後ろから見ることにすると、各クエリを以下のように言い換えることができます:

- 頂点 t から頂点 s までの経路で、長さが mod MOD で r になるようなものが存在するかしないか求めてください。ただし、経路の長さが a の状態で長さ L の辺を通ると経路の長さは $2a + L$ になります。

この問題は以下のように解くことができます:

- まず、今の経路の長さに c を足します。(最初の経路の長さは c であり、最終的に長さが mod MOD で $r+c$ であるような経路を探す問題になります。) (以下、c を足したものを経路の長さと呼びます。)
- こうすることで、現在の経路の長さが $pM+q$ の時に、長さ $aM+c$ の辺を通ることにすると、経路の長さは $(2p+a)M+2q$ になります。
- 最初の経路の長さは $0 \cdot M + c$ であると思うことができます。
- 経路の長さが $pM+q$ の時に長さ $aM+c$ の辺を往復することにすると、経路の長さは $(4p+3a)M+4q$、mod $3M$ で見ると $pM+4q$ になります。よって、 q は自由に 4 倍することができます。
- ここで、(v, p, par) を頂点 v にいて、経路の長さの M の係数が p であり、ここまで通ってきた辺の本数の mod 2 が par であるような状態とおき、互いに到達できるペアを Union-Find 上で繋ぎます。
- すると、$(t, 0, 0)$ と (s, p, par) が同じ連結成分に所属し、$pM + 2^{2N+par} \times c$ と $r+c$ が mod MOD で等しくなるような非負整数 N が存在するかが分かれば良い、ということになります。
- これは、結局 $2^x \times c$ が mod MOD で 何らかの値 num に合同になり、かつ mod 2 が par になるような x が存在するかがわかればよく、これは最初に高々 MOD 回ループを回すことで判定できます。

以上より、この問題を全体で $\mathcal{O}(MOD + (n+m+q)\ \alpha(n))$ 時間で解くことができました。

AGC032 Editorial

writer : many

March 16, 2019

A : Colorful Subsequence

Since we distinguish strings whose contents are the same but made from different positions in the original string, all $2^N - 1$ subsequences will be considered as different.

From the condition, we cannot use the same letter twice. Thus, for a letter c, we have (the number of occurrences of c) + 1 choices. (We either take one of those occurrences, or nothing.)

The answer is the product of these numbers for all letters c, minus 1 for the empty string.

B : Reversi

First, we can see that, in a sequence that can be made, the color of the stones at the both ends of a maximal contiguous subsequence of stones of the same color have not changed. (If the color of such stone has been changed, the two stones adjacent to that stone must both have the same color as that stone.) On the other hand, we can always make a sequence that satisfies above, by performing the operation for both ends of each maximal contiguous subsequence of stones of the same color. Thus, we can count the number of sequences satisfying this condition.

Let DP[i] be the number of ways of coloring the first i stones satisfying the condition. DP[i] can be computed as the sum of DP[j] such that the color of the i-th stone is the same as the $(j+1)$-th stone but different from the j-th stone (for convenience, assume that the 0-th stone is painted in the color different from all other stones).

This is not fast enough, since the time complexity is O(N^2). Now, for each color, let us maintain the sum of DP[j] such that the color of the i-th stone is the same as the $(j+1)$-th stone but different from the j-th stone. Every time we compute one value in the DP table, we add the result to the sum for that color. We can now find one value in the DP table in O(1) time, for a total of O(N) time.

C : Differ by 1 Bit

From the condition that P_i and P_{i+1} differ by exactly 1 bit, we can know the parity of the number of standing bits in P_i. Particularly, P_0 and P_{2^N-1} have different parities of the number of standing bits. Thus, if A and B have the same parity of the number of standing bits, the construction is impossible.

We will now show that, if A and B have different parities of the number of standing bits, the construction is always possible. The proof is done by induction. The construction is obviously possible when $N = 1$. Then, let us assume that the construction is always possible when $N = K$, for some K. Now, let us consider the case when $N = K + 1$. There is at least one bit position we can choose where A and B differ. Let this position be the x-th bit (from the right). Let A', B' be respectively $(N-1)$-bit numbers obtained by removing the x-th bit from A, B. Obviously, A' and B' have the same parity of the number of standing bits. Here, we can take a $(N-1)$-bit number C whose parity of the number of standing bits is different from that of A' and B'. (We can, for example, swap the first bit in A'.) Now, from the inductive assumption, we can take a permutation Q of length 2^{N-1} beginning with A' and ending with C, and a permutation R of length 2^{N-1} beginning with C and ending with B'. Then, we can construct a permutation R that satisfies the condition, as follows:

- $i < 2^{N-1}$: The x-th bit of P_i is the same as the x-th bit of A. The remaining $(N-1)$ bits are the same as the corresponding bits in Q_i.
- $2^{N-1} \leq i < 2^N$: The x-th bit of P_i is the same as the x-th bit of B. The remaining $(N-1)$ bits are the same as the corresponding bits in $R_{i-2^{N-1}}$.

We can easily verify that this permutation P satisfies the condition.

Therefore, it has been proved that the construction is always possible, and we can actually do it by implementing the procedure above. The time complexity is $\mathrm{O}(N^2 2^N)$ or $\mathrm{O}(N2^N)$, depending on implementation.

D : A Sequence of Permutations

For permutation p and q, let pq denote their composition. That is, pq is a permutation whose i-th element is p_{q_i}. With this notation, $f(p,q) = qp^{-1}$.

Now, let us write out first several elements of our sequence. We get the following:

- $a_1 = p$
- $a_2 = q$
- $a_3 = qp^{-1}$
- $a_4 = qp^{-1}q^{-1}$
- $a_5 = qp^{-1}q^{-1}pq^{-1}$
- $a_6 = qp^{-1}q^{-1}p^2q^{-1}$
- $a_7 = qp^{-1}q^{-1}pqpq^{-1}$
- $a_8 = qp^{-1}q^{-1}pqp^{-1}qpq^{-1}$
- ...

Patience is important in this experiment. From this, we can see the same element disappearing many times. Now, let us rewrite it as $a_i = A_i B_i A_i^{-1}$. We get the following:

- $(A_1, B_1) = (id, p)$
- $(A_2, B_2) = (id, q)$
- $(A_3, B_3) = (id, qp^{-1})$
- $(A_4, B_4) = (q, p^{-1})$
- $(A_5, B_5) = (qp^{-1}, q^{-1})$
- $(A_6, B_6) = (qp^{-1}, q^{-1}p)$
- $(A_7, B_7) = (qp^{-1}q^{-1}p, p)$
- $(A_8, B_8) = (qp^{-1}q^{-1}p, q)$
- ...

Note that we could also say $(A_7, B_7) = (qp^{-1}q^{-1}, p)$, but that would make the period less noticeable. Now, it can be seen that we have a period of 6. In fact, six iterations from (A, p) and (A, q) result in $(Aqp^{-1}q^{-1}p, p)$, $(Aqp^{-1}q^{-1}p, q)$, so we can find A_K as a $\left[\frac{K-1}{6}\right]$-fold product of $qp^{-1}q^{-1}p$. For $\{B_i\}$, it has a period of 6, so we can find B_K by simulation. Thus, the problem is solved in $\mathrm{O}(N)$ time.

E: Snuke the Phantom Thief

The key is to add one more condition: "Snuke must steal exactly K jewels", which makes the problem much more approachable.

First, let us consider the one-dimensional version of the problem, that is, the version where there are only conditions with $t_i = L, R$.

The condition "Snuke can only steal at most b_i jewels whose x-coordinate is a_i or smaller" is equivalent to: "when the stolen jewels are sorted in ascending order of x-coordinate, the x-coordinate of the (b_i+1)-th jewel (1-indexed) must be $a_i + 1$ or larger".

Additionally, if we fix the number of jewels to steal, the condition "Snuke can only steal at most b_i jewels whose x-coordinate is a_i or larger" is equivalent to: "when the stolen jewels are sorted in ascending order of x-coordinate, the x-coordinate of the $(K - b_i)$-th jewel must be smaller than $a_i - 1$".

To sum it up, the conditions on jewels to steal can be represented as "when the stolen jewels are sorted in ascending order of x-coordinate, the x-coordinate of the i-th jewel must be between L_i and R_i (inclusive)" by some integers $L_1, L_2, ..., L_K$ and $R_1, R_2, ..., R_K$.

Obviously, we can assume $L_1 \leq L_2 \leq ... \leq L_K$ and $R_1 \leq R_2 \leq ... \leq R_K$. This is because, otherwise, that is, if $L_i > L_{i+1}$, it can be regarded as $L_{i+1} = L_i$.

In fact, we can further rephrase the condition as "for each i, Snuke must steal one jewel between the x-coordinates L_i and R_i". (The proof is left to the reader.)

Now, we can reduce the problem to the allocation problem, that is, the minimum-cost flow. Let us build a graph with 1(source) + K(constraints on x-coordinate) + N(jewels) + 1(sink) vertices and the following edges:

- From the sink to each of the x-coordinate constraint vertices: an edge of cost 0 and capacity 1
- From the i-th x-coordinate constraint vertex to each jewel vertex with x-coordinate between L_i and R_i: an edge of cost 0 and capacity 1
- From each jewel vertex to the sink: an edge of cost -(the jewel's value) and capacity 1

The answer can be found by sending a flow of K through this graph.

This can be easily extended to the two-dimensional version. Now we have two graphs, but we can flip one of them and connect it to the other.

That is, we build a graph with 1(source) + K(constraints on x-coordinate) + N(jewels x) + N(jewels y) + K(constraints on y-coordinate) + 1(sink) vertices and the following edges:

- From the sink to each of the x-coordinate constraint vertices: an edge of cost 0 and capacity 1
- From the i-th x-coordinate constraint vertex to each "jewel x" vertex with x-coordinate between L_i and R_i: an edge of cost 0 and capacity 1
- From "jewel x" vertex to "jewel y" vertex: an edge of cost -(the jewel's value) and capacity 1
- To the i-th y-coordinate constraint vertex from each "jewel y" vertex with y-coordinate between L_i and R_i: an edge of cost 0 and capacity 1

- From each y-coordinate constraint vertex to the sink: an edge of cost 0 and capacity 1

We can then send a flow of K through this graph. For edges with negative costs, we can add 10^{15} to those costs in advance to make them non-negative and subtract $K \times 10^{15}$ from the answer.

The time complexity is $\mathrm{O}(N^4)$, since we send a flow of $O(N)$ through a graph with $V = \mathrm{O}(N), E = \mathrm{O}(N^2)$, and we repeat it $\mathrm{O}(N)$ times. It becomes $\mathrm{O}(N^4 \log N)$ if you use Dijkstra's algorithm, which takes $\mathrm{O}(E \log V)$ time, to find the shortest paths, but we tried to set the constraints so that even this runs fast enough easily.

F: Walk on Graph

Let's see the path in the reverse order. The query asks the following:

- A person is standing at T with a black integer 0. When a person with a black integer x passes through an edge of length c, the black integer changes to $2x+c \bmod MOD$. Can the person reach S with a black integer R?

Let $[v, x]$ denote the state such that the person is at vertex v and the black integer is x. Let's make various observations:

- Suppose that there is an edge of length c between two vertices a and b. If we start with the state $[a, x]$ and repeat traversing this edge back and forth ($[a, x] \to [b, 2x + c] \to [a, 4x + 3c] \to [b, 8x+7c] \to \cdots$), we will eventually return to the original state $[a, x]$ because the map $x \mapsto 2x+c$ is bijective. This means that $[a, x]$ is reachable from $[b, 2x+c]$. Thus, the reachablity is bidirectional, and we can divide the states into connected components (as an undirected graph). We can answer the queries by checking if the two states $[t, 0]$ and $[s, r]$ are in the same component.
- Suppose that a vertex v is incident to two edges of lengths a and b. For an arbitrary integer x, $[v, x]$ and $[v, 4x + 3a]$ are equivalent (by traversing the edge of length a twice), and also $[v, x]$ and $[v, 4x + 3b]$ are equivalent. This means that we can freely add $3(a - b)$ to the black integer at this vertex.
- Let g be the maximum integer such that g is a divisor of MOD, and all edge lengths are equivalent modulo g. By combining the observation above, we only need to care the value of the black integer modulo $gcd(3g, MOD)$.

Therefore, we can assume that MOD is either g or $3g$, and all edge lengths are equivalent to z modulo g (for some integer z).

Let's define red integer as black integer plus z. Also, let's decrease the lengths of all edges by z. Now, when we pass through an edge of length c, the red integer x changes to $2(x - z) + (c + z) + z = 2x + c$. Thus, we can assume that MOD is either g or $3g$, and all edge lengths are divisible by g.

How to divide the states into connected components in this case? Let's fix a certain vertex as the root. All states reachable from $[root, X]$ can be written of the form $[v, pX + qg]$, where v is some vertex, p is a power of two and q is one of $0, 1, 2$. Notice that now the states $[v, x]$ and $[v, 4x]$ are always equivalent (we can reach $[v, x] \to [u, 2x + c] \to [v, 4x + 3c] = [v, 4x]$ using some edge incident to it). Thus, we can limit the values of p to one of $1, 2$.

We summarize the solution to the problem:

- Let's construct DSU with $6N$ states. Each state represents the state $[v, pX + qg]$, where v is some vertex, and $1 \le p \le 2, 0 \le q \le 2$.
- For each edge (a, b, c) and valid pair (p, q), connect the states $[a, pX + qg]$ and $[b, 2(pX + qg) + c]$. (In case this state doesn't exist among the $6N$ states, convert it using the "four times" rule.)

- To answer queries, we check if $[s, r+z]$ and $[t, z]$ are equivalent. This is "YES" if $[t, X]$ and $[s, pX+qg]$ are equivalent for some (p, q), and $(pZ+qg)/(r+z)$ is a power of four in modulo MOD.

This solution works in $\mathcal{O}(MOD + (n+m+q)\ \alpha(n))$ time.

AtCoder Grand Contest 030 解説

writer: semiexp, sugim48, DEGwer

2018 年 12 月 29 日

For International Readers: English editorial starts on page 7.

A: Poisonous Cookies

食べることのできる毒入りのクッキーの枚数は、解毒剤入りのクッキーの枚数に 1 を足したもの以下です。すなわち、$A + B + 1 \geq C$ ならすべての毒入りのクッキーを食べることができるので答えは $B + C$ であり、そうでないなら答えは $B + (A + B + 1)$ です。

B: Tree Burning

高橋君は最初に反時計回りに進み始めるとして一般性を失いません (最初に進む向きを両方ためし、出てきた解のうち大きい方を出力すればよいです)。

高橋君が初めて時計回りに進んで以降、高橋君はまだ燃やしていない木に到達するごとに進む向きを変えるとして良いことが以下のように証明できます: もしある木 i を燃やした後進む向きを変えないのならば、その後方にある木を燃やしたときに木 i の座標まで行って木 i を燃やしていたことにすれば、進む距離の合計を増やすことができます。

さて、以上のような行動の方法は、最初に向きを変える位置を決めることで定まるので、$O(N)$ 通りしかありません。最初に向きを変える位置それぞれに対して高橋君の移動距離を求められればこの問題を解くことができ、これは累積和を使うか、隣り合う位置についての移動距離の差分を考えることで $O(N)$ 時間で動くように実装できます。

C: Coloring Torus

具体的な構成を示します．便宜上，色を $0, 1, \ldots, K-1$ で表すことにします．まず，n が偶数のとき，$K = 2n$ に対して次のような構成があります：

- $r \equiv 0 \pmod 2$ のとき，マス (r, c) を色 $(r + c) \bmod n$ で塗る．
- $r \equiv 1 \pmod 2$ のとき，マス (r, c) を色 $n + ((r + c) \bmod n)$ で塗る．

この塗り方が K 色でのよい塗り方であることはすぐに確かめられます．

ここで，任意の i $(0 \leq i \leq n-1)$ について，この塗り方で色 $i + n$ で塗られているマスをすべて色 i に塗り替えても，(K 色のうちすべての色が塗られているという条件を除いて) よい塗り方であることがわかります．このような塗り替えを k $(0 \leq k \leq n)$ 色に対して行って，適宜色の番号を取り替えることで，$2n - k$ 色での塗り方が得られます．よって，K が一般の $2 \leq K \leq 1000$ の場合にも，$n = 2\lceil \frac{K}{4} \rceil$ とすると，このようにして $n \leq 500$ なるよい塗り方を得ることができます．$K = 1$ の場合はこの方法ではうまくいきませんが，この場合の構成は明らかです．

D: Inversion Sum

各操作を指定した 2 つの要素を確率 1/2 で入れ替える操作だと考えることにすれば、求める値は最終的な数列の反転数の期待値の 2^Q 倍です。期待値の線形性より、最終的に $A_i > A_j$ となる確率をすべての $i < j$ について求めて足し合わせれば良いです。

DP$[t][i][j]$ を、最初の t 回の操作をしたときに $A_i > A_j$ となる確率とします。DP$[0][i][j]$ の値はすべて、最初の数列に関する情報から求めることができます。また、DP$[t+1][i][j]$ の値が DP$[t][i][j]$ の値と異なるのは、i または j の一方以上の位置を変える可能性のある操作をする場合だけであり、このような (i, j) の組は $O(N)$ 個です。以上より、同じ配列を使いまわすことで $O(N)$ 時間でひとつの操作に対する DP 配列の更新ができることが分かり、よって $O(N^2 + NQ)$ 時間でこの問題を解くことができました。

E: Less than 3

文字列の 0 と 1 の間に赤い線を，1 と 0 の間に青い線を書くと，線たちは操作によって図 1 のように変化します．これを観察すると，赤い線と青い線が常に交互に並んでいることが分かります．また，各操作によって，端以外では線を距離 1 だけ動かすことができ，端では線の追加・削除ができることが分かります．

端での挙動を扱いやすくするために，両端に無限個の線が赤青交互に重なっていると考えます．すると，線たちは操作によって図 2 のように変化します．これを観察すると，次のことが分かります：

- 赤い線と青い線が常に交互に並んでいる．
- 端以外で線どうしが重なってはならない．
- 隣り合う線どうしの間隔は 2 以下である．
- 各操作によって，線を距離 1 だけ動かすことができる．

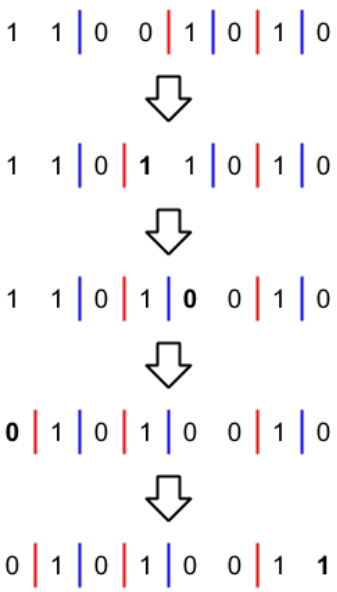


図 1

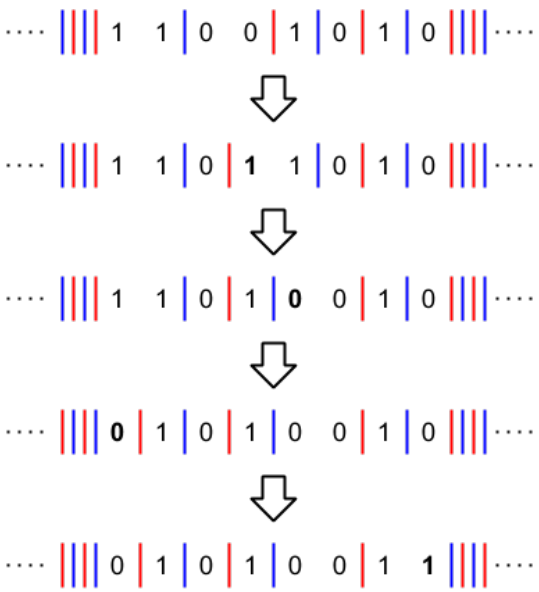


図 2

図 3 のように，s と t の間で線たちの対応が定まっているとします．すると，必要な操作回数の最小値は，対応する線どうしの距離の総和となることが容易に示せます．図 3 の例では，$\cdots+0+0+0+1+0+1+2+3+2+1+0+\cdots=10$ です．線たちの対応の定め方は $\mathrm{O}(N)$ 通りしかありません．なぜならば，あまりに遠くの線どうしを対応させるのは無駄だからです．具体的には，左端の線と右端の線を対応させるような定め方は (ほとんどの場合) 無駄です．(ただし，$s=$ 00, $t=$ 11 などの例外に注意．) よって，線たちの対応を $\mathrm{O}(N)$ 通り全探索することで，$\mathrm{O}(N^2)$ 時間で問題を解くことができます．

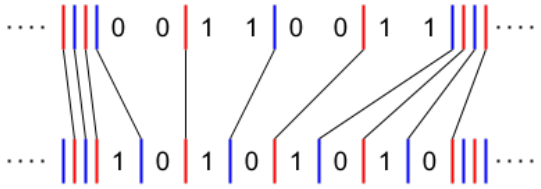


図 3

F: Permutation and Minimum

まず，A_{2i-1} および A_{2i} がともに -1 でないような i は存在しないとして構いません．なぜなら，このような i に対しては B_i は一定なので，数列から A_{2i-1}, A_{2i} を取り除いて，書き込む数も $1, 2, \ldots, 2N$ から A_{2i-1}, A_{2i} を除いた $2N-2$ 個の数としたものに対して場合の数を考えても変わらないためです．

異なる $\{B_i\}$ の個数にのみ興味があるため，一般性を失わず，次の仮定をしてかまいません：

- $A_{2i} = -1$．(さもなくば，A_{2i-1} と A_{2i} を入れ替えてかまいません)
- $A_{2i-1} = A_{2i} = -1$ ならば，A_{2i-1} に書き込まれる整数は A_{2i} に書き込まれる整数より大きい．
- $A_{2i-1} = A_{2i} = -1$ なる i の集合を $S = \{x_1, \ldots, x_k\}$ $(x_1 < \ldots < x_k)$ とする．このとき，$A_{2x_1}, \ldots, A_{2x_k}$ に書き込まれる整数はこの順に降順である．

最後の条件を導入することで，$\{B_i\}$ の個数は変わってしまいますが，最後に $|S|!$ を掛ければ正しい答えが得られます．

整数を $2N, 2N-1, \ldots, 1$ の順に，$\{B_i\}$ 中のどこに現れるか（あるいは，どこにも現れないか）を決定していくことを考えます．整数 n の $\{B_i\}$ 中での出現には次の可能性があります (それぞれに $a(n)$ などの名前をつけておきます)：

- $a(n)$: n は $\{B_i\}$ 中のどこにも現れない．また，$\{A_i\}$ 中にも現れない．
- $a'(n)$: n は $\{B_i\}$ 中のどこにも現れない．一方，$\{A_i\}$ 中に現れる．
- $b(n)$: $A_{2i-1} = A_{2i} = -1$ なる i に対して，B_i に現れる．
- $c_p(n)$: $A_{2i-1} = p$ なる i に対して，B_i に現れる $(p > n)$.
- $d(n)$: $A_{2i-1} \neq -1$ なる i に対して，B_i に現れる．このとき，$A_{2i-1} = n$.

$\{B_i\}$ を，それに対応する各整数の出現の仕方の列で表すことにします．例えば，入力例 1 の $(B_1, B_2, B_3) = (1, 3, 2)$ に対しては，$a(6) \to a(5) \to a(4) \to d(3) \to b(2) \to d(1)$ となります．このとき，異なる $\{B_i\}$ からは異なる列が得られます．よって，正しい $\{B_i\}$ に対応するような異なる列の個数を数えれば答えが得られます．

どのような列が正しい $\{B_i\}$ に対応するかを考えます．まず，$\{B_i\}$ 中に出現する整数は N 個なので，$a(n), a'(n)$ は合わせてちょうど N 回現れる必要があります．次に，n が $\{A_i\}$ 中に現れるなら，n の出現の仕方は $a'(n)$ か $c_p(n)$ のいずれかです．また，n が $\{A_i\}$ 中に現れないなら，n の出現の仕方は $a(n), b(n), d(n)$ のいずれかです．最後に，$b(n), c_p(n), d(n)$ は，それを実現する $\{A_i\}$ への整数の書き方において，n より大きい整数とペアになっている必要があります．すなわち，ある $n' > n$ が存在して，$b(n), d(n)$ は $a(n')$ と，$c_p(n)$ は $a'(p)$ とペアになっている必要があります．(同じ整数が複数のペアに所属することはできません．) 逆に，これらの条件を満たしていれば，そのような列は正しい $\{B_i\}$ に対応することがわかります．

ここで，列が与えられたとき，ペアの構成が可能かどうかは，$n = 2N, 2N-1, \ldots, 1$ の順に，$b(n), c_p(n), d(n)$ が現れたときは今までに現れた，今までにペアになっていない $a(n), a'(p)$ を選んでペアを組むということを行えば良いことがわかります．

関数 $f(n, x, Y)$ を，次の条件を満たす，$2N$ から $n+1$ までの数の出現の列の個数として定めます：

- ペアになっていない $a(i)$ の個数は x.
- $n+1 \leq i \leq n$ であって，ペアになっていない $a'(i)$ の集合は Y に等しい．

$f(n, x, Y)$ で数えられている列に対して n を加えた際の列は次のようになります：

- n が $\{A_i\}$ 中に現れる場合：
  - $a'(n)$ を置いた場合：$f(n-1, x, Y \cup \{n\})$ で数えられる状態．(a)

- $d(n)$ を置いた場合：$f(n-1, x-1, Y)$ で数えられる状態 ($x \geq 1$ の場合に限る).

- n が $\{A_i\}$ 中に現れない場合：
  - $a(n)$ を置いた場合：$f(n-1, x+1, Y)$ で数えられる状態.
  - $b(n)$ を置いた場合：$f(n-1, x-1, Y)$ で数えられる状態 ($x \geq 1$ の場合に限る).
  - $c_p(n)$ を置いた場合：$f(n-1, x, Y \setminus \{p\})$ で数えられる状態 ($p \in Y$ の場合に限る). (b)

ここで, $g(n, x, y) = \sum_{|Y|=y} f(n, x, Y)$ とおくと, (a) は $(n, x, y) \to (n-1, x, y+1)$ の遷移, (b) は $(n, x, y) \to (n-1, x, y-1)$ への y 個の遷移とみなすことができます. よって, 各 g の値を順次動的計画法で計算することができます. n, x, y は $[0, 2N]$ の値をとるので, これは $O(N^3)$ で計算することができ, $g(0, 0, 0)$ が求める値です.

AtCoder Grand Contest 030 Editorial

writer: semiexp, sugim48, DEGwer

December 29th, 2018

A: Poisonous Cookies

The maximum number of poisonous cookies you can eat is $A + B + 1$. Thus, if $A + B + 1 \geq C$, you can eat all cookies and the answer is $B + C$. Otherwise, the answer is $B + (A + B + 1)$.

B: Tree Burning

For simplicity, suppose that Takahashi's home is tree 0 (and he has just finished burning it), and there are $N - 1$ other trees. Let's see how the path of Takahashi looks like.

Suppose that he first start moving right (we call counter-clockwise "right" and clockwise "left"). (He may start moving left, so don't forget to try both directions and output the maximum.) Then, each time he meets with a tree, he burns it. When he arrives at tree a, he may decide to change the direction and start moving left. Then, when he arrives at tree b for some $b < 0$, he may again decide to change the direction and start moving right.

Here, assume that whenever he moves right, the tree number increases, and whenever he moves left, the tree number decreases. For example, when he moves to the left from tree 0, he will meet tree -1 next (but it means tree $n - 1$.

Then, for example, in case he makes five turnings, his path looks like $0 \to c \to b \to d \to a \to e$, where $a < b < 0 < c < d < e$ and $e - a = N - 1$. Here, we can assume that $b - a = 0 - b = e - d = d - c = 1$ (otherwise we can get a longer path in an obvious way). Thus, once we fix the initial direction and the number of turnings, we can uniquely determine the path. Be careful that depending on the parity of number of turnings, the path looks slightly different.

Therefore, there are only $O(N)$ paths we are interested in. With proper pre-computations of prefix sums, the distance of each path can be computed in $O(1)$, so this solution works in $O(N)$ time.

C: Coloring Torus

The following construction works. For simplicity, we represent colors as $0, 1, \ldots, K-1$.

In case n is even, for $K = 2n$, the following grid is valid:

- If $r \equiv 0 \pmod 2$, color the cell (r, c) with color $(r + c) \bmod n$.
- If $r \equiv 1 \pmod 2$, color the cell (r, c) with color $n + ((r + c) \bmod n)$.

It's easy to verify that this is a valid coloring.

Here, notice that for any i $(0 \leq i \leq n-1)$, even if you repaint all cells of color $i + n$ to color i, the grid is still valid (except for the condition that all colors must be used). If you do this operation for k colors $(0 \leq k \leq n)$, you get a valid grid with $2n - k$ colors.

For general K satisfying $2 \leq K \leq 1000$, the construction above works for $n = 2\lceil \frac{K}{4} \rceil$ (and $n \leq 500$ will be satisfied). The case with $K = 1$ is trivial.

D: Inversion Sum

Instead of computing the sum, let's assume that we choose each operation with probability $1/2$ and compute the expected number of inversions in the final array. The answer is 2^Q times this value.

Define $DP[t][i][j]$ as the probability that after t operations, $A_i > A_j$ holds. We can get the values of $DP[0][i][j]$ from the information of the initial array. Then, if you know the values of $DP[t][i][j]$ for all pairs of (i, j), you can get the values of $DP[t+1][i][j]$ for all pairs of (i, j).

This operation looks like $O(N^2)$ per each t, but notice that the value of $DP[t+1][i][j]$ and $DP[t][i][j]$ differs only when at least one of i or j is involved in the $(t+1)$-th swap. There are only $O(N)$ such pairs. Thus, if you reuse the array and update only necessary parts, this update can be done in $O(N)$ per each t.

Because of the linearity of expected values, the expected number of inversions in the final array is the sum of $DP[Q][i][j]$ for all $i < j$. This solution works in $O(N^2 + NQ)$ time.

E: Less than 3

Suppose that you perform an operation on the i-th character, and this character is not at the end of the string. Then, in order for the operation to be valid, the $(i-1)$-th character and the $(i+1)$-th character must differ (otherwise we get a row of three either before or after the operation). This means that unless the operation happens at the end of the string, the number of runs in the string never changes.

Let's draw a red vertical line between `0` and `1`, and a blue vertical line between `1` and `0`. Figure 1 shows an example of the operation with red and blue lines. The red and blue lines appear alternately, and in each operation one of the lines is moved by a unit distance.

Also, in order to handle operations at the ends, assume that there are infinite number of lines at both ends of the string, and red and blue appear alternately. See figure 2.

Now, in terms of red and blue lines, an opeation can be described as follows:

- Red and blue lines appear alternately. You can never change the relative order of these lines.
- Except for the ends, no two line must appear in the same position (i.e., between the same pair of characters).
- The distance between adjacent lines must be either 1 or 2 (except for the ends).
- In each operation, you can choose a line and move it by a unit distance, while keeping the conditions above.

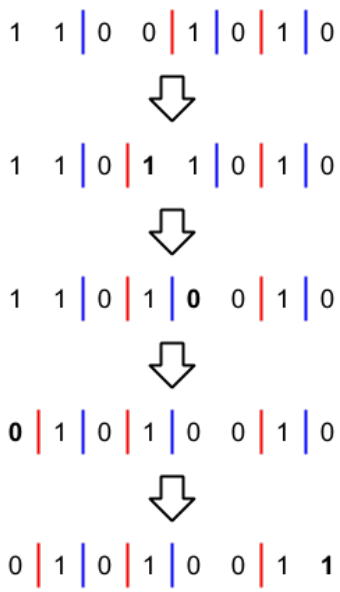


Figure 1:

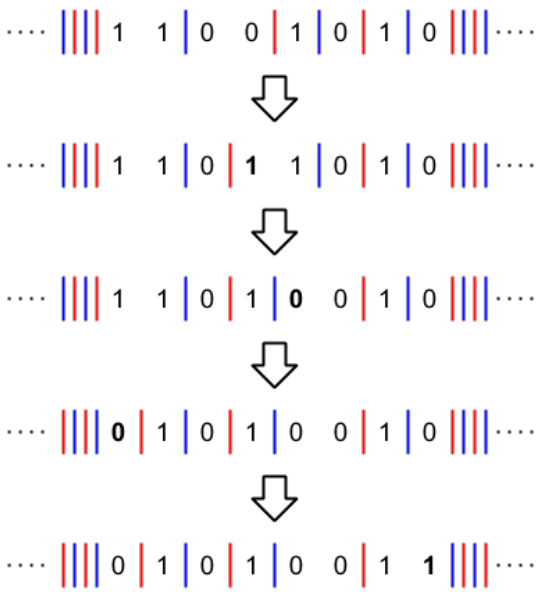


Figure 2:

As in figure 3, suppose that we know which line in s corresponds to which line in t. Clearly, the lower bound of the number of operations required to do this is the sum of distances between each corresponding lines. For example, in the example of 3, $\cdots + 0 + 0 + 0 + 1 + 0 + 1 + 2 + 3 + 2 + 1 + 0 + \cdots = 10$.

It turns out that this lower bound is always achievable. There are three types of lines: moving right, moving left, or staying. Since the distance between two adjacent lines must not exceed 2, a line moving right and a line moving left can't be adjacent. Thus, the lines can be splitted into blocks of "stay, left, ..., left, stay" or "stay, right, ..., right, stay". In both cases it's easy to see that at least one line can be moved to the desired direction.

There only $\mathrm{O}(N)$ ways to decide which lines in s to assign which lines in t. In particular, it's never optimal to assign two or more lines at the left end in s to lines at the right end of t, or vice versa. (Be careful that it's possible that only one such line exists, like $s = \texttt{00}$, $t = \texttt{11}$.)

Therefore, we can try all $\mathrm{O}(N)$ possible ways to assign lines, and this soltuion works in $\mathrm{O}(N^2)$ time in total.

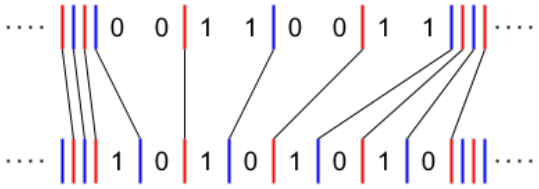


Figure 3:

F: Permutation and Minimum

Suppose there is an index i such that neither A_{2i-1} nor A_{2i} equals -1. Then, B_i is constant regardless of the replacement of A. Hence, we can remove A_{2i-1} and A_{2i} from $\{A_i\}$ and integers A_{2i-1} and A_{2i} from the set of integers to replace $\{A_i\}$, without changing the answer.

Since we are interested only in the number of different sequences $\{B_i\}$, we can make the following assumptions:

- $A_{2i} = -1$ (Otherwise, we can swap A_{2i-1} and A_{2i}).
- If $A_{2i-1} = A_{2i} = -1$ holds, the integer which replaces A_{2i-1} is greater than that which replaces A_{2i}.
- Let $S = \{x_1, \ldots, x_k\}$ $(x_1 < \ldots < x_k)$ be the set of indices i such that $A_{2i-1} = A_{2i} = -1$. Then, the integers which replace $A_{2x_1}, \ldots, A_{2x_k}$ are decreasing in this order.

Please note that, although introducing the last condition changes the number of $\{B_i\}$, we can multiply the number by $|S|!$ to obtain the correct answer.

We decide the position (or absence) of $2N, 2N-1, \ldots, 1$ in $\{B_i\}$, in this order. The following are the possibility of how n appears in $\{B_i\}$ (each possibility is named like $a(n)$):

- $a(n)$: n does not appear either in $\{B_i\}$ or $\{A_i\}$ (before replacing -1).
- $a'(n)$: n does not appear in $\{B_i\}$, but in $\{A_i\}$.
- $b(n)$: n equals B_i, where $A_{2i-1} = A_{2i} = -1$.
- $c_p(n)$: n equals B_i, where $A_{2i-1} = p$ $(p > n)$.
- $d(n)$: n equals B_i, where $A_{2i-1} = n$.

We represent $\{B_i\}$ using the corresponding sequence of appearances of $2N, 2N-1, \ldots, 1$. For example, $(B_1, B_2, B_3) = (1, 3, 2)$ for the Sample Input 1 can be converted to $a(6) \to a(5) \to a(4) \to d(3) \to b(2) \to d(1)$. Here, different $\{B_i\}$ yields different sequence of appearances. Therefore, we can count the number of sequences of appearances which correspond to *correct* $\{B_i\}$ (that is, $\{B_i\}$ which can be found from a valid permutation $\{A_i\}$).

We consider which sequences of appearances correspond to a correct $\{B_i\}$. First, $a(n)$ and $a'(n)$ must appear exactly N times in total, because the length of B is N. Second, if n is in $\{A_i\}$, the appearance of n should be either $a'(n)$ or $c_p(n)$. Similarly, if n is not in $\{A_i\}$, the appearance of n should be either $a(n), b(n)$, or $d(n)$. Finally, in a sequence $\{A_i\}$ which yields $\{B_i\}$, $b(n)$, $c_p(n)$ and $d(n)$ should be paired with an integer greater than n That is, $b(n)$ and $d(n)$ should be paired with $a(n')$ for some $n' > n$, and $c_p(n)$ should be paired with $a'(p)$. Here, an integer cannot belong to more than one pair. On the other hand, if a sequence of appearances satisfies all the conditions above, we can see that it corresponds to a correct $\{B_i\}$.

Given a sequence of apperances, we can check whether there is a valid pairing as follows: for $n = 2N, 2N-1, \ldots, 1$ in this order, if the appearance of n is either $b(n)$, $c_p(n)$, or $d(n)$, we pair it with $a(n')$ or $a'(p)$ which is yet to be paired (the possibility doesn't depend on the choice of n' when there exists multiple $a(n')$ which can be paired with).

We define a function $f(n, x, Y)$ as the number of sequences of appearances *of* $2N, 2N-1, \ldots, n+1$ which satisfies the following conditions:

- The number of unpaired $a(i)$ equals x.
- Y equals the set of $n+1 \leq i \leq n$ such that $a'(i)$ is unpaired.

For a sequence counted by $f(n, x, Y)$, we append an appearance of n at the end of the sequence, and if it is either $b(n)$, $c_p(n)$, or $d(n)$, we pair it with a preceding appearance. The resulting sequences are as follows:

- If n is in $\{A_i\}$:
  - if n appears as $a'(n)$, a sequence counted by $f(n-1, x, Y \cup \{n\})$. (a)
  - if n appears as $d(n)$, a sequence counted by $f(n-1, x-1, Y)$ (only when $x \geq 1$).

- If n is not in $\{A_i\}$:
  - if n appears as $a(n)$, a sequence counted by $f(n-1, x+1, Y)$.
  - if n appears as $b(n)$, a sequence counted by $f(n-1, x-1, Y)$.
  - if n appears as $c_p(n)$, a sequence counted by $f(n-1, x, Y \setminus \{p\})$ (only when $p \in Y$).

Here we let $g(n, x, y) = \sum_{|Y|=y} f(n, x, Y)$. Then, (a) can be seen as a transition from (n, x, y) to $(n-1, x, y+1)$, and (b) can be seen as y distinguishable transitions from (n, x, y) to $(n-1, x, y-1)$. Therefore, each value of g can be computed using dynamic programming. Since n, x and y takes a value in $[0, 2N]$, g can be computed in $O(N^3)$ time. Finally, $g(0, 0, 0)$ is the desired value.

AGC 029 解説

yutaka1999

2018 年 12 月 15 日

A : Irreversible operation

操作として隣合ったオセロの石をともに裏返すのではなくて、それらを裏表そのまま入れ替えると考えて問題ないことに注意します。このとき、各白石の移動した距離の総和が操作回数と一致します。ここで、操作によってもともと同じ色同士であった石の相対順序は入れ替わらず、さらに操作が行えなくなった状態では $W..WB..B$ のようになっていることに注目しましょう。そうすると各白石と相対順序が入れ替わった石の数は、それよりもはじめ左にあった黒石の数と一致します。よって、これ以上操作を行えない状態になったとき、行った操作の回数はその過程によらず $\sum_{S_i=W} \#\{1 \leq j < i | S_j = B\}$ であると分かります。よって、求める最大値は $\sum_{S_i=W} \#\{1 \leq j < i | S_j = B\}$ です。これは $O(N)$ の時間計算量で求めることができます。

B : Powers of two

まず、書いてある数が最大のボールを一つ考えましょう。それを i 番目のボールとします。このとき、i 番目のボールのペアとなるボール j があるとしたら、$A_i < A_i + A_j \leq 2A_i$ より $A_i + A_j$ の値は一意に定まります。(というのも、$A_i + A_j = 2^k$ となっていたとすると、$A_i < 2^k \leq 2A_i$ であるので k は一意に定まります。) よって、$A_i + A_j$ が 2 べきとなるような j があるとしたら、i 番目のボールと j 番目のボールをペアにしてしまってよいことが分かります。これは、ペアの数が最大となる組み方を一つとってきたときに、i 番目のボールがペアとして使用されていれば、それは j 番目のボールと同じ数が書いてあるので j 番目のボールとしてみなしてよく、使用されていない場合は、j 番目のボールが属しているペアを i, j 番目のボールのペアに置き換えてもペアの数が減らないことから示せます。よって、貪欲にペアを作っていって問題ないことが分かるので、シミュレーションをすることでこの問題を解くことができます。時間計算量のボトルネックは、A_i の内最大のものを取ってくるところと $A_i + A_j$ が 2 べきとなる j を探すところで、計算量は $O(N \log N)$ になります。

C : Lexicographic constraints

二分探索を行うことを考えます。つまり、文字を X 種類まで使えるとして、$S_1, ..., S_N$ を構成することができるかどうか考えます。ここで S_0 を空文字列として、各 $1 \leq i \leq N$ に対し S_{i-1} から最適な S_i を順に求めていくことを考えます。長さが A_i で S_{i-1} より辞書順で大きい文字列のうち、辞書順最小な文字列が最適だと分かるので、それを求めればよいです。文字を辞書順に $0, 1, ..., X-1$ とすると求めたい文字列は、$A_{i-1} < A_i$ のときは $A_{i-1}+1, ..., A_i$ 文字目を 0 として S_{i-1} を長さ A_i に拡張して得られる文字列、$A_{i-1} \geq A_i$ のときは S_{i-1} の $A_i + 1, .., A_{i-1}$ 文字目を削除して A_i 文字目を 1 つ大きい文字に置き換えて得られる文字列であることが分かります。(ただし、A_i 文字目が $X-1$ の場合、繰り上がりが生じるので適切に処理する必要があります。) 長さが 10^9 まであるため、実際の文字列の状態を管理するのが難しいように見えますが、ほとんどの文字が 0 であることを考えると、map や配列を適切に用いることで比較的容易に管理することができます。以上より、順に S_i を構成していくことができるので、確かに判定関数を作ることができることが分かります。(X 種類の文字種で構成することができないのは、繰り上がりの際にどの桁も追加できなくなった時です。) よって、$O(N \log N)$ (配列のみを用いると $O(N)$) の時間計算量で二分探索の判定関数を作ることができるので、全体

として $O(N \log N \log \max A)$ の時間計算量で解くことができます。

D : Grid game

高橋君が駒を動かさなかったとき、青木君は自分も駒を動かさずにゲームを終了させるのが最適です。よって、高橋君は動かせる限り駒を動かすことが最適です。このとき、高橋君の行う行動の回数は（高橋君が駒を動かした回数）+1 であるので、高橋君が駒を何回動かせるかを求められればよいです。青木君はゲームを早く終了させたいので、高橋君が駒を動かせなくなるマスにできるだけ早く動かすのが最適です。高橋君は動ける限り必ず動くので、これは青木君一人の選択のみによって定まるゲームとなります。つまり、青木君が駒を (x, y) から $(x+1, y)$ または $(x+1, y+1)$ のいずれかに動かすことができるとき、最小で何回の操作で高橋君が駒を動かせなくなるマスにたどり着けるかを求められればよいです。

$(x, y) \to (x-y, y)$ と座標変換することで、(x, y) から $(x+1, y)$ または $(x, y+1)$ のいずれかに動かせる場合に帰着できます。このとき、高橋君が駒を動かせなくなるマスは $(x+1, y)$ に障害物があるようなマス (x, y) です。（ただし、グリッドの外部にあるマスも障害物とみなします。）よって、そのようなマスの内、障害物にぶつからずにたどり着ける一番近いマスを求められればよいです。障害物にぶつからずにたどり着けるかどうかは、y 座標を小さい順にみていって、障害物にぶつからずにたどり着ける x 座標の範囲を持っておくことで十分高速に判定することができます。よって、一番近いたどり着けるマスを求めることができるので、以上よりこの問題を解くことができます。ボトルネックとなるのは障害物をソートして y 座標の昇順に並べるところであり、$O(N \log N)$ の計算量で解くことができます。

E : Wandering TKHS

便宜上 $c_1 = 1$ とおいて、1 を根として根付きにしたときの、各頂点 $v > 1$ の親を p_v とします。また、p_v から 1 までのパス上にある頂点の番号の最大を m_v とします。（ただし $p_1 = m_1 = -1$）ここで、$c_v - c_{p_v}$ の値を各 $v > 1$ に対して考えます。

$i) v > m_{p_v}$ のとき
$c_v - c_{p_v}$ は（v の子孫（v 含まず）のうち v から m_v 以下の頂点だけを通ってたどり着ける頂点の数）+1 となります。
$ii) v < m_{p_v}$ のとき
$c_v - c_{p_v}$ は v の子孫（v 含まず）のうち v から m_v 以下の頂点だけを通ってたどり着ける頂点の数から、v の子孫（v 含ない）のうち v から m_{p_v} 以下の頂点だけを通ってたどり着ける頂点の数となります。

よって、$Q(v, x) :=$ v の子孫（v 含まず）のうち v から x 以下の頂点だけを通ってたどり着ける頂点の数 が求められればよいことになります。これはデータ構造を使うことでも解けますが、今回は呼ばれるクエリが特殊なので簡潔に解くことができます。まず $Q(v, x) = \sum_c (1 + Q(c, x))$（$c$ は v の子でかつ $c < x$）であるので、この式に従って再帰的に求めることはできます。一般の場合は間に合いませんが、今回は $Q(v, m_v), Q(v, m_{p_v})$ の形式でしか呼び出されないことに注意すると、この漸化式に従ってメモ化再帰を行っても十分高速に動くことが分かります。（各頂点 c に対して、$Q(c, x)$ の x として呼ばれる候補は 1 から c までのパス（c 含まず）の頂点の中で番号上位 3 つのものに絞られます。）よって、$c_v - c_{p_v}$ の値をすべて求めることができるので、すべての c_v を求めることができます。時間計算量は $O(N)$ となります。（メモ化再帰のメモは各頂点に対して行うことで線形になります。）

F : Construction of a tree

まず必要条件を考えます。木の特徴として、どの頂点 v を固定したとしても v を根として根付き木とみなすことで、辺と v 以外の頂点を完全にマッチングさせることができます。この特徴を用いると、任意の頂点 v に対して、v 以外の頂点と E の元の間に完全マッチングが存在することが必要だと分かります。（ただし、ここでは一方が v 以外の頂点、もう一方が辺集合であるような二部グラフであり、(u, E_i) の間には $u \in E_i$ のときに辺が存在するようなものを考えています。）実はこれが十分条件であることが示せます。ある頂点 v での完全マッチングを考え、v 以外の頂点 u に対して u に対応する辺集合を S_u とします。そこで、$a \in S_b$ のときに a から

b に辺を張ってできる N 頂点グラフを考えると、v からすべての頂点にたどり着けることが示せます。というのも、v 以外の任意の頂点 u に対し、u を除いた頂点と辺集合の間の完全マッチングを考えて、その完全マッチングと v を除いた時の完全マッチングの辺を組み合わせて得られる二部グラフを考えます。このとき、v と u のみ次数が 1 なので、v と u を結ぶパスが存在し、そのパスを考えるとそれが先のグラフでの v から u へのパスになっています。よって、v からすべての頂点にたどり着けることが示されました。よって、先のグラフで v から DFS を行って得られる DFS 木を考えると、これが求めたい木となります。以上より、必要十分性が得られました。証明の中で求めたい木の構成が得られているので、判定及び構成がともに可能であることが示されました。ボトルネックとなるのは、ある一つの頂点に対してその頂点を除いたときの完全マッチングを求めるところであり、Dinic 法を用いることにより $O(\sum |E_i|\sqrt{N})$ の時間計算量で解くことができます。

AGC 029 Editorial

yutaka1999

December 15th, 2018

A : Irreversible operation

Note that it makes no difference to regard the allowed operation as swapping two adjacent stones instead of flipping them. Then, the number of operations is the same as the sum of the distance each white stone moves. Here, let's focus on the fact that operations won't change the relative order among stones of the same color, and that the stones should line up like $W..WB..B$ in the end. This means, for each white stone, the number of black stones which switched places with it is the number of black stones on its left in the initial arrangement. Therefore, when you reach a situation where no additional operations can be made, the number of operations you made should be $\sum_{S_i=W} \#\{1 \leq j < i | S_j = B\}$, regardless of how you made them. Hence, the answer is also $\sum_{S_i=W} \#\{1 \leq j < i | S_j = B\}$. This value can be computed in $O(N)$.

B : Powers of two

First, let's think about the ball which has the largest number on it. Assume that the i-th ball has the largest number A_i. Then, if you can pair the i-th ball and the j-th ball, the value of $A_i + A_j$ should be uniquely determined because $A_i < A_i + A_j$ holds: i.e. if there exists k such that $A_i + A_j = 2^k$, k is unique because $A_i < 2^k \leq 2A_i$. Actually this suggests that if you find such j, you can immediately pair the i-th ball and the j-th ball. This can be proved by the fact that, even if the j-th ball is paired with another ball, you can remove that pair and form a pair of i-th ball and j-th ball, with the number of pair unchanged.

Therefore, greedy pairing is sufficient and you can solve this problem just simulating the paring. The time complexity bottleneck is finding the largest A_i and j such that $A_i + A_j$ is a power of 2. The overall complexity is $O(N \log N)$.

C : Lexicographic constraints

Let's consider finding the answer with binary search: is it possible to construct $S_1, ..., S_N$ if you have X characters?

For convenience, let S_0 be an empty string. Let's construct the optimal S_i in turn, based on S_{i-1} for $1 \leq i \leq N$. The optimal S_i is the lexicographically minimal string among strings of length A_i which are lexicographically greater than S_{i-1}. Denote by $0, 1, ..., X-1$ the alphabets ordered from the lexicographically smallest to the largest. Then,

- if $A_{i-1} < A_i$, the optimal S_i is obtained by extending S_{i-1} to length A_i, and assigning 0s to the $A_{i-1}+1, ..., A_i$ -th characters.
- if $A_{i-1} \geq A_i$, the optimal S_i is obtained by deleting the $A_i+1, ..., A_{i-1}$ -th characters, and replacing the A_i-th character with a character bigger by 1. (note that if the A_i-th character is $X-1$, you need to deal with carry.)

Because the length of the strings can be 10^9, you may think it is impossible to maintain and calculate those strings in such a way. However, considering most of the characters are 0, you can see that one can relatively easily maintain them using map or array. This way, S_i can be constructed in order, so the criterion function for binary search can be implemented indeed. (it is impossible to construct $S_1, ..., S_N$ with X characters if you become unable to deal with carry.)

Hence, overall you can solve this problem in $O(N \log N \log \max A)$ as each check is done in $O(N \log N)$ (or $O(N)$ with array),

D : Grid game

If Takahashi didn't make a move, it is optimal for Aoki to finish the game not moving the piece. This means it is optimal for Takahashi to make as many moves as possible without leaving the piece in the same cell. Then, the number of actions Takahashi performs is (the number of moves Takahashi makes) +1, so it suffices to calculate how many times Takahashi can move the piece.

Since Aoki would like to end the game as soon as possible, Aoki should move the piece as soon as possible to cells where Takahashi cannot make a move. Because Takahashi has no other choice but to move the piece, you only need to consider Aoki's choice. That is, you should find the minimum number of actions Aoki need to take, under the condition that Aoki is allowed to move the piece from (x, y) to $(x + 1, y)$ or $(x + 1, y + 1)$.

Transforming the coordinate by $(x, y) \to (x - y, y)$, you can reduce the problem to the case where the destinations are $(x + 1, y)$ and $(x, y + 1)$. In this case, Takahashi cannot make a move from cell (x, y) if $(x + 1, y)$ is occupied by an obstacle, (we consider the coordinates outside of the grid are also occupied by obstacles.) Therefore, you have to find among such cells the nearest cell, which is reachable without hitting obstacles. You can judge efficiently whether some cell is reachable without hitting obstacles, if you check cells in ascending order of y and keep the range of x coordinates of the cells, at which the piece can arrive without facing obstacles. Hence, you can find the nearest cell, and can solve this problem.

The time complexity bottleneck is sorting obstacles in ascending order of y, and the overall complexity is $O(N \log N)$.

E : Wandering TKHS

Let's make the tree rooted at 1. For convenience, let c_1 be 1 and p_v be the parent of node v. We also define m_v as the maximum ID of vertices on the path from p_v to $1(p_1 = m_1 = -1)$, and $Q(v, x)$ as (the number of v's offspring nodes, reachable from v by passing only nodes whose ID is less than or equal to x). Then, let's consider the value of $c_v - c_{p_v}$ for each $v > 1$.

i) The case where $v > m_{p_v}$: $c_v - c_{p_v}$ equals $Q(v, m_v) + 1$.
ii)The case where $v < m_{p_v}$: $c_v - c_{p_v}$ equals $Q(v, m_v) - Q(v, m_{p_v})$.

Therefore, calculating $Q(v, x)$ is enough to find the answer.

This can be done with data structures, but there is an easier way because this time we care only about the particular values of $Q(v, x)$. First, we can see that the values can be calculated recursively because $Q(v, x) = \sum_c (1 + Q(c, x))$(where c is v's child and $c < x$) holds. Generally this takes $O(N^2)$ even with memoization, but considering that the values we need are just $Q(v, m_v), Q(v, m_{p_v})$ of each v, one can see it works faster enough in this case. (for each vertex c, the candidates for x of $Q(c, x)$ are at most 3 vertices on the path between p_c and 1, which have a larger ID than others. Hence, you can find all the values of $c_v - c_{p_v}$ and eventually c_v. The time complexity is $O(N)$. (the linear time can be achieved with the memoization in every vertex.)

F : Construction of a tree

Consider necessary conditions first. Tree has the following characteristic: for any v, edges and vertices except v have a natural matching if we root the tree at v. This characteristic shows that as a necessary condition, for any v, there should be a perfect matching between vertices except v and $\{E_i\}$. (here we consider a bipartite graph which has vertices except v on one side, and the subsets $\{E_i\}$ on the other. The graph has edge (u, E_i) if $u \in E_i$.)

Actually, this condition is also a sufficient condition. Consider the perfect matching obtained for some v. Let S_u be the set of edges on the bipartite graph, one of whose endpoints is u. Next, we construct an extra graph with N vertices by adding edge (a, b) if $a \in S_b$. It can be proved that every vertex in the new graph is reachable from v in the following way: For any $u(\neq v)$, consider the union of two perfect machings obtained by applying the above-mentioned operation to u and v. (this should be a bipartite graph.) Then, there exists a path between v and u because only v and u are the vertices with degree 1, and this path should be also a path in the extra graph. Thus, it is shown that every vertex in the extra graph is reachable from v. This also means when we run DFS from v in the extra graph, the resultant DFS-tree is a tree demanded in this problem. Hence, we find the condition is necessary and sufficient.

As we alredy saw the way of constructing the demanded tree in the proof, it is proved that both of the decision and construction are possible in the problem. The time complexity bottleneck is finding the perfect matching for some vertex, and it can be done in $O(\sum |E_i|\sqrt{N})$ using Dinic's algorithm.

AGC 028 解説

writer : maroonrk

平成 30 年 10 月 13 日

For International Readers: English editorial starts from page 8.

A : Two abbreviations

以下、文字列はすべて 0-indexed で考えます。明らかに L は $lcm(N, M)$ の倍数です。

次に、L が一つ与えられたときに条件を満たすものがあるかどうかを判定します。これは、$a \times L/N = b \times L/M$ なるすべての a, b について、$S_a = T_b$ となっているかどうか調べれば良いです。$S_a = T_b$ を満たさないものがあれば、構成は不可能です。逆にこれらの a, b すべてで $S_a = T_b$ ならば、構成は可能です。

$n = N/gcd(N, M),\ m = M/g(N, M)$ とすれば、$a \times m = b \times n$ なる a, b を調べれば良いです。n, m が互いに素であることから、$(a, b) = (k \times n, k \times m)\ (k = 0, 1, ...gcd(N, M) - 1)$ が解であるとわかります。

L の値によらず、調べる a, b の組は同じです。よって、構成が可能な場合は $L = lcm(N, M)$ とするのが最小です。

以上より、この問題は $O(N + M)$ 時間で解けます。

B : Removing Blocks

以下、すべての演算は mod 10^9+7 で行います。特に、mod の逆元を扱うので、これを知らない方は適宜勉強してください。

ブロック i を取り除くときにブロック i と j が連結である確率を $P(i,j)$ と書くことにします。各 j について、$\sum_{i=1}^{N} P(i,j)$ が求まれば良いです。

ブロック i を取り除くときにブロック i と j が連結であるということは、ブロック i と j の間にあるブロック（ブロック $i,\ j$ 含む）の中で、一番最初に取り除かれるものがブロック i であるということです。よって、$P(i,j) = 1/(abs(i-j)+1)$ とわかります。

$1/1,\ 1/2,\ 1/3,\ \ldots\ 1/N$ の累積和を求めておけば、$\sum_{i=1}^{N} P(i,j)$　は各 j について定数時間で求められます。よってこの問題は $O(N)$ 時間で解けます。

C : Min Cost Cycle

辺のコストを $min(A_x, B_y)$ とする代わりに、A_x と B_y のうち好きな方を選ぶ、としても答えは変わりません。

各頂点を 4 つのタイプに分類します。頂点 v のタイプは、以下の様に分類します。

- タイプ X　サイクルにおいて、v から出る辺の重みは A_v を採用する。v に入る辺の重みは B_v を採用する。
- タイプ Y　サイクルにおいて、v から出る辺の重みは A_v を採用する。v に入る辺の重みは B_v を採用しない。
- タイプ Z　サイクルにおいて、v から出る辺の重みは A_v を採用しない。v に入る辺の重みは B_v を採用する。
- タイプ W　サイクルにおいて、v から出る辺の重みは A_v を採用しない。v に入る辺の重みは B_v を採用しない。

各頂点にタイプを定めると、コストは決定します。サイクルが存在するようなタイプの割り振り方は、以下の 3 通りです。

- 全頂点がタイプ Y。
- 全頂点がタイプ Z。
- タイプ X の頂点の個数とタイプ W の頂点の個数が 1 以上かつ等しい。

1 つめと 2 つめのパターンの計算は簡単です。あとは、3 つめの割り振り方をしたときの最小コストを求められれば良いです。

$A_1, A_2, ...A_N, B_1, B_2, ...B_N$ を昇順にソートします。このとき、最初の N 項の中に A_v と B_v が両方含まれる頂点 v が存在すれば、最初の N 項に含まれる重みを採用するようにタイプ分けを行うことが出来ます。よって、最初の N 項の和が答えになります。

それ以外の場合はタイプ W になる頂点の番号を 1 つ決め打って、残りの $N-1$ 頂点に紐付いた重みを小さい方から貪欲に採用する、というふうにすれば答えが求まります。

予め $A_1, A_2, ...A_N, B_1, B_2, ...B_N$ がソートされており、かつその最初の N 項の中に A_v と B_v が両方含まれる頂点 v が存在しないことから、タイプ W の頂点 w を一つを決め打ったあとで選ばれる辺の重みは、最初の N 項から A_w または B_w を除いたものと、$N+1$ 項目または $N+2$ 項目（$N+1$ 項目が A_w または B_w ならば $N+2$ 項目を採用）であるとわかります。

以上でこの問題を解くことが出来ます。ソートがボトルネックになり、計算量は $O(NlogN)$ です。

D： Chords

円環を切り開いて直線として考えます。

$dp[i][j]$ = 点 i が左端、点 j が右端となるような連結成分が存在する様に $[i, j]$ 内の点を結ぶ方法の数、という DP をします。

$[i, j]$ 内の点が、すでに $[i, j]$ 外の点と結ばれることが確定している場合、$dp[i][j] = 0$ です。

そうでない場合、$[i, j]$ 内で、まだどの点とも結ばれていない点の個数を $f(i, j)$ とします。$f(i, j)$ が奇数なら $dp[i][j] = 0$ です。

そうでない場合、$[i, j]$ 内の点を結ぶ方法は、$g(i, j) = (f(i, j) - 1) \times (f(i, j) - 3) \times ... \times 1$ 通りあります。このうち、点 i を左端とする連結成分の右端が j でない場合の数を引けば、$dp[i][j]$ を求められます。点 i を左端とする連結成分の右端が点 k である場合の数は、$dp[i][k] \times g(k + 1, j)$ となります。

以上の DP でこの問題は解くことが出来ます。計算量は $O(N^3)$ です。

E : High Elements

S の最初の数文字が決定しており、現段階で X, Y の高い項の個数が C_X, C_Y、要素の最大値が H_X, H_Y であるとします。このときに、S の残りの文字を適当に割り振って、よい文字列にできるか、という問題が解ければ良いです。

残りを割り振って S がよい文字列となったとして、そのときの X の高い項が $..., H_X, a_1, a_2, ..a_{|a|}$、Y の高い項が $..., H_Y, b_1, b_2, ...b_{|b|}$ であったとします。このとき、a, b のどちらか一方は、P の高い項のみを含む、としても問題ありません。もし a, b が両方とも P の中で高くない項を含んでいれば、それらを交換することで、P の中で高くない項の個数を減らせるからです。

一般化して、a が P の高い項のみを含むとして考えます（実際のプログラムでは b の場合についても同様に試します）。a, b の中に登場する P の高い個数の合計は固定です。（P の中で高い項はどのような割り振りでも高い項であり続けるため）この個数を Q とします。b が P の中で高い項を k 個含み、P の中で高くない項を m 個含むとします。すると、a の項数は $Q - k$ 個です。S がよい文字列であるためには、$C_X + |a| = C_Y + |b|$ である必要があります。これを変形すると、$2 \times k + m = C_X - C_Y + Q$ となります。

この式の右辺は定数です。これを以下では T と書きます。あとは、左辺の値をうまく調節することを考えます。今解きたい問題は、P でまだ割り振りが決定していない項の部分列 $b = (b_1, b_2, ..., b_{k+m})$ であって、

- $H_Y < b_1 < b_2 \ ... \ < b_{k+m}$
- b 内にある P の高い項の個数を k、それ以外の項数を m とした時、$2 \times k + m = T$

を満たすものを見つけることです。

ところで、$2 \times k + m \geq T$ かつ $2 \times k + m \equiv T \bmod 2$ なる b があれば、$2 \times k + m = T$ なる b は必ず存在します。$2 \times k + m$ を 2 減らすことができるためです。よって、mod2 ごとに、$2 \times k + m$ の最大値を求められれば判定が出来ます。

これは、予め P の後ろの方から LIS を求めるのと同じ要領で DP を行っておけば良いです。ここで必要な操作は、配列のある値を変更する、配列のある範囲の最大値を求める、です。これは SegmentTree を用いれば高速に行えます。

よってこの問題は $O(NlogN)$ で解けます。

F : Reachable Cells

分割統治をします。与えられる盤面は必ず縦の長さが横以上のものだとします。そうでない場合は、縦横を反転させてから問題を解きます。盤面を上下半分ずつに分割して、上半分に X、下半分に Y があるような組 X, Y について問題を解くことができればよいです。$H \times W$ の盤面が与えられているときに $O(HW)$ 時間でこれを解ければ、全体で $O(N^2 logN)$ 時間で解けます。

以下では、各マスに書いてある数字を無視し、単に X, Y のペアの個数を数えます（数字がある場合への拡張は容易です)。上半分の盤面を盤面 U と呼び、そのうち i 行目 j 列目のマスを $U(i, j)$ で表します。盤面 U の行数を H_U とします。同様に、下半分の盤面を盤面 D とよび、そのうち i 行目 j 列目のマスを $D(i, j)$ で表します。盤面 D の行数を H_D とします。まず以下のサブルーチンを準備します。

- $MeetingPoint(a, b)$：$D(1, a), D(1, b)$ のどちらからでも到達できるマスの行番号であって最小のもの（存在しないなら ∞）を $O(1)$ 時間で返す。
- $BothReachable(a, b, l)$：$D(1, a), D(1, b)$ のどちらからでも到達できるマスのうち、行番号が l 以下のものの個数を $O(1)$ 時間で返す。

各 $D(i, j)$ に対して、$Left(i, j)$ を、$D(1, x)$ から $D(i, j)$ に到達可能であるような x の最小値（存在しないなら ∞）と定義します。同様に、$Right(i, j)$ を、$D(1, x)$ から $D(i, j)$ に到達可能であるような x の最大値（存在しないなら $-\infty$）と定義します。また、$Top(j)$ を、$U(y, x)$ から $U(H_U, j)$ に到達可能であるような y の最小値と定義します。同様に、$Bottom(j)$ を、$D(1, j)$ から $D(y, x)$ に到達可能であるような y の最大値と定義します。

累積 min の DP をすると、$Left(i, j) \leq a, b \leq Right(i, j)$ なる $D(i, j)$ の中での i の最小値が求められます。この最小値をとるマスを $D(p, q)$ とします。ここで、$p \leq \min(Bottom(a), Bottom(b))$ であれば、$MeetingPoint(a, b) = p$ です。$Left(p, q), Right(p, q), Bottom(a), Bottom(b)$ を実際に達成するパスを考えると、$D(p, q)$ に $D(1, a), D(1, b)$ から到達可能であることがわかるからです。

また、$p > \min(Bottom(a), Bottom(b))$ の場合、明らかに $MeetingPoint(a, b) = \infty$ となります。これで $MeetingPoint(a, b)$ が実装できました。明らかに前処理 $O(HW)$、クエリ $O(1)$ です。

次に $BothReachable(a, b, l)$ を考えます。まず、$MeetingPoint(a, b) > l$ の場合は、0 を返せば良いです。それ以外の場合は、

- $Left(y, x) \leq a$
- $b \leq Right(y, x)$
- $y \leq l$

を満たす (y, x) の個数を数えれば良いです。上 2 つの条件だけならば、次の様に数えることが出来ます。

- まず、すべての $Left(y, x) \leq Right(y, x)$（つまりどちらも ∞ でない）なる (y, x) の個数を数えて、答えに足す。以下でも $Left(y, x) \leq Right(y, x)$ が成り立つものについてのみ数える。
- $Right(y, x) < b$ となる (y, x) の個数を数えて答えから引く。
- $a < Left(y, x)$ となる (y, x) の個数を数えて答えから引く。

- $a < Left(y,x) \leq Right(y,x) < b$ となる (y,x) の個数を数えて答えに足す。

上記 3 つについては、$y \leq l$ の制約を含めても、累積和をとることで求めることが出来ます。ところで、$a < Left(y,x) \leq Right(y,x) < b$ が成り立つならば、必ず $y < MeetingPoint(a,b)$ が成り立つことは、先ほどと同様にパスを書いてみるとわかります。今、$MeetingPoint(a,b) \leq l$ の場合を考えているので、$a < Left(y,x) \leq Right(y,x) < b$ ならば $y \leq l$ は常に成り立ちます。よって、4 番目については、y の条件を無視しし、これも累積和を取ることで求めることが出来ます。前処理は $O(HW + W^2) = O(HW)$ です（$H \leq W$ より）。また、クエリは $O(1)$ です。

また、これを利用することで、「$Reachable(a) = D(1,a)$ から到達できるマスの個数」も $O(1)$ で求めることが出来ます。

サブルーチンが準備できたので、これから各 $1 \leq y \leq H_U$ について、$X = U(y,x)$、$Y = D(i,j)$ であって、X から Y に到達可能な組 X, Y の個数を $O(W)$ で求めます。

$L(x)$ を、$U(y,x)$ から $U(H_U, j)$ に到達できる最小の j（存在しないなら ∞）と定義します。同様に、$R(x)$ を、$U(y,x)$ から $U(H_U, j)$ に到達できる最大の j（存在しないなら $-\infty$）と定義します。

ここで、L, R はともに x に対して単調増加です。よって、次のような問題が解ければよいことになります。

> マスの集合 S を用意する。最初 S は空である。S に対する次のクエリを均し計算量 $O(1)$ で処理せよ。
>
> - S に $D(1,j)$ を追加する。ただしこのときの j は、今まで追加されたどの j よりも大きい。
> - S から $D(1,j)$ を削除する。ただしこのときの j は、S 内のマスの中で j が最小のもの。
> - S 内のいずれかのマスから到達可能であるマスの個数を数える。

$has[j]$ を $D(1,j)$ から到達でき、かつ、S 内にあるすべての $D(1,j')$ $(j < j')$ から到達出来ないマスの個数と定義します。S を変更するクエリのたびに $has[j]$ を更新できれば良いです。削除クエリの際には $has[j]$ は変化しないので追加クエリのことを考えます。

いま S に $D(1,j)$ を追加したいとします。まず、明らかに $has[j] = Reachable(j)$ です。次に、ある $D(1,j') \in S$ に対して、$Bottom(j') \leq Bottom(j'')$ なる $j'' \in S$ が存在すれば、$has[j']$ は変化しません。なぜなら、$D(1,j')$ と $D(1,j)$ からともに到達可能であるようなマスは明らかに $D(1,j'')$ からも到達可能であり、$has[j']$ にカウントされていないからです。よって、$has[j']$ の変化を考えるべき j' は、$Bottom(j')$ の階段凸法に含まれるものだけです。$Bottom(j')$ の階段凸法に含まれる j' を、$J_1 < J_2 < ... < J_k$ とします。$has[J_k]$ は、$BothReachable(J_k, j, min(Bottom(J_k, j)))$ の分だけ減ります。また、$p < k$ なる p について、$has[p]$ の減少分を考えるとそれは、$D(1,J_p)$ と $D(1,j)$ の両方から到達可能であって、$D(1,J_q)(p < q)$ からは到達出来ないマスの個数です。これは $D(1,J_p)$ と $D(1,j)$ の両方から到達可能であって、行番号が $Bottom(J_{p+1})$ 以上のものと一致します。よってその値は、$BothReachable(J_p, j, min(Bottom(J_p), Bottom(j)))$
$-BothReachable(J_p, j, min(Bottom(J_{p+1}), Bottom(j)))$ です。ところで、$Bottom(J_p) \geq Bottom(j)$ となった段階で、$has[J_q]$ $(q < p)$ は変化しないことがわかります。よって、実際に $has[J_p]$ を更新する回数は、$Bottom(J_p) < Bottom(j)$ なる p の個数 $+1$ 回です。$Bottom(J_p) < Bottom(j)$ なる J_p は以降階段凸法に現れないため、W 回のクエリに対して合計で $O(W)$ 時間での計算が行なえます。

以上より、この問題は解けました。

AGC 028 Editorial

writer : maroonrk

平成 30 年 12 月 23 日

A : Two abbreviations

We use 0-based indices.

Suppose that we are given an integer L that is divisible by both N and M, and let's check if we can find a valid string. For each pair (a, b) such that $a \times L/N = b \times L/M$, we want to check if $S_a = T_b$. (The answer is possible if and only if this condition holds for all pairs.)

Let $n = N/gcd(N, M)$, $m = M/g(N, M)$. Then, $a \times m = b \times n$ must holds. Since n and m are coprime, all possible pairs are $(a, b) = (k \times n, k \times m)$ $(k = 0, 1, ...gcd(N, M) - 1)$.

This set of pairs doesn't depend on the value of L. Thus, in case the answer is not impossible, the answer is $L = lcm(N, M)$.

This solution works in $O(N + M)$ time.

B : Removing Blocks

Let's compute the expected value of the total score (when we choose the order of removals uniformly at random).

Let $P(i, j)$ be the probability such that the blocks i and j are connected, when we remove block i. Then, for each j, we compute the value $B_j := \sum_{i=1}^{N} P(i, j)$. This is the expected number of times the value A_j is added to the total score. Thus, the answer is the sum of $A_j B_j$ for all j.

Notice that $P(i, j)$ happens if and only if the first block that is removed among the blocks $i, i+1, \ldots, j$ is the block i. Thus, $P(i, j) == 1/(abs(i-j)+1)$.

If we compute the values $1/1,\ 1/2,\ 1/3,\ \ldots\ 1/N$ (in modulo $10^9 + 7$) and their prefix sums, we can compute each B_j in $O(1)$.

This solution works in $O(N)$ time.

C : Min Cost Cycle

Instead of assigning the cost $min(A_x, B_y)$ to the edge from x and y, let's add two edges with costs A_x and B_y (and we can choose whichever we want). This doesn't change the problem.

Let's fix a Hamiltonian Cycle of the graph. There are four types of vertices. The type of a vertex v is classified as follows:

- Type X. The cost of the outgoing edge from v is A_v. The cost of the incoming edge to v is B_v.
- Type Y. The cost of the outgoing edge from v is A_v. The cost of the incoming edge to v is not B_v.
- Type Z. The cost of the outgoing edge from v is not A_v. The cost of the incoming edge to v is B_v.
- Type W. The cost of the outgoing edge from v is not A_v. The cost of the incoming edge to v is not B_v.

Here, for example, "The cost of the outgoing edge from v is not A_v" means that if the edge goes to a vertex w, we use B_w instead.

If we decide the types of all vertices, we can compute the cost. Thus, we want to decide all valid assignments of types to vertices.

Let's color the edges along the cycle. We color it red if its cost correponds to A (of its source), and color it blue if its cost corresponds to B (of its sink). If we follow the cycle, we get N pairs of two consecutive edges. Depending on the colors of the two edges, there are four types of two consecutive edges, and those four types correspond to the four types of the vertex in the middle.

There are three cases:

- All edges along the cycle are red.
- All edges along the cycle are blue.
- The cycle contains both red and blue edges. In this case, the numbers of (red, blue) pairs and (blue, red) pairs must be the same and non-zero, but except for that we can arbitrarily decide the frequencies of the four types.

In word of the vertex types, these are:

- All vertices are of type Y.
- All vertices are of type Z.
- The number of vertices of type X and type W are the same, and they are nonzero.

It's easy to compute the cost of the first two cases, so let's compute the optimal cost of the third case.

Let's sort the values $A_1, A_2, ...A_N, B_1, B_2, ...B_N$ in the increasing order. We want to choose N of these, but we must make sure that for some i, we choose both A_i and B_i. Let's fix such i,

and choose $N-1$ smallest elements except for them. Basically, this is the sum of the first $N-1$ numbers of the sorted array, but in case this part contains A_i or B_i, we must remove them and add next elements instead. However, since there are at most two replacements, we can always do it in constant time.

This solution works in $O(NlogN)$ time (sorting is the slowest).

D : Chords

Let $dp[i][j]$ be the number of ways to make pairs, such that there exists a connected component whose minimum is i and maximum is j. (A connected component contains a set of points, and we consider their minimum index and maximum index.) Then, the answer is the sum of these values.

Let X be the set of points with indexes $i, \ldots, j$, and Y be the set of all other points. First, we must not connect a point in X and a point in Y. If there are x unpaired points in X and y unpaired points in Y, the number of such pairings is $g(x)g(y)$, where $g(x) = (x-1)\times(x-3)\times\cdots\times 1$ is the number of ways to make pairs among x things (or $g(x) = 0$ if x is odd).

In all such pairings, we can find a connected component whose minimum is i. However, there may be some $k(i < k < j)$ such that there is a connected component whose minimum is i and maximum is k, and the component doesn't have j. We need to subtract the number of such pairings. The number of such pairings is $dp[i][k] \times g(z) \times g(y)$, where z is the number of unpaired points among points $k+1, \ldots, j$.

If we fill the $dp[i][j]$ table in the increasing order of $j - i$, we can compute all these values. This solution works in $O(N^3)$ time.

E : High Elements

In order to find the lexicographically smallest good string, we want to check if a certain string can be a prefix of a good string. Suppose that we appended some first elements of P to X and Y, and currently, the number of high elements are C_X, C_Y, and the maximum elements are H_X, H_Y, respectively. (In case X is empty, $H_X = -1$.) We want to check if we can make the number of high elements in X and Y same by assiging the remaining elements properly.

Suppose that after assigning the remaining elements, the high elements of X are $..., H_X, a_1, a_2, ..a_{|a|}$ and the high elements of Y are $..., H_Y, b_1, b_2, ...b_{|b|}$. They must satisfy the following conditions:

- $H_X < a_1 < \cdots < a_{|a|}, H_Y < b_1 < \cdots < b_{|b|}$
- $C_X + |a| = C_Y + |b|$
- All high elements in P (in the "remaining" part) are included in a or b. This is because a high element in P will always be a high element in the new sequence.

These conditions are also necessary, that is, if we can find such a and b, we can find a good assignment. We shuold carefully assign remaining elements that are not in a or b such that they don't become high elements in new sequences (and this is always possible). Thus, we want to determine if such a and b exist.

We can also assume that one of the following holds:

- All elements in a are high elements in P.
- All elements in b are high elements in P.

If none of the above two holds, we can remove one non-high element each from a and b, and still keep the conditions above. Thus, assume that the former holds (we should also try the latter case, but it works in the exactly same way.)

Let Q be the number of high elements among "the remaining elements". If we decide b, we can automatically decide a because a must be the set of all high elements that are not chosen in b (and as we described above, we assume that all elements in a are high in P). The conditions can be rewritten only using b:

- $H_Y < b_1 < \cdots < b_{|b|}$
- $C_X + Q - k = C_Y + |b|$. Here, h is the number of high elements in Q that are included in k.

We can even simplify the second condition. Let m be the number of non-high elements in Q that are included in m (i.e., $m = |b| - k$). Then, the second condition is equivalent to $2 \times k + m = C_X - C_Y + Q$.

In words, we are given a sequence and a constant. Some elements of the sequence are marked as "1 point" and others are marked with "2 points". We want to check if the seqeunce contains a increasing subsequence whose total points is exactly the given constant.

Notice that if there is an increasing subsequence whose total points is $c+2$, we can also achieve exactly c points (by removing some points). Thus, to check if we can achieve exactly c points,

we just need the maximum points we can get, with the same parity as c. We can compute it by a DP with a simple segment tree (similar to the computation of LIS). If we do the DP from right to left and keep the DP table, we can answer these questions for all suffixes of P.

This solution works in $O(NlogN)$ time.

F : Reachable Cells

We assume that digits on all cells are 1, since it does not matter much.

Suppose we are given H by W board and let U be the upper half of the board and L be the lower half. We are going to count the number of pairs of cells (c_1, c_2) such that $c_1 \in U$, $c_2 \in L$ and c_2 can be reachable form c_1 in $O(HW)$ time. Using Divide and Conquer, the original problem can be solved in $O(N^2 logN)$ time.

Let's assume $H \geq W$. Define $U(i, j)$ as the cell of U at the i-th row from the top and the j-th column from the left. Define $L(i, j)$ similarly. Let H_U and H_L be the number of rows of U and L respectively. We prepare following functions:

- $MeetingPoint(a, b)$: returns the minimum row number of a cell which can be reachable from both $L(1, a)$ and $L(1, b)$ (returns ∞ if no such cell exits). $O(1)$ time per query.
- $BothReachable(a, b, r)$: returns the number of cells which can be reachable from both $L(1, a)$ and $L(1, b)$ whose row number is at most r. $O(1)$ time per query.
- $Reachable(a)$: returns the number of cells which can be reachable from $L(1, a)$.

Let $Left(i, j)$ be the minimum x such that $L(i, j)$ can be reachable from $L(1, x)$ (or ∞ if no such x exists). Similarly, let $Right(i, j)$ be the maximum x such that $L(i, j)$ can be reachable from $L(1, x)$ (or $-\infty$ if no such x exists). Let $Top(j)$ be the minimum y such that $U(H_U, j)$ can be reachable from $U(y, x)$ for some x. Similarly, let $Bottom(j)$ be the maximum y such that $L(y, x)$ can be reachable from $L(1, j)$ for some x.

We can calculate, for every $a < b$, the minimum i such that $Left(i, j) \leq a < b \leq Right(i, j)$ by dynamic programming. Let $L(p, q)$ be a cell which achieves minimum i. If $p > min(Bottom(a), Bottom(b))$, obviously $MeetingPoint(a, b) = \infty$. Otherwise, since a path from $L(1, Left(p, q))$ to $L(p, q)$ or a path $L(1, Right(p, q))$ to $L(p, q)$ must intersect a path from $L(1, a)$ to $L(Bottom(a), x)$ for some x, $L(p, q)$ can be reachable from $L(1, a)$. Similarly, $L(p, q)$ can be reachable from $L(1, b)$. Since $MeetingPoint(a, b) \geq p$, we get $MeetingPoint(a, b) = p$. Now we have $MeetingPoint(a, b)$. Precalculation requires $O(HW)$ time and one quiery takes $O(1)$ time.

Let's move on to $BothReachable(a, b, r)$. If $MeetingPoint(a, b) > r$, the return value is 0. Otherwise, we want to count the number of cell (y, x) that satisfy:

- $Left(y, x) \leq a \leq b \leq Right(y, x)$
- $y \leq r$

The first condition is equivalent to the following:

1. add the number of $L(y, x)$ such that $Left(y, x) \leq Right(y, x)$
2. subtract the number of $L(y, x)$ such that $Right(y, x) < b$
3. subtract the number of $L(y, x)$ such that $a < Left(y, x)$
4. add the number of $L(y, x)$ such that $a < Left(y, x) \leq Right(y, x) < b$

Conditions 1, 2, and 3 can be combined with the condition $y \leq r$ and we can count the number by prefix sums. For condition 4, since $a < Left(y, x) \leq Right(y, x) < b$ implies $y < MeetingPoint(a, b) \leq r$, we can count the number also by prefix sums. Now we have $BothReachable(a, b, r)$. Precalculation requires $O(HW + W^2) = O(HW)$ time and one quiery takes $O(1)$ time.

$Reachable(a)$ can be calculated by $BothReachable(a, a, H_L)$.

We have prepared all the subroutines. Assume we are given a constant y. We are going to count the number of pairs of cells $U(y, x)$ and $L(i, j)$ such that $L(i, j)$ can be reachable from $U(y, x)$ in $O(W)$ time.

We ignore a cell $U(y, x)$ we can not reach $U(H_U, j)$ for any j. Let $Min(x)$ be the minimum j such that $U(H_U, j)$ can be reachable form $U(y, x)$. Similarly, Let $Max(x)$ be the maxmum j such that $U(H_U, j)$ can be reachable from $U(y, x)$. It is easily seen that both $Min(x)$ and $Max(x)$ monotonically increase as a value of x. Now we are going to solve the following problem:

Let S be a set of cells. Initially, S is empty. Process $O(W)$ queries of following types in $O(W)$ time.

- Add $D(1, j)$ to S. It is assumed that $D(1, j)$ is on the right of cells that were previously added.
- Erase $D(1, j)$ from S. It is assumed that $D(i, j)$ is the leftmost cell in S.
- Count the number of cells which can be reachable from at least one cell in S.

Let $has[j]$ ($D(1, j) \in S$) be the number of cells which can be reachable from $D(1, j)$ which cannot be reachable from $D(1, j')$ for all $j < j',\ D(1, j') \in S$. If we can maintain values of $has[j]$, it is easy to answer to a query of the third type. In processing a query of the second type values of $has[j]$ do not change. So we focus on a query of the first type.

Assume we are going to add $D(1, j)$ to S. Clearly, $has[j] = Reachable(j)$. For all $j' < j,\ j' \in S$, if there exists a j'' such that $j'' \in S,\ j' < j'' < j,\ Bottom(j') \leq Bottom(j'')$, the value of $has[j']$ does not change. Let $J_1 < J_2 < ... < J_k$ be the list of j's such that $has[j']$ can be changed (that is, no j'' satisfies the condition above). It is easily seen that $Bottom(J_1) > Bottom(J_2) > ... > Bottom(J_k)$. The value of $has[J_k]$ decreases by $BothReachable(J_k, j, min(Bottom(J_k), Bottom(j)))$. For all $p < k$, the value of $has[J_p]$ decreases by $BothReachable(J_p, j, min(Bottom(J_p), Bottom(j))) - BothReachable(J_p, j, min(Bottom(J_{p+1}), Bottom(j)))$. If, for some p, $Bottom(J_p) \geq Bottom(j)$ holds, $has[q]$ does not change for all $q < p$. Since J_p for all p such that $Bottom(J_p) < Bottom(j)$ does not appear in the list again, $O(W)$ queries can be processed in $O(W)$ time.

AGC 027 解説

writer : camypaper, sugim48

2018 年 9 月 15 日

For international readers: English editorial starts from page 9.

A: Candy Distribution Again

まず, $\sum_{i=1}^{N} a_i = x$ の場合, 答えは N であり, そうでない場合, 答えは $N-1$ 以下です. 以降, $\sum_{i=1}^{N} a_i \neq x$ と仮定します.

子供たちの部分集合 $S \subsetneq \{1, 2, \ldots, N\}$ について, S 全員を喜ばせるための必要十分条件は $\sum_{i \in S} a_i \leq x$ です. (各 $i \in S$ について子供 i に a_i 個のお菓子を配った後, 余ったお菓子は $j \notin S$ であるような子供 j に押し付ければよいです.) よって, $\sum_{i \in S} a_i \leq x$ を満たすような S の最大サイズを求めるのが目標です. これは, a を昇順にソートした後, $\sum_{i=1}^{k} a_i \leq x$ を満たすような最大の k を求めればよいです. もちろん, k が N である場合, 代わりに $N-1$ を出力することを忘れずに.

B: Garbage Collector

ロボットがゴミを拾ったり，置くときにエネルギーを X 消費するのが厄介です．まずはこれを除去することを考えます．全てのゴミはちょうど 1 回ずつロボットに回収されるため，ゴミを拾うときに消費するエネルギーは 0 と考えても問題ありません (最後に NX を足せばよいです)．ロボットがゴミをゴミ箱に入れた回数 (これを k とします) を固定して考えてみます．すると，ゴミを捨てるときに消費するエネルギーも 0 として考えることができます (同様に，最後に kX を足せばよいです)．このとき，以下の問題が解ければよいことになります．

> k 台のロボットが原点から出発し，それぞれ 0 個以上のゴミを拾い，原点に戻ってくる．全てのゴミが拾われる場合に必要な最小限のエネルギーを求めよ．

各ロボットは拾うことに決めたゴミのうち，遠いほうから 1 つずつ拾うのが明らかに最適です．ロボットが位置 x にあるゴミを i 番目に拾うときに追加で必要なエネルギーを $E(i,x)$ とします．$E(i,x)$ は以下の式で表せます．

$$E(i,x) = \begin{cases} 5x & (i=1) \\ (2i+1)x & (otherwise) \end{cases} \tag{1}$$

$1 \leq i < j$ を満たす任意の (i,j) について $E(i,x) \leq E(j,x)$ が成立していることが重要です．詳細は省きますが，この結果より右から i 番目のゴミは $\lceil \frac{i}{k} \rceil$ 番目に拾えばよいことが言えます．

よって，k を固定した場合の答えは $(N+k)X + \sum_{i=1}^{N} E(\lceil \frac{i}{k} \rceil, a_{N-i+1})$ となります．これを愚直に計算すると計算量は $O(N^2)$ となり，部分点を獲得することができます．

$\lceil \frac{i}{k} \rceil$ が等しいような i たちについて消費するエネルギーはまとめて計算することができます．たとえば，$E(\lceil \frac{1}{k} \rceil, a_N) + E(\lceil \frac{2}{k} \rceil, a_{N-1}) + \ldots + E(\lceil \frac{k}{k} \rceil, a_{N-k+1}) = E(1, \sum_{i=1}^{k} a_{N-i+1})$ です．累積和を用いることで $\sum_{i=1}^{k} a_{N-i+1}$ は前計算 $O(N)$，取得 $O(1)$ で処理できます．以上より k を固定したときの答えは $O(N/k)$ で求めることができます．$O(\sum_{i=1}^{N} \frac{1}{i}) = O(\log N)$ が成立するため，この改善により計算量は $O(N \log N)$ となり $N = 2 \times 10^5$ の場合でも十分高速に動作します．

k の値によっては $(N+k)X + \sum_{i=1}^{N} E(\lceil \frac{i}{k} \rceil, a_{N-i+1})$ が 64 bit 符号付き整数で表せない場合があるため，オーバーフローには注意が必要です．

C: ABland Yard

全ての文字列を列挙して，構成可能かどうかを実際に判定することは当然困難です．そこで，任意の文字列が構成可能である，と同値な条件を探してみましょう．

s を AABB を 10^{100} 回繰り返した文字列とします．s が構成不可能ならば，答えは No です．s が構成可能ならば，実は全ての文字列が構成可能です．

s が構成可能なとき，与えられるグラフには AABB の繰り返しで表されるような (辺素とも点素とも限らない) 閉路が必ず存在しています．これは鳩ノ巣原理から明らかです．このとき，そのような閉路に含まれる全ての頂点はラベルが A であるような頂点と，B であるような頂点の両方に隣接しています．よって，この閉路上を移動し続けることで任意の文字列が構成可能です．

s を実際に構成するのは時間がかかりすぎますが，AABB の繰り返しで表される閉路があるかどうかを判定するだけならば簡単です．例えば，隣接している頂点のラベルの種類が 1 種類以下であるような頂点を取り除く，という操作を繰り返して全ての頂点が取り除かれるかどうかを判定する，という方法があります．これはトポロジカルソートの要領で $O(N+M)$ で実行可能です．その他，頂点を 2 倍に増やし適切に辺を張った有向グラフに閉路があるかを判定する方法もあります．

D: Modulo Matrix

$N = 500$ の場合について解くことができれば，それ以外のケースは容易に解くことができます．以降，$N = 500$ の場合についてだけ考えます．

解の一例について説明します．隣接する 2 つの数について，最大値を最小値で割ったあまりが一定という条件が最も複雑です．そこで，この関係が容易に満たせるような数の割り当て方を考えてみます．

$N \times N$ のマス目を以下の図 1 のように市松模様に塗り分けることを考えます．黒マスについて数を書き込んだのち，白マスは「隣接する全ての黒マスに書かれた数の最小公倍数 $+1$」 を書き込むことにします．すると，どの隣接する 2 つの数についても最大値を最小値で割ったあまりが必ず 1 となります．

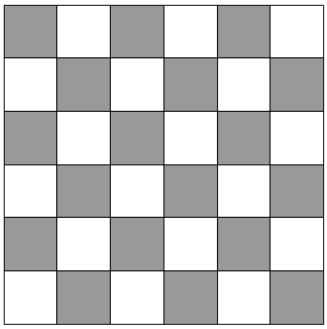


図 1　市松模様

残った問題は数の最大値が 10^{15} 以下であり，数が相異なるようにすることです．白と黒の市松模様ではなく，2 種類の斜め方向の線の重ね合わせの模様で考えてみます．$2N$ 本の線に，素数を小さい方から割り当てて 2 つの数の積を書き込むことにすると，どの数も相異なり，数の最大値が 4 つの素数の積 $+1$ 以下にすることができます．1000 番目の素数は 7919 のため，最大値が 10^{15} 以下になるようにするには割り当て方に多少の注意が必要ですが，さほど特別な工夫をせずとも達成することができます．

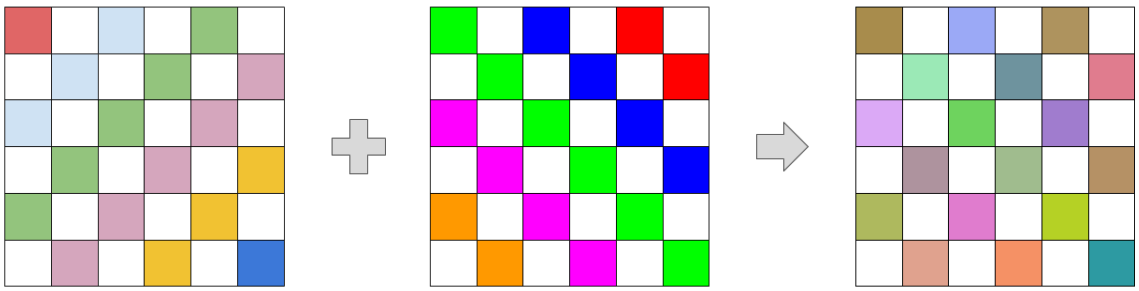


図 2　市松模様の塗り分け

E: ABBreviate

s 中に同一文字が隣り合う場所がない場合，答えは 1 です．以降，s 中に同一文字が隣り合う場所があると仮定します．

まず，操作に関する不変量を見つけましょう．a の重みを 1，b の重みを 2 とし，文字列 s の重みの総和を 3 で割った余りを $p(s)$ と定義します．このとき，$p(s)$ は操作の前後で不変です．

次に，文字列 s と文字 $c \in \{\mathtt{a}, \mathtt{b}\}$ について，s に操作を繰り返して c が得られるための必要十分条件を考えましょう．必要十分条件 $(*)$ は次のとおりです:

- $s = c$．または，
- $p(s) = p(c)$，かつ s 中に同一文字が隣り合う場所がある．

必要性は明らかです．十分性を示します．いま，$|s| \geq 2$，かつ $p(s) = p(c)$，かつ s 中に同一文字が隣り合う場所があることを仮定します．次のように操作を行えば，「s 中に同一文字が隣り合う場所がある」という条件を保ったまま，s を 1 文字分だけ短くすることができます: s 中の同一文字が連続する場所であって極大なものを適当に選び，t とします．t が s 全体である (すなわち，s が同一文字のみからなる) 場合，s の先頭 2 文字に対して操作を行えばよいです．(このとき，不変量の条件より，操作前は $|s| \neq 3$ であることに注意．) t が s 全体でない場合，t の前後いずれかに異なる文字が存在します．異なる文字が t の前に存在するならば，t の先頭 2 文字に対して操作を行えばよいです．逆に，異なる文字が t の後に存在するならば，t の末尾 2 文字に対して操作を行えばよいです．以上より，十分性を示せました．

問題は次のように言い換えられます:

> 文字列 s が与えられる．次の条件 (♣) を満たすような文字列 t は何通りか？
>
> 条件 (♣): うまく s を $|t|$ 個の区間に分割すると，各区間 (と対応する t の文字) において，条件 $(*)$ が成り立つ．

まず，t をひとつ固定したとき，条件 (♣) が成り立つか判定する方法を考えましょう．実は，これは次のようにして可能です: $j = 1, 2, \ldots, |t|$ の順に，$p(\cdot)$ の値が t_j と一致するような s の区間を左から貪欲に取っていきます．$|t|$ 個の区間を取りきることができ，かつ余った文字列の $p(\cdot)$ の値が 0 ならば，条件 (♣) が成り立ちます．例えば，$s = \mathtt{aaababbabababbaaaba}$, $t = \mathtt{abab}$ の場合，$s = \underline{\mathtt{a}}\ \underline{\mathtt{aa}}\ \underline{\mathtt{babb}}\ \underline{\mathtt{abababb}}\ \mathtt{aaaba}$ のように区間を取っていき，余った文字列について $p(\mathtt{aaaba}) = 0$ が成り立つので，この例では条件 (♣) が成り立ちます．

この判定法の正当性を確かめましょう．t がこの判定法に失敗した場合，明らかに条件 (♣) は成り立ちません．以降，t がこの判定法に成功したと仮定し，条件 (♣) が成り立つことを示します．まず，区間を貪欲に取っていることから，各区間は「長さ 1 である」または「同一文字が隣り合う場所がある」のいずれかの性質を持ち，条件 $(*)$ を満たします．よって，残る懸念事項は余った文字列の扱いのみになりました．実は，適切に区間を組み換えることで，余った文字列をいずれかの区間に含めることができます．その方法を説明します．余った文字列に同一文字が隣り合う場所がある場合，余った文字列を最後の区間に含めればよいです．以降，余った文字列に同一文字が隣り合う場所がないと仮定します．この

とき，余った文字列の $p(\cdot)$ の値が 0 なので，余った文字列は abab$\cdots$ab または baba$\cdots$ba の形をしています．ここで，(右から k 番目の区間) $+\cdots+$ (右から 1 番目の区間) $+$ (余った文字列) の形の文字列を考えます．まず，この文字列に同一文字が隣り合う場所が含まれるような最小の k をとります．例えば，$s = \cdots$ b b a b ababab の場合，$k = 4$ です．また，$s = \cdots$ babaa b a b a bababa の場合，$k = 5$ です．次に，右から $1, 2, \ldots, k-1$ 番目の区間を s の右端に詰めます．最後に，余った文字列を右から k 番目の区間に含めます．例えば，$s = \cdots$ b b a b ababab の場合，$s = \cdots$ bbababa b a b となります．また，$s = \cdots$ babaa b a b a bababa の場合，$s = \cdots$ babaababababa b a b a となります．このように区間を組み換えることで，余った文字列をいずれかの区間に含めることができ，かつ各区間が条件 $(*)$ を満たすようにできます．以上より，上述の判定法の正当性を確かめられました．

問題は，上述の判定法に成功するような t の数え上げに帰着されました．これは，文字列の部分列の数え上げと同様の DP によって可能です．時間計算量は $\mathrm{O}(|s|)$ です．

F: Grafting

直感的には，操作回数の最大値は $N-1$ 以下になりそうです．(この直感は実際には間違っていますが) しばらくの間，答えは $N-1$ 以下であることを仮定しておきます．

答えが $N-1$ 以下のとき，操作されない頂点 r が必ず存在します．r を全探索しましょう．A, B ともに，r を根とした根付き木として考えます．r から順番に子孫を見ていきます．操作されない頂点 u を見ているとき，頂点 v が A においても B においても u の直接の子ならば，v もまた操作されない頂点です．そうでないなら，v は操作される必要があり，A においても B においても v を根とする部分木に含まれる頂点は全て操作される必要があります．

操作されるべき全ての頂点たちについて，valid な操作順序が存在するかどうかを調べましょう．操作すべき頂点どうしをつなぐ辺について，A においては子から親に向かって，B においては親から子に向かって辺を向き付けます．以下の図 3 はサンプル 2 のケース 1 です．A, B を操作されない頂点たちを無視して 1 つのグラフにまとめてトポロジカルソートすることが可能ならば，valid な操作順序が存在します．例えば，$(1, 4, 6, 8, 3, 5)$ の順番で操作をすれば A を B にすることができます．辺の数は $O(N)$ で抑えられるので，答えが $N-1$ 以下の場合は $O(N^2)$ で答えを求めることができます．$T \leq 20, N \leq 50$ より十分高速です．

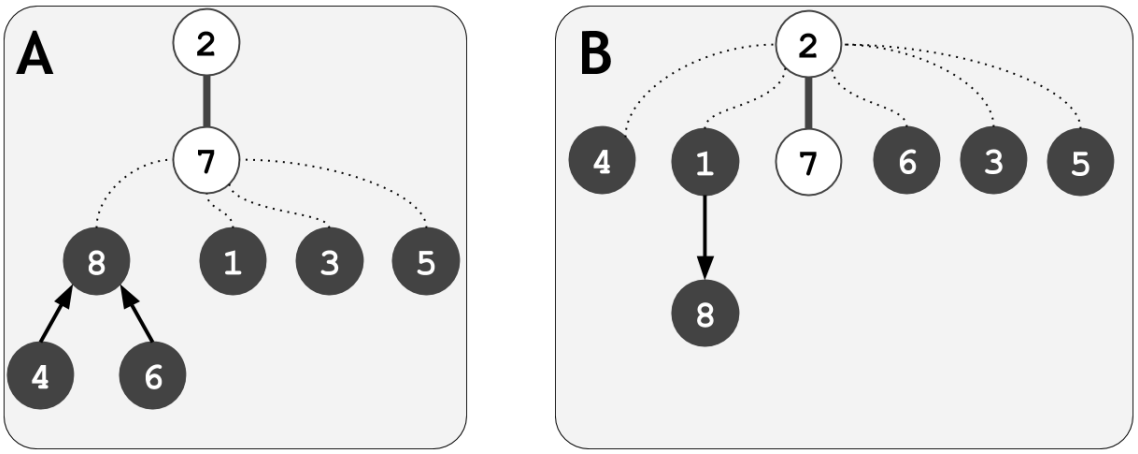


図 3　サンプル 2 ケース 1

上述の通り，操作回数が N の場合が存在します．図 4 はその例です．先程のアルゴリズムはこのケースではうまく動作しません (実際に試してみるとよいです)．このような場合であっても 1 回目の操作を実際に行うことで，操作されない頂点を作ることができます．1 回目の操作方法は $O(N^2)$ 通りなので，各ケースについて $O(N^3)$ で答えを求めることができます．$T \leq 20, N \leq 50$ より十分高速です．

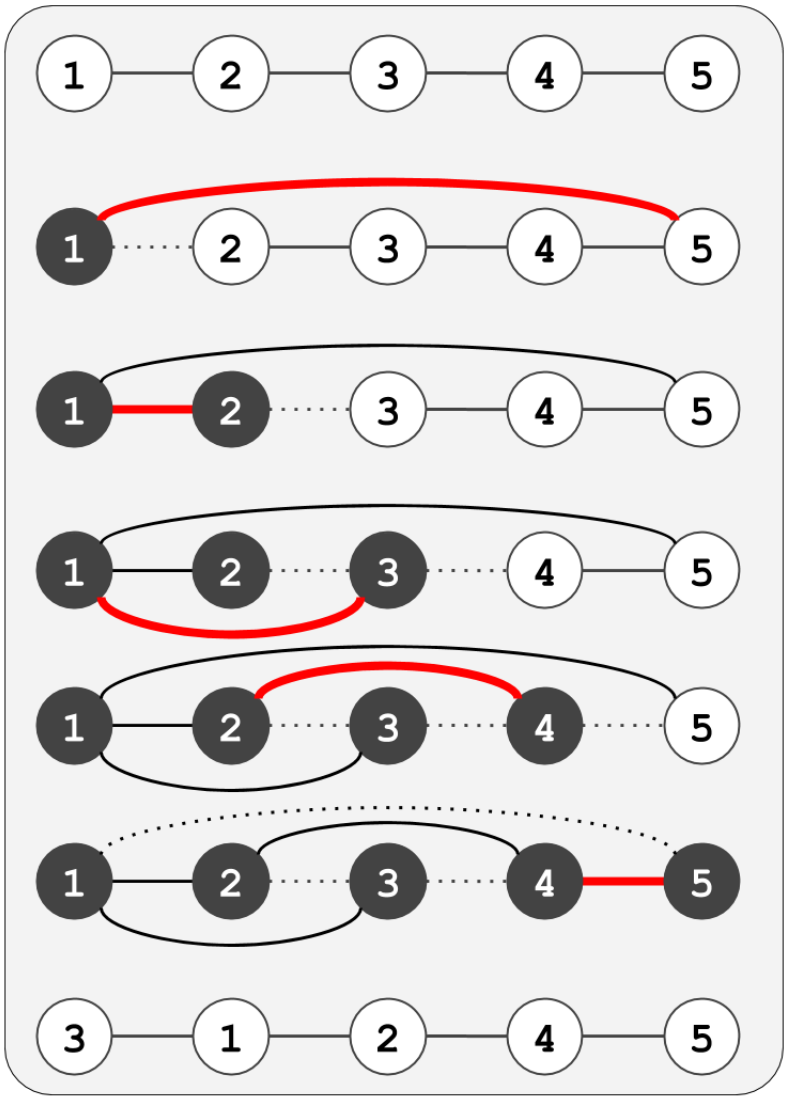


図 4　操作回数が N 回になるケース

AGC 027 Editorial

writer : camypaper, sugim48

September 15, 2018

A: Candy Distribution Again

In case $\sum_{i=1}^{N} a_i = x$, the answer is N, otherwise the answer is $N-1$. From now on, we assume that $\sum_{i=1}^{N} a_i \neq x$.

Let $S \subsetneq \{1, 2, \ldots, N\}$ be a subset of children. We can satisfy all children in S if and only if $\sum_{i \in S} a_i \leq x$. (If this inequality holds, we can give a_i candies for each child i in S, and arbitrarily distribute remaining candies among remaining children.)

Thus, our objective is to find the maximum size of S that satisfies $\sum_{i \in S} a_i \leq x$. To do this, we first sort a in ascending order, and compute the maximum k such that $\sum_{i=1}^{k} a_i \leq x$. In case $k = N$, don't forget to print $N-1$ instead.

B: Garbage Collector

Let's ignore the cost of collecting trashes: since each trash will be collected exactly once, we can ignore it and add NX at the end.

Suppose that the robot starts from the origin, takes some subset of trashes, returns to the origin, and put all trashes it took into the trash bin. When the subset of trashes is fixed, the following strategy is optimal: The robot first moves to the position of the most distant trash (in the subset). Then, it directly returns to the origin. During the return trip, it takes trashes whenever it meets the trash in the subset.

The cost of doing this can be calculated as follows. Suppose that the i-th most distant trash is at position x. Let's define $E(i, x)$ as

$$E(i,x) = \begin{cases} 5x & (i = 1) \\ (2i+1)x & (otherwise) \end{cases} \tag{1}$$

The cost is the sum of $E(i, x)$, plus X for putting trash into the bin.

In general, the movement of robot is of the following form. It consists of k phases. In each phase, the robot starts from the origin, takes a subset of trashes, and returns to the origin.

Let's fix the value of k. Since the cost for putting trash into the bin (kX) is a constant, we want to minimize the value comes from E.

To compute the value comes from E, we multiply each coordinate by some coefficient, and take their sum. The coefficient is $5, 5, 7, 9, 11, \ldots$ when the corresponding trash is the $1, 2, 3, 4, 5, \ldots$-th distant trash in its phase, respectively. Clearly, this value becomes the minimum when the coefficient 5 is assigned to the $2k$ most distant trashes, 7 is assigned to the next k distant trashes, and so on.

By using prefix sums, we can get this value in $O(N/k)$ time for a fixed k. Since $O(\sum_{i=1}^{N} \frac{1}{i}) = O(\log N)$, by trying all possible values of k, we can solve the problem in $O(N \log N)$ time in total.

Note that in some values of k you may get values greater than the limit of 64-bit integers during calculations. Be careful with overflow.

C: ABland Yard

Let s be the infinite repetition of the string `AABB`. If we can't make s, the answer is `No`. If we can make s, it turns out that we can make arbitrary strings.

Suppose that we can make s. Then, there must be a (not necessarily simple) cycle that is a repetition of `AABB`. Each vertex on this cycle is adjacent to both a vertex with `A` and the vertex with `B`. Thus, by keep moving on this cycle, we can make arbitrary strings.

There are various ways to check if we can make s. For example, we keep removing a vertex that is adjacent to only one or zero types of vertices. If a non-empty graph remains after this process, the graph contains s. It's also possible to construct an extended directed graph whose vertices correspond to pairs (the vertex in the original graph, the position in s modulo four), and check if this graph is a DAG.

D: Modulo Matrix

How can we make sure that "max mod min" is a constant? One natural way is the following.

First, paint a $N \times N$ board like a chessboard, as in the picture 1. Write arbitrary numbers on black cells. Then, on white cells, write "The LCM of all neighboring black cells" plus one. This way, the value "max mod min" is always 1.

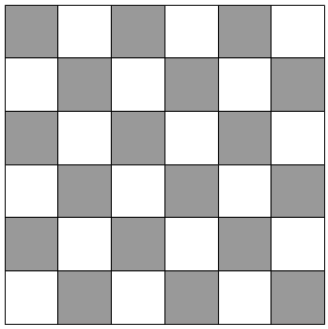


図 1　Chessboard

The main trouble is that, if we don't write numbers on white cells in a right way, the LCMs can be big and exceed the 10^{15} limit. We want to make sure that the LCM of four white numbers (that are adjacent to the same black cell) is always small.

One possible way is as follows. See the following picture. A white cell is always at the intersection of two diagonals. There are $2N$ diagonals in total, and we assign a small prime number on each diagonal. The value in a white cell is the product of two primes assigned to the two diagonals passing through it.

Then, the numbers on black cells will be "the product of four small primes" plus one, which is small enough. (The 1000-th prime is 7919. For example, you can assign the first 500 primes to one direction, and the next 500 primes to the other direction.)

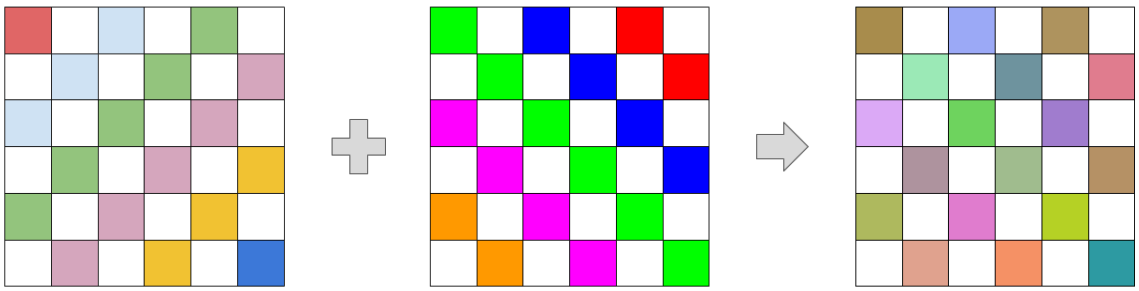


図 2　Diagonals

E: ABBreviate

Let $p(s)$ be the sum of all letters in s, modulo 3. Here, we assume that `a` is 1 and `b` is 2. This is an invariant.

When can we convert a string s into a single letter c? It turns out that the conditions are folloing $(*)$:

- $s = c$. or,
- $p(s) = p(c)$ and s contains two adjacent same letter.

It's clear that these conditions are necessary. Let's prove that this is sufficient. Suppose that $|s| \geq 2$, $p(s) = p(c)$, and s contains two adjacent same letter (i.e., s is not like "ababab..."). Then, by observing a few cases carefully, it turns out that we can always keep performing operations on this string until $|s|$ becomes 1. Since p is an invariant, when $|s|$ becomes 1, it must be c.

We can restate the original problem as follows:

> You are given a string s. Count the number of strings t that satisfies the following:
> By partitioning s into $|t|$ intervals properly, the condition $(*)$ holds for each corresponding pair of an interval and a letter in t.

Let's fix a string t, and check if we can divide s into $|t|$ parts as described above.

First, in case t doens't have two adjacent same letters, the condition is obviously $s = t$. Otherwise, we do this as follows. For each $j = 1, 2, \ldots, |t|$ in this order, take the shortest possible interval of s such that the value of $p(\cdot)$ matches t_j. For example, when $s =$ `aaababbbabababbaaaba`, $t =$ `abab`, the intervals are $s =$ `a` `aa` `babb` `abababb` `aaaba`. (`aaaba` is the remaining part). It turns out that the condition is equivalent to the following: we can successfully take all $|t|$ intervals and the $p(\cdot)$ value of the remaining part is 0.

Again, it's clear that these conditions are necessary. Let's prove that this is sufficient. Since the p value of the remaining part is 0, by attaching it to the last interval, we can almost always get a desired partition. The only concern is that, this way the last part can be of the form "ababab...". However, it's not hard to see that this can be avoided by properly rearranging the intervals.

We can count such t by a simple DP. This solution works in $O(|s|)$ time.

F: Grafting

Suppose that there exists a vertex r such that we never perform an operation on it (i.e., paint it black). (It's possible that we perform an operation on each vertex exactly once - this case is harder, and we handle it later.)

Once we fix such r, we can get almost all information about the operations, as follows:

- Suppose that in both A, B a vertex v is adjacent to the root r. What happens if we move this edge? First, to move this edge, we must perform an operation on v (because we are not allowed to perform an operation on r). Then, immediately before we span an edge between v and r again, we must perform an operation on r again. However this means that we perform two operations on v. Thus, we must never move the edge between r and v.
- Suppose that in both A, B a vertex u is adjacent to v (as mentioned above, it is adjacent to the root in both trees). Then, by a similar reason, we must never move an edge between u and v. This way, by running a DFS from the root, we can get a set of edges that must not be moved. Let's call them "fixed edges".
- If an edge between vertices a and b exists in one of the trees but not in the other tree, this edge must be moved. All edges adjacent to "fixed edges" we got from the DFS above satisfy this property (otherwise more edges will be included in the set of fixed edges by DFS), so all such edges must be moved.
- Consider two fixed edges and a path between them. All edges on the path can't be moved because their endpoints can't be leaves. Thus, the set of fixed edges are connected. This means that we now fully get the information about fixed edges: the edges we got from the DFS are fixed, all other edges are not fixed (and must be moved).
- If a vertex is incident to at least one fixed edge, we can never perform operations on the vertex, because whenever it becaomse a leaf it is incident to a fixed edge. We call such vertices "fixed vertices". Now we get all information about fixed edges and vertices - and by their definition, the number of fixed edges and the number of fixed vertices are always the same.
- We can also uniquely determine "directions". Suppose that r is the root of both trees. In this tree, suppose that there is an unfixed edge between x and y (and x is closer to the root). Then, the only way to make sure that all unfixed edges are moved is to move it from "children", that is, to move the edge between x and y, we must choose y as the leaf.

Now, we know which edges are moved by operations (and each edge is moved exactly once). Also, for each edge, we know from which vertex the edge is moved. (If the parent of x is y in A and z in B, when we perform an operation on x, we must move an edge from $x - y$ to $x - z$.) The only remaining thing we have to decide is the order of operations.

An order of operations is valid if whenever we perform an operation on v, v is a leaf. This condition can be restated as constraints of the form "we must perform an operation on this vertex before on this vertex". If there is no cyclic dependencies among such constraints, a valid order exists.

Thus, for a fixed r, we can get the answer in $O(N)$ time.

The following picture 3 shows the first case of sample 2.

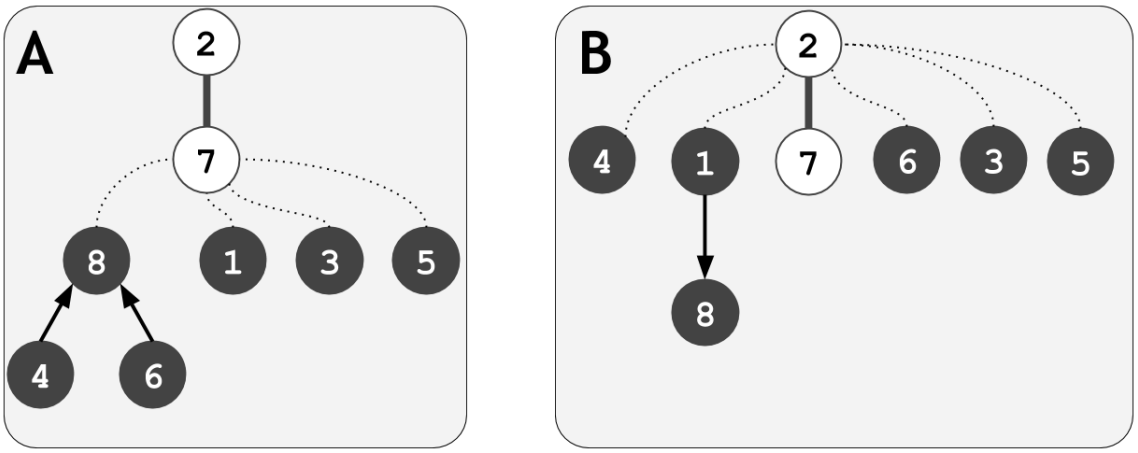


図 3　Sample 2, first case

As mentioned above, in some cases the answer is N. The picture 4 shows such an example.

To handle it, let's try all possible valid moves for the first move - there are only $O(N^2)$ moves. If the vertex v is painted black in the first move, we never perform operations on this vertex again in the future. Thus, we can regard this v as the root in the case above, and we can solve it in $O(N)$.

In total, this solution works in $O(N^3)$ time per testcase.

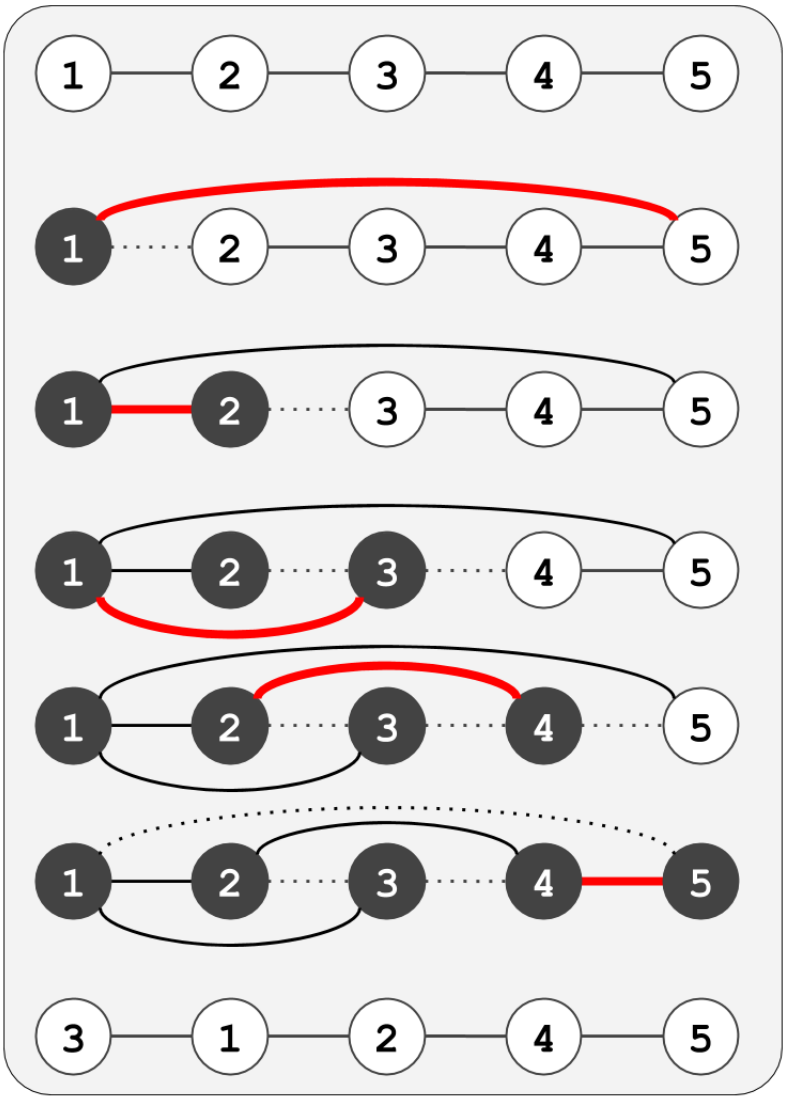


図 4　The answer is N in this case

AGC 026 Analysis

sugim48[*] yosupo[†]

2018 年 7 月 16 日

[*] CDEF Author

[†] B Author, Setter, Analysis Writer

A: Colorful Slimes 2

同じ色が奇数匹並んだ区間については，スライムの色を 1 匹飛ばしで適当に違う色に変えればよいです (例: $(2,2,2,2) \to (2,3,2,3)\text{or}(2,1,2,1)\text{or}, ...)$。同じ色が偶数匹並んだ区間についても，スライムの色を 1 匹飛ばしで適当に違う色に変えます (例: $(2,2,2,2,1,...) \to (2,3,2,3,1,...)$)

このように色を変えるのが最適です。つまり，同じ色のスライムが x 匹並んでいたならば，$\lfloor x/2 \rfloor$ 匹の色を変更します。($\lfloor y \rfloor$ で，y の小数以下を切り捨てた値を表すこととします。)

よって解法は，同じ色のスライムが並んでいる区間ごとに列を分割し (例: $(1,1,2,2,2,3) \to (1,1),(2,2,2),(3)$), 各区間ごとに $\lfloor x/2 \rfloor$(x は区間の長さ) を求め，それを足し合わせたものを求めればよいです。計算量は $O(N)$ です。

B: rng_10s

まず，場合分けで以下の 3 つのケースを取り除きます

- $B > A$: 初日でジュースが買えないため，答えは No です
- $(B \leq A), B > D$: 仮に毎日入荷したとしても購入量に追いつけないため，答えは No です
- $(B \leq A, B \leq D), C \geq B$: 買う量がしきい値以下なため、買切る前に必ず入荷が発生し、(入荷の個数は購入量以上なため) 答えは Yes です。

以上より、$B \leq A, B \leq D, C < B$ を仮定してよいです。

次に，個数の遷移を $\mathrm{mod} B$ で眺めてみます。

- 最初は (個数 $\mathrm{mod}\ B$) は $A \bmod B$ 個です。
- すぬけ君が買っても (個数 $\mathrm{mod}\ B$) は変化しません。
- 入荷すると (個数 $\mathrm{mod}\ B$) は ($\mathrm{mod} B$ の世界で)$D \bmod B$ 個増えます。

そして、この問題の答えは、(個数 $\mathrm{mod}\ B$) が C を超えるタイミングがあるかどうかを判定すればよいことがわかります。このようなタイミングがあれば、その後買えなくなることは簡単にわかり、逆にこのようなタイミングでなければ必ず入荷が発生するため、買いきれません。

以上より、この問題は次のように言い換えられます。

- A からスタートして、D を足していく。個数 $\mathrm{mod}\ B$ が C を超えることはあるか？

A からスタートして、D を足していくとき、個数 $\mathrm{mod}\ B$ の最大は, $g = \gcd(B, D)$ として $B - g + (A \bmod g)$ となり，これを C と比較すればよいです。個数 $\mathrm{mod}\ g$ が常に一定であることを考えるとこれが上界であることは言え、また，$(B/g - 1) \times \mathrm{inv}(D/g, B/g)$ 回 D を足したときにこの上界が達成できます ($\mathrm{inv}(X, Y)$ は，$X \times \mathrm{inv} = 1 \bmod Y$ なる値とします)。

C: String Coloring

文字列を左から N 文字と右から N 文字に分割します。すると，

- 左側で赤く塗られた文字を左から右へ読んだものと，右側で青く塗られた文字を右から左へ読んだものが等しい
- 左側で青く塗られた文字を右から左へ読んだものと，右側で赤く塗られた文字を左から右へ読んだものが等しい

の 2 つが達成された時，またこのときのみに元の問題の条件を達成できることがわかります。

よって，最初に右側の文字列を反転しておき，また右側では赤と青を逆に塗り分けることにすると，条件は以下のようになります。

- 左側で赤く塗られた文字を左から右へ読んだものと，右側で赤く塗られた文字を左から右へ読んだものが等しい
- 左側で青く塗られた文字を右から左へ読んだものと，右側で青く塗られた文字を右から左へ読んだものが等しい

要するに，各色で，左右で塗られた文字たちが全く同じ文字列になるように塗り分ける方法は何通りですか？という問題です。

これは，左右でそれぞれ塗り分け方を全探索し，文字列のペアを key，それを達成する方法を value としたハッシュマップを作れば解くことが出来ます。

ハッシュマップの要素数は高々 2^N なので，$O(N^2 2^N)$ で解くことが出来ます。

D: Histogram Coloring

まずヒストグラムが単一の長方形の場合を考えます。この場合，どのような塗り分け方があるでしょうか？
i 行目より下が全部塗られていて，$i+1$ 行目の塗り分け方を考えているとします。

- (i 行目の色に関わらず，)$i+1$ 行目の色として，i 行目の色を全部反転させたものを塗る
- i 行目の色が赤青赤青..., もしくは青赤青赤... の時，$i+1$ 行目を i 行目の色と全く同じように塗る

この 2 パターンの塗り分け方があることがわかり，またこの場合が全てであることがわかります。

以上の考察を使い，まずヒストグラムを，(最小の値を求め，それを全体から引き，0 の要素で左右に分割する) を繰り返し，長方形からなる木に分解します。

そして木のノードごとに，

- $dp1$:= 長方形とその上の長方形たちの塗り分け方の総数，ただし一番下の列は赤青赤青..., 青赤青赤..., のどちらかの塗り方でないといけない
- $dp2$:= 長方形とその上の長方形たちの塗り分け方の総数

の 2 種類の値を持って，木 DP を行います。

長方形 h の高さを x，子を $c_1, c_2, ..., c_k$，h の幅から子の幅を引いたものを w とすると，遷移式は

- $dp1_h := 2^x \Pi_{i=1..k} dp1_{c_i}$
- $dp2_h := 2^w \Pi_{i=1..k} (dp1_{c_i} + dp2_{c_i}) + (2^x - 2) \Pi_{i=1..k} dp1_{c_i}$

となります。

計算量は木を作るのに $O(N^2)$，その後の木 DP で $O(N \log h)$ です。ただし木は $O(N)$ で作ることも出来ます。

E: Syncronized Subsequence

$S = b_1b_2a_1b_3a_2a_3$ と表記します。

まず，$a = -1, b = 1$ と考え累積和を取り，累積和が 0 になるところで，文字列を分割します。すると，異なる文字列どうしの間に a, b のペアが存在することがないため，取れる部分列というのは文字列ごとに独立になります。もちろん最後にくっつけるため，どのような部分列を文字列ごとに取ればよいのか，というのは非自明なのですが，実は空文字列 or 辞書順最大の文字列の 2 パターンだけ考えればいいことが示せます。

よって，文字列を最初に分割し，独立に元の問題を解けばいいです。

■最初の文字が'a' 仮定より，全ての i について i 番目の a は i 番目の b より先に出現します。

答えの文字列の形は必ず $ababab...$ という形になっています。このような形でないならば，必ず a が隣り合ったところが存在するはず (最初の文字が a で最後の文字が b であることを考えると，鳩の巣原理より) ですが，この場合 $...a_ia_{i+1}...b_i...b_{i+1}...$ という位置関係になっているので，$i+1$ を消したほうが必ず良くなるからです。

よって，最大でどのぐらいの長さの $ababab...$ を作れるかを考えればよいです。これは，a_1, b_1 間の a と，それに対応する b をすべて消し，$a_1b_1...$ という文字列が出来るのですが，この ... について同じことをやる，というのを繰り返せばよいです。

■最初の文字が'b' 仮定より，全ての i について i 番目の b は i 番目の a より先に出現します。

残す i のうち最小を決めたとします。$1, 2, ..., i-1$ を削除したら，b_i と a_i の間には b しか残りません。この b(と，それに対応する a) は当然全て残すべきです。また，この操作により新たに残すことになった a たちの前に b が存在したら，それも当然全て残すべきです。…というのを繰り返していくと，実は累積和が最初と最後以外 0 にならないという仮定より，i 以降全てを残すのが最適であることがわかります。よって，i を全部決め打ち，最も辞書順が大きい文字列を返せばよいです。

最後の結合ですが，後ろから使うか使わないかを順に決め，DP をすれば $O(N^2)$ で解くことが出来ます。今回は文字列の形が特殊で，greedy でも解くことが可能で，この場合 $O(N)$ になります。

計算量は $O(N^2)$ ですが，実は $O(N)$ で解くことも可能です (ただし，Suffix Array を線形時間で構築できることを要求します)。

F: Manju Game

■N が偶数　N が偶数の場合は証明は省略しますが、偶数番目の sum と奇数番目の sum の max が先手の取れる最大です。

■N が奇数　偶奇が重要なため，数列を $a_1b_1a_2b_2...b_ma_{m+1}$ と表すことにします。

- sugim 君は，もし a を選択するならば，端の a を選択し，$\sum a$ を回収してゲームを終了するのが最善です。これは端以外の a を選択した場合であっても，sigma 君は少なくとも $\sum b$ は回収できてしまうことから言えます。
- sugim 君が b を選択した場合，sigma 君が左右どちらかを選び，選んだほうの a を sigma 君が，b を sugim 君が回収し，そして選ばなかった方の数列で同じゲームをすることになります。

以上の考察より，このゲームは，

- sugim 君は，b を 1 つ選択する，そして sigma 君は左右のどちらかを選択する，選択した方の数列 (と sugim 君が選んだ b) は切り落とし，残った数列で同じことをする。
- sugim 君は b を選択せずゲームを終了することもできる。その時は $\sum_{\text{残った数列}} a - \sum_{\text{残った数列}} b + \sum b$ を最終得点として獲得する。

と言い換えられる事ができます。また，sugim 君は $Z = \sum_{\text{残った数列}} a - \sum_{\text{残った数列}} b$ を最大化するゲームとして考えて良いです。ここで、答えで二分探索を行い，ある X について $sugim$ さんが $Z \geq X$ と出来れば勝ちで、そうでなければ負けという勝ち負けの付くゲームへ変換します。すると，この問題は以下のように解けます

■数列の長さが 1 の場合　その要素が X より大きければ sugim の勝ちです。

■数列の長さが 3 以上の場合　左端の 2 要素 a_1, b_1 に注目します。

もし $a_1 \geq X$ かつ $b_1 \geq a_1$ だとしましょう。この場合，sugim 君が最適に行動すると，絶対に a_1, b_1 は切り落とされた状態でゲームが終了するとしてよいです。なぜならば，a_1, b_1 を切り落とさないままゲームを終了する場合，b_1 を選択し，a_1, b_1 を切り落としてからゲームを終了してもスコアが低くならないからです ($a_1 \geq X$ より，sigma 君は b_1 を選択された時に左しか選べないことに注意して下さい)。

よって，sugim 君は最初に b_1 を選択して a_1, b_1 を切り落としてしまって良く，a_2 以降について同じ問題を解けばよいです。

では次に，逆のパターン，つまり $a_1 < X$ または $b_1 < a_1$ の場合を考えます。

- $a_1 < X$: b_1 を選択して sigma が右を選択すると負けるため，明らかに sugim は b_1 を選択出来ません。
- $(a_1 \geq X), b_1 < a_1$: b_1 を選択すると，sigma 君は左を選択するしかなく，a_1, b_1 が切り落とされます。しかし $b_1 < a_1$ より，最適に行動した時，a_1, b_1 が切り落とされており a_2 が切り落とされていないという状態になることはありません。よって sugim は b_1 を選択する旨味がなく，選択しないものとしてよいです。

よって，sugim 君は結局 b_1 を選択することはありません。つまり，a_1, b_1, a_2 は全部切り落とされるか全部生

き残るかのどちらかなので，$a_2 = a_1 + a_2 - b_1$ とし，a_1, b_1 を切り落とし，同じ問題を解けばよいです。

以上より二分探索後の問題は $O(N)$ で解け，まとめて $O(N \log(\sum a))$ で解けます。

A: Colorful Slimes 2

If multiple slimes of the same color are adjacent, we shuold change the color of all even-indexed slimes. For example, $(2, 2, 2, 2) \rightarrow (2, 3, 2, 3)$ or $(2, 2, 2, 2, 2, 1...) \rightarrow (2, 3, 2, 3, 2, 1, ...)$.

Generally, if there is a run of x slimes of the same color, we should change the color of $floor(x/2)$ slimes.

Thus, we should split the array into runs (for example, $(1, 1, 2, 2, 2, 3) \rightarrow (1, 1), (2, 2, 2), (3)$), and the answer is the sum of $\lfloor length/2 \rfloor$ for each part. This solution works in $O(N)$.

B: rng_10s

There are two types of obvious cases:

- $B > A$: since we can't buy the juice in the first day, the answer is "NO".
- $(B \leq A), B > D$: the number of juice in the evening is monotonously (strictly) decreasing, so the answer is "NO".

Thus, assume that $B \leq A, B \leq D$.

The number of cans of juice change as follows:

- Initially, it starts with $A - B$.
- In case the number is x at some evening, the number in the next evening is $x - B$ if $x > C$, and $x - B + D$ if $x \leq C$.

Thus, for sufficiently large i, we know the following about the number of cans in the i-th evening:

- In modulo D, it is equivalent to $A - iB$.
- It is in the range $[C - B + 1, C - B + D]$.

From these two conditions we can uniquely determine the number of cans, and its minimum is the minimum number x such that

- $x \geq C - B + 1$.
- $x \equiv A \pmod{gcd(B, D)}$.

The answer is "YES" when this x is non-negative.

C: String Coloring

Let's cut the string into two halves (N characters each).

The condition in the statement can be restated as follows:

- The string we get by reading all red letters in the left part from left to right is equal to the string we get by reading all blue letters in the right part from right to left.
- The string we get by reading all blue letters in the left part from right to left is equal to the string we get by reading all red letters in the right part from left to right.

Let's reverse the colors of letters in the right half, and also reverse the order of letters in the right half. Then, this condition beocomes the following:

- The string we get by reading all red letters in the left part from left to right is equal to the string we get by reading all red letters in the right part from left to right.
- The string we get by reading all blue letters in the left part from left to right is equal to the string we get by reading all blue letters in the right part from left to right.

For each half, try all possible colorings (there are 2^N ways), and for each coloring assign a pair of two strings (the red letters from left to right, the blue letters from left to right). We can solve the problem by putting these pairs into a map.

This solution works in $O(N^2 2^N)$ time.

D: Histogram Coloring

Suppose that the histogram is a reactangle. How can we color it in this case?

Suppose that we decided the colors for all cells in row i, and we now want to color cells in row $i+1$ (while satisfying constraints for all 2x2 squares on these two rows). There are two possible ways:

- (Regardless of the colors of the row i), for each j, the color of $(i+1, j)$ is different from the color if (i, j).
- In case row i is either RBRB... or BRBR..., for each j, the color of $(i+1, j)$ is the same as (i, j).

This suggests the following solution. For a histogram H, define the following two values:

- $dp1(H)$:= The number of ways to color this histogram, with the additional constraint that the bottommost row of H must be either RBRB... or BRBR...
- $dp2(H)$:= The number of ways to color this histogram.

Keep these two values and do DP.

Consider a histogram H. We'll decompose it into multiple sub-histograms, compute the values of $dp1$ and $dp2$ for each of the sub-histograms, and then compute $dp1(H)$ and $dp2(H)$.

Let x be the height of the shortest column of H. If we remove the bottom x rows from H, it may be splitted into multiple parts: call them $c_1, c_2, ..., c_k$. Let w be the number of completely removed columns (i.e., the number of columns with height exactly x).

Then, we can compute the values of $dp1(H)$ and $dp2(H)$ using the following recursion:

- $dp1_h := 2^x \Pi_{i=1..k} dp1_{c_i}$
- $dp2_h := 2^w \Pi_{i=1..k} (dp1_{c_i} + dp2_{c_i}) + (2^x - 2) \Pi_{i=1..k} dp1_{c_i}$

This solution works in $O(N \log h)$ time (or $O(N^2)$, depending on the way you decompose the histogram).

E: Syncronized Subsequence

We call the i-th occurrence of 'a' "a_i" (and define b_i similarly).

Let's split the string into as many parts as possible, such that each part contains the same number of 'a's and 'b's. For each i, a_i and b_i belong to the same part.

First, let's consider a single part. There are two cases in a single part:

- For all i, $a_i < b_i$.
- For all i, $a_i > b_i$.

(Otherwise the part can be splitted into multiple parts).

First, assume that the entire string contains a single part.

■In case $a_i < b_i$ for all i The answer will always be of the form $ababab...$.

Otherwise, there must be two consecutive 'a's in the answer (because the answer string starts with 'a', ends with 'b', and contains the same number of 'a's and 'b's). It will look like $...a_i a_j ... b_i ... b_j ...$. However, in this case we can get a better result by erasing both a_j and b_j.

What is the longest possible length of the string of the form $ababab...$ we can get? This can be done greedily: we should choose a_1 and b_1, then we should choose a_i and b_i where i is the minimum index such that a_i is to the right of b_1, and so on.

■In case $a_i > b_i$ for all i Suppose that we choose a_i and b_i, but not a_{i+1} and b_{i+1}, for some i. In this case, we can always improve the result by inserting both a_{i+1} and b_{i+1}. (Notice that these characters appear in the order $...b_i...b_{i+1}...a_i...a_{i+1}...$).

Thus, in the optimal answer, we should choose $a_i, b_i, a_{i+1}, b_{i+1}, ...$ (i.e., all characters indexed with i or greater). We can get the optimal answer by trying all values for i.

What should we do when the string contains multiple parts?

We split the string into parts, and for each part, we get the optimal answer as described above. Then, for each part, we should either choose the optimal answer or choose an empty string. This is because, in case of $a_i > b_i$, all other strings we can get is not a prefix of the optimal string.

Thus, by doing DP from right from left, we can get the optimal string in $O(N^2)$. (It's also possible to do it in $O(N)$ if we use the property of strings we get this way: we should choose a string s if it's greater than or equal to all strings after that.)

This solution works in $O(N^2)$ time ($O(N)$ is also possible with Suffix Arrays).

F: Manju Game

Let's color the boxes black and white. The 1st, 3rd, 5th, ... boxes are black, and the 2nd, 4th, 6th, ... boxes are white. Let B be the sum of values in all black boxes and let W be the sum of values in all white boxes.

■In case N is even If the first player takes the leftmost (rightmost) box, all remaining moves will be forced and the first player's score will be B (W). Thus, the first player can make sure to get at least $max(B, W)$ points.

Suppose that the first player takes a black box in the first turn. Then, if the second player takes its left box, all moves will be forced until all boxes to the left are taken; then, if the second player takes the rightmost box, he can make sure to get at least W points. Similarly, if the first player takes a white box in the first turn, the second player can ensure B points.

Thus, the first player's score will be $max(B, W)$ and the second player's score will be $min(B, W)$ in this case.

■In case N is odd By a similar observation, we get the following:

- The first player can ensure B points.
- In case the first player takes a black box in the first turn, the second player can ensure W points.

Thus, the main question is whether the first player can get more than B points by taking a white box in the first turn. If the first player starts with a white box, the game will proceed as follows:

- Initially, the first player chooses a white box and takes it (call it p).
- The second player chooses "left" or "right".
- For example, if the second player chooses "left", the game proceeds as follows. The first player takes p and all white boxes to the left of p. The second player takes all black boxes to the left of p. Then, they start the same game with all boxes to the right of p, starting from the first player.

In this game, the strategy of the first player can be completely described using a binary tree. The root of the tree corresponds to the white box the first player takes in the first turn. The root's left subtree corresponds to the game in case the second player chooses "right" in his first turn - i.e., the root's left child is the white box the first player takes in his second turn (among all boxes to the left of the root), in case the second player chooses "right" in his first turn. The root doesn't have a left child if the first player wants to start taking black boxes in this case. Similarly, define its right subtree.

Suppose that a binary tree (that describes the first player's strategy) is given. How many points can he ensure with this strategy?

Let $W_1, \ldots, W_k$ be the white boxes that appear in this tree, from left to right. The first player will take all these boxes, no matter what the second player does. These k boxes split the sequence of boxes into $k+1$ parts. For one of these parts, the first player takes all black boxes and the second player takes all white boxes. For all other parts, the first player takes all white boxes and the second player takes all

black boxes. The choice of the "one part" depends on the second player's strategy.

Let's fix a parameter X. The first player can ensure $W + X$ points if he can choose white boxes $W_1, \ldots, W_k$ such that for each of $k+1$ parts we get when we split the sequence with chosen white boxes, (The sum of black boxes in the part) minus (The sum of white boxes in the part) is at least X.

Let's do a binary search on X. We can check if we can choose white boxes as described above in $O(N)$ by a simple DP. This solution works in $O(N \log(\sum a))$ time in total.

AGC 25 解説

yutaka1999

2018 年 6 月 3 日

For International Readers: English editorial starts at page 7.

A : Digits Sum

求める答えは、N が 10 のべき乗でないとき N の各位の和、N が 10 べきの場合は 10 となります。

N の各位の和が下限になるのは明らかです。（筆算を考えましょう。）ただし、N が 10 べきの場合は明らかに構成不可能なので、10 が答えになります。これも、筆算から明らかです。

B : RGB Coloring

G で塗られたブロックを、R と B で同時に塗られたブロック、と考えることにします。こうすることで、3 色で塗るのではなく、各ブロックを 2 色で塗る、ただし、同じマスを 2 回塗ってもよいという問題になります。

よって、この問題は以下のような問題になります。

> $A \times a + B \times b = K$ となる $0 \leq a, b \leq N$ に対する ${}_NC_a \times {}_NC_b$ の和を求めよ。

これは a を全探索することで全体として $O(N)$ の時間計算量で解くことができます。よって、この問題を解くことができました。

C : Interval Game

高橋君の最適な移動の仕方は、すでに含まれている場合は動かない、そうでない場合は、与えられた区間の近い方の端点まで動く、というものです。これは、区間の近い方の端点まで移動してからまた動くとしたら、そのステップが終わったあとにその分を動けばよいからです。

さて、高橋君の最適戦略は分かったので、青木君の最適戦略を考えましょう。ここで、青木君はすべての区間を使う必要がないとしてもよいです。つまり、何個かの区間を余して途中でやめたとしても、答えが大きくなることはありません。これは簡単に証明できます。

この明らかなことが何に使えるかということですが、実は青木君は不必要な区間を考えないことで、高橋君の動きを単純化することができます。まず、高橋君が 2 回連続で同じ方向、つまり、連続で右に移動したり連続で左に移動する場合は無視できます。これは、先の高橋君の最適戦略を考えれば明らかです。よって、青木君は不必要な区間を無視することで、高橋君が移動する向きを毎回変えるように動くようにすることができます。このことを考えると、青木君は高橋君を右に動かすための区間と左に動かすための区間の 2 つの役割に、各区間を分けて考えることができるようになります。

つまり、各区間を、右向きに L_i まで動かす、または、左向きに R_i まで動かすためのものとして考えることができます。そして、移動距離への影響はどうなるかというと、右向きに使うなら $2L_i$、左向きに使うなら $-2R_i$ かかります。よって、その影響が大きいものから順番に貪欲に使っていけばよさそうです。ただし、右向きに使うものと左向きに使うものの個数の差は 1 以内に収まらなければならないので、その条件を満たすように貪欲に取らなければいけません。

ここで気になるのは、同じ区間を両方の向きで使ってしまわないかということですが、各区間について $2L_i - 2R_i < 0$ なので、同じ区間を両方の向きで使うことが最適になることはありません。

以上をまとめると、右向きに使う個数と左向きに使う個数を決め打ったときに、どの区間を用いるかは貪欲に使っても問題ないということです。よって、後は適当にソートなどをすることで、$O(N \log N)$ の計算量で解くことができます。

D : Choosing Points

$4N^2$ 個の点から N^2 個の点を選ぶ、しかも制約が 2 個、という時点で勘がいい人は気づくかもしれませんが、これは 2 つの二部グラフが与えられたときに、どちらでも独立集合となる $\frac{V}{4}$ サイズの頂点集合を求める問題です。

ということで本質は格子点の集合が与えられたときに、距離 $\sqrt{D}$ である 2 点間に辺を張ってできるグラフが 2 部グラフとなることです。これを D の帰納法により示していきましょう。

D が奇数のときは簡単です。2 点間の距離が $\sqrt{D}$ になるとき、x 座標と y 座標の差をそれぞれ s, t とすると、$s^2 + t^2 = D$ が奇数より、s, t の偶奇が異なることになります。つまり、各点 (x, y) を $x + y$ の偶奇で 2 色に塗ることを考えると、それらが今考えてるグラフの 2 彩色になります。よって、D が奇数のときはできました。

次に、$D \equiv 2 \pmod 4$ のときを考えます。このときは、x 座標と y 座標の差をそれぞれ s, t とすると、$s^2 + t^2 \equiv 2 \pmod 4$ より、s, t がともに奇数となります。よって、各点 (x, y) を x の偶奇で 2 色に塗ることを考えると、それらが今考えてるグラフの 2 彩色になります。よって、$D \equiv 2 \pmod 4$ のときもできました。

最後に、$D \equiv 0 \pmod 4$ のときを考えましょう。同様にすると、s, t はともに偶数になります。よって、各点 (x, y) を $(x \bmod 2, y \bmod 2)$ で振り分けて、それぞれを適当に $\frac{1}{2}$ 倍に収縮すると、$\frac{D}{4}$ の場合に帰着することができ、帰納的に 2 部グラフになることが示せます。

以上より、距離 $\sqrt{D}$ である 2 点間に辺を張ってできるグラフが 2 部グラフであると示せたので、あとは最初に述べた問題を解けばよいだけです。

まず、各点についてそれぞれの距離で 2 彩色のどちらに属するかを求めます。それによって、$2 \times 2 = 4$ パターンの塗り分けが生じますが、そのうち一番点が多いパターンを選ぶと、そこには $\frac{4N^2}{4} = N^2$ 個以上の点が属しているので、同じパターンだけから N^2 個の点を選ぶことができます。すると、それらの間はどれも距離 $\sqrt{D_1}, \sqrt{D_2}$ にならないので、条件を満たします（これはグラフの作りかた、および、彩色の仕方から明らかです）。よって、この問題を解くことができました。

計算量はグラフの構成が一番重いですが、粗い評価でも $O(N^3)$ に収まります。実際は距離が $\sqrt{D_1}, \sqrt{D_2}$ になるような (s, t) の候補が少ないので、十分高速に動きます。

E : Walking on a Tree

まず、答えの上界を考えてみましょう。木の各辺 i に対して、その辺を通る散歩の個数 c_i を考えます。このとき、明らかに $\sum \min(c_i, 2)$ は上界となります。というのも、各辺につき向きは 2 通りしかないので、その辺を通ることで得られる楽しさは高々 2 であるからです。

では実際にこの上界を達成できることを示しましょう。これは帰納法によって示すことができます。まず、$n = 1$ のときはそもそも辺がないので明らかです。よって、n がより小さい場合に帰着できればよいです。

まず、適当な葉を v とし、v に隣接する辺 e を考えます。$c_e = 0$ の場合は v を無視してしまえばよいから明らかですし、$c_e = 1$ の場合も、v を端点にもつ散歩の端点を、v から e のもう一方の端点に変えればよいので、明らかに帰着できます。なので、$c_v \geq 2$ の場合を考えるとしてよいです。このとき、v を端点に持つ散歩を 2 つ取れます。それらの v でない方の端点を a, b としましょう。このとき、これらの散歩の向きを、$a \to v \to b$ もしくは $b \to v \to a$ と定めることで、これらの散歩の共通部分は両向きで通ったことにしたうえで a から b への散歩を追加する、という状況と同一視できます。よって、v を通る残りの散歩も、端点を v から e のもう一方の端点とみなすことで、$n - 1$ サイズの場合に帰着することができます。よって、帰納的に上界を達成できることが示せました。

実際の構成は上の方法を素直に実装すれば各ステップ $O(N + M)$ で可能です。なので、全体としては $O(N(N + M))$ の計算量で解くことができます。

F : Addition and Andition

X, Y の 2 進表記に対応する文字列 S, T を、説明の都合上以下のような配列として定義しなおします。

- X の i bit 目が 1 ならば $S_i = 1$、0 ならば $S_i = 0$ とする。
- Y の i bit 目が 1 ならば $T_i = 1$、0 ならば $T_i = 0$ とする。

ここで、問題文に書かれている操作を配列に対する操作に言い換えると、以下のようになります。

- $S_i = T_i = 1$ となる i すべてに対して $S_i = T_i = 0$ とする。
- 次に、先の i すべてに対して S_{i+1} と T_{i+1} に 1 を足す。
- その後、S_i, T_i の繰り上がりを処理していく。

ここで、「繰り上がりを処理」する方法はいくつかありますが、以下のようにして行うことを考えてみましょう。

- i を大きい方から小さい方へ順に見ていって、$S_i = 2$ となる i を見つけたら以下を繰り返す。
  - j を $i, i+1, i+2, ...$ とインクリメントしていく。
  - $S_j = 2$ なら $S_j = 0$ にして S_{j+1} に 1 を足す。
  - そうでないなら j に対する操作をやめて、i のネストに戻る。

この方法で繰り上がりを計算することを考えると、問題文の操作は次のように言い換えられます。

- i を大きい方から小さい方へ順に見ていって、$S_i = T_i = 1$ となる i を見つけたら $S_i = T_i = 2$ として以下を繰り返す。…(X)
  - j を $i, i+1, i+2, ...$ とインクリメントしていく。
  - $S_j = 2$ なら $S_j = 0$ にして S_{j+1} に 1 を足す。
  - $T_j = 2$ なら $T_j = 0$ にして T_{j+1} に 1 を足す。
  - どちらでもないなら j に対する操作をやめて、i のネストに戻る。

この操作を K 回繰り返した後の S, T の状態を求めるのがこの問題です。これを愚直に表すと以下のようになります。

- $t = 0, 1, ..., K-1$ に対して、以下を行う。
  - i を大きい方から小さい方へ順に見ていって...

そこで一つの仮説が立ちます。つまり、このループの順番を逆にしても得られる最終状態は同じではないかということです。ここで、ループの順番を逆にするとは、以下のようにして操作を行うことを指しています、

- i を大きい方から小さい方へ順に見ていって、以下の操作を行う。
- ただし、残り何回操作できるかを表すカウンター z 持っておく。最初は $z = K$ である。
  - j を $i, i+1, i+2, ...$ とインクリメントしていく。
  - $S_j = T_j = 1$ かつカウンター z が正なら、$S_j = T_j = 0$ として S_{j+1}, T_{j+1} に 1 を足し、z から 1 を引く。ただし、$z < 0$ ならばその操作を行わずに終了する。
  - $S_j = 2$ なら $S_j = 0$ にして S_{j+1} に 1 を足す。
  - $T_j = 2$ なら $T_j = 0$ にして T_{j+1} に 1 を足す。
  - いずれでもないなら j に対する操作をやめて、i のネストに戻る。

なぜこのように、ループの順番を変えてもいいのかを示しましょう。注目すべきなのは、操作 (X) において、操作後は $S_i = T_i = 0$ となることです。これを使うと、今注目してる i より小さなところで操作 (X) を行ったとしても、増える値の合計は高々 $2^i - 1$ であるから、i bit 目より上の bit には影響を与えないことが分かります。この事実を用いると、$S_i = T_i = 1$ なる最大の i を考えたときに、その i に対して (X) を行った際、i より上の位 j で $S_j = T_j = 1$ となって終わったとすると、i より小さいところでの操作をする前に先に j に対する操作を行ってしまっても、結果が変わらないことが分かります。よってこれを繰り返し用いることで、ループの順番を変える、つまり、先にあげたような操作をした際の最終状態を求めればよいと分かります。

後はループの順番を変えた後の操作について解けばいいだけです。このパートも簡単ではありませんが、先程よりは説明が簡明にできると思うので、分かりやすいと思います。まず、先程の操作を言い換えてみましょう。

- i を大きい方から小さい方へ順に見ていって、$S_i = T_i = 1$ ならば $S_i = T_i = 0$ として以下の操作を行う。
- ただし、残り何回操作できるかを表すカウンター z および下の桁から繰り上がりでくる桁を表す文字列 x を持っておく。
- 最初は $z = K, x = 11$ である。ここで、$x = 10$ ならば S のみ、$x = 01$ ならば T のみ、$x = 11$ ならば両方繰り上がりがあることを指す。
  - j を $i+1, i+2, ...$ とインクリメントしていく。
  - $(S_j, T_j, x) = (0, 0, 10)$ ならば $(S_j, T_j) = (1, 0)$ にして操作を終える。
  - $(S_j, T_j, x) = (0, 0, 01)$ ならば $(S_j, T_j) = (0, 1)$ にして操作を終える。
  - $(S_j, T_j, x) = (0, 0, 11)$ ならば $(S_j, T_j, x) = (0, 0, 11), z \mapsto z - 1$ とする。ただし、$z < 0$ となるならば $(S_j, T_j) = (1, 1)$ として操作を終える。
  - $(S_j, T_j, x) = (1, 0, 11)$ ならば $(S_j, T_j, x) = (0, 1, 10)$ にする。
  - $(S_j, T_j, x) = (1, 0, 10)$ ならば $(S_j, T_j, x) = (0, 0, 10)$ にする。
  - $(S_j, T_j, x) = (1, 0, 01)$ ならば $(S_j, T_j, x) = (0, 0, 11), z \mapsto z - 1$ とする。ただし、$z < 0$ となるならば $(S_j, T_j) = (1, 1)$ として操作を終える。
  - $(S_j, T_j) = (0, 1)$ のときも同様。

ただし、$(S_j, T_j) = (1, 1)$ となるところまで操作でたどりつかないことに注意してください。これは先のループの順序を入れ替えてよいということと同様にして言えます。

このように操作を整理してしまうと、$(S_j, T_j) = (0, 0)$ なる場所はほとんど機械的に処理できると分かります。よって、$(S_j, T_j) = (1, 0), (0, 1)$ となっている j だけを保持しておけばよさそうです。

具体的には、$(S_j, T_j) = (1, 0), (0, 1)$ となっている j を、 j が大きいほど深くにあるように、stack に積みます。この stack を A としましょう。すると操作はどうなるかというと、$S_i = T_i = 1$ なる i を見つけたときに、A の一番上を取り出して、そこまでの $(0, 0)$ のケースを処理します。そして、実際に一番上の要素まで操作でたどりつくならば、そのケースを処理して、次の要素に行きます。ただし、操作後に $(S_j, T_j) = (0, 0)$ とならない場合はその情報を stack に最後に追加しないといけないので、適当な配列に覚えておきます。これを繰り返して、先のシミュレーションを終わらせた後に、途中で生じたあまりを順に stack に積みます。これでシミュレーションが完全に実行できます。

このシミュレーションの計算量を解析しましょう。操作のアルゴリズムをよく見ると、操作後に $(S_j, T_j) = (0, 0)$ とならない場合は $x = 11$ である場合のみで、特に $x = 11$ の後は必ず $x = 10, 01$ のいずれかになります。$x = 10, 01$ の場合は A の先頭の要素数が減り、$x = 11$ の場合も増えはしないから、シミュレーションの 2 回に 1 回に A の要素数が減ることになります。また、A の要素数が増えるのは、各 i に対して $(S_j, T_j) = (0, 0)$ にぶつかって終了するときのみであるから、全体として N 回しか要素数が増えません。よって、シミュレーションの 2 回に 1 回 A の要素数がへり、A に追加される要素数の合計は N であるから、stack の要素を見る回数は高々 $2N$ 回です。よって、stack の出し入れは $O(1)$ でできることから、全体で $O(N)$ の計算量で解くことができます。ただし、「A の要素数」に途中で生じたあまりの個数も含んでいることに注意してください。

A : Digits Sum

We can compute the answer by brue force, but here's more interesting solution: the answer is 10 if N is a power of 10, otherwise the answer is the sum of digits of N.

It's clear that the answer can't be smaller than the sum of digits of N: consider computing the sum on paper. The sum of the sum of digits decreases whenever we get a carry. However, when N is a power of 10, obviously the answer can't be 1, so the answer will be 10 (it must be the same in modulo 9).

B : RGB Coloring

Let's assume that a layer painted by green is a layer painted by both red and blue (and ignore green). Then, in this problem, we paint layers by two colors (but we may paint the same layer with two colors).

The problem will be the following:

> Find the sum of $\binom{N}{a} \times \binom{N}{b}$ for all $0 \leq a, b \leq N$ such that $A \times a + B \times b = K$.

Here, a is the number of layers painted by red, and b is the number of layers painted by blue.

By trying all possible values for a, we can compute the value above in $O(N)$ time.

C : Interval Game

Takahashi's optimal strategy is as follows. If he is already contained in the new interval, he shouldn't move at all. Otherwise, he should move to the nearest endpoint of the new interval. It doesn't make sense to move more than that - instead, he can "postpone" the extra movement and the total distance he moves won't be greater.

What is Aoki's optimal strategy? Since we know Takahashi's optimal strategy, we can regard the game as Aoki's single-player game. Whenever Aoki puts an interval, Takahashi moves in a deterministic way (as we mentioned above), and Aoki wants to maximize the total distance Takahashi moves. Let's modify the rule of the game a bit: Aoki doesn't necessarily have to put all intervals (and after he stops putting intervals, Takahashi returns to the origin). Anyway, since ignoring some intervals never make Takahashi's distance greater, the answer won't change.

Now, since we can omit unnecessary intervals, we can assume that we never put an interval that contains Takahashi. Also, we never move Takahashi to the same direction twice in a row (in this case we can omit the first interval). Thus, we need to consider only two types of movements by Aoki:

- 1. When Takahashi is to the left of L_i, move him to L_i by choosing the interval i.
- 2. When Takahashi is to the right of R_i, move him to R_i by choosing the interval i.

Also, note that these two types of movements must be performed alternately.

To simplify things, instead of forcing Takahashi to start/end the trip at origin, let's add an interval $[0, 0]$ to the set of intervals. How can we compute the total distance if we know the set of intervals used by Aoki? If k intervals with indices $i_1, \ldots, i_k$ are used for the first type of movements, and k intervals with indices $j_1, \ldots, j_k$ are used for the second type of movements (since those two types must be performed alternately, the numbers of intervals must be the same), the total distance will be:

$2(L_{i_1} + \cdots + L_{i_k} - R_{j_1} - \cdots - R_{j_k})$

Thus, for a fixed k, we can just choose intervals greedily, and by trying all possible values of k we can solve the problem in $O(N \log N)$ time.

Note that we will never choose the same interval twice (in two types of operations) in the optimal solution because for each interval $2L_i - 2R_i \leq 0$ holds.

D : Choosing Points

It turns out that in this problem, you are given two bipartite graphs with V vertices, and you are asked to choose $\frac{V}{4}$ vertices that is an independent set on both graphs.

First, let's prove that the following graph is a bipartite graph: the set of vertices is the set of integer points on a plane, and there is an edge between two integer points if the distance between them is $\sqrt{D}$. Let's do an induction on D.

In case D is odd, this is easy. If there is an edge between (x, y) and $(x+s, y+t)$, since $s^2+t^2 = D$ is odd, the parities of s and t are different. Thus, we can color a point (x, y) based on the parity of $x + y$ (standard chessboard coloring).

Suppose that D is even. If two cells p, q are at the distance of $\sqrt{D}$, they must be painted in the same color in a chessboard (assume that they are black cells). Now consider only black cells in a chessboard: these cells form a new square grid that is $\sqrt{2}$ times larger than the original grid (and rotated by 45 degrees). If we change the unit distance by a factor of $\sqrt{2}$, the (original) distance of $\sqrt{D}$ corresponds to the distance of $\sqrt{D/2}$ in the new grid. Thus, we can reduce to the case with $D/2$, and by the induction, we prove that the graph is bipartite.

Therefore, we now see that the graphs are biartite, and we now want to solve the problem mentioned at the beginning.

First, let's color the cells black and white, such that it becomes a bipartite coloring for the distance $\sqrt{D_1}$. Similarly, let's color the cells red and blue for the distance $\sqrt{D_2}$. Each cell is painted in two colors, and there are $2 \times 2 = 4$ possible patterns for a pair of colors in a cell. Thus, there is a pattern (a pair of colors) such that there are $\frac{4N^2}{4} = N^2$ or more cells that are painted in this pattern. From the definition of the colors, it's clear that these points satisfy the conditions.

The slowest part is the construction of graphs. It is obviously $O(N^3)$, and a detailed analysis will show that it will work much faster (because there are not so many pairs (s, t) such that $s^2 + t^2 = D_1 or D_2$).

E : Walking on a Tree

First, let's get the upper bound of the happiness. For each tree edge i, let c_i be the number of walks that pass through this edge. Since there are only two directions for this edge, the number of happiness we can get from this edge is at most $\min(c_i, 2)$. Thus, the upper bound of the total happiness is $\sum \min(c_i, 2)$.

We'll prove that we can always achieve this upper bound, in a constructive way. We do an induction on n, the number of vertices. In case $n = 1$, there's no edge and this is trivial. Suppose that $n > 1$, and let's reduce to a case with smaller n.

Take an arbitrary leaf, and call it v. Let e be the only edge incident to v. In case $c_e = 0$, we can simply remove v from the tree. In case $c_e = 1$, we can also remove v from the tree. Here, the walk that involves v will be converted as follows: if this walk is between v and x, it will be converted to a walk between w and x, where w is an endpoint of e to the opposite of v. Thus, we assume that $c_v \geq 2$.

Take two walks that involve v. Suppose that they are (v, a) and (v, b). Let (v, c) be the intersection of these two paths. Now, we add a restriction: if we choose $a \to v$ we must choose $v \to b$, and if we choose $v \to a$ we must choose $b \to v$. With this restriction, we can ensure that the edges between v and c are covered in both directions, and in the future we can ignore those edges. Thus, $a \to v$ and $v \to b$ is equivalent to $a \to b$, and $v \to a$ and $b \to v$ is equivalent to $b \to a$. Now we can remove two walks (v, a) and (v, b) and add (a, b) instead. By repeating this process while $c_v \geq 2$, we will eventually get a case with $c_v < 2$. Therefore, by induction, we can always achieve the upper bound.

The proof above shows a construction. Each step can be done in $O(N + M)$, and this solution works in $O(N(N + M))$ time in total.

F : Addition and Andition

Define two strings S, T as follows:

- If the i-th bit in the binary representation of X is 1, $S_i = 1$, otherwise $S_i = 0$.
- Similarly define T for Y.

In each operation, we perform the following for the strings:

- For each i such that $S_i = T_i = 1$, let $S_i = T_i = 0$.
- Then, for each such i, add 1 to both S_{i+1} and T_{i+1}.
- Then, handle "carries" of S_i, T_i.

Let's handle "carries" as follows:

- In the decreasing order of i, when we find i such that $S_i = 2$, repeat the following:
  - For $j = i, i+1, i+2, \ldots$ in this order, do the following.
  - If $S_j = 2$, let $S_j = 0$and add 1 to S_{j+1}.
  - Otherwise break the loop (and proceed to the next i).

Now we can restate the operations in the statement as follows:

- In the decreasing order of i, when we find i such that $S_i = T_i = 1$, let $S_i = T_i = 2$ and repeat the following$\cdots(X)$
  - For $j = i, i+1, i+2, \ldots$ in this order, do the following.
  - If $S_j = 2$, let $S_j = 0$and add 1 to S_{j+1}.
  - If $T_j = 2$, let $T_j = 0$and add 1 to T_{j+1}.
  - If neither of above holds, break the loop (and proceed to the next i).

In this task, we are asked to compute the states of S, T after we repeat the operation above K times. Thus, we do the following:

- For each $t = 0, 1, ..., K-1$, do the following.
  - In the decreasing order of i, ...

Here we have a hypothesis: the order of the two loops (a loop for t and a loop for i) doesn't matter. If we change the order of the loops, we get the following:

- In the decreasing order of i, do the following.
- Let $z = K$ be a counter that represents the number of remaining operations we may perform.
  - For $j = i, i+1, i+2, \ldots$ in this order, do the following.
  - If $S_j = T_j = 1$ and z is positive, let $S_j = T_j = 0$, add 1 to both S_{j+1} and T_{j+1}, and decrement z by one. (However, if $z \leq 0$, break the loop.)
  - If $S_j = 2$, let $S_j = 0$and add 1 to S_{j+1}.
  - If $T_j = 2$, let $T_j = 0$and add 1 to T_{j+1}.
  - If neither of above holds, break the loop (and proceed to the next i).

Why can we swap the order of the loops like this? Notice that after the operation (X), $S_i = T_i = 0$ will always be satisfied. Therefore, even if we perform (X) for positions before current i, since the total increase of the value is at most $2^i - 1$, it doesn't affect the $i+1$-th bit or more significant bits. Using this fact, we get the following observation. Consider the maximum i such that $S_i = T_i = 1$. Suppose that when we perform (X) for position i, the operation ends at position j (with $S_j = T_j = 1$). Then, even if we perform an operation for j before performing operations before position i, the result doesn't change. By repeating this observation, we can prove that the order of the loops doesn't matter.

Now, it is sufficient to simulate the process quickly after the loops are swapped. Let's restate the problem as follows:

- In the decreasing order of i, when we find i such that $S_i = T_i = 1$, let $S_i = T_i = 0$ and do the following:
- We keep a counter z (the number of remaining operations we may perform) and a string x.
- Initially, $z = K, x = 11$. The length of x is always 2, and it "carries" from lower digits. If $x = 10$ we have a carry only from S, if $x = 01$ we have a carry only from T, and if $x = 11$ we have carries from both strings.
  - For $j = i, i+1, i+2, \ldots$ in this order, do the following.
  - If $(S_j, T_j, x) = (0, 0, 10)$, let $(S_j, T_j) = (1, 0)$ and finish the operation.
  - If $(S_j, T_j, x) = (0, 0, 01)$, let $(S_j, T_j) = (0, 1)$ and finish the operation.
  - If $(S_j, T_j, x) = (0, 0, 11)$, let $(S_j, T_j, x) = (0, 0, 11), z \mapsto z - 1$. However, in case $z < 0$, let $(S_j, T_j) = (1, 1)$ and finish the operation.
  - If $(S_j, T_j, x) = (1, 0, 11)$, let $(S_j, T_j, x) = (0, 1, 10)$.
  - If $(S_j, T_j, x) = (1, 0, 10)$, let $(S_j, T_j, x) = (0, 0, 10)$.
  - If $(S_j, T_j, x) = (1, 0, 01)$, let $(S_j, T_j, x) = (0, 0, 11), z \mapsto z - 1$. However, in case $z < 0$, let $(S_j, T_j) = (1, 1)$ and finish the operation.
  - The case with $(S_j, T_j) = (0, 1)$ is similar.

Note that the case $(S_j, T_j) = (1, 1)$ never occurs (we can prove this in the same way as the proof for swapping the order of loops). Since we can easily handle position j when $(S_j, T_j) = (0, 0)$, let's keep all j such that $(S_j, T_j) = (1, 0), (0, 1)$. In particular, let's hold all such j in a stack (call the stack A). The elements in A are sorted in the increasing order from top to bottom. Then, when we find i such that $S_i = T_i = 1$, pop the first element of A and handle all $(0, 0)$ cases till the element. In care we can really reach the first element, handle it, and go to the next element. However, when we don't finish the operation with $(S_j, T_j) = (0, 0)$, we keep this information in some array (these values should be added to the stack later). We repeat this, and after the simulation mentioned above finishes, we push all residues we get during the simulation. This way we can perform the simulation.

Let's analyze the time complexity of this simulation. If we look at the algorithm above carefully, the only case where we don't get $(S_j, T_j) = (0, 0)$ after the operation is $x = 11$. After we get $x = 11$, x will be 10 or 01. In case $x = 10, 01$, the number of elements of A decreases, and in case $x = 11$, the number of elements of A doesn't increase. Thus, the number of elements of A decrease in every two steps of the simulation. Also, since the number of elements of A increases only when $(S_j, T_j) = (0, 0)$ (and this happens at most once for each i), this happens at most N times. Thus, we perform operations for the stack at most $2N$ times. Since we can perform push/pop for a stack in $O(1)$, this solution works in $O(N)$ time in total. Note that "the number of elements in A" includes the number of new elements added during the simulation.

AGC 024 解説

DEGwer

2018/05/20

For International Readers: English editorial starts on page 6.

A: Fairness

高橋君の持っている整数から中橋君の持っている整数を引いた値は、操作を 1 回行うとどのように変化するかを考えてみましょう。

高橋君、中橋君、低橋君がそれぞれ整数 $(A+x, A, B)$ を持っているとしましょう。1 回の操作後、3 人はそれぞれ $(A+B, A+B+x, 2A+x)$ を持っています。すなわち、所望の値は x から $-x$ に変化します。より正確に言えば、絶対値は符号が反転します。

よって、操作回数が偶数回なら答えは $A-B$ で、奇数回なら $B-A$ です。

B: Backfront

先頭や末尾に移動されることのない整数たちは、その相対的な順序が変更されることはなく、さらにそれらの間に新たに整数が挿入されることもありません。よって、移動されない整数を小さい順に $x_1, ..., x_k$ とすれば、これらは以下の条件を満たす必要があります。

- 全ての i に対し、$x_i + 1 = x_{i+1}$
- 全ての i に対し、元の順列で x_i は x_{i+1} より前にある

逆に、この条件が満たされれば、$x_1 - 1, ..., 1$ の順に整数を先頭に、$x_k + 1, ..., N$ の順に整数を末尾に移動することで列を昇順に並び替えることができます。よって、このような $x_1, ..., x_k$ に対する k の最大値を求めればよいです。

これは、全ての i に対し $Q[P[i]] = i$ として得られる列 Q を前から見ていくことで簡単に求められます。よってこの問題を $O(N)$ 時間で解くことができました。

C: Sequence Growing Easy

操作によって $X_1 > 0$ や $X_{i+1} - X_i > 1$ となることはないので、$A_1 > 0$ の場合と $A_{i+1} - A_i > 1$ なる i がある場合は答えは -1 です。以下、そうでない場合を考えます。

$A_i = t$ とします。このとき、操作によって X_i に t が代入されるためには、$t = 0$ であるか、または $X_{i-1} = t - 1$ であるようなタイミングが必要です。この議論を繰り返し行うことで、$1 \leq s \leq t$ に対し、

X_{i-t+s} に s が代入されているタイミングが必要となることがわかります。よって、このような $(i-t+s, s)$ の組としてありうるものの個数で求める最小値を下から抑えることができます。この値は、$X_{i-1}+1=X_i$ なら答えに 1 を、そうでないなら答えに X_i を足すことで求めることができます。

逆に、以下を繰り返すことによって、その回数の操作で X と A を一致させることができます。

- $X=A$ なら終了する
- そうでない場合、r を $X_r \neq A_r$ なる最大の r とする
- 各 $r-X_r+1, ..., r$ に対し、順に $X_i = X_{i-1}+1$ とする

よって、この問題を $O(N)$ 時間で解くことができました。

D: Isomorphism Freak

木に対し、その直径で同じ頂点を二度通らない最長のパスの長さ (パス上の辺の本数) を表します。また、直径を構成するパス上で最も遠い頂点までの距離が最小になる頂点を木の中心と呼びます。

最も遠い頂点までの距離が異なる二頂点は合同になりえないため、与えられる木の直径を D とすれば、互いに合同でない頂点は少なくとも $\lceil \frac{D}{2} \rceil$ 種類あります。逆に、この下界を達成できることを以下で示します。

D が偶数の場合、与えられる木の中心は 2 個あります。下界を達成するためには、これらの頂点のうち近い方からの距離が同じ頂点たちはすべて合同にする必要があります。すなわち、そのような頂点たちの次数はすべて等しくする必要があります。逆に、この条件が満たされれば、下界は達成されることが容易に確認できます。

よってこのとき、中心のうち近い方からの距離ごとに、そのような頂点の次数の最大値を求め、すべて掛け合わせたものの 2 倍が葉の個数の最小値になります。

D が奇数の場合、中心は 1 個あります。中心を c とします。下界を達成するためには、c からの距離が同じ頂点たちをすべて合同にするか、あるいは中心の隣の頂点 v をひとつ指定し、c, v のうち近い方からの距離が同じ頂点たちをすべて合同にする必要があります。

前者の場合は、D が偶数の場合と同様に解くことができます。後者の場合は、v の候補をすべて試し、D が偶数の場合と同様の処理を行えばよいです。

よって、$O(N^2)$ 時間でこの問題を解くことができました。なお、葉の個数の最小値は 64 ビット符号付き整数の範囲に収まることが (中心からの距離 k の頂点の個数が葉の個数に与える影響を評価することにより) 証明できます。

なお、(大きな整数を $O(1)$ 時間で扱えると仮定すれば) この問題は $O(N)$ 時間で解くこともできます。

E: Sequence Growing Hard

長さ k の数列に対し、その位置 $0, 1, ..., k$ で 1 文字目の前、1, 2 文字目の間、……、k 文字目の後の位置を表すことにします。

空列に整数を挿入していくことで数列を作っていくことを考えましょう。位置 i に整数 t を挿入できる条件は、便宜上数列の末尾に整数 0 が置かれているものとして考えれば、以下のいずれかの場合です。

- 数列の $i+1$ 項目の値が t より小さい

- 数列の $i+1$ 項目が t であり、その後 t 以外で初めて現れる整数が t より小さい

さて、後者の場合は ($i+1$ 項目以降初めて t 以外の整数が現れるのが r 項目であるとすれば) 位置 $r-1$ に t を挿入するとしてもできる数列は変わらないので、前者の操作のみ行うことができるとしてよいです。逆に、前者の操作でできる数列はすべて異なるので、前者の操作を繰り返し行う時の操作列の数を数えればよいです。特に、整数 t を挿入するとき、数列の t より小さい項の個数を X とすれば、$X+1$ 通りの挿入の方法があります。

さて、$DP[k][n]$ を入力 (n,k) に対する答えとします。求めたいものは $DP[K][N]$ です。1 以上 k 以下からなる長さ n の数列が、時刻 i に長さ i になるように作られていくような操作列に対し、整数 $k+1$ を合計 l 個、それぞれ時刻 X_1 と X_1+1 の間、……、時刻 X_l と時刻 X_l+1 の間に挿入するとき、上述の考察より、$k+1$ を追加する操作の操作列は $(X_1+1)(X_2+1)...(X_l+1)$ 個存在します。時刻を後から順に見ていくことを考えれば、最初全ての n に対し $x[n][n]=DP[k][n]$ として初期化し、

- $x[n][i]$ から $x[n][i-1]$ へ、係数 1 で遷移 (時刻を前に動かすことに対応)
- $x[n][i]$ から $x[n+1][i]$ へ、係数 $i+1$ で遷移 (整数 $k+1$ を時刻 i と $i+1$ の間の時刻に挿入することに対応)

という遷移式で DP を行えば、この問題を解くことができます。計算量は $O(KN^2)$ です。

F: Simple Subsequence Problem

長さ N 以下の全ての文字列に対し、その文字列を部分列として含むような S の要素の個数を求めることを考えましょう。これが求まれば、この問題を解くのは容易です。

以下のようなグラフを考えます。

- 長さ N 以下の全ての文字列 s に対し、頂点 $v_{s,0},...,v_{s,|s|}$ を用意する。
- 全ての文字列 s と $0 \leq i \leq |s|-1$ に対し、以下のように辺を張る。
  - s の先頭 i 文字を取り出してできる文字列を s_ϕ とし、$v_{s,i}$ から $v_{s_\phi,i}$ に辺を張る
  - s の $i+1$ 文字目以降で初めて 0 が出てくるのが r_0 文字目であるとする。もし r_0 が定義されるなら、s の i 文字目以前からなる文字列と r_0 文字目以降からなる文字列 (境界を含む) をつなげてできる文字列を s_0 とし、$v_{s,i}$ から $v_{s_0,i+1}$ に辺を張る
  - s の $i+1$ 文字目以降で初めて 1 が出てくるのが r_1 文字目であるとする。もし r_1 が定義されるなら、s の i 文字目以前からなる文字列と r_1 文字目以降からなる文字列 (境界を含む) をつなげてできる文字列を s_1 とし、$v_{s,i}$ から $v_{s_1,i+1}$ に辺を張る

さて、このグラフにおいて、$v_{s,0}$ から $v_{t,|t|}$ へは、t が s の部分列のときちょうど 1 本の、そうでないとき 0 本のパスがあることが示せます。これが示されれば、このグラフの $v_{s,0}$ の形をした頂点に $s \in S$ なら 1 を、そうでないなら 0 を書き、すべての辺に係数 1 をつけて DP を行うことで、全ての t について「$s \in S$ を用いて $v_{s,0}$ とあらわせる頂点からのパスの個数」、すなわち t を部分列として含むような $s \in S$ の個数を求めることができ、よってこの問題を解くことができます。

以下、このことを示します。t が s の部分列かどうかは、t を前から見る貪欲法で判定できることを思い出しましょう。まず、$v_{a,i}$ から $v_{b,j}$ へ辺があるとき b は a の部分列なので、$v_{s,0}$ から $v_{t,|t|}$ へのパスがある場

合 t は s の部分列です。また、t が s の部分列のとき、t を前から順に見て、今見ている文字が 0 なら s_0 への、1 なら s_1 への、t の末尾に到達したなら s_ϕ への辺を辿れば、$v_{s,0}$ から $v_{t,|t|}$ へ辿り着くことができます。さらに、それ以外の辿り方では $v_{t,|t|}$ には辿り着けないことも、頂点 $v_{a,i}$ から辿り着ける頂点に対応する文字列の先頭 i 文字は a と一致することからわかります。

よって主張は示され、この問題を解くことができました。時間計算量は、グラフの頂点数が $O(2^N N)$ であり、辺数がその定数倍なので、合計で $O(2^N N)$ です。

AGC 024 Editorial

DEGwer

2018/05/20

A: Fairness

Let d be Takahashi's integer minus Nakahashi's integer. How does d change after each operation?

Suppose that initially $d = x$. The three people have integers $(A + x, A, B)$ for some A, B. After one operation, the three integers will be $(A + B, A + B + x, 2A + x)$, and $d = (A + B) - (A + B + x) = -x$.

Thus, in each operation the value of d is multiplied by -1. If the number of operations is even, the answer is $A - B$, otherwise the answer is $B - A$.

B: Backfront

Suppose that we can sort the sequence without performing operations on integers $x_1, \ldots, x_k (x_1 < x_2 < \ldots)$. Then, the following conditions must be satisfied:

- Those integers will form a consecutive interval in the sorted sequence. Thus, for each i, $x_i + 1 = x_{i+1}$ holds.
- The relative positions of those integers won't change. Thus, for each i, x_i must be to the left of x_{i+1} in the initial sequence.

On the other hand, if these conditions are satisfied, we can sort the sequence in $N - k$ steps (without moving those integers). For example, we can move integers $x_1 - 1, ..., 1$ to the beginning in this order, then move integers $x_k + 1, ..., N$ to the end in this order. Thus, we want to compute the maximum possible value of k for such $x_1, ..., x_k$.

This can be done by using a permutation Q such that $Q[P[i]] = i$. This solution works in $O(N)$ time.

C: Sequence Growing Easy

By repeating operations, we can never satisfy $X_1 > 0$ or $X_{i+1} - X_i > 1$. Thus, in case $A_1 > 0$ or $A_{i+1} - A_i > 1$ for some i, the answer is -1. From now on, we assume that $A_1 = 0$ and $A_{i+1} - A_i \leq 1$ (and in this case, it turns out that we can always achieve the goal, as we see below).

First, let's compute the lower bound on the number of operations. Suppose that $A_i = t$. In order to satisfy $X_i = t$, at some point during the operations, either $t = 0$ or $X_{i-1} = t - 1$ must be satisfied. By repeating this observation, for each $s (1 \leq s \leq t)$, at some point during the operations, $X_{i-t+s} = s$ must be satisfied.

Thus, the number of different pairs of $(i - t + s, s)$ (that can be obtained this way) gives the lower bound on the number of operations. This value can be computed by adding 1 in case $X_{i-1} + 1 = X_i$, and X_i otherwise.

On the other hand, we can always achieve the goal within the number of operation computed above, as follows:

- If $X = A$, we achieved the goal, we stop performing operations.
- Otherwise, let r be the maximum integer such that $X_r \neq A_r$. For each $i - X_r + 1, ..., r$ in this order, perform an operation $X_i = X_{i-1} + 1$.

This solution works in $O(N)$ time.

D: Isomorphism Freak

Let D be the diameter of the given tree. Since two vertices on the diameter are not congruent unless they are at symmetric positions of the diameter, (and the diameter never decreases by performing operations), the colorfulness of the tree must be at least .

We can always achieve this colorfulness, and let's compute the minimum number of leaves in this case.

In case D is odd

Since $\lfloor \frac{D+1}{2} \rfloor + 1 > \lfloor \frac{D}{2} \rfloor$, in order to achieve the minimum colorfulness, we must not change the diameter by operations.

In this case, the tree has two centers. To keep the diameter, we must keep these two centers, and each newly added vertices must be within the distance of $D/2$ from the centers. Here, the distance between a vertex and the centers is defined as the distance between the vertex and the center closer to the vertex.

It's easy to see that we can achieve the minimum colorfulness if and only if (we don't change the centers and the diameter and) all vertices at the same distance from the centers have the same degree. Thus, in this case, the minimum number of leaves can be computed as follows:

- Ignore the edge between the centers and split it into two rooted trees (roots are the centers).
- For each $d (0 \leq d \leq \lfloor \frac{D}{2} \rfloor)$, let x_i be the maximum number of children of vertices at depth d.
- The minimum possible number of leaves is the product of all x_i times two (since we'll get to isomorphic rooted trees).

In case D is even

Since $\lfloor \frac{D+1}{2} \rfloor + 1 - \lfloor \frac{D}{2} \rfloor$, there are two cases: we don't change the diameter, or we increase the diameter by one by operations.

In this case, the tree has a single center. Let's call it c.

There are two cases:

- Keep c as the unique center, and keep the diameter to D.
- For some vertex v adjacent to c, make c, v two centers, and change the diameter to $D + 1$.

The former case is similar to the solution described above. The latter case can be handled by trying all possibilities for v. Thus, this solution works in $O(N^2)$ time.

Note that by using the fact that $\sum x_i \leq N$, we can prove that the answer always fit in a 64-bit integer (because we can bound the value of $\prod x_i$). Also, note that it is also possible to solve this problem in linear time (left as an exercise for readers), assuming that we can perform basic operations on big integers in constant time.

E: Sequence Growing Hard

When can we insert an integer x to the left of an integer y, and make the sequence lexicographically larger? There are two cases:

- $x > y$.
- $x = y$, and the first integer that is not x after this position is greater than x.

However, we don't need to consider the second case. In the second case, if the first integer that is not x is r, we can insert x immediately to the left of this r, and this change doesn't affect the sequence. On the other hand, if we allow only the first case, if we insert an element at different positions we always get different sequences, thus we want to count the number of ways to perform operations when we are allowed to perform operation only in the first case.

Let's convert the sequence of operations to a rooted tree in the following way.

- Start with a tree with a single node. In this tree each vertex has two values: id and label. The only vertex has $id = 0, value = 0$.
- Suppose that in the k-th (1-based) operation, we insert an integer t to the left of the integer that was inserted in the p-th operation (or $p = 0$ in case we add it to the end of the sequence). Then, create a vertex with $id = k, value = t$, and its parent is a vertex with $id = p$.

Now, what we want to compute is the number of rooted trees with the following properties:

- Each vertex has two values: id and label.
- id are assigned sequencially in the order $0, 1, \ldots$.
- The label of each vertex is between 0 and K.
- For each node, its id and its label are strictly greater than parent's id and parent's label, respectively.

Let $DP[n][x]$ be the number of rooted trees (with the properties above) with n vertices such that the label of the root is x. The answer is $DP[N + 1][0]$.

To compute this DP, notice that the parent of a vertex with $id = 1$ must be a vertex with $id = 0$. If there are k vertices in the subtree rooted at $id = 1$, there are $\sum_{y>x} DP[n-k][x] * DP[k][y] * comb(n-2, k-1)$ ways to decide a tree.

Now it's easy to make it $O(KN^2)$ using prefix sums.

F: Simple Subsequence Problem

For each string s such that $|s| \leq N$, let's compute the number of elements in S that contains s as a subsequence. After getting these values, the original problem can be easily solvable.

Our plan is to construct a DAG with the following properties:

- For each string s such that $|s| \leq N$, there is a red vertex labelled with s.
- For each string s such that $|s| \leq N$, there is a blue vertex labelled with s.
- Additionally, some black vertices may exist.
- If t is a subsequence of s, there is exactly one path from red s to blue t.
- Otherwise, there is no path from red s to blue t.

Once we construct this DAG, we can get desired values by writing 1 on each red vertex labelled with an element in S, and run a DP on the DAG.

Let's construct the DAG. First, how can we list all subsequences of a particular string s? Remember that to check if a string t is a subsequence of s, we shuold choose each character in t from s greedily from left to right.

Let $[s]$ denote the set of all subsequences of s. For example, how can we compute [000110101]?

If t is a subsequence of 000110101, there are three cases:

- t starts with 0. In this case the remaining part of t must be a subsequence of 110101. These strings can be represented by 0[00110101] (a single 0 followed by a subsequence of 00110101).
- t starts with 1. In this case the remaining part of t must be a subsequence of 10101. These strings can be represented by 1[10101].
- An empty string.

Thus, [000110101] is a disjoint union of 0[00110101], 1[10101], and [].

Now, we can see that the following construction satisfies all the properties:

- For each pair of strings (s, t) such that $|s| + |t| \leq N$, prepare a black vertex labelled with $s[t]$.
- If t contains 0 and the (0-based) position of the first occurrence of 0 in t is i, add an edge from black $s[t]$ to black $s0[t']$, here t' is the suffix of t with length $|t| - i - 1$.
- If t contains 1 and the (0-based) position of the first occurrence of 1 in t is i, add an edge from black $s[t]$ to black $s1[t']$, here t' is the suffix of t with length $|t| - i - 1$.
- If t is empty, add an edge from black $s[t]$ to black $s[]$.
- For each string s, add an edge from red s to black $[s]$, and add an edge from black $s[]$ to blue s.

The number of vertices in the DAG is $O(2^N N)$, and this solution works in $O(2^N N)$ time.

Atcoder Grand Contest 023 解説

writer : maroonrk

平成 30 年 4 月 28 日

For International Readers: English editorial starts from page 7.

A : Zero-Sum Ranges

S を長さ $N+1$ の数列とし、$S_0 = 0$, $S_i = S_{i-1} + A_i$ とします。ある連続する部分列の総和は、S の 2 要素の差です。よって、和が 0 になる連続する部分列の数は、S の中で値の等しい 2 つの要素を選ぶ方法の数に等しいです。これは、S の要素を sort するなどして、同じ値の個数を数えることで計算出来ます。sort に $O(NlogN)$ かかるため、この問題は $O(NlogN)$ で解けました。

B : Find Symmetries

ある (A, B) が条件を満たすことと、$(A+1, B+1)$ が条件を満たすことは同値です。($A+1$ や $B+1$ が $N+k$ と表される場合は k と同一視してください) よって、(A, B) の組として、$(0, 0)$, $(0, 1)$, ..., $(0.N-1)$ をチェックし、条件を満たすものの個数を N 倍したものが答えです。一回のチェックは $O(N^2)$ で行えます。チェックは N 回行うので、合計で $O(N^3)$ でこの問題は解けました。

C : Painting Machines

期待値の線形性から、各 K について、K 回マシンを動かして、すべてのマスが黒く塗られているような順列の個数、が求まれば良いです。

結論から言えば、このような順列の個数は $C(K-1, N-1-k) \times K! \times (N-1-K)!$ です。ここで、C は二項係数です。

このコンビネーションは、次のように考えると導出されます。まず、$N-1$ 個のマシンについて、動かした or 動かしていない を決めると、その状態になる順列は $K! \times (N-1-K)!$ 個あります。これは、動かした K 個のマシンを動かす順番と、動かしていない $N-1-K$ 個のマシンを動かす順番を考えることに対応しています。

動かした or 動かしていない の決め方について考えます。$N \leq 3$ のケースは手で処理してしまうとして、$N \geq 4$ と仮定します。まず、すべてのマスが塗られるためには、マシン 1 及び マシン $N-1$ は必ず動かしている必要があります。そして、動かしていないマシンは必ず、2 つの動かしたマシンに挟まれている必要があります。動かしたマシンは K 個ですから、その隙間は $K-1$ 個です。$K-1$ 個の隙間から、動かしていないマシンを置く $N-1-K$ 個の場所を選ぶ方法は、$C(K-1, N-1-K)$ 通りです。

これで、コンビネーションの導出が出来ました。コンビネーションの計算は、階乗とその逆元を前計算しておけば 1 回あたり定数時間で行なえます。前計算に $O(N)$ かかり、コンビネーションも $O(N)$ 回計算するので、この問題は $O(N)$ で解けました。

D : Go Home

マンション 1 に住む社員が A 人、マンション N に住む社員が B 人だとしましょう。

$A \geq B$ のとき、バスは、マンション N よりもマンション 1 を先に訪れます。これは、仮にマンション N の直前までバスが来たとしても、そこからマンション 1 側の全員が負の方向へ投票し続ければ良いことからわかります。また、バスがマンション 1 についてからマンション N に向かう手順は明らかで、直進するだけです。このことから、バスがマンション N に到着する時刻はマンション 1 に到着する時刻に定数を足したものであるとわかります。

投票の際、各社員は、自分の帰宅時間が短くなるような方へ投票します。よって、投票を行う際、ほとんどのケースでは、マンション 1 の住人とマンション N の住人の動きは一致します。唯一の例外は、バスがマンション N の直前にいる場合です。しかしこの場合は、先程考えたように、マンション 1 側にバスが移動してしまいます。

よって、マンション N の住人はマンション 1 の住人と同じ投票をすると思っても良いとわかります。これは、マンション N がなくなって、代わりに $P[1] = P[1] + P[N]$ としたのとほとんど変わりありません。唯一の違いは、最後にマンション N を訪れるかどうかです。

$A < B$ のときも同様にして、マンションの数を減らして考えることが出来ます。

以上の操作を再帰的に繰り返せば、最終的に問題は自明な形（すべてのマンションがバスの初期位置から見て一方にある状態）になります。一回の再帰に必要なのは定数回の演算なので、この問題は $O(N)$ で解けました。

E : Inversions

$Cnt[k] = A_i \geq k$ なる A_i の個数 $-(N-k)$ とすると、ありうる P の個数は $\prod_{k=1}^{N} Cnt[k]$ となります。この数を S とおきます。ここで、$Cnt[k] = 0$ なる k がある場合は、ありうる順列は存在しないため、答えとして 0 を出力します。以下、$Cnt[k] \geq 1$ とします。

各 i, j について、$P_i > P_j$ なる P の個数がわかれば良いです。ここではまず、$A_i \leq A_j$ となる i, j について数えることにします。

この問題の本質部分は、$P_i > P_j$ なる P の個数は、$A_j := A_i$ としたときに条件を満たす P の個数のちょうど半分になるということです。P_i と P_j を swap してできる P が一意に定まり、またこれも条件を満たすことから明らかです。

よって、各 i, j について、$A_j := A_i$ としたときに条件を満たす P の個数を求めれば良いです。

$A_j := A_i$ とすることで、Cnt は範囲 $[A_i + 1, A_j]$ の値が 1 減ることになります。

そこで予め、$D[k] = \prod_{i=1}^{k} (Cnt[i] - 1)/Cnt[i]$ と定義しておきます。

すると、$A_j := A_i$ としたときの、P の個数は、$D[A_i]/D[A_j] \times S$ となります。ここで、$D[A_j]$ が 0 の場合は式が壊れてしまいますが、その対策は後ほど解説することにして、とりあえず $D[k] \geq 1$ と思っておきます。

以上より、結局、各 j について、$i < j$, $A_i \leq A_j$ なるすべての i について、$1/D[A_i]$ の総和が求まれば良いとわかります。これは、j を増やしながら、$1/D[A_i]$ の和を BIT を管理すれば高速にできます。

ところで、$D[A_j]$ が 0 の場合は式が壊れていたのでした。そこで、$D[k]$ を形式的に、$D[k] \times 0^{x[k]}$ という形だと思ってみます。すると、$D[A_i]/D[A_j]$ が意味を持つ、つまり 0 でないのは、$x[A_j] = x[A_i]$ のときです。また明らかに $x[k]$ は広義単調増加です。よって、更新は先程とほぼ同様に行い、BIT で和を求めるときは、$x[A_j] = x[k]$ なる範囲の和だけを取れば良いです。

$A_i > A_j$ となるのも大体同様にできますが、こちらの場合は転倒していないペアの個数を数えることになるため、A の転倒数を求めてこれから引き算する操作が必要になります。

BIT で $O(N)$ 回のクエリ処理を行うため、$O(NlogN)$ でこのアルゴリズムは動作します。

F : 01 on Tree

より一般的な問題を考えます。各頂点には 0 と 1 からなる数列が書かれていて、頂点 i にかかれている数列には 0 が $C0_i$ 個、1 が $C1_i$ 個含まれているとします。そして、これらの頂点を、先祖が子孫の頂点の右側に置かれないよう並び替え、頂点にかかれている数列を並び替えた順に取り出して連結し、出来た数列の転倒数を最小化したいとします。なおここで、もともと各頂点に書かれている数列内での転倒数は考えないことにします。(これは並び替えによらず一定なため)

ここで、根以外の頂点の中で、$C0_i/C1_i$ ($C1_i = 0$ のときは ∞ と考える) が最大の頂点を選び、この頂点を v とします。v の親の頂点を p とします。ここで、最終的な並び替えにおいて、p のすぐ右隣に v があるような最適な並び替えが存在することがわかります。もし p と v の間に他の頂点が挟まる最適解があるならば、v の選び方から、v を左にずらしても決して損しないことからこれはわかります。

これで、多項式解法が得られました。つまり、$C0_i/C1_i$ が最大の頂点を選び、それと親の頂点をマージし、一つの頂点にしてしまう、という操作を $N-1$ 回繰り返す解法です。転倒数は、頂点をマージする際に簡単に計算できます。

この解法はそのままでは $O(N^2)$ なので、改善が必要です。そのために、Union-Find と Priority-Queue を使用します。Union-Find は、頂点のマージを管理します。Priority-Queue は、各頂点の $C0_i/C1_i$ を管理します。この 2 つのデータ構造を使えば、$C0_i/C1_i$ 最大の頂点を選び、親の頂点とマージする、という一連の動作が $O(logN)$ 時間で行えるようになり、全体で $O(NlogN)$ でこの問題が解けます。

Atcoder Grand Contest 023 Editorial

writer : maroonrk

April 28, 2018

A : Zero-Sum Ranges

Let S be a sequence of length $N+1$ defineed as follows: $S_0 = 0$, $S_i = S_{i-1} + A_i$. (This is a sequence of prefix sums). Then, the sum of all integers in a contiguous subsequence is the difference of two elements in S.

Thus, the answer is the number of ways to choose two elements from disticnt positions in S such that their values are the same. This can be done by sorting the sequence S, and it works in $O(N \log N)$ time.

B : Find Symmetries

A pair (A, B) saisfies the condition if and only if a pair $((A+1)\%N, (B+1)\%N)$ satisfies the condition. This is because if two cells are at symmetric positions in the first pair, they are also at symmetric potisions in the second pair.

Thus, we only need to check the following pairs for (A, B): $(0, 0)$, $(0, 1)$, ..., $(0.N-1)$. The answer is the number of good pairs among them, multiplied by N.

This solution works in $O(N^3)$ time.

C : Painting Machines

For each K, we want to count the number of permutations such that after activating the first K machines, all cells become black. Then, the answer can be represented as additions/subtractions of these numbers.

Only the set of first K machines matter. For simplicity, assume that $N \geq 3$. First, in order for all cells to be black, machines 1 and $N-1$ must be in the set of first machines. If the set of first K machines are $1 = a_1 < a_2 < \cdots < a_K = N-1$, for each i, $a_{i+1} - ai$ must be either one or two. Thus, there are $C(K-1, N-1-K)$ ways to choose them (here C denotes binomial coefficients).

Since we can freely permute machines among the first K machines or remaining $N-1-K$ machines, the number we want to compute is $C(K-1, N-1-k) \times K! \times (N-1-K)!$.

If we pre-compute factorials and their inverses, this works in $O(N)$ time.

D : Go Home

Let A, B be the number of employees in apartments 1 and N, respectively.

If $A \geq B$, the bus visits apartment 1 before apartment N. This is because, after apartment $N-1$ is visited for the first time, everyone except for residents of apartment N will vote for negative direction, and the bus will visit apartment 1 before apartment N. After it visits apartment 1, it directly goes to apartment N because this is the only remaining apartment. Thus, the time when the bus reaches apartment N is, the time when the bus reaches apartment 1 plus constant (constant is the distance between these two apartments).

When a voting takes place, each employee vote for the direction that makes his/her arrival time earlier. Thus, for employees of apartment N, the vote is almost always the same as the vote of residents of apartment 1. The only exception happens after apartment $N-1$ is visited. However, in this case, the bus will go to negative direction anyway, regardless of the votes from residents from apartment N. Thus, even if they always make the same votes as the residents in apartment 1, it won't change the movement of the bus (except that in the end the bus needs to travel from X_1 to X_N).

Therefore, the answer won't change if we do the following. Move all residents of apartment N to apartment 1 (i.e., $P[1] = P[1] + P[N]$), and decrement N by one. Add $X_N - X_1$ to the answer.

Similarly, when $A < B$, we can also decrease the number of apartments.

We can repeat the operation above recursively while there are at least one apartmens for both directions. Since each recursion can be performed in constant time, this solution works in $O(N)$ time.

E : Inversions

For each pair of integers $i, j (i < j)$, we want to count the number of valid permutations such that $P_i > P_j$. The answer is the sum of these numbers for all pairs.

First, only consider pairs i, j such that $A_i \leq A_j$. An essential observation is that the number of valid permutations such that $P_i > P_j$ can be computed as follows: Replace A_j with A_i and count the number of valid permutations. Then the number we want compute is exactly a half of the number of valid permutaions (after the replacement). Thus, for each pair i, j such that $i < j$ and $A_i \leq A_j$, we want to compute the number of valid permutations when $A_j := A_i$ is performed.

Let $Cnt[k]$ be (the number of elements such that $A_i \geq k$) minus $(N - k)$. Then the number of valid permutations is $S := \prod_{k=1}^{N} Cnt[k]$. From now on, we assume that $Cnt[k] \geq 1$ for all k (otherwise there is no valid permutation and the answer is obviously zero).

What happens if we perform $A_j := A_i$? For each k in the interval $[A_i+1, A_j]$, the value of $Cnt[k]$ will be decremented by one. Define $D[k] = \prod_{k=1}^{N}(Cnt[k] - 1)/Cnt[k]$. Then, when we perform $A_j := A_i$, the number of valid permutations will be $D[A_j]/D[A_i] \times S$ (unless $D[A_i] = 0$).

Thus, for each j, we want to compute the sum of $1/D[A_i]$ for all i such that $i < j$, $A_i \leq A_j$. This can be done with BIT just like standard inversion number: while we increase j, we keep the sums of $1/D[A_i]$.

Make sure to handle cases with $D[A_j] = 0$ correctly. For example, you can keep $D[k]$ of the form $D[k] \times 0^{x[k]}$. Then, $D[A_i]/D[A_j]$ is nonzero only when $x[A_j] = x[A_i]$. Since $x[k]$ is monotonously non-decreasing, the positions with the same value of $x[k]$ will form an interval. In each interval, you can compute the sum of $D[A_i]/D[A_j]$ using the method mentioned above.

You can handle the cases with $A_i > A_j$ similarly. Note that this time you need compute the product of the inversion number of the origional sequence and the original value of S, and subtract what we compute from the product.

We perform $O(N)$ queries with BIT, and it works in $O(N \log N)$ time.

F : 01 on Tree

Let's generalize the problem. Each vertex contains multiple 0s and 1s: the vertex i contains $C0_i$ zeroes and $C1_i$ ones. We want to arrange the vertices in a row as in the statement, and minimize the inversion number. Here, we don't count a pair of a zero and a one in the same vertex to the inversion number.

Let v be the non-root vertex with the maximum value of $C0_i/C1_i$ (when $C1_i = 0$, define it as ∞). Let p be its children. Then, we can prove that, in an optimal sequence, v should be immediately after p. (If v is immediately after w and w is not p, we can legally swap w and v, and its inversion number won't be greater.)

Thus, we get the following polynomial-time solution. We start with the given tree, and repeat the following steps $N - 1$ times. In each step, you choose a vertex with the maximum value of $C0_i/C1_i$ (call it v), and merge it with its parent p. When you merge v and p, you should add $C0_v \times C1_p$ to the answer. This solution works in $O(N^2)$ time.

To make it faster, we use Disjoint Set Union and Priority Queue. By Disjoint Set Union, you keep the set of merged vertices. By Priority Queue, you keep the values of $C0_i/C1_i$ of all non-root vertices. This way, each step described above can be performed in $O(\log N)$, and the entire solution works in $O(N \log N)$ time.

AGC 022 Editorial (Japanese)

zscoder

2018 年 4 月 1 日

For International Readers: English editorial starts on page 10.

A. Diverse Word

S の長さが 26 未満の場合、辞書順で次の多彩な文字列は、S にまだ現れていない中で最も小さい文字を S の後ろに付け足したものです。

そうでない場合、S の長さは 26 です。$S = p_1p_2p_3...p_{26}$ として、$p_ip_{i+1}...p_{26}$ を S の接尾辞であって文字が左から右に降順に並んでいる（すなわち、$i \leq j \leq 25$ に対して $p_j > p_{j+1}$ である）ようなもののうち最長のものとします。$i = 1$ のとき、S は辞書順で最も大きい多彩な単語であるため、答えは -1 です。そうでなければ、$p_i, p_{i+1}, ..., p_{26}$ のうち p_{i-1} より大きいものの中で最も小さい文字を q として、辞書順で次の多彩な文字列は $p_1p_2...p_{i-2}q$ となります。（このような文字は必ず存在します。なぜなら、そうでないと仮定すると $p_{i-1} > p_i$ となって接尾辞の最長性に矛盾するからです。）

B. GCD Sequence

まず、各要素 a_i と残りの要素の和の最大公約数（以下 gcd）を考える代わりに、各要素 a_i とすべての要素の和 S の gcd を考えても構いません。$\gcd(a_i, S - a_i) = \gcd(a_i, S)$ であるためです。

実は、制約が解法のヒントを多少含んでいました。この厳しい制約は、1 から N までの数のうち 2/3 程度を選ぶ必要があるを意味します。

アイデアの重要部分は、S を何らかの小さい素数の倍数とすれば、a_i たちがその小さい素数で割り切れ、かつそれらの gcd が 1 であればよくなる、というものです。ここから、k を何らかの整数として $S = 6k$ とする案が浮かびます。すると、a_i がいずれも 2 か 3 の倍数であって、それらすべての gcd が 1 となるように選べばよいことになります。

偶然にも、1 から N までの数のうち 2/3 程度が 2 か 3 で割り切れます（つまり、$6k+2, 6k+3, 6k+4, 6k$ のいずれかの形式の数です）。さらに、2 と 3 を両方選べば、選んだ要素の gcd は必ず 1 となります。

これでほぼ解けました。あとは、N が小さいケースを処理して、S が確実に 6 の倍数になるようにすればよいです。$N \leq 5$ のときは、それぞれ $\{2, 5, 63\}, \{2, 5, 20, 63\}, \{2, 5, 20, 30, 63\}$ を選べばよいです（他にも解はあります。また、このうち最初の二つは都合よくサンプルにありました）。以降、$N \geq 6$ と仮定します。

$6k+2, 6k+3, 6k+4, 6k+6$ の形式の数を小さい順に選んでいきましょう。唯一の問題は、和が 6 にならないかもしれないことです。和を 6 で割った余りは $0, 2, 3, 5$ のいずれかになることが容易にわかるので、これらのケースを個別に処理しましょう。

- S が 6 で割り切れるとき: 何もする必要はありません。
- $S \equiv 2 \pmod 6$ のとき: 8 を集合から削除して、次の 6 の倍数を追加すればよいです。
- $S \equiv 3 \pmod 6$ のとき: 9 を集合から削除して、次の 6 の倍数を追加すればよいです。
- $S \equiv 5 \pmod 6$ のとき: 9 を集合から削除して、次の $6k+4$ の形式の数を追加すればよいです。

より単純な解もあります。$N = 3$ のときはサンプルにある解を使います。N が偶数のときは $2, 10, 3, 9, 4, 8, 6, 12, ...$（次の 8 個はこれらにそれぞれ 12 を足したもの、以下同様）の順に追加すればよく、N が奇数のときは $6, 2, 10, 3, 9, 4, 8, 12, ...$（次の 8 個はこれらにそれぞれ 12 を足したもの、以下同様）の順に追加すればよいです。

C. Remainder Game

まず、それぞれの数 v に対して、その数に x で割る操作を適用したら、それ以降その数を $y \geq x$ で割る操作を適用しても値が変わらないため無意味です。したがって、操作は k の (strict な) 降順に行われると仮定して構いません。特に、それぞれの値はたかだか一度しか使われません。

$2^x > 2^{x-1} + 2^{x-2} + ... + 2^0$ より、実質的には a_i を b_i に変える辞書順最小の操作列を求めることになります。

明らかに、50 より大きい数を使うことはありません。次の問いに答える必要があります。

操作で使う数の集合 S を固定したとする。S の要素のみを使って目標を達成できるか？

この問いに答えることができれば、まず 50 から始めて、$\{1, 2, ..., 49\}$ を操作で使って目標を達成できるか判定します。可能なら 99 に進み、$\{1, 2, ..., 48\}$ を操作で使って目標を達成できるか判定します。使わざるを得ない整数が出てきたら、それを S に恒久的に追加します。この過程を続けることで、目標を達成するのに必要な辞書順最小の数の集合が求まります。目標が達成不可能であるかどうかは、$S = \{1, 2, ..., 50\}$ に対して同じ問いに答えることでわかります（ある単純な条件の成立を確認してもわかりますが)。

では、部分問題の解き方に移りましょう。次のような有向グラフを考えます。頂点には 0 から 50 の番号がついていて、各整数 v に対して、すべての $t \in S$ について v から $v \bmod t$ への有向辺があります。目標が達成可能なのは、各 b_i が a_i から到達可能なときで、そしてそのときに限ります。(必要性はグラフの定義から明らかで、十分性も、各ペアが到達可能なときは同じ値による操作を一度の操作でまとめて行えるため、明らかです。）これは BFS か DFS で確認できます。

最悪ケースでは、部分問題を $O(N)$ 回解くことになります（便宜上 $N = 50$ とします)。それぞれの部分問題を解くために、関係する頂点で BFS か DFS を行うか、Floyd-Warshall のアルゴリズムを実行することができます。辺が $O(N^2)$ 本存在しうるため、これらのアルゴリズムはどちらも最悪ケースで $O(N^3)$ 時間を要します。したがって、アルゴリズム全体の実行時間は $O(N^4)$ となります（かかる定数は小さいですが)。

部分問題ごとにグラフを再構築したり BFS/DFS をやり直したりしないことで、問題を全体で $O(N^3)$ 時間で解くことも可能ですが、これは読者への練習問題とします。

D. Shopping

本質的には、すべての駅を一度以上訪れて帰れるまでの電車の最小の往復回数を求めていることに注意します。電車が路線を一方向に走りきる回数を求めて L 倍することにします（とはいえこの回数は偶数なので答えは $2L$ の倍数です)。

まず、二乗時間の解法を説明します。座標 0 にダミーの駅を置きます。各駅について、電車がその駅に右から入って右から出ていくなら値 $+1$ を（最初、電車は座標 0 に右から入るものとします)、電車がその駅に左から入って左から出ていくなら値 -1 を、その他の場合（左から入って右から出るかその逆）は値 0 を割り当てます。電車がとりうる経路は、すべての駅への値 $1, 0, -1$ の割り当てであって、次の条件を満たすようなものとして表せます：駅の列のどの prefix（全体を除く）についても値の和が正であり、すべての値の和が 0 である。それぞれの値を各駅に割り当てるコストは値 x_i, t_i を用いて $O(1)$ 時間で求められます。ここで、$dp[i][j]$ を「最初の i 駅に値を割り当てて、それらの和を j とするのに必要な最小のコスト」とすると、$O(N^2)$ 時間の解法が得られ、部分点を取る上では十分です。

では、線形時間の解法に移ります。

まず、すべての t_i を $2L$ で割って余りをとります。ただし、t_i が $2L$ で割り切れる場合、その駅を取り除くわけにはいかない（一度は通るようにしなければならない）ので注意が必要です。以下、$0 \leq t_i < 2L$ とします。

各駅は組 (x, y) で特徴付けられます。ここで、x は、電車がその駅に左から来たときに降りて買い物をした場合に、次の電車が右から来るなら、つまり $t_i \leq 2(L - x_i)$ であれば 0 であり、次の電車が（次の往復に入ってから）左から来るなら x は 1 です。y は、電車がその駅に右から来たときに降りて買い物をした場合に、次の電車が左から来るなら、つまり $t_i \leq 2x_i$ であれば 0 であり、そうでなければ x は 1 です。($2L$ で割ったあと）$t_i = 0$ であるような駅は $(1, 1)$ の駅として、あとで答えを適切に減らすことにしましょう。

電車の行程は、左端（座標 0）から出発してこれらの駅を何らかの順番に訪れるような動く点とみなせます。$x = 0$ であるような駅に左から入ると、進行方向が反転して右から左に動くようになり、$x = 1$ であるような駅に左から入ると、そのまま通り抜けてコストが 1 増加します。駅に右から入る場合も同様です。点が左端や右端（座標 $0, L$）に達すると、やはり進行方向が反転してコストが 1 増加します。我々の目標は、行程の総コストを最小化することです（最後に答えに適切な定数を足し、L 倍します)。

$(1, 1)$ の駅はほぼ無視できることに注意します。単に通り抜けてコストが 1 増えるだけだからです。ただし、経路がすべての駅を一度は必ず通るようにあとで考慮する必要があります。当面の間、$(1, 1)$ の駅は存在しないと仮定します。

また、定義より $(1, 0)$ の駅は $(0, 1)$ の駅より前に現れることがない点にも注意します。

駅の並びが $(0,1),(0,1),...,(0,1),(0,0),(1,0),...,(1,0)$ という形であると仮定します。ここで、$(0,0)$ の駅の数は 0 か 1 であるとします。S を駅の数とします。ここでのアイデアは、それぞれの駅について、その駅はコスト 1 で外側に通り抜けるか、その駅でコスト 0 で外側方向に進行方向が反転するかのどちらかである、というものです。ところが、このような形で駅が配置されているため、次の駅に追加コストを発生させずに入ることはできません（次はより外側にある駅をコスト 1 で通り抜けるか、座標 0 か L で反転するかです)。駅が追加コストを発生させないのは、それが最後に訪れる駅である場合のみです。しかし、$(1,0)$ の駅が一つ以上存在する場合、少なくとも一度は右端で反転しなければならないことがいえ、少なくとも $S+1$ のコストが発生することが証明できます。$(1,0)$ の駅が存在しない場合は、以下の場合を除き最小コストは $S-1$ です。その場合というのは、最後の駅（$(0,1)$ か $(0,0)$）より遠くに駅が存在し、その駅に一度は寄らなければならない場合で（$(1,1)$ の駅か、後述の「削除済み」駅のために起こる)、この場合の答えは変わらず $S+1$ となります。

一般には、駅の並びは $(0,0),(0,1)$ や $(0,0),(0,0)$ や $(1,0),(0,0)$ といった形の部分列を含むことがあります。駅の並びからこれらのペアを最も多く組む方法であって、最後のペアの一番目の駅の添え字が最小のものを求めましょう。これは線形時間で貪欲に行うことができます。ここでのアイディアは、これらのペアはコストに影響を与えずに駅の並びから取り除くことができる、というものです。なぜなら、ペアの二番目の駅に先に行き、反転して一番目の駅に行ってそこからまた反転することができるからです。よって、これらの駅は無視することができ、残りの経路が一番目の駅を一度は通ることさえ保証すればよいです。ペアを最も多く組むのが最適なのは、これらのペアを取り除くと答えは $S+1$ か $S-1$ になるため、常に S を最小化するべきだからです。

これらのペアを取り除いたあと、$(0,1),(0,1),...,(0,1),(0,0),(1,0),...,(1,0)$ という形の駅の並びが残り、ここで $(0,0)$ の駅の数は 0 か 1 です。ここでの最小コストが $S-1$ か $S+1$ のどちらになるかを計算すればよいです。この解法の所要時間は $O(N)$ です。

E. Median Replace

まず、どのようなビット列が美しいか分析しましょう。

筆者の解法より説明しやすいため、ここでは admin の解法を説明します。

文字列を、最初の 1 の前の 0 の数、隣り合う 1 と 1 の間の 0 の数、最後の 1 の後の 0 の数を並べることで、配列として表しましょう。例えば、文字列 0010001011 に対応する配列は $[2,3,1,0,0]$ です。

文字列に対する操作を、配列に対する操作に「翻訳」することができます。行えるのは、配列中の $x \geq 3$ を $x-2$ に変えること、連続する二つの要素 x, y を $x+y-1$ に置き換えること、両端以外で二つ連続する 0 を削除すること、です。

2 より大きい奇数の要素をすべて 1 まで減らし、2 より大きい偶数の要素をすべて 2 まで減らしても操作に本質的な影響はないため、以下、配列の全要素は必ず区間 $[0,2]$ 内にあるとします。

すると、行える操作は以下のように簡略化されます：二つの連続した偶数であって「両方とも 0」ではないものを 1 で置き換える、1 を配列から削除する、両端以外で二つ連続する 0 を削除する。目標は、すべてが 0 であるような配列を得ることです。

1 は即座に取り除いてよいことに注意します。したがって、以下では配列は 0 と 2 のみを含むと仮定します。

実は、以上のように縮小された配列 $[a_1, a_2, ..., a_k]$ は、二つの添え字 $p < q$ であって $a_p = a_q = 0$ であり、p が奇数、q が偶数であるようなものが存在するとき、そしてそのときに限って目標を達成できます。

実際、この条件を満たさない配列にどのように操作を行っても、得られる配列はやはり条件を満たさず、また、条件を満たさないような小さい配列は美しくないことがわかります。さらに、この条件が満たされる場合、a_p と a_q の間の要素をすべて削除し、a_p より前の要素をすべて削除し（ただし先頭に 0 を二つ残しても可)、最後に後ろから要素を削除していく（ただし末尾に 0 を二つ残しても可）ことで配列のすべての要素を 0 にすることができます。

まとめると、はじめに最初の 1 の前の 0 の個数、1 と 1 に挟まれた 0 の個数、最後の 1 の後の 0 の個数を求め、配列に書き込みます。次に、1 より大きい偶数をすべて 2 に減らし、1 より大きい奇数をすべて 1 に減らし、1 をすべて削除します。残った配列に $a_p = a_q = 0, p < q, p$ が奇数で q が偶数であるような p, q があれば、その文字列は美しく、なければ美しくありません。

この判定条件を用いて、単純な DP を行うことで美しい文字列の数を $O(N)$ 時間で数えることができます。

筆者はこの問題に別の方向からアプローチしました。結果は以下の通りです。

・文字列が悪い（美しくない）のは以下の条件が満たされる場合で、またその場合に限られる。

- 文字列中で 1 が連続するのは最大で 3 個までである。
- 文字列が 3 個の連続する 1 を含むなら、それより左側の部分と右側の部分の長さは偶数でなければならず、またそれらは 00-bad （$b_i \in \{00, 01, 10\}$ として $0b_1b_2...b_k0$ という形の文字列）でなければならない。
- 文字列が 2 個の連続する 1 を含むなら、偶数長の部分は 00-bad でなければならず、奇数長の部分は悪くなければならない。
- 文字列が 2 個の連続する 1 を含まないなら、1 で始まって 1 で終わってはならない。

この性質を用いれば、$O(N)$ 時間で動作する DP の解法を素直に書くことができます。

F. Checkers

駒の位置は、係数（桁の値）が負になることを許した X 進法で考えることができます。すなわち、はじめ i 番目の駒の位置は $(0, 0, ..., 0, 1, 0, ..., 0)$ で表されます。ここで、1 となっているのはタプルの i 番目の要素です。

位置 a の駒が位置 b の駒を飛び越すとき、新たな駒が位置 $2b - a$ に生まれ、元々の二つの駒は取り除かれます。$N - 1$ 回の操作後の最後のタプルがどのようなものになるか分析しましょう。

重要なのは、タプルに現れる係数からなる多重集合だけである点に注意します。ある係数の多重集合が達成可能であると分かれば、その係数を並び替える方法の数をかけて答えに足せばよいです。したがって、以降は係数の多重集合がどのようになるかのみを考えます。

はじめ、$\{2^0\}$ が N 個あります。各ステップでは、二つの集合 A, B を新たな集合 $2B \cup -A$（重複した要素はそれぞれ加える）に併合します。最初にわかることは、係数は必ず 2 の累乗であることです。次にわかることは、どの集合の係数の和も常に 1 であることです。

さらに、どの集合に対しても、もし 2^i か -2^i がその集合に存在するなら、2^{i-1} か -2^{i-1} も必ずその集合に存在することにも注意します。最後に、集合には $\pm 2^0$ が必ずちょうど一つ存在します。これらはすべて必要条件であり、そのことは容易に示せます。

では、上記の条件を満たす集合はすべて最後に残りうる集合なのでしょうか？答えは No です。最小の反例は $\{-2^0, -2^1, -2^1, -2^1, -2^1, -2^1, 2^2, 2^3\}$ で、どのように操作してもこの係数の集合を得ることはできません。というわけで、必要条件をさらに見つけなければなりません。

この問題の難所は、以下の必要条件を見つけるところです。

すべての整数 i に対し、集合中の要素 $\pm 2^i$ の符号を変えることで、$0 \leq j \leq i$ であるような要素 $\pm 2^j$ の和を 1 にすることができる。

この条件を見つけたら、これが必要であることを証明するのは難しくありません（この条件を満たす二つの集合を併合するとどうなるか考えてみてください）。

そして実は、この条件は十分でもあります。数学的帰納法によりこれを示します。ここでのアイデアは、現時点での集合を $2B, -A$ という形の二つのよい部分集合に分けることが常に可能である、というものです。S を現時点での集合とします。$-2^0 \in S$ である場合のみを考えます（もう一つの場合も同様であるため）。k を、$+2^k$ が集合に存在するような最小の整数とします。もし、$1 \leq i < k$ のすべてに対し、S に -2^i が二度以上現れる場合は、$B = \{-2^0, -2^1, ..., -2^{k-2}, +2^{k-1}\}$ として、残りの要素を（符号を反転させて） A とします。もし、-2^i が S にちょうど一度現れるような i が存在する場合は、そのよう

な i のうち最小のものを考えます。$i = 1$ なら、$B = \{-2^0\}$ を選んで続けます。一方、$i > 1$ のときは $-2^0 - 3(2^1 + ... + 2^{i-1}) = -2^i + 1 - 2(2^1 + ... + 2^{i-1}) < -2^i - 1$ であるため、-2^i の符号を変えて和を 1 にすることは不可能であり、条件に矛盾します。

よい係数の集合の判定条件がわかったので、ありうるタプルの数を数える段階に進めます。小さい方から大きい方へと 2 の累乗を加えていくことで、タプルを構築します。また、必要が生じるたびに二項係数を掛けます。$dp[i][j]$ を、長さ i のありうるタプルであって、含まれる 2 の累乗たちの和が $1 + j \cdot V$ であるようなものの数とします。ここで、直前にタプルに含まれていた最大の 2 の累乗を $\frac{V}{2}$ としています（DP の遷移を行う上で、V の値を知る必要はないことに注意してください）。DP の遷移には、使われる $+V$ と $-V$ の数を列挙し、$\pm V$ の符号を付け替えて和を 1 とすることができるかを確認すればよいです。各状態から $O(N^2)$ 個の遷移があり、状態の数は $O(N^2)$ 個であるため、時間計算量は $O(N^4)$ となります（$O(N^5)$ の解法も前計算をせずとも問題なく通ります）。

AGC 022 Editorial

zscoder

30 March 2018

A. Diverse Word

If the length of the string is strictly less than 26, then the next diverse string in lexicographical order is the string appended by the smallest letter that has not yet appeared in the string.

Otherwise, the string has length 26. Let $S = p_1p_2p_3...p_{26}$ and suppose $p_ip_{i+1}...p_{26}$ is the longest suffix such that the letters are in descending order from left to right, i.e. $p_j > p_{j+1}$ for $i \leq j \leq 25$. If $i = 1$, the answer is -1, because S is the largest possible diverse string. Otherwise, the next smallest diverse string is $p_1p_2...p_{i-2}q$, where q is the smallest letter among $p_i, p_{i+1}, ..., p_{26}$ that is strictly larger than p_{i-1}. (this letter exist because otherwise $p_{i-1} > p_i$, contradicting the maximality of the suffix).

B. GCD Sequence

Firstly, instead of considering the gcd of each element a_i and the sum of the remaining elements, we can consider the gcd of each element a_i with the sum of all elements, S, because $\gcd(a_i, S - a_i) = \gcd(a_i, S)$. The constraints actually contain a small hint to the solution. The tight constraints suggests that we have to take around two-thirds of the numbers between 1 to N.

The main idea is that if we pick S to be a multiple of some small primes, then we just have to ensure a_i are divisible by the small primes and the gcd of everything is 1. This inspires the choice of $S = 6k$, for some integer k. Then, all we need to do is to pick a_i such that they are all multiples of 2 or 3, and make sure the gcd of every element is 1.

Coincidentally, around two-thirds of the numbers from 1 to N are divisible by 2 or 3, i.e. the numbers of the form $6k + 2, 6k + 3, 6k + 4$ and $6k$. Furthermore, if we pick both 2 and 3, then the gcd of the chosen elements will always be 1.

We're almost done. We just need to take care of some small cases and ensure that S is a multiple of 6. For $N \leq 5$, we can take $\{2, 5, 63\}, \{2, 5, 20, 63\}, \{2, 5, 20, 30, 63\}$ respectively (there are also other constructions, and note that the first 2 are conveniently given in the samples). Thus, from now we assume $N \geq 6$.

Let's take the numbers of the form $6k + 2, 6k + 3, 6k + 4, 6k + 6$ from small to large. The only issue is that the sum might not be divisible by 6. It can be easily seen that the sum is either congruent to $0, 2, 3, 5$ modulo 6 respectively. Let's consider each of them separately :

- If S is divisible by 6, we're done.
- If $S \equiv 2 \pmod 6$, then we remove 8 from our set and insert the next multiple of 6.
- If $S \equiv 3 \pmod 6$, then we remove 9 from our set and insert the next multiple of 6.
- If $S \equiv 5 \pmod 6$, then we remove 9 from our set and insert the next number of the form $6k + 4$.

Here's another simpler construction :

For $N = 3$, use the construction in the samples. If N is even, we can add the numbers in the order $2, 10, 3, 9, 4, 8, 6, 12, ...$ (the next 8 terms are the first 8 terms added by 12 and so on). If N is odd, we can add the numbers in the order $6, 2, 10, 3, 9, 4, 8, 12, ...$ (the next 8 terms are the first 8 terms added by 12 and so on).

C. Remainder Game

First, note that for each number v, if we have applied an operation on it with x, then it doesn't make sense to apply an operation on it with any $y \geq x$, since it will not change the value anymore. Thus, we may assume that the operations are done in strictly decreasing value of k. In particular, each value should be used at most once.

Since $2^x > 2^{x-1} + 2^{x-2} + ... + 2^0$, essentially we need to find the lexicographically smallest sequence of operations to convert a_i into b_i.

Clearly, we will not use any numbers larger than 50. We need to answer the question :

Suppose we fixed a set of values S to use. Is it possible to achieve the goal using only the elements in S?

If we can answer this question, then we can start from 50 and determine if we can finish the task using the set $\{1, 2, ..., 49\}$ in our operations. If it is possible, we move on to 49 and determine if we can finish the task using $\{1, 2, ..., 48\}$ in our operations. Whenever we reach an integer that we must use, we add it to the set S permanently. Continuing this process, we get the lexicographically smallest set of values needed to achieve the goal. We can also check whether the goal is impossible by answering the same question with $S = \{1, 2, ..., 50\}$ (though there is also a simple condition to check it).

Now, we focus on answering the subproblem. Construct a directed graph where the vertices are the numbers 1 to 50 and for each integer v, there is a directed edge from v to $v \bmod t$ for all $t \in S$. The task is possible if and only if each integer b_i is reachable from a_i (one direction follows from the definition of the graph, the other direction is also obvious since if each pair can be reached individually, we can combine the operations with the same value into one operation). This can be checked with BFS or DFS.

In the worst case, we'll answer the question $O(N)$ times (we let $N = 50$ for convenience), and to answer each question, we can do a bfs or dfs on each relevant vertex or just run Floyd-Warshall algorithm. Both of these algorithms take $O(N^3)$ time in worst case, since there can be $O(N^2)$ edges. Thus, the entire algorithm runs in $O(N^4)$ time (albeit with a small constant).

It is also possible to solve the problem in $O(N^3)$ time by not reconstructing the graph and redoing bfs/df for each question. This is left as an exercise for the reader.

D. Shopping

Note that we are essentially just calculating the minimum number of round trips the train should make before we can visit every station at least once and go back home. We'll calculate the number of trips in one direction (call this a round) made by the train and multiply the answer by L. (though note that the number of trips must be even, so the answer is always divisible by $2L$)

First, we describe the quadratic-time solution. Add a dummy station at coordinate 0. For each station, we will assign a value $+1$ if in our trip, the train enters the station from the right and departs from the right from this station after shopping (assume that the train enters the station at coordinate 0 from the right at the start), -1 if the train enters the station from the left and departs from the left, and 0 otherwise (enters from left, departs from right or vice versa). A valid path of the train can be described as an assignment of values $1, 0, -1$ to the stations such that every proper prefix of stations have positive sum and the sum of all values are 0. The cost of assignment of each value to each station can be computed in $O(1)$ time using the values of x_i and t_i. Now, we can let $dp[i][j]$ be the minimum cost to assign values for the first i stations so that the sum is j. This solution works in $O(N^2)$ time. This is enough to get the partial points.

Now, we move on to the linear-time solution.

Firstly, we can take all t_i modulo $2L$. However, care must be taken when t_i is divisible by $2L$, since we can't just remove the station (we will have to make sure we pass through it at least once). Anyway, from here we will assume $0 \leq t_i < 2L$.

Each station can be characterized by a pair (x, y), where x is 0 if when we stop at the station when the train is travelling from the left, and after we finish shopping the next train will come from the right, i.e. $t_i \leq 2(L - x_i)$, and 1 if the next train will come from the left (after going through one more round). y is 0 if when we stop at the station when the train is travelling from the right, and after we finish shopping the next train will come from the left, i.e. $t_i \leq 2x_i$, and 1 otherwise. Let's treat the stations with $t_i = 0$ (after reduction mod $2L$) as $(1, 1)$ stations, and subtract the answer appropriately later.

We can consider the train trip as a moving point, where we start from the left side (coordinate 0), and enter these stations in some order. Whenever we enter a station with $x = 0$ from the left, we rebound in the opposite direction (so now it moves from right to left), while if we enter a station with $x = 1$ from the left, we pass through it and increase our cost by 1. The move pattern is similar when entering a station from the right. When our point hit the left and rightmost point (coordinates 0 and L), it rebounds and the cost is also increased by 1. Our goal is to minimize the total cost of the trip (in the end we add the answer

with an appropriate constant and multiply the answer by L).

Note that we can almost ignore $(1, 1)$ stations, as we just pass through them and increase the cost by 1. However, we have to make sure our path passes through all these stations at least once later. For now, suppose there are no $(1, 1)$ stations.

Note that by definition, $(1, 0)$ can never appear before $(0, 1)$ stations.

Suppose our sequence of stations is of the form $(0, 1), (0, 1), ..., (0, 1), (0, 0), (1, 0), ..., (1, 0)$, where there can be 0 or 1 $(0, 0)$ stations. Let S denote the number of stations. The idea is that for each station, either we pass through it with the side with cost 1, or we make a rebound on the side with cost 0. However, the arrangement of stations of this form means that it is impossible to enter the next station without additional cost (either you pass through the new station on the side with cost 1, in which case we can ignore it, or you rebound from coordinate 0 or L). The only way a station will not incur an additional cost is if it's the last station visited. However, if at least one $(1, 0)$ station exist, we can that we're forced to rebound from the right end at least once, so we can prove the cost is at least $S + 1$. If no $(1, 0)$ stations exist, then the minimum cost is $S - 1$, unless there exist a station beyond the last station $(0, 1)$ or $(0, 0)$ which we must touch at least once (caused by either a $(1, 1)$ station or removed stations that we will see later), in which case the answer is still $S + 1$.

In general, our sequence of stations can have subsequences of the form $(0, 0), (0, 1)$, $(0, 0), (0, 0)$ and $(1, 0), (0, 0)$. Let's find the maximum pairing of these subsequences in our sequence of stations, where the index of the first station of the last matched pair is minimal. This can be done in linear time greedily. The idea is that we can remove these subsequences without affecting our cost, because we can go to the 2nd station of the pair first, rebound to the first station of the pair, and rebound from there again. So, we can ignore these stations and only have to make sure our remaining path passes through the first station at least once. The maximum matching is optimal because once we remove these pairs, our answer is either $S + 1$ or $S - 1$, so it is always best to minimize S.

After removing these pairs, we're left with a sequence of stations of the form $(0, 1), (0, 1), ..., (0, 1), (0, 0), (1, 0), ..., (1, 0)$, where there can be 0 or 1 $(0, 0)$ stations, and we can compute whether the minimum cost is $S - 1$ and $S + 1$. This solution takes $O(N)$ time.

E. Median Replace

Firstly, we'll analyze which binary strings are beautiful.

Below, I will describe the admin's solution since it's simpler to describe.

Let's represent our string as an array, denoting the number of zeroes before the first occurrence of 1, between two consecutive 1s and after the last occurrence of 1. For example, the array for the string 0010001011 is $[2, 3, 1, 0, 0]$.

The operations on the string can be translated to operations on the array. We can either change $x \geq 3$ in the array to $x-2$, change replace consecutive elements x, y with $x + y - 1$, or delete two consecutive 0s that are in the middle of the array.

Below we will assume that all elements of the array are always in the range $[0, 2]$, where all odd elements larger than 2 has been reduced to 1 and all even elements larger than 2 has been reduced to 2, since it doesn't really affect our operations.

Then, the operations are simplified to replace two consecutive even numbers which are not both 0 with 1, remove 1 from our array and remove 2 consecutive 0s that are in the middle of the array. Our goal is to reach an array with all zeroes.

Note that we can remove all the 1s in the array immediately. Thus, from now we will assume our array only consists of 0s and 2s.

We claim that the reduced array $[a_1, a_2, ..., a_k]$ works if and only if there exist two indices $p < q$ such that $a_p = a_q = 0$, p is odd, q is even.

Indeed, note that any operation on an array that doesn't satisfy the condition will also give an array that doesn't satisfy the condition, and we can see that the small arrays not satisfying these conditions are bad. Also, if this condition holds, we can remove all the elements between a_p and a_q, remove all the elements before a_p (except potentially a pair of leading 0s), and finally remove elements from the back (except potentially a pair of suffix 0s) until all elements of the array becomes 0.

To summarize, first we find the number of 0s before the first 1, between each pair of 1s and after the last 1 and write it into an array. Then, we reduce all even numbers larger than 1 to 2 and reduce all odd numbers larger than 1 to 1. Remove all the 1s in the array. If in the remaining array we can find p, q such that $a_p = a_q = 0$, $p < q$, p odd and q even, then the string is beautiful. Otherwise, it is not.

With the criteria above, we can do a simple dp to count the number of beautiful strings in $O(N)$ time.

The author approached the problem from a different direction and here's the result :

A string is bad (not beautiful) if and only if it satisfies the following :

- It contains at most 3 consecutive ones.
- If it contains 3 consecutive ones, the parts to the left and right must have even length, and must be 00-bad (which is of the form $0b_1b_2...b_k0$, where $b_i \in \{00, 01, 10\}$).
- If it contains 2 consecutive ones, the even-length part must be 00-bad while the odd-length part must be bad.
- If it does not contain 2 consecutive ones, it must not both start and end with 1.

With this property, it is straightforward to write a dp solution that works in $O(N)$ time.

F. Checkers

We can consider the positions of the checkers in base-X, possibly with negative coefficients. Thus, initially the i-th checker can be represented by $(0, 0, ..., 0, 1, 0, ..., 0)$, where the i-th element of the tuple is 1.

When a checker at position a hops over a checker at position b, a checker is created at position $2b - a$ and the two previous checkers are removed. Let's analyze how the final tuple looks like after $N - 1$ operations.

Note that the only important thing is the multiset of coefficients that appear in the tuple. Once we know that a multiset of coefficients is achievable, we can multiply it by the number of ways to arrange the coefficients and add it to the answer. Thus, from now on we'll only consider how the multiset of coefficients look like.

Initially, we have N copies of $\{2^0\}$. Every step, we can merge two sets A, B into a new set $2B \cup -A$ (elements are counted with multiplicity). The first observation is that the coefficients are always powers of 2. The next observation is that the sum of coefficients of every set is always 1. Also, note that for any set, if 2^i or -2^i exist in the set, then 2^{i-1} or -2^{i-1} must exist in the set. Finally, $\pm 2^0$ must appear exactly once in the set. All these are necessary conditions and are very easy to prove.

However, are all sets satisfying the above conditions valid final sets? The answer is no. The smallest counterexample is the set $\{-2^0, -2^1, -2^1, -2^1, -2^1, -2^1, 2^2, 2^3\}$. No matter how you try, you can never form this set of coefficients by the operations. Thus, we still need to find more necessary conditions.

The hard part of the problem is to find the following necessary condition :

For all integers i, it is possible to change the signs of the elements $\pm 2^i$ in the set such that the sum of the elements $\pm 2^j$ with $0 \le j \le i$ is equal to 1.

Once you find this condition, it is not hard to prove that it is necessary (just consider what happens when we merge two sets satisfying this condition).

In fact, this condition is also sufficient. We can use induction to prove this fact. The idea is that we can always find a partition of the current set into two good subsets of the form $2B$ and $-A$. Let S be our current set. We'll only look into the case where $-2^0 \in S$ since the other case is similar. Let k be the smallest integer such that $+2^k$ exist in the set. If for all $1 \le i < k$, -2^i appears at least twice in S, then we can choose $B = \{-2^0, -2^1, ..., -2^{k-2}, +2^{k-1}\}$ and the remaining elements (negated) goes to A. If some i exist so that -2^i appears exactly once in S, then consider the smallest such i. If $i = 1$, we can just pick

$B = \{-2^0\}$ and proceed. However, if $i > 1$, then $-2^0 - 3(2^1 + ... + 2^{i-1}) = -2^i + 1 - 2(2^1 + ... + 2^{i-1}) < -2^i - 1$, so it is impossible to change the sign of -2^i so that the sum is 1, which contradicts the condition.

Once we have the criterion of good sets of coefficients, we can proceed to the count the number of valid tuples. We build the tuples starting from the smallest power to the largest power. We will multiply binomial coefficients whenever necessary. Let $dp[i][j]$ be the number of valid tuples of length i such that the sum of the powers are $1+j \cdot V$, where the previous largest power of 2 is $\frac{V}{2}$. (Note that it is not necessary for us to know the value of V to do the dp transitions). For the dp transitions, we just iterate through the number of $+V$ and $-V$ taken and check whether it is possible to reassign the signs for the $\pm V$s so that the sum is 1. The dp transitions are $O(N^2)$ and there are $O(N^2)$ states, so the time complexity is $O(N^4)$. ($O(N^5)$ solutions can also pass without problems even without precomputation)

AGC 021 解説

DEGwer

2018/02/24

For International Readers: English editorial starts on page 6.

A: Digit Sum 2

与えられた数 N が 10 進法で K 桁からなるとします。最上位の桁が c であるとすれば、N 以下の正の整数は、

- 下から K 桁目は c 以下
- それ以外の位は 9 以下

です。よって、答えは $c+9(K-1)$ 以下となります。

また、答えが $c+9(K-1)$ のとき、条件を満たす整数は $c999\cdots999$ のみです。この値が N 以下となるような N は $c999\cdots999$ の形をしているものしかありえないため、この場合のみ答えが $c+9(K-1)$ となります。

そうでない場合、整数 $(c-1)999\cdots999$ は N 以下で、各桁の和は $c+9(K-1)-1$ となります。よってこれが最適で、すべての場合について答えが求まりました。

時間計算量は $O(\log N)$ です。

B: Holes

R を大きくしていくことを考えましょう。与えられた点集合の凸包上にない穴について、りんごさんの選ぶ点であって、その穴にすぬけ君が落ちるような領域の面積は定数で抑えられます。よって、R を大きく取れば、求める確率は 0 に収束します。また、凸包の辺上にある場合も、そのような領域の面積は $O(R)$ なので、R を大きく取れば求める確率は 0 に収束します。

そうでない場合、穴 p について確率を求めることを考えましょう。与えられた点集合の凸包上で p に隣り合う穴をそれぞれ q,r とします (q,r は同一の穴であることもあります)。りんごさんの選ぶ点であって、穴 p にすぬけ君が落ちるような領域の面積は、pq の垂直二等分線と qr の垂直二等分線によって 4 つに分けられた領域のうちの p を含む部分 (以後これを T と呼ぶ) から、p 付近の高々有限の面積の領域を除いたものです。この高々有限の面積の部分は極限操作によって無視されることになるので、T と R の共通部分の面積について考えればよいことがわかります。

さて、2 本の垂直二等分線の交点を原点に持っていったとき、T の面積の変化分は $O(R)$ です。よって極限操作によってそのずれは無視され、結局、2 本の垂直二等分線の角度が 2π に対して占める割合が、求める確

率となります。

以上は、$O(N \log N)$ 時間で動くように実装できます。

C: Tiling

面積の条件から、$2(A+B) \leq NM$ が必要です。

さて、以下の 4 通りに分けて考えることにしましょう。

N, M がともに偶数の場合

奇数列目を黒、偶数列目を白に塗ると、縦向きのタイルは同じ色のマス 2 つを、横向きのタイルは異なる色のマス 2 つを覆います。各色のマスは偶数個ずつあるので、もし $2(A+B) = NM$ なら、横向きのタイルは偶数枚ある必要があります。縦向きのタイルについても同様です。

逆に、その条件が満たされる場合、以下のようにして条件を満たすタイルの置き方が構成できます。

- 全体を 2×2 の領域に区切る。各領域を、縦向きのタイル 2 枚または横向きのタイル 2 枚で適切に埋める。
- 余分なタイルを取り除く。

N が偶数、M が奇数の場合

先ほどと同様、奇数列目を黒、偶数列目を白に塗ることを考えると、白のマスは $(M-1)N/2$ 個しかないので、横向きのタイルは $(M-1)N/2$ 枚しか置くことはできません。また、もし $2(A+B) = NM$ なら、先ほどと同様、横向きのタイルは偶数枚ある必要があります。

逆に、その条件が満たされる場合、以下のようにして条件を満たすタイルの置き方が構成できます。

- 右端の列を、縦向きのタイルで埋める。
- 残った部分を 2×2 の領域に区切る。各領域を、縦向きのタイル 2 枚または横向きのタイル 2 枚で適切に埋める。
- 余分なタイルを取り除く。

N が奇数、M が偶数の場合

先ほどの場合と対称です。

N, M がともに奇数の場合

先ほどと同様、横向きのタイルは $(M-1)N/2$ 枚しか置くことはできません。同様に、縦向きのタイルは $(N-1)M/2$ しか置くことはできません。

逆に、その条件が満たされる場合、以下のようにして条件を満たすタイルの置き方が構成できます。

$NM-1=2(A+B)$ でないか、縦向きのタイルの枚数と、置ける縦向きのタイルの最大枚数の偶奇が等しい場合

- 右端の列の下端のマス以外を、縦向きのタイルで埋める。
- 下端の列の右端のマス以外を、横向きのタイルで埋める。
- 残った部分を 2×2 の領域に区切る。各領域を、縦向きのタイル 2 枚または横向きのタイル 2 枚で適切に埋める。
- 余分なタイルを取り除く。

そうでない場合

- 右下の 3×3 領域に、縦向きのタイル 2 枚と横向きのタイル 2 枚を四畳半敷きの塩梅で置く。
- 右端の列の残った部分を、縦向きのタイルで埋める。
- 下端の列の残った部分を、横向きのタイルで埋める。
- 残った部分を 2×2 の領域に区切る。各領域を、縦向きのタイル 2 枚または横向きのタイル 2 枚で適切に埋める。
- 余分なタイルを取り除く。

正当性は、N か M が 1 の場合は必ず前者の場合になることと、前者と後者で最後から 2 番目のステップで置ける横向きのタイルの枚数の偶奇が異なることがわかるので、証明できます。

以上で全ての場合について解けたので、この問題を解くことができました。

D: Reversed LCS

T と T を反転した文字列 T' の最長共通部分列はどのようなものかを、まず考えることにしましょう。

$T_{i_1},T_{i_2},\cdots,T_{i_k}$ と $T_{j_1},T_{j_2},\cdots,T_{j_k}$ (ただし、$i_1<\cdots<i_k, j_1>\cdots>j_k$ が成り立つ) が列として等しいとします。このとき、ある整数 p が存在し、$i_p<j_p$ かつ $i_{p+1}\geq j_{p+1}$ となります (ただし、便宜上 $i_{k+1}=|T|+1, j_{k+1}=-1$ とします)。

さて、$i_{p+1}>j_{p+1}$ のとき、列 $T_{i_1},\cdots,T_{i_p},T_{j_p},\cdots,T_{j_1}$ と列 $T_{j_k},\cdots,T_{j_{p+1}},T_{i_{p+1}},\cdots,T_{i_k}$ はともに回文で、長さの合計は $2k$ です。すなわち、T の部分列であって、回文であり、長さが k 以上のものが存在します。

$i_{p+1}=j_{p+1}$ のときも同様に、列 $T_{i_1},\cdots,T_{i_p},T_{i_{p+1}},T_{j_p},\cdots,T_{j_1}$ と列 $T_{j_k},\cdots,T_{j_{p+2}},T_{i_{p+1}},T_{i_{p+2}},\cdots,T_{i_k}$ はともに回文で、長さの合計は $2k$ です。すなわち、T の部分列であって、回文であり、長さが k 以上のものが存在します。

回文は T と T' の共通部分列となるので、最長共通部分列としては回文であるもののみ考えればよいです。よって、$DP[l][r][x]$ を、区間 $[l,r]$ の文字を x 個まで変更したときの、その区間の部分列であって回文であるものの最大長として DP を行うことで、この問題を $O(N^3)$ 時間で解くことができます。

E: Ball Eat Chameleons

投げ入れる赤のボールの個数で場合分けをします。赤のボールを R 個、青のボールを B 個投げ入れるときの答えを高速に求められれば、$R+B=K$ なる非負整数の組 (R,B) すべてに対してそれらを足し合わせる

ことで、求める答えを得ます。

$R < B$ のとき

いずれかのカメレオンは、食べた青のボールの個数が食べた赤のボールの個数より多くなります。そのカメレオンは最終的に青色となるので、求める場合の数は 0 です。

$R = B$ のとき

全てのカメレオンが、赤と青のボールを等しい個数ずつ食べます。赤と青のボールを等しい個数ずつ食べたカメレオンが最終的に赤になっているための必要十分条件は、ボールを 1 つ以上食べ、かつ最後に食べたボールが青であることです。

全てのカメレオンの最後に食べたボールが青であることから、最後にりんごさんが投げ入れるボールは青である必要があります。また、すべてのカメレオンは最後に赤いボールを食べた後に青いボールを一度以上食べるので、りんごさんが投げ入れたボールを順に並べた列から、(赤, 青) とこの順に並ぶ (連続するとは限らない) 組が disjoint に N 組取れる必要があります。

逆に、これらの条件が満たされれば、(最後に青いボールを食べるカメレオンに、組にならないボールをすべて食べさせることで) 全てのカメレオンを最終的に赤色にすることができます。

格子上を x 座標または y 座標の増加する方向に $R+B$ 回進んで (R, B) に至る経路をりんごさんの投げ入れたボールを順に並べた列に自然に対応付けることにすれば、この条件は、

- $(R, B-1)$ を通る
- $y - x \geq B - N + 1$ なる領域を通らない

と言い換えられます。これは、後述の方法で求められます。

$R > B$ のとき

上の場合とだいたい同じですが、赤いボールのほうを多く食べるカメレオンが存在するため、最後に投げ入れるボールが青である必要はありません (赤いボールのほうを多く食べるカメレオンに、余計なボールをすべて食べさせれば良いです)。

赤と青のボールを同数食べるカメレオンの数は最小で $N - (R - B)$ なので、上の場合と同様の言い換えをすれば、条件は

- $y - x \geq R - N + 1$ なる領域を通らない

と書くことができます。これは、後述の方法で求められます。

さて、あとは $y - x \geq T$ なる領域を通らずに $(0, 0)$ から (X, Y) まで格子上を最短距離で進む経路の個数を高速に求められれば良いです。

$(0, 0)$ から (X, Y) まで行く経路のうち、$y - x \geq T$ なる領域を通る経路について、初めて $y - x = T$ となった地点から先の部分を折り返せば、$(0, 0)$ から $(Y - T, T + X)$ まで行く経路となります。逆に、$(0, 0)$ から $(Y - T, T + X)$ まで行く経路を同様に折り返せば、$(0, 0)$ から (X, Y) まで行く経路のうち、$y - x \geq T$ なる領域を通る経路 になります。よって、両者の個数は等しいので、求める場合の数は、$(0, 0)$ から (X, Y)

まで行く経路の個数 $\binom{X+Y}{X}$ から、$(0,0)$ から $(Y-T,T+X)$ まで行く経路の個数 $\binom{X+Y}{Y-T}$ を引いた値となります。

これは適切な前処理をしておけば定数時間で求められるので、$O(K)$ 時間でこの問題を解くことができました。

F: Trinity

$DP[k][p]$ を、p 行 k 列のマス目のいくつかを黒く塗ったときの列の組 (A,B,C) であって、$A_i = k+1$ なる i が存在しないようなものの個数とします。すなわち、すべての行に黒く塗られたマスが存在する必要があります。求める答えは $DP[M][p]$ たちを用いて簡単に計算できるため、以下、この DP を計算することを考えます。

$DP[k][p]$ から $DP[k+1][p+q]$ への遷移につく係数は何でしょうか？ 以下の 2 通りに分けて考えてみましょう。

$q=0$ の場合、黒く塗られたマスの存在する行が新たに出現することはありません。すなわち、k 列目までには黒く塗られたマスがなく、$k+1$ 列目に初めて存在するような行はありません。

このとき、A_i の値が新たに定義されることはありません。B_{k+1}, C_{k+1} の値が新たに定義されますが、これは $k+1$ 行目に黒く塗られたマスが存在しない場合 1 通り、存在する場合 $\binom{p+1}{2}$ 通りです。よって、遷移の係数は $\binom{p+1}{2}+1$ です。

$q>0$ の場合、p 個の行の間に新たに q 個の行が挿入され、$A_l = k+1$ なる l が新たに q 個出現します。このような l の位置から、新たに出現した行の位置は一意的に復元できます。さて、B_{k+1} がこの $p+q$ 個の行のうち上から $i+1$ 番目の行の番号に等しくなるためには、上から i 個の行は元からあった行でなければなりません。同様に、C_{k+1} がこの $p+q$ 個の行のうち下から $j+1$ 番目の行の番号に等しくなるためには、下から j 個の行はもとからあった行でなければなりません。それ以外は任意に決められるため、遷移につく係数は、

$$\sum_{i+j\le p} \binom{p-i-j+q}{q} = \sum_{i=0}^{p} (i+1)\binom{p+q-i}{q} \tag{1}$$

$$= \sum_{i=0}^{p} \binom{p+q+1-i}{q+1} \tag{2}$$

$$= \binom{p+q+2}{q+2} \tag{3}$$

となり、$O(N^2M)$ 時間でこの DP が計算できます。

さて、後者の遷移につく係数をよく見てみましょう。$\binom{p+q+2}{q+2} = \frac{(p+q+2)!}{p!(q+2)!}$ より、遷移の係数は p の式、q の式、$p+q$ の式の積に分解できます。よって、$X_p = \frac{DP[k][p]}{p!}, Y_q = \frac{1}{(q+2)!}$ として FFT などで畳み込みを行い、係数 $(p+q+2)!$ を後からかけることで、遷移を $O(N\log N)$ 時間で計算できます。前者の遷移は $O(N)$ 時間で計算できるので、$O(NM\log N)$ 時間でこの問題を解くことができました。

AGC 021 Editorial

DEGwer

2018/02/24

A: Digit Sum 2

Suppose that N has K digits, and the leftmost (the most significant) digit is c. Consider the following two cases:

- N is of the form $c999\cdots999$. In this case, for all $x \leq N$, the K-th (from the smallest) digit of x is at most c, and each of the other digits is at most 9, thus the sum is at most $c + 9(K-1)$. By choosing $x = N$, we can achieve the sum $c + 9(K-1)$. Therefore, the answer is $c + 9(K-1)$.
- Otherwise, the answer must be less than $c + 9(K-1)$. On the other hand, the integer $(c-1)999\cdots999$ is less than N (because the leftmost digit is smaller), and achieves the sum $c+9(K-1)-1$. Therefore, the answer is $c + 9(K-1) - 1$.

This solution works in $O(\log N)$ time.

B: Holes

Since R is extremely large, we can assume that a randomly chosen point is sufficiently distant from the holes.

Let's call the chosen point s. When will it fall into a hole p? For each hole q (other than p), $dist(s,p) < dist(s,q)$ must hold. Since s is sufficiently distant from the holes, this condition holds if and only if the angle $\angle spq > \pi/2$.

This gives an very simple $O(N^2 \log N)$ solution. Fix a hole p. Compute the directions from p to each of the other holes, and sort those directions. The probability that p is chosen is $\max\{(\text{the longest gap between two adjacent directions} - \pi), 0\}/(2\pi)$.

Note that we can improve it to $O(N \log N)$. Let's take the convex hull of all holes. If a hole p is not a vertex of the convex hull, the probability that p is chosen is 0. Otherwise, let q, r be the two vertices of the convex hull adjacent to p. Then, he probability that p is chosen is $(\pi - \angle qpr)/(2\pi)$.

C: Tiling

The following three conditions are necessary:

- $A + B \leq \lfloor \frac{NM}{2} \rfloor$ (compare areas)
- $A \leq N\lfloor \frac{M}{2} \rfloor$ (we can put $\lfloor \frac{M}{2} \rfloor$ tiles per row)
- $B \leq M\lfloor \frac{N}{2} \rfloor$ (we can put $\lfloor \frac{N}{2} \rfloor$ tiles per column)

Let's define $S := \lfloor \frac{NM}{2} \rfloor, S_A := N\lfloor \frac{M}{2} \rfloor, S_B := M\lfloor \frac{N}{2} \rfloor$. Then, $A + B \leq S, A \leq S_A, B \leq S_B$ must hold.
On the other hand, we can come up with the following way to put tiles:

- First, put 2×2 squares from the top-left corner (in the most natural way). We will get $\lfloor \frac{N}{2} \rfloor \dot{\lfloor} \frac{M}{2}$ squares.
- We divide each square vertically or horizontally: for each square, we will get two tiles of the same type.
- Additionally, we can put $\lfloor \frac{M}{2} \rfloor$ 1×2 tiles in case N is odd, and $\lfloor \frac{N}{2} \rfloor$ 2×1 tiles in case M is odd.

By this method, we can achieve $(A, B) = (S_A, S-S_A), (S_A-2, S-S_A+2), (S_A-4, S-S_A+4), \ldots, (S-S_B, S_B)$. (And obviously, if $A' \leq A$ and $B' \leq B$, we can achieve (A', B') if we can achieve (A, B).)

Therefore, the only non-trivial cases are $(A, B) = (S_A - 1, S - S_A + 1), (S_A - 3, S - S_A + 3), \ldots, (S - S_B + 1, S_B - 1)$.

It turns out that the answer for these cases depend on the parity of NM:

In case NM is odd

Note that the non-trivial cases don't exist when $N = 1$ or $M = 1$. Thus, when N and M are odd, we can assume that $N, M \geq 3$.

Take a 3×3 square from the bottom-right corner, and put two tiles of each type. In the previous method, this part was filled by an odd number of tiles of each type. Thus, we can achieve the different parity this way. By filling the remaining part in a similar way, we can achieve $(A, B) = (S_A - 1, S - S_A + 1), (S_A - 3, S - S_A + 3), \ldots, (S - S_B + 1, S_B - 1)$.

In case NM is even

Paint the entire grid black and white: a cell in odd-numbered rows are black and other cells are white. Then, each 1×2 tiles contains even number of black cells, and each 2×1 tiles contains odd number of black cells. Thus, the parity of total number of 2×1 tiles must match the parity of total number of black cells.

Therefore, in this case, it turns out that the answer for non-trivial cases are "NO".

D: Reversed LCS

First, let's find a simple way to compute the length of LCS of T and T'.

Suppose that the length of LCS is k. Then, there exist indices $i_1 < \cdots < i_k$ and $j_1 > \cdots > j_k$ such that the sequences $T_{i_1}, T_{i_2}, \cdots, T_{i_k}$ and $T_{j_1}, T_{j_2}, \cdots, T_{j_k}$ are the same.

Then, there exists an integer p such that $i_p < j_p$ and $i_{p+1} \geq j_{p+1}$. (For convenience, let $i_{k+1} = |T| + 1$ and $j_{k+1} = -1$.)

If $i_{p+1} > j_{p+1}$, both the subsequence $T_{i_1}, \cdots, T_{i_p}, T_{j_p}, \cdots, T_{j_1}$ and the subsequence $T_{j_k}, \cdots, T_{j_{p+1}}, T_{i_{p+1}}, \cdots, T_{i_k}$ are palindromes, and the sum of their lengths is $2k$. Thus, T contains a sub-palindrome whose length is at least k.

Similarly, if $i_{p+1} = j_{p+1}$, both the subsequence $T_{i_1}, \cdots, T_{i_p}, T_{i_{p+1}}, T_{j_p}, \cdots, T_{j_1}$ and the subsequence $T_{j_k}, \cdots, T_{j_{p+2}}, T_{i_{p+1}}, T_{i_{p+2}}, \cdots, T_{i_k}$ are palindromes, and the sum of their lengths is $2k$. Thus, T contains a sub-palindrome whose length is at least k.

Therefore, we proved that the length of LCS of T and T' is actually the length of the longest (not necessarily continuous) sub-palindrome of T.

Now, we are interested in the longest possible sub-palindrome we can make by changing at most K characters in T. Define $DP[l][r][x]$ as the length of the longest possible sub-palindrome we can make by changing at most x characters in $T[l..r)$ (a substring of T). This DP works in $O(N^3)$ time.

E: Ball Eat Chameleons

A chameleon becomes red if one of the following holds:

- It ate more red balls than blue balls.
- It ate the same (nonzero) number of red balls and blue balls, and the last ball it ate is blue.

Suppose that there are R red balls and B blue balls in total. For each pair (R, B) such that $R+B = K$, we will compute the number of valid sequences of R red balls and B blue balls. A sequence of R red balls and B blue balls is valid if we can divide it N disjoint (possibly non-continuous) subsequences such that for each subsequence, one of the two conditions above hold. Let's compute the number of such sequences.

In case $R < B$

Obviously, the answer is 0.

In case $R = B$

In this case, all subsequences must contain the same number of reds and blues, and end with a blue. Thus, the following conditions are necessary:

- The last element of the sequence must be a blue.
- It must be possible to choose N disjoint subsequences "red, blue".

It turns out that these conditions are sufficient: we assign each of the N pairs to different chameleons, and assign everything else to the chameleon with the last blue ball.

Now, let's plot a sequence on an xy-grid. A sequence corresponds to a path from $(0, 0)$ to (R, B). We see the sequence from left to right, and when we see an element red (blue), we extend the path to the right (up).

Then, the conditions above correspond to the following:

- It is a path from $(0, 0)$ to (R, B) that goes only up and right, and passes through $(R, B - 1)$.
- It never enters the region with $y - x \geq B - N + 1$.

The number of such paths can be computed in $O(1)$, and it will be described later.

In case $R > B$

It's very similar to the case above, but this time we are not interested in the last ball. A sequence is good iff:

- It must be possible to choose $\max N - (R - B), 0$ disjoint subsequences "red, blue".

(note that this time an arbitrary chameleon with (red) > (blue) receives "everything else")
and if we describe it in the word of paths,

- It is a path from $(0, 0)$ to (R, B) that goes only up and right.
- It never enters the region with $y - x \geq R - N + 1$.

Therefore, we can compute the answer for each (R, B) in $O(1)$, and the solution works in $O(K)$ time.
The only remaining thing is the computation of the number of paths with the following conditions:

- It is a path from $(0, 0)$ to (X, Y) that goes only up and right.
- It never enters the region with $y - x \geq T$.

This is a well-known problem (for computation of Catalan numbers).

Since we can easily compute the number of paths from $(0, 0)$ to (X, Y), we are interested in the number of such paths that enters the region $y - x \geq T$ at least once (let's call it a bad path).

Let's fix a bad path P. Suppose that P enters the region for the first time at the point (p, q). Define a path Q as follows:

- From $(0, 0)$ to (p, q), Q follows P.
- After that, Q is a "reflection" of P: when P goes right, Q goes up, and when P goes up, Q goes right.

Then, Q is a path from $(0, 0)$ to $(Y - T, T + X)$. (Notice that $p - q = T$).
This gives a bijection between a bad path and a path from $(0, 0)$ to $(Y - T, T + X)$.
Therefore, the answer for this is simply $\binom{X+Y}{X}$ minus $\binom{X+Y}{Y-T}$.

F: Trinity

Let $DP[k][p]$ be the answer of the problem for the grid with p rows and $k$columns, with an additional condition that in each row we must paint at least one cell black. Since the answer is $\sum DP[M][p]\binom{N}{p}$, from now on, we will focus on the computation of this DP table.

Fix a grid with k rows and p columns (this time, some rows may be empty). We have three arrays A, B, C that corresponds to this grid. What happens to the three arrays if we attach the $(p+1)$-th column to the right?

- If the i-th row is empty before the attachment, A_i will be $p+1$. Otherwise, the value of A_i doesn't change.
- The first p elements of B, C don't change.
- We get two values B_{p+1}, C_{p+1}: these two values depend on the newly added column.

Let's call a grid nice if each row is non-empty. We can make a nice grid with $p+q$ rows and $k+1$ columns from a grid with p rows and k columns as follows:

- Initially we have a nice grid with p rows and k columns.
- Insert q new empty rows into some positions. Now we have a grid with $p+q$ rows and k columns.
- Attach a new (not necessarily empty) column to the right. Cells that correspond to empty rows must be black. Now we have a nice grid with $p+q$ rows and $k+1$ columns.

We update the DP table as in the process above. We perform "$DP[k+1][p+q] += DP[k][p] * (coefficient)$;" for each pair (k, p, q) in the increasing order of k. The coefficient is the number of ways to get a new three arrays A, B, C in the process above: it turns out that the initial content of the three values doesn't affect this coefficient.

How to compute this coefficient?

If $q = 0$, we only care the last values of two arrays B, C. These are the positions of the topmost/bottommost black cell in the newly added column. There is one way for an empty column, and there are $\binom{p+1}{2}$ ways for a non-empty column. Thus, the coefficient is $\binom{p+1}{2} + 1$.

What happens if $q > 0$? We insert q new rows into p old rows. We can see this as a sequence of p gray balls and q black balls (the black balls corresponds to newly inserted rows). The positions of the black balls corresponds to the positions of l such that $A_l = k+1$.

Furthurmore, we should think about the last elements of B, C. In the word of balls, these correspond to the leftmost/rightmost positions of black balls. However, the problem is that, we don't know the information about gray balls: it can be either black or not black. Therefore, the combinatorial interpretation of the coefficient is as follows:

- Imagine a sequence of p gray balls and q black balls.
- Consider all gray balls that are to the right of the rightmost black ball: you can choose at most one of them and paint it red.

- Consider all gray balls that are to the left of the leftmost black ball: you can choose at most one of them and paint it red.

The annoying part is "at most". Instead, we attach an extra gray ball to each end, and now it becomes as follows:

- Imagine a sequence of $p+2$ gray balls and q black balls. Both ends are gray.
- Consider all gray balls that are to the right of the rightmost black ball: you can choose exactly one of them and paint it red.
- Consider all gray balls that are to the left of the leftmost black ball: you can choose exactly one of them and paint it red.

Now, we can regard red balls as black, and it's just a sequence of p gray balls and $q+2$ black balls. Thus, the coefficient turns out to be $\binom{p+q+2}{q+2}$.

Now we get the coefficient. This solution works in $O(N^2M)$ time, and is fast enough to get partial scores.

How can we improve this? We have two arrays x, y. Initially x is given and y is empty. For each (p, q) such that $q > 0$, we perform the following:

$y_{p+q} += x_p \times \binom{p+q+2}{q+2}$

(We can ignore the case $q = 0$ because it can be done in $O(N)$ obviously).

We want to perform this operation quickly. Let's take a close look at the coefficient:

$\binom{p+q+2}{q+2} = \frac{(p+q+2)!}{p!}$

Thus, by defining $X_p = \frac{x_p}{p!}$ and $Y_q = \frac{y_q}{(q+2)!}$, it changes to the following:

$y_{p+q} += X_p \times \frac{1}{(q+2)!}$

This is just a standard convolution and can be done in $O(N \log N)$. Now the solution works in $O(NM \log N)$.

AtCoder Grand Contest 020 Editorial (Japanese)

writer: tourist

2018 年 1 月 14 日

For International Readers: English editorial starts on page 9.

A: Move and Win

まず、プレイヤーが負けになるのは、動かしたい駒がもう一つの駒と紙切れの端に挟まれた場合のみであることに注意します。

二つの駒の間の距離を $D = B - A$ とします。毎ターンに D が $+1$ か -1 変動することに注意すると、D の偶奇は毎ターン変化することがわかります。

仮に、アリスの最初のターンの前の D の初期値が偶数であるとします。すると、アリスにターンが回ってきたとき D は必ず偶数で、ボリスにターンが回ってきたとき D は必ず奇数となります。

ここで、二つの駒が隣接するとき、必ず $D = 1$ であることに注意します。したがって、D が偶数であれば、二つの駒は隣接しておらず、アリスは必ず駒を動かすことができます。

視点を変えると、アリスは必ず駒を動かすことができるため、いずれ駒をマス $N - 1$ に動かすことができます。このとき、ボリスの駒はマス N に閉じ込められ、アリスの勝ちになります。

同様に、D の初期値が奇数である場合、ボリスが駒をマス 2 に動かして勝ちます。

よって、答えは $B - A$ の偶奇に依存し、この値が偶数ならアリスの勝ち、奇数ならボリスの勝ちとなります。

B: Ice Rink Game

ラウンド i の直前の時点で脱落せず残っている子供の人数を X とします。ラウンド i の直後には何人残るでしょうか？作られるグループの個数は明らかに $\lfloor \frac{X}{A_i} \rfloor$ であるため、$\lfloor \frac{X}{A_i} \rfloor \cdot A_i = X - X \bmod A_i$ 人が残ることになります。以後、$f_p(X) = X - X \bmod p$ と書きます。

この問題では、次のような N の値をすべて求めることが要求されています。

$$g(N) = f_{A_K}(f_{A_{K-1}}(\dots f_{A_1}(N)\dots)) = 2$$

主に二つの方針が考えられます。

線形走査

最終的に 2 人が生き残るような、ラウンド i の直後での最小の生存人数を L_i とし、同様に最大の生存人数を R_i とします。$L_K = R_K = 2$ であることが分かっていて、求めたいものは L_0, R_0 です。

L_i, R_i から L_{i-1}, R_{i-1} を復元することができるでしょうか？答えは Yes です。

まず、ラウンド i の直後での生存人数は A_i で割り切れなければなりません。もし L_i と R_i の間に A_i の倍数が存在しない場合、答えは -1 となります。

そうでない場合、区間 $[L_i, R_i]$ 内の A_i の倍数のうち最小のものと最大のものをそれぞれ X_i, Y_i とすると、

$$\begin{aligned} L_{i-1} &= X_i, \\ R_{i-1} &= Y_i + A_i - 1. \end{aligned}$$

となります。

ここで、$X_i = \lceil \frac{L_i}{A_i} \rceil \cdot A_i, Y_i = \lfloor \frac{R_i}{A_i} \rfloor \cdot A_i$ であるため、結局、

$$\begin{aligned} L_{i-1} &= \lceil \tfrac{L_i}{A_i} \rceil \cdot A_i, \\ R_{i-1} &= \lfloor \tfrac{R_i}{A_i} \rfloor \cdot A_i + A_i - 1. \end{aligned}$$

となります。

この解法の計算量は $O(K)$ です。

二分探索

f には、広義単調増加であるという便利な性質があります。正式には、$X \le Y$ のとき $f_p(X) \le f_p(Y)$ となります。

広義単調増加同士の合成関数もまた広義単調増加であることに注意します。今回は、$X \le Y$ のとき $g(X) \le g(Y)$ となります。

二分探索を用いて、$g(N) \ge 2$ であるような最小の N を求め、また別に二分探索を用いて、$g(N) \le 2$ であるような最大の N を求めることができます。明らかに、これらの値が問題の答えです（前者が後者を上回る場合は -1）。

答えが $2 + K \cdot \max(A_i)$ 以下であることは簡単にわかります。なぜなら、毎ラウンドの脱落者の数はたかだか $\max(A_i)$ 人であるためです。したがって、この解法の計算量は $O(K \log(K \cdot \max(A_i)))$ となります。

C: Median Sum

$S_0 = 0$ とします。これは A の空の部分列に対応します。S_{2^N-1} が A のすべての要素の和であることに注意します。

A のすべての部分列を二つずつペアにします。このとき、すべてのペアについて、A のすべての要素がペアのちょうど一方に含まれるようにします（したがって、それぞれの部分列は自分自身と相補的な部分列と組むことになります)。明らかに、これらのペアの数は 2^{N-1} です。

任意の部分列のペア (P_i, Q_i) を考え、P_i, Q_i の要素の和をそれぞれ $\Sigma_{P_i}, \Sigma_{Q_i}$ とします。一般性を失うことなく、$\Sigma_{P_i} \leq \Sigma_{Q_i}$ と仮定します。ペアの組み方より $\Sigma_{P_i} + \Sigma_{Q_i} = S_{2^N-1}$ であるため、$\Sigma_{P_i} \leq \frac{1}{2} S_{2^N-1}, \Sigma_{Q_i} \geq \frac{1}{2} S_{2^N-1}$ となります。

すべての P_i は、$\frac{1}{2} S_{2^N-1}$ という値を隔てて、すべての Q_j から分離されていることがわかります。したがって、Σ_{P_i} は S の前半 $S_0, S_1, \ldots, S_{2^{N-1}-1}$ に属し、Σ_{Q_i} は S の後半 $S_{2^{N-1}}, \ldots, S_{2^N-1}$ に属するとみなして構いません。

よって、$S_{2^{N-1}}$ の値を求めるには、A の部分列であって、和が $\frac{1}{2} S_{2^N-1}$ 以上で最小であるものを求める必要があります。

これには動的計画法を用いることができます。A の最初の i 項の部分列であって、和が j であるものが存在するか否かを $f(i, j)$ で表します。$f(i, j)$ の値はブール値であり、遷移は bitwise OR で行うことができるため、この動的計画法は bitset （例えば C++ の `std::bitset`、Java の `java.util.BitSet` がありますが、手書きの bitset でも十分で、実装も容易です）を用いて高速化することができます。

この解法の計算量は $O(\frac{N^2 \max(A_i)}{64})$ となります。

D: Min Max Repetition

クエリを一つ一つ処理しましょう。$f(A,B)_{C..D}$ を求めます。

まず、$f(A,B)$ の最長の同じ文字からなる部分文字列の長さ、K を決定しましょう。$A < B$ とします。文字 A が A 個あり、B 個の文字 B を間に挟む必要があります。文字 B を入れる場所は $A+1$ 箇所存在するため、鳩の巣原理より、これらの場所のうち一箇所は文字 B を $\lceil \frac{B}{A+1} \rceil$ 個以上含みます。この数は、文字 B を文字 A の間に均等に入れることで、容易に達成することができます。$A \geq B$ の場合も同様です。結局、

$$K = \lceil \frac{\max(A,B)}{\min(A,B)+1} \rceil.$$

となります。

文字列を左から右に一文字ずつ構成していきましょう。辞書順最小の文字列を求めたいため、明らかに、以下のような状況では次の場所に A を置くべきです。

- 置くことが可能である（置いても A が $K+1$ 連続で現れない）。
- 置いても、最長の同じ文字からなる部分文字列の長さが K 以下となるように文字列を完成させることが可能であることがわかっている。

これらの条件を用いることで、部分点向けの解法を容易に実装することができます。

分析を進めましょう。しばらくは、第二の条件は無関係です。もし第二の条件が存在しなければ、文字列の先頭部分は AA...ABAA...ABAA...AB...（A は各グループで K 個ずつ）のようになります。

では、現在の場所に A を置くと第二の条件に反するような状況に初めて遭遇した時を考えます。A をあと A 文字、B をあと B 文字使わなければならないとします。もし A を置くと、あと $A-1$ 文字残ることになります。文字列を適切に完成させることができるための条件は何でしょうか？ある程度考えると、$B \leq A \cdot K$ がこのための必要十分条件であることがわかります（B のグループを A 組作ることができ、各グループの大きさはたかだか K です）。

この条件が成立しなくなると、$B > A \cdot K$ となります。この条件が再び成り立つまで、B のみを置いていくことになります。

したがって、B を $B - A \cdot K$ 個置くことになります。このあと、$B = A \cdot K$ となります。すると、文字列の残りの部分は ...ABB...BABB...BABB...B のようになります（B は各グループで K 個ずつ）。これは、A を置くたびに直ちに B を K 個置いて第二の条件を再び成り立たせる必要があるためです。

これらの分析から、$f(A,B)$ は次の二つのパーツを連結したような形式であることがわかります。

- AA...ABAA...ABAA...AB...（A は各グループで K 個ずつ）の接頭辞
- ...ABB...BABB...BABB...B（B は各グループで K 個ずつ）の接尾辞

最後に残る問題は、この二つをどこで連結するべきかという点です。

これは、二分探索で容易に求められます。$f(A,B)$. の第一のパーツに含まれる A の個数、N_A を求めましょう。二分探索では、N_A の値を仮定し、まず第一のパーツに含まれる B の個数、N_B を $N_B = \max(0, \lfloor \frac{N_A-1}{K} \rfloor)$ から求めます。すると、第二パーツには $A - N_A$ 個の A と $B - N_B$ 個の B が残されることになります。

もし $B - N_B \leq (A - N_A + 1) \cdot K$ であれば、第二のパーツを作ることは可能で、実際の N_A の値は現在の値より小さくはありません。そうでない場合、第二のパーツを作ることは不可能で、実際の N_A の値は現在

の値より小さいです。

N_A の値が判明すれば、$f(A,B)_{C..D}$ のそれぞれの文字は $O(1)$ で容易に求められます。この解法の計算量はクエリごとに $O(\log(A+B)+(D-C+1))$ となります。

E: Encoding Subsets

S のすべてのサブセットについての合計を計算する代わりに、デコードすると S のサブセットになるようなエンコード済み文字列の個数を一斉に数えましょう。文字列 S に対するこの問題の答えを $f(S)$ とします。

これを求めるには、エンコード済み文字列の最初の文字に注目します。次の二つの可能性があります。

- 最初の文字は桁 `0` または `1` である。この場合、この文字は文字列の残りの部分のエンコーディングとは無関係で、残りの部分をエンコードする方法は $f(S_{2..|S|})$ 通り存在します。$S_1 = \texttt{1}$ の場合はこの値を 2 倍するべき（エンコード済み文字列の最初の文字は `0` でも `1` でもよい）で、$S_1 = \texttt{0}$ の場合はこの値を 1 倍するべき（エンコード済み文字列の最初の文字は `0` でなければならない）です。
- 最初の文字は開きカッコ (である。この場合、このエンコード済み文字列の先頭部分は（$P\mathtt{x}K$）（P はある文字列のエンコード結果）となります。$K \cdot |A| \leq |S|$ のもとで、正の整数 K と $|A|$ の組み合わせをすべて試しましょう。$AA\ldots A$（A を K 個連結したもの）が $S_{1..K|A|}$ のサブセットでなければならず、これは、A が $S_{1..|A|}$ のサブセットであり、同時に $S_{|A|+1..2|A|}$ のサブセットであり、…、同時に $S_{(K-1)|A|+1..K|A|}$ のサブセットであることと同値です。さらに言えば、これは A がこれらすべての文字列の logical `AND` ($\wedge$) であることと同値です。そして、この文字列の残りの部分をエンコードする方法が $f(S_{K|A|+1..|S|})$ 通り存在します。

$f(S)$ の漸化式の全体は次のようになります。

$$f(S) = (1 + S_1)f(S_{2..|S|}) + \sum_{|A|=1}^{|S|} \sum_{K=1}^{\lfloor \frac{|S|}{|A|} \rfloor} f(w(S, K, |A|)) \cdot f(S_{K|A|+1..|S|}),$$

ここで、$w(S, K, |A|) = S_{1..|A|} \wedge S_{|A|+1..2|A|} \wedge \ldots \wedge S_{(K-1)|A|+1..K|A|}$ です。

メモ化することにより、$f(S)$ を直接計算しましょう。f は何通りの引数で呼ばれるでしょうか？明らかに、上界は $O(2^{|S|})$ です。しかし、この解法は十分高速であり、正解できることがわかります。

これを確かめる方法は少なくとも二つあります。

- プログラマー流。$f(S)$ を呼ぶ代わりに、V を長さ $N = |S|$ のビットマスクのベクトルとして $f(V)$ を呼ぶことにしましょう。このベクトルは f の引数になりうる文字列に対応します。V_i は、文字 i が元の文字列で対応する文字たちの logical `AND` であることを表します。このとき、$N = 100$ に対して $f(\{\{1\}, \{2\}, \ldots, \{N\}\})$ を呼んでみましょう。1 から N までのすべての長さのベクトル V や f の引数の個数を数えます。すると、このようなベクトルで長さが 12 を超えるものは 41703 個であることがわかります。また、このようなベクトルで長さが 12 以下のものは 8190 個であることもわかります。したがって、状態数は合計でたかだか 49893 通りです。公式テストデータには、45000 通り以上の状態数のデータが含まれていました。

- 数学者流。f が T を呼ぶとすると、T は入力される文字列に以下の二つの操作を繰り返して得ることができます。
  - 部分文字列をとる。
  - "K-fold" する $(K \geq 2)$。 $S_{1..K|A|}$ を $S_{1..|A|}$, $S_{|A|+1..2|A|}$, ..., $S_{(K-1)|A|+1..K|A|}$ の AND に変換する。

  Fold の操作をすると必ず文字列の長さが半分以下になるので、$|T| > N/8$ のときは Fold を高々二回しかしていないことが分かり、また、二回する方法は以下の三通りのみです。
  - 部分文字列をとる, 2-fold, 部分文字列をとる, 2-fold, 部分文字列をとる
  - 部分文字列をとる, 2-fold, 部分文字列をとる, 3-fold, 部分文字列をとる
  - 部分文字列をとる, 3-fold, 部分文字列をとる, 2-fold, 部分文字列をとる

  例えば最初の場合では、得られる文字列はパラメータ i, j, k を用いて四つの文字列 $S_{i..i+k-1}$, $S_{i+k..i+2k-1}$, $S_{j..j+k-1}$, $S_{j+k..j+2k-1}$ の AND として表せることが分かります。同様に、すべてのケースで、得られる文字列の個数は $O(N^3)$ であることが分かります。

F: Arcs on a Circle

一般性を失うことなく、最長の円弧の長さを L_N とします。この円弧の位置を適当に決めてしまいましょう(対称性より、問題ありません)。円周上に座標系を導入し、上記の N 番目の円弧の両端の座標が $0, L_N$ となるようにし、円周上のその他の点は区間 $[0, C)$ 内の座標値を持つようにします。

円弧 i の左端の座標を X_i とすると、右端の座標は $(X_i + L_i) \bmod C$ となります。問題文より、X_i の値は $[0; C)$ から一様ランダムに選ばれます。

$P_i = \lfloor X_i \rfloor, F_i = X_i - P_i$ を用いて、$X_i = P_i + F_i$ と表します。すなわち、P_i, F_i はそれぞれ X_i の整数部分と小数部分です。X_i を $[0; C)$ から選ぶ代わりに、整数 P_i を $[0, C-1]$ から選び、実数 F_i を $[0, 1)$ から選ぶことにしても同等です。また、$X_N = P_N = F_N = 0$ となります。

$i \neq j$ に対し $F_i = F_j$ となる確率は 0 であるため、F_i はすべて異なるとみなせます。

F_i を昇順に並べた際の順序を表す順列 $A_1, A_2, \ldots, A_{N-1}$ を固定しましょう。すなわち、$F_{A_1} < F_{A_2} < \ldots < F_{A_{N-1}}$ とします(F_N はすでに 0 に固定されています)。このような順列は $(N-1)!$ 通り存在し、対称性よりこれらの事象の発生確率はすべて等しいです。

これで、$O((N-1)! \cdot C^{N-1} \cdot N \log N)$ 時間の解法が得られます。F_i を昇順に並べた際の順序($(N-1)!$ 通り)と P_i の値(C^{N-1} 通り)を固定し、円弧が演習全体を覆うか確認する($O(N \log N)$)というものです。F_i の値を知る必要はなく、これらの大小関係のみが必要である点に注意してください。

この解法は遅すぎますが、正解への大きなヒントとなります。F_i の大小関係を固定したあと、P_i のすべての組み合わせを探索するのではなく、動的計画法を用いましょう。

円弧の端になりうるすべての座標を昇順に列挙してみましょう。

$$\begin{gathered} 0;\ 0 + F_{A_1};\ 0 + F_{A_2};\ \ldots;\ 0 + F_{A_{N-1}}; \\ 1;\ 1 + F_{A_1};\ 1 + F_{A_2};\ \ldots;\ 1 + F_{A_{N-1}}; \\ \ldots \\ C-1;\ C-1 + F_{A_1};\ C-1 + F_{A_2};\ \ldots;\ C-1 + F_{A_{N-1}}. \end{gathered}$$

このような座標は CN 個存在します。これらを $x_0, x_1, \ldots, x_{CN-1}$ と書き換えましょう。

$f(i, s, j)$ を、集合 s に含まれる円弧の左端として x_i 未満の座標値を割り当て、x_j 以下のすべての点が一本以上の円弧に覆われるようにする方法の個数とします。

起点となるケースは $f(0, \{N\}, L_N \cdot N) = 1$ で、求めたい値は $f(CN, \{1, 2, \ldots, N\}, CN)$ です。

$f(i, s, j)$ からの一つの遷移は、x_i を左端とする円弧がない場合で、$f(i+1, s, j)$ へと遷移します。もう一つの遷移については、x_i が左端となりうる円弧は $A_{i \bmod n}$ のみであり、さらに $i \bmod n \neq 0, A_{i \bmod n} \notin s$ である場合に限られることに注意します。すると、$f(i, s, j)$ からのもう一つの遷移は $f(i+1, s \cup \{A_{i \bmod n}\}, \min(CN, \max(j, i + L_{A_{i \bmod n}} \cdot N)))$ への遷移であることがわかります。

この解法の全体の計算量は $O((N-1)! \cdot (CN)^2 \cdot 2^{N-1})$ となります。

AtCoder Grand Contest 020 Editorial

writer: tourist

January 14, 2018

A: Move and Win

First, note that the only way to lose is when the token to be moved has the other token on one side and the outside of the strip on the other side.

Let the distance between tokens be $D = B - A$. Note that every turn D is changed by $+1$ or -1. Thus, after every turn the parity of D changes.

Suppose that initially, before Alice moves, D is even. Then D will always be even when Alice moves, and D will always be odd when Borys moves.

Note that whenever the tokens of Alice and Borys are in neighboring cells, $D = 1$. Thus, if D is even, the tokens are not in neighboring cells, and Alice can always make a move.

On the other hand, since Alice can always make a move, she can eventually move her token to cell $N - 1$. In this case, Borys's token will always be blocked in cell N, and Alice will win.

Similarly, if initially D is odd, Borys wins by moving his token to cell 2.

Thus, the answer depends on the parity of $B - A$: if it's even, Alice wins, otherwise Borys wins.

B: Ice Rink Game

Suppose there were X children left in the game before round i. How many children will be left after round i? Clearly, the number of groups they will form is $\lfloor \frac{X}{A_i} \rfloor$, therefore, $\lfloor \frac{X}{A_i} \rfloor \cdot A_i = X - X \bmod A_i$ children will be left. Let's denote $f_p(X) = X - X \bmod p$ from now on.

The problem asks us to find all N such that

$$g(N) = f_{A_K}(f_{A_{K-1}}(\dots f_{A_1}(N)\dots)) = 2.$$

There are two main directions which lead to a solution of the problem.

Linear scan

Let L_i and R_i be the smallest and the largest number of children that could be in the game after round i which lead to exactly 2 children in the end. It's known that $L_K = R_K = 2$, and we need to find L_0 and R_0.

Can we recover L_{i-1} and R_{i-1} from L_i and R_i? Yes, we can!

First, the number of children after round i must be divisible by A_i. If no integer between L_i and R_i is divisible by A_i, the answer is -1.

Otherwise, let X_i and Y_i be the smallest and the largest integers in $[L_i; R_i]$ divisible by A_i. Then,

$$\begin{aligned} L_{i-1} &= X_i, \\ R_{i-1} &= Y_i + A_i - 1. \end{aligned}$$

But $X_i = \lceil \frac{L_i}{A_i} \rceil \cdot A_i$ and $Y_i = \lfloor \frac{R_i}{A_i} \rfloor \cdot A_i$. Thus, overall,

$$\begin{aligned} L_{i-1} &= \lceil \tfrac{L_i}{A_i} \rceil \cdot A_i, \\ R_{i-1} &= \lfloor \tfrac{R_i}{A_i} \rfloor \cdot A_i + A_i - 1. \end{aligned}$$

The complexity of this solution is $O(K)$.

Binary search

A useful property of function f is that it's monotonically non-decreasing. Formally, if $X \le Y$, then $f_p(X) \le f_p(Y)$.

Note that a composition of monotonically non-decreasing functions is also monotonically non-decreasing. In our case, if $X \le Y$, then $g(X) \le g(Y)$.

We can use binary search to find the smallest N such that $g(N) \ge 2$, and another binary search to find the largest N such that $g(N) \le 2$. Clearly, these two values are the answer to the problem (or -1 if the first value exceeds the second one).

It's easy to see that the answer doesn't exceed $2 + K \cdot \max(A_i)$ — in every round, at most $\max(A_i)$ children leave the game. Thus, the complexity of this solution is $O(K \log(K \cdot \max(A_i)))$.

C: Median Sum

Let $S_0 = 0$, which corresponds to an empty subsequence of A. Observe that S_{2^N-1} is equal to the sum of all elements of A.

Divide all subsequences of A in pairs such that every element of A belongs to exactly one subsequence in every pair (thus, every subsequence is paired with its complementary subsequence). Obviously, there are 2^{N-1} pairs.

Consider an arbitrary pair of subsequences (P_i, Q_i), and let Σ_{P_i} and Σ_{Q_i} be the sum of elements in P_i and Q_i, respectively. Without loss of generality, let $\Sigma_{P_i} \leq \Sigma_{Q_i}$. By the way the pairs are formed, $\Sigma_{P_i} + \Sigma_{Q_i} = S_{2^N-1}$. Thus, $\Sigma_{P_i} \leq \frac{1}{2} S_{2^N-1}$ and $\Sigma_{Q_i} \geq \frac{1}{2} S_{2^N-1}$.

We see that every P_i is separated from every Q_j by the value of $\frac{1}{2} S_{2^N-1}$. Therefore, we can safely assume that Σ_{P_i} belongs to the first half of S, which is $S_0, S_1, \ldots, S_{2^{N-1}-1}$, while Σ_{Q_i} belongs to the second half of S, which is $S_{2^{N-1}}, \ldots, S_{2^N-1}$.

Thus, to find the value of $S_{2^{N-1}}$, we need to find the subsequence of A with the smallest sum greater than or equal to $\frac{1}{2} S_{2^N-1}$.

This can be done using dynamic programming: $f(i, j)$ denotes if there exists a subsequence of the first i elements of A with sum j. Since the values of $f(i, j)$ are boolean and the transitions can be performed using bitwise OR, this DP can be optimized using bitsets (for example, `std::bitset` in C++ or `java.util.BitSet` in Java, but even hand-written bitsets are good enough and easy to implement).

The complexity of this solution is $O(\frac{N^2 \max(A_i)}{64})$.

D: Min Max Repetition

Consider queries one by one — we need to find $f(A, B)_{C..D}$.

First, let's determine K — the length of the longest substring of $f(A, B)$ consisting of equal letters. Let $A < B$. There are A letters `A`, and B letters `B` have to placed in between. Since there are $A + 1$ places to put letters `B`, by pigeonhole principle, one of these places will have at least $\lceil \frac{B}{A+1} \rceil$ letters `B`. But this number is also easy to achieve if letters `B` are placed uniformly between `A`'s. The situation with $A \geq B$ is similar. Overall,

$$K = \lceil \frac{\max(A,B)}{\min(A,B)+1} \rceil.$$

Let's build the string letter-by-letter from left to right. Clearly, since we want to get the lexicographically smallest string, we should place `A` on the next place whenever:

- it's possible (we won't form $K + 1$ letters `A` in a row);
- we know we will be able to finish the string so that the longest substring of equal letters will still have length at most K.

It's easy to use these conditions for implementing a solution for partial points.

Let's analyze further. Until some moment, the second condition will not bother us. If there were no second condition, the string would start like `AA...ABAA...ABAA...AB...`, where `A` is repeated K times in each group.

Then, for the first time we encounter a situation that the second condition is false if we place `A` on the current position. Suppose we had A remaining letters `A` and B remaining letters `B`. If we place letter `A`, we'll have $A - 1$ letters `A` remaining. What is the condition that we can finish the string appropriately? It's not hard to see that we can do that if and only if $B \leq A \cdot K$ (we'll be able to form A groups of letters `B`, and each group will have size at most K).

Once this condition is false, we have $B > A \cdot K$. We will have to place only letters `B` until this condition is true again.

Thus, we will have to place $B - A \cdot K$ letters `B`. After this, we'll have $B = A \cdot K$. Now, the rest of the string will look like `...ABB...BABB...BABB...B`, where `B` is repeated K times in each group. That's true because now every time we place letter `A`, we'll have to place K letters `B` immediately to make the second condition true again.

Overall, this analysis tells us the form of $f(A, B)$ — it will consist of two parts merged together:

- a prefix of `AA...ABAA...ABAA...AB...`, where `A` is repeated K times in each group;
- a suffix of `...ABB...BABB...BABB...B`, where `B` is repeated K times in each group.

The only question left is: where is the merging place?

The merging place is easy to find using a single binary search. Indeed, let's find N_A — the number of letters `A` belonging to the first part of $f(A, B)$. In binary search, for some supposed value of N_A, first we find $N_B = \max(0, \lfloor \frac{N_A - 1}{K} \rfloor)$, which is the number of letters `B` in the first part of $f(A, B)$. Then, we'll have $A - N_A$ letters `A` and $B - N_B$ letters `B` left for the second part.

If $B - N_B \leq (A - N_A + 1) \cdot K$, the second part is possible to obtain, and the actual value of N_A is not lower than the current one. Otherwise, the second part is impossible to obtain, and the actual value of N_A is strictly lower than the current one.

After we know N_A, it's easy to find every character of $f(A, B)_{C..D}$ in $O(1)$. The complexity of this solution is $O(\log(A + B) + (D - C + 1))$ per query.

E: Encoding Subsets

Instead of doing summation over all subsets of S, let's count encodings which are decoded into a subset of S all together. Let $f(S)$ denote the answer to the problem for string S.

How to calculate it? Consider the first character of the encoding. There are two options:

- It is a digit, `0` or `1`. Then this character is independent from encoding of the rest of the string, and there are $f(S_{2..|S|})$ ways to encode it. This should be multiplied by 2 if $S_1 =$ `1` (the first character of the encoding may be `0` or `1`) or multiplied by 1 if $S_1 =$ `0` (the first character of the encoding has to be `0`).
- It is an opening bracket, `(`. Then this encoding starts with `(`P`x`K`)`, where P is an encoding of some string A. Let's loop over positive integers K and $|A|$ subject to $K \cdot |A| \le |S|$. We need $AA \dots A$ (A repeated K times) to be a subset of $S_{1..K|A|}$. This is equivalent to A being a subset of $S_{1..|A|}$, a subset of $S_{|A|+1..2|A|}$, ..., and a subset of $S_{(K-1)|A|+1..K|A|}$ at the same time. This, in turn, is equivalent to A being a subset of logical `AND` ($\wedge$) of all these strings. And there are $f(S_{K|A|+1..|S|})$ ways to encode the rest of the string.

The overall formula for $f(S)$ is:

$$f(S) = (1 + S_1) f(S_{2..|S|}) + \sum_{|A|=1}^{|S|} \sum_{K=1}^{\lfloor \frac{|S|}{|A|} \rfloor} f(w(S, K, |A|)) \cdot f(S_{K|A|+1..|S|}),$$

where $w(S, K, |A|) = S_{1..|A|} \wedge S_{|A|+1..2|A|} \wedge \ldots \wedge S_{(K-1)|A|+1..K|A|}$.

Let's calculate $f(S)$ directly with memoization. How many different arguments will f be called with? Obviously, the upper bound is $O(2^{|S|})$. But it turns out that the solution is fast enough to get accepted.

There are at least three ways to see this:

- Programmer's way. Instead of calling $f(S)$, let's call $f(V)$ where V is a vector of bitmasks of length $N = |S|$. This vector corresponds to a string which is a potential argument of f. V_i means that character i is logical `AND` of the corresponding characters in the original string. Now, call $f(\{\{1\}, \{2\}, \ldots, \{N\}\})$ for $N = 100$. Count the number of vectors V, arguments of f, of every length from 1 to N. It turns out that the number of these vectors of length more than 12 is 41703. At the same time, the number of strings of length up to 12 is 8190. Thus, the total number of states is at most 49893. The official test data had cases with more than 45 thousand states.
- Mathematician's (handwaving?) way. Again, there are only $2^{13} - 2$ strings of length at most 12. It can be shown that for each length more than 12, there are only $O(N^2)$ ways to get a string of this length using the

operations in f. In fact, the number of states can be bounded by $O(N^3 + 2^{N/8})$, with a fairly small constant in front of N^3.

- Believer's way. Run your solution on cases of maximum length with different number of ones and see that it is very fast.

Appendix: Detail of Mathematician's way.

If we get a string T as an argument of f, T must be obtained from the input string by repeating the following two types of operations:

- Take a substring.
- "K-fold" a string ($K \geq 2$). That is, from $S_{1..K|A|}$, get logical `AND` ($\wedge$) of the strings $S_{1..|A|}$, $S_{|A|+1..2|A|}$, ..., $S_{(K-1)|A|+1..K|A|}$.

If $|T| > N/8$, we can perform "fold" operations at most twice (because each fold operation will halve the length). There are three ways to perform two "fold" operations:

- Take a substring, 2-fold it, take a substring, 2-fold it, and take a substring.
- Take a substring, 2-fold it, take a substring, 3-fold it, and take a substring.
- Take a substring, 3-fold it, take a substring, 2-fold it, and take a substring.

For example, in the first case, the string we get will be the logical `AND` ($\wedge$) of four strings $S_{i..i+k-1}$, $S_{i+k..i+2k-1}$, $S_{j..j+k-1}$, $S_{j+k..j+2k-1}$ for some parameters i, j, k. Thus, there are at most $O(N^3)$ such strings. The second and the third cases are similar.

F: Arcs on a Circle

Without loss of generality, suppose the longest arc has length L_N. Let's fix the position of this arc on the circle arbitrarily (it doesn't matter due to symmetry). Introduce a coordinate system on the circle in such a way that the ends of the N-th arc have coordinates 0 and L_N, while all other points on the circle have coordinates from $[0; C)$.

Let X_i be the coordinate of the left end of arc i. Then, its right end has coordinate $(X_i + L_i) \bmod C$. By the problem statement, X_i is chosen uniformly from $[0; C)$ at random.

Let's represent $X_i = P_i + F_i$, where $P_i = \lfloor X_i \rfloor$ and $F_i = X_i - P_i$. That is, P_i and F_i are the integer and the fractional parts of X_i, respectively. Instead of choosing X_i from $[0; C)$, let's choose integer P_i from $[0; C-1]$ and real F_i from $[0; 1)$ independently, which is equivalent. Also, $X_N = P_N = F_N = 0$.

We can assume that all F_i are distinct, since the probability that $F_i = F_j$ for $i \neq j$ is 0.

Let's fix the permutation $A_1, A_2, \ldots, A_{N-1}$ which determines the order of F_i: $F_{A_1} < F_{A_2} < \ldots < F_{A_{N-1}}$ (F_N is already fixed at 0). There are $(N-1)!$ permutations, and all such orders are equally likely due to symmetry.

Now we can build a solution which works in $O((N-1)! \cdot C^{N-1} \cdot N \log N)$. Fix the order of F_i ($(N-1)!$ ways), the values of P_i (C^{N-1} ways), and check if the arcs cover the whole circle ($O(N \log N)$). Note that we don't need to know the values of F_i, we only need to know how they compare to each other.

This is too slow, but gives a big hint to the full solution. After fixing the order of F_i, instead of brute forcing all combinations of P_i, let's use dynamic programming.

Consider all coordinates which can potentially turn out to be the ends of arcs in increasing order:

$$\begin{gathered} 0;\ 0 + F_{A_1};\ 0 + F_{A_2};\ \ldots;\ 0 + F_{A_{N-1}}; \\ 1;\ 1 + F_{A_1};\ 1 + F_{A_2};\ \ldots;\ 1 + F_{A_{N-1}}; \\ \ldots \\ C-1;\ C-1 + F_{A_1};\ C-1 + F_{A_2};\ \ldots;\ C-1 + F_{A_{N-1}}. \end{gathered}$$

There are CN such coordinates. Let's encode them by $x_0, x_1, \ldots, x_{CN-1}$.

Let $f(i, s, j)$ denote the number of ways to assign left ends' coordinates less than x_i to arcs in set s so that all points with coordinates up to x_j are covered by at least one arc.

The base case is $f(0, \{N\}, L_N \cdot N) = 1$, and the value we want to know is $f(CN, \{1, 2, \ldots, N\}, CN)$.

One transition from $f(i, s, j)$ is to $f(i+1, s, j)$ if no arc's left end is at x_i. Otherwise, note that the only arc which can start at x_i is $A_{i \bmod n}$, and only if $i \bmod n \neq 0$ and $A_{i \bmod n} \notin s$. Then, the other transition from $f(i, s, j)$ is to $f(i+1, s \cup \{A_{i \bmod n}\}, \min(CN, \max(j, i + L_{A_{i \bmod n}} \cdot N)))$.

The overall complexity of this solution is $O((N-1)! \cdot (CN)^2 \cdot 2^{N-1})$.

AtCoder Grand Contest 019 解説

writer: tourist

2017 年 8 月 27 日

A: Ice Cream Store

必要なリットル数、N が整数であることに注意します。

もし 0.25 リットルサイズを奇数個買ってしまうと、買う量の合計を 0.5 で割った余りが 0.25 になってしまい、合計が整数リットルになりません。よって、0.25 リットルサイズは必ず偶数個買うべきです。0.25 リットルサイズを二つずつ組にすることで、0.25 リットルサイズ二つを 0.5 リットルサイズ一つとして扱えます。したがって、$H := min(H, 2 \times Q)$ とすることで、0.25 リットルサイズの存在を忘れてもよくなります。

同様の議論により、0.5 リットルサイズも必ず偶数個買うべきです。同じように、 $S := min(S, 2 \times H)$ とすることで、0.5 リットルサイズの存在を忘れてもよくなります。

これで、1 リットルサイズを一つ H 円、2 リットルサイズを一つ D 円で買えるという状況になりました。以下の二つの場合を考えます。

- $2 \times S \leq D$ のとき、2 リットルサイズを買うのは決して最適ではありません。その代わりに 1 リットルサイズ二つを常により安く買えるからです。したがって、この場合の答えは $N \times S$ です。
- $2 \times S > D$ のとき、2 リットルサイズをできるだけ多く買うべきです。2 リットルサイズを $(N/2)$ 個買うべきことが容易にわかり、$N \mod 2 = 1$ の場合はさらに 1 リットルサイズを一つ買います。この場合の答えは $(N/2) \times D + (N \mod 2) \times S$ です。

まとめると、答えは以下のようになります。

$$(N/2) \times min(8 * Q, 4 * H, 2 * S, D) + (N \mod 2) \times min(4 * Q, 2 * H, S).$$

B: Reverse and Compare

$A_i = A_j$ であるような i, j を選んだとします。すると、部分文字列 $i+1..j-1$ を反転した結果は、部分文字列 $i..j$ を反転した結果と同じです。これにより、$A_i = A_j$ であるような組 (i, j) は答えに影響しないため無視することができます。

後ほど、$A_i \neq A_j$ であるような組 $i < j$ のそれぞれから、A のうち部分文字列 $i..j$ を反転することで異なる文字列が得られることを証明します。

答えは ($i < j$ かつ $A_i \neq A_j$ であるような組 (i, j) の個数) $+1$ です。この組の個数を数えるには、すべての組の個数 $n \cdot (n-1)/2$ から $i < j$ かつ $A_i = A_j$ であるような組の個数を引けばよいです。同じ文字の組の個数は、“a” から “z” までのそれぞれの文字 c について $z(c) \cdot (z(c)-1)/2$ を足し合わせた和になります。ここで、$z(c)$ は A に c が出現する回数です。

この解法の計算量は $O(n)$ となります。

上記の証明:

$B(i, j)$ を A のうち部分文字列 $i..j$ を反転して得られる文字列とします。ここで $A_i \neq A_j$ と仮定します。すると、$B(i, j)$ と A で文字が異なる最も左の位置は i です。同様に、$B(i, j)$ と A で文字が異なる最も右の位置は j です。

今度は、$B(i, j) = B(x, y)$ と仮定します。$B(i, j)$ と $B(x, y)$ の A と文字が異なる最も左の位置は同じでなければならないため、$i = x$ を得ます。同様に、$B(i, j)$ と $B(x, y)$ の A と文字が異なる最も右の位置は同じでなければならないため、$j = y$ を得ます。よって、$B(i, j) = B(x, y)$ であれば $i = x$ かつ $j = y$ が成立します。この対偶を取ると、$i \neq x$ または $j \neq y$ であれば $B(i, j) \neq B(x, y)$ となります。ゆえに、$A_i \neq A_j$ であるような部分文字列 $i..j$ を反転して得られる文字列はすべて異なります。

C: Fountain Walk

(x_1, y_1) から (x_2, y_2) までの経路を考えます。s を経路が通過する区間の個数（噴水は無視する)、p を噴水がある交差点で 90 度曲がる回数、q を噴水がある交差点で 180 度曲がる回数とします。例えば、問題文の三つの例では $(s, p, q) = (9, 2, 0), (3, 0, 0), (2, 0, 1)$ です。この経路の長さは以下のようになります。

$$100s - (20 - 5\pi)p + (10\pi - 20)q \tag{1}$$

したがって、s, q は少ないほどよく、p は大きいほどよいです。s にかかる係数が圧倒的に大きいため、まずこれを最小化しなければなりません（厳密な証明は後ほど)。一般性を失うことなく、$x_1 \leq x_2, y_1 \leq y_2$ と仮定します。すると、$s = (x_2 - x_1) + (y_2 - y_1)$ となります。

s を最小化しつつ、通過することのできる噴水は最大で何個でしょうか？この個数を k とし、通過する噴水の位置を順に $(a_1, b_1), \ldots, (a_k, b_k)$ とします。この噴水の列は、次の条件が満たされるときに、またそのときに限って経路の一部となりえます。

- $x_1 \leq a_1 < \cdots < a_k \leq x_2$
- $y_1 \leq b_1 < \cdots < b_k \leq y_2$

k はこの条件を満たす最も長い噴水の列の長さであり、LIS（Longest Increasing Sequence: 最長増加列）として計算できます。

さて、理想的には $p = k$ かつ $q = 0$ としたいところです。実は、$k = \min\{x_2 - x_1, y_2 - y_1\} + 1$ の場合以外ではこれが可能であることがわかります。(各通りが噴水を一つまでしか含まないため、$k \leq \min\{x_2 - x_1, y_2 - y_1\} + 1$ であることに注意します)

$k < \min\{x_2 - x_1, y_2 - y_1\} + 1$ の場合

$(a_1, b_1), \ldots, (a_k, b_k)$ を、長さ k の噴水の列としてありうるものの一つとします。ここで $a_1 > x_1$ と仮定できます（そうでないなら、$b_1 > y_1$ であり同様の主張ができます)。k の最大性により、噴水列のすべての噴水を通過するときに他の噴水は通過しないため、それらは無視できます。

まず、 $y_1 \leq y \leq y_2$ であって東西方向の通り y が噴水を含まないような y を選びます。(特殊な場合として、$a_k = x_2$ のときは $y = y_2$ を選ぶ必要があります。) そして、次のルールに従って噴水に出入りします。

- $b_i < y$ なら、 (a_i, b_i) に $(a_i - 1, b_i)$ から入って $(a_i, b_i + 1)$ へと出る。
- $b_i > y$ なら、(a_i, b_i) に $(a_i, b_i - 1)$ から入って $(a_i + 1, b_i)$ へと出る。

このようにすれば、$p = k$ かつ $q = 0$ であるような経路が得られます。

$k = \min\{x_2 - x_1, y_2 - y_1\} + 1$ の場合

$y_2 - y_1 \leq x_2 - x_1$ と仮定できます。すると、このとき y_1 から y_2 まで（両端含む）のそれぞれの南北方向の通りが噴水を一つ含みます。これらの噴水が長方形 $[x_1, x_2] \times [y_1, y_2]$ を二分割しており、一方の領域から他方へと移動するには一回 180 度曲がらなければなりません。

その一方で、$(p, q) = (k-1, 1)$ とすることは可能です。任意の $k-1$ 個の噴水を選び、前の場合と同じことをすればよいためです。

したがって、この場合の最適な経路においては $p = k-1$ かつ $q = 1$ です。

s を最小化すべきことの証明

まず、経路が長方形 $[x1, x2] \times [y1, y2]$ の外部に出ないことを証明します。

経路が長方形の下にはみ出したと仮定します。経路中の最小の y 座標を $y_0 < y_1$ とします。すると、経路はある時点で $y_0 + 1$ から y_0 へと進み、直線 $y = y_0$ に沿って直進し、$y_0 + 1$ に戻ったことになります。

この経路の $y = y_0$ の部分の y 座標を $y = y_0$ から $y_0 + 1$ に変えることで、別の経路を得ることができます。これにより、s は 2 減り、p は高々 1 減り、q は高々 1 増えるため、新しい経路の方が短くなります。よって、長方形の外部に出るべきではありません。

これより、経路は (x_1, y_1) から $(x_1 + 1, y_1)$ または $(x_1, y_1 + 1)$ へと進むことがわかります。経路の残りの部分は、その位置から (x_2, y_2) への最短経路でなければならないため、同様の「長方形」の議論により s を最小化すべきことが証明されます。(厳密には、その新たな交差点が噴水を含む場合を注意して扱わなければなりませんが、証明は同様であり、細部は省略します。)

D: Shift and Flip

B が 0 のみからなるとき、A のビットを反転することはできません。よって、A も 0 のみからなるときの答えは 0、それ以外の場合の答えは -1 です。以降、B に 1 が一つ以上含まれるとします。

次のような「カウンター」があると想定します。このカウンターの初期値は 0 であり、A を右にシフトするたびに値が 1 増え、左にシフトするたびに値が 1 減ります。ここで、l を操作列の実行中にカウンターが記録した最小の値とします。同様に、r を操作列の実行中にカウンターが記録した最大の値とします。

l, r, d の値が既知であるとします。次の問いを考えましょう:「A を B に変換する操作列は存在するか？存在するなら、そのような最短の操作列の長さは何か？」

A の個別のビット一つ (A_i とします) を考えます。最終的に、このビットは位置 $(i+d) \mod n$ に移動します。よって、$A_i = B_{(i+d) \mod n}$ であれば A_i を反転させる必要はなく、そうでなければどこかのタイミングで反転させる必要があります。操作列の実行中に、ビット A_i は $(i+l) \mod n$ から $(i+r) \mod n$ のすべての位置（両端含む）を訪れます。もし $B_j = 1$ であるような j が存在すれば、A_i が j を訪れたときに反転させることができます。そうでなければ A_i を反転させることはできず、条件を満たす操作列は存在しません。

もし l, r, d の値に対応する操作列が存在することが分かれば、そのような最短の操作列の長さは容易に計算できます。その長さは、反転の回数とシフトの回数の和です。必要な反転の回数は当然、$A_i \neq B_{(i+d) \mod n}$ であるような添字 i の個数です。また、必要なシフトの回数が $2 \cdot (r-l) - |d|$ であることは容易に示せます。例えば、$d \geq 0$ であれば、左に $-l$ 回シフトし、右に $r-l$ 回シフトして左に $r-d$ 回シフトするのが最適です。

ここで、最適解において $|l|, |r|, |d| \leq n$ であることに注意します。実際、例えば右に n 回シフトを行うと、A のすべてのビットがすべての場所を訪れたことになり、また B には少なくとも一つ 1 が含まれるため、A のすべてのビットを反転させる機会がすでにあったのであり、それ以上 A を右にシフトする必要はありません。

簡単な前計算を行うと、これで $O(n^3)$ の解法が得られます。

これを最適化するには、まず d の値を固定します。一般性を失うことなく $d \geq 0$ と仮定します。再び、A の個別のビット一つ (A_i とします) を考えます。もし $A_i = B_{(i+d) \mod n}$ であれば、問題ありません。そうでなければ、A_i を反転する必要があります。もし i と $(i+d) \mod n$ の間に $B_j = 1$ であるような j が存在すれば、A_i が位置 j に来たところで反転させることができるので、他に何もする必要はありません。そうでなければ、A_i が $B_j = 1$ であるような位置 j に来るように A をシフトする必要があります。ここで、x_i を B での位置 i より左側の最も近い 1 までの距離とし、y_i を B での位置 $(i+d) \mod n$ より右側の最も近い 1 までの距離とします。また、$X = -l, Y = r-d$ とします。すると、$X \geq x_i$ と $Y \geq y_i$ のうち少なくとも一方が成立する必要があります。

このようにして複数の組 (x_i, y_i) が得られ、次の部分問題を解くことになります。「条件 ($X \geq x_i$ または $Y \geq y_i$) のもとで、$X+Y$ を最小化する $X \geq 0, Y \geq 0$ を求めよ。」組を x_i の値でソートすると、この問題は単純な平面走査で解けます。これを d の $O(n)$ 個の値について行う必要があるため、全体で $O(n^2 \log n)$ の解法が得られます。ここで、x_i の値が n であることを利用して組のソートを $O(n)$ で行うと $O(n^2)$ の解法となります。

E: Shuffle and Swap

a と b を個別にシャッフルする代わりに、次の同等な手順を考えます。

- まず、a_i と b_i の対応を固定します。対応は $k!$ 通り存在します。
- そして、組 (a_i, b_i) の順序を決定します。順序は $k!$ 通り存在します。

第一ステップがすでに済んだものとして、$A = B$ となるような順序の個数を数えます。n 頂点からなる有向グラフを考えます。各 i に対し、a_i から b_i に辺を描きます。各頂点 i の出次数は A_i, 入次数は B_i です。$A_i = B_i = 0$ であるような i（孤立点）を無視すると、このグラフの各連結成分はパスまたはサイクルです。

サイクルに含まれる辺の順序は関係ありません。それらの辺は 1 と 1 を入れ替えるのに対応しているためです。

しかし、任意のパス $p_1 - > p_2 - > \ldots - > p_t$ については、適切な辺の順序はただ一つです。はじめ、$A_{p_1} = A_{p_2} = \ldots = A_{p_{t-1}} = 1, A_{p_t} = 0$ です。また、$B_{p_1} = 0, B_{p_2} = B_{p_3} = \ldots = B_{p_t} = 1$ です。この状態から、0 を位置 p_t から p_1 に移動させる必要があり、そのような方法はただ一つであることは容易にわかります: まず $A_{p_{t-1}}$ と A_{p_t} を入れ替え（0 を p_{t-1} に移動)、次に $A_{p_{t-2}}$ と $A_{p_{t-1}}$ を入れ替え（0 を p_{t-2} に移動)、…、最後に A_{p_1} と A_{p_2} を入れ替え（0 を p_1 に移動)。

$A_i = B_i = 1$ であるような添字 i の個数を m、$A_i = 1$, $B_i = 0$ であるような i の個数を e とします。A と B に含まれる 1 の個数は等しいため、$A_i = 0$, $B_i = 1$ であるような i の個数も e となります。明らかに、グラフにはちょうど e 本のパス（と何個かのサイクル）があります。グラフの辺の本数は $m + e$ であり、パスまたはサイクルのいずれかに含まれる頂点が m 個あります。

このとき、m 個の頂点を e 本のパスに配分する方法の数を数え、同時に問題文の条件を満たす辺の順序の数を数えるのは、比較的単純な組合せの練習問題のようなものです。定数倍を除くと、求めたいものは次の関数です。

$$f(0,0) = 1,$$
$$f(0,j) = 0 \text{ for } j > 0,$$
$$f(i,j) = \sum_{u=0}^{j} \frac{f(i-1,j-u)}{(u+1)!}.$$

ここで、$f(i,j)$ はいずれか j 個の頂点を最初の i 本のパスに配分するのに対応します。

すると、問題の答えは 0 から m までの j に対して $f(e,j)$ を足し合わせた和を $e! \cdot m! \cdot (e+m)!$ 倍したものであることが示せます。これにより、$O(n^3)$ の解法が得られます。

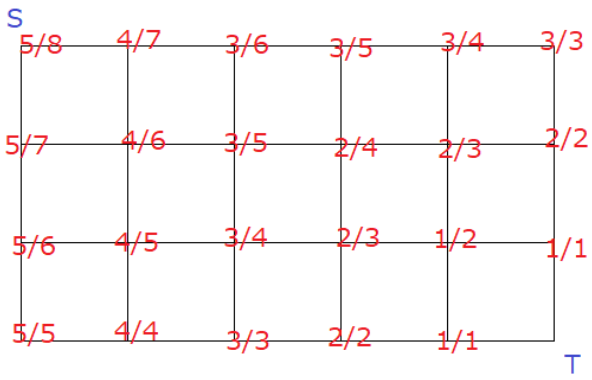
この解法を二段階で最適化することで満点解法が得られます。第一段階は、多項式 $p_i(x) = \sum_{j=0}^{m} f(i,j) \cdot x^j$ を考えることです。すると、$p_i(x)$ から $p_{i+1}(x)$ を高速フーリエ変換により $O(n \log n)$ 時間で計算することができます。これにより、$O(n^2 \log n)$ の解法が得られます。

第二段階は次のようになります。$q(x) = \sum_{j=0}^{m} \frac{x^j}{(j+1)!}$ とします。すると、$p_{i+1}(x) = p_i(x) \cdot q(x)$ （の最初の $m+1$ 項）となります。これと $p_0(x) = 1$ より、$p_e(x) = q(x)^e$ が成立します。よって、繰り返し二乗法により、$O(\log n)$ 回の多項式の掛け算で $p_e(x)$ を計算できます。これで、$O(n \log^2 n)$ の解法が得られました。

F: Yes or No

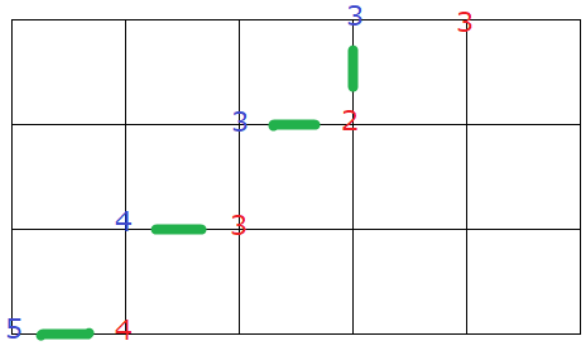
最適な戦略においては明らかに、答えがマルの問題が残り A 問、バツの問題が残り B 問であることが分かっているならば、$A \geq B$ であればマル、そうでなければバツと答えるべきです。したがって、このような状況では次の問題に確率 $\max(A, B)/(A + B)$ で正解できます。

マルとバツの列を $N \times M$ のマス目の左上隅から右下隅への経路として表しましょう。



求めるべきものは、上の図の S から T への無作為なパス上の数の和の期待値です。

それぞれの「斜線」を独立に扱うことにします。例えば、次の図は二本の斜線を示します。



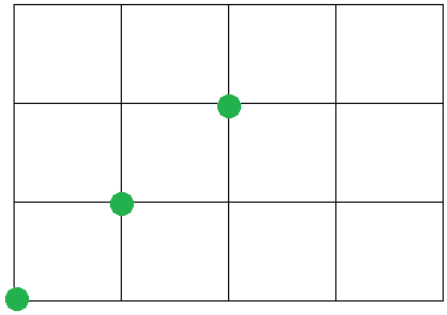
それぞれの経路はちょうど一つの「青い」数を含みます。すべての経路についてこの数の和を求め、5（共通の分母）と $(N + M)!/N!M!$（経路の数）で割って答えに加えます。これをすべての斜線について行います。

斜線を右下から左上の順に扱っていきます。すべての経路に対する「赤い」数の和をすでに計算したとしま

しょう。このとき、どのようにすれば青い数の和が求まるでしょうか？

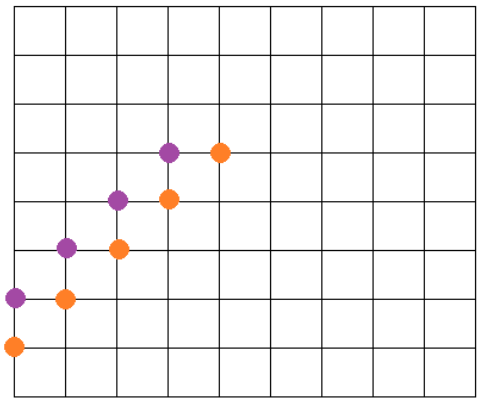
任意の経路において、(青い数) - (赤い数) は 0 または 1 であり、1 となるのは経路が緑の辺を通るときであることに着目します。すると、(青い数の和) = (赤い数の和) + (緑の辺を通る経路の数) であることがわかります。

横の緑の辺と縦の緑の辺を別に扱うことにします。横の緑の辺を通る経路の数は、次のマス目で緑の点を通る経路の数と等しいです（マス目が横方向に 1 縮んでいることに注意してください)。



これを高速に求めるのは困難です。しかし、一つ前の斜線における計算結果を再利用することができます。

分かりやすくするため、より大きな例を示します。



この図で、紫の点を通る経路の数を求めたいとします。ところが、一つ前の斜線における計算中に、橙の点を通る経路の数がすでに分かっています。この二つの数の差は単純な二項係数で表されるため、O(1) で計算できます。

AtCoder Grand Contest 019 Editorial

writer: tourist

August 26, 2017

A: Ice Cream Store

Note that N, the required number of liters, is integer.

If we buy an odd number of 0.25-liter bottles, we will never get an integer number of liters — remainder of division of our bought volume by 0.5 will be 0.25. Therefore, we must always buy an even number of 0.25-liter bottles. We can arrange 0.25-liter bottles in pairs and consider two 0.25-liter bottles as one 0.5-liter bottle. Thus, we can set $H := min(H, 2 \times Q)$ and forget about 0.25-liter bottles completely.

Now, by a similar argument, we must buy an even number of 0.5-liter bottles. Similarly, we can set $S := min(S, 2 \times H)$ and forget about 0.5-liter bottles as well.

Now we're in a situation where we can only buy 1-liter bottles for S yen each, and 2-liter bottles for D yen each. There are two cases:

- If $2 \times S \leq D$, it's never optimal to buy 2-liter bottles — we can always buy two 1-liter bottles at a better price instead. Therefore, the answer is $N \times S$.
- If $2 \times S > D$, we should buy as many 2-liter bottles as possible. It's easy to see we should buy (N div 2) 2-liter bottles, and if $N \mod 2 = 1$, we should buy an additional 1-liter bottle. The answer is $(N \text{ div } 2) \times D + (N \mod 2) \times S$.

Overall, the answer is:

$$(N \text{ div } 2) \times min(8 * Q, 4 * H, 2 * S, D) + (N \mod 2) \times min(4 * Q, 2 * H, S).$$

B: Reverse and Compare

Suppose we choose such i and j that $A_i = A_j$. Then, reversing substring $i+1..j-1$ results in the same string as reversing substring $i..j$. This means we can safely ignore pairs (i, j) with $A_i = A_j$, since they don't influence the result.

Later, we'll prove that for every pair $i < j$ such that $A_i \neq A_j$, we'll obtain a unique string by reversing substring $i..j$ in A.

The answer is 1 plus the number of pairs (i, j) such that $i < j$ and $A_i \neq A_j$. To count these pairs, we can subtract the number of pairs such that $i < j$ and $A_i = A_j$ from the total number of pairs, $n \cdot (n-1)/2$. The number of pairs of equal characters is the sum of $z(c) \cdot (z(c)-1)/2$ over all characters c from "a" to "z", where $z(c)$ is the number of occurrences of c in A.

The complexity of this solution is $O(n)$.

Proof:

Let $B(i, j)$ be the string obtained by reversing substring $i..j$ in A. Assume $A_i \neq A_j$. Then, the leftmost position where $B(i, j)$ and A differ is position i. Similarly, the rightmost position where $B(i, j)$ and A differ is position j.

Now assume that $B(i, j) = B(x, y)$. Since the leftmost position where $B(i, j)$ and $B(x, y)$ differ from A must be the same, we get that $i = x$. Similarly, the rightmost position where $B(i, j)$ and $B(x, y)$ differ from A must be the same, so we get $j = y$. Thus, if $B(i, j) = B(x, y)$, then $i = x$ and $j = y$. The negation of this statement is: if $i \neq x$ or $j \neq y$, then $B(i, j) \neq B(x, y)$. It follows that all strings obtained by reversing substring $i..j$ where $A_i \neq A_j$ are unique.

C: Fountain Walk

Consider a path from (x_1, y_1) to (x_2, y_2). Let s be the number of sections it passes through (when we ignore foutains), p be the number of 90-degree turnings at fountains, and q be the number of 180-degree turnings at fountains. For example, in the three samples in the statement, $(s, p, q) = (9, 2, 0), (3, 0, 0), (2, 0, 1)$. The length of this path is:

$$100s - (20 - 5\pi)p + (10\pi - 20)q \tag{1}$$

Thus, we want s and q to be small and p to be big. Since the coefficient of s is by far the largest, we must minimize it (we will give a formal proof later). Without loss of generality, we can assume that $x_1 \leq x_2, y_1 \leq y_2$. Then, $s = (x_2 - x_1) + (y_2 - y_1)$.

What is the largest possible number of fountains we can pass through while minimizing s? Let's call this value k, and suppose that the fountains are at $(a_1, b_1), \ldots, (a_k, b_k)$, in order. This sequence can be a part of the path if and only if the following conditions are satisfied:

- $x_1 \leq a_1 < \cdots < a_k \leq x_2$
- $y_1 \leq b_1 < \cdots < b_k \leq y_2$

k is the longest possible sequence of fountains satisfying these constraints, and it can be computed as a LIS (Longest Increasing Sequence).

Now, ideally, we want to make $p = k$ and $q = 0$. It turns out to be possible except for the special case $k = \min\{x_2 - x_1, y_2 - y_1\} + 1$. (Note that $k \leq \min\{x_2 - x_1, y_2 - y_1\} + 1$ because each street/avenue contains at most one fountain.)

The case with $k < \min\{x_2 - x_1, y_2 - y_1\} + 1$

Let $(a_1, b_1), \ldots, (a_k, b_k)$ be one possible sequence of fountains of length k. We can assume that $a_1 > x_1$ (otherwise, $b_1 > y_1$ and the same claim holds). Because of the maximality of k, when we passes through all of them, we don't pass through any other fountains, so we can ignore others.

First, we choose y such that $y_1 \leq y \leq y_2$ and the street y contains no fountains. (As a special case, when $a_k = x_2$, we must choose $y = y_2$.) Then, we enter/leave fountains according to the following rule:

- If $b_i < y$, we enter (a_i, b_i) from $(a_i - 1, b_i)$ and leaves to $(a_i, b_i + 1)$.
- If $b_i > y$, we enter (a_i, b_i) from $(a_i, b_i - 1)$ and leaves to $(a_i + 1, b_i)$.

This way, we can find a path with $p = k$ and $q = 0$.

The case with $k = \min\{x_2 - x_1, y_2 - y_1\} + 1$

We can assume that $y_2 - y_1 \leq x_2 - x_1$. Then, in this case, each street between y_1 and y_2, inclusive, contains one fountain. These fountains splits the rectangle $[x_1, x_2] \times [y_1, y_2]$ into two parts and it is necessary to use one 180-degree turning to go from one part to the other.

On the other hand, $(p, q) = (k - 1, 1)$ is possible: choose arbitrary $k - 1$ fountains and do the same as the previous case.

Thus, in this case, the optimal path has parameters $p = k - 1$ and $q = 1$.

Proof that s should be the smallest

First, we'll prove that the path never goes out of the bounding box $[x1, x2] \times [y1, y2]$.

Suppose that the path goes below the bounding box. Let $y_0 < y_1$ be the minimum y-coordinate of the path. Then, at some point, the path goes from $y_0 + 1$ to y_0, runs along the line $y = y_0$, and return to $y_0 + 1$.

We can change the path by changing y-coordinates of all parts of the path at $y = y_0$ to $y_0 + 1$. This way, s decreases by 2, p decreases by at most 1, and q increases by at most 1, thus we get a shorter path. Thus, we should never go out of the bounding box.

Now, we know that the path goes to $(x_1 + 1, y_1)$ or $(x_1, y_1 + 1)$ from (x_1, y_1). The remaining part of path should be the shortest path from that position to (x_2, y_2), so the same bounding-box argument will prove that s should be smallest possible. (Strictly speaking, we shuold carefully handle the case when the new intersection contains a fountain, but the proof is simlar and we omit details).

D: Shift and Flip

If B consists only of 0's, we can't flip any bits in A. Thus, if A also consists only of 0's, the answer is 0, otherwise it's -1. From now on, assume B contains at least one 1.

Consider a counter which is initialized with zero, increases by 1 whenever we shift A to the right, and decreases by 1 whenever we shift A to the left. Let l be the smallest value ever contained by the counter during executing some sequence of operations. Similarly, let r be the largest value ever contained by the counter, and let d be the resulting value of the counter.

Suppose we know l, r and d. Let's answer the following question: does there exist a sequence of operations which transforms A into B, and if yes, what is the length of the shortest such sequence?

Consider an individual bit in A, say A_i. In the end, it will move to position $(i+d) \mod n$. Then, if $A_i = B_{(i+d) \mod n}$, we don't need to flip A_i. Otherwise, we have to flip it at some moment. During the process, bit A_i visited all positions from $(i+l) \mod n$ to $(i+r) \mod n$, inclusive. If there exists j in this range such that $B_j = 1$, we can flip A_i at the moment it visits position j. Otherwise, we can't flip A_i, so there is no valid sequence of operations.

If we found out that a sequence of operations with corresponding l, r and d exists, we can easily calculate the length of the shortest such sequence. This length is equal to the number of flips plus the number of shifts. We know the required number of flips — it's the number of indices i such that $A_i \neq B_{(i+d) \mod n}$. It's easy to show that the required number of shifts is $2 \cdot (r-l) - |d|$. For example, if $d \geq 0$, it's optimal to make $-l$ shifts to the left, then $r-l$ shifts to the right, and then $r-d$ shifts to the left.

Note that in the optimal solution $|l|, |r|, |d| \leq n$. Indeed, once we make n shifts to the right (for example), it means that every bit in A has been to every single position, and since B contains at least one 1, we've already had a chance to flip every bit in A, so we don't need to shift A to the right anymore.

With some easy preprocessing, this gives an $O(n^3)$ solution.

To optimize it, let's fix the value of d first. Without loss of generality, assume $d \geq 0$. Again, consider an individual bit in A, say A_i. If $A_i = B_{(i+d) \mod n}$, we're fine. Otherwise, we need to flip A_i. If there exists j between i and $(i+d) \mod n$ such that $B_j = 1$, we can flip A_i at the moment it gets to position j, so we don't need to do anything else. Otherwise, we have to perform shifts on A in such a way that A_i gets into some position j with $B_j = 1$. Let x_i be the distance to the closest 1 in B to the left of position i, and y_i be the distance to the closest 1 in B to the right of position $(i+d) \mod n$. Also, put $X = -l$ and $Y = r - d$. Then, we need at least one of $X \geq x_i$ and $Y \geq y_i$ to be true.

Finally, we have several pairs (x_i, y_i), and we need to solve the following subproblem: find $X \geq 0$ and $Y \geq 0$ subject to ($X \geq x_i$ or $Y \geq y_i$) minimizing $X+Y$. After we sort all pairs by x_i, this subproblem can be solved with a simple linear scan. Since we have to do that for $O(n)$ values of d, overall we have an $O(n^2 \log n)$ solution due to sorting, or we can arrive at an $O(n^2)$ solution if we use the fact that x_i doesn't exceed n, so we can sort pairs in $O(n)$.

E: Shuffle and Swap

Instead of shuffling a and b independently, let's perform an equivalent procedure:

- First, fix the matching of a_i and b_i. There are $k!$ matchings.
- Then, fix the order of pairs (a_i, b_i). There are $k!$ orders.

Suppose we've done the first step and want to count the number of orders which result in $A = B$. Consider a directed graph with n vertices. Draw an edge from a_i to b_i for every i. Every vertex i has outgoing degree A_i and ingoing degree B_i. After ignoring i with $A_i = B_i = 0$ (which are isolated vertices), every connected component in this graph is either a path or a cycle.

The order of edges in cycles doesn't matter, since every cycle edge corresponds to swapping 1 with 1.

For any path $p_1 -> p_2 -> \ldots -> p_t$, though, there is only one good order of edges. Initially, $A_{p_1} = A_{p_2} = \ldots = A_{p_{t-1}} = 1$ and $A_{p_t} = 0$. Also, $B_{p_1} = 0$ and $B_{p_2} = B_{p_3} = \ldots = B_{p_t} = 1$. We need to move 0 from position p_t to position p_1, and it's easy to see that there is exactly one way to do that: swap $A_{p_{t-1}}$ with A_{p_t} (moving 0 to position p_{t-1}), then swap $A_{p_{t-2}}$ with $A_{p_{t-1}}$ (moving 0 to position p_{t-2}), ..., then swap A_{p_1} with A_{p_2} (moving 0 to position p_1).

Denote the number of indices i with $A_i = B_i = 1$ by m, and the number of indices i with $A_i = 1$, $B_i = 0$ by e. Since the number of 1's in A and B is equal, the number of indices i with $A_i = 0$, $B_i = 1$ is also e. Clearly, we have exactly e paths in our graph (along with some cycles). The number of edges in the graph is $m + e$, and there are m vertices which might belong either to a path or to a cycle.

Now it's a relatively simple combinatorial exercise to count the number of ways to distribute some of m vertices into e paths, while also counting the number of orders of edges which comply with the problem statement. It can be shown that, up to a constant, we're interested in the following function:

$$f(0,0) = 1,$$
$$f(0,j) = 0 \text{ for } j > 0,$$
$$f(i,j) = \sum_{u=0}^{j} \frac{f(i-1,j-u)}{(u+1)!}.$$

Here, $f(i,j)$ corresponds to distributing some j vertices into the first i paths.

Then, it can be shown the answer to the problem is the sum of $f(e,j)$ over j from 0 to m, multiplied by $e! \cdot m! \cdot (e+m)!$. This gives an $O(n^3)$ solution.

It can be optimized to full solution in two steps. The first step is to consider polynomials $p_i(x) = \sum_{j=0}^{m} f(i,j) \cdot x^j$. Then, we can calculate $p_{i+1}(x)$ from $p_i(x)$ in $O(n \log n)$ using Fast Fourier Transform. This gives an $O(n^2 \log n)$ solution.

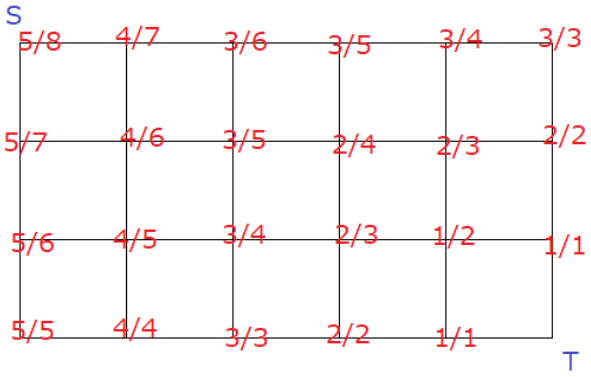
Here is the second optimization. Let $q(x) = \sum_{j=0}^{m} \frac{x^j}{(j+1)!}$. Then, $p_{i+1}(x) = p_i(x) \cdot q(x)$ (cut to the first $m+1$ coefficients). Since $p_0(x) = 1$, it follows that

$p_e(x) = q(x)^e$. We can calculate $p_e(x)$ in $O(\log n)$ polynomial multiplications using exponentiation by squaring. Finally, this gives an $O(n \log^2 n)$ solution.

F: Yes or No

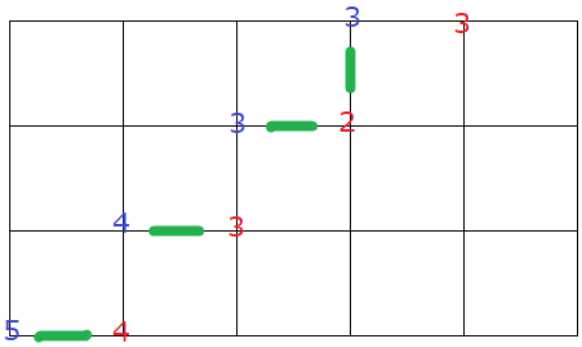
Obviously, in the optimal strategy, if you know that there are A remaining questions with "Yes" and B remaining questions with "No", you should answer "Yes" if $A \geq B$, and "No" otherwise. Thus, in such situation, you can correctly answer the next question with probability A/B.

Let's represent a sequence of "Yes" and "No" as a shortest path from the top-left corner to the bottom-right corner in an $N \times M$ grid.



In the picture above, what we want to compute is the expected sum of numbers on a random path from S to T.

We will handle each diagonal independently. For example, the following picture shows two diagonals:

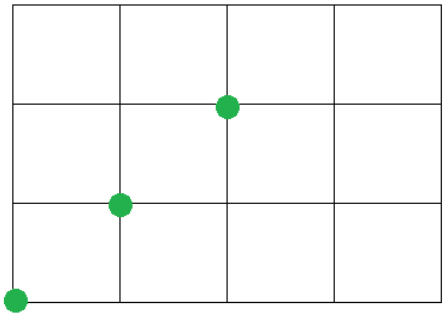


Each path contains exactly one blue number. We should compute the sum of this number over all paths, divide it by 5 (the common denominator) and $(N+M)!/N!M!$ (the number of paths), and add it to the answer. We should do this for all paths.

Let's handle diagonals in the order from bottom-right to top-left. Suppose that we've already computed the sum of red numbers over all paths. How should we compute the sum of blue numbers then?

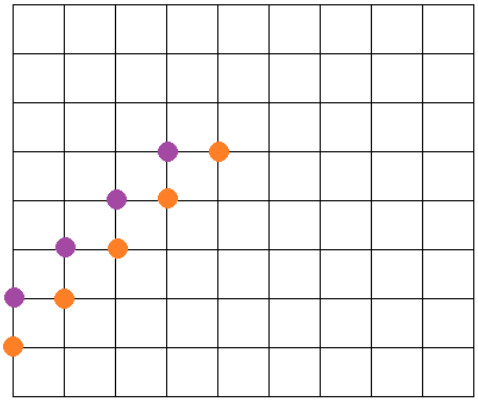
Notice that in any path, (the blue number) minus (the red number) is either zero or one, and it is one when the path passes through a green edge. Thus, (the sum of blue numbers) = (the sum of red numbers) + (the number of paths with a green edge).

We handle horizontal green edges and vertical green edges separately. The number of paths with a horizontal green edge is equal to the number of paths with a green vertex in the following grid (Note that the grid is compressed by one horizontally):



This is hard to compute quickly. However, we can again use the result from previous diagonals.

For the ease of understanding, here's a bigger example:



In this diaglam, we want to compute the number of paths with a purple vertex. However, we already know the number of paths with an orange vertex (during the computation of previous diagonal). Since the difference between those two numbers can be represented as a simple binary coefficient, it can be computed in O(1).

Atcoder Grand Contest 018 解説

writer : maroonrk

平成 29 年 7 月 23 日

For International Readers: English editorial starts from page 6.

A : Getting Difference

A_1,A_2,...,A_N 全ての最大公約数を G とします。また、A_1,A_2,...,A_N の最大値を M とします。このとき、K の書かれたボールを箱に入れることができるための必要十分条件は、K が G で割り切れる M 以下の整数であることです。後はこれを証明します。どんな操作をしても、箱の中のボールに書かれた数の最大公約数が G のままであり、また箱の中のボールに書かれた数の最大値が M のままであることは簡単にわかります。よって、必要性は示されました。次に、十分性を説明します。二つのボールに書かれた差を取るという操作を繰り返すと、ユークリッドの互除法のような操作が行えます。そのため、ある二つのボールの最大公約数のボールを箱に入れることが可能になり、特に、すべての数の最大公約数である G の書かれたボールを箱に入れることができます。そのあとは、M と G の差を取り、それと G の差を取り、…と繰り返すことで、M,$M-G$,$M-2G$,...$2G$,G の書かれたボールはすべて箱に入ります。よって、K の書かれたボールを箱に入れることができました。

B : Sports Festival

まず、すべてのスポーツを実施すると仮定して、どのスポーツに何人の人が参加するか調べます。ここで、最も参加人数の多いスポーツがスポーツ P で、その参加人数が Q 人だとします。当り前ですが、この段階で、参加人数の最大値が Q の解が得られています。よって、あと考えるべきは、参加人数の最大値が Q 未満の解です。そして、そのような解において、スポーツ P は必ず実施されません。なので、Q 未満の解を求めるには、スポーツ P を候補から外した上で、もう一度同じ問題を解きなおせばよいです。これを再帰的に行っていけば、最終的な解を求めることができます。再帰するごとに候補の数が減るので、再帰は合計 M 回呼ばれます。誰がどのスポーツを選ぶのか愚直にシミュレーションすると $O(NM)$ かかるので、全体で $O(NM^2)$ でこの問題は解けました。なお、シミュレーションを工夫すると、全体で $O(NM)$ でこの問題を解くこともできます。

C : Coins

まず初めに、コインを持っている人を全員、持っている金のコインの枚数 – 持っている銀のコインの枚数 で昇順にソートし、左から右へ並ばせておきます。すると、最適解の一つにおいて、金

のコインをくれる全ての人は、銀のコインをくれる全ての人よりも右に立っていることがわかります。これは、そうでない場合、つまり、ある二人の人がいて、左に立っている方が金のコインを、右に立っている方が銀のコインをくれる場合、左から銀のコイン、右から金のコインをもらうように変えても損しないことからわかります。上の主張を言い換えると、ある整数 K が存在して、列の中で左の K 人のうち、Y 人から銀のコイン、$K-Y$ 人から銅のコインをもらい、列の中で右の $X+Y+Z-K$ 人のうち、X 人から金のコイン、$Y+Z-K$ 人から銅のコインをもらう、という形の最適解が存在することを言っています。よって、すべての K について、左側からもらうコインの最大枚数、右側からもらうコインの最大枚数を求めればよいです。ある K について、左側からもらうコインの枚数を最大化することを考えます。このとき、左側の人の中で、持っている銀のコインの枚数 – 持っている銅のコインの枚数 の大きい方から Y 人選び、彼らから銀のコインを、残りから銅のコインをもらうのが最適な戦略になります。K を小さい方から大きい方へと動かしてゆき、持っている銀のコインの枚数 – 持っている銅のコインの枚数 を優先度付きキューなどで管理すれば、K を一つ大きくする際に $O(log(X+Y+Z))$ で、左からもらうコインの最大枚数を更新できるので、全ての K について、合計で $O((X+Y+Z)log(X+Y+Z))$ で求めることができます。右側からもらうコインの最大枚数についても、同様にして求めることができます。よってこの問題は、$O((X+Y+Z)log(X+Y+Z))$ で解けました。

D : Tree and Hamilton Path

完全グラフ上での移動ではなく、木上で移動するものと考えます。まず、各辺について、その辺を通る回数の上限を考えます。辺 i で木を切った時に、残る二つの木のうち小さいものの頂点数を S_i とすると、辺 i を通る回数は、小さい方の木に入って出るという操作が高々 S_i 回しか行えないため、高々 $2\times S_i$ 回です。以下では、ここで求めた、$2\times S_i$ を、辺 i の上限と呼ぶことにします。ここで、2 通りの場合分けを行います。

$S_i = N/2$ となる辺がある場合。$S_i = N/2$ なる辺を辺 e と呼ぶことにします。辺 e を通る回数は、先ほど求めた上限では N 回ですが、そもそも、N 頂点のパスは当然 $N-1$ 回しか移動しないので、辺 e を通る回数は高々 $N-1$ 回です。ここで、辺 e で木を切ったあとに残る二つの木の頂点を、それぞれ、X_1,X_2,...,$X_{N/2}$、Y_1,Y_2,...,$Y_{N/2}$ と置きます。特に、辺 e が結ぶ二つの頂点が X_1 と $Y_{N/2}$ であるようにします。すると、$X_1 \to Y_1 \to X_2 \to Y_2 \to ...X_{N/2} \to Y_{N/2}$ というハミルトンパスを考えると、このパスに沿って移動した場合、辺 e 以外のすべての辺について、その辺を通る回数が上限に達しており、また辺 e を $N-1$ 回通っています。よってこれが最適な動き方だと分かり、最適解を求めることができます。

$S_i = N/2$ となる辺がない場合。木の重心（この場合、必ず一意に決定する）を G とおきます。G に接続する辺について、先ほど求めた通る回数の上限を足し合わせると $2(N-1)$ となりますが、これらの辺を通る回数の合計は、$2(N-1)$ 回にはなりません。なぜなら、G に接続する辺すべてを上限いっぱいの回数通ることにすると、G 以外の頂点からパスが始まることも終わることもできなくなり、明らかにありえないからです。よって、G に接続する辺のうち、少なくとも 1 本は、上限より少ない回数しか通りません。ここで、G に接続する辺のうち、最も短い辺を e とします。辺 e 以外の辺はすべて上限いっぱいの回数通り、辺 e だけは上限 -1 回通るような動き方があれば、これは明らかに最適な動きです。そして、このような動きは必ず可能です。木から G を取り除いた後に残る木の頂点数を、T_1,T_2,...,T_K(ここで、$T_1 \geq T_2 \geq ...T_K$) とおきます。木の重心の性質と、$S_i = N/2$ となる辺がない、という条件から、$T_1 \leq T_2 + T_3 + ... + T_K$ が成り立ちます。すると、

頂点の列であって、列の中で隣接する頂点間のパスに必ず G が含まれているようなものであって、G で始まり、好きな頂点で終わるものを作ることができます。特に、辺 e が結ぶ頂点のうち G でない方をパスの最後におくことができ、このようなパスは、辺 e 以外の辺を上限回、辺 e を上限 -1 回通っているので、最適な動きになります。

上で上げたどちらの場合も、木の重心を求めたあと、簡単な処理を施すだけで求めることができるので、この問題は $O(N)$ で解けました。

E : Sightseeing Plan

問題を要約すれば、次のようになります。

> 3 つの長方形領域が北西から南東に向かって並んでいる。それぞれの長方形領域から 1 マスずつ選び、選んだ 3 つのマスを通る最短経路の個数の総和を求めよ

上の問題にとりかかる前に、まず、次の問題を考えてみましょう。

> 整数 W, K が与えられる。マス $(1, 1)$ から出発し、マス (x, K) で終わる最短経路の数を、$1 \leq x \leq W$ を満たす全ての x について足し合わせた値を求めよ。

この問題の答えは、マス $(1, 1)$ からマス $(W, K + 1)$ へ向かう最短経路の個数になります。これは、二項係数をこねるか、グリッドをグッと睨むとわかります。

続いて、次の問題を考えてみましょう。

> 整数 W, H が与えられる。マス $(1, 1)$ から出発し、マス (x, y) で終わる最短経路の数を、$1 \leq x \leq W$,$1 \leq y \leq H$ を満たす全ての x, y について足し合わせた値を求めよ。

この問題の答えは、マス $(1, 1)$ からマス $(W + 1, H + 1)$ へ向かう最短経路の個数 -1 になります。これは、先ほどの問題の答えを使って、再びグリッドをグッと睨むとわかります。

最後に、次の問題を考えてみましょう。

> 整数 P, Q, R, S が与えられる。マス $(1, 1)$ から出発し、マス (x, y) で終わる最短経路の数を、$P \leq x \leq Q$,$R \leq y \leq S$ を満たす全ての x, y について足し合わせた値を求めよ。

この問題の答えは、

$$
\begin{aligned}
&+ \text{マス } (1, 1) \text{ からマス } (P, R) \text{ へ向かう最短経路の個数} \\
&- \text{マス } (1, 1) \text{ からマス } (P, S + 1) \text{ へ向かう最短経路の個数} \\
&- \text{マス } (1, 1) \text{ からマス } (Q + 1, R) \text{ へ向かう最短経路の個数} \\
&+ \text{マス } (1, 1) \text{ からマス } (Q + 1, S + 1) \text{ へ向かう最短経路の個数}
\end{aligned}
$$

になります。これは、二次元累積和と同じ考え方を用いると、先ほどの問題の答えから導けます。

以上の結果を用いると、元の問題において、3 つある長方形領域のうち真ん中以外の 2 つは、長方形ではなくて正負の重みのついた 4 つのマスと考えてもよいとわかります。よって、次のような問題が解ければよいです。

マス A、長方形領域、マス B が北西から南東に向かって並んでいる。マスからマスへの最短経路全てについて、通る長方形領域内のマスの数を重みとして足し合わせた値を求めよ。

ここで、次の問題を考えます。

長方形領域、マス A が北西から南東に向かって並んでいる。長方形領域の最も北西のマスからマス A への最短経路すべてについて、通る長方形領域内のマスの数を重みとして足し合わせた値を求めよ。

この問題は、長方形領域から出る位置を決め打ちすると、通る長方形領域内のマスの数が一定になっているので、長方形領域から出る位置を全通り試せば答えを求めることができます。適切な前処理を行っておけば、長方形領域から出る位置 1 つにつき $O(1)$ で答えが求まるので、全体で $O($座標の値$)$ で求まります。

この問題が解ければ、二次元累積和と同じ考え方を用いて、元の問題も同様にして解くことができます。これで、全体で $O($座標の値$)$ で答えを求めることができ、この問題は解けました。

F : Two Trees

ある整数列 X_1,X_2,...,X_N が条件を満たしていたとします。そして、頂点 v が木 1 では C_v 個の子を、木 2 では D_v 個の子を持っていたとします。条件が満たされている場合、$1 \equiv -1 \pmod 2$ なので、$X_v + C_v \equiv X_v + D_v \equiv 1 \pmod 2$ となります。ここで、明らかに、$C_v \equiv D_v \pmod 2$ がわかります。つまり、$C_v \equiv D_v \pmod 2$ が全ての頂点について成り立っていることが、条件を満たす整数列が存在することの必要条件になります。

次に、すべての頂点について $C_v \equiv D_v \pmod 2$ が満たされている場合、実際に条件を満たす整数列を作れることを示します。まず、X_v の偶奇を調べます。そして、$X_v \equiv 0 \pmod 2$ となる X_v については、$X_v = 0$ と決定してしまいます。次に、$X_v \equiv 0 \pmod 2$ となる X_v については、$X_v = 1, -1$ とすることにします。ここで、木 2 のことはいったん忘れて、木 1 において条件を満たすようにすることを考えます。N 頂点からなるグラフを考え、以下の条件を満たすようなマッチングを作ることを考えます。

- 木 1 の任意の部分木について、その部分木内に、$X_v = 1, -1$ となるような頂点が $2K + 1$ 個あった時に、そのうち $2K$ 個がマッチングを作る。

このようなマッチングは、DFS で求めることができます。こうしてできたマッチングには、次のような性質があります。

- 各ペア (u, v) について、$X_u = -X_v$ となっていれば、整数列 X_1,X_2,...,X_N は木 1 に関して条件を満たす。

これは、マッチングの性質から、どの部分木についてもペア同士で値を打ち消しあうことからわかります。今、木 1 に関して条件が満たされるようにしましたが、同じようなマッチングを木 2 についても求めてみます。そして、木 1 で求めたマッチングと木 2 で求めたマッチングを、グラフの辺だと思って一つのグラフにまとめてみます。こうして作られたグラフは、次の性質を持っています。

- 各辺 $e = (u, v)$ について、$X_u = -X_v$ となっていれば、整数列 X_1,X_2,...,X_N は木 1、木 2 両者に関して条件を満たす。

ここで、二つのマッチングを重ね合わせてできたグラフは、奇数長閉路を持つことができないことから、二部グラフです。よってこのグラフにおいて、各辺 $e = (u, v)$ について、$X_u = -X_v$ となる値の割り振りが可能です。辺を張る DFS や二部グラフに値を割り振る DFS は $O(N)$ で行えるので、この問題は $O(N)$ で解けました。

Atcoder Grand Contest 018 Editorial

writer : maroonrk

平成 29 年 7 月 23 日

A : Getting Difference

Let G be the greatest common divisor of A_1,A_2,...,A_N. Also, let M be the maximum of A_1,A_2,...,A_N.

When two integers x, y are both multiples of G, their absolute difference $|x - y|$ is also a multiple of G. Also, when $x, y \leq M$, $|x - y| \leq M$. Thus, all integers in the box is always a multiple of G, and at most M.

Next, let's prove that we can construct all such integers. Let X be the smallest integer in the box. If all integers in the box are multiples of X, it means that $X = G$. Otherwise, we can find an integer $Y = kX + Z$ that is not a multiple of X. Using this, we can construct $(k-1)X+Z, (k-2)X+Z, \ldots, Z$, so we got a new integer Z that is smaller than X. By repeating this process, we will eventually get G. Once we get G, we can construct $M, M-G, M-2G, \ldots, G$, and these are everything we want to get.

In summary, we shuold output "POSSIBLE" if and only if K is a multiple of G and $K \leq M$.

B : Sports Festival

First, suppose that we select all sports. In this case, let P be the sport that is chosen by the largest number of participants, and let this number Q. Obviously, the answer of the original problem is at most Q.

Can the answer be smaller than Q? In case we choose the sport P, at least Q people will choose this sport, no matter what subset of the remaining $M-1$ sports we choose. Thus, from now on, we can assume that we never choose the sport P. Recursively, we can solve a problem with N participants and $M-1$ sports.

Since each step of the recursion reduces the number of sports by one, the recursion terminates after M steps. In total, this solution works in $O(NM^2)$. If we implement this solution efficiently, it's also possible to solve this problem in $O(NM)$.

C : Coins

First, we sort the people in the increasing order of (the number of gold coins) minus (the number of silver coins), from left to right. That is, we assume that $A_1 - B_1 \leq A_2 - B_2 \leq \ldots$. In one optimal solution, all gold people (people who give Snuke gold coins) are to the right of all silver people. (Otherwise, if a gold person g is to the left of a silver person s, even if Snuke receives silver coins from g and gold coins from s, the total number of coins will not decrease.)

Therefore, there exists an integer K, such that

- Among the leftmost K people, there are Y silver people and $K - Y$ bronze people.
- Among the remaining people, there are X gold people and $Y + Z - K$ brone people.

For simplicity, let's assume that each person has exactly zero bronze coins. Otherwise, we can replace a person (A_i, B_i, C_i) with $(A_i - C_i, B_i - C_i, 0)$, and add C_i to the final answer. In this case, for a fixed K, we want to compute the sum of following values:

- The sum of Y largest integers among $B_1, \ldots, B_k$.
- The sum of X largest integers among $A_{K+1}, \ldots, A_{X+Y+Z}$.

It's not hard to compute these values for all K in $O((X + Y + Z)log(X + Y + Z))$ time with priority queues.

D : Tree and Hamilton Path

Instead of Hamiltonian Paths on the complete graph, imagine that we visit all vertices of the tree in some order. How many times do we pass through an edge i? If we cut the tree by the edge i, we get two connected components. Let S_i be the size of the smaller of those two components. It's easy to see that we can pass through this edge at most $2S_i$ times. Thus, the answer is at most $\sum 2S_iC_i$. Actually we can prove that the answer is always $\sum 2S_iC_i$ if we consider Hamiltonian Cycles, but in case of Haniltonian Paths, the answer is slightly smaller.

In this problem, centroids play an important role. It is well-known that there are two types of centroids:

Case I. There is a unique edge e such that $S_e = N/2$

In this case, $2S_e = N$, but obviously, we can pass through the edge e at most $N-1$ times. Thus, the answer is at most $\sum 2S_iC_i - C_v$.

On the other hand, we can achieve this value in the following way. Let X_1,X_2,...,$X_{N/2}$ be vertices in one component, and Y_1,Y_2,...,$Y_{N/2}$ be vertices in the other component. In particular, X_1 and $Y_{N/2}$ are the two endpoints of e. Then, the path $X_1 \to Y_1 \to X_2 \to Y_2 \to \cdots \to X_{N/2} \to Y_{N/2}$ achieves the bound.

Therefore, in this case, the answer is $\sum 2S_iC_i - C_v$.

Case II. There is a unique vertex G such that $S_G < N/2$

Suppose that G is incident to k edges $e_1, \ldots, e_k$. If we remove G from the tree, k connected components will appear: call them $T_1, \ldots, T_k$. Here T_i corresponds to e_i.

We know that $S_i = |T_i|$. If we want to pass through the edge e_i $2S_i$ times, we must start the path outside T_i and end the path outside T_i. It is impossible to satisfy this for all i: then, we must start/end the pass outside $T_1, \ldots, T_k$, but there's only one such vertex (G). Thus, the answer is at most $\sum 2S_iC_i - \min\{C_{e_1}, \ldots, C_{e_k}\}$.

Without loss of generality, we can assume that $\min\{C_{e_1}, \ldots, C_{e_k}\} = C_{e_1}$. We construct a path of length $\sum 2S_iC_i - C_{e_1}$. Let G, v be the endpoints of e_1. Here, we use the following fact (the proof is not very hard, an exercise for readers):

> There are $M \geq 2$ balls of various colors. In each color, there are at most $M/2$ balls. Then, we can always arrange balls in a circle such that no two balls of the same color are adjacent.

Because of this, we can arrange all $N-1$ vertices of the tree except for G in a circle, such that no two adjacent vertices are from the same T_i. We split this cycle next to v, and construct a sequence that ends with v, and append G to the beginning of the sequence. This sequence achieves the cost $\sum 2S_iC_i - C_{e_1}$.

Therefore, in this case, the answer is $\sum 2S_iC_i - \min\{C_{e_1}, \ldots, C_{e_k}\}$.

In both cases, this solution works in $O(N)$.

E : Sightseeing Plan

The problem asks the following:

> You are given three rectangular regions A, B, C. Choose a point from each region, and count the number of paths that passes through all chosen points. Compute the sum of these values.

First, we state some easy lemmas. Let $C(x, y) = \frac{(x+y)!}{x!y!}$ be the number of paths from $(0, 0)$ to (x, y).

> $\sum_{y=0}^{Y} C(X, y) = C(X + 1, Y)$

Consider a path from $(0, 0)$ to $(X+1, Y)$. It passes through an edge between (X, k) and $(X+1, k)$ for some k, and the number of such paths is $C(X, k)$.

> $\sum_{x=0}^{X} \sum_{y=0}^{Y} C(x, y) = C(X + 1, Y + 1) - 1$

Apply the previous lemma twice.

> $\sum_{x=X_1}^{X_2} \sum_{y=Y_1}^{Y_2} C(x, y) = C(X_1, Y_1) - C(X_1, Y_2 + 1) - C(X_2 + 1, Y_1) + C(X_2 + 1, Y_2 + 1)$

This is similar to a well-known trick where you compute the sum in a subrectangle by adding/subtracting rectangles four times.

In the original problem, we choose the start from a rectangle region. However, because of this lemma, we can assume that there are only four (weighted) possible positions for the start. In particular,

- $(X_1 - 1, Y_1 - 1)$ with weight $+1$.
- $(X_1 - 1, Y_2)$ with weight -1.
- $(X_2, Y_1 - 1)$ with weight -1.
- (X_2, Y_2) with weight $+1$.

Similarly, the goal is also considered as four points, so we can solve the original problem by solving the following simpler problem $4 \times 4 = 16$ times:

> You are given a point A, a rectangular region B, and a point C. Choose a point from B, and count the number of paths that passes through all chosen points. Compute the sum of these values.

We should count the number of paths from A to C that passes through B at least once. However, if the size of the intersection of a path P and the rectangle B is k, this path should be counted with multiplicity k.

The following works:

- For each $X_3 \leq x \leq X_4$, count the number of paths that enters B by $(x, Y_3 - 1) \to (x, Y_3)$ with multiplicity $-(x + Y_3)$.
- For each $Y_3 \leq y \leq Y_4$, count the number of paths that enters B by $(X_3 - 1, y) \to (X_3, y)$ with multiplicity $-(X_3 + y)$.
- For each $X_3 \leq x \leq X_4$, count the number of paths that leaves B by $(x, Y_4) \to (x, Y_4 + 1)$ with multiplicity $x + Y_4 + 1$.
- For each $Y_3 \leq y \leq Y_4$, count the number of paths that leaves B by $(X_4, y) \to (X_4 + 1, y)$ with multiplicity $X_4 + y + 1$.

In total, this solution works in $O(MAXCOORDINATE)$, with pre-computation of factorials and inverses of factorials.

F : Two Trees

Since $1 \equiv -1 \pmod 2$, we know the parity of the sum of each subtree. Suppose that a vertex v has C_v children in the first tree. Then, X_v must be even if C_v is odd, and X_v must be odd if C_v is even. Similarly, if a vertex v has D_v children in the first tree, X_v must be even if D_v is odd, and X_v must be odd if D_v is even.

If $C_v \equiv D_v \pmod 2$ for all v, the answer is always "POSSIBLE", as we describe later. Otherwise, the answer is "IMPOSSIBLE".

Author's solution

Construct a new graph with $2N + 1$ vertices in the following way:

- The vertices are named $P_1, P_2, \ldots, P_N, Q_1, Q_2, \ldots, Q_N, R$.
- If there is an edge between i and j in the first tree, add an edge between P_i and P_j.
- If there is an edge between i and j in the second tree, add an edge between Q_i and Q_j.
- If p is the root of the first tree, add an edge between P_p and R.
- If q is the root of the first tree, add an edge between Q_q and R.
- If X_v is odd, add an edge between P_v and Q_v.

This graph is connected and the degree of each vertex is even, so it has a Eulerian Cycle. Find one Eulerian Cycle, and direct it in an arbitrary way. Define $X_v = 0$ if X_v is even, $X_v = 1$ if the Eulerian Cycle is directed from P_v to Q_v, and $X_v = -1$ if the cycle is directed in the opposite way. It's easy to see that this satisfies the constraints.

Admin's solution

As we see, we know the parity of X_v. If $X_v \equiv 0 \pmod 2$, we decide $X_v = 0$, and call it "even variable". Otherwise, we decide that X_v is either 1 or -1, and call it "odd variable".

Now, let's forget about the second tree for a while, and try to satisfy the condition in the first tree. We want to find some disjoint pairs of odd variables with the following property:

- For each subtree T of the first tree, if T contains $2K + 1$ odd variables, there are K pairs among those odd variables.

Such pairs can be easily found by a DFS.

Then, for each pair (u, v), let's force a constraint that $X_u = -X_v$. If all pairs satisfy this condition, obviously, the sum in each subtree will be either 1 or -1.

Now, let's construct a graph with N vertices $1, 2, \ldots, N$. If u and v are paired in the first tree, add a red edge between vertices u and v. Since the pairs are disjoint, the red edges form a matching of the graph. Similarly, do the same for the second tree, and add a blue edge between two vertices paired in the second tree. Now this graph has two matchings: red and blue.

Consider a cycle in this graph. Since a red edge and a blue edge must appear alternately in this cycle, this graph doesn't contain odd cycles, so it is bipartite. This means that we can satisfy constraints $X_u = -X_v$ for both red and blue edges at the same time. This is what we want to get.

AtCoder Grand Contest 017 解説

writer: semiexp

2017 年 7 月 9 日

For International Readers: English editorial starts on page 6.

A: Biscuits

A_i がすべて偶数の場合は，どのように袋を選んでも食べるビスケットの枚数は偶数である．よって，$P = 0$ の場合は答えは 2^N，$P = 1$ の場合は答えは 0 である．

A_i の中に奇数が含まれているものとすると，A_k が奇数であるような k をとることができる．k 番目の袋以外の $N-1$ 個の袋の選び方は 2^{N-1} 通りである．これらの $N-1$ 個の袋から選んだ袋のビスケットの枚数の合計を S とするとき，

- S が奇数であれば，k 番目の袋を選ぶと合計は偶数，選ばないと合計は奇数.
- S が偶数であれば，k 番目の袋を選ぶと合計は奇数，選ばないと合計は偶数.

であるから，これらの 2^{N-1} 通りの選び方のそれぞれについて，k 番目の袋を選ぶか選ばないかでちょうど 1 通り，合計を偶数にする方法および奇数にする方法が存在する．よって，この場合の答えは P によらず 2^{N-1} である．

B: Moderate Differences

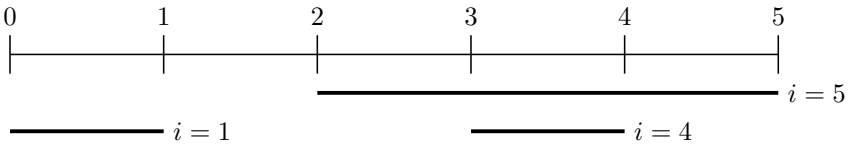
左から i 番目のマスに書く整数を X_i とする ($X_1 = A, X_N = B$)．$Y_i = X_{i+1} - X_i$ とおく ($i = 1, 2, \ldots, N-1$)．このとき，隣接するマスの整数の差の条件は,「各 $i = 1, 2, \ldots, N-1$ に対して，$-D \leq Y_i \leq -C$ または $C \leq Y_i \leq D$ が成り立つ」と言い換えられる．また，$\sum_{i=1}^{N-1} Y_i = \sum_{i=1}^{N-1} (X_{i+1} - X_i) = X_N - X_1 = B - A$ が成り立つ．逆に，これを満たすような整数列 $\{Y_i\}$ が存在すれば，$X_2 = X_1 + Y_1, X_3 = X_2 + Y_2, \ldots$ と順次定めていくことにより，条件をみたす $\{X_i\}$ を得ることができる．

$-D \leq Y_i \leq -C$ を満たす i が m 個存在するとする．このとき，他の $N-1-m$ 個の i については $C \leq Y_i \leq D$ が成り立つ．これらの条件式を各 i について足し合わせることで，$C(N-1-m) - Dm \leq \sum_{i=1}^{N-1} Y_i \leq -Cm + (N-1-m)D$ が成り立つ．よって，$C(N-1-m) - Dm \leq B - A \leq -Cm + (N-1-m)D$ が必要であるが，逆にこれが成り立つとき，条件をみたすような $\{Y_i\}$ をとることができる．実際，最初 Y_i の値を $-D$ または C にしておき，総和が $B - A$ 未満である限り値を $Y_1, Y_2, Y_3, \ldots, Y_{N-1}, Y_1, Y_2, \ldots$ の順で増やしていけばよい．

m を固定した際に条件をみたす書き方が可能かどうかは上のように $O(1)$ で判定できるので，$m = 0, 1, \ldots, N-1$ をすべて試し，1 つでも可能なものがあれば `YES`，いずれも不可能であれば `NO` を出力すればよい．

C: Snuke and Spells

整数 i が書かれたボールの個数を N_i とする．下図のように，各 $i = 1, 2, \ldots, N$ に対して，区間 $[i - N_i, i]$ を考える (下図は，ボールに書かれている整数が $1, 4, 5, 5, 5$ である場合に対応する)．すると，魔法を何回か唱えることでボールをすべて消滅させられることと，これらの区間が $[0, N]$ を覆い尽くすことは同値である．



整数 i が書かれたボールを j に変更するのは，区間 $[i-N_i,i]$ を縮めて $[i-N_i+1,i]$ にし，区間 $[j-N_j,j]$ を伸ばして $[j-N_j-1,j]$ にすることに相当する．特に，1 回の変更操作では，新たに覆える $[x-1,x]$ は高々 1 箇所である．よって，最初の状態で覆われていない $[x-1,x]$ が L 箇所あるとき，答えは L 以上である．

一方，L 回の操作で $[0,N]$ を覆い尽くすことは可能である．これを示すには，1 回の操作により覆われていない $[x-1,x]$ を 1 つ減らせることを示せばよい．$[j-1,j]$ が覆われていないような $j(1\leq j\leq N)$ をとる．$i-N_i<0$ であるような i があれば，i が書かれたボールを 1 個 j に変更すればよい．そのような i が存在しなければ，2 つ以上の区間で覆われているような $[r-1,r]$ が存在する．そのような r のうち最小のものを取ってくると，$i-N_i=r-1$ となるような (すなわち，$r-1$ を起点とするような) 区間が少なくとも 1 つ存在するから，この i が書かれたボールを 1 個 j に変更すればよい．

以上の議論より，$[x-1,x]$ が覆われていないような $x=1,2,\ldots,N$ の個数を数えれば答えが得られることがわかる．この問題では，複数の変更クエリを処理する必要があるが，そのためには「各 x に対し，$[x-1,x]$ を覆う区間の数」「$[x-1,x]$ が覆われていないような $x=1,2,\ldots,N$ の個数」を順次更新していけばよい (前者を更新する際に，後者の値も適切に更新する)．区間を 1 縮める，伸ばす操作でこの更新は $O(1)$ でできるから，全体として更新クエリは $O(M)$ で処理できる．初期化コスト $O(N)$ と合わせて，この問題は $O(N+M)$ で解くことができる．

D: Game on Tree

木に対して，その木から始めてゲームをしたときの Grundy 数を考える．

根 (頂点 1) から複数本の辺が出ている場合は，それぞれの辺ごとに別の根を持ってきて，(より辺の本数が少ない) 複数の木に対するゲームにすることができる．Grundy 数の性質より，元の木の Grundy 数は，このようにして得た木の Grundy 数すべてを XOR したものになる．

根からちょうど 1 本の辺が出ている場合は，Grundy 数に対して次の定理が成り立つ．

定理. 根付き木 T の *Grundy* 数を $G(T)$ で表す．T の根 r から新たに 1 本の辺を生やし，その辺の端点のうち r でないほうを新たに根とした木を T' とするとき，$G(T')=G(T)+1$ が成り立つ．

証明. 数学的帰納法により示す．

T が 1 頂点のみである場合は，$G(T)=0,G(T')=1$ が容易に確かめられるからよい．

T が $n-1$ 頂点以下の場合示されたと仮定して，T が n 頂点の場合を示す．T からある辺を取り除き，根を含まない連結成分も取り除いてできる木は，辺の選び方により $n-1$ 通り考えられるが，これらを $S_1,S_2,\ldots,S_{n-1}$ とおく．また，S_i の根から新たに 1 本の辺を生やし，その辺の r でない端点を根とした木を S_i' とおく．すると，帰納法の仮定より，$G(S_i')=G(S_i)+1$ が成り立つ．また，T' から遷移可能な木は，1 頂点のみからなる木 I (辺 rr' を取り除いた場合) または S_i' のいずれか (それ以外の辺を取り除いた場合) である．

Grundy 数の定義より，$G(S_1),\ldots,G(S_{n-1})$ の中には，$0,1,\ldots,G(T)-1$ はすべて含まれるが，$G(T)$ は含まれない．よって，$G(I),G(S_1')=G(S_1)+1,\ldots,G(S_{n-1}')=G(S_{n-1})+1$ の中には，$0,1,\ldots,G(T)$ はすべて含まれるが，$G(T)+1$ は含まれない．ゆえに $G(T')=G(T)+1$ である．よって示された． □

上の議論により，根から DFS を行いつつ各点を根とした部分木の Grundy 数を求めることで，木全体の Grundy 数を求めることができる．

E: Jigsaw

ピース i の**型**を，次のようにして定める．

- $C_i = 0$ ならば，$l = +_{A_i}$ とする．$C_i \neq 0$ ならば，$l = -_{C_i}$ とする．
- $D_i = 0$ ならば，$r = +_{B_i}$ とする．$D_i \neq 0$ ならば，$r = -_{B_i}$ とする．
- このとき，ピース i の型は (l, r) である．

$+_1, \ldots, +_H$ をまとめて $+$，$-_1, \ldots, -_{H-1}$ をまとめて $-$ で表すことにする．すると，型 (l, r) のピースのすぐ右に型 (l', r') のピースを置けるための必要十分条件は，次のいずれかを満たすことであるとわかる (*)：

- r, l' がともに $+$.
- ある k に対し，$r = +_k, l' = -_k$.
- ある k に対し，$r = -_k, l' = +_k$.

また，一番左のピースの型 (l, r) について，l は $+$ であり，一番右のピースの型 (l', r') について，r は $+$ である $(\star)$.

ここで，$2H$ 個の頂点 $u_1, \ldots, u_H, v_1, \ldots, v_H$ を持った有向グラフを考える．このグラフは，多重辺，自己辺なども許すものとする．このグラフの辺を，次のようにして与える：

- このグラフには辺 $1, 2, \ldots, N$ の N 個の辺がある．
- ピース i の型が (l, r) であるとき，辺 i の起点は
  - $l = +_k$ なら u_k.
  - $l = -_k$ なら v_k.
- ピース i の型が (l, r) であるとき，辺 i の終点は
  - $r = +_k$ なら v_k.
  - $r = -_k$ なら u_k.

グラフの辺をたどることを，そのピースを今までで最後に置いたピースの右の位置に置くことに対応させる．すると，(*) で示した 3 つの場合の並べ方は，グラフ上で次のたどり方に対応する：

- r, l' がともに $+$ の場合：最後に通った辺の終点が v_* であるとき，ある u_* に移動して，その点から出ている辺をたどる．
- ある k に対し，$r = +_k, l' = -_k$ の場合および，ある k に対し，$r = -_k, l' = +_k$ の場合：最後に通った辺の終点から出ている辺をたどる．

また，$(\star)$ に対応して，最初に通る辺の始点は u_*，最後に通る辺の終点は v_* である．

すなわち，グラフ上で「v_* から u_* に辺を 0 本以上増やすことで，ある u_* からある v_* までのオイラー路が作れるか」を判定すればよいことがわかる．

グラフに新たな頂点 w を追加する．v_* から u_* に辺を生やすときは，v_* から w の辺と w から u_* の辺を 1 本ずつ増やすことにする．さらに，最初に通る辺の始点を u_s，最後に通る辺の終点を v_t として，v_t から w，w から u_s の辺も 1 本ずつ増やすことにする．すると，元の条件が成り立てば，この新しいグラフ上で w を含むオイラー閉路が存在する．一方，このようなオイラー閉路が存在すれば，閉路中の $v_* \to w$, $w \to u_*$ の形の辺を 1 本ずつ取り除き，残る $v_* \to w \to u_*$ の辺をすべて $v_* \to u_*$ に変更することで，元の条件のオイラー路が作れることがわかる．

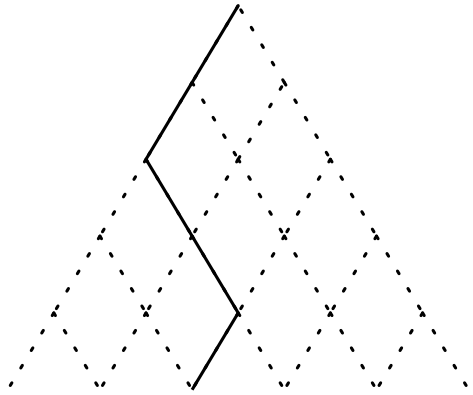
さて，有向グラフにおいてオイラー閉路が存在するためには，全体が (強) 連結でありかつ，すべての頂点において入次数と出次数が一致していることが必要十分である．v_* からは w への辺を増やすことしかできないので，v_* において出次数が入次数より大きい場合は不可能である．出次数が入次数以下の場合は，次数の差だけ w へ辺を増やす必要がある．u_* についても同様である．一方，この議論から，辺を増やした後のグラフは一意に定まる．また，このようにすると，w 以外のすべての頂点において入次数と出次数が一致するため，w においてもこれらは一致する．よって，次数の条件から不可能でない場合は，辺を増やした後，w を含む連結成分がすべての辺を含むかどうか判定すればよい．

F: Zigzag

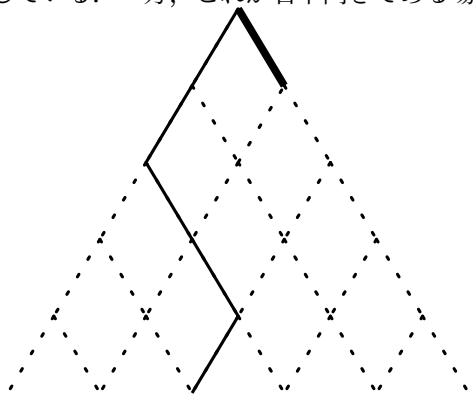
折れ線 1 から順に，現在の折れ線が直前の折れ線よりも左に来ることがないように，折れ線を 1 回目，2 回目，…，$N-1$ 回目の移動の順番で決定していくことを考える．

1 つの方法として，状態として「現在の折れ線の番号 i」「現在の折れ線で今までに行った移動の回数 j」「前回の折れ線と，現在の折れ線において，j 回目までの移動の後の横方向の位置の差 」「現在の折れ線で j 回目までの移動の向き」「直前の折れ線で，$j+1$ 回目以降の移動の向き」を持ってビット DP を行う方法がある．ここで,「現在の折れ線で j 回目までの移動の向き」「直前の折れ線で，$j+1$ 回目以降の移動の向き」は合わせて $N-1$ 箇所の向きであるから，この 2 つによる状態数は合わせて $O(2^n)$ である．よって，このビット DP の状態数は $O(n^3 2^n)$ であり，計算量が大きすぎる．

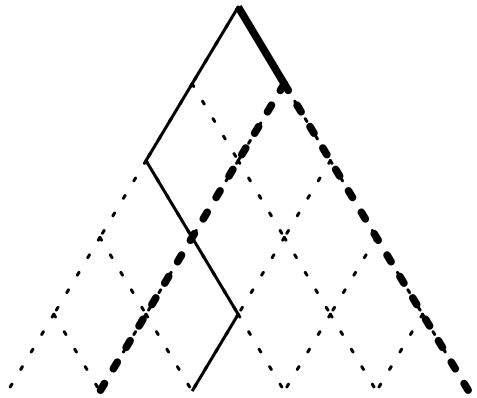
ここで，現在の折れ線が超えては行けない限界 (左の折れ線の位置) を，現在の折れ線の先端の観点から考える．下の図において，実線で示した折れ線が前回の折れ線であるとし，これより左に来ないように現在の折れ線を描くことを考える．



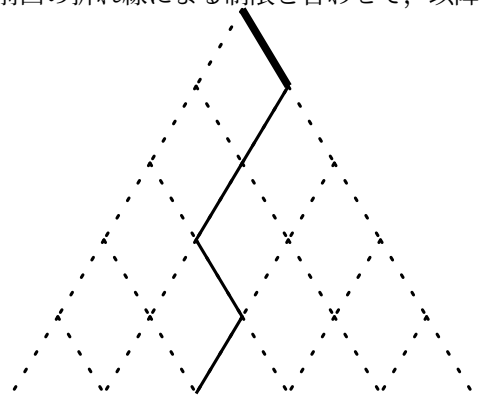
現在の折れ線の最初の移動が左下向きである場合は，ここまでの移動において，前回と今回の折れ線の先端位置は一致している．一方，これが右下向きである場合は，先端位置が 1 ずれる．



このとき，現在の先端から到達可能な部分を図示すると，下の太点線で囲まれた領域になる．



前回の折れ線による制限と合わせて，以降の移動において超えられない限界は，下の細実線で示した折れ線になる．



このようにすると，折れ線の先端が前回とずれていても，現在の先端を起点とする別な折れ線を限界とすることができる．この新たな限界は，次のようにして求められる．

- 元々の限界における最初の移動と同じ向きに移動している場合は，元々の限界と同じ．
- 元々の限界における最初の移動と異なる向きに移動している場合は,「元々の限界では左下，今回の移動では右下」という移動に限られる．元々の限界において右下の移動がない場合は，新たな限界は「最後まで左下の移動を繰り返す」折れ線である．一方，右下の移動がある場合は，その中で一番最初の移動を単に取り除いてできるものが新たな限界の折れ線となる．

これを用いると，ビット DP の状態として「現在の折れ線の番号 i」「現在の折れ線で今までに行った移動の回数 j」「現在，折れ線が超えてはいけない限界」を持っておけばよく，状態数は $O(n^2 2^n)$ となる．ビット DP における更新は，各状態から高々 2 つの別の状態に値を足し込めばよいが，ここで「新たな限界」を表すビット列を求める必要がある．これはビット演算を用いると効率よく行うことができる．現在の限界が左下向き，次の折れ線の向きが右下向きである場合のみ考える (他の場合は，限界を更新する必要がない)．ビット列の Least Significant Bit (LSB) から順に 1 回目，2 回目，...，$N-1$ 回目の移動に対応させ，0 を左下への移動，1 を右下への移動に対応させる．現在の限界が `m` であるとき,「j ビット目以降で最初の 1 を 0 にし，j ビット目を 1 にした」ビット列が新たな限界であるから，これは `((m - 1 << j) & m) | (1 << j)` で得られる．

よって，各状態からの更新も $O(1)$ で行えるから，全体で $O(n^2 2^n)$ で問題を解くことができる．

AtCoder Grand Contest 017 Editorial

writer: semiexp

July 9th, 2017

A: Biscuits

If all A_i are even numbers, the sum is always even. Thus, if $P = 0$ the answer is 2^N, and otherwise the answer is 0.

Otherwise, there exists an integer k such that A_k is odd. There are 2^{N-1} ways to choose a subset of $N-1$ bags (except for the k-th bag). Let S be the total number of biscuits in this subset.

- If S is odd, if you choose the k-th bag, the total number of biscuits will be even. Otherwise the total number of biscuits will be odd.
- If S is even, if you choose the k-th bag, the total number of biscuits will be odd. Otherwise the total number of biscuits will be even.

In both cases, for a given subset of $N-1$ bags, regardless of the value of P, there is exactly one way to make the total number P modulo 2. Therefore, if at least one bag contains odd number of biscuits, the answer is always 2^{N-1}.

B: Moderate Differences

Let X_i be the integer in the i-th square ($X_1 = A, X_N = B$). Let $Y_i = X_{i+1} - X_i$.

- For each i, $-D \leq Y_i \leq -C$ or $C \leq Y_i \leq D$ must be satisfied.
- $\sum_{i=1}^{N-1} Y_i = B - A$ must be satisfied because $\sum_{i=1}^{N-1} Y_i = \sum_{i=1}^{N-1} (X_{i+1} - X_i) = X_N - X_1 = B - A$.

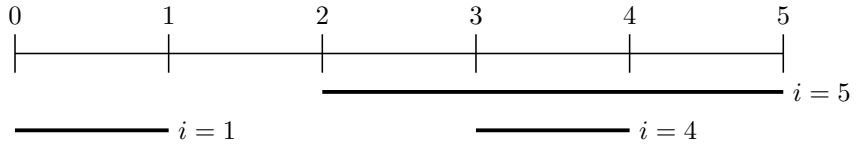
If you can find Y_i that satisfies the conditions above, you can construct a desired sequence by $X_2 = X_1 + Y_1, X_3 = X_2 + Y_2, \ldots$. Thus, these conditions are sufficient.

Suppose that m of Y_i satisfy $-D \leq Y_i \leq -C$ and the remaining $N-1-m$ Y_i satisfy $C \leq Y_i \leq D$. By adding these inequalities, we get $C(N-1-m) - Dm \leq \sum_{i=1}^{N-1} Y_i \leq -Cm + (N-1-m)D$. Thus, $C(N-1-m) - Dm \leq B - A \leq -Cm + (N-1-m)D$ must be satisfied. On the other hand, when this inequality holds, we can find an example of Y_i. We can first set the values of Y_i to $-D$ or C, and while the sum is smaller than $B - A$, we can increase integers one by one in the order $Y_1, Y_2, Y_3, \ldots, Y_{N-1}, Y_1, Y_2, \ldots$.

We can try all values of $m = 0, 1, \ldots, N-1$, and for each m we can check the inequality in $O(1)$ as we described above. If we find at least one m that satisfies the inequality, the answer is `YES`, otherwise the answer is `NO`.

C: Snuke and Spells

Suppose that we have N_i balls with an integer i. For each i, consider the interval $[i - N_i, i]$. For example, the following diagram shows the case with $1, 4, 5, 5, 5$.



We can vanish all balls by the spells if and only if these intervals entirely fill $[0, N]$. We claim that, when the total length of uncovered parts is L, we need L modifications (the proof is attached later).

Therefore, we can solve the problem by counting the number of x such that $[x - 1, x]$ is not covered. In order to handle multiple queries, we should keep the following values:

- For each x, the number of intervals that covers $[x - 1, x]$.
- The total length of uncovered parts.

When we update the former value, we should also update the latter value properly. Each query can be handled in $O(1)$, so this solution works in $O(N + M)$.

Proof.

- If we change an integer i to integer j, the interval $[i - N_i, i]$ will be shrunk to $[i - N_i + 1, i]$ and the interval $[j - N_j, j]$ will be enlarged to $[j - N_j - 1, j]$. In particular, in one modification, the total length of uncovered parts can decrease by at most one. Thus, we need at least L modifications.
- Suppose that some part of $[0, N]$ is not covered. In this case,
  - If there exists i such that $i - N_i < 0$, one of balls with an integer i is unnecessary.
  - Otherwise, choose the smallest integer r such that $[r, r + 1]$ is covered multiple times. There exists at least one i such that $i - B_i = r$, and one of balls with an integer i is unnecessary.

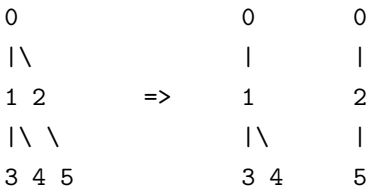
  If $[j-1, j]$ is not covered by any intervals, we can change an integer i to j and we can decrease the total uncovered length by one. By repeating this, we can cover $[0, N]$ in L modifications.

□

D: Game on Tree

We want to compute the Grundy Number of a given tree. If the Grundy Number is zero, Bob wins. Otherwise Alice wins.

Suppose that the root has $k > 1$ children. In this case, we can decompose the original tree into k smaller trees, and the Grundy Number of the original tree is XOR of the Grundy Numbers of new k trees.



If the root has exactly one child, we can compute the Grundy Number using the following theorem:

Theorem. *Let $G(T)$ be the Grundy Number of a rooted tree T. Let T' be a rooted tree obtained by attaching an edge to the root of T (and the new root is the new node attached to the old root). Then, $G(T') = G(T) + 1$.*

Proof. We can prove it by the induction on the number of vertices in T.

Let's compute the Grundy Number of T'.

- If we remove the new edge, we get a single node, and its Grundy Number is zero.
- Suppose that we remove one of other edges, and we get a new tree S. This tree is of the form "(one of trees reachable from T) + (one edge)". By the induction hypothesis, in the set of Grundy Numbers of these trees, $0 + 1, 1 + 1, \ldots, G(T) - 1 + 1$ will appear, but $G(T) + 1$ won't.

Therefore, $G(T') = G(T) + 1$. □

By combining the observations above, we can get the Grundy Number of the original tree by a simple DFS.

E: Blocks

Define the **type** of piece i as follows:

- If $C_i = 0$, $l = +_{A_i}$. If $C_i \neq 0$, $l = -_{C_i}$.
- If $D_i = 0$, $r = -_{B_i}$. If $D_i \neq 0$, $r = +_{B_i}$.
- The **type** of piece i is defined as (l, r).

We can put a piece of type (l', r') immediately to the right of a piece of type (l, r) if one of the following holds:

- l is positive and r' is negative.
- $r = l'$.

Also, when you arrange all intervals, the left part of the leftmost piece must be positive, and the right part of the rightmost part must be negative.

Now, consider a directed graph with $2H$ vertices. The vertices are labeled with $1, 2, \ldots, H$, and $-1, -2, \ldots, -H$. When we have a piece of type (l, r), we add an edge from l to r.

From the observation above, we want to determine if we can decompose this graph into one more more (directed) paths, such that each path starts from a positive vertex and ends at a negative vertex.

It turns out that this is equivalent to the following conditions:

- For each positive vertex, $outdegree \geq indegree$ holds.
- For each negative vertex, $outdegree \leq indegree$ holds.
- In each (weakly) connected component, there must be at least one vertex whose outdegree and indegree are different.

It is clear that these conditions are necessary. On the other hand, when these conditions hold, we can decompose the graph into a set of

- A path from a positive vertex to a negative vertex.
- A cycle.

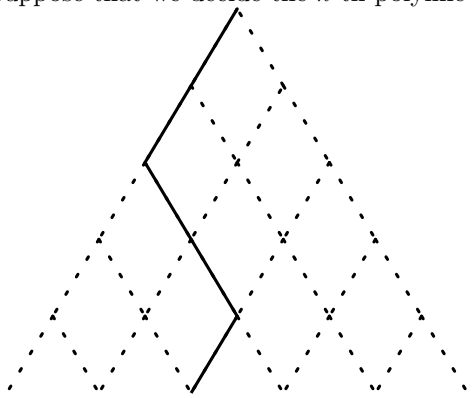
(This decomposition can be obtained by greedy connecting edges in any order).

Now, by the third condition, whenever we have a cycle, there exists at least one path or cycle that is adjacent to this cycle. Thus, we can merge them and we can decrease the number of cycles. By repeating this, we get a desired set of paths.

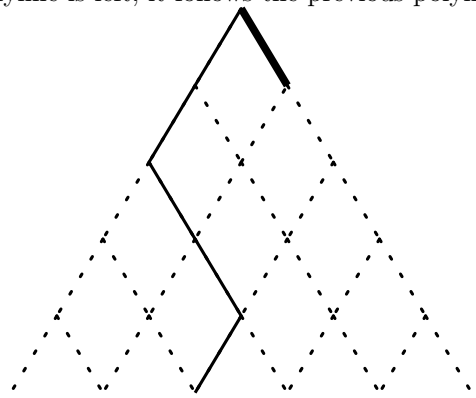
F: Zigzag

We can represent the shape of each polyline as a bitmask. We decide the polylines from polyline 1 in order. Define $dp[k][mask]$ as the number of ways to choose the first k polylines, such that the k-th polyline is $mask$. This DP will lead to $O(4^N * poly(N))$, or it can be improved to $O(2^N * N^3)$, but still it's too slow.

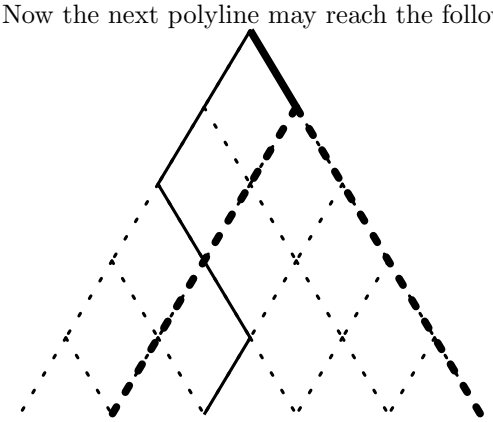
Suppose that we decide the k-th polyline as follows:



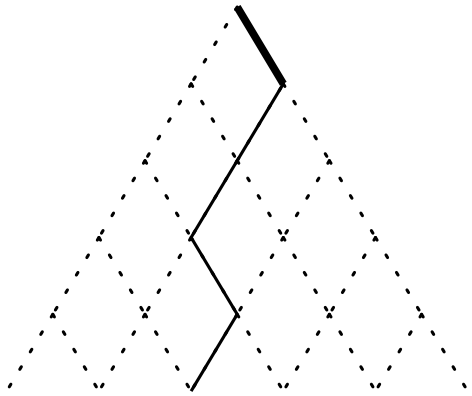
Now, we decide the $k+1$-th polyline from top to bottom. If the direction of the first (topmost) part of the next polyline is left, it follows the previous polyline. If the direction is right, it will look as follows:



Now the next polyline may reach the following part:



However, it is not allowed to go to the left of previous polyline. Thus, the region is limited as follows:



Now we can think as if the previous polyline was indeed this polyline. This way, even this case can be handled in the same way as the former case: now the $k+1$-th polyline follows the k-th polyline.

Let $P = (p_1, \ldots, p_{N-1})$ be the previous polyline, and let $Q = (q_1, \ldots, q_k)$ be the current polyline (it may not have reached the bottom yet). In general, we modify P as follows:

- If Q currently follows P, we don't modify P.
- Otherwise, $q_k = 1$ and $p_k = 0$. If there is no $i > k$ such that $p_i = 1$, we simply change p_k to 1 and now Q follows P. Otherwise, choose the smallest such i, change p_k to 1, and change p_i to 0.

All of these operation can be handled in constant time using bitwise operations.

Now, the state of the DP is $dp[i][j][mask]$. It means the number of ways to choose the first i polyline and the top j parts of the $i+1$-th polyline, with the following constraints:

- In the top j parts, the $i+1$-th polyline follows $mask$.
- In the future, the $i+1$-th polyline is not allowed to go to the left of $mask$.

Now we reduced the number of states to $O(n^2 2^n)$, and it works in $O(n^2 2^n)$ time.

AGC #016 Editorial

writer : sugim48

2017 年 6 月 19 日

For International Readers: English editorial starts on page 5.

A: Shrinking

英小文字 c をひとつ固定します．最終的に s が単一の文字 c のみからなるようにしたいとします．すると，各操作では c を優先的に選べばよいです．よって，シミュレーションによって必要な操作回数が求まります．以上より，c を `a` から `z` まで全探索し，操作回数の最小値を求めればよいです．

B: Colorful Hats

すべての猫の色の種類数を A とします．自分と同じ色の猫が他にいないような猫を alone な猫と呼ぶことにします．このとき，猫 i が alone ならば，$a_i = A - 1$ です．また，猫 i が alone でないならば，$a_i = A$ です．よって，a_i の最小値と最大値の差が 2 以上ならば，答えは `No` です．以降は，a_i の最小値と最大値の差が 0 の場合，1 の場合に分けて説明します．

まず，a_i の最小値と最大値の差が 0 の場合，すなわち a_i がすべて同一の場合を説明します．この場合，猫はすべて alone であるか、すべて alone でないかのどちらかです．すべて alone の場合，$A = N$ より $a_i = N - 1$ です．また，すべて alone でない場合，$2A \leq N$ より $2a_i \leq N$ です．よって，$a_i = N - 1$ または $2a_i \leq N$ のどちらかが成り立つならば `Yes` で，そうでないならば `No` です．

次に，a_i の最小値と最大値の差が 1 の場合を説明します．この場合，A は a_i の最大値です．また，$a_i = A - 1$ ならば猫 i は alone で，$a_i = A$ ならば猫 i は alone ではないです．alone な猫の匹数を x，alone でない猫の匹数を y とします．すると，$x < A$ かつ $2(A - x) \leq y$ ならば `Yes` で，そうでないならば `No` です．

C: Subrectangles

H が h の倍数であり，かつ W が w の倍数である場合，答えは明らかに `No` です．この場合，$H \times W$ の長方形は $h \times w$ の部分長方形たちに分割できるので，部分長方形がすべて負ならば，全体の長方形も負になってしまうからです．逆に，H が h の倍数でないか，W が w の倍数でない場合，答えは `Yes` です．以降は，解の構成法を説明します．

W が w の倍数でない場合を説明します．さらに，$H = h = 1$ とします．この場合に解が構成できれば，その解を縦に H 行分繰り返せば，H や h が一般の場合の解となります．解を $a = (a_1, a_2, \ldots, a_W)$ とします．

また，この累積和を $s = (s_0, s_1, \ldots, s_W)$ とします．すなわち，$s_0 = 0$ かつ $s_i = s_{i-1} + a_i$ です．a の代わりに s を構成することにします．解が条件を満たすためには，$s_W > 0$ かつ $s_i > s_{i+w}$ が必要です．W が w の倍数でないので，これは常に構成できます．s が構成できれば，$a_i = s_i - s_{i-1}$ として a を構成できます．

D: XOR Replace

まず，操作をよく観察してみます．$x = a_1 \oplus a_2 \oplus \cdots \oplus a_N$ とします．今，a_i を x に置き換えたとします．すると，全要素の XOR は

$$\begin{aligned} & a_1 \oplus \cdots \oplus a_{i-1} \oplus x \oplus a_{i+1} \oplus \cdots \oplus a_N \\ = & (a_1 \oplus \cdots \oplus a_{i-1} \oplus a_{i+1} \oplus \cdots \oplus a_N) \oplus (a_1 \oplus \cdots \oplus a_N) \\ = & a_i \end{aligned}$$

となります．これは，「全要素の XOR」と a_i を入れ替えたと見ることができます．そこで，仮に $a_0 = a_1 \oplus a_2 \oplus \cdots \oplus a_N$ とすると，操作は「a_0 と a_i $(1 \leq i \leq N)$ を入れ替える」のと等価です．同様に，仮に $b_0 = b_1 \oplus b_2 \oplus \cdots \oplus b_N$ とすると，この問題の目標は，「a_0 と a_i $(1 \leq i \leq N)$ を入れ替える」という操作を繰り返し，$(a_0, a_1, \ldots, a_N)$ を $(b_0, b_1, \ldots, b_N)$ に一致させる，と言い換えることができます．以降は，この言い換えのもとで説明をします．

まず，$(a_0, a_1, \ldots, a_N)$ と $(b_0, b_1, \ldots, b_N)$ が多重集合として一致していなければ，目標は達成不可能です．逆に一致していれば，目標は達成可能です．

(最短とは限らない) 操作列によって，$(a_0, a_1, \ldots, a_N)$ が $(b_0, b_1, \ldots, b_N)$ へ変わったとします．このとき，各 i $(1 \leq i \leq N)$ について，$a_i = x_i^0 \to x_i^1 \to \cdots \to x_i^{(t_i)} = b_i$ と変わっていったとします．ここで，次のようなグラフを考えます．$(a_0, a_1, \ldots, a_N)$ を (多重ではない) 集合としたものを頂点集合とします．また，各 i $(1 \leq i \leq N)$ ごとに，有向辺 $(x_i^0 \to x_i^1)$, $(x_i^1 \to x_i^2)$, ..., $(x_i^{(t_i-1)} \to x_i^{(t_i)})$ を張ります．このグラフは，b_0 を始点，a_0 を終点とするオイラーグラフであることが分かります．逆に，任意の $b_0 \to a_0$ オイラーグラフは，何らかの操作列に対応することが分かります．以上より，最短の操作列を求めるためには，$a_i = x_i^0 \to x_i^1 \to \cdots \to x_i^{(t_i)} = b_i$ をうまく決め，得られるグラフが $b_0 \to a_0$ オイラーグラフであるという条件を満たしつつ，その辺数を最小化すればよいです．

まず，各 i $(1 \leq i \leq N)$ について，$a_i = b_i$ の場合 $a_i = x_i^0 = b_i$ とし，$a_i \neq b_i$ の場合 $a_i = x_i^0 = x_i^1 = b_i$ としてみます．このグラフの辺数は明らかに下界です．よって，このグラフが $b_0 \to a_0$ オイラーグラフならば，この辺数が答えです．このグラフの特徴を観察してみます．$(a_0, a_1, \ldots, a_N)$ と $(b_0, b_1, \ldots, b_N)$ が多重集合として一致していることに注目します．すると，$a_0 = b_0$ の場合，全頂点で (入次数) = (出次数) です．また，$a_0 \neq b_0$ の場合，頂点 b_0 では (入次数) + 1 = (出次数) であり，頂点 a_0 では (入次数) = (出次数) + 1 であり，それ以外の頂点では (入次数) = (出次数) です．よって，$b_0 \to a_0$ オイラーグラフであるための必要十分条件は，始点 b_0，終点 a_0，辺集合がすべて単一の連結成分に属していることです．

下界のグラフが $b_0 \to a_0$ オイラーグラフでない場合を考えます．例えば，$(a_0, a_1, a_2, a_3, a_4) = (15, 1, 2, 4, 8)$, $(b_0, b_1, b_2, b_3, b_4) = (15, 2, 1, 8, 4)$ の場合を考えます．下界のグラフは図 1 です．このグラフは，(始点 / 終点も合わせて) 連結成分が 3 個になっています．そこで，$a_1 = 1 \to 15 \to 2 = b_1$, $a_3 = 4 \to 2 \to 8 = b_3$ と変えてみます．すると，グラフは図 2 へ変わります．このグラフは，始点 b_0，終点 a_0，辺集合がすべて単一の連結成分に属しており，$b_0 \to a_0$ オイラーグラフになっています．このように，下界のグラフの辺数が e，連結成分数が c の場合，答えは $e + c - 1$ となります．

図 1

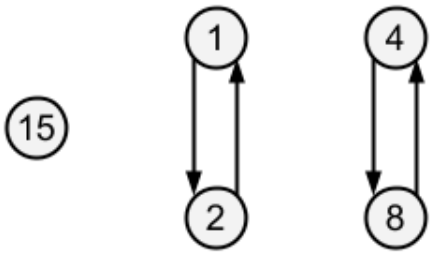
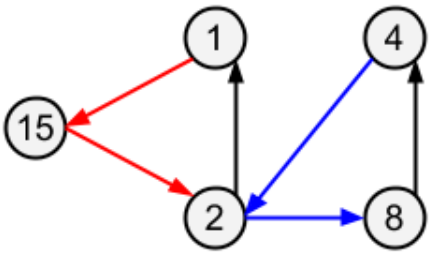


図 2



E: Poor Turkeys

まず，鳥 v が単独で生き残る可能性があるかを判定します．集合 S^i を，男性 1, 2, ..., i が行動を終えた時点で生き残っているべき鳥の集合とします．まず，$S^M = \{v\}$ です．$i = M, M-1, ..., 1$ の順に男性 i の行動を考え，S^i から S^{i-1} を更新していきます．とりあえず，$S^i \supseteq S^{i-1}$ なので，$S^i = S^{i-1}$ とします．$x_i \in S^{i-1}$ かつ $y_i \in S^{i-1}$ の場合，鳥 v は単独で生き残ることはありません．ここで処理を打ち切ります．$x_i \in S^{i-1}$ かつ $y_i \notin S^{i-1}$ の場合，S^i に y_i を追加します．$x_i \notin S^{i-1}$ かつ $y_i \in S^{i-1}$ の場合，S^i に x_i を追加します．$x_i \notin S^{i-1}$ かつ $y_i \notin S^{i-1}$ の場合，何もしません．以上の処理が最後まで終われば，鳥 v は単独で生き残る可能性があると判定できます．具体的には，男性 i は鳥 x_i, y_i のうち，S^i に含まれない方を選んで食べればよいです (以上の処理が最後まで終わったことから，これは常に可能です)．後の処理のため，S^0 を S_v として保存しておきます．

それぞれ単独で生き残る可能性がある鳥 u, v のペアについて，同時に生き残る可能性があるかを判定します．結論から述べると，鳥 u, v のペアは同時に生き残る可能性があるための必要十分条件は，$S_u \cap S_v = \emptyset$ です．もし，$S_u \cap S_v = \emptyset$ ならば，男性 i は鳥 x_i, y_i のうち，S_u^i にも S_v^i にも含まれない方を選んで食べていくことで，鳥 u, v のペアは同時に生き残ります．S_u および S_v の構成により，鳥 x_i, y_i がともに S_u^i にも S_v^i にも含まれることはありえません．逆に，もし $S_u \cap S_v \neq \emptyset$ ならば，S_u および S_v の構成により，鳥 x_i, y_i がともに S_u^i にも S_v^i にも含まれるような男性 i が存在することが示せます．

全体の時間計算量は $O(NM + N^3)$ です．

F : Games on DAG

まず，グラフ G' がひとつ固定されたときに勝敗を判定する方法を考えます．これは各頂点の Grundy 数を計算し，頂点 1, 2 の Grundy 数が等しいか見ることで判定できます．次に，各頂点の Grundy 数がひとつ固定されたときに，それと整合するグラフ G' が何通りあるか数える方法を考えます．今，ある頂点 v を固定します．v を始点とする辺集合の on / off の組合せが何通りか数えます．v の Grundy 数を x とします．頂点集合を Grundy 数ごとに分割し，順に $S_0, S_1, ...$ とします．まず，各 $y = 0, 1, \ldots, x-1$ について，v から S_y への辺集合のうち少なくとも 1 本は on でなければなりません．次に，v から S_x への辺集合はすべて off でなければなりません．最後に，各 $y = x+1, x+2, \ldots$ について，v から S_y への辺集合の on / off は自由です．以上より，v を始点とする辺集合の on / off の組合せが何通りか数えられました．すべての頂点についてこの値の積を計算すれば，G' が何通りあるか数えられます．

Alice が勝つような Grundy 数の組合せを全探索し，各組合せに対して上述の値の総和を計算すれば，答えが求まります．しかし，この方法では TLE になります．そこで，bit DP を用いて高速化します．Grundy 数が 0, 1, ... の順に頂点集合 S_0, S_1, ... を追加していくことを考えます．ただし，頂点 1, 2 は同時に追加されないようにして，Alice が勝つことを保証します．また，頂点集合 T を追加するときに，T 内の頂点を終点とする辺集合の on / off の組合せ数を掛けることにします．既に追加された頂点集合を S とし，今追加しようとしている頂点集合を T とし，$S \cup T$ の補集合を U とします．すると，S から T への辺集合の on / off は自由で，T 内の辺集合はすべて off でなければなりません．また，U から T への辺集合については，各 $v \in U$ に対し，v から T への辺集合のうち少なくとも 1 本は on でなければなりません．以上の遷移において，今の grundy 数は状態として持つ必要がないことに注意してください．

各 $v \in V$, $S \subseteq V$ の組に対して，v から V への辺集合の本数を前計算しておくことで，全体の時間計算量は $\mathrm{O}(N \cdot 3^N)$ となります．

AGC #016 Editorial

writer : sugim48

June 19, 2017

A: Shrinking

Fix a lowercase letter c, and suppose that we want to convert s such that all characters become c.

We simulate the operations. In each operation, we are asked to choose one of t_i and t_{i+1}. If both are c, t'_i must be c. If one of them is c, we should choose c as t'_i. If none of them is c, it doesn't matter which to choose.

This way, we can compute the minimum number of operations for a fixed c. Try all c from `a` to `z`, and the answer is the minimum of those numbers.

B: Colorful Hats

Suppose that there are A colors in total.

We call a cat *alone* if no other cat wears a hat of the same color as this cat. If the cat i is alone, $a_i = A - 1$, and otherwise $a_i = A$. Therefore, the difference between the maximum of a_i and the minimum of a_i must be at most one.

Case I. max = min

There are two cases: all cats are alone or all cats are not alone. In the former case, $a_i = N - 1$, and in the latter case, $2a_i \leq N$. Thus, if one of the above is true, the answer is `Yes`, otherwise the answer is `No`.

Case II. max - min = 1

In this case, A must be the maximum of a_i. Let x be the number of alone cats (the number of $A - 1$), and let y be the number of non-alone cats. The number of colors among alone cats is x, and the number of colors among non-alone cats is between 1 and $y/2$. Thus, if $x+1 \leq A \leq x+y/2$, the answer is `Yes`, otherwise the answer is `No`.

C: Subrectangles

If H is a multiple of h and W is a multiple of w, the answer is `No`. We can divide the entire rectangle into disjoint $h \times w$ subrectangles. The sum in each subrectangle must be negative while the sum of the entire rectangle must be positive, which is obviously impossible.

Otherwise, we can prove that the solution always exists. Without loss of generality, we can assume that H is not a multiple of h.

The following construction works:

- If the row-number (0-based) of a cell is a multiple of h, write a positive number in the cell.
- Otherwise, write a negative number in the cell.

This way, the ratio of positive numbers in an $h \times w$ subrectangle is always $1/h$, and the ratio of positive numbers in the entire rectangle is strictly greater than that. Thus, if we properly choose a positive number and a negative number such that the sum in each subrectangle is only slightly smaller than zero, it satisfies the conditions. For example, we can use $1000(h-1)-1$ for positive cells and -1000 for negative cells.

D: XOR Replace

Let $x = a_1 \oplus a_2 \oplus \cdots \oplus a_N$. If we perform an operation and replace a_i with x, the new XOR of the entire sequence will be the following:

$$\begin{aligned} & a_1 \oplus \cdots \oplus a_{i-1} \oplus x \oplus a_{i+1} \oplus \cdots \oplus a_N \\ = \; & (a_1 \oplus \cdots \oplus a_{i-1} \oplus a_{i+1} \oplus \cdots \oplus a_N) \oplus (a_1 \oplus \cdots \oplus a_N) \\ = \; & a_i \end{aligned}$$

It means that the new XOR of the entire sequence is a_i. Let $a_0 = a_1 \oplus a_2 \oplus \cdots \oplus a_N$. The operation just swaps two elements a_0 and a_i!

Now, the problem can be reduced to the following:

> You are given two sequences $a_0, \ldots, a_N$ and $b_0, \ldots, b_N$. In each operation you can swap the 0-th element and the i-th element for some i. How many operations are required to convert a into b?

In case $a_0, \ldots, a_N$ and $b_0, \ldots, b_N$ are not the same as multisets, it is impossible to do this. Otherwise, this is always possible. As a special case, if the two sequences are the same, the answer is zero. We assume that the two sequences are distinct.

Fix a (not necessarily the shortest) sequence of operations that converts a into b. Let's construct a colored directed graph that (partially) describes this sequence of operations. If we perform an operation between the 0-th element and the i-th element when the 0-th element is x and the i-th element is y,

- Add an edge from vertex x to vertex y of color 0.
- Add an edge from vertex y to vertex x of color i.

If we consider only edges of color i, those edges must form an Eulerian path from vertex a_i to vertex b_i. Since each edge of color 0 corresponds to the reverse of some other edge, edges with nonzero colors must form an Eulerian path from vertex b_0 to vertex a_0. Thus, the graph with only nonzero colored edges will satisfy the following properties:

- The entire graph is an (Eulerian) path from b_0 to a_0.
- This graph can be decomposed into a path from a_1 to b_1, a path from a_2 to b_2, ..., a path from a_N to b_N. Here, if $a_i = b_i$, the i-th path can be empty.

The number of edges in this graph is the number of operations, so we want to find such a graph with minimum possible number of edges.

Now, construct a new graph G. Initially G is empty.

- For each $i (1 \leq i \leq N)$, if $a_i \neq b_i$, add two vertices a_i and b_i to G (in case they are not currently in the graph), and add an edge from a_i to b_i. Note that this graph may contain multiple edges.
- Add two vertices a_0 and b_0 to G, in case they are not added yet (even if $a_0 = b_0$).

It is clear that the number of edges in G gives the lower bound of the number of edges in a desired graph. Furthermore, since we want the graph to be Eulerian, the graph must be connected. In case there are c connected components in G, we need extra $c - 1$ edges to connect them (see the picture below). Therefore, if the number of edges and the connected components in G are e and c, respectively, the answer is $e + c - 1$.

Figure 1:

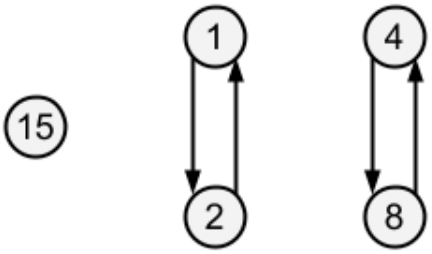
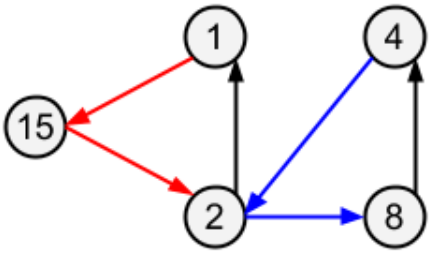


Figure 2:



Strictly speaking, we only gave a lower bound, and we must construct a sequence of length $e + c - 1$ explicitly. It is relatively easy to do this on paper, but it's lengthy to describe the details in words, and we leave details as an exercise for readers. Basically, in each step we can resolve one "mismatch", but for each connected component that doesn't contain a_0 or b_0, we need an extra move in the beginning.

E: Poor Turkeys

First, we generalize the question.

> Let S be a subset of turkeys, and let t be an integer. Is it possible that all turkeys in S survive after t steps?

There are four cases:

- If $x_t \in S$ and $y_t \in S$, we are sure that the answer is "No". Regardless of the state after $t-1$ steps, at least one of x_t and y_t will be eaten after the t-th step.
- If $x_t \in S$ and $y_t \notin S$, all turkeys in S survive after t steps if and only if all turkeys in $S \cup \{y_t\}$ survive after $t-1$ steps.
- If $x_t \notin S$ and $y_t \in S$, all turkeys in S survive after t steps if and only if all turkeys in $S \cup \{x_t\}$ survive after $t-1$ steps.
- If $x_t \notin S$ and $y_t \notin S$, all turkeys in S survive after t steps if and only if all turkeys in S survive after $t-1$ steps.

Let's determine if a single turkey v has a possibility to survive. First, start with a set $S = \{v\}$. In the order $i = M, M-1, ..., 1$, we update the set S, according to the rule we stated above. If we can complete the process without getting "No", the answer is "Yes". Let S_v be the set we get after this process, when we start with $S = \{v\}$. We can compute these sets in $O(NM)$.

Now, let's return to the original problem. How can we check if two turkeys u, v can survive at the same time? If we follow the rules above, we can understand that this is equivalent to the following conditions:

- The turkey u must has a possibility to survive.
- The turkey v must has a possibility to survive.
- $S_u \cap S_v = \emptyset$.

This part requires $O(N)$ time per each pair of turkeys.
The total complexity of this solution is $\mathrm{O}(NM + N^3)$.

F : Games on DAG

When a graph G' is given, we can determine the winner by checking whether the Grundy numbers of two vertices 1 and 2 are the same. In this task, we are asked to count the number of G' where the Grundy number of the two vertices become the same.

$O(Bell(N) \times poly(N))$ solution

We fix a sequence $(g_1, \ldots, g_N)$. How can we count the number of G' such that the Grundy number of the vertex i is g_i?

Let S_i be the set of vertices whose grundy numbers are i. Consider a vertex v, and let $x = g_i$. The Grundy number of v depends on the set of outgoing edges from v, and it must satisfy the fllowing conditions:

- For each $y = 0, 1, \ldots, x-1$, there must be at least one edge from v to S_y.
- There must not be any edges from v to S_x.
- For each $y = x+1, x+2, \ldots$, there is no restriction about the edges from v to S_y.

The number of sets of outgoing edges from v that satisfy the conditions above can be computed easily. By computing this number for all possible sequences $(g_1, \ldots, g_N)$, we can get the answer. This works in $O(Bell(N) \times poly(N))$, where $Bell(N)$ is the N-th Bell Number.

$O(3^N N)$ solution

However, the former solution was (at least was supposed to be) slow, so we want to improve it to an $O(3^N N)$ DP.

Let S be a set of vertices. S contains either both or none of the two vertices 1 and 2. Define $dp[S]$ as the number of ways to choose a subset of edges inside S, such that the Grundy numbers of the vertices 1 and 2 becomes the same.

One possible way is not to add edges at all inside this set. There is one way to do that. In other cases, we divide the set S to two disjoint non-empty sets T and U, such that the Grundy number of each vertex in T is nonzero and the Grundy number of each vertex in U is zero.

- T must contain either both or none of the two vertices 1 and 2.
- We must not add edges inside U.
- The edges from U to T are arbitrary.
- For each vertex in T, there must be at least one edge to U.
- And here is the key observation: the number of ways to choose the set of edges inside T is $dp[T]$!

With proper pre-calculation, this solution works in $\mathrm{O}(N \cdot 3^N)$.

AGC 015 解説

DEGwer

2017/05/27

For International Readers: English editorial starts on page 4.

A: A +...+ B Problem

N 数の最大値は最小値以上なので、$A > B$ のとき、答えは 0 です。

それ以外で $N = 1$ のとき、最大値も最小値も、その 1 つの値に等しくなるので、$A \neq B$ なら答えは 0 で、そうでないなら答えは 1 です。

それ以外のとき、最小値と最大値以外の $N - 2$ 数は、A 以上 B 以下の範囲で自由に選ぶことができます。それらの合計は全部 A にする場合に最小、全部 B にする場合に最大になり、その間の値は全て作ることができるので、$(N-1)A + B$ 以上 $A + (N-1)B$ 以下の $(B-A)(N-2)+1$ 個の値を作ることができ、これが答えです。時間計算量は $O(1)$ です。

B: Evilator

i 階に上向きの矢印ボタンのみがあるとき、i 階より上の階へは 1 回エレベーターに乗れば到着できます。i 階より下の階には、最上階で乗り換えることで 2 回エレベーターに乗れば到着できます。

i 階に下向きの矢印ボタンのみがあるとき、i 階より下の階へは 1 回エレベーターに乗れば到着できます。i 階より上の階には、最下階で乗り換えることで 2 回エレベーターに乗れば到着できます。

ある階より上/下にある階の個数は簡単に求められるので、$O(N)$ 時間でこの問題を解くことができます。

C: Nuske vs Phantom Thnook

黒マス全体は、4 方向に隣接するマスを隣り合うとしたとき、森になっています。グリッドを特定の長方形領域に制限しても、この性質は変わりません。

さて、森の各連結成分である木は、頂点数が辺数より 1 だけ大きいという性質を持っています。すなわち、各クエリに対する答えは、その領域の 頂点数 − 辺数、すなわち黒マスの数から、黒マスが隣接する箇所の数を引いたものになります。

必要な情報は、黒マスの数、黒マスが縦方向に連続する箇所の数、黒マスが横方向に連続する箇所の数のそれぞれについて、2 次元累積和をとっておけば定数時間で求めることができます。時間計算量は $O(NM + Q)$ となります。

D: A or...or B Problem

$A = B$ のとき、答えは 1 です。そうでないとき、A と B の二進表記を上の桁から順に見ていったときに初めて異なるところが、下から r 桁目であるとします。

下 r 桁以外の値がすべて等しい整数たちの or をとっても下 r 桁以外は変わらないので、これ以降下 r 桁以外を無視し、 A も B も r 桁の整数であるとします。特に、A の最上位は 0 で、B の最上位は 1 です。

まず、or をとる操作で数が小さくなることはないので、作れる最小の整数は A です。また、A 以上 2^{r-1} 未満の整数は、すべて作ることができます。また、2^{r-1} 未満の整数たちの or では 2^{r-1} 未満の整数しか作れないため、それ以上の整数を作るためには最上位が 1 の整数を使う必要があります。

最上位が 1 の整数のみを使う場合、B を上の桁から見て、最上位以外で初めて 1 が現れるのが下から k 桁目だとすると、2^{r-1} 以上 $2^{r-1} + 2^k$ 未満の整数をすべて作ることができ、それ以外は作ることはできません。(すべての $1 \leq i \leq k-1$ に対し、$2^{r-1} + 2^i$ が使えるため)

最上位が 1 の整数と最上位が 0 の整数の両方を使う場合、$2^{r-1} + A$ 以上 2^r 未満のすべての整数を作ることができ、それ以外は作ることができません。

以上で作れる整数の区間が列挙できたので、桁数の線形時間でこの問題を解くことができます。2, 3 個目に挙げた区間は重なることがあることに注意してください。

E: Mr.Aoki incubator

まず、高橋君 i を時刻 0 に青木君にしたときに、最終的に青木君になる高橋君がどのような高橋君であるかを考えます時刻 0 での座標が X_i 以下の高橋君の中での速度の最大値を R_i、時刻 0 での座標が X_i 以上の高橋君の中での速度の最小値を L_i とすると、速度が L_i 以上 R_i 以下の高橋君全員が青木君になることが分かります。

以上の考察より、この問題は次のように言い換えることができます。

問題: 区間 $[L_i, R_i]$ が与えられる。これらの区間のうちいくつかを選び、V_i たちすべてを被覆する方法は何通りあるか。

また、L_i も R_i も X_i に対して単調増加であることがわかるので、これらの区間を L_i も R_i も単調増加になるように並べることができます。

適切に座標圧縮を行い、V_i たちを 1 から高橋君の人数までの整数としておきます。 $DP[i][j]$ を、最初の i 個の区間を使い、座標 j までを被覆するような区間の選び方の数として更新すれば、$O(N^2)$ の DP ができます。この DP は、区間が L_i も R_i も単調増加になるように並んでいるという性質を使い、配列を使いまわして累積和を適切に使うことで、$O(N)$ に改善でき、この問題を解くことができます。

F: Kenus the Ancient Greek

以下、$F[0] = 1, F[1] = 1, F[k+2] = F[k+1] + F[k]$ で定まる数列 (フィボナッチ数列) の k 項目を $F[k]$ であらわします。

2 以上の整数 k に対し、次の条件を満たす整数対 (a, b) を「レベル k の良いペア」と呼ぶことにします。

- ユークリッドの互除法の反復回数が k である
- $a, b \leq F[k+2]$

例えば、レベル 3 の良いペアは $(3,5),(4,7),(5,7),(5,3),(7,4),(7,5)$ の 6 つです。

入力で与えられる整数対を X, Y とします。$X \leq Y$ として一般性を失いません。また、$X = 1$ または $(X, Y) = (2, 2)$ のときは反復回数の最大値が 1 で、その最大値をとるペアの個数は XY であるので、それ以外の場合を考えます。簡単な帰納法により、$F[k] \leq X, F[k+1] \leq Y$ なる最大の k が、ユークリッドの互除法の反復回数の最大値となることが分かります。

さて、この最大値をとるペアは、1 ステップ互除法を進めることを考えれば、あるレベル k の良いペア (a, b) と整数 t を用いて $(a, b + ta)$ と書くことができることが分かります。よって、レベル k の良いペアをすべて列挙して、各ペアについて先の条件を満たす t がいくつあるかを割り算で求めることで、この問題を解くことができます。

さて、レベル k の良いペアはいくつあるでしょうか?実は、ちょうど $2k$ 個あることが証明でき、また、下記の証明から自然に導かれるアルゴリズムを用いて (すべての k に対して合計で) $O(k^2)$ 時間で列挙することが可能です。N 以下のフィボナッチ数の個数は $O(logN)$ 個なので、良いペアを最初にすべて列挙しておけば、$N \leq 10^{18}$(入力の整数の上限) として $O(log^2 N + QlogN)$ でこの問題を解くことができます。

以下にレベル k の良いペアがちょうど $2k$ 個あることの証明の概要を示します。フィボナッチ数の一般項の式 (が近似的に指数関数であらわせること) を用いて適切に不等式評価を行うことで、各良いペアに対し、互除法を走らせたときに商が 2 以上となるような反復は高々 1 回で、さらにそのときの商は 2 であることが分かります。よって、$a < b$ なるレベル k の良いペア (a, b) は、いつ商を 2 にするか (あるいは一度も商を 2 にしないか) で表され、ちょうど k 個です (最初の反復の商は 1 であることに注意してください)。

また、同じく帰納法を用いて、レベル k の良いペア (a, b) のうち $a < b$ であるものは全て $F[k] \leq a \leq F[k+1], F[k+1] \leq b \leq F[k+2]$ を満たすことなども分かり、実装の補助となります。

AGC 015 Editorial

DEGwer

2017/05/27

For International Readers: English editorial starts on page 7.

A: A +...+ B Problem

- If $N = 1$, the only number must be the maximum and the minimum. If $A = B$, the answer is 1, otherwise the answer is 0.
- If $N > 1$,
  - If $A > B$, the answer is obviously 0.
  - Otherwise, the maximum must be B, the minimum must be A, and you can arbitrarily choose the remaining $N - 2$ numbers between A and B. The sum of those $N - 2$ numbers is between $(N - 2)A$ and $(N - 2)B$, so the answer is $(B - A)(B - 2) + 1$.

B: Evilator

- If the direction on the i-th floor is 'U', for each $j > i$, you can reach the j-th floor in one step. For each $j < i$, you can reach the j-th floor in two steps: first go to the topmost floor and transfer there.
- If the direction on the i-th floor is 'D', for each $j < i$, you can reach the j-th floor in one step. For each $j > i$, you can reach the j-th floor in two steps: first go to the bottommost floor and transfer there.

Thus, for each i, you can compute the sum of distances from i to j in constant time. Overall this solution works in $O(N)$.

C: Nuske vs Phantom Thnook

Let's construct a graph. The vertices correspond to the blue cells, and when two blue cells are adjacent, we add an edge between them. From the constraints, this graph must be a forest. Even if we construct graphs for subrectangles, this property still holds.

It is well-known that in a tree, the number of vertices minus the number of edges is always 1. When a forest has k connected components, each connected component is a tree and the number of vertices minus the number of edges in the forest is k. Thus, we can compute the number of connected components in a forest as the number of vertices minus the number of edges.

Therefore, for each query, we can compute the answer as follows:

(The number of blue cells in the rectangle)
- (The number of horizontally adjacent pairs of two blue cells in the rectangle)
- (The number of vertically adjacent pairs of two blue cells in the rectangle)

Each term in this formula can be computed in $O(1)$ if we pre-compute 2-dimensional sums. Overall it works in $O(NM + Q)$.

D: A or...or B Problem

If $A = B$, the answer is obviously 1. We assume that $A < B$.

Let r be the largest integer such that the r-th (0-based) bit of A and the r-th bit of B are different. By the definition of r, the $(r+1)$-th bit or higher bits are the same for all integers between A and B. Thus, we can ignore them, and we can assume that $0 \leq A < 2^r$ and $2^r \leq B < 2^{r+1}$.

Let $T = 2^r$, $X = \{A, A+1, \ldots, T-1\}$, and $Y = \{T, T+1, \ldots, B\}$.

- If we use only integers from the set X, we can construct integers between A and $T-1$, inclusive (choose one element). We can only construct these numbers (The minimum possible OR is A, and the OR of all elements is $T-1$).
- If we use only integers from the set Y, we can construct all integers between T and the OR of all integers in Y. Let k be the largest integer such that $2^r + 2^k$ is in the set Y. Since all of $2^r, 2^r + 2^0, \ldots, 2^r + 2^k$ are in the set Y, we can construct all integers between 2^r and $2^r + 2^{k+1} - 1$. It's easy to see that the OR of all elements in Y is $2^r + 2^{k+1} - 1$.
- If we use both integers from X and integers from Y, we can construct integers between $T + A$ and $2T - 1$, inclusive (choose T and one element from X). We can only construct these numbers (The minimum possible OR is $T + A$, and the OR of all elements is $2T - 1$).

The set of possible integers is the union of the three intervals above. Note that the second and the third intervals may overlap.

E: Mr.Aoki incubator

We call Takahashi "red point", and call Aoki "black point".

First, consider the case where we color only one point red at time 0. Let i be the index of the red point.

Suppose that $X_j < X_i$ and $V_j > V_i$. In this case, point j passes through point i at some time, so point j becomes red. Suppose that point k lies between i and j after a sufficiently long time (i.e., $V_i < V_k < V_j$. It is easy to see that point k also becomes red.

Without loss of generality, we assume that the points are sorted in the increasing order of V. For each i, we define R_i as the maximum j such that $X_j \leq X_i$. From the observation above, all points between i and R_i will be red. Similarly, we define L_i as the minimum j such that $X_j \geq X_i$. When we only choose point i at time 0, the set of red points after a sufficiently long points will be the points between L_i and R_i, inclusive.

We can compute the arrays L and R in linear time. These arrays have a special property: both L and R are monotonously increasing.

Therefore, we can restate the problem as follows:

You are given N intervals $[L_1, R_1], \ldots, [L_N, R_N]$, such that $L_1 \leq L_2 \leq \ldots$ and $R_1 \leq R_2 \leq \ldots$. Each interval is a subinterval of I. In how many ways can you choose a subset of those N intervals such that the union of chosen intervals becomes I?

Construct a DAG with $N + 2$ vertices. The vertices are numbered 1 through N, and there are two special vertices S and T.

- When L_i is the leftmost element of I, we add an edge from S to i.
- When R_i is the rightmost element of I, we add an edge from i to T.
- When $i < j$ and $R_i \geq L_j - 1$, we add an edge from i to j.

The answer is the number of paths from S to T in this DAG. It can be computed in $O(N)$ DP and prefix sums.

F: Kenus the Ancient Greek

We denote the Euclidean step of a pair (a, b) as $f(a, b)$.

When are we interested in a pair (a, b)? If there exists a pair (a', b') such that $a' \leq a, b' \leq b, f(a', b') > f(a, b)$, the pair (a, b) never become the pair with the maximum steps for any query. We call a pair (a, b) *good*, if such (a', b') doesn't exist.

An example of good pair is Fibonacci numbers. Let $F_0 = 1, F_1 = 1$ and $F_i = F_{i-1} + F_{i-2}$ for bigger i. We can see that $f(F_k, F_{k+1}) = k$ for positive k. Also, it's easy to see that this is the "minimum" pair with k Euclidean steps: that is, if $f(x, y) = k$ and $x < y$, $x \geq F_k$ and $y \geq F_{k+1}$ hold.

Ideally we want to generate all good pairs, but it turns out there are too many good pairs. For example, $(2, 2017)$ is a good pair. Most of good pairs are very "thin", and the situation becomes better after one Euclidean step. We can prove that after one step, a good pair becomes an *excellent pair*, defined as follows:

A pair of integers (a, b) is called an *excellent pair of level k* if:

- $f(a, b) = k$
- $a, b \leq F_{k+2}$

For example, there are six excellent pairs of level 3: $(3, 5), (4, 7), (5, 7), (5, 3), (7, 4), (7, 5)$.

Here is a proof: let $(x, px + y)$ be a good pair and $f(x, px + y) = k + 1$. After one step, it becomes (y, x), and $f(y, x) = k$. If this is not an excellent pair, $x > F_{k+2}$. However, we know that $f(F_{k+2}, F_{k+3}) = k + 2$, and since $F_{k+2} < x$ and $F_{k+3} = F_{k+2} + F_{k+1} < px + y$, this contradicts that the pair $(x, px + y)$ is good. Here we used $y \geq F_k$ - this comes from $f(y, x) = k$ and the fact that Fibonacci numbers are the optimal. Thus, we prove that after one step, a good pair becomes an excellent pair.

Now, the only remaining thing to do is to generate all excellent pairs. It turns out that there are exactly $2k$ excellent pairs of level k for each k, but we don't need this fact to solve the problem. Since excellent pair is also an excellent pair, after one step excellent pair becomes an excellent pair, and we can generate the list of all excellent pairs of level $k + 1$ from the list of all excellent pairs of level k. This way, all excellent pairs can be generated in $O(log^2 MAX)$ time.

To handle a query (X, Y), we first compute the maximum Euclidean steps (call it k). Then, we iterate through all $2(k - 1)$ excellent pairs of level $k - 1$, and count the number of good pairs within the rectangle (X, Y) such that after one step, it becomes the desired excellent pair. Be careful with the special case where $k = 1$.

This solution works in $O(log^2 MAX)$ pre-computation and $O(logMAX)$ per query.

AGC 14 解説

yutaka1999

2017 年 5 月 6 日

For International Readers: English editorial starts at page 5.

A : Cookie Exchanges

操作を行える場合、各人の持っているクッキーの個数の集合を $\{A, B, C\}$ とすると、操作後の集合は $\{\frac{A+B}{2}, \frac{B+C}{2}, \frac{C+A}{2}\}$ になる。よって、$A \leq B \leq C$ とすると持っているクッキーの個数の最大と最小の差が操作前は $C - A$、操作後は $\frac{C-A}{2}$ となり、操作を行うと、最大と最小の差がちょうど $\frac{1}{2}$ になると分かる。よって、$A = B = C$ ではない場合は、$M = 10^9$ として、操作を高々 $\log_2 M$ 回しか行わないことになり、愚直にシュミレーションすることで解くことができる。逆に、$A = B = C$ の場合は、どれも偶数のときは無限に操作を行うことができ、どれも奇数の時は一回も操作を行うことができないので、この場合も簡単に解くことができる。以上より、$O(\log M)$ でこの問題を解くことができた。

B : Unplanned Queries

どの頂点においてもクエリの a_i, b_i として現れる回数が偶数回であることが必要十分であることを示す。つまり、全ての木に対して、その条件を満たすとき必ずすべての辺の数が偶数になり、そうでないときある辺の数が奇数になることを示す。

適当な頂点を根として、根付き木にする。この根を r とする。まず、クエリ (a, b) を考えたとき、これを $(r, a), (r, b)$ と分解することができる。これは、(a, b) の LCA を p としたとき、クエリ (a, b) ではパス $a - p, b - p$ に $+1$ しており、クエリ $(r, a), (r, b)$ ではパス $a - p, b - p$ に $+1$、パス $r - p$ に $+2$ するため、mod 2 で考えると同一視できるので明らかである。

よって、クエリ全体は (r, v) を $f(v)$ 回行う、という形に変形でき、mod 2 で考えているため、$f(v) = 0, 1$ としてよく、今示したいことは $f(v) = 0$ が題意が成立するための必要十分条件であることである。これは $f(v) = 1$ なる v で深さが一番深いものを考えると、v の直接の親を p としたとき、$v - p$ に書かれた数が奇数になるため、明らかに成立する。

よって、クエリの a_i, b_i として現れる回数が偶数回であるかどうかを判定すればこの問題を解くことができ、$O(N + M)$ で解くことができる。

C : Closed Rooms

まず、高橋君が今の場所から K 回以内でたどり着ける場所を列挙する。そして、その各場所から以下の操作を行って何回で目標地点にたどり着けられるか求められれば良い。

- まず、閉じられている部屋を K 個まで選び、それらを開いた状態にする。

- その後、K 回まで移動する。

ここで、この操作は以下の操作と同値になることを示す。

- 閉じられている閉じられていないに関わらず、隣接するマスに移動することを K 回まで繰り返す。

先の操作で移動できるならばこの操作で移動できることは明らかなので、その逆を示せばよい。これは、後者の操作で移動するときに通る、閉じられている部屋を開くことで、先の操作でも移動できるから明らか。

よって、後者の操作を行うとしてよく、これは BFS をしたり、1 行目、1 列目、H 行目、W 列目までの距離を計算することで解くことができる。

以上より、$O(HW)$ でこの問題を解くことができる。

D : Black and White Tree

与えられた木に完全マッチングが存在することと後手勝ちであることが同値であることを示す。帰納法でこれを示す。頂点数が $1, 2$ の木の場合は明らかなので、頂点数が $n-2$ のときに成立するならば頂点数 n の時も成立することを示せればよい。

完全マッチングが存在するならば、後手は先手が前に塗った頂点に対応する頂点を塗ることで、どの白の頂点も黒の頂点に隣接するようにできるため、明らかに後手勝ちである。よって、完全マッチングが存在しないときに先手勝ちを示せればよい。

まず、木を適当な頂点を根として根付き木にする。このとき、深さ最大の葉の親は子がすべて葉なので、特に子がすべて葉であるような頂点 v が存在する。ここで、v の子が 2 つ以上あるとき、先手は v を白に塗り、次の手番で先手は v の子の内まだ塗られていない頂点を白に塗れば、最終的にその葉は黒に隣接しないので、先手勝ちとなる。よって、v の子が 1 つしかない場合を考えればよい。その子を l とし、今の木から v, l を取り除いて得られる木を T とする。

このとき、T に完全マッチングが存在するならば、そのマッチングに $v-l$ の組を加えることで、今の木に完全マッチングが存在することになり、これは矛盾。よって、T には完全マッチングは存在せず、特に、T だけでゲームを行った場合、帰納法の仮定より先手勝ちである。

よって、まず先手は v を白に塗り、後手が l を黒に塗らなかったら次の手番で l を白に塗り、後手が l を黒に塗ったら、残りは T だけのゲームと同一視できるため、先手勝ちとなるから、結局このゲームは先手勝ちとなる。

以上より、頂点数が n の時も、与えられた木に完全マッチングが存在することと後手勝ちであることが同値であることが示されたので、帰納法よりこれは成立。

よって、後は与えられた木に完全マッチングがあるかどうかの判定ができればよい。これは先と同様に適当な頂点で根付き木にし、葉から貪欲にマッチングを組んでいくことで判定できる。よって、この問題は $O(N)$ で解くことができる。

E : Blue and Red Tree

まず、青から赤へ木を作り替えられると仮定して、青い辺を取り除かれる順に $b_1, b_2, ..., b_{N-1}$、赤い辺を追加される順に $r_1, r_2, ..., r_{N-1}$ とする。

このとき、任意の $i \in [1, N-1]$ に対して $b_i, ..., b_{N-1}$ からなる森の連結成分集合を $f(i)$ とすると、操作の性質より、r_i, b_i が繋いでいる $f(i+1)$ の 2 つの連結成分は一致する。

よって、$f(N) = \{\{1\}, \{2\}, ..., \{N\}\}$ であることを考慮すると、青から赤へ木を作り替えられるならば、はじめ連結成分集合が $\{\{1\}, \{2\}, ..., \{N\}\}$ という状態から、

異なる 2 連結成分を繋ぐ青い辺と赤い辺を選んで、その 2 つを 1 つの連結成分にまとめる

という操作を繰り返して、連結成分を 1 つにまとめることができる。

逆に、この操作を行うことができるとき、青から赤へ木を作り替えることができることを示す。先の操作で明

らかに同じ辺を 2 回以上使うことがないので、青い辺と赤い辺を使う順に $b_1, b_2, ..., b_{N-1}, r_1, r_2, ..., r_{N-1}$ とすると、$(b_{N-1}, r_{N-1}), (b_{N-2}, r_{N-2}), ..., (b_1, r_1)$ の順で元の問題の操作を行えば、明らかに青から赤へ木を作り替えられる。

よって、異なる 2 連結成分を繋ぐ青い辺と赤い辺を選んで、その 2 つを 1 つの連結成分にまとめる、という操作を繰り返して、連結成分を 1 つにまとめられるかどうかを判定できればよい。言い換えると、同じ 2 頂点間を結んでいる青い辺と赤い辺を選んで、その 2 頂点を縮約することを繰り返して、1 つの頂点にまとめられるかどうかを判定できればよい。同じ 2 頂点間を結んでいるような青い辺と赤い辺があれば貪欲に縮約してよいので、縮約を高速に行い、そのような辺の組を高速に発見できればよい。これは、縮約時に残っている辺の数が少ない方をマージし、マージした際に青い辺と赤い辺が同じ頂点を指していれば、その辺を消して縮約する 2 点として追加することで、全体として $O(N \log^2 N)$ で解くことができる。

よって、この問題を $O(N \log^2 N)$ で解くことができた。これは十分高速である。

F : Strange Sorting

まず、順列の内 i から N の値の部分の動きには 1 から $i-1$ の値の部分が影響しないことに注意し、i から N の部分だけ見たときに、これらがソートされる回数を $T(i)$ で表すことにする。まず、$T(i) = T(i+1)$ or $T(i+1)+1$ であることを示す。

これは、$T(i+1)$ 回目の操作後にはじめて $i+1$ から N の値の部分がソートされ、その時に i が $i+1$ より左にあればその時点で i から N の部分がソートされており、そうでないならもう一回操作を行うことで、i から N の部分がソートされるので、確かに成立する。よって、$T(i)$ が $T(i+1)$ と $T(i+1)+1$ のどちらになるか高速に判定できればよい。

ここで、$T(i+1) = 0$ の時は i が元の順列で $i+1$ より左にあるかどうかだけで判定できるのでよい。よって、$T(i+1) > 0$ の時のみ考えればよい。ここで、$T(i) > 0$ のとき、$T(i)$ 回目の操作を行う時点で i から N の中で先頭にある数を $f(i)$ で表すことにする。このとき、$T(i)$ 回目の操作ではじめて i から N がソートされることより、$f(i) \neq i$ である。ここで、$T(i)$ が $T(i+1)$ と $T(i+1)+1$ のどちらであるかは、元の順列で $i, i+1, f(i+1)$ がどの順番で並んでいるかだけに依存することを示す。

まず、$T(i+1)$ 回目の操作を行う時点での $i, i+1, f(i+1)$ の順番について考えると、この時点で $f(i+1), i, i+1$ の順に並んで入れば $T(i) = T(i+1), f(i) = f(i+1)$ であり、$f(i+1), i+1, i$ もしくは $i, f(i+1), i+1$ の順で並んでいれば $T(i) = T(i+1)+1, f(i) = i+1$ である。(ここで、$f(i+1), i+1$ はこの順に並ぶことに注意する。) よって、$T(i+1)$ 回目の操作時点でのこの 3 つの値の相対順序が元の順列でのこれらの相対順序と一致することを示せれば、判定を $O(1)$ で行うことができ、$f(i)$ についても求めることができる。以下ではこの事実を示す。(ただし、ここでいう相対順序では $(a,b,c), (b,c,a), (c,a,b)$ という 3 つの順序を同一視し、つまり、順列を円環状に並べたときの相対順序を指していることに注意する。)

特に、1 回目から $T(i+1)-1$ 回目までの操作のいずれにおいても、これらの相対順序が変化しないことを示せば十分。そのためにまず、$f(i+1)$ はこれらの操作において、$i+1$ から N だけを見たときに、先頭以外で高い項とならないことを示す。

$i+1$ から N だけを見たときに先頭以外で高い項になったとすると、その範囲だけみたとき、その操作後に $f(i+1)$ の一項前が $f(i+1)$ より小さくなる。このとき、$f(i+1)$ が再び $i+1$ から N だけを見たときに先頭に来ることがないことが示せる。なぜならば、$f(i+1)$ が先頭に来る前に必ず $f(i+1)$ の一項前の数が高い項として選ばれるが、このときその数が $f(i+1)$ よりも小さいため、$f(i+1)$ も高い項として選ばれ、移動してしまう。よって、ひとたび $f(i+1)$ が先頭以外で高い項となれば、次も必ず先頭以外で高い項となってしまうため、先頭として現れることはなくなる。しかし、$f(i+1)$ の定義より、$T(i+1)$ 回目の操作で $i+1$ から N だけを見たときに先頭となるため、これは矛盾。よって、確かに 1 回目から $T(i+1)-1$ 回目までの操作において、$f(i+1)$ は先頭以外で高い項にはならない。

よって、これらの操作の内一つを固定して、その操作での i から N の部分での先頭を k とすると、$k = f(i+1)$ のときは $k > i, i+1$ より明らかに $i, i+1, f(i+1)$ の相対順序は変化せず、$k \neq i, f(i+1)$ の時も、先の性

質より、$i, i+1, f(i+1)$ の相対順序は変化しない。よって、$k=i$ の時だけ考えればよいが、このとき i の次の数を l とすると、$l=i+1, f(i+1)$ のときは明らかに変化せず、$l \neq i+1, f(i+1)$ の時は、$l > i+1$ 及び先の性質より、この場合も相対順序は変化しない。

よって、いずれの場合も変化しないことが示されたので、$T(i+1)$ 回目の操作時点での $i, i+1, f(i+1)$ の相対順序と元の順列での相対順序は変化しない。よって、$T(i), f(i)$ を $T(i+1), f(i+1)$ を用いて $O(1)$ で求めることができるので、特に、$T(1)$ を求めることができ、これが求める答えとなる。

よって、この問題は $O(N)$ で解くことができる。

Atcoder Grand Contest 014 Editorial

writer : yutaka1999

2017 年 5 月 6 日

A : Cookie Exchanges

We prove that the operation stops in at most $O(\log M)$ steps (where $M = \max A, B, C$), except for the special case where $A = B = C$.

Suppose that the three people have A, B, C cookies, respectively. Without loss of generality, we can assume that $A \leq B \leq C$, and the difference between the maximum and the minimum is $C - A$. If an operation can be performed, after an operation, the number of cookies will be $\{\frac{A+B}{2}, \frac{B+C}{2}, \frac{C+A}{2}\}$, respectively. Here, the difference between the maximum and the minimum is $(C - A)/2$. Thus, the operation finishes in $O(\log M)$ unless $A = B = C$.

In case $A = B = C$, the answer is -1 if they are even, and 0 if they are odd. Otherwise, brute force works as described above. This solution works in $O(\log M)$.

B : Unplanned Queries

The answer is "YES" if and only if each vertex appears even number of times in the queries.

In fact, the shape of the tree doesn't matter. Choose an arbitrary vertex as a root, and call it r. If we only consider the parities, the following two things are equivalent:

- Add 1 to the path between a and b.
- Add 1 to the path between r and a, and then add 1 to the path between r and b.

Let p be the loweast common anscestor of a and b. Then, the second operation adds 1 to the path between a and b, and 2 to the path between r and p, so they are equivalent in modulo 2.

Let $f(v)$ be the number of appearances of vertex v in the queries. As we see above, the queries are equivalent to adding $f(v)$ to the path between r and v, for each v.

If $f(v)$ is even for all v, obviously any tree satisfies the condition. If $f(v)$ is odd for some v, consider one of the deepest such vertex. In this case, the edge that connects v and its parent will be labeled with an odd number, so the condition won't be satisfied.

Therefore, we only need to check whether each vertex appears even number of times, and the problem can be solved in $O(N + M)$.

C : Closed Rooms

In the statement, the following operations will be repeated X times, for some X:

- Move to an adjacent (unlocked) room at most K times.
- Then, select and unlock at most K locked rooms.

This is equivalent to the following:

- Move to an adjacent (unlocked) room at most K times.
- Then, repeat the following $X-1$ times: "select and unlock at most K locked rooms, and move to an adjacent (unlocked) room at most K times".

Note that the final unlocking of at most K rooms is irrelevant for reaching the goal.
Furthermore, this is equivalent to the following:

- Move to an adjacent (unlocked) room at most K times.
- Then, move to an adjacent room, this time not necessarily unlocked, at most $(X-1)K$ times.

Now the problem becomes easy. First, compute the set of rooms that can be reached within the distance of K from the initial position, using BFS. From each such room, compute the minimum number of operations of the following type required to reach one of the rooms with an exit:

- Move to an adjacent room, this time not necessarily unlocked, at most K times.

This can be done by a simple division. From (x, y), the minimum number of such operations is $ceil(\min\{x-1, H-x, y-1, W-y\}/X)$.
This solution works in $O(HW)$.

D : Black and White Tree

We prove that the second player wins if and only if there exists a perfect matching.

When a perfect matching exists, let $(a_1, b_1), \ldots, (a_k, b_k)$ be the edges of perfect matching. The second player uses the following strategy: when the first player paints a_i for some i, he paints b_i, and when the first player paints b_i for some i, he paints a_i. This way, exactly one of (a_i, b_i) will be painted black. Thus, after all vertices are colored, each white vertex is adjacent to at least one black vertex, so the second player wins.

Otherwise, the first player follows the following strategy. Consider the tree as a rooted tree. At each his turn, he chooses one of the deepest uncolored vertices, and call it x. Unless x is the only uncolored vertex, x is adjacent to exactly one uncolored vertex. Let's call it y. Then the first player's strategy is to color y. Then, in the next turn, the second player is forced to color x (otherwise the first player colors x in the next turn and he wins).

There are two cases:

- If y has more than one (ucolored) children, the first player wins. For example, suppose that y has two children x and z. Since x is one of the deepest node, both x and z are leaves, and the second player is forced to color them in the next turn. However, he can color at most one of them, and the second player loses.
- If x is the only child of y, after the first player colors y and the second player colors x, we can completely forget about these two vertices and continue the game as if these two vertices don't exist.

By repeating this process, we will either construct a perfect matching or the first player wins. Thus, when a perfect matching doesn't exist, the first player wins.

It's easy to check whether a tree has a perfect matching. We just repeat choosing edges greedily from leaves. This solution works in $O(N)$.

E : Blue and Red Tree

Consider a valid sequence of operations that converts the blue tree to the red tree. Suppose that we cut an edge e in the first operation, and let X and Y be the two connected components of blue edges we get after the removal of edge e. (X and Y are sets of vertices). Then we add a red edge. Let's call it f. Obviously, f must connect one vertex from X and one vertex from Y. After that, whenever we remove a blue edge in X, we must add a red edge that connects two vertices in X, and whenever we remove a blue edge in Y, we must add a red edge that connects two vertices in Y. Thus, only one red edge (that is, f) connects between X and Y.

By similar observation, we get the following. Let $b_1, b_2, ..., b_{N-1}$ be the blue edges in the order of removal, and let $r_1, r_2, ..., r_{N-1}$ be the red edges in the order of addition. Let B_i be the set of connected components obtained by the blue edges $b_i, ..., b_{N-1}$, and define R_i similarly for red edges. Then, for each i, B_i and R_i are the same.

Therefore, when there exists a valid sequence of operations, we can do the following:

- Initially, there is a graph with N vertices and no edges.
- Then, repeat this $N-1$ times: choose a blue edge and a red edge that connect the same pair of connected components, and connect them.

On the other hand, in case we can do this, the answer is "YES". Let $p_1, p_2, \cdots, p_{N-1}$ and $q_1, q_2, \cdots, q_{N-1}$ be blue and red edges used in this process (in the order they are used), respectively. Then, in the original problem, we can perform the opertaions in the order $(p_{N-1}, q_{N-1}), (p_{N-2}, q_{N-2}), \cdots, (p_1, q_1)$.

In summary, we want to the following. We are given a graph with N vertices and $2(N-1)$ edges (we don't need to distinguish blue and red edges). Whenever we find a pair of edges that connects the same pair of vertices, contract those two vertices. Can we contract the entire graph into a single vertex?

Here is one possible implementation. We keep the following information:

- The queue of all pairs of vertices that have at least two edges between them.
- For each pair of vertices, the number of edges between them (for example, map in C++).
- For each vertex, the set of all edges incident to it (for example, multiset in C++).

While the queue is non-empty, pop one element, and call it (x, y). Suppose that $deg(x) \geq deg(y)$. Then, we remove the two edges between x and y, and then for each edge that connects y and some other vertex z, delete it and add an edge connecting x and z. Of course, the information we mentioned above should be kept properly. We repeat this, and if we can perform $N-1$ contractions before the queue becomes empty, the operation is successful.

It is well-known that this way each edge is moved at most $O(\log N)$ times, so this solution works in $O(N \log^2 N)$.

F : Strange Sorting

First, notice that the existence of the element with value 1 doesn't affect the type (high or low) of the other elements. Even if we remove 1 from the original sequence and perform the operations, the behavior is very similar.

Let T be the number of operations required to sort the sequence, when we ignore 1. What happens if we perform T operations on the original sequence (including 1)? The integers $2, 3, \ldots, N$ will appear in the sequence in this order, and 1 will be inserted somewhere. If the integer 1 becomes the first element after T operations, the answer is T. In other cases, we need one more operation to sort the original sequence and the answer is $T + 1$.

How can we check if the answer is T or $T + 1$? The case with $T = 0$ is very easy. Assume that $T > 0$, and consider the state of the sequence after $T - 1$ operations. Let f be the integer that appears first in the sequence among the integers $2, 3, \ldots, N$, after $T - 1$ operations. By the definition of T, we can prove that $f > 2$. (Otherwise the integers $2, 3, \ldots, N$ are sorted in $T - 1$ operations or not sorted in T operations). We can see that after $T - 1$ operations, if 1 appears between f and 2, the answer is T, and otherwise the answer is $T + 1$.

Now, we claim that the "cyclic order" of the integers $1, 2, f$ never changes in the first $T - 1$ operations. Here, in "cyclic order", we assume that the orders $(a, b, c), (b, c, a), (c, a, b)$ are identical. Thus, we can compute the answer by computing the values of T and f, and checking the cyclic order of the integers $1, 2, f$ in the initial sequence. In order to compute T, we can repeat the similar process recursively. The detail will be described later.

Let's prove the claim. First, we prove that f never become high element in the first $T - 1$ operations unless it comes first among the integers $2, 3, \ldots, N$. Suppose that at some point, an integer x is not the first element and x is a high element. Since the first element is always a high element, after one operation x comes directly to the right of another integer y, such that $y < x$. Then, we repeat more operations on this sequence. While x and y are of the same type, these two integers always move together, and x is always directly to the right of y. They can be separated only when x becomes high and y becomes low. However, in this case, x is again a high element at not the first position, so these process will be repeated. Therefore, x never comes first. In particular, when f is not the first among the integers $2, 3, \ldots, N$, f is always a low element.

Now, let's return to the proof. Consider the sequence at some point. Let's verify that the cyclic order of $1, 2, f$ doesn't change in the next opertaion.

- If 1 is the first element,
  - If 2 is the second element, 1 and 2 are high and f is low. The cyclic order doesn't change.
  - If f is the second element, 1 and f are high and 2 is low. The cyclic order doesn't change.
  - Otherwise, 1 is high and 2 and f are low. The cyclic order doesn't change.
- If 2 is the first element, 2 is high and 1 and f are low. The cyclic order doesn't change.
- If f is the first element, f is high and 1 and 2 are low. The cyclic order doesn't change.
- Otherwise, all of $1, 2, f$ are low. The cyclic order doesn't change.

Here is the summary of the solution. Let T_i be the number of operations required to sort the sequence, when we only consider the integers $i, i+1, \cdots, N$. Let f_i be the integer that comes the first after $T_i - 1$ operations, when we only consider the integers $i, i+1, \cdots, N$. (In case $T_i = 0$, f_i is undefined).

First, we compute q_i, the position of i in the initial sequence (i.e., $p_{q_i} = i$). Then, we compute the values T_i, f_i in the order $i = N, N-1, \cdots, 1$, and the answer is T_1. When $i < N$, the values can be computed as follows:

- If $T_{i+1} = 0$,
  - If $q_i > q_{i+1}$, $T_i = 1$ and $f_i = i + 1$.
  - Otherwise, $T_i = 0$ and f_i is undefined.
- Otherwise,
  - If $q_{f_{i+1}}, q_i, q_{i+1}$ are in this cyclic order, $T_i = T_{i+1}$ and $f_i = f_{i+1}$.
  - Otherwise, $T_i = T_{i+1} + 1$ and $f_i = i + 1$.

This way, the problem can be solved in $O(N)$ time.

Atcoder Grand Contest 013 解説

writer : maroonrk

2017 年 4 月 17 日

For International Readers: English editorial starts from page 5.

A : Sorted Arrays

A_1 と A_2 の間で数列が切られているような最適な分割があったとします。この時、A_2 と A_3 の間を切って、A_1 と A_2 をくっつければ、これもまた最適な分割となります。よって、A_1 と A_2 の間で数列が切られていないと考えても最適解は変わりません。次に、A_2 と A_3 の間について考えます。A_1,A_2,A_3 という数列が単調非減少でも単調非増加でもない場合、A_2 と A_3 の間は必ず切られています。A_1,A_2,A_3 という数列が単調非減少または単調非増加である場合は、A_1 と A_2 の時と同じ議論を用いると、A_2 と A_3 の間を切らない最適な分割があると分かります。続く要素についてもこのように考えていくと、結局、数列の先頭から条件を満たすできるだけ長い列を切り出す、という操作を続けることが最適であると分かります。この操作は $O(N)$ で行えるので、この問題を解くことが出来ました。

B : Hamiltonish Path

まず最初に、どんなに短くても良いので適当なパスを作ります。次に、そのパスが条件を満たすかどうか調べて、もし条件を満たすならば、そのまま出力します。条件を満たさない場合、パスの端点と直接結ばれているのに、パスに含まれていないような頂点があることになります。そこで、その頂点を新しいパスの端点として追加し、パスを伸ばします。パスを伸ばしたら、再び条件を満たすか確認し、条件を満たさないならば、更にパスを伸ばして・・・という風に繰り返して行きます。パスを伸ばせる回数は高々 $N-1$ 回しかないので、このアルゴリズムは条件を満たすパスをいつかは出力します。計算量は、各頂点について、パスの端点として追加される回数が高々 1 回であることと、各辺について、パスの端点とパス内を結んでいると分かること、パスの端点とパス外を結んでいると分かること、がそれぞれ 1 回ずつであることから、このアルゴリズムは $O(N+M)$ で動き、この問題を解くことが出来ました。

C : Ants on a Circle

問題の設定を次のように言い換えます。

N 匹の蟻がいて、それぞれの蟻は 1 から N まで番号の書かれたゼッケンをつけている。二つの蟻が同じ場所についた瞬間、その二つの蟻はゼッケンを交換し、そのまますれ違って同じ方向に歩き続ける。T 秒後のゼッケンの番号と座標の対応を答えよ。

どの瞬間においても、1 のゼッケン、2 のゼッケン、…N のゼッケンは、この順に時計回りに分布することになります。すると、二つの蟻がゼッケンを交換する時、時計回りに歩いてる蟻のゼッケンの番号は 1 増え、反時計回りに歩いている蟻のゼッケンの番号は 1 減ります。ある二つの蟻が T 秒間にすれ違う回数は、$O(1)$ で求めることができます。よって、最初に 1 のゼッケンをつけていた蟻が、T 秒間に他の蟻とすれ違う回数の合計は、$O(N)$ で分かります。他の蟻とすれ違う回数が分かるので、最初に 1 のゼッケンをつけていた蟻が T 秒後に付けているゼッケンの番号も分かります。これで、T 秒後にどの番号のゼッケンがどの座標にあるかが一つ分かります。ところで、T 秒後にゼッケンのある座標の集合は、当然、T 秒後に蟻のいる座標の集合と同じなので、T 秒後にそれぞれの蟻がどこにいるかを求めると、番号は分からないが、どの座標にゼッケンがあるのかが分かります。ゼッケンの番号が時計回りに $1, 2, ...N$ となっているので、一つでもゼッケンの番号がわかれば他の番号も分かりますが、先ほど一つゼッケンの番号と位置の情報を得たので、これで求められます。ゼッケンのある座標の集合を昇順にソートする部分がボトルネックで、$O(NlogN)$ でこの問題は解けました。

D : Piling Up

ある実行可能な積み方 X があったとします。そして、最初に赤い積み木を A 個、青い積み木を $N - A$ 個箱に入れた時に、X が実行可能であるような最小の A がとれます。この時、最初に赤い積み木を A 個、青い積み木を $N - A$ 個箱に入れた状態から X を実行すると、箱の中の赤い積み木が 0 個である瞬間が存在します。もし存在しないとすれば、$A = A - 1$ としても X が実行可能であることから、これは分かります。よって、初期状態と積み方の組み合わせとして、箱の中の赤い積み木が 0 個である瞬間が存在するようなものだけを考えるようにします。すると、ある積み方は一度しか数えられない事になり、また全ての積み方は数えられる事になります。そしてこれは、次のような DP で数えることができます。

$DP[i][j][k] = i$ 回目の操作の直後において、箱に j 個の赤い積み木があり、箱の中の赤い積み木が 0 個になったことが $(k = 0 \rightarrow$ない$)(k = 1 \rightarrow$ある$)$ 時の場合の数

この DP の初期値は、$DP[0][0][1] = 1, DP[0][1][0] = 1, DP[0][2][0] = 1, ...DP[0][N][0] = 1$ となります。そして、最終的に求める答えは $DP[M][0][1] + DP[M][1][1] + ...DP[M][N][1]$ となります。この DP は遷移が $O(1)$ で求まるため、全体で $O(NM)$ で計算でき、この問題は解けました。

E : Placing Squares

たくさんの長方形を置く代わりに、問題を次のように言い換えます。

長さ N の棒の上に、いくつかの仕切りを立てる。仕切りは棒の左端からの距離が整数の場所にのみ立てる。棒の両端には必ず仕切りを立て、逆に印のついている部分の真上には必ず仕切りを立てない。二つの仕切りの間には、赤いボールと青いボールを置く。ボールが置かれる座標は、棒の左端から距離が $i.5(i$ は整数$)$ という形で表されるものだけを考える。二つのボールが同じ座標にあっても構わない。この時、仕切りとボールの置き方は全部で何通りあるか。

この問題が元の問題と同値であることは、仕切りを置く位置を正方形の置き方に、ボールの置く位置の自由度を正方形の面積に対応させればすぐに分かります。この問題は、次のような DP で解けます。

$DP[i][j] =$ 棒の左端から距離 i の地点までの仕切りとボールの設置を終えており、最も右にある仕切りの右側に既に置いてあるボールが j 個であるような場合の数

この DP の遷移は、印のある位置でも無い位置でも $O(1)$ で行えるので、$O(N)$ で解くことができます。し

かし、DP の遷移の形に注目すると、これは 3×3 行列で表せるので、行列累乗が行えます。これで、印の無い連続する区間での DP の遷移が $O(logN)$ で行えるようになるので、全体で $O(MlogN)$ でこの問題は解けました。

F : Two Faced Cards

まず C_i の値を昇順にソートしておきます。また、$B_i = min(A_i, B_i)$ としておきます。この問題を解くためには、

$ans_v = C_v$の値の書かれたカードを X に加えてゲームを行った時のスコアの最大値

が 1 から $N+1$ までのすべての v について前計算できればよいですが、まずは最初に、v の値がひとつ与えられた時に、ans_v を計算する方法を考えます。

この問題は、二部マッチングを作る問題とも考えられます。そこで、$cnt[i] = C_i$以下の値を使っている Z のカードの数 $-i$ とすると、cnt の全要素が非負になっていれば条件を満たしていることになります。なので、配列 cnt を保持して、スコアを最大化しつつ、cnt の全要素を非負にすることを目標にアルゴリズムを実行します。まず、$cnt[i] = -i$ で初期化します。次に、全ての $i(1 \leq i \leq N)$ について、$cnt[A_i], cnt[A_i+1], ...cnt[N+1]$ の値を全て 1 増やします。これは、とりあえず X のカードを表面のままにしておくことに対応します。ここからできる操作は、次の二つです。

操作 1 $v \leq i \leq N+1$ なる全ての i について、$cnt[i]$ の値を 1 増やす。（新しいカードを使うことに対応）

操作 2 $B_j \leq C_i < A_j$ なる全ての i について、$cnt[i]$ の値を 1 増やす。（X のカード j をひっくり返すことに対応）

あとは、上記 2 種類の操作を行って cnt を非負にして、そのなかで操作 2 を行う回数を最小化すればよいです。これは、次のような貪欲法で計算できます。

まず、上記の操作は区間加算と見ることができるので、操作の行える区間の集合を S とする。また、区間の集合 T を空集合としておく。整数 k を、$1, 2, ...N+1$ と動かしながら、以下の操作をする。$0 \leq cnt[k]$ なら、次の k に移る。そうでない場合、k を含む区間を S から T へ移す。$cnt[k] < 0$ である限り、T から右端が最も右にある区間を取り出し、区間加算をする、という操作を繰り返す。$cnt[k] < 0$ であるのに操作が行えなくなれば、割り当てが存在しないと報告する。

T から右端が最も右にある区間を取り出す操作は、他の操作よりも損しないため、この貪欲法は最適になります。区間の集合を優先度付きキューで管理すれば、この貪欲法は $O(NlogN)$ で動くので、v がひとつ与えられた場合は解けます。

v が決まっていない場合について考えます。ここで、v を考えずに、操作 2 だけを考えて、先ほどの貪欲法を実行してみます。すると、不完全な解が得られます。この時の解で使用した区間の集合を U とします。すると、v を決めて先ほどの貪欲法を実行した時の解で使う操作 2 の区間の集合は、U の部分集合となることが証明できます。これは、アルゴリズムの動き方から分かります。よって、全ての v について、U から除いても問題ない区間を選んで、その個数を最大化する問題になります。まず、$cnt[i] < 0, i < v$ を満たす i が存在する時、その v については割り当てが存在しません。次に、U 内の区間全てについて、

区間の左端 $=$ その区間を T から取り出した時の k の値

とします。この時、$cnt[i] < 0$ なる i を含むような区間は、v の値によらず取り除けません。そして、それ以外の区間からなる U の部分集合 W について、それを U から取り除いても問題ないことの必要十分条件は、W 内の区間の左端が全て v 以上で、かつどの二つの区間も重ならないことになります。

これは、後ろの方から見る区間スケジューリング問題と考えることができ、v を大きい方から求めていくと、効率的に全ての v について答えが求められます。このアルゴリズムは適切に実装すれば $O(NlogN)$ で動くので、この問題を解くことが出来ました。

Atcoder Grand Contest 013 Editorial

writer : maroonrk

2017 年 4 月 15 日

A : Sorted Arrays

Suppose that $A_1, A_2, \ldots, A_i$ is an (either non-increasing or non-decreasing) monotonous sequence. In this case, for each $j < i$, it never makes sense to cut the sequence between A_j and A_{j+1}. For example, in some solution, if the first two subarrays are $A_1, \ldots, A_j$ and $A_{j+1}, \ldots, A_k$ $(k > i)$, you can change them to $A_1, \ldots, A_i$ and $A_{i+1}, \ldots, A_k$ without making the solution worse.

Therefore, you can repeat the following: find the longest monotonous prefix from the sequence, delete it, again find the longest monotonous prefix from the remaining sequence, delete it, and so on.

Note that you should carefully handle equal numbers when you check whether a sequence is monotonous. First you should check whether there are two adjacent terms x, y such that $x < y$, and also check whether there are two adjacent terms x, y such that $x > y$. When both of these are found, the sequence is not monotonous.

This solution works in $O(N)$.

B : Hamiltonish Path

First, consider an arbitrary path. Check if this path satisfies the conditions. If yes, we are done. Otherwise, we can find a vertex that is adjacent to one of the endpoints of the path. We can extend the path using this vertex and we get a longer path. We repeat this process, and this process finishes after at most $N-1$ steps since the length of any path is at most $N-1$.

For example, we can implement this idea as follows:

1. Create a deque with single vertex: $\{1\}$.
2. Let x be the first element of the deque, and y be the last element of the deque. Repeat the following:
   (a) For each vertex z that is adjacent to x, check if z is contained in the path. If not, append it to the beginning of the deque and go to step 2.
   (b) For each vertex z that is adjacent to y, check if z is contained in the path. If not, append it to the end of the deque and go to step 2.
3. End the process. We found a solution.

In step 2(a) and step 2(b), each vertex is processed at most once. Therefore, this solution works in $O(M)$.

C : Ants on a Circle

We will restate the problem as follows:

Let's assign a card to each ant. Initially, the ant i has a card labelled with i. When two ants meet, instead of changing their directions, they swap their cards (and their directions won't be changed). After T seconds, we want to know the positions of each card.

Now, since the ants don't change directions, we can easily compute the positions of ants after T seconds. The main difficulty is to find the correspondence between ants and cards.

- The relative positions of the cards never changes. That is, the cards $1, 2, \ldots, N$ are aligned clockwise in this order.
- We can determine the card assigned to ant 1 after T seconds in the following way. If this ant is moving clockwise, each time it meets with another ant, the number on the card assigned to this ant increases by 1. Similarly, if this ant is moving counter-clockwise, each time the number decreases by 1. Thus, we can determine the number by counting the total number of meetings with other ants (which can be done in $O(N)$).

If we combine the two observations above, we can get the correspondence between ants and cards. This solution works in $O(NlogN)$.

D : Piling Up

Suppose that the box currently contains x red bricks and $N - x$ blue bricks. You will perform M operations from now, and each operation is one of the following four types:

- Operation 'RR': Take a red brick from the box, put a red brick and a blue brick into the box, and take a red brick from the box. This is possible only when $x > 0$, and x decreases by 1 after the operation.
- Operation 'RB': Take a red brick from the box, put a red brick and a blue brick into the box, and take a blue brick from the box. This is possible only when $x > 0$, and x remains unchanged after the operation.
- Operation 'BR': Take a blue brick from the box, put a red brick and a blue brick into the box, and take a red brick from the box. This is possible only when $x < N$, and x remains unchanged after the operation.
- Operation 'BB': Take a blue brick from the box, put a red brick and a blue brick into the box, and take a blue brick from the box. This is possible only when $x < N$, and x increases by 1 after the operation.

The problem asks the number of possible sequences of these operations.

Now, it is natural to define $dp[i][j]$: the number of possible sequences of the first i operations that end with j red bricks after the operations. The problem is that, we may count the same sequence multiple times this way.

For example, the following two sequences should not be distinguished because they have the same sequence of operations:

- Start with one red brick, perform an 'RR' operation, and end with two red bricks.
- Start with two red bricks, perform an 'RR' operation, and end with three red bricks.

In order to avoid double-countings, we add another restriction: you must perform at least one special operation. We call an operation "special" if it has the smallest possible value of x when the operation is performed. That is, a 'RR' or 'RB' operation when $x = 0$, or a 'BB' or 'BR' operation when $x = 1$. It is easy to see that any sequence of operations has exactly one way to perform with at least one special operation.

Now, define $dp[i][j][k]$ as the number of possible sequences of the first i operations that end with j red bricks after the operations, and additionally k is a boolean value that shows whether we have performed special operations. This solution works in $O(NM)$.

E : Placing Squares

The key observation in this problem is to rephrase the statement in a combinatorial way. For example, when we put k squares of sizes $a_1, \ldots, a_k$ from left to right (in a valid way), we want to count this configuration $a_1^2 a_2^2 \ldots a_k^2$ times. How can we do that?

It turns out that the problem is equivalent to the following:

> There are N cells. Some of the borders between two adjacent cells may be marked. How many ways are there to put some separators and red/blue balls such that:
>
> - There must be a separator at the left border of the leftmost cell and the right border of the rightmost cell.
> - You may put separators between two adjacent cells that are not marked.
> - You may put balls into cells (a cell may contain both red and blue balls).
> - Between each pair of adjacent separators, there must be exactly one red ball and exactly one blue ball.

The relation between this problem and the original problem is clear: the separators correspond to squares, and balls are added to add the factor of $a_1^2 a_2^2 \ldots a_k^2$.

Now the problem suddenly becomes very easy!

Define $dp[x][k]$ as the number of ways to determine the placement of separators and balls to the first x cells such that currently we put k balls after the last separator. This solution works in $O(N)$, and it is straightforward to improve this solution using matrix exponentiation. The solution with matrix exponentiation works in $O(MlogN)$.

F : Two Faced Cards

This problem is very challenging, and this is the main reason of the extended contest duration. Before tackling the original problem, let's prepare several things.

First, without loss of generality, we can assume that $C = \{0, 1, \ldots, N\}$. We should first sort the given C, and change the smallest number to 0, the second smallest number to 1, and so on. We should also change the values of A_i, B_i, D_i, E_i accordingly to preserve the relative relation between these numbers and C_i. More specifically, when $C_{i-1} < x \leq C_i$, the number x should be converted into $i - 1$.

Next, suppose that we have two arrays $p_1, p_2, \ldots$ and $q_1, q_2, \ldots$ (here each element is up to N). How can we check if we can permute the elements in p, q such that $p_i \leq q_i$ is satisfied for each i? One obvious solution is to sort each of them. However, in this solution, we use the following.

Consider an array of integers that is initially filled with zeroes. For each p_i, we add 1 to the interval $[p_i, N]$. For each q_i, we add -1 to the interval $[q_i, N]$. The condition is satisfied if and only if the array obtained this way consists of non-negative integers.

Now, let's return to the original problem. First, we create an array with zeroes, and for each i from 0 to N, add -1 to the interval $[i, N]$. This corresponds to the cards in deck Y. Then, for each i, we add 1 to the interval $[a_i, N]$. This corresponds to the cards in deck Z when we use front sides of all cards, except for the single added card.

This array may still contain negative numbers. We want to change all elements of this array to non-negative numbers by the following operations:

- Flip some cards in deck Z. This makes sense only when $b_i < a_i$, and it adds 1 to the interval $[b_i, a_i)$.
- Add a single card (depending on the query). It adds 1 to the interval $[x, N]$ for some integer x.

And our objective is to minimize the number of operations of the first type.

How can we handle queries? Suppose that the given query is (d, e). We consider two cases independently: whether we use the front side (d) or the back side (e) of the given card. Thus, in the problem above, it is sufficient if we can compute the minimum number of operations of the first type for each x from 0 to N.

Let's summarize what we have to do from now. (Note that the variable names in the problem below has nothing to do with the original problem, but the correspondence should be clear.)

> You are given an array of integers $a_0, \ldots, a_{N-1}$. This array may contain positive or negative numbers. You are also given a (multi)set of intervals $[L_i, R_i)$. In one operation, you can choose one of the intervals and add 1 to all elements of the array corresponding to this interval. Each interval can be used at most once. For each x between 0 and $N - 1$, solve the following problem:
>
> - How many operations are required to convert the array such that $a_0 \geq 0, a_1 \geq 0, \ldots, a_x \geq 0, a_{x+1} \geq -1, \ldots, a_{N-1} \geq -1$?

In the remaining part of this editorial, we will describe how to solve this problem.

First, find the rightmost element in the array that is smaller than -1. Regardless of the value of x, we must choose at least one of the intervals that contain this element. Among them, consider the interval whose left end is the leftmost (let's call it I). We will claim that, regardless of the value of x, we should use this interval.

Proof. Let p be the rightmost position such that $a_p < -1$. Let c_i be the distance from the current value of the i-th element to the target value of this element. For example, if this element is currently -4 and this element must be at least -1 after choosing some intervals, the distance is defined as 3. Now, regardless of the value of x, it is easy to see that $c_p > 0$ and for each $q > p$, $c_p > c_q$ is satisfied. In the optimal solution, there are at least c_p chosen intervals that contain the position p. Among them, consider the one with the smallest right end. It does no harm even if we change this interval to I. Thus, we can assume that we always use the interval I.

Now, we can greedily repeat this process. Traverse the elements of the array from right to left. When we find an element that is smaller than -1, we find an unused interval that contains this element and use it. In case there are multiple such intervals, choose the one with the smallest left end. This way, we get an array whose elements are at least -1 in all positions.

In the remaining part, it's easier to find a correct strategy. Since each element is now at least -1, we are now only interested in the leftmost occurrence of -1. Thus, we can repeat the following process. This time, traverse the elements of the array from left to right. When we find an element that is smaller than 0, we find an unused interval that contains this element and use it. In case there are multiple such intervals, choose the one with the largest right end.

Finally, we solved the problem. This solution works in $O(NlogN)$ (we need priority queues to simulate the solution above).

AGC 012 解説

writer : camypaper

2017 年 4 月 1 日

For international readers: English editorial starts from page 8.

A : AtCoder Group Contest

結論から言えば，「$2, 4, 6, \ldots, 2N$ 番目に強い参加者の強さの和」が答えである．参加者を弱い順に並べたとき、最初の N 人を N 個のチームの最も弱いメンバーとすることができる．このとき $N+1$ 番目の参加者はどのチームに入ってもチームで 2 番目に強いメンバーとなる．すると，$N+2$ 番目の人は $N+1$ 番目の参加者と同じチームにするのがよいことが分かる．このようにしてチームを作っていくと，上記の値が得られる．

計算量は，強さの順にソートする部分が最も重く， $O(N \log N)$ である．

B : Splatter Painting

操作を逆順に見ていくことにする．このとき，頂点の色が上書きされるという条件は一度でも色が塗られた頂点の色が変わることはない，と言い換えられる．すると，「頂点 v から距離 d 以内にある頂点たちの色を塗る」という操作が一度でも行われたならば，(操作を逆順に見たときの) それ以降の操作によって頂点 v から距離 d 以内にある頂点の色が変わることはないことがわかる．これを利用して動的計画法によりこの問題を解くことが可能である．

$\mathrm{dp}(v, d)$ を「頂点 v から距離 d 以内にある頂点たちの色を塗る」という操作が行われた (操作を逆順に見たときの) 最初の時点，とする．頂点 v から距離 d 以内にある頂点の色を塗るという操作が行われたとき，v と隣接している頂点 u について「頂点 u から $d-1$ 以内にある頂点を塗る」，という操作が行われる．また，「頂点 v から距離 $d-1$ 以内にある頂点の色を塗る」という操作も行われていると考えてよい．

最終的な頂点の色は $\mathrm{dp}(v, 0)$ から何番目の操作によって色が塗られたかを知ることで求められる．ただし，操作が行われていない場合頂点の色は 0 である．これらの処理は $O(Q + (N + M) \max d_i)$ の計算量で達成可能であり，十分高速である．

C : Tautonym Puzzle

p を $(1, 2, 3, \ldots, n)$ を並び替えた数列として，$s = (p_1, p_2, p_3, \ldots, p_n, 1, 2, 3, \ldots, n)$ とする．このとき，s の部分列であって，良い文字列であるようなものの個数は p の増加部分列の数 $f(p)$ で表される．以下のような問題を解くことが可能ならば，元の問題も解くことが可能である．

> 長さ 100 以下の順列 p であって，$f(p) = N$ を満たすような p の一例を示せ．

ここで，$p = (1, 2, 3, \ldots, n)$ のとき，$f(p) = 2^n - 1$ である．p の $i-1$ と i の間に $n+1$ を挿入した数列を p'_i としたとき，$f(p'_i) = 2^n - 1 + 2^{i-1}$ となり，$f(p)$ から 2^i だけ増加する．さらに，$p = (1, 2, 3, \ldots, n)$ に $n+1, n+2, \ldots$ を上記のように挿入していくとき，新たに挿入された数たちが降順に並ぶように挿入すると，「$f(p)$ を 2^i だけ増加させる」という独立な操作として扱うことが可能である．これを利用すると，$2^n - 1 + 2^a + 2^b + \ldots = N$ となるように順列を構成すればよいことが分かる．以下の C++ のソースコードで表されるように構成を行うと，長さ $2 \log N$ 以下での構成が可能である．$N \leq 10^{12}$ より，構成された順列 p の長さは 80 以下となり，必ず条件を満たす．

```
vector<int> construct(long long N){
    int n;
    vector<int> p;
    for(n=40;n>0;n--){
        if(N>=(1LL<<n)-1)break;
    }
    for(int i=1;i<=n;i++)
        p.push_back(i);
    N-=(1LL<<n)-1;

    for(int i=n-1;i>=0;i--){
        if(N<1LL<<i)continue;
        n++;
        p.insert(p.begin()+i,n);
        N-=1LL<<i;
    }
    return p;
}
```

D : Colorful Balls

ボールの色が 1 種類の場合，答えは 1 である．以降，ボールの色は 2 種類以上存在すると仮定する．一般に，3 つのボール a, b, c が存在して，a と b，a と c の位置がどちらも交換可能ならば，b と c の位置も必ず交換可能である．これは，図 1 に示されるように「a と b の位置を交換」，「a と c の位置を交換」，「a と b の位置を交換」という手順で操作を行うことで達成可能である．

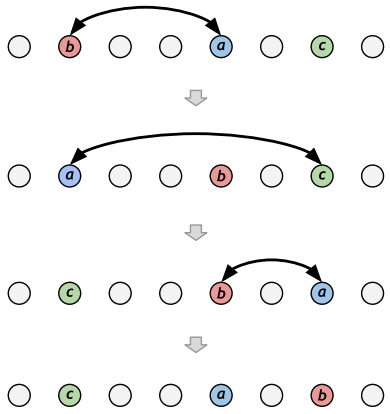


図 1　a と b，a と c が位置を交換可能なときに，b と c の位置を交換する操作

ある色 i に着目しているとき，色が i であるボールのうち最も軽いボールを x_i とする．このとき，色が i であって，操作 1 で x_i と交換可能なボールの重さは x_i の重さと等しいとしてもよいことが示せる．x_i と操作 1 により操作が交換可能なボール a，x_i と操作 2 により位置が交換可能なボール b があるとする．先程の事実から a と b の位置も交換可能であることが分かる．よって，操作 2 によって a と位置の交換が可能どうかは x_i の重さによってのみ定まることが言える．

操作 2 による位置の交換については N 個のボールのうち最も軽いボール y と，y と色が異なるボールのうち最も軽いボール z についてのみ考えれば十分である．これは，以下のことから示すことが可能である．

- y と色が異なり，かつ y と位置の交換が不可能なボールは，他のどのボールとも操作 2 では位置の交換が不可能．
- y と色が同じで，かつ z と位置の交換が不可能なボールは，他のどのボールとも操作 2 では位置の交換が不可能．

ここで，y から y と操作 2 により位置が交換可能なボールに辺を張り，z についても同様の操作を行ったグラフを考える．このグラフ上の同じ連結成分に含まれるボールたちは互いに位置の交換が可能なことが示せる (詳細は略)．最終的に互いに位置が交換可能なボールたちの並べ方を調べればよく，これは容易に達成可能である．全ての処理は $O(N)$ で達成可能であり，十分高速である．

E : Camel and Oases

まず，ラクダが N 箇所のオアシスを全て訪れることは可能か？という問題について考える．ラクダがまた訪れていないオアシスであって，歩いて到達可能なオアシスが存在するならば，先にそのようなオアシスを訪れた方がよいのは明らかである．ここから，ラクダが大ジャンプをするとき，現在の位置から歩いて到達可能なオアシスは必ず存在しないと考えてよい．

ラクダが大ジャンプをするとき，オアシスから移動する必要はないことも明らかであろう．すると，ラクダが大ジャンプ可能な回数を K とすると，K は $O(\log V)$ 程度となる．$V \leq 2 \times 10^5$ より，K は高々 18 以下である．以上のことをまとめると，この問題は以下のように言い換えられる．

> $d_0 = V, d_i = \lfloor \frac{d_{i-1}}{2} \rfloor$ を満たす長さ K の数列 d が与えられる．数直線上に並べられた N 個の点の間にいくつか仕切りを入れて $k(k \leq K)$ 個のグループに分割したい．グループには $0, 1, 2, \ldots, k-1$ の番号を割り振る．左から i 番のグループに割りふられた番号を p_i として，以下の制約を満たすように分割することは可能か？
>
> - 左から i 番のグループに含まれるどの隣り合った二点間の距離も d_{p_i} 以下である．
> - 左から $i-1$ 番目のグループの右端の点と，i 番目のグループの左端の点の距離は d_{p_i} より大きい．
> - 左から i 番目のグループの右端の点と，$i+1$ 番目のグループの左端の点の距離は d_{p_i} より大きい．

グループに左から順番に番号を割り振っていくときに K 個以下のグループで N 個の点を全てグループに割り振ることが可能かどうかを調べることにする．これは， dp(使った番号の集合) = 次のグループの始点として選べる点のうち，最も右側にある点，という bitDP が解ければよい．各状態において今いるグループに割り当てる番号の数は $O(K)$ 個である．さらに，次のグループの始点がどこになるかは $O(NK)$ で前計算しておけば $O(1)$ で調べられる． であるから，$O((N+V)\log V)$ でこの問題を解くことが可能である．

さて，元の問題では始点を全て求める必要があった．これは左から bitDP をした結果と，右側から bitDP した結果の 2 つを組み合わせることで解くことが可能である．0 番以外の番号付けをしたときに，残った点たちが存在するかどうか，残った点たちを 1 つのグループにすることが可能かどうか，の 2 つを調べればよい．これは $O(V)$ で可能である．よって，全体として $O((N+V)\log V)$ でこの問題を解くことが可能である．

F: Prefix Median

簡単のため, 与えられる数列が順列である場合から考える．このとき, $b_i < i$ あるいは $b_i > 2N-i$ を満たすような b は明らかに存在しない．その他，図 2 に示されるように，$2i-1$ 個の要素からなる集合にどのように 2 つ新しい要素を追加しても，以下のどちらかを満たすことが分かる．

- 新しい中央値も元の中央値のままである．
- 新しい中央値は元の中央値に隣接した位置にある．

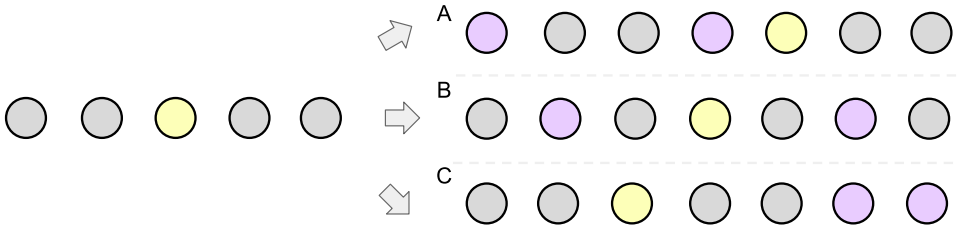


図 2　元の中央値と新しい中央値の位置関係

これらの事実は，数列 b に以下のような制約を与える．

- 制約 $1: i \leq b_i \leq 2N-i (1 \leq i \leq N)$
- 制約 $2: b_j < b_i < b_{j+1}$ なる $i, j (i < j)$ は存在しない．
- 制約 $3: b_j > b_i > b_{j+1}$ なる $i, j (i < j)$ は存在しない．

制約 $2, 3$ のうちどちらかに違反したとすると，上述した性質に違反するため，そのような数列が存在しないことが示せる．逆に，これらの性質を満たす数列であれば，実は必ず構成可能である．数学的帰納法を用いて示す．任意の自然数 k について長さ $2k-1$ の順列 a が与えられたとき，上の性質を満たす数列 b が全て構成可能である，と仮定する．$k=1$ のときは明らかに成立する．$k=i$ のときに成立するならば $k=i+1$ のときも成立することを示す．$k=i+1$ のとき，$b_{i+1}=i+1$ である．このとき，上の制約より b_i は $i, i+1, i+2$ のいずれかである．それぞれ場合分けをして考える．

$b_i = i$ のとき

図 3 に示されるように，$1, 2, 3, \ldots, i$ と $i+1, i+2, \ldots, 2i-1$ の 2 つのグループに分ける．後者のグループから $b_1, b_2, \ldots, b_{i-1}$ に含まれない値であって，最も i に近いものから 2 つ取り除く．後者のグループの大きさは $i+1$ であるため，必ず取り除くことが可能である．制約 2, 3 より，上のように 2 つの数を取り除いた集合において，b_{i-1} と b_i が等しい，あるいは b_{i-1} と b_i が隣接するという条件を満たすようにすることが可能である．$b_i = i+2$ の場合もほぼ同様である．

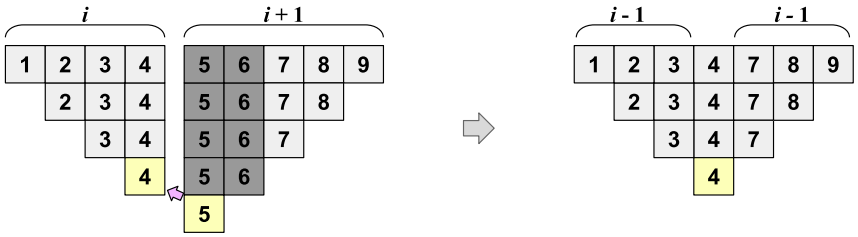


図 3　$b_i = i$ のときの 2 つの数の取り除き方

$b_i = i+1$ のとき

図 4 に示されるように，$1, 2, \ldots, i$ と $i+2, i+3, \ldots, 2i-1$ のグループに分ける．2 つのグループから $b_1, b_2, \ldots, b_{i-1}$ に含まれない値であって，最も $i+1$ に近いものから 1 つずつ取り除く．2 つのグループの大きさはどちらも i であるため，必ず取り除くことが可能である．制約 2, 3 より，上のように 2 つの数を取り除いた集合において，b_{i-1} と b_i が等しい，あるいは b_{i-1} と b_i が隣接するという条件を満たすようにすることが可能である．

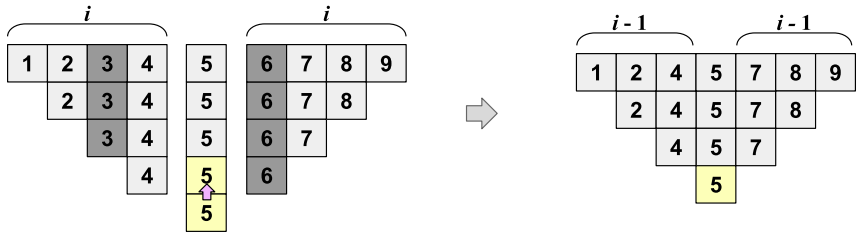


図 4　$b_i = i+1$ のときの 2 つの数の取り除き方

その後，残った数に $1, 2, 3, \ldots, 2i-1$ となるように数を振り直すことにすると，$k = i$ の場合に帰着させることが可能である．b_i の値に関わらず $k = i$ の場合に帰着させることが可能であり，$k = i$ の場合は構成可能であると仮定したので，$k = i+1$ も構成可能であることが示された．

以上より，与えられる数列が順列である場合，この問題は以下のように言い換えられる．

整数 N が与えられる．以下の条件を満たす数列 b はいくつあるか？

- $i \leq b_i \leq 2N - i (1 \leq i \leq N)$
- $b_j < b_i < b_{j+1}$ なる $i, j (i < j)$ は存在しない．
- $b_j > b_i > b_{j+1}$ なる $i, j (i < j)$ は存在しない．

これらの制約から， b が以下の特徴を持つことが分かる．

- b_i が取りうる値の候補の数は b_{i+1} のものから 2 ずつ増える．
- 一度飛び越した候補は 2 度と使うことができない．

以上のことから，$\mathrm{dp}(i, j, k) = i$ 項目以降が確定した時点で，j 個の候補が残っており，候補のうち k 番目の値が b_iであるような場合の数，という動的計画法で解くことが可能である．これを $\mathrm{dp}(N, 1, 1)$ から順番に埋めていけばよい．遷移は以下の 3 通りである．

- $\mathrm{dp}(i, j, k)$ から $\mathrm{dp}(i - 1, j + 2, k + 1)$ に加算
- $\mathrm{dp}(i, j, k)$ から $\mathrm{dp}(i - 1, j', k - |j + 2 - j'|)$ へ加算 $(j + 3 - k \leq j' \leq j + 2)$
- $\mathrm{dp}(i, j, k)$ から $\mathrm{dp}(i - 1, j', k + 2)$ へ加算 $(k + 2 \leq j' \leq j + 2)$

状態の数が $O(N^3)$，遷移が $O(N)$ となり，合わせて $O(N^4)$ で十分高速である．

最後に与えられる数列 a が順列でない場合について考える．a はソート済みであるとしても一般性を失わない．一般の数列の場合，値が同じもののうち，N に最も近い位置だけ中央値として採用されうる，とすると順列の場合とほぼ同様である．結論として以下の条件を満たす数列を数え上げればよい．

- $a_i \leq b_i \leq a_{2N-i} (1 \leq i \leq N)$.
- $b_j < b_i < b_{j+1}$ なる $i, j (i < j)$ は存在しない．
- $b_j > b_i > b_{j+1}$ なる $i, j (i < j)$ は存在しない．

b_i が取りうる値の候補の数が b_{i+1} のものから最大で 2 つ増えることがある，という条件のもと先程と同様に数え上げればよい．こちらについても計算量は $O(N^4)$ となる．

AGC 012 Editorial

writer : camypaper

April 1st, 2017

A : AtCoder Group Contest

Without loss of generality, we can assume that $a_1 \geq a_2 \geq \cdots \geq a_{3N}$. We prove that the answer is $a_2 + a_4 + a_6 + \cdots + a_{2N}$.

Sort the teams in the descending order of their strengths, and let p be the second strongest member in the i-th strongest team. Then, we know that at least $2i - 1$ people are stronger than p: two members each in the first $i - 1$ teams and the strongest member in the i-th team. Thus, the strength of p is at most a_{2i}, and the sum of team strengths is at most $a_2 + a_4 + a_6 + \cdots + a_{2N}$.

On the other hand, we can achieve this sum by forming teams in the following way: (a_1, a_2, a_{3N}), $(a_3, a_4, a3N - 1)$, (a_5, a_6, a_{3N-2}), $\cdots$ (i.e., when you form a new team, always choose the two strongest remaining people and the weakest reaming people).

This solution works in $O(N \log N)$.

B : Splatter Painting

Perform the operations in the reverse order: when you perform the operation i, you should color all uncolored vertices that are within a distance d_i from vertex v_i, in color c_i. (You never overpaint vertices). This will give the same answer as the original problem.

How do you paint all vertices that are within a distance d from vertex v? When you do this, you call a function $func(v, d)$. When $d > 0$, a vertex is within a distance d from v if one of the following is satisfied:

- The vertex is within a distance $d - 1$ from vertex v.
- The vertex is within a distance $d - 1$ from one of neighbors of v.

Thus, when $d > 0$, $func(v, d)$ recursively calls $func(v, d - 1)$ and $func(u, d - 1)$ for each adjacent vertex u. You should memoize this solution: when $func(v, d)$ is called, you memoize this fact, and never perform the recursions again when this is called again. The final color of a vertex v can be obtained by the first time the function $func(v, 0)$ is called.

This solution works in $O(Q + M \max d_i)$.

C : Tautonym Puzzle

Let p be a permutation of $(1, 2, 3, \ldots, n)$, and let $s = (p_1, p_2, p_3, \ldots, p_n, 1, 2, 3, \ldots, n)$.

How many good subsequences does s have? If tt is a subsequence of s, t must be a subsequence of both $(p_1, \ldots, p_n)$ and $(1, \ldots, n)$. Thus, the number of good subsequences of s is the same as the number of (non-empty) increasing subsequences of $(p_1, \ldots, p_n)$.

Let $f(p)$ be the number of (possibly empty) increasing subsequences of $(p_1, \ldots, p_n)$. We should solve the following problem:

> Construct one permutation of length up to 100 such that $f(p) = N + 1$.

We use the following facts:

- $f(emptysequence) = 1$
- For any permutation q of length k, $f(q_1, \ldots, q_k, k+1) = 2f(q_1, \ldots, q_k)$.
- For any permutation q of length k, $f(k+1, q_1, \ldots, q_k) = f(q_1, \ldots, q_k) + 1$.

You define a function that returns p such that $f(p) = X$ for a given X. When X is an even (or odd) number, it recursively calls the function with the parameter $X/2$ (or $X - 1$). This way we can construct a permutation whose length is up to $2lgN = 80$.

D : Colorful Balls

Construct a graph with N vertices. We add an edge between two balls s and t if s and t are swappable.

For three balls a, b, c, when a and b are swappable, and a and c are also swappable, we can prove that b and c are also swappable:

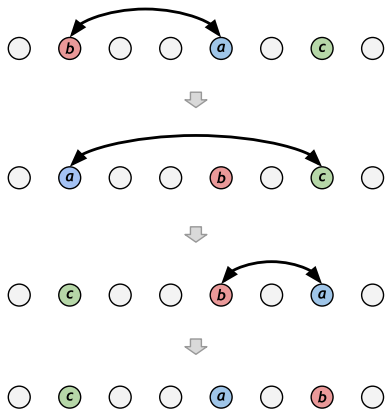


図 1　Swapping b and c

By performing possibly multiple operations, you can arbitrarily permute balls in the same connected component in this graph. Thus, our main objective in this problem is to find connected components in this graph. If we add edges between all pairs of swappable balls, the number of edges can be as large as N^2. Instead, we reduce the number of edges while keeping the connected components.

First, consider Operation 1, and in particular, consider balls with color i. Let x_i be the lightest ball of color i, and y_i, z_i be other two balls of color i. When $y_i + z_i \leq X$, we should add an edge betwen y_i and z_i. However, in this case, there are edges between x_i and y_i, and x_i and z_i. It means that even if we ignore the edge between y_i and z_i, it doesn't change the connected components. In Operation 1, we only need to consider pairs that involve the ball x_i, and we need at most $O(N)$ edges.

Next, consider Operation 2. Without loss of generality, we can assume the following:

- The colors are numbered 1 through C.
- For each i, the lightest ball with color i is a_i.
- $a_1 \leq a_2 \leq \cdots$

We claim that we only need to consider pairs that involve either a_1 or a_2.

When there is an edge between two balls s and t (i.e., their colos are different and $weight(s)+weight(t) \leq Y$),

- If $color(s) \neq 1$ and $color(t) \neq 1$, there is a path $s - a_1 - t$.
- If $color(s) \neq 2$ and $color(t) \neq 2$, there is a path $s - a_2 - t$.
- If $color(s) = 1$ and $color(t) = 2$, there is a path $s - a_2 - a_1 - t$.

This way, we again need only $O(N)$ edges.

The entire solution works in $O(N)$.

E : Camel and Oases

First, the camel visits some oases by walking. The set of visited oases in this step is an interval, and for each adjacent pair of two oases in this interval, the distance between them must be at most V.

Then, the camel jumps to another place and starts walking again. The set of visited oases in this step is again an interval, and this time the distance between adjacent oases must be at most $floor(V/2)$. The camel jumps again and starts walking, and so on.

In summary, the problem can be restated as follows:

> Let $\{v_0, v_1, \ldots, v_{k-1}\}$ be the sequence $\{V, floor(V/2), floor(V/4), \ldots, 0\}$ (it stops after the first appearance of zero). Can we divide the N points (oases) into k disjoint intervals such that for each i, the distance between two adjacent points in this interval is at most v_{p_i} (here p is some permutation of length k)? Also, for each point, determine if the point can be in the interval with v_0.

We do the following bitmask DP: for each subset S of $\{v_1, v_2, \ldots, v_{k-1}\}$, compute the length of the longest prefix that can be covered by the intervals corresponding to elements in S. Call it $dpL[S]$. Similarly, do the same thing for suffixes: call it $dpR[S]$. Since $k = O(logV)$, we can compute these arrays in $O(2^k k) = O(VlogV)$.

How to check if the camel can start from a particular point p? Let $[l, r)$ be the interval of points that can be reached only by walking from the point p. The answer is "Possible" if we can divide the set $\{v_1, v_2, \ldots, v_{k-1}\}$ into two disjoint sets S_1 and S_2, such that $dpL[S_1] \geq l$ and $dpR[S_2] \geq N - r$.

In total, this solution works in $O((N + V) \log V$.

F: Prefix Median

For simplicity, consider the case where $a_i = i$ for each i.

Obviously, $i \leq b_i \leq 2N - i$. Also, we can prove that for each i, b_i and b_{i+1} are either the same or adjacent in the sequence $a_1, \ldots, a_{2i+1}$. See the following picture:

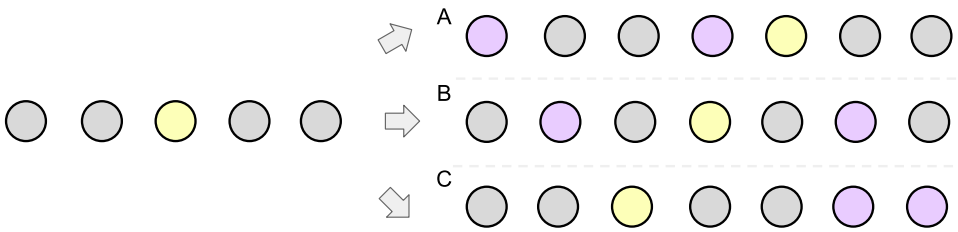


図 2　The median of a sequence and the median of new sequence after adding two elements

Thus, b must satisfy the following conditions:

1. $i \leq b_i \leq 2N - i (1 \leq i \leq N)$
2. No $(i, j)(i < j)$ satisfies $b_j < b_i < b_{j+1}$.
3. No $(i, j)(i < j)$ satisfies $b_j > b_i > b_{j+1}$.

We claim that these conditions are sufficient. By induction, it is sufficient to prove the following:

Let $b_1, \ldots, b_i + 1$ be the sequence that satisfies the three conditions above. That is,

- b_{i+1} is the median of the $2i + 1$ numbers.
- b_i is one of the three middle numbers of the $2i + 1$ numbers.
- b_{i-1} is one of the five middle numbers of the $2i + 1$ numbers.
- :
- No $(p, q)(p < q)$ satisfies $b_q < b_p < b_{q+1}$.
- No $(p, q)(p < q)$ satisfies $b_q > b_p > b_{q+1}$.

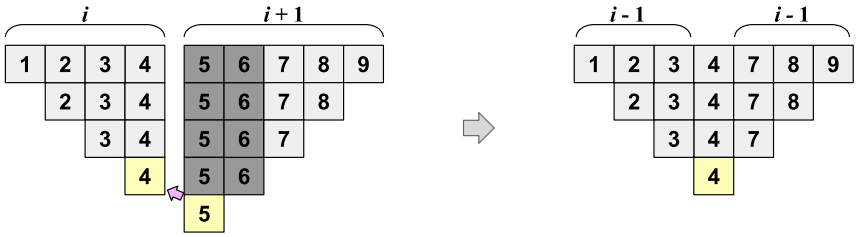
Then, we can remove two of numbers $1, 2, \ldots, 2i + 1$ such that:

- The numbers $b_1, \ldots, b_i$ remain after the removal of two numbers.
- b_i is the median of the remaining $2i - 1$ numbers.
- b_{i-1} is one of the three middle numbers of the remaining $2i - 1$ numbers.
- b_{i-2} is one of the five middle numbers of the remaining $2i - 1$ numbers.
- :

There are three cases: $b_{i+1} = i, i+1, i+2$. We consider each of these cases separately:

The case where $b_i = i$

Consider the numbers $\{i+1, \ldots, 2i+1\}$. Since there are $i+1$ numbers, at least two of these don't appear in $b_1, \ldots, b_{i-1}$. It is easy to prove that if we remove the two smallest such numbers, the conditions will be satisfied.

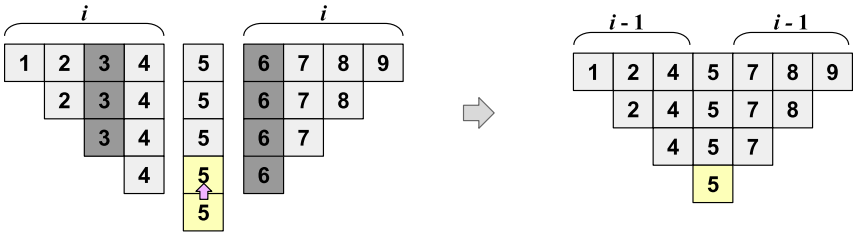


The case where $b_i = i + 2$

Similar to the case above.

The case where $b_i = i + 1$

Remove the greatest number in the set $\{1, \ldots, i\}$ that doesn't appear in $\{b_1, \ldots, b_{i-1}\}$. Also, remove the smallest number in the set $\{i+2, \ldots, 2i+1\}$ that doesn't appear in $\{b_1, \ldots, b_{i-1}\}$. Again, it is easy to prove that if we remove these two numbers, the conditions will be satisfied.



In general case (where a_i are not necessarily pairwise distinct), the conditions (and the proof) are very similar. Assume that $a_1 \leq a_2 \leq \cdots \leq a_{2N-1}$. Then,

1. b_i is one of the numbers $\{a_i, a_{i+1}, \ldots, a_{2N-i}\}$.
2. No $(i, j)(i < j)$ satisfies $b_j < b_i < b_{j+1}$.
3. No $(i, j)(i < j)$ satisfies $b_j > b_i > b_{j+1}$.

How to count the number of such sequences? Determine the sequence in the order $b_N, \ldots, b_1$.

Then, the last two conditions mean the following: when we "jump over" a number x, we can never use x later in the sequence.

This will lead to an easy DP. Define $dp[i][j][k]$ as the number of ways to choose the first i elements of the sequence (in the reverse order), such that there are j candidate numbers for the next element, and the number we chose the last (i.e., the i-th element) is the k-th smallest among the j candidate numbers. It works in $O(N^4)$.

AtCoder Grand Contest 011 解説

writer: semiexp

2017 年 3 月 12 日

For International Readers: English editorial starts on page 6.

A: Airport Bus

バス B に b 人，バス B' に b' 人の乗客が乗っているとする．バス B' に，バス B のある乗客より到着時刻の遅い乗客が乗っているとき，バス B' の乗客は全員，バス B のどの乗客よりも到着時刻が遅いとしてよい．さもなくば，バス B, B' のいずれかの乗客のうち，到着時刻が早い方から b 人を新たにバス B に，残った b' 人を新たにバス B' に乗せることができる．

よって，到着時刻が早い乗客から順に,「まずその人がバスに乗っていないなら，その人をバスに乗せ，さらに乗せられるだけ到着時刻が早い人から乗せていく」ことを繰り返せばよいことがわかる．これは，T_i をソートした後,「今残っている乗客のうち，先頭の乗客と同じバスに乗る乗客」を乗せられるだけ先頭から選んでいくことを繰り返せば解くことができる．これは $O(N \log N)$ で可能である．

B: Colorful Creatures

生き物を大きさでソートすることで，一般性を失わず $A_1 \leq A_2 \leq \cdots \leq A_N$ としてよい．$2\sum_{i=1}^{k} A_i < A_{k+1}$ であるような最大の k を t とおく．(このような k が存在しないときは $t = 0$ とする) $1, 2, \ldots, t$ 番目の生き物の色が最後まで残るためには，大きさが $\frac{A_{t+1}}{2}$ より大きくなってから $t+1$ 番目の生き物を吸収する必要があるが，$1, 2, \ldots, t$ 番目の生き物すべてが合体しても不可能であることから，このようなことはできない．(もちろん，$t+1$ 番目より大きい生き物を先に吸収することもできない) 一方，$t+1, t+2, \ldots, N$ 番目の生き物は，小さい生き物から順に吸収していくことで，最終的に残ることができる．

よって，上で示した t を計算し (左辺の総和は，$k = 1, 2, \ldots, N$ の順番で試していけば全部で $O(N)$ で求められる)，$N - t$ を出力すればよい．

C: Squared Graph

問題を一般化して，次のような問題を考える：

> 無向グラフ $A = (V_A, E_A), B = (V_B, E_B)$ が与えられる．グラフ C を，頂点集合は $V_A \times V_B$ であり，辺集合は「$(a, b), (a', b')$ 間に辺があることと，A において a, a' 間に辺がありかつ B において b, b' 間に辺があることは同値」として定めるとき，C の連結成分の個数を求めよ．

この問題は，上の問題において $A = B$ の場合に対応するから，上の問題が解ければ元の問題も解けることがわかる．

まず，A, B がいずれも連結である場合を考える．C において $(a, b), (a', b')$ 間に辺があることと，ある整数 l が存在し，A において a, a' 間に，B において b, b' 間に，それぞれ長さ l のパスが存在することは同値であることに注意する．

A, B いずれも頂点数が 2 以上の場合，任意のパスの長さを 2 伸ばすのは容易であるため，この条件は「A において a, a' 間に，B において b, b' 間に，長さの偶奇の等しいパスが存在する」と言い換えられる．

- A, B の少なくとも一方の頂点数が 1 のとき
  C には辺は存在しないため，連結成分の個数は $|A| \cdot |B|$ である．

- A, B の両方の頂点数が 2 以上であり，A, B の少なくとも一方が二部グラフではないとき
  このときは，C の連結成分の個数は 1 であることを示す．そのためには，A において a, a' 間に辺があり，B において b, b' 間に辺があるとき，C において $(a, b), (a', b)$ 間，$(a, b), (a, b')$ 間の両方に辺があることを示せばよい．

  B が二部グラフではないと仮定する．(A が二部グラフではない場合も同様) すると，B には長さが奇数のサイクル $v_1 v_2 \dots v_k v_1$ (k は奇数，この列において隣り合う頂点間には辺がある) が存在する．よって，b から v_1 を経由して，この奇数長のサイクルを通り v_1 に戻り，最初と同じパスを経由して b を戻ることで，B において b から b への奇数長のパスを得る．a から a' の間には奇数長のパスがあるのは明らかであるから，$(a, b), (a', b)$ 間 に辺がある．これと $(a', b), (a, b')$ 間に辺があることと合わせ，$(a, b), (a, b')$ 間にも辺がある．

- A, B の両方の頂点数が 2 以上であり，A, B の両方が二部グラフであるとき A の頂点集合 V_A を 2 つの部分集合 S_A, T_A に分け，すべての辺が S_A, T_A の間にまたがっているようにできる．B についても同様に V_B を S_B, T_B に分ける．

  先程と同様に，$a \in S_A, b \in S_B$ であるような (a, b) および，$a' \in T_A, b' \in T_B$ であるような (a', b') はすべて C で同じ連結成分に属することがわかる．$a \in S_A, b \in T_B$ であるような (a, b) および，$a' \in T_A, b' \in S_B$ であるような (a', b') についても同様である．一方，$a \in S_A, b \in S_B$ であるような (a, b) および，$a' \in S_A, b' \in T_B$ であるような (a', b') については，S_A 中の 2 頂点間には偶数長のパスしかなく，S_B, T_B にまたがる 2 頂点間には奇数長のパスしかないことから，この 2 つは同じ連結成分に属さない．よって，この場合は連結成分の個数は 2 である．

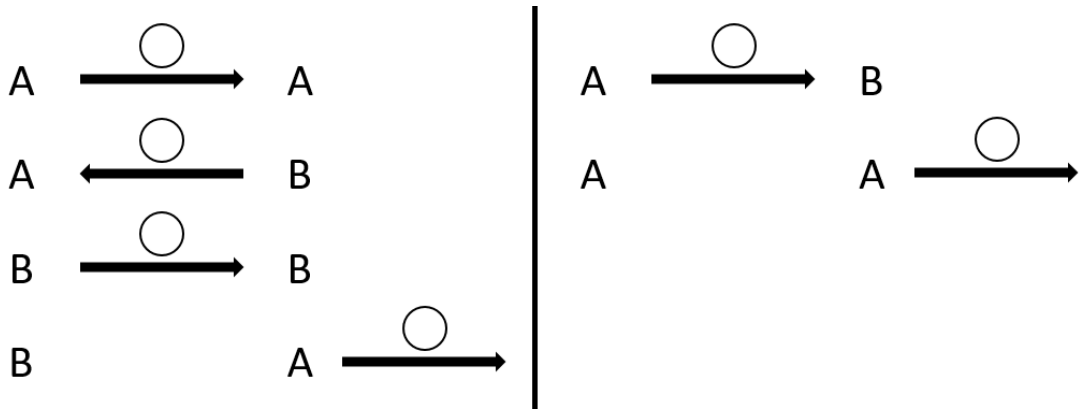
A, B が一般の場合には，連結成分のペアごとに問題を解いて，それぞれの連結成分の個数の合計を求めればよい．A, B の全頂点数を N_A, N_B，次数 0 の頂点数を i_A, i_B，二部グラフでない連結成分の個数を p_A, p_B，二部グラフである連結成分の個数を q_A, q_B とおく．このとき，連結成分のペアごとに考えることで，全体の連結成分の個数は

$$(i_A i_B + i_A(N_B - i_B) + i_B(N_A - i_A)) + p_A p_B + p_A q_B + q_A p_B + 2 q_A q_B$$

であることがわかる．これを計算するのは，グラフ連結成分を求め，それぞれの連結成分の頂点数が 0 か，また二部グラフかを判定するのが $O(V + E)$ で行えることから，$O(V + E)$ で行うことができる．

D: Half Reflector

1 回ボールを入れたときの装置の状態の変化を考える．まず，一番左の装置の状態が `A` であれば，この装置の状態が `B` になり，左端からボールが飛び出してくるだけである．一番左の装置の状態が `B` である場合を考える．ボールはこの装置を通過し，一番左の装置の状態が `A` になった状態で，次はこの装置の右端から飛び出してくる．さて，この「状態 `A` の装置の右端からボールが飛び出してくる」状況から，どのような状態変化が起こるかを考えると，下図のようになる．



この図から,「状態 A の装置 E の右端からボールが飛び出してくる」状況においては，E の状態は E の右の装置の元々の状態と反対の状態になり，E の右の装置が状態 A になって，その装置の右端からボールが飛び出してくる状態に変化することがわかる．さらに，装置 N は最終的には必ず状態 A になることもわかる．

以上をまとめると，1 回ボールを入れたときの装置の状態変化は，次のようになる．

- 一番左の装置の状態が A のときは，この装置の状態が B に変化する．それ以外の装置の状態は変化しない．
- 一番左の装置の状態が B のときは，$1, 2, \ldots, N-1$ 番目の装置の状態は，それぞれボールを入れる前における $2, 3, \ldots, N$ 番目の装置の状態と逆の状態になる．N 番目の装置の状態は，必ず A になる．

一番左の装置の状態が B のときの状態変化は，状態列を文字列をみなしたとき,「1 文字目を取り除き，他の文字については A, B を入れ替えた上で，末尾に A を加える」ことに対応する．ここで，文字列の偶数文字目の文字をあらかじめ反転させておくと，これは「1 文字目を取り除き，末尾に (N の偶奇に応じて) A または B を加える」ことと同じになる．この操作は deque を用いると $O(K)$ で行える．また，K が大きい場合も，$2N$ 回のシミュレートを行った後は状態の周期が 1 もしくは 2 になることを利用すると，$2N + O(1)$ 回で全体のシミュレートを完了できる．このアルゴリズムは $O(N)$ で動作し，十分高速に問題を解くことができる．

E: Increasing Numbers

10 進法において各桁がすべて 1 であるような数を **レピュニット** と呼ぶ．(i 桁からなる) i 番目のレピュニットを R_i で表すことにする．たとえば，$R_1 = 1, R_2 = 11, R_3 = 111$ である．便宜上 $R_0 = 0$ としておく．

整数 M が増加的であることと，M が高々 9 個のレピュニットの和で書けることは同値になる．実際,

- M が増加的なとき，i 以上の数が現れる桁のうち最も大きい箇所を D_i とすると (1 の位の位置を 1 とする．そのような箇所が存在しないときは $D_i = 0$ とする) $M = R_{D_1} + R_{D_2} + \cdots + R_{D_9}$ と表される．
- M が高々 9 個のレピュニットの和で書けるとき，$M = R_{D_1} + R_{D_2} + \cdots + R_{D_9}$ かつ $D_1 \geq D_2 \geq \cdots \geq D_9 \geq 0$ であるような整数 $D_1, \ldots, D_9$ が存在する．

よって，N が高々 k 個の増加的数の和で書けることと，N が高々 $9k$ 個のレピュニットの和で書けることは同値である．

任意のレピュニットは，非負整数 r に対して $\dfrac{10^r - 1}{9}$ と表される．この形で表される整数には 0 も含まれる．よって，N が高々 k 個の増加的数の和で書けるとき,

$$9N = \sum_{i=1}^{9k} (10^{r_i} - 1) = \sum_{i=1}^{9k} 10^{r_i} - 9k$$

を満たすような整数 $r_1, r_2, \ldots, r_{9k}$ が存在する．このような $9k$ 個の整数が存在するかどうかを知るには，$9N + 9k$ の各桁の数の和が $9k$ 以下であるかどうかを確認すればよい．

任意の整数は，$99\ldots9, 199\ldots9, \ldots, 999\ldots9$ のような形の増加的数を引くことで 1 桁小さい整数にできることに注意すると，L 桁の整数に対するこの問題の答えは高々 L である.「N が高々 k 個の増加的数の和で書けるか？」の判定は，上で示した方法を用いると $O(L)$ でできるので，k を二分探索で決定することによりこの問題は $O(L \log L)$ で解くことができる.

なお，この問題は $O(L)$ で解くこともできる．そのためには，$k = 1, 2, \ldots$ と小さい方から上の判定を順次行う．k が 1 増えたとき，$9N + 9k$ の値は 9 増える．多倍長整数に 9 増やす操作を最大 L 回行う必要があるが，繰り上がりが起きなくなった地点で加算処理を終了することで，これらの処理は全体で $O(L)$ で行うことができる．9 増やす際に，各桁の数の和も適切に更新することで，各 k における判定を効率よく行うことができ，$O(L)$ で問題が解ける.

いずれの解法においても，多倍長整数の実装が必要であるが，必要とされるものは「整数を 9 倍する処理」「各桁の数の和を求める処理」および，$O(L \log L)$ 解法では「int 型に収まる程度の正の整数を足す処理」，$O(L)$ 解法では「各桁の数の和を更新しながら 9 を足す処理」のみであり，比較的容易である.

F: Train Service Planning

どの列車も，反対方向の列車とすれ違いを行う位置は同じであることに注意する．駅 $0, N$ 発の列車をそれぞれ適当に選んで T, T' で表すことにする.

所要時間が $\frac{K}{2}$ より大きい単線区間が 1 個でも存在する場合，条件を満たすように列車を走らせることは不可能である．なぜならば，そのような区間では K 分の間に駅 $0, N$ 発の列車をそれぞれ，同時にこの区間にいることはないように走らせる必要があるためである．一方，そのような区間が存在しない場合は，条件を満たす列車の走らせ方が存在する．実際，駅 0 発の列車は駅 $0, 1, \ldots, N-1$ のすべてにおいて出発時刻を K の倍数とし，駅 N 発の列車は駅 $N-1, N-2, \ldots, 0$ のすべてにおいて到着時刻を K の倍数とするように列車を走らせればよい.

全区間が単線の場合

まず，全区間が単線の場合を考える．このとき，両方向の列車がすれ違うことができるのは駅においてのみである．列車 T がすれ違いを行う駅 (便宜上，駅 $0, N$ も含めて考える) を固定して考える．列車 T がすれ違いを行わない駅ですれ違いを行う列車は存在しないから，すれ違いを行う駅同士の間はまとめて 1 つの単線区間 (所要時間は，それらの駅の間に元々あった区間の所要時間の合計) とみなしてよい．すると，列車 T はすべての駅ですれ違いを行うとみなすことができる．このとき，列車 T' もすべての駅ですれ違いを行う.

所要時間の和を最小にする列車の走らせ方の中に，列車 T の停車時間がどの駅でも 0 であるようなものが存在することを示す．列車 T のある駅 i の停車時間が T_i $(T_i > 0)$ であるとき，次の操作を行うことができる.

- 駅 0 から駅 N へのすべての列車の，駅 $i, i+1, \ldots, N-1$ の出発時刻および駅 $i+1, i+2, \ldots, N$ の到着時刻を T_i 早くする.
- 駅 N から駅 0 へのすべての列車の，駅 $N, N-1, \ldots, i+1$ の出発時刻および駅 $N-1, N-2, \ldots, i$ の到着時刻を T_i 早くする.

条件を満たす列車の走らせ方に対してこの操作を行っても，相変わらず条件は満たされており，また所要時間の和は変化しない．一方，列車 T の駅 i での停車時間は 0 になり，この列車の他の駅での停車時間は変化しない．よって，所要時間の和を最小にする適当な運転計画から始めて，この操作を列車 T の停車時間が 0 より大きい駅すべてに対して行うことで，所要時間の和が最小かつ列車 T の停車時間がどの駅でも 0 であるような列車の走らせ方を得ることができる.

次に，列車 T の停車時間が 0 であるような列車の走らせ方において，列車 T' の所要時間を考える．先程の操作と同様にして，列車 T' は駅 s $(s = N-1, N-2, \ldots, 1)$ において，すれ違いを行う駅 0 発の列車が発車するのと同時に発車するとしてよいことがわかる．さらに，列車 T' の駅 $N-1$ への到着は，駅 $N-1$ の出発時刻と同時としてよいこと

もわかる．よって，区間が端から $A_1, A_2, \ldots, A_N$ であり，列車 T が各駅ですれ違いを行うときの列車の所要時間の和は，次で表される：

- $N = 1$ のときは，$2A_1$
- $N \geq 2$ のときは，$2A_1 + K(N-2) + 2A_N$

すれ違い駅を決めたときの所要時間の和がわかったので，次にすれ違い駅を決める方法を考える．駅 0 から駅 i までの区間でうまくすれ違い駅を決めたときの，「(最初の区間の所要時間の和) $\times 2 +$ (区間の数 $- 1$) $\times K$」の最小値を X_i とする．各 i について X_i が計算できれば，駅 i から駅 N まですれ違いなしで行くときの所要時間の和の最小値が計算でき，結果として答えを求めることができる．

まず，駅 0 から駅 i までの所要時間の和が $\frac{K}{2}$ 以下であれば，X_i はこの所要時間の和の 2 倍に等しい．さもなくば，駅 j から駅 i までの所要時間の和が $\frac{K}{2}$ 以下であるような最小の j に対して，$X_i = \min_{j \leq i' < i}(K + X_{i'})$ となる．このような j は尺取法などにより容易に計算できるので，X_i を順次 Segment Tree を用いた動的計画法で計算することができる．ところで，X_i は広義単調増加である．これは，駅 0 から駅 i までの途中のすれ違い駅の決め方は，そのまま駅 $i-1$ までのすれ違い駅の決め方として使うことができることからわかる．よって，$X_i = \min_{j \leq i' < i}(K + X_{i'})$ とするかわりに，$X_i = K + X_j$ としてしまってかまわないことがわかる．

以上より，$i = 0, 1, \ldots, N$ に対して X_i を $O(N)$ で決定できることがわかったので，全区間単線の場合を $O(N)$ で解けることがわかった．

複線区間が存在する場合

複線区間では，区間の途中でも自由にすれ違いを行うことができる．これは，途中に勝手に駅を設置することができる (ただし，区間の所要時間の合計は保たれなければならない) ような単線区間と同等と思うことができる．

このもとで，先程の X_i を計算することを考える．X_i に相当する値を，仮想的に複線区間の途中でも考えることにすると，この値は駅 0 からの所要時間の合計に対して広義単調増加である．つまり，直前のすれ違い地点を決めるときには，相変わらずできるだけ現地点から遠い地点を選ぶべきである．

X_i を計算するときに，駅 i から所要時間の和が $\frac{K}{2}$ であるような手前の地点が単線区間の途中であった場合には，前のすれ違い地点は (先程と同様に) その単線区間の終わりとする．一方，この地点が複線区間の途中であった場合には，その地点をすれ違い地点として，さらに $\frac{K}{2}$ だけ遡って，すれ違い地点が駅となる地点を探す．つまり，所要時間の和を $\frac{K}{2}$ ずつ遡っていったとき，初めてぶつかる単線区間の途中はどこであるか，を求めたい．これは，「最後に出てきた単線区間の位置」を $\bmod \frac{K}{2}$ で順次更新していけば求められる (実際には，所要時間のほうをすべて 2 倍したほうが簡便である)．この操作は，C++ の set などを使えばならし $O(\log N)$ で実装できる．

複線区間が存在する場合，最も駅 N に近いすれ違い地点が複線区間の途中になる可能性があるため，X_i を計算しただけでは答えを求めることができない．しかし，両方向の列車の所要時間の和を最小にしつつ，すれ違い地点が駅に一致するような箇所は存在するとしてよい．これは，駅 0 から駅 N への列車の時刻すべてを同様に，連続的に大きくしていくと，すべてのすれ違い地点が少しずつ駅 0 側に移動し，時刻をちょうど K ずつ増やしたときにはすれ違い地点は最初と同じになっていることからわかる．(この間に，すれ違い地点が単線区間の途中になって条件を満たさなくなる可能性があるが，最初に条件を満たさなくなる瞬間の直前には，そのすれ違い地点はちょうど駅に一致していたはずである．) よって，駅 i から駅 N までの区間でうまくすれ違い地点を決めたときの，X_i と同様の値の最小値を Y_i としたとき，答えは $\min_{i=0,1,\ldots,N}(X_i + Y_i)$ として求められる．この Y_i の計算は X_i とまったく同様に行うことができる．

A: Airport Bus

Suppose that b people take bus B and b' people take bus B'. If some passenger on bus B' arrives at the airport after some passenger on bus B, we can assume that all passengers on bus B' arrive at the airport after all passengers on bus B. (Otherwise, among those $b + b'$ people, we can assign the first b people to bus B and the last b' people to bus B').

Therefore, we should repeat the following: "Choose the first person who haven't taken bus yet, create a new bus, put the person to the new bus, and assign the largest possible number of people to the new bus (in the increasing order of arriving time)".

First, sort the people by T_i, and assign people to buses from left to right. Let i be the smallest index that is not assigned to a bus yet. Then, we should find the maximum j such that $T_j - T_i \leq K$ and $j - i + 1 \leq C$, and assign them to the bus. (The next i will be $j + 1$). We can get the answer by repeating this process. This solution works in $O(NlogN)$.

B: Colorful Creatures

Suppose that $A_1 \leq A_2 \leq \cdots \leq A_N$. Let t be the largest integer k such that $2\sum_{i=1}^{k} A_i < A_{k+1}$. (If such k doesn't exist, $t = 0$.) We claim that the last color will be one of $t+1, \ldots, N$.

If one of $1, 2, \ldots, t$ wants to survive, its size must be at least $\frac{A_{t+1}}{2}$ (and the eats $t+1$). However, even if all of $1, 2, \ldots, t$ merges, its size doesn't reach $\frac{A_{t+1}}{2}$. On the other hand, each of $t+1, t+2, \ldots, N$ can survive by eating in the increasing order of size.

Thus, we should compute such t and the answer is $N - t$.

C: Squared Graph

In general, consider the following problem:

> You are given two graphs $A = (V_A, E_A), B = (V_B, E_B)$. Define a graph C: the set of vertices are $V_A \times V_B$ and there is an edge between $(a, b), (a', b')$ iff there is an edge between a, a' in A and there is an edge between b, b' in B. Compute the number of connected components in C.

(The original problem corresponds to the case $A = B$).

First, suppose that both A and B are connected.

If $|V_A| = 1$, the answer is obviously $|V_B|$. Similarly, if $|V_B| = 1$, the answer is obviously $|V_A|$.

Otherwise, each graph contains at least two vertices. There is an edge between $(a, b), (a', b')$ in C iff there is a path of legnth l between a, a' in A and there is a path of legnth l between b, b' in B for some integer l. Since we can easily extend the length of paths by two, we are only interested in parities.

If both A and B are bipartite, there will be two connected components in C. (If V_A are divided into S_A, T_A and V_B are divided into S_B, T_B in bipartite coloring, the two connected components are $S_A \times S_B \cup T_A \times T_B$ and $S_A \times T_B \cup T_A \times S_B$).

Otherwise, C will be connected because we can always find a path with given parity between two vertices of non-bipartite graph.

When A, B are not connected, we can solve the problem for each pair of connected components and compute the sum. Let N_A, N_B be the number of vertices, i_A, i_B be the number of isolated vertices, p_A, p_B be the number of non-bipartite components, q_A, q_B be the number of bipartite components (except for isolated vertices). Then, the answer is:

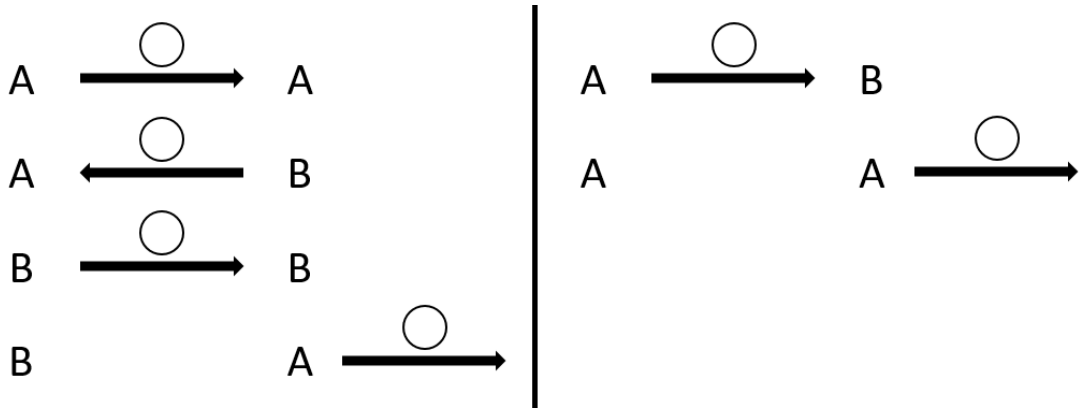
$$(i_A i_B + i_A(N_B - i_B) + i_B(N_A - i_A)) + p_A p_B + p_A q_B + q_A p_B + 2 q_A q_B$$

This solution works in $O(M)$.

D: Half Reflector

First, let's see what happens when the operation is performed only once. If the leftmost device is in state `A`, the state of this device will be `B` after the operation and nothing else happens. If the leftmost device is in state `B`, the ball first changes the state of the leftmost device to `A` and then enters the second device from the left side.

See the following picture (the left picture shows the case where the second device is in `A`, the left picture shows the case where the second device is in `B`):



From this, you see that the state of the first device after the operation is the negation of the state of the second device before the operation. Then, the ball will enter the third device and similar changes will happen. Finally, the ball leaves the contraption from the right side of the device N, and the state of this device always end with `A`.

Here is the summary:

- If the leftmost device is in state `A`, it will change into `B` and the other devices won't change.
- If the leftmost device is in state `B`, the states of devices $1, 2, \ldots, N-1$ after the operation is the negation of the states of devices $2, 3, \ldots, N$ after the operation, respectively. The state of the device N is always `A`.

In the latter case, we can see the operation as "delete the first character, negate all characters, and append `A` at the end". If we negate all characters at even indices beforehand, the only operations we need to do is "delete the first character and append a character `A` or `B` (depending on the parity of N)". This can be done in $O(1)$ using a deque.

This way the problem can be solved in $O(K)$. Also, notice that after $2N$ operations, the strings we get after each operation will be periodic, and the period is at most 2. Thus, we need only $O(N)$ simulations and this algorithm works in $O(N)$.

E: Increasing Numbers

An integer is called *Repunit* if it only consists of ones in its decimal representation. For example, $1, 11, 111, ...$ are Repunit.

It is easy to see that an integer is increasing if and only if the integer can be represented as the sum of at most 9 Repunit numbers. Thus, an integer N can be written as the sum of at most k increasing numbers if and only if N can be written as the sum of at most $9k$ Repunit numbers.

A Repunit number can be written of the form $(10^r - 1)/9$ using a non-negative integer r. Suppose that

$$N = \sum_{i=1}^{9k} (10^{r_i} - 1)/9$$

This is equivalent to

$$9N + 9k = \sum_{i=1}^{9k} 10^{r_i}$$

In order to check if such $r_1, \ldots, r_{9k}$ exist, we need to check if the sum of digits of the decimal representation of $9N + 9k$ is at most $9k$.

Let L be the number of digits of N. We can prove that the answer of the problem is at most L (always choose the greatest integer of the form $99\ldots9, 199\ldots9, \ldots, 999\ldots9$ greedily and subtract it from N). Thus, by using binary search, this problem can be solved in $O(L \log L)$.

It is also possible to solve this problem in $O(L)$. Instead of using binary search, compute the decimal representations of $9N$, $9N + 9$, $9N + 18$, ... one by one. The time complexity of one addition can be $O(L)$, but since the big carries will not happen after a big carry, its amortized complexity is $O(L)$.

We need big integers, but the required operations are extremely simple: we only need to increment big integers.

F: Train Service Planning

Consider a train that goes from station 0 to station N. For this train, define $p_0, \ldots, p_{N-1}$ as follows:

- The train departs station 0 at time p_0.
- After A_0 minutes, it arrives at station 1.
- The train waits for p_1 minutes there.
- After A_1 minutes, it arrives at statopn 2.
- The train waits for p_2 minutes there, and so on.

Of course, $p_1, \ldots, p_{N-1}$ must be non-negative.

Then, consider a train that goes from station N to station 0 in a strange way. We assume that this train goes from station 0 to station N, but spends negative amount of time. Define $q_0, \ldots, q_{N-1}$ as follows:

- The train departs station 0 at time $-q_0$.
- After $-A_0$ minutes, it arrives at station 1.
- The train waits for $-q_1$ minutes there.
- After $-A_1$ minutes, it arrives at statopn 2.
- The train waits for $-q_2$ minutes there, and so on.

Again, $q_1, \ldots, q_{N-1}$ must be non-negative.

What happens if the section s is single-tracked?

- One train leaves station $s-1$ at time $A_0 + \ldots + A_{s-1} + p_0 + \ldots + p_{s-1}$ and arrives at station s at time $A_0 + \ldots + A_s + p_0 + \ldots + p_{s-1}$.
- The other train "leaves" station $s-1$ at time $-(A_0 + \ldots + A_{s-1}) - (q_0 + \ldots + q_{s-1})$ and "arrives" at station s at time $-(A_0 + \ldots + A_s) - (q_0 + \ldots + q_{s-1})$.

The constraint on this single-tracked section is as follows: The value $p_0 + \ldots + p_{s-1} + q_0 + \ldots + q_{s-1}$ is not in the (open) interval $(-2(A_0 + \ldots + A_s), -2(A_0 + \ldots + A_{s-1})$, modulo K. Formally, no number in this open interval is equivalent to $p_0 + \ldots + p_{s-1} + q_0 + \ldots + q_{s-1}$ modulo M. Note that when $2A_s$ is greater than K, this constraint is unsatisfiable and you need to output -1.

Intuitively, the constraint only depends on the total waiting time in the stations $[0, s)$ (of two trains). Let $x_i = p_0 + \ldots + p_{s-1} + q_0 + \ldots + q_{s-1}$ (i.e., the total waiting time). The problem asks to find the minimum possible value of $x_{N-1} - x_0$ of non-decreasing sequence $x_0, \ldots, x_{N-1}$ under the constraints above.

Now, we can state a simpler general version of this problem: (Note that after the reduction this N corresponds to the number of single-tracked sections.)

You are given a modulo M and N intervals $[L_i, R_i]$. You are asked to find a non-decreasing sequence $x_0 \leq x_1 \leq \ldots \leq x_{N-1}$ that minimizes the value $x_{N-1} - x_0$, under the constraint that for each i, x_i must be in the interval $[L_i, R_i]$, modulo M. (Formally, there must be $L_i \leq y_i \leq R_i$ such that $y_i \equiv x_i \pmod{M}$.

The easier understanding of this problem is as follows. Suppose that someone is standing on a number line. He can move to the right at arbitrary speed (or he can stay at the same place), but he is not allowed to go left. At time i, he must be in the interval $[L_i, R_i]$, modulo M. What is the minimum distance he must travel?

Now, it is easy to prove that we are only interested in "important coordinates" $L_0, \ldots, L_{N-1}, R_0, \ldots, R_{N-1}$. We can assume that in one of optimal solutions all x_i are one of these values. Any reasonable DP that uses this fact will run in polynomial time and get 500 points.

How can we solve this problem faster? First we define values $dpL[i], dpR[i]$. $dpL[i]$ means the following: "Suppose that you are at L_i at time i. What is the minimum distance you have to travel in the future (to satisfy the constraints)? Define $dpR[i]$ similarly.

Unless you are forced to do so, it makes no sense to move. Instead, you can move to the right later when you are forced. Thus, we can compute the value of $dpL[i]$ in the following way. First, find minimum j such that $j > i$ and $L[i]$ is not in the interval $[L[j], R[j]]$ modulo M. (If such j doesn't exist, $dpL[i] = 0$. Then, $dpL[i] =$ (the distance you have to move from $L[i]$ to $L[j]$) + $dpL[j]$. The computation of dpR is similar. If you compute these values in the decreasing order of i, and if you use RMQ (Range Minimum Query) to find such j, it works in $O(NlogN)$.

How to get the answer? The optimal solution will look like one of the following:

- Stay at L_i until time i, and then move $dpL[i]$ in total. This is valid only when L_i is in the interval $[L_j, R_j$ for each $j < i$.
- Stay at R_i until time i, and then move $dpR[i]$ in total. This is valid only when R_i is in the interval $[L_j, R_j$ for each $j < i$.

Thus, there are only $2N$ possibilities. We can check the validity of each of these possibilty in $O(NlogN)$, again using RMQ. Therefore, this solution works in $O(NlogN)$.

AGC 10 解説

yutaka1999

2017 年 2 月 4 日

For International Readers: English editorial starts on page 4.

A : Addition

偶奇が等しい 2 つの数を足すと、その和は偶数になる。よって、$N \geq 2$ より、最後に残る数は偶数でなくてはならない。

逆に、和が偶数ならば、もとの数の中に奇数は偶数個あることになるので、奇数を 2 つずつのペアにしてその和に置き換えることで、全ての数を偶数にできる。このとき、どのような順番でも数は 1 つにできるので十分である。

よって、合計が偶数かどうかの判定ができればいいので、$O(N)$ で解くことができる。

B : Boxes

一回の操作で取り除かれる石の個数はちょうど $\frac{n(n+1)}{2}$ 個であるので、はじめにある石の個数から、操作をする回数が分かる。これを k とすると、ちょうど k 回操作を行って、全ての数が等しくなるようにできるかを判定すればよい。

ここで、$d_i = A_{i+1} - A_i$ の変化を考えると、一回の操作で $\{d_i\}$ の内、ちょうど一つが $+(n-1)$ され、その他が -1 される。よって、$\{d_i - k\}$ のうち毎回一つを $+n$ するとして、全て 0 にできるか判定すればよい。

これは、各 d_i が負であり、かつ n の倍数であることが必要十分であるから、$O(N)$ で判定可能である。($\sum d_i = 0$ であることを用いれば、必要な $+n$ の回数が必ず k 回であることが分かる。)

C : Cleaning

適当な頂点 r を根として、根付き木として考える。さらに、根以外の各頂点に対して、その頂点とその親を結ぶ辺を通る操作の回数を通る回数と呼ぶことにする。このとき、明らかに各葉に対しては通る回数が分かる。ここで、葉でない頂点 v について、v の子での通る回数が分かっているときに、v の石を全て取り除けるかどうかと、その頂点の通る回数を調べる。

各子の通る回数を $c_1, c_2, ..., c_k$ と置くと、子を通る操作は v と v の親と通るパスを通るか、v で他の子を通る操作とつながるかの 2 通りで、どちらも v から石を 1 つ取り除くから、前者のパスの個数、つまり、v の通る回数を T と置くと、$T + \frac{\sum c_i - T}{2} = A_v$ となる。よって、この情報から、T の値が分かるので、後は異なる子を通るパス 2 つをペアにしたものを $\frac{\sum c_i - T}{2}$ 個作れるかどうかを判定すればよい。

つまり、$L = \frac{\sum c_i - T}{2}$ として、1 から k までの各数 i が c_i 回まで使えるときに、ペアを L 個作れるかとい

う判定ができればよく、そのようなペアは最大で $\min\left(\max c_i, \frac{\sum c_i}{2}\right)$ 個であることが示せるので、この判定は容易にできる。

以上より、各頂点に対して、その子の情報から今の頂点の情報を処理することができたので、根付き木の下の頂点から順に処理していくことで $O(N)$ で解くことができる。

D : Decrementing

毎回の操作で後者の操作を行わない、つまり、GCD で割らないときのことを考える。このとき、$\sum (A_i - 1)$ の偶奇で勝敗が決まるので、同様にして、偶奇に注目して勝敗を調べる。

まず、A_1 から A_n の中に奇数があれば、その GCD も奇数なので、GCD で割った後も各数の偶奇は入れ替わらない。さらに、A_1 から A_n の GCD が 1 のとき、少なくとも一つ奇数がある。この性質を用いると、以下のように場合分けできる。

i)　A_1 から A_n の中に偶数が奇数個あるとき
偶数が一つ以上あるため、偶数を一つ選んで -1 し、相手に渡す。このとき、操作前に奇数が 1 つ以上あり、操作後には奇数が 2 つ以上あるため、相手の操作時の GCD も奇数となる。よって、相手から戻ってくるならば、その時も偶数が奇数個あるため、先手はこれを繰り返すことで必勝である。

ii)　A_1 から A_n の中に偶数が偶数個あるとき
奇数が 2 つ以上あるならば、どうやっても相手に渡したときに偶数が奇数個あるため、i) より、後手必勝になる。ただし、奇数がちょうど 1 つあるときは、その奇数を消すことで、GCD が偶数になるため、後手必勝とは限らない。よって、この場合は、後手必勝になるか操作が一意に定まるかのいずれかなので、これを繰り返すことでどちらが必勝かわかり、GCD が偶数になる回数は高々 $O(\log(\max A_i))$ 回なので、十分高速に動く。

以上より、$O(N \log(\max A_i))$ でこの問題を解くことができる。

E : Rearranging

まず、後手の最適手を考える。$A_1, A_2, ..., A_N$ の順に並んでいるとする。互いに素な数のみが順序を入れ替えられるため、$\gcd(A_i, A_j) \neq 1$ であれば、これらの相対的な順序は変化しない。

よって、$i < j, \gcd(A_i, A_j) \neq 1$ となる (i, j) に対して、$i \to j$ と有向辺を張ることで DAG ができ、このトポロジカル順序で辞書順最大のものが求めるものとなる。これは簡単に求められるので、後手の最適手は簡単に求められる。

よって、先手の最適な順序を求めればよい。自然数からなる集合 S に対して、各要素間に GCD が 1 でないなら辺を張る、というようにして作ったグラフを $g(S)$ とおき、$g(S)$ が連結グラフであるような集合を連結集合と呼ぶことにする。このとき、先の後手の動きからもわかるように、各連結集合ごとの順序が定まれば後手の差す手は決まるので、それぞれ独立に考えてよい。

ここで一般化して、「連結集合 S に全順序を定めて後手の操作の後に辞書順最小になるようにする。ただし、後手がどのように操作しても先頭が X と互いに素となってはならない。」という問題を考える。この答えとなる順序の一つを $f(S, X)$ と置く。

まず、X との GCD が 1 でない元の中で最小の元を x とする。このとき、明らかに先頭は x 以上でなくてはならないが、$g(S)$ が連結なため、x を始点とする DFS 順序にそってほかの要素を並べることで、後手が最適に行っても、先頭を x のままにすることができる。よって、$f(S, X)$ の先頭は x としてよい。このとき、$S \backslash \{x\}$ の各連結集合 T での順序が求まれば、それを適当につなぎ合わせることで $f(S, X)$ が得られるため、その順序を考えることにする。

まず、T を並べたとき、その先頭が y であるとする。$(x, y) = 1$ ならば、x と y を入れ替えることができるため、明らかに不適である。逆に、$(x, y) \neq 1$ ならば、どのようにしても x と y の順序は入れ替えらないため、T の並びとしては、先頭が x と互いに素ではない並びで最適な並び、つまり、$f(T, x)$ が最適であることが分かる。

以上をまとめると、$f(S, X)$ は X との GCD が 1 でない元で最小の元 x を先頭にし、S から x を取り除いたときの各連結成分 T に対する $f(T, x)$ を適当に連結したものとすればよい。

求める先手の順序は $S = \{A_1, A_2, ..., A_n\}$（ただし、$S$ は multiset として考える）として、$f(S, 0)$ であるから、先頭の元を取り除く操作が N 回、各連結成分を求める操作が $O(N \log N)$ 程度で可能なので、全体として $O(N^2 \log N)$ で解くことができる。

ただし、連結成分を求める際は、あらかじめすべての数の素因数を計算し、数と素因数の間に辺を張ってグラフを構築するものであるから、その各数の素因数の計算に $O(\sqrt{\max A_i})$ かかる。よって、全体としては、$O(N(N \log N + \sqrt{\max A_i}))$ となる。

F : Tree Game

最初に高橋君が駒を置く頂点を一つ固定する。このとき、各頂点について $O(N)$ で解ければよい。ここで、木は二部グラフであるから、どの頂点に対してもそこから石をとるプレイヤーは一意に定まることに注意する。

その選んだ頂点を根として、木を根付き木として考える。このとき、以下のように再帰的に各頂点 v に対して $f(v)$ を定める。

- v が葉であるとき、$f(v) = A_v$
- v が葉でないとき、v の直接の子での $f(v)$ が全て A_v 未満なら、$f(v) = \inf$
  そうでないなら、$f(v) = A_v$

このとき、葉でない頂点 v に対して、v を根とする部分木で v に駒を置いて始めたゲームの勝敗と $f(v) = \inf$ かどうかが一致することを帰納的に示す。

まず、v の直接の子がすべて葉の時、$f(v) = \inf$ ならば、ある子 c で $A_c < A_v$ となる。このとき、先手は v から常に c に動かすことで勝つことができる。逆に、$f(v) = A_v$ の時は、どの頂点 c に対しても $A_c \geq A_v$ となるため、先手はどのようにしても勝つことができない。よって、この場合は示された。

次に、v のすべての子で f の値が定まっているときを考える。このとき、$f(u) = \inf$ である頂点 u に動かすと後手が勝ちになるため、このような頂点 u に先手は動かすことができない。さらに、そうでない頂点 u に対しては、先手が u に動かしたときに後手は v に動かすのが最適である。なぜならば、後手が u の子に動かすとき、u を根とする部分木だけ動かす場合は先手勝ちであるから、後手が勝つには途中で u から v へ戻る必要がある。しかし、結局 u を根とする部分木において、後手陣地である u の石が減るだけであるから、また v から u へ戻ったとしても、その部分木内だけ動かした場合は先手勝ちであることに変わりない。よって、u からその子へ行ったとしても、u の石の個数を減らすだけで、結局 v に戻る必要があるため、意味がない。よって、後手は u から v に動かすのが最適である。

以上より、v のすべての子で f の値が定まっている場合も、v の子がすべて葉である場合と同じように処理することができるため、正当性が示された。

さて、先のような f の値は根付き木を根から再帰的に調べることで $O(N)$ で求めることができるため、根をすべて試すことで、$O(N^2)$ で解くことができる。

AGC 10 Editorial

yutaka1999

February 4th, 2017

A : Addition

Suppose that after the operations you have only one integer. This integer is obtained as a sum of two integers of the same parity, so it must be even. This means that the sum of A_i is even.

On the other hand, if the sum of A_i is even, we can prove that the answer is always "YES". In this case initially there are even number of odd integers. Divide them into pairs and perform the operation for each pair. This way all integers will be even and after that we can add any pair of integers.

Thus, we just need to check whether the sum of A_i is even, and it can be done in $O(N)$.

B : Boxes

In each operation we remove $\frac{n(n+1)}{2}$ stones. Let s be the total number of stones. Obviously, s must be a multiple of $\frac{n(n+1)}{2}$. Let $k = s/\frac{n(n+1)}{2}$. We want to check whether after exactly k operations, the number of stones in each box can be the same.

Let $d_i = A_{i+1} - A_i$, the difference of number of stones in adjacent boxes. Usually an operations reduces this number by 1, but one of the values of $\{d_i\}$ is incremented by $n-1$. It means that for some non-negative integer x, $d_i - (k-x) + (n-1)x = 0$ or $k - d_i = nx$ (here x is the number of "unusual" operations happened for d_i). Thus, for each i, $k - d_i$ must be non-negative and it must be a multiple of n. We can also prove that this condition is sufficient: the sum of $(k-d_i)/n$ is always k because the sum of d_i is zero. We can solve this problem in $O(N)$ time.

C : Cleaning

Let's assign labels to the edges. Initially all the edges are labeled with zero. When an operation is performed on a path connecting two different leaves, we increment the labels of all edges on the path by one.

When can we remove all stones? Each leaf vertex on a path is incident to an edge on the path, and each non-leaf vertex on a path is incident to two edges on the path. Thus, we can see that this is equivalent to the following:

- For each leaf v, the label assigned to the only edge incident to v is A_v.
- For each non-leaf vertex v, the sum of labels assigned to the edges incident to v is $2A_v$.

Now, consider the tree as a rooted tree. From the conditions above, we can uniquely determine the labels of all edges in the order from leaves to the root. Note that this is not always possible - make sure to check that all labels are non-negative and the condition is satisfied at the root.

In case we can successfuly assign labels, how can we check whether we can decompose it into a set of paths connecting leaves? Consider a certain non-leaf vertex, and let $c_1, \ldots, c_k$ be the labels assigned to these edges. These $\sum c_i$ things must be separated into pairs, and each pair must consist of two things from different edges. Thus, the condition is that, $\sum c_i$ must be even and for each i, $c_i \leq \sum c_i/2$ must be satisfied.

The solution above works in $O(N)$ time.

D : Decrementing

First consider an easier version of problem: we don't divide the integers by their GCD. In this case, obviously the answer only depends on the parity of the total number of stones. The original problem also has something to do with parities.

Suppose that in your turn all integers are odd. If all integers are one, you can't perform any operations and you lose. If not, you can perform an operation, and after the operation exactly one integer is even and the others are odd (here the GCD doesn't matter - since GCD is odd in this case, it doesn't change the parities). However, if your opponent perform an operation on the only even number, you will again face the situation where all numbers are odd, and you will lose eventually.

If exactly one integer in your turn is even, you can always win. By performing an operation in the only even number, you can force the opponent to lose (as described above).

By similar observations, we get the following:

1. In case there are odd number of even integers, you win. Your strategy is as follows. In your turn you choose one of even numbers and perform an operation on it. After the operation there will be two or more odd integers, so the GCD operation doesn't change the parities. Thus, in the opponent's turn there will be even number of even integers, and in your next turn you will again have odd number of even integers. By repeating this strategy, you always win.
2. In case there are even number of even integers and two or more odd integers, you lose. No matter what you do, after your operation the opponent will be given a winning state, as described above.
3. In case there are even number of even integers and exactly one odd integer. In this case, it's clear that if you perform an operation on one of even integers, you lose (your opponent will face a winning state). Thus, you must perform an operation on the odd integer, and after that divide all integers by their GCD. It's not clear what happens in this case - just simulate the game and solve the problem recursively.

When the simulation happens, all the integers are divided by at least two. Thus, the number of simulations is $O(\log(\max A_i))$. The time complexity of the algorithm above is $O(N \log(\max A_i))$.

E : Rearranging

Let's think about the second player's optimal strategy first. He wants to maximize the permutation by swapping two adjacent coprime elements. If two numbers x and y are in the permutation in this order, and if they are not coprime, he can never change the relative order of these elements. We can also prove that this condition is sufficient: he can get any permutation that satisfies the condition above by repeating operations.

For convenience, assume that the input is sorted ($A_1 \leq \cdots \leq A_N$). Construct a graph with N vertices. There is an edge between vertices i and j if A_i and A_j are not coprime. After the first player decides his arrangement, each edge in this graph is directed. The second player wants to find the greatest topological sort of this directed graph.

Next, let's think about the first player's optimal strategy. From the observations above, we can handle each connected component independently. When we merge two connected components, we first get two sequences from each connected component, and find the greatest sequence we can get by merging the two sequences. From now, we assume that the graph is connected.

Let x be the smallest element in the permutation. The first player can force that, even after the second player's operations, the first element of the permutation becomes x. For example, take an arbitrary spanning tree of the graph. Arrange the numbers according to the DFS-order of this spanning tree. Then, each element is topologically greater than x in the directed graph, so the second player can't change the first element. Threrefore the first player should put x first.

Then, remove x from the graph and again compute connected components. For the same reason as above, we can consider each connected component independently. The only difference is that, if y comes first in a connected component, y must be non-coprime to x (i.e., x and y must be adjacent in the graph). Otherwise the second player can swap x and y and make the permutation greater. We do this recursively in the same manner.

How can we implement it efficiently? Run dfs in the graph and construct DFS-trees of the graph. When there are multiple adjacent vertices, visit the smallest vertex first. This way, we get a DFS-tree with the following properties:

- Each edge connects an ancestor and a descendant (like all DFS-trees).
- x is the smallest element in the subtree rooted at x.
- After we remove x, we can get new connected components as the subtrees rooted at x's children.

This is exactly what we need to compute. After computing this DFS tree, we assign a

sequence to each vertex. In order to compute the sequence assigned to vertex x, first we find the lexicographically greatest sequence that can be obtained by merging sequences assigned to x's childrem. Then, append x to the front of this sequence, and this is the sequence we want to assign for x. We compute this from leaves to the root, and the sequence assigned to the root is the answer of the problem.

The complexity of this algorithm is $O(N^2 \log \max A_i)$.

F : Tree Game

Suppose that Takahashi initially puts the piece on vertex r. Let r be the root of the tree. For each vertex v, we define $state(v)$ as follows:

Consider a subtree rooted at v. The two players play the game using this subtree (they are not allowed to move the piece out of the subtree), and initially the piece is at vertex v. If the first player can win in this game, define $state(v) = W$. Otherwise $state(v) = L$.

Note that $state(r)$ gives the result of the entire game when the piece is initially at r.

We claim the following:

1. If there exists a child c of v such that $A_c < A_v$ and $state(c) = L$, then $state(v) = W$.
2. Otherwise, $state(v) = L$. (In particular, if v is a leaf, $state(v) = L$.)

The proof of 1. Suppose that this is your turn and the piece is currently at v. First you take a stone from v (this is possible because A_v is not zero) and move the piece to c. Whenever the opponent tries to move the piece from c to v, you can refuse to do that by moveing it back to c (this is possible because $A_v > A_c$). This way, your opponent is forced to play the game within the subtree rooted at c. Since $state(c) = L$, your opponent loses and you win. (Strictly speaking, your opponent can also reduce the value of A_c by moving the piece between c and v back and forth, but this doesn't change the state of vertex c.)

The proof of 2. If v is a leaf, you can't move the piece and you lose. Otherwise you can move the token from v to one of v's children, w. There are two cases: $state(w) = W$ or $A_w \geq A_v$. If $state(w) = W$, your opponent never moves the piece back to v and play the rest of the game completely within the subtree rooted at w. Since $state(w) = W$, this way your opponent wins and you lose. If $A_w \geq A_v$, your opponent refuses to move the piece from v to c by moving it back to v (this is possible because $A_w \geq A_v$). Thus, you lose in this case.

This way, for a fixed position of the initial piece, we can solve the problem in $O(N)$. In total the complexity of this solution is $O(N^2)$. (Exercise: can you improve it to $O(N)$?)

AGC 009 解説

DEGwer

2017 年 1 月 22 日

For International Readers: English editorial starts on page 6.

A: Multiple Array

数列の最後の要素の値を変える方法は、最後のボタンを押す以外ありません。つまり、最後のボタンを押すべき回数が $modB_N$ で定まり、すなわち最後のボタンを押すべき最小回数がわかります。最後から 2 番目の要素の値を変えるためには、最後のボタンか最後から 2 番目のボタンを押すしかありません。最後のボタンを押す最小回数が求まっているので、最後のボタンと最後から 2 番目のボタンを押す回数の合計の最小の値も求まります。

数列を後ろから順に見て、以下同じことを繰り返していく貪欲法を行えば、この問題を解くことができます。計算量は $O(N)$ です。

B: Tournament

人 x が y に負けたとき、y の親を x としたような木上での DP を考えます。

$DP[v]$ を、人 v に対応する頂点を根とした部分木だけに人を限定して考えたときのトーナメントの深さの最小値とします。求めるべきものは $DP[1]$ です。

では、更新はどのように行えばいいでしょうか。v に子が k 個あるとします。この k 個の子に対応する人たちはみな v に直接負けています。つまり、v は v の子たちと、1 回ずつ試合をしています。

v がその子たちと、$v_1, ..., v_k$ の順に試合をしたとすると、v_i を根とする部分木で最もトーナメント上で深い位置にいる人は、v を根とする部分木に含まれる人たちの中で優勝をするためには、$DP[v_i] + i$ 回の試合をする必要があります。つまり、$DP[v]$ の値は、すべての子の並べ方に対する、$max_i DP[v_i] + i$ の最小値です。

あとは、v の子に対する DP 値が与えられたときに、上述の値を求められればよいです。これは、もし $i < j$ に対し $DP[v_i] \leq DP[v_j]$ なら $max(DP[v_i]+i, DP[v_j]+j) \geq max(DP[v_i]+j, DP[v_j]+i)$ なので、子の DP 値を降順にソートして順に $v_1, .., v_k$ とする貪欲法で求めることができます。

計算量は全体で $O(NlogN)$ となります。

C: Division into 2

$A \leq B$ として一般性を失いません。$A > B$ なら、 A と B を swap すればよいです。

まず、$S_i + A > S_{i+2}$ なる i が存在する場合、こたえは 0 であることがわかります。

$DP[i][j]$ を、小さいほうから i 番目の要素 S_i までを振り分け、最後に Y に振り分けた要素が小さいほうから j 番目の要素 S_j であるような場合の数とした DP を考えます。

この DP を愚直に計算すると間に合わないので、高速化をする必要があります。i 番目の要素を X に振り分けるとき、つぎの 2 つの場合があります。便宜上、0 番目の要素として負の無限大が入っており、これは Y に振り分けられるとして考えます。

$S_i - S_{i-1} < A$ のとき、もし S_i を X に振り分けるならば、S_{i-1} は Y に振り分けられなければなりません。すなわち、$DP[i][0] = ... = DP[i][i-2] = 0$ です。また、$DP[i][i-1] = DP[i-1][i-1]$ となることも容易にわかります。

そうでないとき、S_i を X に、何の制約もなく振り分けることができます。すなわち、$DP[i][0] = DP[i-1][0], ..., DP[i][i-1] = DP[i-1][i-1]$ です。

また、S_i を Y に振り分けるとき、$DP[i][i] = \sum_{s_i - s_j \geq B} DP[i-1][j]$ として更新を行えばいいです。

さて、$DP[i][x]$ が最初に 0 でなくなる位置のポインタを常に保持しておけば、このポインタの位置は単調に増加するので、この DP は、同じ配列を使いまわすことで全体で $O(N)$ 回の操作で更新を行うことができます。

時間計算量は尺取法のような実装で $O(N)$ 、Y に振り分ける時の和をとる範囲を求めるところに二分探索を用いたり、和をとるのに BIT を用いたりしても $O(NlogN)$ となります。

D: Uninity

まず、問題文のような木の構成法について、ウニ度 k の木を作るときの中心の点として初めて追加された点に値 k を書くことにすると、木のウニ度が k であることは、全ての i に対して、値 i の書かれた任意の 2 つの頂点間のパス上に少なくとも 1 つの、値 $i+1$ 以上の書かれた頂点が存在することと同値です。これは、ウニ度最大の点で再帰的に木を分割していくことを考えると簡単に示すことができます。

さて、これ以降、このような値の書き込み方をするときに、書かれている値が最大のものを最小化すること考えます。

まず、木を適当な頂点を根として DFS して根つき木とします。値の書き込み方をひとつ固定します。

頂点 v と値 x に対し、v から v を根とする部分木に含まれる頂点 u へのパス (端点含む) であって、そのパス上の頂点に書かれた値が u 以外では x 未満であり、u では x であるような頂点 u が存在するとき、頂点 v から値 x が見えると呼ぶことにします。さらに、各頂点 v について、全ての値 x に対し、頂点 v から値 x が見えるときには 2^x を、見えないときには 0 を加算してできる値を $p[v]$ と書くことにします。

定義より、最大の値が書かれた頂点は根から必ず見えるので、p[根] の値の全ての値の書き込み方に対する最小値がわかれば、求める最小値もわかります。

では、p[根] の最小値はどのように計算すればいいでしょうか。実は、葉から順に p[頂点] としてありうる値の最小値を順に定めていく貪欲法で、この値を求めることができます。

以下これを証明します。頂点 v の子が順に $c_1, .., c_k$ であり、$c_1, .., c_k$ を根とする部分木に含まれる頂点に対する値の割り当てが決まっているとします。

このとき、v に書き込める値の最小値は、$p[c_1], ..., p[c_k]$ の二進表記のうち 2 つ以上で立っているビットの中で一番上位のものより上のビットのうち、$p[c_1], ..., p[c_k]$ の二進表記のいずれでも立っていないビットの中で一番下位であるようなビットに対応する値です。この値を t として、 t を頂点 v に書き込むことにすれば、$p[v]$ の値は、$p[c_1] or ... or p[c_k]$ の下位 $t-1$ ビットを 0 で埋めた後、t ビット目を立てたような値です。ただし、or は bitwise or を表します。

さて、このように $p[c_1], ..., p[c_k]$ の値を受け取って、v に書き込める最小の値を v に書き込んだときの $p[v]$ の値を返す関数を f とします。

まず、上述のビットの立て方より、v には v に書き込める最小の値を書き込んだときが一番 $p[v]$ の値が小さくなることがわかります。

あとは、$f(p[c_1], ..., p[c_k]) \leq f(p[c_1]+1, ..., p[c_k])$ を示せば、この不等式を繰り返し用いることによって、このアルゴリズムがすべての頂点 v で、特に根で、$p[v]$ としてありうる最小の値を求めてくることがわかります。これは、値の二進表記に 1 を足す操作が、下位に連続する立ったビットを 0 にして、そのすぐ上の立っていないビットを 1 にする操作であるということに気を付けて場合分けを行うと、証明することができます。

以上より、この貪欲法が最小の p[根] の値を求めてくることがわかりました。「木の重心に最大の値を書き、木をいくつかに分割して再帰的に書き込む」というアルゴリズムで、ウニ度が $logN$ 以下の解が得られるので、各頂点 v での $p[v]$ の値は高々 $logN$ ビットであり、全体で $O(NlogN)$ でこの問題を解くことができます。

E: Eternal Average

x が K 進表記で $0.x_1x_2x_3...x_t(x_t \neq 0)$ と書けるとします。最終的に値 x が残るような操作列は、どのようなときに存在するでしょうか。

0 の書かれた葉を N 個と 1 の書かれた葉を M 個用意し、$a_1, ..., a_K$ を消してその平均をとる操作を行ったとき、新しくその平均の書かれた頂点をつくり、$a_1, ..., a_k$ の書かれた頂点 K 個の親に設定してできるような K 分木を考えます。最終的に残る値は、各 1 の書かれた葉 v に対し、この木上でのその葉の深さを d_v としたときの、K^{-d_v} の総和です。

つまり、最終的に x が残るとき、正整数 $a_1, ..., a_M$ を用いて $x = K^{-a_1} + ... + K^{-a_M}$ と書き表すことができます。

よって、最終的に x が残るためには、$x_1+x_2+...+x_t \leq M$ が必要です。0 と 1 を入れ替えて、値 $1-x$ について同様の議論を行うことで、$(K-1-x_1)+(K-1-x_2)+...+(K-1-x_{t-1})+(K-x_t) \leq N$ も必要なことがわかります。

さらに、$x = K^{-a_1} + ... + K^{-a_M}$ として x を K 進表記で計算するとき、繰り上がりが発生することがありますが、繰り上がりによってその各桁の和を $K-1$ で割ったあまりは変わらないので、最終的に x が残るためには $x_1 + x_2 + ... + x_t \equiv M(mod(K-1))$ も必要です。

逆に、これらの 3 つの条件が満たされれば、0 が $(K-1-x_1)+(K-1-x_2)+...+(K-1-x_{t-1})+(K-x_t)$ 個と 1 が $x_1 + x_2 + ... + x_t$ 個になるまで 0,1 の個数を (0 だけ、もしくは 1 だけの平均をとることで) 減らした後、x_i たちの値に応じて適切に平均をとることで、x を残すことができます。

上の 3 条件を満たすような x の個数は、$DP[i][j]$ を、x の i 桁目までを決定し、i 桁目までの各桁の和が j となるような場合の数として更新していくことで得られる情報から簡単に求めることができます。計算量は $O((N+M)^2)$ となります。

AGC 009 Editorial

DEGwer

A: Multiple Array

Let x_i be the number of times the i-th element is incremented. It is easy to see that $x_1 \geq \cdots \geq x_N$ must be hold. On the other hand, when this inequality holds, it is always possible to find such a way to press buttons (specifically, press the i-th button $x_i - x_{i+1}$ times). The total number of button pressings is x_1.

Thus, we want to minimize x_1 under the constraints that:

- For each i, $A_i + x_i$ is a multiple of B_i.
- $x_1 \geq \cdots \geq x_N \geq 0$

In the optimal solution, we can assume that $x_N < B_N$: otherwise we can replace x_N by $x_N - B_N$ and the condition still holds. Thus, x_N should be the minimum value that satisfies the condition $x_N \geq 0$ and $A_N + x_N$ is a multiple of B_N. Similarly, we can determine the values of $x_N - 1, \cdots, x_1$ greedily in this order.

The time complexity is $O(N)$.

B: Tournament

Construct a rooted tree with N vertices: the vertex 1 is the root, and the vertex a_i is the root of the vertex i. (The constraints guarantee that this is a valid rooted tree). Let's call it G.

When can it be a rooted tree that corresponds to a tournament of depth k? Consider a full tournament of depth k (that is, a tournament with 2^k people) and let G_k be the tree that corresponds to this tournament. The answer of the solution is the minimum k such that G_k contains G.

The tree G_k has the following structure:

- G_0 is a single vertex.
- The root of G_k has k children: the children corresponds to the roots of $G_0, \ldots, G_{k-1}$.

We solve this task by the following DP. For each vertex v in G, define $dp[v]$ as the minimum t such that the subtree of G rooted at v is contained in G_t.

How to compute the value of $dp[v]$? Let $v_1, \ldots, v_c$ be the children of v. From the observation above, $dp[v] \leq k$ if and only if $c \leq k$ and $dp[v_i] \leq k - i$ possibly after permutating the children. It's easy to show that the children should be sorted in the decreasing order of the dp value. Thus, after the children are sorted in this order, $dp[v] = \max\{dp[v_i] + i + 1\}$.

The complexity is $O(NlogN)$.

C: Division into 2

Define $DP_X[i][j]$ as the number of ways to divide the first i integers into two sets such that:

- The last (i-th) element is in X.
- The last element that is in Y is the j-th integer.

Similarly, define $DP_Y[i][j]$.

What are transitions from $DP_X[i][j]$? First, when $S_{i+1} - S_i \geq A$, we can try to include S_{i+1} into the set X. In this case, for each j, we should add $DP_X[i][j]$ to $DP_X[i+1][j]$. Also, when $S_{i+1} - S_j \geq B$, we can try to include S_{i+1} into the set Y. For each j that satisfies the inequality above, we shuold add $DP_X[i][j]$ to $DP_Y[i+1][i]$.

Now, instead of creating a 2D array, we keep an array $DP_X[j]$ that represents $DP_X[i][j]$ in the definition above for fixed i. What happens when we increment i? First, compute the sum of $DP_X[j]$ for all j that satisfies the condition $S_{i+1} - S_j \geq B$, and remember this value. Then, if $S_{i+1} - S_i < A$, "clear" the DP_X array. Then, add the remembered value to $DP_Y[i]$. Do a similar thing for DP_Y array.

Thus we need a data structure that supports the following:

- Compute the sum of numbers in the given range.
- Update the value of one element.
- Clear the data structure.

This can be done by Binary Indexed Tree and the set of positions that are updated. When you want to clear the data structure, you write zero to all updated positions one by one. This way, when you are given $O(Q)$ update queries, a single clear query may be slow, but the amortized complexity will be $O(Q)$.

The total complexity of this solution is $O(NlogN)$. Also, if you use two-pointer method, you can get an $O(N)$ solution (but a bit complicated).

D: Uninity

Assign a label to each vertex. When you construct a tree of uninity k as described in the statement, you label the center vertex with the label k. This way, you get the labelling with the following property:

- When two distinct vertices s, t have the same label ($label(s) = label(t) = k$), there exists a vertex u on the path between s and t such that $label(u) > k$.

In this task, you are asked to label the given tree with the property above (and minimize the maximum label).

Assume that the given tree is a rooted tree. Let v be an arbitrary vertex of the given tree, and suppose that you decide the labels for all vertices in the subtree rooted at v. We say that a label k is *visible* from v if

- $label(v) = k$, or
- There exists a vertex u such that $label(u) = k$ and no label on the path from u to v is greater than k.

In other words, when k is visible from v, you can't attach a vertex with label k to v.

Now, for each vertex, along with its label, let's write the bitmask of visible labels. For example, when $2, 3, 5$ are visible from v, we write $2^2 + 2^3 + 2^5 = 44$ on v. If we can compute the minimum possible bitmask for the root, we get the answer of the problem.

What is the minimum possible bitmask for the vertex v when it has two children p, q and their bitmasks are known? (For simplicity we assume that the number of children is two, but this is not essential). Let M be a very big constant, and define $weight(mask)$ as the number we get when we read $mask$ as a M-ary number. For example, $weight(44) = M^2 + M^3 + M^5$ (basically we compare the masks lexicographically). It is easy to see that the minimum possible bitmask for the vertex v is the minimum x such that $weight(x) > weight(mask[p]) + weight(mask[q])$. This shows that we can determine the labels greedily: we determine the labels from the leaves to the root, and we always choose a label that minimizes the bitmask.

If we do the centroid decomposition, we see that the answer is always $O(logN)$. Thus, the time complexity is $O(NlogN)$.

E: Eternal Average

Let's describe the process as a rooted tree. The tree has $N+M$ leaves: N of them are labelled with 0, and the others are labelled with 1. Each non-leaf node has exactly K children and it is labelled with the average of labels of its children. The label of the root corresponds to the last number.

Let $x_1, \ldots, x_N$ be the depths of the leaves labelled with 0, and $y_1, \ldots, y_M$ be the depths of the leaves labelled with 1. In this case, the last number will be $\sum K^{-y_i}$. When can we construct such a tree?

It turns out that the condition is $\sum K^{-x_i} + \sum K^{-y_i} = 1$. In general, when $\sum K^{-d_i} = 1$ and d is sorted in decreasing order, we can prove that $d_1 = \ldots = d_K$. We can merge these K leaves of depth d_1 into a single node of depth $d_1 - 1$, and if we repeat this process, we finally get a root.

Thus, in this task, we are asked to count the number of rational numbers $z (0 < z < 1)$ such that:

- z can be written as the sum of exactly N powers of $1/K$.
- $1 - z$ can be written as the sum of exactly M powers of $1/K$.

Let's write z as a d-ary number. z must be able to written with finite number of digits. Let $z = 0.z_1 z_2 \cdots z_l (z_l \neq 0)$. We can prove that z can be written as the sum of exactly N powers of $1/K$ if and only if $\sum z_i \leq N$ and $\sum z_i \equiv N \pmod{K-1}$. (The proof is similar to the observation above).

Thus, we want to count the number of sequences $z_1, \ldots, z_l$ such that

- $0 \leq z_i \leq K - 1$
- $z_l > 0$
- $\sum z_i \leq N$ and $\sum z_i \equiv N \pmod{K-1}$
- $\sum (K - 1 - z_i) \leq M - 1$ and $\sum (K - 1 - z_i) \equiv M - 1 \pmod{K-1}$

This can be solved by a straightforward DP in $O(NM)$ time.

AGC 008 Editorial

writer : sugim48

2016 年 12 月 25 日

For International Readers: English editorial starts on page 8.

A : Simple Calculator

最短の操作列は

(B を 0 or 1 回押す) → (A を 0 回以上押す) → (B を 0 or 1 回押す)

の形に限ることが示せます (証明は後述).

そこで,「最初に B を押す / 押さない」と「最後に B を押す / 押さない」の組合せを計 4 通り試すことにします．最初に B を押す場合，x の符号を反転しておきます．また，最後に B を押す場合，y の符号を反転しておきます．すると，残りの操作は「A を 0 回以上押す」だけです．よって，$x \leq y$ が成り立っていれば，A を $y - x$ 回押すことで目標を達成できます．このようにして求めた操作回数のうち最小値が答えです．

(証明) 操作列中の “BB” を “” へ，“ABA” を “B” へ，それぞれ置き換えても結果は変わらず，また操作列はより短くなります．よって，最短の操作列に “BB” や “ABA” は含まれません．よって，最短の操作列に “B” が含まれるならば，それは先頭か末尾に限ります．

B : Contiguous Repainting

この問題のように何かを上書きしていく設定に対しては，操作列を逆順に見るという考え方が有効です．元々の設定は次のようでした．

> 最初，すべてのマスは白色である．「長さ K の区間と，白色または黒色を選び，区間に含まれるすべてのマスを選んだ色で上書きする」という操作を繰り返す．

操作列を逆順に見ると，設定は次のように変わります．

> 最初，すべてのマスは未確定である．「長さ K の区間と，白色または黒色を選び，区間に含まれるすべての未確定のマスを選んだ色に確定させる」という操作を繰り返す．色が確定したマスはその後上書きされない．

一般に，後者の設定の方が扱いやすいことが多いです．以降は後者の設定のもとで考えます．

とりあえず，(操作列を逆順に見たとき) 最初の操作によって，どこかの長さ K の区間がすべて白色かすべて黒色に確定します．実は，この区間以外のすべてのマスは，1 個ずつ自由に色を確定させることができます．たとえば，最初の操作で選んだ区間のすぐ右隣のマスの色を確定させたければ，次の操作で選ぶ区間を 1 マスだけ右にずらせばよいです．同様にして，残りのマスも 1 個ずつ色を確定させていけばよいです．

以上より，次のような方法で解くことができます．まず，長さ K の区間を全探索し，その区間をすべて白色にするかすべて黒色にするかを両方試します．そして，その区間以外の各マス i について，$a_i \geq 0$ ならば黒色にし，$a_i < 0$ ならば白色にします．このようにして求まるスコアのうち最大値が答えです．

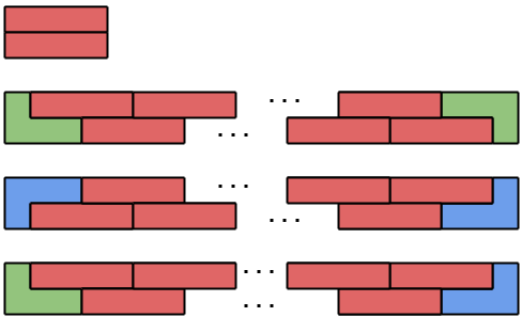
この方法を愚直に行うと，時間計算量が $\mathrm{O}(N^2)$ で間に合いません．そこで，前計算として，a_i の累積和と $b_i := \max\{0, a_i\}$ の累積和をそれぞれ計算しておきます．すると，時間計算量が $\mathrm{O}(N)$ となって間に合います．

C : Tetromino Tiling

まず，T, S, Z 型を使うことはできません．これらのテトロミノが長方形に含まれるとすると，そのテトロミノによって長方形は左右に分断されます．このとき，分断された左右の領域はともに奇数個のマスからなります．テトロミノはどれも 4 マスからなるので，どうしても奇数個のマスからなる領域を埋めることはできません．よって，T, S, Z 型を使うことはできません．

次に，O 型はすべて使うことができます．余っている O 型があれば，それを長方形の右端に追加することは常に可能です．また，O 型はすべて長方形の右端に寄せることができます．長方形の中間に O 型が挟まれていれば，それを抜いて長方形の右端に追加することは常に可能です．よって，I, J, L 型だけで長方形を作るときの最大個数と，a_O の和が答えとなります．

以上より，I, J, L 型だけで長方形を作るときの最大個数を求めればよいことになりました．I, J, L 型だけで作る長方形は，次の 4 種類のパーツを左右に連結したものに限られます．



後半 3 種類のパーツにおいて，I 型の連続部分をできるだけ I + I 型のパーツへ分解すると，ありうるパーツは次の 4 種類に限定されます．



ここで，I + J + L 型のパーツは高々 1 個しか使う必要はありません．なぜなら，I + J + L 型のパーツ 2 個を組み直すと，I + I 型のパーツと J + J 型のパーツと L + L 型のパーツへ分解できるからです．また，I + J + L 型のパーツを 0 個使うか 1 個使うかを決めると，後は I + I 型のパーツと J + J 型のパーツと L + L 型のパーツを貪欲に作るだけになります．よって，I + J + L 型のパーツを 0 個使うか 1 個使うかを両方試して最適な方を選ぶことで，I, J, L 型だけで長方形を作るときの最大個数が求まります．

D : K-th K

まず，数列 a の x_1, x_2, ..., x_N 番目には，それぞれ 1, 2, ..., N を置かなければなりません．残りの場所に整数を置くとき，どのような条件が成り立つようにすればよいでしょうか？ これは，各 $1 \leq i \leq N$ について，数列 a の x_i 番目より左にちょうど $i-1$ 個の整数 i が置かれており，数列 a の x_i 番目より右にちょうど $N-i$ 個の整数 i が置かれている，という条件になります．この条件が成り立つように，残りの場所に整数を置いていきましょう．

まずは，「各 $1 \leq i \leq N$ について，数列 a の x_i 番目より左にちょうど $i-1$ 個の整数 i が置かれている」という条件を成り立たせるように整数を置いていきましょう．このために置く整数はできるだけ左側に詰めて置くのがよいことが分かります．ここでは，左端から順に 1 個ずつ整数を置いていくことにします．このとき，最初に置いていく整数は，x_i が最も小さいような整数 i_1 にするのが最適であることが分かります．整数 i_1 をちょうど i_1-1 個置き終えたら，次に置いていく整数は，x_i が 2 番目に小さいような整数 i_2 にするのが最適であることが分かります．このような順番で，各 $1 \leq i \leq N$ についてちょうど $i-1$ 個ずつ整数 i を置いていきます．

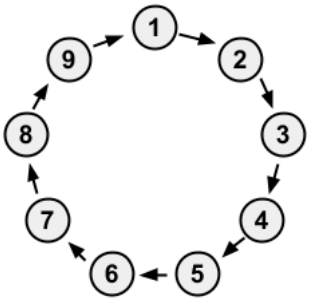
次に，「各 $1 \leq i \leq N$ について，数列 a の x_i 番目より右にちょうど $N-i$ 個の整数 i が置かれている」という条件を成り立たせるように整数を置いていきましょう．このために置く整数はできるだけ右側に詰めて置くのがよいことが分かります．ここでは，右端から順に 1 個ずつ整数を置いていくことにします．このとき，最初に置いていく整数は，x_i が最も大きいような整数 i_1 にするのが最適であることが分かります．整数 i_1 をちょうど $N-i_1$ 個置き終えたら，次に置いていく整数は，x_i が 2 番目に大きいような整数 $N-i_2$ にするのが最適であることが分かります．このような順番で，各 $1 \leq i \leq N$ についてちょうど $N-i$ 個ずつ整数 i を置いていきます．

以上のようにして，最適な方法ですべての場所に整数を置くことができました．最後に，成り立つべき条件が実際に成り立っているかチェックします．成り立っていれば，答えは `Yes` であり，構成した数列 a を出力すればよいです．成り立っていなければ，他にどのようにしても条件が成り立つようにできないので，答えは `No` です．

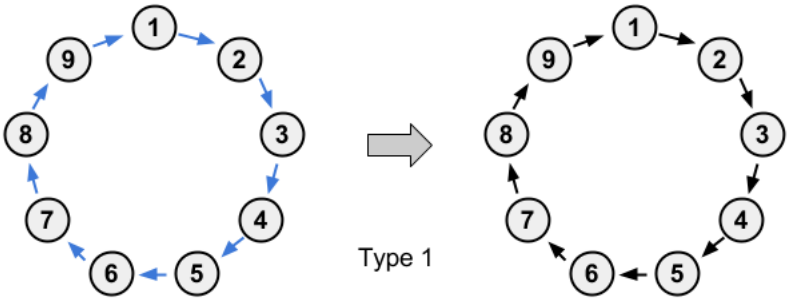
E : Next or Nextnext

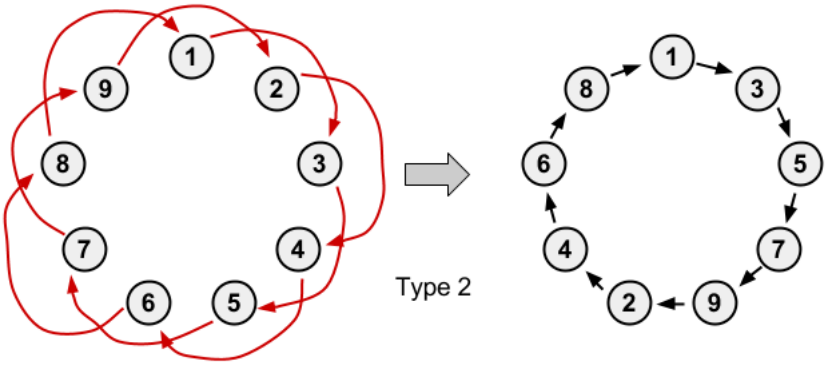
$(1, 2, ..., N)$ の順列 $(p_1, p_2, ..., p_N)$ が与えられたとき，頂点 i から頂点 p_i へそれぞれ有向辺を張ることで，有向グラフを作ることができます．同様にして，数列 $(a_1, a_2, ..., a_N)$ からも有向グラフを作ることができます．a_i の値として p_i または p_{p_i} が選べるとき，数列 $(a_1, a_2, ..., a_N)$ のグラフはどのような形がありうるでしょうか？

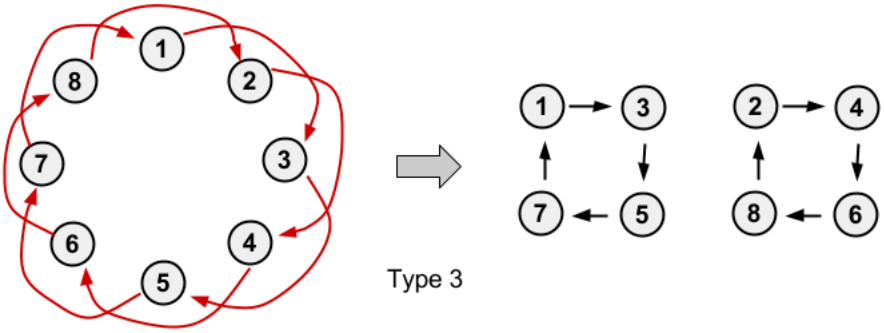
まず，順列 $(p_1, p_2, ..., p_N)$ のグラフはいくつかの閉路からなることが分かります．それら閉路のうちひとつに注目して考えることにします (次図)．

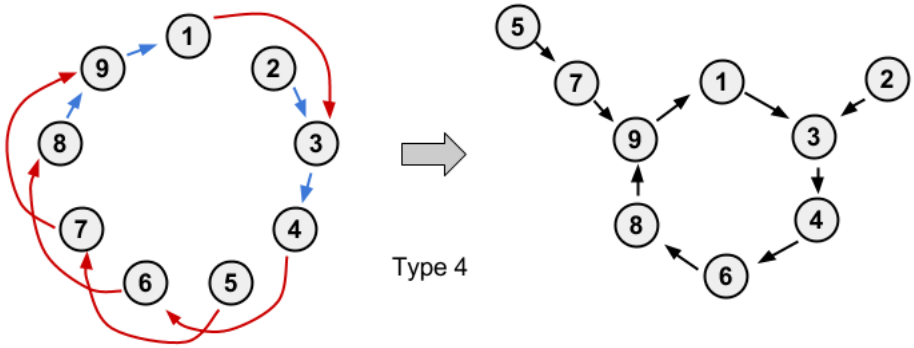


この閉路において，a_i の値として p_i または p_{p_i} が選べるとき，数列 $(a_1, a_2, ..., a_N)$ のグラフの形は次図の 4 種類に大別できます．なお，$a_i = p_i$ を青い矢印で，$a_i = p_{p_i}$ を赤い矢印で表しています．







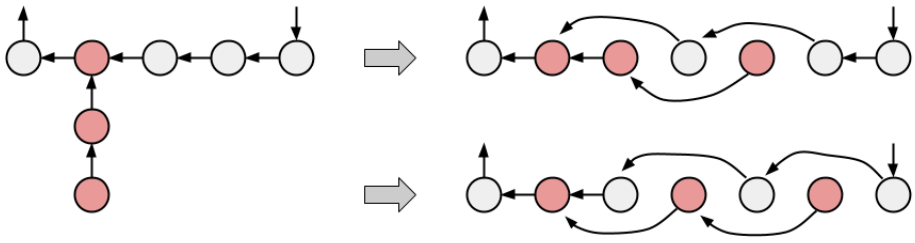


Type 1 は，前と後でグラフの形がまったく同じです．Type 2 は，閉路のサイズが 3 以上の奇数の場合のみ発生し，前と後で頂点の順番のみが変わっています．Type 3 は，閉路のサイズが偶数の場合のみ発生し，サイズが半分の閉路 2 個に分裂します．Type 4 は，閉路に何本か足が生えた形になっています．このとき，足に分岐はなく，また同じ箇所から複数本の足が生えることはありません．

以上の観察をもとに，元の数え上げ問題の解法を考えていきます．まず，与えられた数列 $(a_1, a_2, ..., a_N)$ から先述のグラフを作ります．このグラフはいくつかの連結成分に分かれていますが，各連結成分はそれぞれ Type 1 - 4 のどれかによって発生したはずです．よって，もし足に分岐があったり，同じ箇所から複数本の足が生えていたりしたら，すぐに答えは 0 と分かります．以降は，各連結成分は「ただの閉路」または「閉路に適切な足が何本か生えたもの」であるとします．これらの元々の姿としてありうるグラフを数え上げます．

まずは，「ただの閉路」たちの元々の姿としてありうるグラフを数え上げます．同じサイズの閉路が 2 個ある場合，それらは元々 1 個の閉路だったものが Type 3 によって分裂した可能性があります．よって，「ただの閉路」たちは独立に元々の姿を数え上げることはできず，閉路のサイズ k ごとにその個数 n_k をカウントしておき，まとめて元々の姿を数え上げる必要があります．これは $O(n_k)$ の DP で可能です．DP の状態としては，未処理の閉路の個数を持ちます．未処理の閉路のうち適当にひとつ選んできたときに，その閉路は Type 1 によって発生したか，k が 3 以上の奇数ならば Type 2 によって発生したか，または他の未処理の閉路とペアになって Type 3 によって発生したか，のどれかです．これらを DP の遷移とすればよいです．なお，Type 3 の場合に閉路 2 個をペアにするとき，閉路 2 個の噛み合わせ方は k 通りあることに注意してください．

次に，「閉路に足が生えたもの」たちの元々の姿としてありうるグラフを数え上げます．「閉路に足が生えたもの」たちは Type 4 のみによって発生したので，他のものとペアになって発生したということはありえません．よって，「閉路に足が生えたもの」たちは独立に元々の姿を数え上げることができます．「閉路に足が生えたもの」は元々ただの閉路だったので，足の部分をうまく閉路の部分に収納してあげる必要があります．このとき，それぞれの足の収納方法は次図の 2 通りです．



実際には，足の収納スペースがどれくらいあるかで場合分けが必要です．今注目している足の辺数を l_1，今注目している足の付け根から次の足の付け根までの辺数を l_2 とします．$l_1 < l_2$ ならば，前図の 2 通りの収納方法が可能です．$l_1 = l_2$ ならば，前図のうち上の 1 通りの収納方法のみが可能です．$l_1 > l_2$ ならば，この足は収納できず，すぐに答えは 0 と分かります．以上の場合分けをすべての足について行えばよいです．

以上の解法をまとめます．まず，与えられた数列 $(a_1, a_2, ..., a_N)$ から有向グラフを作り，連結成分ごとに形をチェックします．このとき，足の付け根や途中に分岐があれば，すぐに答えは 0 と分かります．そうでなければ，「ただの閉路」たちと「閉路に足が生えたもの」たちのみが存在することになります．「ただの閉路」たちの元々の姿の通り数は，DP によって求まります．「閉路に足が生えたもの」たちの元々の姿の通り数は，足の収納方法の通り数の積で求まります．これらの積が答えとなります．計算量は $\mathrm{O}(N)$ です．

F : Black Radius

まず思いつくのは，お気に入りの頂点 v を全探索するという方法です．v を固定して距離 d を自由に動かすとき，何通りの配色がありうるかというのは簡単に求められます．しかし，これらの通り数をすべて合計しても正しい答えは求まりません．なぜなら，異なる v から同一の配色が作られる場合に，それらを重複して数えてしまうからです．重複が発生しないようにするためには，何を全探索すればよいでしょうか？

どのような配色でも，黒い頂点はひと繋がりの部分木をなすことがすぐに分かります．この黒い部分木の center vertex および center edge を全探索することを考えます．center vertex および center edge が異なれば当然黒い部分木も異なるため，重複は発生しません．以降は，center vertex および center edge を固定したときに何通りの黒い部分木がありうるかを考えることにします．

まずは，center vertex を頂点 v に固定したときに何通りの黒い部分木があるかを考えます (このとき，必ずしも v はお気に入りの頂点とは限らないことに注意してください)．center vertex が v である黒い部分木の半径 r をひとつ決めたとします．すると，当然 v からの距離が r より大きい頂点は，すべて白色でなければなりません．さらに，実は v からの距離が r 以下の頂点は，すべて黒色でなければならないということも分かります．よって，半径 r をひとつ決めると，対応する黒い部分木は (存在するならば) 一意に定まります．以上より，黒い部分木が存在するような r の個数を求めればよいことになります．

実は，黒い部分木が存在するような r というのは，ある下限と上限の間の連続区間になっています．まずは上限から考えてみましょう．黒い部分木の半径がちょうど r となるためには，v にぶら下がっている各部分木のうち，黒い頂点を深さちょうど r に含むようなものが，少なくとも 2 本なければなりません．よって，v にぶら下がっている各部分木のうち，深さが 2 番目に大きいものの深さを d_2 とすると，$r \leq d_2$ でなければなりません．よって，r の上限は d_2 です．

次に，r の下限を考えてみましょう．もし，v がお気に入りの頂点ならば，v を選んで距離 $d = r$ とすることで，$0 \leq r \leq d_2$ の範囲の r をすべて実現できます．よって，この場合 r の下限は 0 です．では，v がお気に入りの頂点でない場合はどうすればよいでしょうか？ この場合，v 以外のお気に入りの頂点 w を代わりに選び，うまく距離 d を設定することで，黒い部分木の center vertex が v かつ半径が r になるようにしなければなりません．v にぶら下がっている各部分木のうち，頂点 w を含むものを T_w と書くことにします．距離 d を設定する際には，T_w の頂点がすべて黒くなる必要があることが分かります．T_w に白い頂点が残っていると，黒い部分木全体の center vertex が v になりえないからです．逆に，T_w の頂点がすべて黒くなるように距離 d を十分大きくすれば，あとは $r \leq d_2$ の範囲で r をいくらでも大きくできます．ただし，T_w で最も深い頂点が黒いという前提があるので，r は T_w の深さより小さくはできません．よって，お気に入りの頂

点 w としてどのようなものを選ぶべきかというと，v にぶら下がっている各部分木のうち，最も深さの小さい部分木に含まれるお気に入りの頂点を選ぶべきだと分かります．以上より，v にぶら下がっている各部分木でお気に入りの頂点を含むもののうち，最も深さの小さいものの深さを d_1 とすると，$d_1 \leq r \leq d_2$ の範囲の r をすべて実現できます．よって，r の下限は d_1 です．

では次に，center edge を辺 (u, v) に固定したときに何通りの黒い部分木があるかを考えます．この場合も，center vertex を固定したときと同様に，半径 r を決めると黒い部分木が一意に定まります．すなわち，頂点 u 側の頂点たちについて，頂点 u からの距離が r 以下ならば黒色，r より大きければ白色となります．頂点 v 側の頂点たちについても同様です．ここで，お気に入りの頂点を u 側の頂点たちから選んできたとします．このとき，先の議論と同様にして，頂点 u 側の頂点はすべて黒くなる必要があることが分かります．頂点 v 側についても同様なので，結局，頂点 u 側か頂点 v 側のどちらかはすべて黒くなっているということがいえます．また，辺 (u, v) が center edge となるためには，頂点 u 側の部分木と頂点 v の部分木のうち，より浅い方 (同じ場合はどちらでもよい) がすべて黒くなる必要があります．よって，実は center edge を辺 (u, v) に固定したときの黒い部分木は (存在すれば) 1 通りです．存在を判定するには，頂点 u 側の部分木と頂点 v の部分木のうち，より浅い方 (同じ場合はどちらでもよい) にお気に入りの頂点が含まれるかをチェックすればよいです．

以上の計算に必要なすべての情報は，全方位木 DP によって計算量 $\mathrm{O}(N)$ で求められます．

AGC 008 Editorial

writer : sugim48

December 25, 2016

A : Simple Calculator

You can prove that the optimal solution is always of the following form:

(Push B 0 or 1 times) → (Push A 0 or more times) → (Push B 0 or 1 times)

Proof: In the optimal solution, the substrings "BB" and "ABA" never appear because they are equivalent to "" and "B". Thus, "B" appears only at the ends of the operation sequence.

Now you can try four patterns: whether you push B at the beginning (change x to $-x$) or not, and whether push B at the end (change y to $-y$) or not. After changing the signs of x and y, if $x \leq y$ holds, this sequence is possible with $y - x$ more operations.

B : Contiguous Repainting

Let's see the operations in the reverse order. The problem will be as follows:

> Initially all cells are unpainted. You repeat the following operation: choose K consecutive cells and paint it black or white. However, once a cell is painted, the color of the cell never changes.

Obviously, after all cells are painted, there is at least one subsegment of length K that is painted with the same color. (The K cells that are colored in the first operation satisfy this condition).

On the other hand, this condition is sufficient: once you color K consecutive cells with the same color, you can determine the color of the remaining cells arbitrarily.

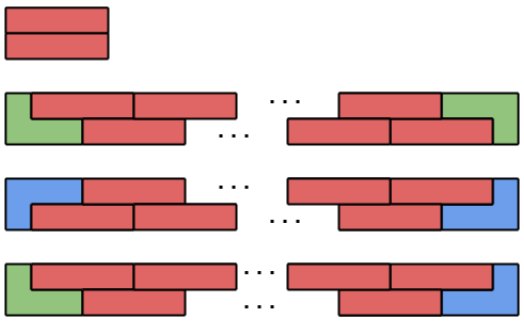
Thus, we get the following $O(N^2)$ solution: brute force all subsegments of length K, and for this segment and for each of the other cells, determine the color greedily. With proper pre-computation, this can be done in $O(N)$ time.

C : Tetromino Tiling

We can't use T, S, Z minoes: if you put one of these minoes on a $2 \times 2K$ board, the board will be splitted into two parts whose areas are odd.

We can always use all O minoes: when we can create a $2 \times 2K$ rectangle with an O mino, we can create a $2 \times 2K - 2$ rectangle without it. Thus, the answer is the sum of a_O and the maximum size of the rectangle that can be formed by only using I, J, L minoes.

When we can use only I, J, L minoes, it is easy to see that there are only four patterns (and their concatenations):



In the last three patterns, when there are two or more I minoes, we can remove it and create an I + I type rectangle. Thus, it is sufficient to consider the following four patterns:



Also we can assume that we use the I + J + L pattern at most once (otherwise we can create I + I, J + J, and L + L). Thus, we can try two possibilities for the number of used I + J + L patterns, and the number of the other patterns can be computed easily.

D : K-th K

We should divide the integers $1, \ldots, N^2$ into the following sets:

- Integers $x_1, \ldots, x_N$
- Sets $L_1, \ldots, L_N$. L_i corresponds to the positions of is to the left of i-th i. $|L_i| = i - 1$ and each element in it must be smaller than x_i.

- Sets $R_1, \ldots, R_N$. R_i corresponds to the positions of is to the right of i-th i. $|R_i| = N - i$ and each element in it must be greater than x_i.

Now, in the order $p = 1, \ldots, N^2$, we classify p into one of the sets described above.

When $p = x_i$ for some i, it is obvious. Now we assume that there is no such i.

We say p can be assigned to L_i if the current size of L_i is still smaller than the desired size of it and p is smaller than x_i. Similarly, define "p can be assinged to R_i".

In case there is no set that p can be assigned to, the answer is "No". In case there are muultiple such sets, how can we choose a set?

First, when p can be assigned to both L_i and R_j for some i, j, we can prove that p should be assigned to an L-type set. This is because if we assign p to R_j and, in the future, if we assign q to L_i, we can swap p and q.

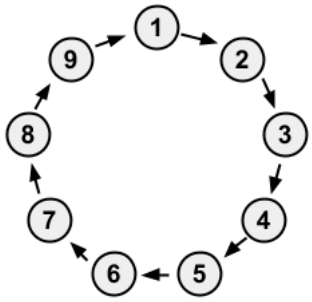
When p can be assigned to both L_i and L_j ($x_i < x_j$), p should be assigned to L_i for a similar reason.

Also, when p can be assigned to multiple R-type sets, it doesn't matter which to choose - again for a similar reason.

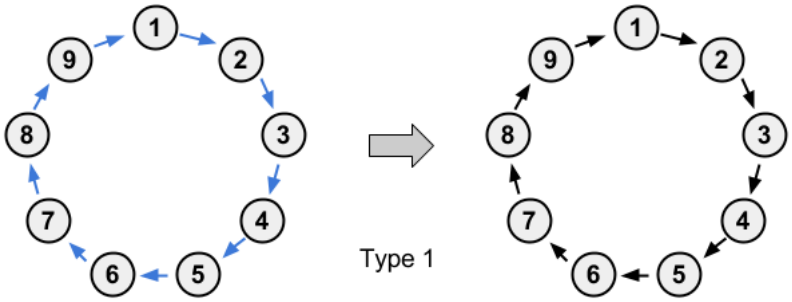
This way we can greedily determine the set p is assigned to, in $O(N^3)$ or $O(N^2 logN)$ time.

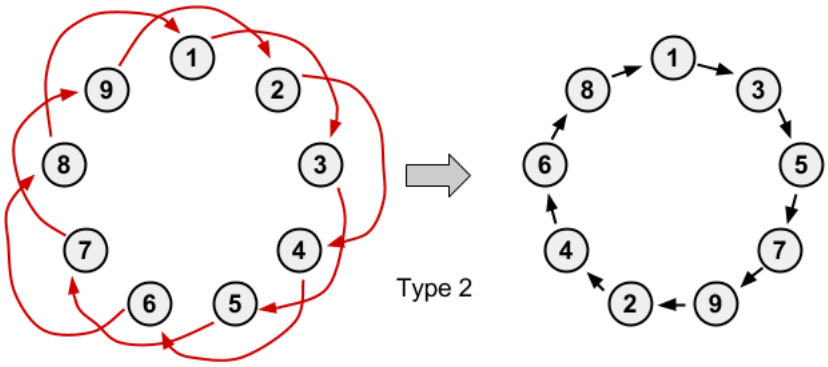
E : Next or Nextnext

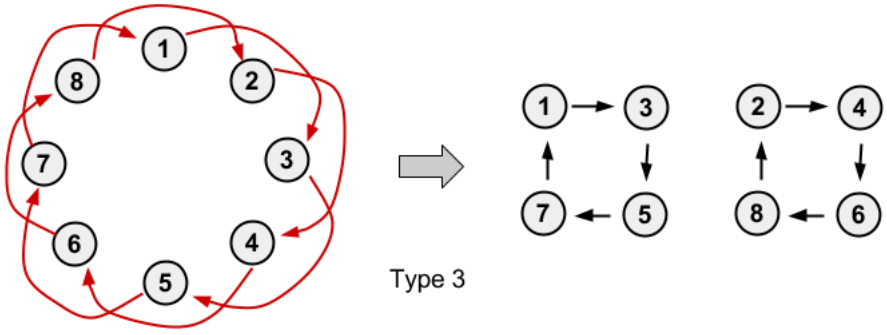
Construct a graph with N vertices and for each i, add an edge from i to p_i. This graph is a set of cycles. Consider one of the cycles in the graph.

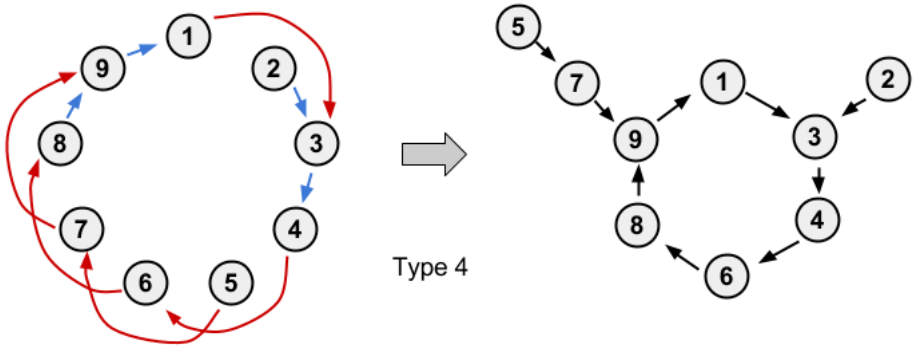


We construct a new graph by adding edges from x to p_x or p_{p_x} for each vertex x in this cycle: There are four possible types for the new graph:


1
2
3
4
5
6
7
8
9
Type 1


1
2
3
4
5
6
7
8
9
Type 2


1
2
3
4
5
6
7
8
Type 3



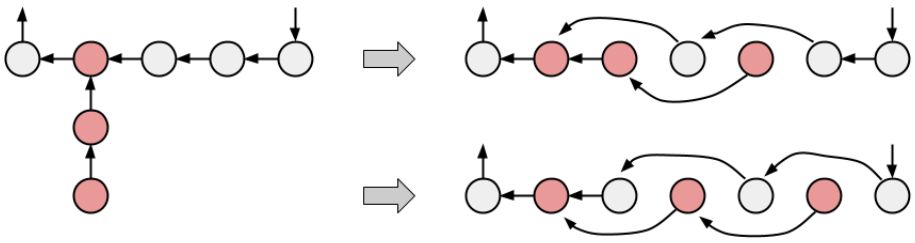
- In Type 1 we get the same graph.
- Type 2 occurs only when the size of the cycle is an odd number (except for 1), and we get an isomorphic graph.
- Type 3 occurs only when the size of the cycle is an even number, and the cycle is splitted into two cycles of the half size.
- Type 4 is a cycle with "feet". No two feet can grow from the same vertex and there are no branches in the feet.

Let's return to the original problem. First construct a similar graph for the sequence a (that is, add edges from i to a_i). Each connected component in this graph must be a cycle or "a cycle with feet". Otherwise the answer is zero.

First, let's think about cycles. Two cycles of the same size may come from the same cycle in the graph of p (because of Type 3).

Suppose that there are n_k cycles of the size k. We can count the number of ps that corresponds to these cycles in $O(n_k)$ DP. Each cycle may come from Type 1 or Type 2, or it may be paired with another cycle and come from Type 3. Note that when we use Type 3, there are k ways to "merge" two cycles.

Next, consider "a cycle with feet". Since this comes from Type 4, we can compute the number of ways for each connected component independently. For each foot, we can merge it with the "cycle part" in at most two ways:



Let l_1 be the length of the foot. Let l_2 be the distance between the root of current foot and the root of previous foot.

- If $l_1 < l_2$, there are 2 ways.
- If $l_1 = l_2$, there is 1 way.
- If $l_1 > l_2$, there are 0 ways.

In summary,

- First constcuct a graph with edges from i to a_i.
- If there is an "invalid component", the answer is zero.
- Then, we first handle simple cycles: count the frequency of cycles by size and do DP.
- Then, we handle cycles with feet: for each root of foot, there are 0 or 1 or 2 ways to merge it with the cycle part.
- The answer is the product of these numbers.

In total, we can solve the problem in O(N).

F : Black Radius

Let $f(x, d)$ be the set of vertices y such that the distance between x and y is at most d. The partial score of this task asks the number of sets that can be written of the form $f(x, d)$ (and we call such sets "good").

Let S be a good set. In general, it can be written of the form $f(x, d)$ in multiple ways, so in order to avoid duplicates it is important to define the "canonical form". For simplicity, we assume that S is not the entire set. Note that the entire set is always a good set.

Suppose that $S = f(x, d_1) = f(y, d_2)$ for some two distinct vertices x, y. Let $x, v_1, v_2, \ldots, v_{k-1}, y$ be the path from x to y. If we analyze this situation carefully, we can prove that there is some vertex z on this path and the following holds:

$$f(x, d_1) = f(v_1, d_1 - 1) = f(v_2, d_2 - 1) = \ldots = f(z, d_3) = \ldots = f(v_{k-1}, d_2 - 1) = f(y, d_2) \quad (1)$$

In other words, for each set there is a canonical form (in the case above $f(z, d_3)$), and other representations of this set is of the form $f(v, d_3 + dist(v, z))$. Note that the set $f(v, d_3 + dist(v, z))$ may not be the same for all vertices v: however, the set of such vs form a subtree (connected subset in the tree).

Now, how can we get the partial score? For each vertex x, we compute the maximum integer d such that

- $f(x, d)$ is not the entire set.
- $f(x, d)$ is not the same as $f(y, d - 1)$ for any vertex y adjacent to x.

In order to compute d, first we see the tree as a rooted tree rooted at x. Then, for each child y of x, compute the depth of the deepest vertex in the subtree of y. This can be done in $O(N)$ tree DP in total for all x: make sure not to write $O(N^2)$ solution when the given tree is a "star". With this pre-computation the value of d can be computed easily.

Now, the sets $f(x,0),\ldots,f(x,d)$ corresponds to the good sets written in canonical forms, so the answer is the sum of $d+1$ for each vertex x plus one (for the entire set).

How can we get the full score?

We see that the good sets can be written in "canonical form" in a unique way, and that corresponds to $\{\ f(x,i) | i \leq high_x\ \}$ for some array $high_x$. Now, for each $f(x,i)$ that is written in canonical form, we want to decide if it can be rewritten of the form $f(y,j)$ using a good vertex y.

- When x is a good vertex, of course this is possible.
- When x is not a good vertex, there is a number low_x such that the set $f(x,i)$ can be rewritten if and only if $i \geq low_x$.

Thus, we can compute the array low and the answer is the sum of $high - low + 1$ plus one.

AtCoder Grand Contest 007 Editorial

writer : dreamoon

English editorial starts on page 8.

A : Shik and Stone

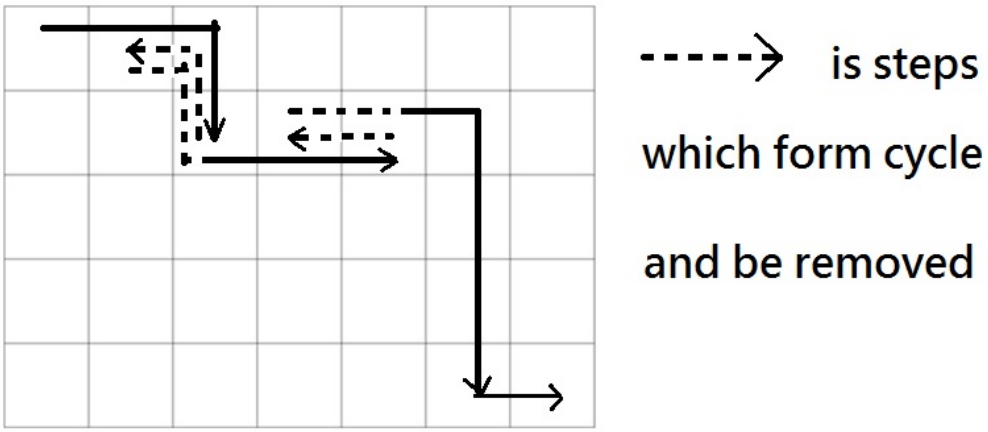
この問題には簡潔な解法が存在します。入力中の文字 `#` の個数を数えて、その個数が $H+W-1$ と等しければ `Possible`、等しくなければ `Impossible` を出力すればよいです。以下、この解法の妥当性を示します。

駒が上下左右に 1 マス動くことを「ステップ」と呼ぶことにします。まず、左上隅のマスから右下隅まで、右または下へのステップのみを使って移動するときのステップ列の集合 (この集合を S_A とします) と、左上隅のマスから右下隅まで最少のステップ数で移動するときのステップ列の集合 (この集合を S_B とします) が等しいことを示します。

i 番目に駒が通ったマスの行番号と列番号の和を g_i とします。i 番目のステップが右または下へのステップなら $g_{i+1} = g_i + 1$ 、左または上へのステップなら $g_{i+1} = g_i - 1$ が成り立ちます。したがって、数列 g_i の隣接する二つの要素の左の絶対値は常に 1 となります。駒が行ったステップの数を n とすると、$g_1 = 2$ 、$g_{n+1} = H + W$ となります。よって、n は少なくとも $H+W-2$ であり、$n = H+W-2$ のときはすべての i について $g_{i+1} = g_i + 1$ となることになります。ゆえに、S_A と S_B はともに、$n = H+W-2$ であるようなステップ列全体の集合となります。

次に、'#' の個数は駒が一度以上通ったマスの個数であるため、$n+1 \geq$ (`#` の個数) が成り立ちます。`#` の個数が $H+W-1$ より大きいと、n が $H+W-2$ より大きくなり、ステップ数が最少でないことになります。したがって、この場合は `Impossible` が答えとなります。

最後に、`#` の個数が $H+W-1$ と等しければ `Possible` を答えとしてよいことを示します。問題の制約より、入力された情報に対応するステップ列が存在し、そのうちの一つを選びます。このステップ列から、通るマスの列がサイクルを形成するようなステップを取り除きます（下図を参照してください)。`#` の個数を減らすことはできない（減らせるようであれば問題の制約に反する）ため、このようなステップを取り除くことは常に可能です。このようなステップを取り除くと、ステップ数が (`#` の個数) -1 であるようなステップ列が得られます。これは最少のステップ数であり、このステップ列は入力で与えられる情報と整合します。



おまけ: この問題から「問題文および a で与えられる情報と整合するような駒の動き方が存在する。」という制約を取り除いた場合、同じ解法が使えるでしょうか？もし使えないならば、どうすればよいでしょうか？

B : Construct Sequences

まず解の一例を示し、それからその解がすべての条件を満たすことを示すことにします。以下が解の一例です。

$r_{p_i} = i\ (1 \leq i \leq n)$ として、
$A_i = 30000 \times i\ (1 \leq i \leq n)$
$B_i = 30000 \times (n - i) + r_i\ (1 \leq i \leq n)$

正当性の証明（概略）は以下の通りです。
$30000 \times 1 \leq A_i \leq 30000 \times n \leq 30000 \times 20000 \Rightarrow 1 \leq A_i \leq 10^9$.
$30000 \times 1 + 1 \leq B_i \leq 30000 \times n + n \leq 30000 \times 20000 + 20000 \Rightarrow 1 \leq B_i \leq 10^9$.
$A_i = 30000 \times i < 30000 \times (i + 1) = A_{i+1}$.
$B_i = 30000 \times (n - i) + r_i > 30000 \times (n - i) = 30000 \times (n - i - 1) + 30000 > 30000 \times (n - i - 1) + r_{i-1} = B_{i-1}$.
$A_{p_i} + B_{p_i} = 30000 \times p_i + 30000 \times (n - p_i) + r_{p_i} = 30000 \times n + i < 30000 \times n + i + 1 = 30000 \times p_{i+1} + 30000 \times (n - p_{i+1}) + r_{p_{i+1}} = A_{i+1} + B_{i+1}$.

C : Pushing Balls

球を一つ転がしたあと、残りの球と穴に番号を振りなおすと、やはり i 番の球が i 番の穴と $i+1$ 番の穴の間にあることがポイントです。球を一つ転がしたとき、球の転がし方として可能なもの

すべてを考慮して、隣接する球と穴の間の距離の期待値を更新します。計算すると、期待値の列 $\mathbf{E}[d_i]$ はやはり等差数列をなし、その長さは 1 短くなっています。

よって、求めたい期待値を再帰的に求めることができます。最初に転がす球が移動する距離の期待値を求め、残った球が移動する距離の期待値を「サイズの減った元の問題」として再帰的に求めた結果と足し合わせればよいです。

D : Shik and Game

以下、ゲームが行われる直線は左右に伸び、右に進むほど座標が大きくなるものとします。プレイヤーがゴールと反対方向に戻ってコインを拾いに行くとき、まだ拾っていないコイン（まだ出現していないものも含む）のうち最も左にあるものの位置まで戻り、もしまだ出現していなければ出現まで待つべきです。でないと、そのコインを取りに再び戻る必要が生じてしまいます。したがって、このゲームにおける戦略は、クマたちを何個かの連続する区間に分割したもので表されます。この分割が表す具体的な行動は、プレイヤーがまず最初の区間にいるクマたち全員にキャンディを与え、この区間の左端のクマの位置まで戻ってすべてのコインを拾い、次の区間に進む、というものです。

戦略における区間の個数を M 、i 番目の区間の左端の座標を L_i 、右端の座標を R_i とおくと、この戦略における所要時間は $E + \sum_{i=1}^{M} \max(t, 2 \times (R_i - L_i))$ となります。左から i 匹のクマからすべてのコインを回収するのに要する最短時間を $dp[i]$ とおくと、これは $dp[i] = \min_j(dp[j] + \max(t, 2 \times (a_i - a_{j+1})))$ とする動的計画法により $\mathrm{O}(N^2)$ 時間で計算できます。

この解法の計算量を改善しましょう。上式の $\max$ の部分に注目し、j が動く範囲を $t > 2 \times (a_i - a_{j+1})$ であるような範囲と $t \leq 2 \times (a_i - a_{j+1})$ であるような範囲に分割します。各 i に対し、しきい値 p が存在して前者の範囲が $j > p$ 、後者の範囲が $j \leq p$ となることがわかります。さらに、p の値が i に関して広義単調増加することもわかり、すべての i に対する p の値を合計 $\mathrm{O}(N)$ 時間で求めることができます。

範囲 $t > 2 \times (a_i - a_{j+1})$ に対しては、動的計画法の式の右辺は $\min_j(dp[j] + t)$ となります。配列 dp の値は広義単調増加するため、この値は範囲内の最も小さい j で最小値をとります。この最小値はすべての i に対し合計 $\mathrm{O}(N)$ 時間で求めることができます。

範囲 $t \leq 2 \times (a_i - a_{j+1})$ に対しては、動的計画法の式の右辺は $\min_j(dp[j] + 2 \times (a_i - a_{j+1}))$ となります。これを $2 \times a_i + \min_j(dp[j] - 2 \times a_{j+1})$ と変形します。動的計画法に用いるもう一つの配列 $dp2$ を用意し $dp2[i] = \min_{j=1}^{i}(dp[i] - 2 \times a_{i+1})$ として、この値をすべての i に対し合計 $\mathrm{O}(N)$ 時間で求めることで、$2 \times a_i + \min_j(dp[j] - 2 \times a_{j+1}) = 2 \times a_i + dp2[j]$ をすべての i に対し合計 $\mathrm{O}(N)$ 時間で計算することができます。

以上より、元の解法の計算量 $\mathrm{O}(N^2)$ を $\mathrm{O}(N)$ に改善することができました。

E : Shik and Travel

直感的に、この問題を解く上で最初に答えの値に対する二分探索を行うべきであると感じた人がいるかもしれません。ここで説明する解法もその方針をとります。すなわち、ある金額 V に対して、「負担金額が V 以下となるような、木の葉に相当する都市（以下、これらを葉都市と呼びます）の訪問順序が存在するか？」という部分問題を解くことを考えます。

まず、次の点を踏まえます。旅行中、i 番の道を一度目に通ったら、二度目にその道を通るまでに、$i+1$ 番の都市を根とする部分木に存在するすべての都市を訪れなければなりません。つまり、ある都市を一度目に訪れたら、その日からその都市を根とする部分木に存在するすべての葉都市で宿泊していくことになります。より正確には、都市 i を根とする部分木に存在する葉都市の個数を n_i とすると、ある数 st_i が存在し、以下の期間には都市 i を根とする部分木の内部にいることになります: 「旅行の st_i 日目の途中から」、「st_i+1 日目から st_i+n_i-1 日目の間ずっと」、「st_i+n_i 日目の途中まで」。そして、その他の期間には部分木の外部にいることになります。

このことから、部分問題への再帰的な愚直解が得られます。各都市で、二つの子が持つ訪問列を再帰的に併合していき、その結果が条件を満たすか判定していきます。以下に疑似コードを記します。

```
function f
argument: root city i of some subtree
return value: a set S_i of ordered pair of numbers

S_i に要素 (a, b) が含まれるとき、葉都市の訪問順序であって以下のようなものが存在することを表す。
- st_i 日目に発生する料金のうち、この部分木の内部で発生する分は a である。
- st_i + n_i 日目に発生する料金のうち、この部分木の内部で発生する分は b である。
- st_i + 1 日目から st_i + n_i - 1 日目に発生する料金はいずれも V 以下である。

f(i) {
    if n_i = 1:
        return {(0, 0)}
    set S_i as empty set
    denote the number of two son cities of city i as j and k
    let S_j = f(j) and S_k = f(k)
    for all combination of element (a,b) in S_j and element (c,d) in S_k:
        if (b+c+v_j+v_k <= V) then add (a+v_j, d+v_k) to S_i
        if (a+d+v_j+v_k <= V) then add (c+v_k, b+v_j) to S_i
    return S_i
}
```

この $f(1)$ を呼び、S_i が空であるかどうかが部分問題の答えとなります。

問題は $|S_i|$ が大きくなりすぎることで、これを小さく抑えたいです。目標は S_1 が空であるかどうかを知ることのみであるため、S_i の要素 (a,b) であって、$a' \leq a$ and $b' \leq b$ を満たすような他の要素 (a',b') が存在するようなものは S_i から省くことができます。

このようにすると、$|S_i| \leq \min(\ |S_j|, |S_k|)\ \times 2$ が成立します。

$|S_j| \leq |S_k|$ とします。各 (a, b) について、それを用いて生成される新たなペアは $(a + v_j, u)$ または $(u, b + v_j)$ (u は任意の数) のいずれかの形式で表されます。このそれぞれに対して、その形式で表されるような最終的に残るペアはたかだか一つです。よって $|S_i| \leq |S_j|$ が示されました。$|S_k| \leq |S_j|$ の場合も同様です。

この性質は、さまざまな点において役立ちます。特に、$\sum_{i=1}^{N} |S_i| = \mathrm{O}(NlogN)$ が成り立ちます。以下にこれを示します。

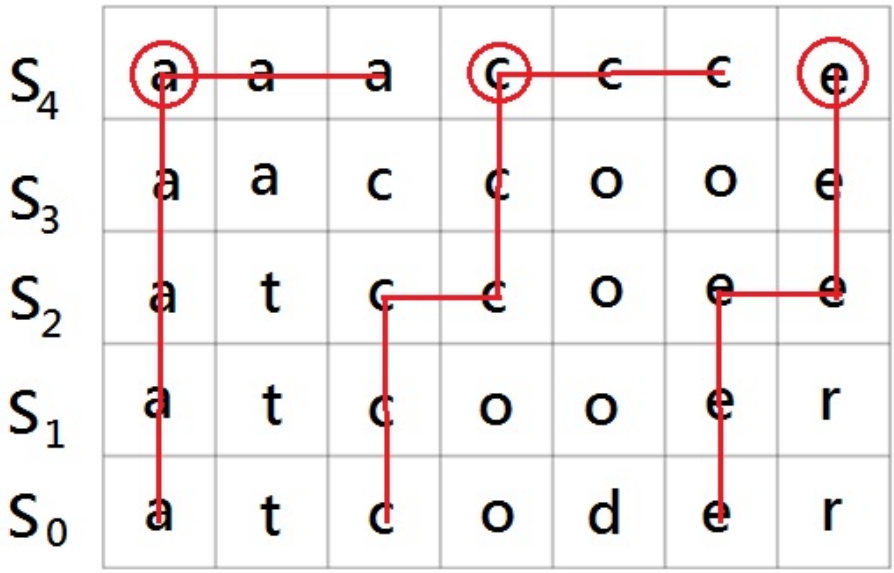
i 番の都市から 1 番の都市までの距離（深さ）を d_i とします。各都市 i について、i を根とする部分木において d_j が最小となるように都市 j を選びます。すると、$|S_i| \leq 2^{d_j - d_i}$ となります。したがって、$|S_i|$ の値を、i を根とする部分木に存在する深さ d_j のすべての都市に 1 ずつ「償却」することができます。この「償却」を行っていく過程において、$|S_i|$ の和がすべての頂点の値の和を超えることはありません。また、各頂点に「償却」される値はたかだか $log_2(N + 1)$ です。よって、$\sum_{i=1}^{N} |S_i| \leq N \times log_2(N + 1)$ となります。

以上から、尺取り法を用いて S_i をソートされた状態に保ち、$(a + v_j, u)$ と $(u, b + v_j)$ のそれぞれの形式における u の最小値を求めていくことにより、部分問題を時間計算量 $\sum_{i=1}^{N} |S_i| = \mathrm{O}(NlogN)$ で解くことができ、元の問題を $O(NlogN \times log\ answer)$ 時間で解くことができます。

F : Shik and Copying String

まず、$S_0 = T$ のとき、答えは 0 です。この場合は例外として扱います。

下図に、操作が行われていき文字列が S_0 から T に変遷する様子を示します。



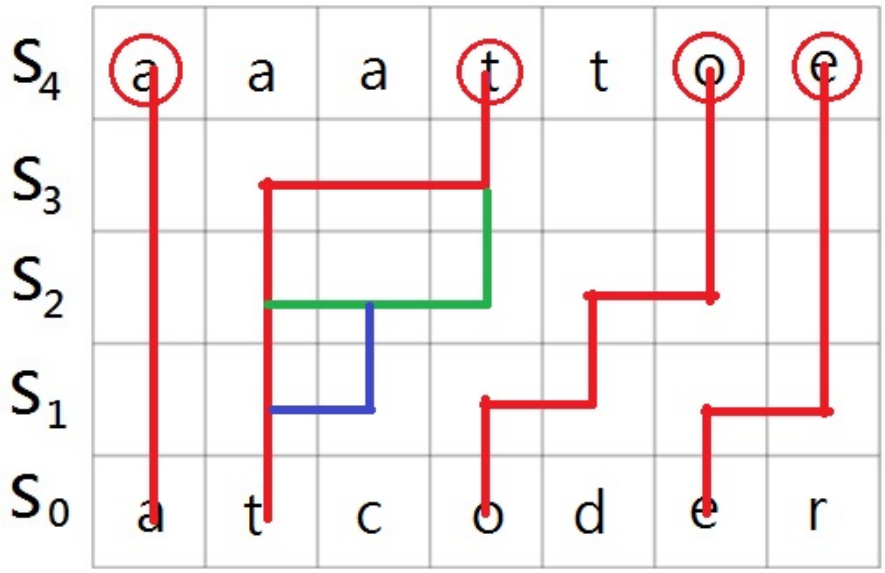
これは、$S_0 = \texttt{atcoder}$、$T = \texttt{aaaccce}$ のときの操作列であって、$S_4 = T$ であるようなものを示しています（最小の操作回数を求めているわけではありません）。同じ文字が連続するとき、それらのうち最初のもののみに注目すればよいことがわかります（図ではそのような文字は丸で囲まれています）。そのような T の丸で囲まれた文字を、操作の途中過程における文字列に含まれる同じ文字と線で結んでいき（この線は他の線と重なってはいけません）、最後に S_0 に含まれる同じ文字に到達することができます。線を引ける方向は左と下のみです。

操作の過程の途中で文字列がどのようなものであるかが不明であっても、T の丸で囲まれた文字と S_0 の文字を結ぶことさえできれば、そこから操作列を構成することができます。よって、元の問題の代わりに、$S_i = T$ の丸で囲まれた文字すべてから S_0 の同じ文字に線を引くことができるような最小の i を求めればよいです。

i を定めたとき、$S_i = T$ であるような線の引き方が存在するかどうかは、以下のような貪欲なアルゴリズムにより判定できます。T の丸で囲まれた文字を後ろから見ていき、それぞれの文字から可能な限り線を下に引いていき、左に引いていかざるを得ないときのみ左に引いていきます。もし途中で線がマス目からはみ出るようなことがあれば、$S_i = T$ であるような線の引き方は存在しません。

この正当性は以下のように示せます。$S_i = T$ であるような線の引き方が存在するとき、上のような貪欲な引き方でも線を引くことができることを示します。

任意の線の引き方に対し、それを貪欲な引き方で得られるようなものに変形していくことが以下のステップを繰り返すことで可能です。各ステップでは、引き方が貪欲なステップで得られるような引き方と最初に相違する箇所を見つけます。そこでは、貪欲な引き方が下に線を引いていっているところで、元の引き方では（我々から見て）左に曲がっているはずです。この左に曲がるのを 1 マス分遅らせ、曲がったら直前の引き方と一致するまで左に線を引き続けます。このように線の引き方を変形していく過程を下図に示します（赤 $\rightarrow$ 緑 $\rightarrow$ 青）。



この貪欲法と二分探索を組み合わせると答えを得られますが、時間計算量が $\mathrm{O}(log(N) \times N^2)$ となってしまいます。

この解法を高速化することを考えます。配列 H を用意し、$H[j]$ に線が j 列目を通ったときの縦方向の位置のうち最も上のものを記録します。はじめ、すべての j に対し $H[j] = 0$ とします。k を N から 1 にイテレートします。各イテレーションでは、答えが少なくとも $H[k]$ であること ($T[k] = T[k+1]$ のとき)、もしくは少なくとも答えが $H[k+1]$ であること ($T[k] \neq T[k+1]$ のとき) がわかります。$T[k]$ が丸で囲まれた文字であるときに H の値を更新します。まず、文字 $S_0[l]$ であって、$T[k]$ と同じ文字であり、$h[l] = 0$ であるようなもののうち最も右にあるものを求めます。すると、l 以上 $k-1$ 以下の r に対し、新しい $H[r]$ の値は (古い $H[r+1]$ の値) $+1$ となります（なぜなら、r 列目の線は $(H[r+1]+1)$ 行 $(r+1)$ 列目のマスから入ってくるからです)。これはマス目の行数を表す i の値によりません。したがって、二分探索が不要となり、解法の時間計算量が $\mathrm{O}(N^2)$ となりますが、まだ不十分です。

そこで、「l 以上 $k-1$ 以下の r に対し、(新しい $H[r]$ の値) $=$ (古い $H[r+1]$ の値) $+1$」という操作に注目します。この操作は、配列 H の $H[l+1]$ から $H[k]$ の区間にあるそれぞれの値を左隣に移して 1 を加える、という操作とみなすことができます（$H[k]$ 以降の値は以後無視されます)。したがって、もう一つの配列 h と二つの値 $\mathrm{offset}_{\mathrm{value}}$, $\mathrm{add}_{\mathrm{value}}$ を管理し、$h[i + \mathrm{offset}_{\mathrm{value}}] + \mathrm{add}_{\mathrm{value}} = H[i]$ を成り立たせることができます。このとき、操作を行う際は単に $\mathrm{offset}_{\mathrm{value}}$ と $\mathrm{add}_{\mathrm{value}}$ に 1 を加えればよいです（実際にはもう少し複雑な処理が必要ですが、省略します)。これにより、解法の時間計算量が $\mathrm{O}(N)$ となります。

AtCoder Grand Contest 007 Editorial

writer : dreamoon

A : Shik and Stone

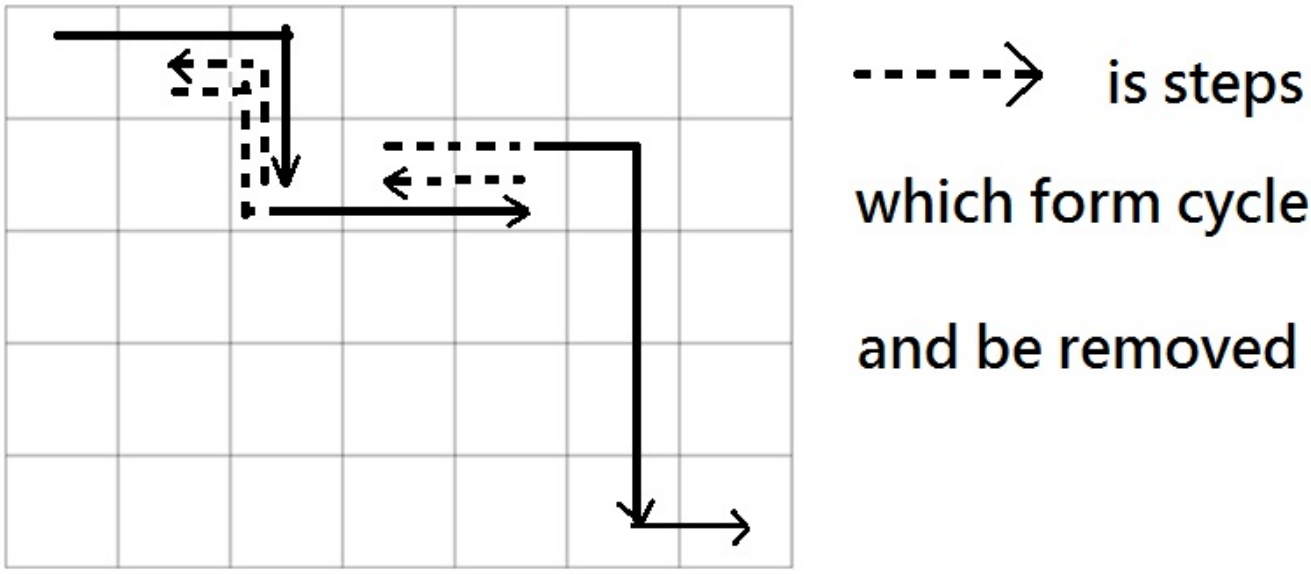
The solution of this problem is quite short. We can get the answer by counting the number of '#'s in the input. If the number is equal to $H+W-1$, the answer is 'Possible'. Otherwise, the answer is 'Impossible'. Now we want to prove this conclusion.

Firstly, the set of steps which only uses right and down moves(denoted as S_A) is equal to the set of steps using minimum moves(denoted as S_B). Let's prove it.

Denote g_i as the sum of row number and column number of the i-th position of stone. If using only right and down moves in i-th step, $g_{i+1} = g_i + 1$. Otherwise, for left and up moves, $g_{i+1} = g_i - 1$. That is, the absolute difference of two consequtive numbers in sequence g_i is always 1. Suppose Shik uses n steps to move the stone. We know that $g_1 = 2$ and $g_{n+1} = H + W$. Therefore, we can know the minimum value of n is $H + W - 2$ and $g_{i+1} = g_i + 1$ for all i must be satisfied when $n = H + W - 2$. Then we get that S_A and S_B are all sequences of moves such that $n = H + W - 2$.

Secondly, because '#' denotes that the stone had ever located at this position, $n + 1 \geq$ the number of '#'s. If the number of '#'s is larger than $H + W - 1$, n will be larger than $H + W - 2$, then Shik doesn't use the minimum moves. So in this condition the answer is always 'Impossible'.

Finally, because there always exists a valid sequence of moves for Shik in inputs. We can choose any of valid sequence first, then remove these steps such that the position sequence of stone form cycles (you can see the picture below for understanding). The removing is always possible becuase it's impossible to decrease the number of '#'s. If it is possible, it will conflict with the fact that the input is valid. After removing all unnecessary cycle steps, we can get a moving sequence which the number of steps added one is equal to the number of '#'s. Now we get a possible moving sequence using minimum steps which match the input.



Bonus problem: If the input dosn't need to be generate by some valid moving sequence. Can we use the same method to determine whether such input is generate by Shik with using only right and down moves in all steps.? If not, how to solve it?

B : Construct Sequences

First we give a possible solution then we justify that it satisfies all of the conditions:

Let $r_{p_i} = i$ for $1 \leq i \leq n$. $A_i = 30000 \times i$ for $1 \leq i \leq n$. $B_i = 30000 \times (n - i) + r_i$ for $1 \leq i \leq n$.

Now the (sketch of the) proof:

$30000 \times 1 \leq A_i \leq 30000 \times n \leq 30000 \times 20000 => 1 \leq A_i \leq 10^9$. $30000 \times 1 + 1 \leq B_i \leq 30000 \times n + n \leq 30000 \times 20000 + 20000 => 1 \leq B_i \leq 10^9$. $A_i = 30000 \times i < 30000 \times (i + 1) = A_{i+1}$. $B_i = 30000 \times (n-i) + r_i > 30000 \times (n-i) = 30000 \times (n-i-1) + 30000 > 30000 \times (n-i-1) + r_{i-1} = B_{i-1}$. $A_{p_i} + B_{p_i} = 30000 \times p_i + 30000 \times (n - p_i) + r_{p_i} = 30000 \times n + i < 30000 \times n + i + 1 = 30000 \times p_{i+1} + 30000 \times (n - p_{i+1}) + r_{p_{i+1}} = A_{i+1} + B_{i+1}$

C : Pushing Balls

Notice that after rolling one ball, if we renumber all the remaining balls and holes, the balls still satisfy that the i-th ball is between the i-th hole and the $(i + 1)$-th hole. Let's also recalculate the expected value of the distance between neighbouring items for all possible rolls of the last ball. We can find that the the expected value $\mathbf{E}[d_i]$ is still an arithmetic sequence with N decreased by 1.

So we can calculate the answer recursively: calculate the expected value of rolling distance of the first ball, and sum it with the remaining similar problem in reduced size.

D : Shik and Game

If the player decided to go back to pick up some coins, he will go back to the very first coin he hasn't picked up or even wait for the first coin to appear, otherwise he needs to go back for that coin again. So, we could partition all the bears into continuous segments to represent a strategy of the game. More specifically, we first give candies to all the bears in a segment, go back to collect all the coins in this segment and then proceed to the next segment.

Assuming we have M segments and the i-th segment starts at position L_i and ends at position R_i, the answer to the problem is $E + \sum_{i=1}^{M} \max(t, 2 \times (R_i - L_i))$. Let $dp[i]$ be the answer to the problem with only first i bears, we have $dp[i] = \min_j(dp[j] + \max(t, 2 \times (a_i - a_{j+1})))$, an $O(N^2)$ dp solution.

To accelerate the solution, we can split the max part in the above equation into two parts: $t > 2 \times (a_i - a_{j+1})$ and $t \leq 2 \times (a_i - a_{j+1})$. It's trivial that for each i we can find a pivot p such that the first part consists of all $j > p$ and the second part holds for all $j \leq p$. Furthermore, it's easy to show that p is non-decreasing for all i, so we can maintain p using amortized $O(N)$ time for all i.

For the first part, the equation is now $\min_j(dp[j]+t)$. $dp[]$ is non-decreasing, thus the smallest possible j will leads to the optimal solution in all j. We can calculate this part greedily using $O(N)$ time for all i.

For the second part, the equation is $\min_j(dp[j] + 2 \times (a_i - a_{j+1}))$. We can rewire it as $2 \times a_i + \min_j(dp[j] - 2 \times a_{j+1})$. By maintaining another dp array $dp2[i] = \min_{j=1}^{i}(dp[i] - 2 \times a_{i+1})$ in $O(N)$ time for all i, we can calculate $2 \times a_i + \min_j(dp[j] - 2 \times a_{j+1}) = 2 \times a_i + dp2[j]$ in $O(N)$ time for all i.

To this point, we have improved the time complexity of the solution from $O(N^2)$ to $O(N)$.

E : Shik and Travel

You may have the intuition that the first step of solving this problem is to apply binary search on answer. It's also the first step of the solution provided here. So, we are going to study the sub problem here: whether there exists an arrangements of leaf cities to stay during the travel such that the answer is not larger than a certain value V.

Firstly, we should know one thing: in the travel, once the employee passed a road i, he/she will visit all cities belong to the descendants in the subtree rooted at city $i + 1$ before passing

the road i again. So for any given city, Shik always stay at all leaf cities in the subtree rooted from this city in consecutive days. More precisely, if the subtree of city i contains n_i leaves, there exist some number st_i such that in the last part of day st_i, the first part of day $st_i + n_i$ and all days from day $st_i + 1$ to day $st_i + n_i - 1$, Shik is travelling in the subtree of city i. Nevertheless, in other days the employee won't be in this subtree.

After knowing that, we give a straightforward recursive algorithm to solve this sub problem. The main idea is to recursively merge possible answers from both children to produce all possible answers of a vertex. Following is the pseudo code:

```
function f
argument: root city i of some subtree
return value: a set S_i contains ordered pairs (a,b),
              each representing there exists one possible
              order of stay sequence of leaf cities such
              that the sum of tolls at the st_i day in
              this subtree is a, the sum of tolls at the
              day t_i + n_i in this subtree is b and sum
              of tolls in any days from day st_i + 1 to day
              st_i + n_i - 1 is not larger than V.

f(i) {
    if vertex i is leaf:
        return (0,0)
    set S_i as empty set
    let two children cities of city i to be j and k
    let S_j = f(j) and S_k = f(k)
    for all combinations of (a,b) in S_j and (c,d) in S_k:
        if(b + c + v_j + v_k ≤ V) add (a + v_j, d + v_k) to S_i
        if(a + d + v_j + v_k ≤ V) add (c + v_k, b + v_j) to S_i
    return S_i
}
```

After calling $f(1)$ and getting S_1, we can know the answer by whether S_1 is empty or not.

But the function may produce quite large $|S_i|$. So we want to reduce it. Because our purpose is only to know whether S_1 is empty or not. We can remove all (a, b) in S_i if there is another pair (a', b') in S_i satisfying $a' \leq a$ and $b' \leq b$.

Now we want to prove after performing such removal, we have inequality: $|S_i| \leq 2\min(|S_j|, |S_k|)$:

Assuming $|S_j| \leq |S_k|$. When a specific pair $(a, b) \in S_j$ matching all pairs in S_k, the generated pairs will be in the form of $(a + v_j, u)$ and $(u, b + v_j)$ (exact value of u depends on the matched pair in S_k). Each aforementioned form will remain at most one pair for a given pair $(a, b) \in S_j$ (the one with minimum u among all possible values of u). So the maximum size of resulting set is at most double of $|S_j|$. The proof for $|S_k| \leq |S_j|$ part is similar.

Such inequality make many things wonderful! The most important thing for us is that

$\sum_{i=1}^{N} |S_i| = O(NlogN)$. Proof is given as the following:

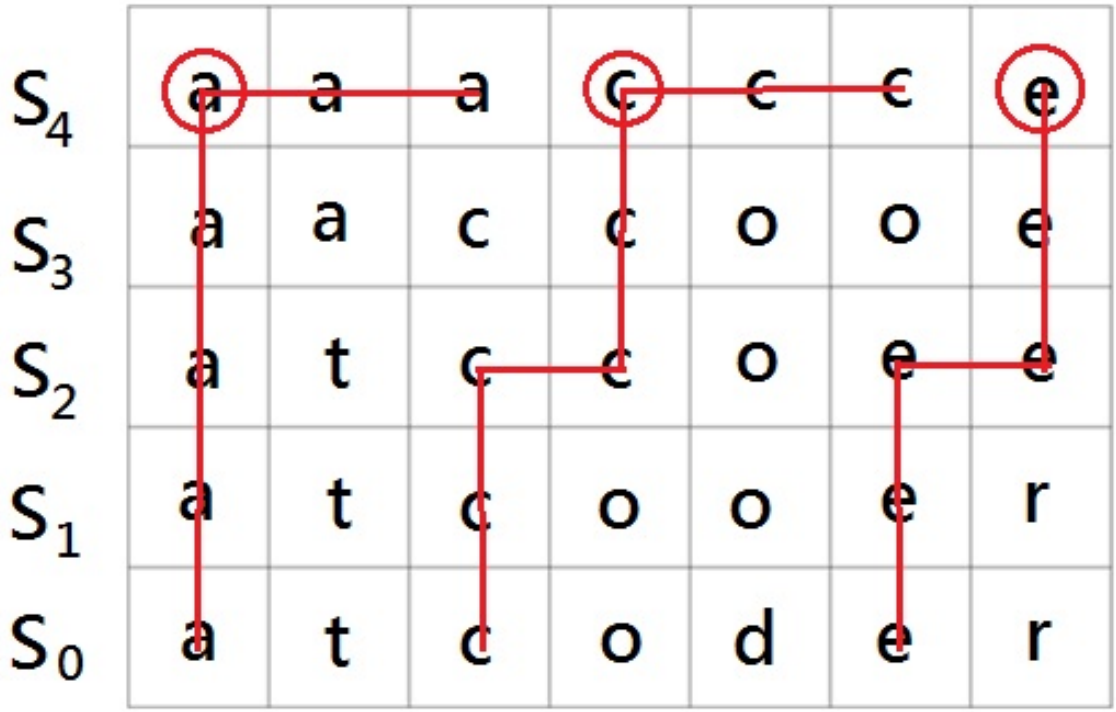
Denote the distance between node i and the root (i.e. depth) as d_i. For each city i, let j be a leaf city in the subtree of city i which has minimum d_j, we have $|S_i| \leq 2^{d_j - d_i}$. So, we can amortize the value of $|S_i|$ to all cities in the subtree of city i with depth exactly equals to d_j by adding value 1 to each of such city. Note that there are exactly $2^{d_j - d_i}$ such cities because d_j is the smallest depth in the subtree. The amortized value in one vertex is at most $log_2(N+1)$; This is because only the parents of a node who has a maximum distance of $\log_2(N+1)$ to the node has the chance to add value to the node, otherwise the size of the tree would be larger than N (since we have a complete subtree of depth greater than $log_2(N+1)$). As a conclusion, we get $\sum_{i=1}^{N} |S_i| \leq N \times log_2(N+1)$ so $\sum_{i=1}^{N} |S_i| = O(NlogN)$.

With the fact we can use two-pointer technique to find the minimum value of u in the two forms $(a+v_j, u)$ and $(u, b+v_j)$ since we can maintain that S to be sorted. The time complexity of the sub problem will be $\sum_{i=1}^{N} |S_i| = O(NlogN)$. Go back to the original problem. The time complexity will be $O(NlogN \times log\ answer)$.

F : Shik and Copying String

Note that we don't consider the case $S_0 = T$. It's answer is 0 obviously.

In the beginning, I'd like to roughly give a picture about how the string copying works and what's the relation between string S_0 and string T. Please see picture below.



In this picture, we suppose $S_0 =$ "atcoder" and $T =$ "aaaccce". It shows one possible process such that $S_4 = T$ (Note that this picture does NOT show the process of the optimize solution for minimum i). We are only concerned about the first letter in and these letters which is different to previous letter of themself (the circled letter in the picture). We can connect these circled letters to their source letters one by one with disjoint lines and finally stop at letters in S_0. You may notice that these line can only go either left or down.

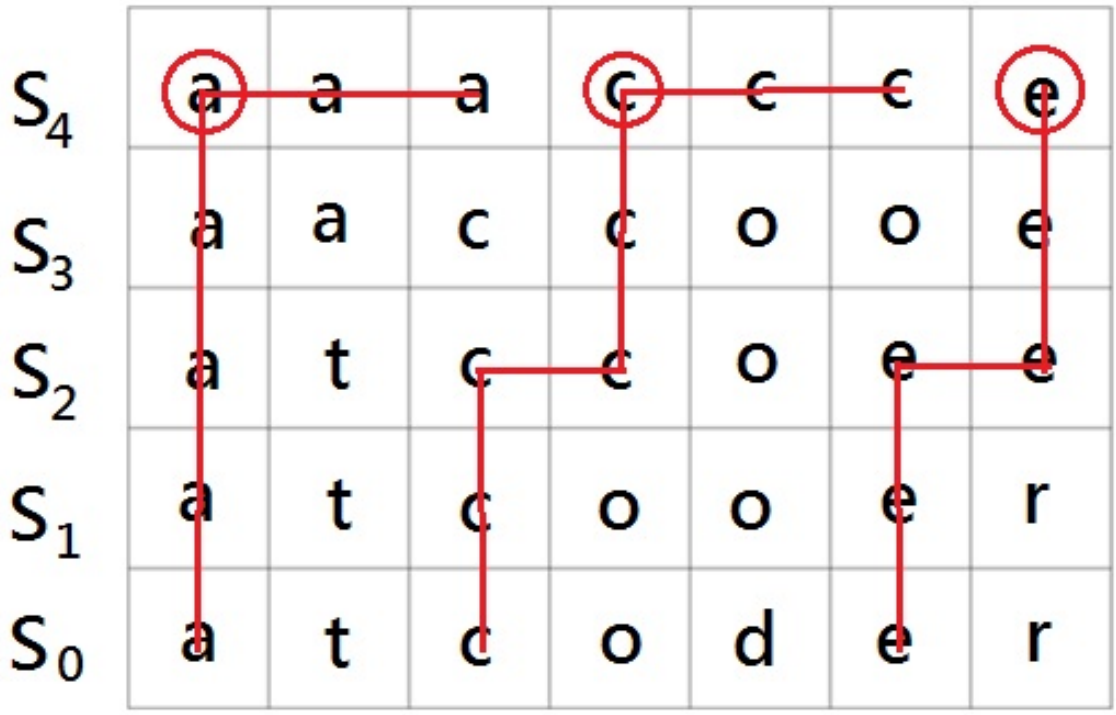
In the other face, suppose we don't know the process between S_0 and T. But if we can draw lines from all circled letters in T to the same letter in S_0, we can construct one possible process easily. So we can transform the original problem of finding minimum i to check if we can draws lines from $S_i = T$ to S_0.

Now we give a greedy algorithm in intuition to test whether exist configures of line for some i with $S_i = T$: for each circled letter from back to front, we draw line to down first if possible, otherwise we draw it to left. If in some step the drawn lines go out of bound, then there exists no valid process such that $S_i = T$.

This solution can be proven easily. What we should prove is, if there exists some valid process for $S_i = T$, then the algorithm in this solution will also construct a valid process.

We can always adjust any valid configuration of lines to match the lines produced by our algorithm step by step. In each step, we find the first time that the lines don't draw by our algorithm and change the direction(always from left to down) of the drew line at this point

for one unit, and continue to draw the line to left until meeting the lines in previous step. The picture below show the steps from red lines to meet our algorithm (red -¿ green -¿ blue).



Using this greedy algorithm we can get a binary search answer + simulation solution for this problems. But the time complexity is $O(log(N) \times N^2)$.

For speeding up this algorithm, we maintain an array $H[j]$ denoting the highest position passed by the line in column j. Initially, $H[j] = 0$ for all j. We iterate k from N to 1. In each iteration, we can know the answer should at least be $H[k]$ (if $T[k] = T[k+1]$) or $H[k+1]$ (if $T[k] \neq T[k+1]$). If $T[k]$ is a circled letter, we will update values in array H. Next, we will find the most right letter $S_0[l]$ which is the same with $T[k]$ and $h[l] = 0$. Then the new $H[r]$ will be equal to old $H[r+1]+1$ for r from l to $k-1$ (it's because the line in the r-th column is come from the $(H[r+1]+1)$-th row in the $(r+1)$-th column) no matter which i satisfies $S_i = T$. Now we have an algorithm without binary search and time complexity is $O(N^2)$. But it's still not fast enough to solve this problem.

Now we focus on the operation: new $H[r]$ = old $H[r+1]+1$ for r from l to $k-1$. This operation just give the the part of array with index from $l+1$ to k value of H in array one unit of left offset and add one to all of them and we don't care the value in array H not small than k more! So we can maintain another array h along with two values $offset_{value}$ and add_{value} such the the value $h[i + offset_{value}] + add_{value}$ equals to $H[i]$. After that, in each time we want to do this operation we can only increase both $offset_{value}$ and add_{value} by one(with

some more detail not covered here). Finally, we get an algorithm with time complexity $O(N)$ which can solve this problem!

AtCoder Grand Contest 006 解説

writer : sugim48

English editorial starts on page 8.

A : Prefix and Suffix

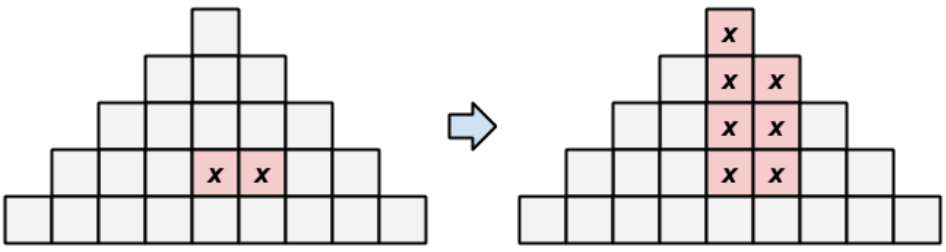
長さの上限は $2N$ です．なぜならば，s と t をそのまま連結した文字列は条件を満たすからです．よって，長さを N から $2N$ まで小さい順に試していって，条件を満たす文字列が見つかった時点でその長さを答えとすればよいです．

長さが l $(N \leq l \leq 2N)$ であるとき，条件を満たす文字列が存在するか，どう判定すればよいでしょうか？ 条件を満たす文字列は，s と t を $2N - l$ 分だけオーバーラップさせたような文字列になります．そのため，s の末尾 $2N - l$ 文字と，t の先頭 $2N - l$ 文字が，完全に一致しているか判定すればよいです．

B : Median Pyramid Easy

まず，$x = 1,\ 2N - 1$ の場合は `Impossible` です．これらは順列の最小値 / 最大値なので，下から 2 段目の時点で完全に消えてしまうからです．逆に，$2 \leq x \leq 2N - 2$ ならば必ず `Possible` です．以降，解を具体的に構成する方法を説明します．

$N = 2$ ならば，解は適当に $(1,\ 2,\ 3)$ などとすればよいです．以降は $N \geq 3$ とします．重要な観察として，左図の赤い 2 マスが x になれば，右図のように自然と頂上のマスも x になります．（このとき，他のマスの値は関係ありません．）



よって，左図の赤い 2 マスを x にするような順列を構成すればよいです．これは例えば

$$(\ldots,\ x-1,\ x,\ x+1,\ x-2,\ \ldots)$$

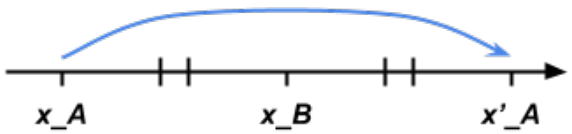
などとすればよいです．ただし，この構成は $x=2$ の場合のみ不適です．$x=2$ の場合は代わりに

$$(\ldots,\ x+1,\ x,\ x-1,\ x+2,\ \ldots)$$

などとすればよいです．

C : Rabbit Exercise

まず，「うさぎ A がうさぎ B に関して対称な座標へ移動する」という操作はどういうことかを考えます．うさぎ A, B の今の座標をそれぞれ x_A, x_B とおき，うさぎ A の新しい座標を x'_A とおきます．すると，x_A と x'_A の中間の値がちょうど x_B なので，$(x_A + x'_A)/2 = x_B$ です．この式を変形することで，$x'_A = 2x_B - x_A$ が得られます．「うさぎ A がうさぎ B に関して対称な座標へ移動する」という操作を数式で書くことができました．



確率論的な現象のままでは考えづらいので，決定論的な現象に言い換えます．今，うさぎ i が，うさぎ $i-1$ またはうさぎ $i+1$ を等確率で選び，選ばれたうさぎに関して対称な座標へ移動するとします．うさぎ $i-1, i, i+1$ の今の座標をそれぞれ x_{i-1}, x_i, x_{i+1} とおき，うさぎ i の新しい座標を x'_i とおきます．このとき，確率 $1/2$ で $x'_i = 2x_{i-1} - x_i$ となり，確率 $1/2$ で $x'_i = 2x_{i+1} - x_i$ となります．よって，x'_i の期待値 $\mathrm{E}[x'_i]$ は

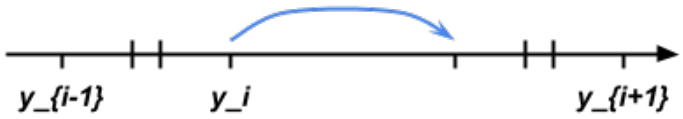
$$\begin{aligned}\mathrm{E}[x'_i] &= \frac{1}{2}\mathrm{E}[2x_{i-1} - x_i] + \frac{1}{2}\mathrm{E}[2x_{i+1} - x_i] \\ &= \frac{1}{2}(2\mathrm{E}[x_{i-1}] - \mathrm{E}[x_i]) + \frac{1}{2}(2\mathrm{E}[x_{i+1}] - \mathrm{E}[x_i]) \\ &= \mathrm{E}[x_{i-1}] + \mathrm{E}[x_{i+1}] - \mathrm{E}[x_i]\end{aligned}$$

と計算できます．よって，x_i の期待値を y_i とおくことで，問題は次のように言い換えられます．

> 数列 $y_1, y_2, \ldots, y_N$ がある．この数列に何回か操作を行う．1 セット分の操作列は，次のような合計 M 回の操作からなる．j 回目の操作では，$i := a_j$ とし，$y_i \leftarrow y_{i-1} + y_{i+1} - y_i$ と置き換える．以上の合計 M 回の操作を 1 セット分の操作列として，K セット分の操作列を続けて繰り返す．最終的な数列を求めよ．

決定論的な現象に言い換えることができました．

以上の考察により，$K = 1$ の場合は愚直にシミュレーションすることで解くことができます．K が大きい場合はどう解けばよいでしょうか？ まず思いつくのは，1 セット分の操作列によって数列がどう変わるかを行列の形で表し，行列累乗をするという方法です．しかし，この方法の計算量は $\mathrm{O}(N^3 \log K)$ と大きすぎます．ここで，$y_i \leftarrow y_{i-1} + y_{i+1} - y_i$ という操作をよく観察すると，$y_i - y_{i-1}$ の値と $y_{i+1} - y_i$ の値が交換されていることに気づきます．



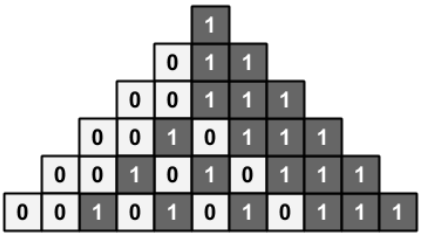
よって，$\mathrm{d}y_i := y_{i+1} - y_i$ と長さ $N-1$ の数列を定義することで，y_i に対する操作を，$\mathrm{d}y_i$ に対する隣接要素のスワップと捉え直すことができます．すると，1 セット分の操作列の結果は $\mathrm{d}y_i$ に対する置換として表すことができます．長さ N の置換どうしの積は $\mathrm{O}(N)$ で計算できるので，行列累乗の要領で置換の累乗をすると，計算量は $\mathrm{O}(N \log K)$ となります．これは十分高速です．最終的な数列 $\mathrm{d}y_i$ が求まれば，その累積和をとることで数列 y_i が復元できます．

D : Median Pyramid Hard

愚直にすべてのマスを埋めていく方法だと，$\mathrm{O}(N^2)$ 時間掛かって間に合いません．そこで，頂上のマスの値 x を二分探索することを考えます．具体的には，「頂上のマスの値が x 以上か」という条件を用い，二分探索することを考えます．

「頂上のマスの値が x 以上か」という判定問題を高速に解くにはどうすればよいでしょうか？「頂上のマスの値が x 以上か」ということだけが気になっているので，最下段の順列の各値を，x 以上ならば 1，x より小さいならば 0，と表してしまいましょう．さらに，これから埋めていくマスの値についても，同様に 0/1 で表してしまいましょう．このように各値を 0/1 へ表してしまっても，マスを埋めていくルールはまったく同じものが適用できることが分かります．具体的には，「あるマスの値は，そのマスの左下，真下，右下の 3 マスの値の多数決で決まる」というルールになります．この単純化された問題設定ならば，頂上のマスの値を高速に計算できそうです．以降，具体的な計算方法を説明します．

まず，すぐに分かるのは，0 または 1 が複数マス連続している区間については，その真上の区間は変化しないということです．よって，頂上のマスがこのような区間に含まれていれば，頂上のマスの値は簡単に求まります．では逆に，0/1 が交互に並んでいる区間については，その真上の区間はどう変化していくでしょうか？ たとえば，図のように変化していきます．



つまり，0/1 が交互に並んでいる区間については，0/1 が反転しながら，幅が左右 1 マスずつ狭くなっていきます．一方，左右の連続区間については，幅が 1 マスずつ広くなっていきます．そして，左右の連続区間がぶつかったところで，変化が止まります．この挙動をよく観察することで，頂上のマスの値を計算できます．ただし，最下段全体で 0/1 が交互に並んでいる場合に注意してください．

以上の方法を用いると，頂上のマスの値を $\mathrm{O}(N)$ 時間で計算できます．この値が 0 か 1 かで二分探索の分岐をすることで，頂上のマスの真の値が求まります．全体の計算時間は $\mathrm{O}(N \log N)$ で，十分に間に合います．

E : Rotate 3x3

まずは，Rotate 2x2 という問題を考えてみましょう．つまり，マス目の縦幅が 2 であり，正方形の大きさが 2×2 であるような，簡単な問題設定です．2×2 の正方形内の配置を 180° 回転するとはどういうことか考えてみましょう．これは，隣り合う 2 列の位置をスワップし，同時に各列を上下反転する操作と見ることができます．操作を行うたびに列が上下反転されると考えづらいので，次のように見方を工夫することにします．マス目のうち偶数列目の位置にある列は，常に上下が反転して表示されるとします．すると，先述の操作は，隣り合う 2 列の位置をスワップするだけの操作に見えることになります．よって，初期配置と目標配置のそれぞれについて偶数列目を上下反転しておけば，あとはこれらの配置がスワップ操作のみによって移り合えるか判定するだけの問題になります．これは結局，初期配置と目標配置で列の集合が一致しているか判定すればよいです．以上より，Rotate 2x2 が解けました．

さて，元の問題である Rotate 3x3 に戻りましょう．まずは，3×3 の正方形内の配置を 180° 回転するとはどういうことか考えてみましょう．これは，左の列と右の列の位置をスワップし，同時に各列を上下反転した後，さらに真ん中の列も上下反転する操作と見ることができます．ここでも，Rotate 2x2 と同様に見方を工夫することで，問題設定を簡単にします．つまり，3, 4, 7, 8, 11, 12, . . . 列目の位置にある列は，常に上下が反転して表示されるとします．すると，先述の操作は，左の列と右の列の位置をスワップした後，真ん中の列を上下反転する操作に見えることになります．また，初期配置と目標配置のそれぞれについて，3, 4, 7, 8, 11, 12, . . . 列目を上下反転しておきます．このように簡単化された問題設定において，解法を考えていきます．

「左の列と右の列の位置をスワップした後，真ん中の列を上下反転する」操作では，奇数列目ど

うし，または，偶数列目どうしがスワップされます．よって，奇数列目の並べ替えと，偶数列目の並べ替えは，ある程度独立に考えることができます．ここでは，奇数列目の並べ替えに注目してみましょう．とりあえず，「隣り合う列のスワップ」と「ある列の上下反転」の 2 種類の操作が好きな順番で行えるとしたとき，初期配置（の奇数列目）から目標配置（の奇数列目）へ変形できないならば，答えは `Impossible` とすぐ分かります．変形できるならば，「隣り合う列のスワップ」の最小回数と，「ある列の上下反転」の最小回数を計算し，それぞれ inv_o, $flip_o$ としておきます．同様に，偶数列目についても，変形できるかどうかを判定し，inv_e, $flip_e$ を計算しておきます．実は，inv_o, $flip_o$, inv_e, $flip_e$ の偶奇をチェックすることで，`Possible` か `Impossible` かを判定できます．

まず，inv_o, $flip_o$, inv_e, $flip_e$ の偶奇に関する必要条件を考えてみましょう．「左の列と右の列の位置をスワップした後，真ん中の列を上下反転する」操作を行うと，inv_o と $flip_e$ の偶奇がともに変化するか，または，inv_e と $flip_o$ の偶奇がともに変化します．よって，inv_o と $flip_e$ の偶奇は一致している必要があり，かつ，inv_e と $flip_o$ の偶奇は一致している必要があります．実は，$N \geq 5$ ならば，これらは十分条件になっていることが示せます（証明は後述）．以上より，Rotate 3x3 が解けました．転倒数を計算するパートが最も重く，全体の計算量は $\mathrm{O}(N \log N)$ です．

(十分性の証明) inv_o と $flip_e$ の偶奇が一致していて，かつ，inv_e と $flip_o$ の偶奇が一致していれば，`Possible` であることを示します．「左の列と右の列の位置をスワップした後，真ん中の列を上下反転する」操作をうまく組み合わせることで，奇数列目から任意の 2 列，または，偶数列目から任意の 2 列を選び，それらをともに上下反転できる，ということが示せれば十分です．これを示すためには，距離がちょうど 2 だけ離れた任意の 2 列を選び，それらをともに上下反転できる，ということが示せれば十分だと分かります．以降，これを示します．

マス目のうち，連続する 5 列を好きな位置から取り出してきます．この 5 列を `a b c d e` と表すことにします．さらに，上下反転した列を `A` のように大文字で表すことにします．次のような操作列を構成してみます．

- `a b c d e`
- `c B a d e`
- `c B e D a`
- `e b c D a`
- `e b a d c`
- `a B e d c`
- `a B c D e` (1)
- `a D C B e`
- `C d a B e`
- `C B A d e`
- `A b C d e` (2)

(1) および (2) を見ると，距離がちょうど 2 だけ離れた 2 列をともに上下反転できていることが分かります．連続する 5 列は好きな位置から取り出せるので，距離がちょうど 2 だけ離れた任意の 2 列を選べることになります．以上より，十分性が証明できました．

F : Blackout

とりあえず，次のように問題を言い換えます．

> N 頂点 M 辺の有向グラフがある．辺 $(x, y), (y, z)$ が存在し，辺 (z, x) が存在しないならば，辺 (z, x) を張ることができる．可能な限り辺を張ったとき，最終的な辺の本数を求めよ．

明らかに，グラフの弱連結成分（有向辺を無向辺と見たときの連結成分）どうしは独立に計算することができ，答えはそれらの総和になります．よって，以降はグラフ全体が弱連結であると仮定します．

突然ですが，N 個の頂点に A, B, C のどれかのラベルを付けて，次の条件が満たされるようにしましょう．

- M 本の辺のそれぞれについて，その始点と終点のラベルは (A → B) または (B → C) または (C → A) のようになっている．

このようなラベル付けは一意ではないですし，そもそもどうラベル付けしても矛盾してしまう場合もあります．しかし，DFS などをすることで，「ラベル付けをひとつ求める」か「矛盾を検出する」ことができます．

ラベル付けの結果で場合分けをすることで，最終的な辺の張られ方が次のように求まります．証明は後述します．

- A, B, C すべてのラベルが使われていない場合
  新たに辺を張ることができないので，最初の辺集合のままである．
- A, B, C すべてのラベルが使われている場合
  (A → B) または (B → C) または (C → A) のような辺はすべて張ることができる．それ以外の辺は張ることができない．
- どうラベル付けしても矛盾してしまう場合
  任意の頂点から任意の頂点へ辺を張ることができる．特に，任意の頂点から自己ループを張ることができる．

どの場合においても，最終的な辺の本数は簡単に計算できます．よって，答えを求めることができました．計算量は $O(N + M)$ です．

（A, B, C すべてのラベルが使われていない場合の略証）この場合は，辺 $(x, y), (y, z)$ の組が存

在しないので，新しく辺を張ることができません．□

（A, B, C すべてのラベルが使われている場合の略証）この場合は，辺 (x,y), (y,z) の組がどこかに存在するはずなので，辺 (z,x) を新しく張って，頂点 x, y, z からなる三角形を作っておきます．今，x, y, z のラベルがそれぞれ A, B, C であるとしても，一般性を失いません．最初の目標は，「任意の頂点 v について，v が x, y, z のうち 2 個と辺で結ばれている」ようにすることです．たとえば，v のラベルが A ならば，辺 (v,y), (z,v) が存在するようにします．これは，三角形 x, y, z に近い頂点から順に，所望の辺を張っていくことで達成できます．「任意の頂点 v について，v が x, y, z のうち 2 個と辺で結ばれている」ようにできれば，その繋がりを利用して，(A $\to$ B) または (B $\to$ C) または (C $\to$ A) のような辺をすべて張ることができます．逆に，それ以外の辺は張れないこともすぐに分かります．□

（どうラベル付けしても矛盾してしまう場合の略証）目標は，ある頂点について自己ループを張ることです．これができれば，ひとつ前の略証と同様にして，任意の頂点から任意の頂点へ辺を張ることができます．まず，どうラベル付けしても矛盾するようなサイクルをひとつ見つけてきます．（このサイクルは必ずしも有向閉路である必要はなく，無向辺として見たときにサイクルであればよいです．）このサイクルには必ず辺 (x,y), (y,z) の組がどこかに含まれるので，辺 (z,x) を新しく張り，サイクルのサイズをひとつだけ小さくします．このとき，ひとつだけ小さくなったサイクルも，どうラベル付けしても矛盾することが分かります．この操作を繰り返していくことにより，サイクルのサイズを 1 まで小さくすることができます．これは欲しかった自己ループそのものです．□

AtCoder Grand Contest 006 Editorial

writer : sugim48

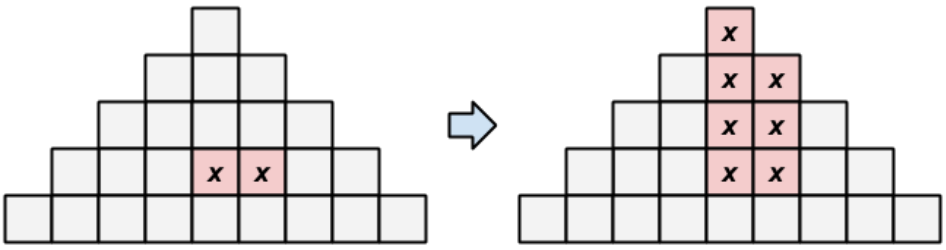
A : Prefix and Suffix

The answer is always at most $2N$: the concatenation of s and t satisfies the conditions. Thus, for each l ($N \leq l \leq 2N$), we want to check whether there exists a string of length l that satisfies the conditions. Such strings exist if and only if the last $2N - l$ characters of s and the first $2N - l$ characters of t are the same.

B : Median Pyramid Easy

In case $x = 1,\ 2N-1$, the answer is `Impossible`. This is because since 1 and $2N-1$ are the max/min of the permutation, these numbers can't appear in the second row from the bottom. Otherwise, the answer is always `Possible`, as explicitly constructed as follows:

Suppose that $N \geq 3$ (if $N = 2$, the answer is $(1,\ 2,\ 3)$ for example). An important observation is that, if the two red cells in the left picture are x, the topmost cell will also become x as the right picture shows.



Thus, it is sufficient to come up with a permutation that makes the two red cells x. For example, when $x \neq 2$, the permutation can be

$$(\ldots,\ x-1,\ x,\ x+1,\ x-2,\ \ldots)$$

and when $x = 2$,

$$(\ldots,\ x+1,\ x,\ x-1,\ x+2,\ \ldots)$$

C : Rabbit Exercise

Everything about this task is to notice a very simple, but yet hard-to-notice trick. If you notice the trick, the task looks like an *A + B problem* and you can solve the task immediately, while if you don't the task looks like *the hardest task you've ever seen* and you will never solve the task forever in your life!

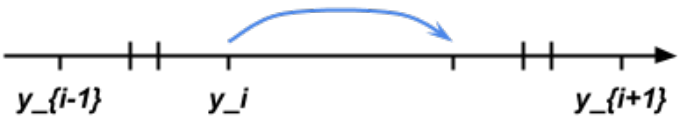
When an operation is performed on the rabbit i, the expected position of this rabbit after the operation is:

$$\frac{1}{2}(2x_{i-1} - x_i) + \frac{1}{2}(2x_{i+1} - x_i) = x_{i-1} + x_{i+1} - x_i$$

Thus, an operation on the rabbit i is equivalent to:

$$x_i := x_{i-1} + x_{i+1} - x_i$$

What does this operation mean? Let's draw a diagram that shows the positions of $y_i, y_{i-1}, y_{i+1}, y_{i-1} + y_{i+1} - y_i$:



Here the trick comes. The operation just swaps (the distance between the rabbits i and $i+1$) and (the distance between the rabbits i and $i-1$)!

Now everything becomes easy. Let $\mathrm{d}x_i := x_{i+1} - x_i$, and an operation on the rabbit i means

$$swap(\mathrm{d}x_{i-1}, \mathrm{d}x_i);$$

Thus, the problem can be solved in $\mathrm{O}(N \log K)$ time by computing an exponentiation of a permutation.

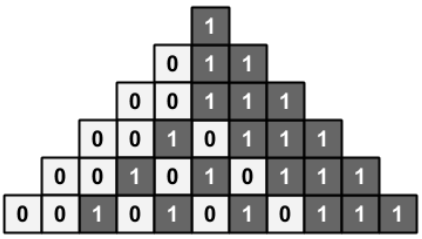
D : Median Pyramid Hard

The straightforward simulation is $\mathrm{O}(N^2)$. It may be surprising, but we can use binary search to achieve a better complexity. For a given value x, let's try to decide whether the value on the top is at least x or not. If we can do this, we can compute the value on the top by repeating $\mathrm{O}(logN)$ questions of this type by using binary search.

In order to answer this question, we don't need to distinguish integers less than x, and we don't need to distinguish integers greater than or equal to x. The only important thing is that the integers in the cells are at least x or not. Thus, let's replace all integers less than x with 0, and the other integers with 1. The rule for computing the median is exactly the same: when we decide a value of a cell, we check three integers (directly or diagonally) under the current cell, and fill it with the mode (the most frequent value) among the three integers. This simplifies the simulation. Indeed, when the values are only 0 and 1, we can compute the value on the top in linear time, as described below.

Suppose that two zeroes are horizontally adjacent in the bottom row. In this case, you can see that all values in these two columns will be zero. The same happens for horizontally adjacent ones.

What happens when 0 and 1 appear alternately? See the following diagram:



When there are two horizontally adjacent same numbers, this number also appears on top of it, and the "width" of the range of this number increases as we go up, and makes a V-shape. When two shapes of this type meet, they stop in the middle. In other areas, the value of a cell is the negation of the value directly below it. You can compute the value on the top by using these properties. (Make sure not to forget the case where the bottom row entirely alternates).

This way, when the values are only 0 or 1, we can compute the value on the top in $O(N)$ time. With binary search, we can solve the original problem in $O(N \log N)$ time in total.

E : Rotate 3x3

What happens when you rotate a 3×3 subrectangle by 180 degrees? This is equivalent to the following operation: swap the left column and the right column, and then flip all three columns.

Thus, even if you repeat multiple operations, the set of columns won't change. For each k, the three numbers $3k+1, 3k+2, 3k+3$ will be always in the same column. However, the order of these numbers may be reversed (here "reverse" means "upside down"). In this column, the numbers are $3k+1, 3k+2, 3k+3$ or $3k+3, 3k+2, 3k+1$ from top to bottom.

Also, you can notice that odd-indexed columns are always swapped with odd-indexed

columns. For each integer l, the numbers $6l+1, 6l+2, 6l+3$ will be always in an odd-indexed column. The same holds for even-indexed columns: the numbers $6l+4, 6l+5, 6l+6$ will be always in an even-indexed column.

It is clear that these conditions are necessary - but are these sufficient? To check this, the easiest way is to count the number of reachable configurations. For example, when $N = 10$, you can check that there are 3686400 reachable configurations by writing a brute force program. The conditions above will give $120 \times 120 \times 2^{10} = 14745600$ configurations. This is 4 times bigger than the actual number and the conditions are not sufficient - and it suggests that there are two types of "parities".

Let's think about odd-indexed columns. Consider two operations: "swap two consecutive (that is, two columns with distance 2) odd-indexed columns" and "reverse an odd-indexed column". Clearly, if you can't reach the final configuration from the initial configuration by using these two types of operations, the answer is `Impossible`. If you can, let inv_o be the number of operations of the former type, and $flip_o$ be the number of operations of the latter type. The values of inv_o and $flip_o$ can't be uniquely determined, but their parities are unique. Similarly, compute inv_e and $flip_e$ for even-indexed columns.

We want to come up with the necessary conditions for inv_o, $flip_o$, inv_e, and $flip_e$. If you perform the operation "swap the left column and the right column, and then flip all three columns", one of the following two things will happen:

- The parities of inv_o and $flip_e$ change.
- The parities of inv_e and $flip_o$ change.

Thus, the parities of inv_o and $flip_e$ must be the same, and the parities of inv_e and $flip_o$ must be the same. Now we found two "parities". We can prove that, when $N \geq 5$, these conditions are indeed sufficient (proved later). Finally we solved the problem. The time complexity is $O(N)$ in total.

(Proof) Now, we'll give a proof that the conditions above are indeed sufficient. That is, we want to prove that whenever the parities of inv_o and $flip_e$ are the same, and the parities of inv_e and $flip_o$ are the same, the answer is `Possible`. It is sufficient to prove that, by repeating the operation "swap the left column and the right column, and then flip all three columns", you can reverse arbitrary two odd-indexed (or even-indexed) columns. Furthermore, it is sufficient to prove that you can choose two rows with distance 2, and reverse both columns.

Let `a b c d e` be consecutive 5 columns. We'll denote reversed columns as an uppercase letter, for example `A` is a reversed `a`.

Consider the following sequence of operations:

- `a b c d e`

- c B a d e
- c B e D a
- e b c D a
- e b a d c
- a B e d c
- a B c D e (1)
- a D C B e
- C d a B e
- C B A d e
- A b C d e (2)

- a b c d e
- C B A d e
- C B E D a
- e b c D a
- e b A d C
- a B E d C
- a B c D e (1)
- a d C b e
- c D A b e
- c B a d e
- A b C d e (2)

If we see (1) and (2), we can see that we reversed two rows with distance 2. Therefore, we proved that the conditions we gave are sufficient.

F : Blackout

Let's state the problem in terms of graph theory.

> You are given a graph with N vertices and M edges. When there are two edges $x \to y$ and $y \to z$, you can add an edge $z \to x$. You keep adding edges while you can. How many edges will be there at the end?

Obviously, we can compute the answer for each weakly-connected component (that is, the connected component when we ignore directions), and the answer is the sum of these numbers. We'll assume that the graph is (weakly) connected.

Let's do some experiments. Consider a path graph. That is, the vertices are labeled with

integers, and there is an edge from vertex i to $i+1$ for each integer i. If you repeat operations on this graph, you can see that there will be an edge from s to t if and only if $t \equiv s + 1($ (mod 3)). It suggests that the problem has something to do with the length of path modulo 3.

Let's try to label the N vertices with labels `A`, `B`, `C` such that:

- For each of the M edges, the labels of the endpoints of the edge is (`A` $\to$ `B`) or (`B` $\to$ `C`) or (`C` $\to$ `A`).

Such labelling may not exist, but when it exists the labelling is unique (except for cyclic shift). By DFS, you can construct such an labelling (or conclude that it doesn't exist).

There are three cases depending on the result of labelling. We'll first state the conclusion, and later we'll give the proof.

- When the labelling is possible, and not all of `A`, `B`, `C` are used
  You can't perform any operations and the set of edges remain unchanged.
- When the labelling is possible, and all of `A`, `B`, `C` are used
  You can add an edge between all pairs with labelling (`A` $\to$ `B`), (`B` $\to$ `C`), and (`C` $\to$ `A`).
  You can't add any other edges.
- When the labelling is impossible
  You can add an edge between any pair of vertices, including self loops.

In all cases, we can easily compute the number of edges in the final configuration. The total complexity of this task is $O(M)$.

Now we'll give a proof. In the proof we'll denote the edge $x \to y$ as (x, y).

When the labelling is possible, and not all of `A`, `B`, `C` are used If two edges (x, y) and (y, z) exist, all labels will be used and this is a contradiction. Thus, in this case there is no such pairs of edges, and you can't perform any operations. □

When the labelling is possible, and all of `A`, `B`, `C` are used In this case, there are some x, y, z such that both edges (x, y), (y, z) exist. We can add an edge (z, x), and these edges from a triangle. Let's assume that the labels of x, y, z are `A`, `B`, `C`, respectively. Consider a vertex v that is adjacent to one of x, y, or z. For example, when there is an edge (v, x), a triangle with v, x, y will be formed. Similarly, we can prove that v is always adjacent to two of x, y, z. Then, consider another vertex w that is adjacent to one of v, x, y, z, and prove that w is always adjacent to two of x, y, z. By repeating this process, we can prove that each vertex is adjacent to two of x, y, z, and by using this we can add all edges between (`A` $\to$ `B`), (`B` $\to$ `C`), and(`C` $\to$ `A`). On the other hand, it is clear that we can't add other edges. □

When the labelling is impossible Let's prove that at least one self-loop will be formed. If we prove this, the remaining part of the proof is almost equivalent to the former case.

First, find a cycle of the graph that will cause a contradiction in modulo 3. (This cycle is not necessarily a directed cycle: this is an undirected cycle in general.) This cycle must contain two edges (x, y) and (y, z), and thus we can make the size of the cycle smaller by adding an edge (z, x). This smaller cycle will also cause a contradiction in modulo 3. By repeating this process, we will get a self loop. $\square$

AtCoder Grand Contest 005 Editorial

Kohei Morita(yosupo)

October 1st, 2016

A: STring

まず、問題分には 10^{10000} 回という非常に大きな数が書いてありますが、実際には多くとも削除は $|X|/2$ 回しか行えないため、それ以上は意味がありません。

0.1 部分点

文字列から ST を探すのは、$O(|X|)$ で可能です。文字列検索はほぼ全ての言語に標準ライブラリとして入っているはずです。また、文字列の途中から ST を削除するのも $O(|X|)$ で可能です。以上より、一回の操作は $O(|X|)$ で行うことが出来ます。

そして、操作は $|X|/2$ 回しか意味がないため、それ以上は行わなくて良いです。よって、$O(|X|^2)$ でこの問題は解くことが出来ます。

0.2 満点

$O(|X|^2)$ では間に合わないため、更に高速化する必要があります。
実はスタックを使い、以下の操作を行うと $O(|X|)$ で解くことが出来ます。

- 文字列の文字を先頭から見ていき、
- 見ている文字が S ならば、スタックに push
- 見ている文字が T かつ、スタックの先頭の文字が T or 空ならばスタックに push
- 見ている文字が T かつ、スタックの先頭の文字が S ならばスタックを pop
- を文字列の最後まで繰り返す。

この操作によりスタックに残ったものが答えの文字列です。

イメージとしては、文字列を前からスキャンし、ST を発見したら適時削除していくイメージです。

B: Minimum Sum

この問題は、${}_N\mathrm{C}_2$ 個の区間それぞれについて最小値を求め、それを足しあわせたものを求める問題です。区間は $O(N^2)$ 個ありますが、$O(N^2)$ ではTLE するので、区間それぞれについて最小値を求めていては間に合いません。よって、視点を変える必要があります。

ここで、”最小値が a_i となる区間はいくつあるか？”をすべての i について求めることを考えます。

- a_i より左側で、a_i より小さいもののうち最も a_i に近いものを a_l
- a_i より右側で、a_i より小さいもののうち最も a_i に近いものを a_r

とすると、最小値が a_i となる区間は、左側が $l+1, l+2, ..., i$ のうちどれかで、右側が $i, i+1, i+2, ..., r-1$ となる区間です。よって、$(i-l+1)\times(r-i+1)$ 個の区間で最小値が a_i となります。これを足し合わせれば答えが得られるので、すべての i についてこの l, r を求めれば良いことがわかります。

$a_i = N, N-1, N-2, ...$ の順で l, r を求めていくことを考えます。すると、l, r は今まで見た要素のなかで、i 以下最大のものと i 以上最小のものであることがわかります。これは、平衡二分木 (例えば C++ならば std::set) に今まで見た要素の位置を入れていけば求めることが出来ます。

以上より、この問題は $O(NlogN)$ で解くことが出来ました。

C: Tree Restoring

はじめに、頂点 u, v について、$\mathrm{dist}(u, v)$ を 2 点間の距離とします。

仮に条件を満たす木が存在した場合、その木の直径 (木から 2 頂点 u, v を選んだ時の $\mathrm{dist}(u, v)$ の最大値) はいくつになるでしょうか。当然 $\max(a_1, a_2, ..., a_N)$ となります。$\mathrm{dist}(a, b)$ が木の直径となる時、パス $a - b$ 上の頂点について考えます。実は、$\mathrm{dist}(a, b)$ が木の直径であるとき、任意の p について

$$\max(\mathrm{dist}(p, 1), \mathrm{dist}(p, 2), ..., \mathrm{dist}(p, N)) = \max(\mathrm{dist}(p, a), \mathrm{dist}(p, b))$$

という性質があります。

この性質を使うと、直径を K とすると、パス $a - b$ 上の頂点の、他の頂点との距離の最大値は

- K が偶数の時、$K, K-1, K-2, ..., K/2+1, K/2, K/2+1, ..., K-1, K$
- K が奇数の時、$K, K-1, K-2, ..., (K+1)/2+1, (K+1)/2, (K+1)/2, (K+1)/2+1, ..., K-1, K$

となることがわかります。

よってまずはこのパスを構築します。もし構築できないならば Impossible です。そして、このパスのどこに頂点をつけても最大距離は偶数ならば $K/2+1$ 以上、奇数ならば $(K+1)/2+1$ 以上になります。なのでこれ未満が残っていたら、結果は Impossible です。これ以上のものしか無いとします。するとそれらを、それより最大距離が 1 小さいパス上の頂点につけます。すると条件を満たす木が構築できます。よって Possible です。

D: ～K Perm Counting

はじめに、この問題は "どれかの i について $|a_i - i| = K$ を満たすもの" を数えても良いので、こちらを数えます。更に条件を、$a_i - i = K$ と $a_i + i = K$ という $2N$ 個の条件に分解しておきます。すると数えるべきは、この $2N$ 個の条件のうちどれか 1 つは満たす順列の個数です。

実は、長さ N の順列と、頂点数 $2 \times N$ の二部完全グラフのマッチングには全単射が存在するので、二部グラフの完全マッチングで考えます。全単射は、二部完全グラフを $L_1, L_2, ..., L_N, R_1, R_2, ..., R_N$ として、$a_i = j$ と辺 (L_i, R_j) を対応させると作れます。

するとこの問題で数えるべきは、辺 (L_i, R_{K+i})、辺 (L_i, R_{K-i}) のうちどれか 1 つは使う完全マッチングの個数、と考えることができます。ここで包除原理を使用します。すると、上記の辺のうちいくつかを選び、それらを全て含む完全マッチングの個数を数える問題になります。

ここで注目したいのは、辺を A 本選んだとすると、条件を満たす完全マッチングの個数は、0 か $(N - A)!$ であるということです。もし選んだ辺たちがマッチングではないならば当然 0 個で、マッチングならば残りの頂点の割り当て方が $(N - A)!$ 通りあります。

これにより、$2N$ 個について包除しているため本来 2^{2N} 通りについて調べる必要があるのが、選んだ個数ごとにまとめることができます。つまり、$2N$ 個の辺からマッチングとなるように A 個選ぶ方法は何通りか？という問題を $A = 1, 2, ..., 2N$ について解けば良いことがわかります。

辺たちの形が非常に単純で、直線型のグラフが何本か合わさった形になっているので、これは簡単な DP で $O(N^2)$ で求めることができます。

以上よりこの問題は $O(N^2)$ で解けました。

おまけ: 以上の考察をもう少し進めると、この問題は $O(NlogN)$ で解くことが出来ます。

E: Sugigma: The Showdown

この問題はしぐま君とすぎむ君がそれぞれ木を持っていて、その上でしぐま君は逃げて、すぎむ君は追う問題です。

まず、しぐま君の辺 (u, v) について、$u - v$ 間のすぎむ君の辺での距離が 3 以上ならば、その辺をつかうとしぐま君は永遠に逃げ続けられます。その辺の端点で待機し、すぎむ君が近づいてきたら辺を使い移動する、を繰り返えすだけです。よって、そのような辺が生えている頂点はしぐま君にとっての必勝頂点 (その頂点でしぐま君が手番をとったら永遠に逃げ続けられる) としてよいです。

以上の考察より、しぐま君は、距離 1,2 の辺 + いくつかの頂点が必勝頂点を持っていると考えられます。

また、すぎむ君は、常にしぐま君に近づくように動くのが最善戦略です。なぜならば、もしすぎむ君が離れたら、しぐま君はすぎむ君が戻ってくるまでパスをし続けることでターン数を稼ぐことができるからです。よって、実はすぎむ君のするべき行動は一意に定まり、あとはしぐま君の行動について考えればよいです。

すぎむ君の木を、すぎむ君が最初にいる頂点を根とした根付き木に変形し、その上で考えると見通しが良くなります。すると、「根付き木が与えられ、しぐま君はどこかにいて長さ 1 や 2 のジャンプが出来て、すぎむ君は最初は根にいて長さ 1 のジャンプが出来る。また、いくつかの頂点でしぐま君が手番を取るとしぐま君の勝ち」という問題になります。長さ 1 や 2 のジャンプしか出来ない時に重要な性質として、「しぐま君がすぎむ君を飛び越すと、その次のターンでしぐま君は捕まる」という性質があります。もし飛び越えても、すぎむ君の隣にしか行けないからです。そして、飛び越えてもターン数が伸びるわけではないので、飛び越える利点はないです。よって、「しぐま君はすぎむ君を飛び越えることはない」としてよいです。

よってしぐま君は常にすぎむ君の下にいて、すぎむ君は毎ターンしぐま君の方に降りていく、という行動をすることになります。これは、「しぐま君は i 回目の手番では深さ i+1 以上のところのみを動ける」、という解釈にすることが出来ます。

このように解釈すると、ゲームからすぎむ君を消して、しぐま君の一人ゲームとして考えることが出来ます。しぐま君が、毎ターン strict に範囲が狭まっていく木の中で、一人で逃げ続けるゲームです。すると、

- しぐま君は、後戻りをして得をすることはない
- しぐま君は、一度パスをしたならばその後ずっとパスをし続けるのが最善戦略

ということが簡単に示せます。

なので、しぐま君の動きを、しぐま君の木において dfs をすることで全探索し、この問題は、$O(N)$ で解くことが出来ます。

F: Many Easy Problems

まず、K が固定されている場合から考えます。もちろん、${}_N\mathrm{C}_K$ 通りそれぞれについて調べていてはとても間に合わないので、視点を変える必要があります。

辺ごとに、その辺が部分木に使われるような選び方は何通りか？というのを求め、それらの総和を取ることを考えます。辺の左右の部分木からそれぞれ 1 個以上頂点を選んだ場合、その辺は部分木に含まれることがわかります。よって、辺がサイズ $A, N-A$ の木をつないでいるとすると、この辺を使う選び方は ${}_N\mathrm{C}_K - {}_A\mathrm{C}_K - {}_{N-A}\mathrm{C}_K$ 通りです。

なので、まず $A, N-A$ を最初に列挙しておきます。そしてそれらを並べ、数列 a_i としておくと、

$$(N-1){}_N\mathrm{C}_K - \sum_{i=1}^{2(N-1)} {}_{a_i}\mathrm{C}_K$$

が答えとなります。

これは階乗を前計算しておけば $O(1)$ で求めることができるので、K が固定されている場合は $O(N)$ で解くことが出来ました。

満点解法を考えます。まず、数列 b_i を、$b_i = (a_1, a_2, ...$ のうちの i の個数 $)$ とすると、

$$(N-1){}_N\mathrm{C}_K - \sum_{i=1}^{N-1} b_i \times {}_i\mathrm{C}_K$$

を求める問題になります。

ここで、${}_N\mathrm{C}_K = \frac{N!}{K!(N-K)!}$ を使うと、

$$\sum_{i=1}^{N-1} b_i \times {}_i\mathrm{C}_K$$

$$= \sum_{i=1}^{N-1} b_i \frac{i!}{K!(i-K)!}$$

$$= \frac{1}{K!} \sum_{i=1}^{N-1} (b_i \times i!) \frac{1}{(i-K)!}$$

と変形できます。よって、$c_i = b_i \times i!$、$d_i = 1/i!$ とおくと、$c_i \times d_{i-K}$ の総和を求めれば良いことがわかります。これは NFT を用いることで $K = 1, 2, ..., N$ について、まとめて $O(NlogN)$ で求めることが出来ます。

今回は mod が $10^9 + 7$ では無く 924844033 です。1 を引いて素因数分解をしてみると、$924844033 = 2^{21} * 441 + 1$ であることがわかります。これが今回 mod を変更した理由で、$Z/[924844033]$ において、2^{21} 乗すると初めて 1 になる元が存在します。なので、他に特殊な mod を用意しなくても、直接

NFT を行うことが出来ます。(Time Limit は他の mod を経由して NFT をしても十分に間に合うように調整しました。) 余談ですが、原始元は 5 なので、$5^{441} = 44009197$ が 2^{21} 乗すると初めて 1 になる元です。

以上より、この問題は $O(NlogN)$ で解けました。

AtCoder Grand Contest 005 Editorial

Kohei Morita(yosupo)

October 1st, 2016

A: STring

Instead of 10^{10000} operations, $|X|/2$ operations are sufficient.

0.1 200 points

You can find the leftmost occurrence of the string "ST" in the given string in $O(|X|)$. Also, you can remove this substring from the string in $O(|X|)$. Since there are at most $|X|/2$ operations, you can solve this problem in $O(|X|^2)$ by simulation.

0.2 300 points

You can get an intuitive understanding of this task by replacing 'S' to '(' and 'T' to ')'. In each step, you remove a pair of matching parenthesis.

Prepare an empty stack. Scan the string from the first character the last character, and do the following:

- If the current character is 'S' (or '('), push it to the stack
- If the current character is 'T' (or ')'), and the topmost character in the stack is 'T' (')') or the stack is empty, push it to the stack
- If the current character is 'T' (or ')'), and the topmost character in the stack is 'S' ('('), pop the topmost character from the stack

The answer is the size of the stack after this process. This way you can solve the task in $O(|X|)$ time.

B: Minimum Sum

In this task, for each interval you compute the minimum, and the answer is the sum of these minimums. There are $O(N^2)$ intervals and straightforward solutions will get TLE.

Instead, for each i, compute the number of intervals whose minimum is a_i. Let $f(i)$ be this number. Then, the answer is the sum of $a_i f(i)$.

For each index i, we compute l and r:

- l is the biggest integer such that $l < i$ and $a_l < a_i$
- r is the smallest integer such that $r > i$ and $a_r < a_i$

Then, in order for a_i to be minimum, the leftmost value in the interval must be one of $l+1, l+2, \ldots, i$, and the rightmost value must be one of $i, i+1, i+2, \ldots, r-1$. Thus, there are $(i-l) \times (r-i)$ such intervals.

We can compute the values of l and r in the increasing order of a_i. We keep a set of indices (for example std::set in C++), and in the decreasing order of a_i, add i to the set. The values of l, r are the two integers adjacent to i in this set.

Thus, this problem can be solved in $O(NlogN)$.

C: Tree Restoring

Let $\text{dist}(u, v)$ be the distance between two vertices u and v.

The diameter of the tree must be $\max(a_1, a_2, ..., a_N)$. Let a, b be the two endpoints of a diameter.

In general, we can prove that for any vertex p, the following holds:

$$\max(\text{dist}(p, 1), \text{dist}(p, 2), ..., \text{dist}(p, N)) = \max(\text{dist}(p, a), \text{dist}(p, b)) \quad (1)$$

Let K be the length of the diameter. There are $K + 1$ vertices on the path between a and b, and by using the property above, the values that correspond to these vertices are:

- When K is even, the values are
  $\{K, K-1, K-2, ..., K/2+1, K/2, K/2+1, ..., K-1, K\}$
- When K is odd, the values are
  $\{K, K-1, K-2, ..., (K+1)/2+1, (K+1)/2,$
  $(K+1)/2, (K+1)/2+1, ..., K-1, K\}$

First, from the given multiset of values, remove values that correspond to these values. If this is impossible, the answer is clearly "Impossible".

Consider a vertex x that is not on this path. We can see that when K is even, the value corresponds to x must be at least $K/2 + 1$, and when K is odd, the value must be at least $(K + 1)/2 + 1$. On the other hand, by choosing an appropriate vertex on the path and add a vertex adjacent to this vertex, we can construct all other cases.

Thus, if all the remaining values are at least this bound, the answer is "Possible". Otherwise the answer is "Impossible".

D: K Perm Counting

Consider a bipartite graph with $2N$ vertices. The vertices are labelled $L_1, \ldots, L_N, R_1, \ldots, R_N$. A perfect matching in this bipartite graph corresponds to a permutation. A permutation $p_1, \ldots, p_N$ corresponds to a matching that uses the edges (L_i, R_{p_i}). In this task, you are asked to count the number of perfect matchings that doesn't use edges (L_i, R_{i+K}) and (L_i, R_{i-K}).

By inclusion-exclusion principle, we get the following. Consider a bipartite graph that consists of bad edges: (L_i, R_{i+K}) and (L_i, R_{i-K}) for each i. Let M_k be the number of matchings of size k in this graph. Then, the answer is

$$\sum_{i=0}^{N} M_i (N-i)!(-1)^i \tag{2}$$

Thus, it is sufficient to compute the value of M_k for each k.

Since this graph is a disjoint union of paths, the values of M_k can be easily computed in $O(N^2)$ time in DP. Therefore, this problem can be solved in $O(N^2)$.

Exercise: can you solve this problem in $O(NlogN)$?

E: Sugigma: The Showdown

In this task, each of Sigma and Sugim has a tree (on the same set of vertices). Sigma is the first player. He starts from the vertex X and tries to escape from Sugim using the red tree. Sugim is the second player. He starts from the vertex Y and tries to capture Sigma using the blue tree.

In this editorial, let $D_r(u, v)$ be the distance between the vertices u and v along the red tree. Define $D_b(u, v)$ similarly for the blue tree.

Suppose that there is an edge $u - v$ in the red tree, and $D_b(u, v) \geq 3$. Let's call this type of edges *long*. If Sigma reaches one of the vertices u or v and Sugim can't capture him right after that, we can prove that Sigma can escape forever. When Sigma is at the vertex u, he waits at this vertex until Sugim reaches one of the vertices adjacent to u (in the blue tree). Then, right after Sugim reaches one of the adjacent vertices (let's call it x), he moves to the vertex v. Since $D_b(u, v) \leq 3$ and $D_b(u, x) = 1$, $D_b(v, x) \leq 2$ and he can't capture Sigma in the next turn. Similarly, when Sigma is at the vertex v, he wais there until Sugim reaches one of the adjacent vertices, and then return to u again. This way Sigma can escape from Sugim forever.

Consider the blue tree as a rooted tree rooted at Y. Some vertices of this rooted tree are marked as "special". These vertices correspond to the endpoints of the long edges, and if Sigma reaches one of thesse vertices (and Sugim can't capture him right after that), Sigma wins. Also, there are some additional red edges. When a red edge connects two vertices u and v, $D_b(u, v)$ is 1 or 2.

We can consider these red edges as "jumps". The game becomes a game on a rooted tree. Sigma jumps around the tree (the jumps are always short), and tries to reach one of the special vertices or tries to escape from Sugim as long as Sugim.

An important observation is that, we can assume that Sigma never jumps over Sugim. When Sigma jumps over Sugim, since the jump length is at most two, both endpoints of the jump must be adjacent to Sugim, and Sigma will be captured right after the jump. It is not better than staying at the vertex he jumped from. In other words, Sigma is always in the subtree rooted at Sugim.

We can assume that Sugim always move downwards to Sigma's direction. If Sugim follows this strategy, whenever Sigma reaches vertex x after t steps such that $D_b(Y, x) \leq t$, he can capture Sigma. On the other hand, as long as Sigma keeps the condition $D_b(Y, x) > t$, obviously Sugim can't capture Sigma. Thus, this is the optimal strategy for Sugim.

Now this game can be regarded as a single-player game for Sugim. In

Sigma's path, if the i-th (0-based) vertex is v_i, it must satisfy $D_b(v_i, Y) > i$. Under this constraints, if he can reach one of the special vertices, he wins (and the answer is -1). Otherwise, the answer corresponds to the maximum length of such paths. This can be done by a simple dfs.

Therefore, this problem can be solved in $O(N)$ time.

F: Many Easy Problems

First, solve the problem for a fixed value of K.

For each edge e, count the number of ways to choose K vertices such that the edge e is included in the subtree. If we compute the sum of these values for all edges, we can compute the sum of number of edges in all subtrees defined by K vertices. Since the number of vertices in a tree is the number of edges plus one, the answer is this sum plus the total number of ways to choose K vertices from the N vertices.

Assume that if we cut the given tree by the edge e, we get two subtrees of the sizes A and $N-A$. Then, the number of ways to choose K vertices that include this edge is $\binom{N}{K} - \binom{A}{K} - \binom{N-A}{K}$. Thus, for each we can compute this value, and the answer is the sum of these values plus $\binom{N}{K}$. This can be done in $O(N)$.

We can simplify this solution a bit. First, compute the frequency list of the sizes of subtrees obtained by cutting a single edge. We can convert it to a sequence of coefficients $b_0, b_1, \ldots, b_N$ ($b_i = N$ if $i = N$, otherwise b_i is minus the number of subtrees of size i), and the answer is simply the following:

$$\sum b_i \binom{i}{K} \tag{3}$$

Let's compute this value for all K efficiently.

By using $\binom{i}{K} = \frac{i!}{K!(i-K)!}$,

$$\sum b_i \binom{i}{K} \tag{4}$$

$$= \sum b_i \frac{i!}{K!(i-K)!} \tag{5}$$

$$= \frac{1}{K!} \sum (b_i \times i!) \frac{1}{(i-K)!} \tag{6}$$

Let $c_i = b_i \times i!$ and $d_i = 1/(-i)!$. Then, this value can be computed by the sum of $c_i \times d_{K-i}$. This can be seen as a convolution, thus we can compute these values for all K using FFT in $O(NlogN)$.

Since the modulo is 924844033, we can use NFT instead of FFT to avoid precision errors. The modulo is $924844033 = 2^{21} * 441 + 1$, and thus there exists a g such that $g^{2^{21}} = 1$ and any smaller powers of g is not one. For example, if we use a primitve root 5, we can get $g = 5^{441} = 44009197$. We can use this g instead of a root of unity in FFT.

In summary, this problem can be solved in $O(NlogN)$ time.

AtCoder Grand Contest 004 解説

writer : sugim48

A : Divide a Cuboid

$A \times B \times C$ 個のブロックからなる直方体を 2 つの直方体へ分割し，2 つの直方体に含まれるブロックの個数の差を最小化する問題です．分割の方法をすべて試すと TLE してしまうので，より速い方法を考えなければなりません．

まず，A，B，C のどれかが偶数の場合を考えます．例えば，A が偶数であるとして，$A = 2a$ とおきます．この場合，$a \times B \times C$ の直方体 2 つへ分割すると，ブロックの個数の差は 0 になります．よって，答えは 0 です．

次に，A，B，C がすべて奇数の場合を考えます．例えば，A の方向に分割することにして，$A = 2a + 1$ とおきます．この場合，$a \times B \times C$ の直方体と $(a + 1) \times B \times C$ の直方体へ分割すると，ブロックの個数の差は $B \times C$ で最小になります．同様に，B の方向に分割する場合の最小値は $C \times A$ であり，C の方向に分割する場合の最小値は $A \times B$ です．よって，答えは $\min\{B \times C, C \times A, A \times B\}$ です．

B : Colorful Slimes

魔法を唱える回数を k に固定してみます．すると，最終的に色 i のスライムが欲しい場合，色 i，$i-1$，...，$i-k$ のスライムのどれかを適切なタイミングで捕まえればよいです．ただし，色 1 のひとつ前は色 N であるとします．例えば，$K = 2$ で最終的に色 3 のスライムが欲しい場合，次の 3 通りの方法があります．

- 魔法を唱える → 魔法を唱える → スライム 3 を捕まえる
- 魔法を唱える → スライム 2 を捕まえる → 魔法を唱える
- スライム 1 を捕まえる → 魔法を唱える → 魔法を唱える

どのタイミングでスライムを捕まえるかは自由なので，最も a_i が小さいタイミングで捕まえればよいです．よって，最終的に色 i のスライムを手に入れるためには，$\min\{a_i, a_{i-1}, \ldots, a_{i-k}\}$ 秒が掛かります．これを $b_i(k)$ 秒とおくと，全色のスライムを手に入れるためには，合計で $k \times x + \sum_i b_i(k)$ 秒が掛かります．

以上の考察より，k は 0 から $N-1$ まで試せば十分であることが分かります．k を 0 から $N-1$ まで全探索し，$k \times x + \sum_i b_i(k)$ の最小値を求めれば，それが答えです．しかし，各 $b_i(k)$ を計算するのに $\mathrm{O}(N)$ 時間掛けてしまうと，全体で $\mathrm{O}(N^3)$ 時間となり TLE してしまいます．ここで，$b_i(k) = \min\{b_i(k-1), a_{i-k}\}$ であることを用いると，各 $b_i(k)$ を $\mathrm{O}(1)$ 時間で計算できます．すると，全体で $\mathrm{O}(N^2)$ 時間となり，間に合います．

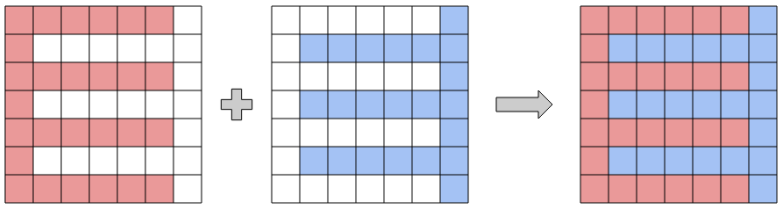
C : AND Grid

与えられた紫色のマスの配置に対して，臨機応変に赤いマス目と青いマス目のペアを構成するのは大変です．ここでは，どのような紫色のマスの配置に対しても，機械的に赤いマス目と青いマス目のペアを構成できる方法を考えてみます．

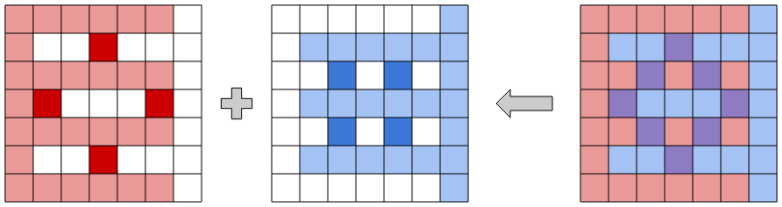
次のような条件を満たす赤いマス目と青いマス目のペアを準備しておくと，嬉しいことが分かります．

- 赤いマス目と青いマス目を重ねると，各マスは赤または青のどちらかである．
- 赤いマスも青いマスも上下左右に連結である．
- 赤いマス目において，最も外側のマス以外のマスをひとつ選んで赤く塗っても，赤いマスは上下左右に連結のままである．青いマス目においても同様．

例えば，次図のようなマス目のペアが条件を満たします．

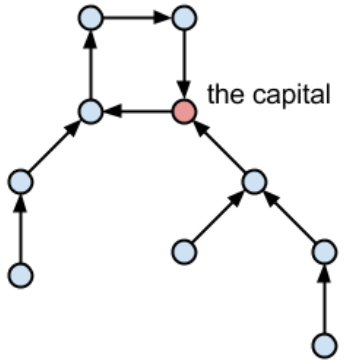


このようなマス目のペアを準備しておくと，どのような紫色のマスの配置に対しても，紫色の各マスに対応するマスを赤または青で塗ることで，機械的に赤いマス目と青いマス目のペアを構成できます（次図）．

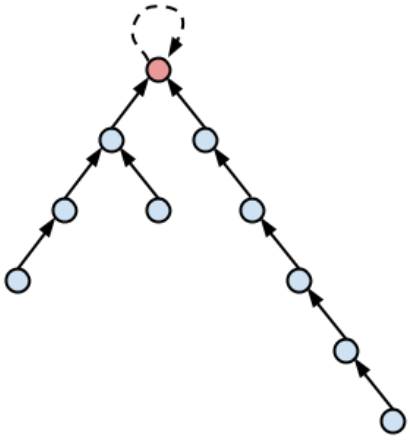


D : Teleporter

各 $1 \leq i \leq N$ について，頂点 i から頂点 a_i へ有向辺を張ったグラフを考えます（次図）.「どの町から出発しても，テレポーターを何回か使うことで首都へ辿り着ける」という制約より，グラフは連結です．グラフの辺の行き先をいくつか変えて，「どの頂点から出発しても，辺をちょうど K 回辿ると，最終的に首都にいる」という条件が成り立つようにするのが目標です．

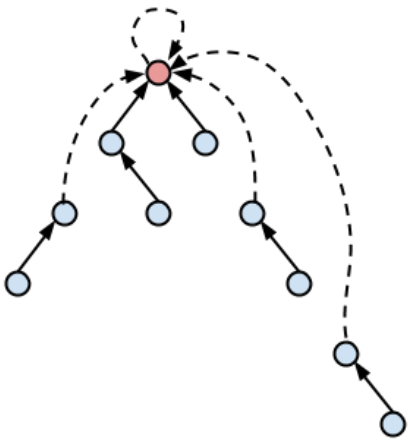


まず，首都から出る辺は首都へ戻る自己ループでなければならないことを示します．条件が成り立つようにグラフを変形できたとき，仮に $a_1 \neq 1$ であったとします．条件が成り立っているので，首都から辺をちょうど K 回辿ると，最終的に首都にいます．そのためには，頂点 a_1 から辺をちょうど $K-1$ 回辿ると，最終的に首都にいなければなりません．よって，頂点 a_1 から辺をちょうど K 回辿ると，最終的に頂点 a_1 $(\neq 1)$ にいます．これは条件に矛盾します．以上より，首都から出る辺は首都へ戻る自己ループでなければなりません．首都から出る辺を首都へ戻る自己ループに変えると，グラフは首都を根とする根付き木のようになります（次図）.



首都に自己ループがあるので，条件は「どの頂点から出発しても，辺を K 回以下辿れば，首都

へ辿り着ける」と言い換えられます．また，辺の行き先を変えるときは，できるだけ早く首都へ辿り着きたいので，辺の行き先を首都へ変えるのが最善であることが分かります．以上の考察に従うと，例えば $K = 2$ の場合，次図のように辺の行き先を変えればよいです．



ここまで来ると，問題は次のように言い換えられます．

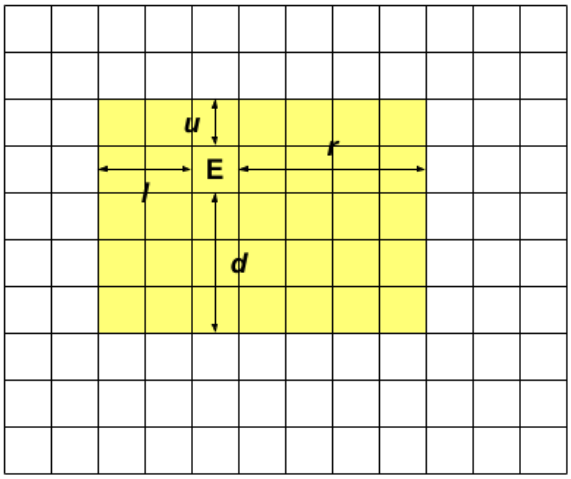
> 首都を根とした根付き木から，できるだけ少ない本数の辺を取り除き，分断されたそれぞれの木が次の条件のどちらかを満たすようにせよ．
>
> - 根が首都であり，高さが K 以下である．
> - 根が首都ではなく，高さが $K - 1$ 以下である．

この問題は次のようにして解くことができます．各頂点の高さを計算するのと同じ要領で，根付き木を深さ優先探索します．ただし，頂点 v を見ているとき，高さが $K - 1$ に達しており，かつ，v の親が首都でなければ，頂点 v を親から分断します．このとき，親へ伝える高さを 0 へリセットします．深さ優先探索が終わった後，分断した回数が答えです．時間計算量は $\mathrm{O}(N)$ です．

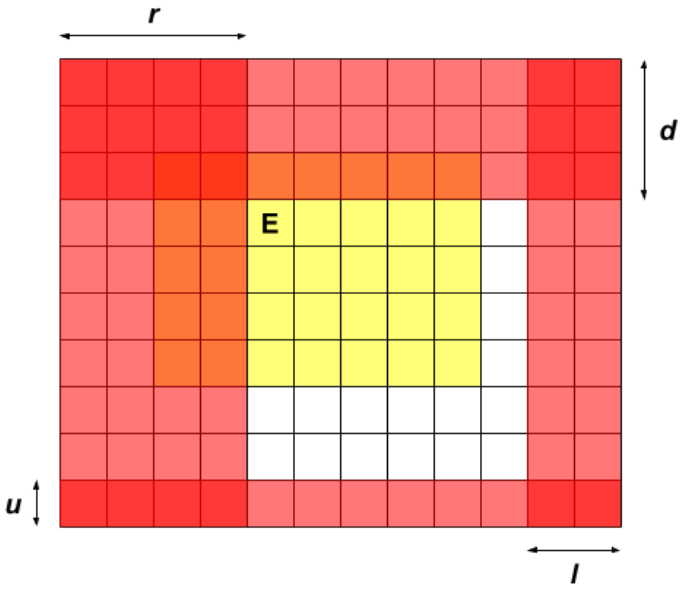
E : Salvage Robots

ロボットを動かす代わりに，壁と出口を動かすと考えます．例えば，すべてのロボットを左向きに 1 マスだけ動かすというのは，壁と出口を右向きに 1 マスだけ動かすのと同じです．すると，問題設定は「壁と出口の動かし方を工夫し，壁が触れる前に出口が触れるロボットの個数を最大化せよ」となります．

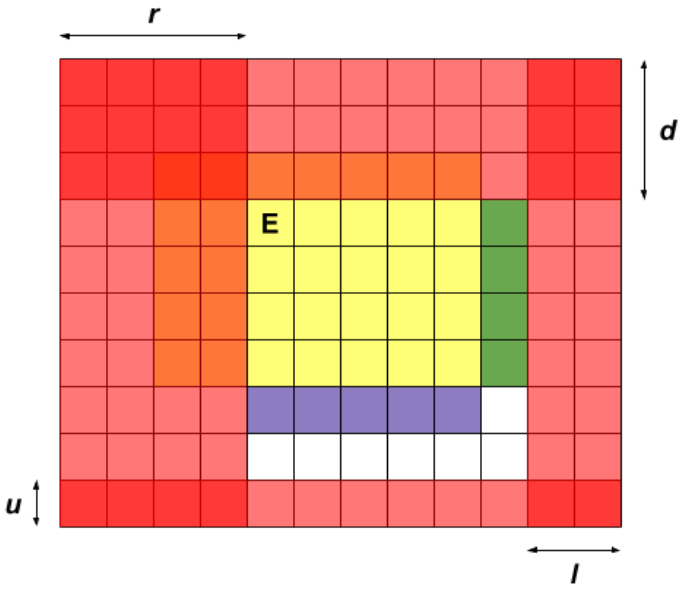
今，出口が初期位置から左向きへ最大 l マス，上向きへ最大 u マス，右向きへ最大 r マス，下向きへ最大 d マス離れたことがあるとします（次図）．このとき，出口はできるだけ多くのロボットを救出するように工夫して動いてきたと仮定します．



出口が動くのと同時に壁も動くので，次図の赤い長方形内にロボットは生き残っていません．（ただし，壁に触れて爆発する前に出口が触れて救出された可能性はあります．）また，黄色の長方形内にもロボットは生き残っていないことが分かります．というのも，出口が黄色の長方形内で動く分には l，u，r，d は新たに大きくならないので，赤い長方形の範囲も大きくなりません．よって，黄色の長方形内にロボットが生き残っていれば，必ず救出しに行けます．出口はできるだけ多くのロボットを救出するように工夫して動いてきたと仮定したので，黄色の長方形内にはロボットは生き残っていないはずです．以上より，色のないマスのロボットはすべて生き残っており，色のあるマスのロボットはすべて消えていると確定できます．



さらに出口が動いていくと，いずれ l，u，r，d のどれかが $+1$ されます．例えば，r が $+1$ されると，次図の緑の長方形内のロボットすべてが新たに救出できます．また，d が $+1$ されると，次図の紫の長方形内のロボットすべてが新たに救出できます．l または u が $+1$ されると，新たに救出できるロボットはありません．

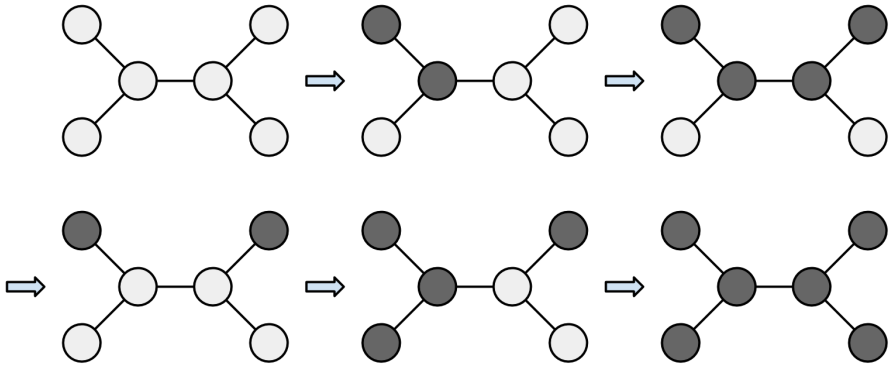


$dp[l][u][r][d]$ を，「出口が初期位置から左向きへ最大 l マス，上向きへ最大 u マス，右向きへ最大 r マス，下向きへ最大 d マス離れたことがあるとき，今までに救出したロボットの個数の最大値」と定義します．すると，以上の考察により，$dp[l][u][r][d]$ の値を用いて，$dp[l+1][u][r][d]$，$dp[l][u+1][r][d]$，$dp[l][u][r+1][d]$，$dp[l][u][r][d+1]$ の各値を更新していくことができます．この DP テーブルを埋めた後，最も大きい黄色の長方形に対応する dp の値が答えです．時間計算量は $\mathrm{O}(H^2W^2)$ です．

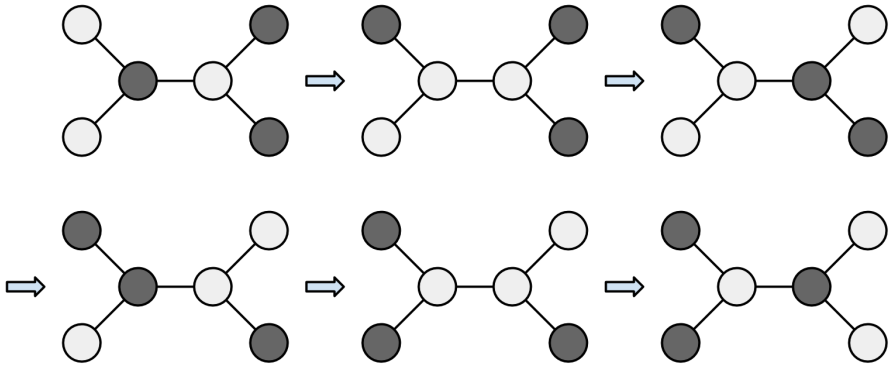
F : Namori

まずは，$M = N - 1$ の場合，すなわちグラフが木の場合を考えます．今，可能な操作は 白-白 $\leftrightarrow$ 黒-黒 ですが，このままでは考察が難しいです．そのため，次のように見方を変えることにします．

まず，木の頂点たちを白黒交互に塗り分けたものを用意し，これを市松模様と呼ぶことにします．以降，木の各頂点の色を見るときには，この市松模様との XOR を取ったものを代わりに見ることにします．すると，白-白 $\leftrightarrow$ 黒-黒 の操作だったものは，白-黒 $\leftrightarrow$ 黒-白 の操作に見えることになります．サンプル 1 を例として，見え方がどう変わるかを次図に示します．



市松模様との XOR を取る前



市松模様との XOR を取った後

白い頂点を空きマス，黒い頂点を駒のあるマスとして捉え直すと，問題は次のように言い換えられます．

> 木の頂点のうちいくつかに駒が乗っている．駒は隣り合う頂点へ動かすことができるが，駒同士を重ねることはできない．駒の初期配置と最終配置が与えられるので，初期配置から最終配置へ変えられるか判定せよ．変えられるならば，必要な操作回数の最小値を求めよ．

かなり見通しが良くなりました．

言い換え後の問題は，次の最小費用流問題として定式化できます（証明は後述）．まず，初期配置において駒がある各頂点に，流出量 1 の source をそれぞれ設置します．また，初期配置において駒がある各頂点に，流入量 1 の sink をそれぞれ設置します．さらに，各辺を容量 ∞，コスト 1 に設定します．とりあえず，source の個数と sink の個数が異なる場合，答えは当然 Impossible です．source の個数と sink の個数が等しい場合，実際にフローを流して得られる最小コストが答えになります．

しかし，この問題では $N \leq 10^5$ とグラフが大きいため，実際にフローを流すことはできません．そこで，最小コストを次のようにして計算します．ある辺 $e = (u, v)$ で木を二分割したとします．

頂点 u 側の部分木を見たとき，この部分木内の (source の個数) − (sink の個数) を f_e とおきます．すると，頂点 u から v への流量は f_e に一致し，辺 e でのコストは $|f_e|$ となります．よって，最小コストは $\sum_e |f_e|$ として計算できます．この時間計算量は $\mathrm{O}(N)$ です．以上で，グラフが木の場合を解くことができました．

次に，$M = N$ の場合を考えます．この場合，グラフにはちょうど 1 つだけ閉路が含まれます．この閉路が偶閉路か奇閉路かで場合分けをします．

まずは，偶閉路の場合を考えます．偶閉路の場合は，木の場合と同様にして，グラフを白黒交互に塗り分けられるので，市松模様との XOR を取ることができます．よって，駒を動かす問題設定へ言い換えられ，さらに最小費用流問題として定式化することができます．しかし，この最小コストは木の場合と同じ方法では計算できません．

最小コストは次のようにして計算できます．まず，偶閉路に含まれる辺を適当にひとつ選びます．この辺を $e_0 = (u, v)$ とします．頂点 u から v への流量 x を決めた後，辺 e_0 をグラフから取り去ることを考えます．すると，残ったグラフは木になるので，木の場合と同様にして最小コストを求めることができます．こうして求まるグラフ全体のコストの総和を $F(x)$ とおくと，実は $F(x)$ は x に関して下に凸であることが示せます．よって，x について三分探索を行うことで，$\min F(x)$ を $\mathrm{O}(N \log N)$ 時間で求めることができます．

$F(x)$ は x に関して下に凸であることを示します．x を決めたとき，辺 e でのコストを $f_e(x)$ と書くことにします．まず，辺 e が閉路に含まれない場合，$f_e(x)$ は定数関数なので無視します．辺 e が閉路に含まれる場合，$f_e(x) = |x - a_e|$ (a_e は整数定数) という形になっています．これは x に関して下に凸の関数です．よって，$f_e(x)$ の総和である $F(x)$ も x に関して下に凸です．

次に，奇閉路の場合を考えます．奇閉路の場合は，そもそもグラフを白黒交互に塗り分けられません．しかし，できるだけ白黒交互になるように塗り分けたとします．このとき，同じ色が隣り合うような頂点のペアがちょうど 1 つ存在します．これを $e_0 = (u, v)$ とおきます．すると，辺 e_0 以外の辺での操作は 白-黒 $\leftrightarrow$ 黒-白 ですが，辺 e_0 での操作だけは 白-白 $\leftrightarrow$ 黒-黒 となります．言い換えると，頂点 u，v に同時に駒を置くか，同時に駒を取り去ることができます．

辺 e_0 での例外的な操作により，駒を増やしたり減らしたりすることができるようになりました．そのため，初期配置と終了配置で駒の個数が異なっても，初期配置から最終配置へ変えられる可能性があります．ただし，駒の増減は 2 個刻みなので，初期配置と終了配置で駒の個数の偶奇が異なる場合，答えは Impossible です．初期配置と終了配置で駒の個数の偶奇が一致する場合，適切に駒を増減することで駒の個数を揃えられます．また，駒を増やしたり減らしたりする操作の回数は一意に決まります．駒を増やす操作を k 回行う場合，頂点 u，v に source を k 個ずつ設置すればよいです．駒を減らす操作を k 回行う場合，頂点 u，v に sink を k 個ずつ設置すればよいです．その後，辺 e_0 をグラフから取り去ると，残ったグラフは木になるので，木の場合と同様にして最

小コストを求めることができます．この最小コストに $|k|$ を足せば，答えが求まります．時間計算量は $\mathrm{O}(N)$ です．

TODO：最小費用流問題として定式化できることの証明

AtCoder Grand Contest 004 Editorial

writer : sugim48

A : Divide a Cuboid

In this editorial "box" means "rectangular parallelepiped".

In this task, you are asked to divide an $A \times B \times C$ box into two boxes and minimize the difference between the number of blocks in the two boxes.

If at least one of A,B,C is even, for example, when $A = 2a$, you can divide the box into two boxes of size $a \times B \times C$, thus the answer is 0.

Suppose that all of A,B,C are odd. If $A = 2a+1$ and you want to divide the box by a plane orthogonal to this direction, it is optimal to divide the box into two boxes of sizes $a \times B \times C$ and $(a+1) \times B \times C$, and the difference is $B \times C$. Similarly, you can get the differences of $C \times A$ and $A \times B$ using other directions. Therefore, when all of all of A,B,C are odd, the answer is $\min\{B \times C, C \times A, A \times B\}$.

B : Colorful Slimes

Suppose that Snuke casts the spell k times in total. In order to get a slime of color i, he needs to capture one of slimes of colors i,$i-1$,...,$i-k$ (indices are modulo N). For example, if $K = 2$ and you need a slime of color 3, there are 3 ways to get it:

- Cast a spell → Cast a spell → Capture a slime of color 3
- Cast a spell → Capture a slime of color 2 → Cast a spell
- Capture a slime of color 1 → Cast a spell → Cast a spell

Thus, in order to get a slime of color i, he needs $\min\{a_i, a_{i-1}, \ldots, a_{i-k}\}$ seconds (except for the time spent for casting spells). Let $b_i(k)$ be this value. Then, he needs $k \times x + \sum_i b_i(k)$ seconds to get all types of slimes.

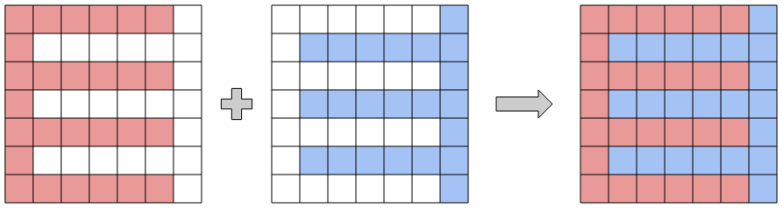
If you try all possible values of k from 0 to $N-1$ and compute the minimum of $k \times x + \sum_i b_i(k)$, you can get the answer. The straightforward implementation of this algorithm is $O(N^3)$. If you use the fact that $b_i(k) = \min\{b_i(k-1), a_{i-k}\}$, you can compute $b_i(k)$ in $O(1)$ for each pair (i, k), and thus the solution works in $O(N^2)$.

C : AND Grid

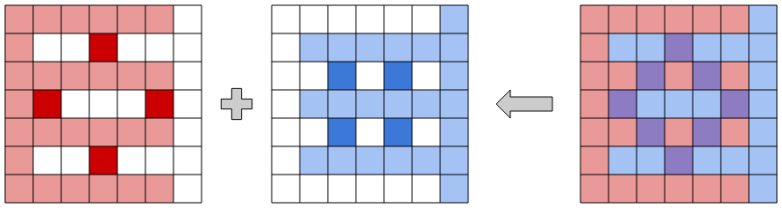
First, we construct a pair of a red grid and a blue grid with the following properties:

- If you overlaid the red grid on the blue grid, each cell becomes either red or blue. In other words, blue cells are the complement of the red cells.
- Red cells are 4-connected. Blue cells are also 4-connected.
- In the red grid, if you choose an arbitrary subset of white cells and paint them red, the grid remains 4-connected. The same holds for blue grid.

For example, the following pair satisfies the conditions.

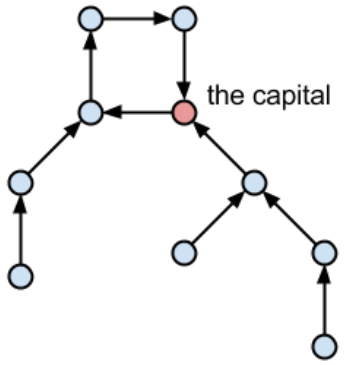


By using this pair, you can construct a solution for arbitrary set of purple cells as in the following diagram:



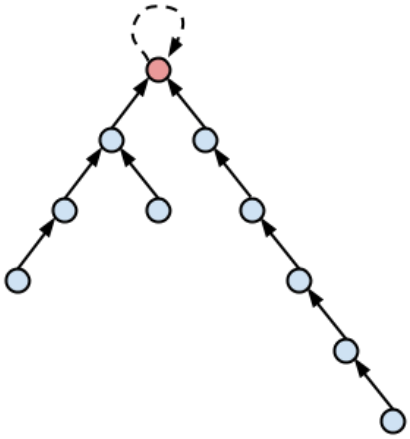
D : Teleporter

Consider a graph with N vertices. For each $1 \leq i \leq N$, there is a directed edge from the vertex i to the vertex a_i. By the condition "one can get to the capital from any town by using the Teleporters some number of times.", (if we ignore the orientations) the graph is connected. In this task, you are asked to change the heads of the smallest possible number of edges and satisfy the condition "Starting from any town, one will be at the capital after using the Teleporters exactly K times in total."



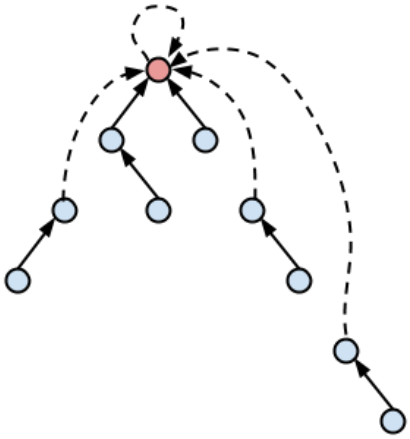
First, we prove that after we change the graph, the edge from the vertex 1 should go to the vertex 1 itself. When you follow the edges k times from the vertex 1, you should be at the vertex 1 (from the condition in the statement). Thus, there must be a cycle containing the vertex 1, and the length of this cycle must be a divisor of K. Suppose that there is an edge $1 \to r$. Since r is in the cycle mentioned above, if you follow the edges k times from the vertex r, you will reach the vertex r. From the statement, you must be at 1m thus we proved that $1 = r$.

If we change the tail of the edge from vertex 1 to 1, the graph will be a tree rooted at the vertex 1:



In this rooted tree, the condition can be restated as "Starting from any town, one will be at the capital after using the Teleporters *at most* K times in total." Also, when we change the tails, we can see that it is optimal to change it to the capital.

When $K = 2$, the following diagram shows an example of optimal solution:



Now the problem can be restated as follows.

> You are given a rooted tree. You want to remove the minimum number of edges such that in each connected component, one of the following is satisfied:
>
> - The root is the capital and the depth is at most K.
> - The root is not the capital and the depth is at most $K - 1$.

This problem can be solved as follows. Run a dfs from the root. When we call $dfs(x)$, we basically compute the depth of the subtree rooted at the vertex x. However, when we call $dfs(x)$, if there is a child of x (call it y) such that $dfs(y) = K - 1$ and x is not the root, we

cut the edge between x and y. The answer is the number of cuts. The time complexity of this solution is $O(N)$.

Another solution (easier but a bit more complicated to implement) is as follows. First, choose all the vertices in decreasing order of depth, and mark them unvisited, and repeat the following.

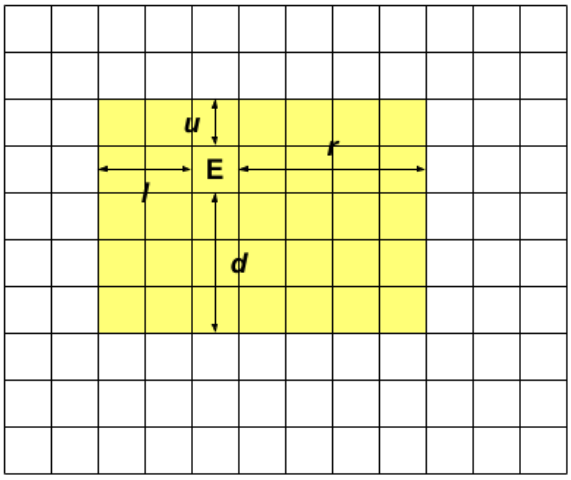
Choose a deepest (unvisited) vertex of the tree. Let's call it x. If $depth(x) \leq K$, we are done. Otherwise, we must cut at least one edge from the vertices $x, parent(x), ..., parent^{K-1}(x)$ (to their parent), and we can easily prove that it is optimal to choose an edge from the vertex $parent^{K-1}(x)$. After you cut an edge from the vertex y, you should mark all descendants of y as visited.

If you simulate this, you'll get an $O(N)$ solution.

E : Salvage Robots

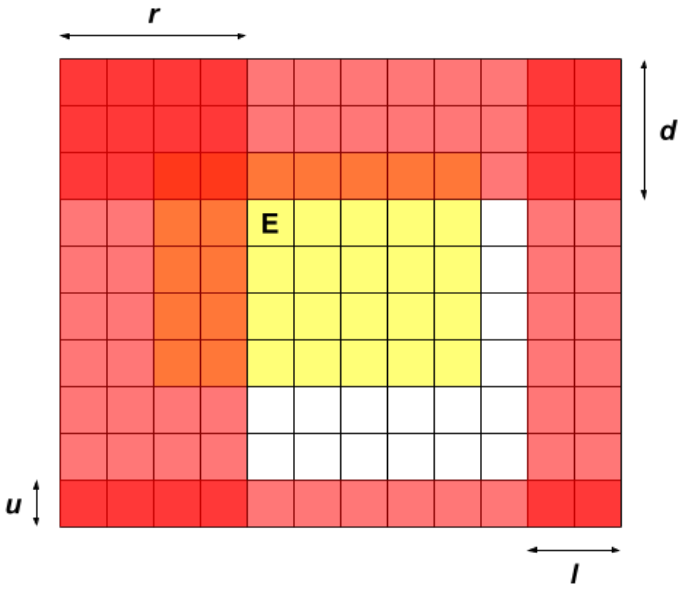
Instead of moving robots, we assume that we move the boundaty of the grid and the exit. For example, moving all robots to the left by a unit cell is equivalent to moving the boundary and the exit to the right by a unit cell. In this task, you are asked to maximize the number of robots that touch the exit before go out of the boundary (by finding the optimal movement of the boundary and the exit).

Suppose that the leftmost cell reached by the exit so far is l units to the left of the initial position of the exit. Similarly, define u, r, and d for other directions. In the diagram below, the yellow rectangle shows the bounding box of the cells reached by the exit.



When we know the values of l, u, r, and d, we can determine the set of cells that have

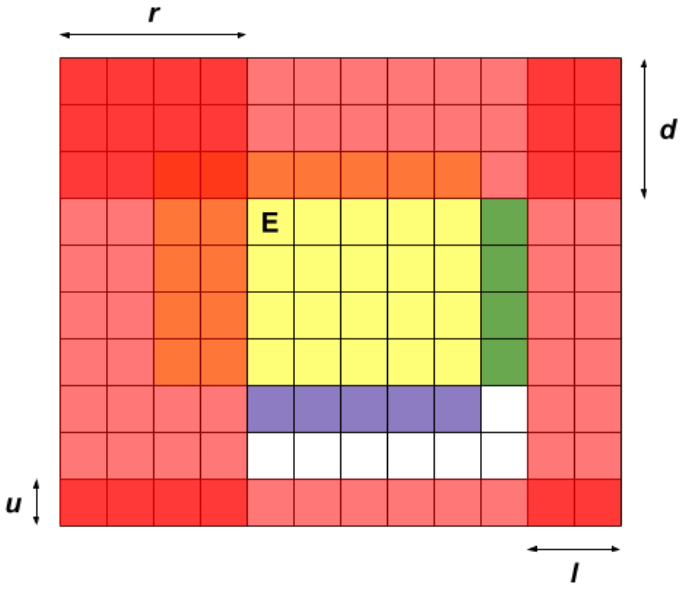
been out of the boundary. For example, the rightmost l columns have no robots because it already went out of the boundary. In the diagram below, no robots are remaining in red cells. (However, it is possible that some of them are rescued before explosion).



Notice that the set of red cells only depends on the *bounding box* of the yellow cells. Thus, you don't need to know the exact set of cells reached by the exit, instead the bounding box is enough. If there is at least one robot in the yellow region in the diagram above, we can rescue it without extending the red cells. Thus, it is enough to consider the case where there is no robot in colored cells. Also, since neither of the grid and the boundary have reached the white cells, we know that all robots that were initially at the white cells still remain.

Therefore, we can define the current state by the four integers l, u, r, and d. Define $dp[l][u][r][d]$ as the maximum number of robots you could have saved so far, when the position of the yellow bounding box is defined by l, u, r, d, and the exit has reached all cells in the bonding box.

Let's see how to update the dp array. When the exit goes out of the bounding box, one of the values l,u,r,d will be incremented. For example, if we increment r, we can save all robots in the green rectangle. Similarly, if we increment d, we can save all robots in the purple rectangle. If we increment l or u, no new robots can be saved.



In summary, you can update the values of $dp[l+1][u][r][d]$,$dp[l][u+1][r][d]$,$dp[l][u][r+1][d]$,$dp[l][u][r][d+1]$ using the value of $dp[l][u][r][d]$. The answer is the value of dp corresponding to the entire rectangle. The time complexity of this algorithm is $O(H^2W^2)$.

F : Namori

This task may look overwhelming - it is 2200 points, and probably you won't get any progress for a while. However, if you notice a single clever observation, the task suddenly becomes much more tractable.

We'll write a hint first. If you haven't solved this task by yourself and get stuck, we'll recommend you to try again after reading the hint. Can you solve it?

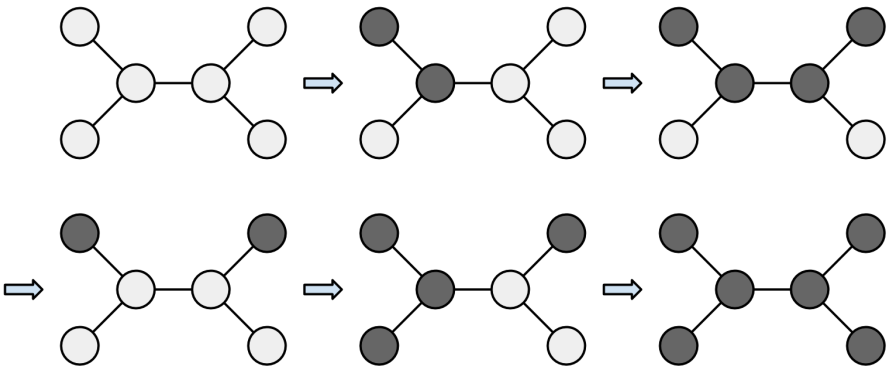
Hint: Trees are always bipartite. Let's color the given tree red and blue. Now, flip all colors in red vertices - black in red vertices becomes white, and vice versa. What does the operation mean after this transformation?

This page is intentionally left (almost) blank.

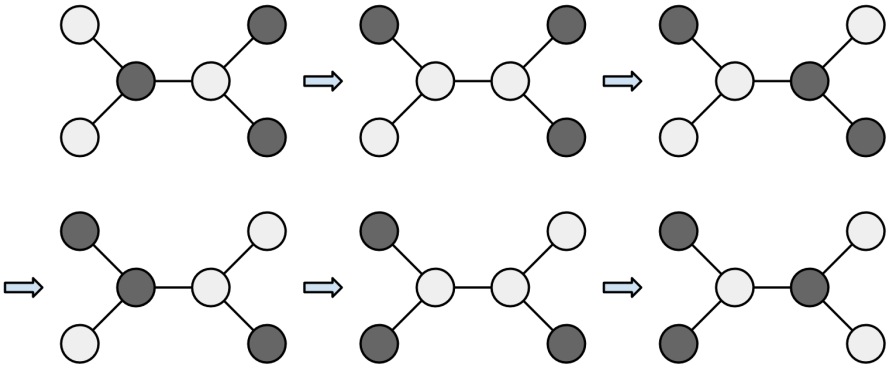
First, consider the case $M = N - 1$, that is, the graph is a tree. Currently you can convert white-white $\leftrightarrow$ black-black, but this looks difficult. Thus, we transform the graph in the following way.

Let's color the vertices of the tree red and blue, and flip all colors in red vertices. Then, the operation white-white $\leftrightarrow$ black-black becomes white-black $\leftrightarrow$ black-white.

As an example, we use Sample Input 1.



Before the transformation



After the transformation

Let's regard white vertices as empty vertices and black vertices as vertices with tokens. Then, the problem can be restated as follows:

> You are given a tree. Some vertices may contain tokens. In each operation, you can "slide" a token in a vertex into adjacent empty vertex. You are given the initial state and the final state. (State can be represented by a set of vertices with tokens). Determine if you can convert from the initial state to the final state by repeating the operations, and if this is possible compute the minimum number of operations.

Now the problem looks much easier!

We first explain an intuitive solution, and then give a formal proof. Let G be the input graph, S be the initial state, and T be the final state. (Also, assume that the graph is transformed as we described above).

Tree Case

In this case G is a tree.

First, it is obvious that the number of tokens never changes. Thus, when the number of tokens in S and T are different, the answer is -1. Otherwise, it is always possible.

Consider an edge e in G. If we cut the graph G by the edge e, we get two connected components. Let's call those components G_1 and G_2. Let x be the number of tokens in G_1 in S, and let y be the number of tokens in G_1 in T. Then, $|x - y|$ tokens must pass through the edge e.

For each edge, compute the number of tokens that pass through this edge. The answer is the sum of these values for all edges.

Even Cycle Case

When $M = N$, the graph contains exactly one cycle. We handle two cases separately depending on the parity of the length of the cycle. In this section suppose that the cycle length is even.

In this case, since the graph is bipartite, we can transform the colors in the same way and it is sufficient to solve "slide tokens" version.

Choose an arbitrary edge in the cycle and call it $e_0 = (u, v)$. Let x be the number of tokens that pass through this edge in $u \to v$ direction. (If the tokens pass through the other direction, this number is negative).

Remove this edge from the graph. If we know the value of x, by a similar observation as in the tree case, we can compute the number of tokens that pass through each edge, and these values are of the form $x + (constant)$.

Thus, the total number of operations can be represented as the sum of functions of the form $|x + (constant)|$. Since each term of this function is convex, this function is also convex, and we can compute the minimum using ternary search. (It is also possible to find the minimum by computing the median of the constants.)

Odd Cycle Case

Choose an arbitrary edge in the cycle (call it $e_0 = (u, v)$), and remove it from the graph. After the removal the graph becomes a tree, thus it is almost sufficient to solve "slide tokens"

version on a tree. The only difference is that, there are two distinct vertices in the tree A, B and the following additional operations are possible:

- If both A and B are empty, put tokens to both vertices.
- If both A and B contain tokens, remove tokens from both vertices.

Now we can change the number of tokens using these exceptional operations. However, these operations don't change the parity of the number of tokens. When the parity of the number of tokens in S and T are different, the answer is -1.

Otherwise, without loss of generality (since the operations are revertable), we can assume that the number of tokens from S to T is non-decreasing.

Suppose that the number of tokens increases by $2k$ from S to T. In this case, we need perform the operation "Put tokens on A and B" k times (and this costs k operations). Then, imagine that we add k more tokens to each of A and B, and compute the minimum cost in the same way as the tree case. The answer is this minimum cost plus k.

The time complexity of this algorithm is $\mathrm{O}(N)$.

Formal Proof

Now the formal proof of solutions described above. In the solution above we proved that we need *at least* certain number of operations, but we didn't prove why that number of operations is enough.

Let G be an arbitrary graph, and S and T be two states. Consider a "slide tokens" problem on this graph.

First, for each edge in G, we assign an integer with direction (this is the number of tokens that pass through this edge). The integer assigned to $u \to v$ is the negative of the integer assigned to $v \to u$, and for each vertex v in G, the following condition must be satisfied:

(The number of tokens in v in S) + $\sum value(w \to v)$ = (The number of tokens in v in T)

We want to prove that if we can assign integers to each edge with the condition above, we can convert from S to T with at most (the sum of absolute values of assigned integers) numbers of operations.

Let K be the number of tokens (in S and T). First, when an integer $x > 0$ is assigned to the edge $u \to v$, we add x arrows from u to v. By merging these arrows, you get K paths. Each path starts from a token in S and ends at a token in T. Thus, each token in S is now assigned to a token in T (and this relation is bijective), and the total length of paths connecting each pair is the required number of operations. In general, there are multiple optimal matchings between tokens in S and T. In this case, suppose that we want to maximize the number of

matchings to itself (that is, a token at vertex v in S is assigned to a token at vertex v in T).

Consider a pair of matching. A token at vertex p in S is assigned to a token at vertex q in T. When $p \neq q$, we can prove that the vertex p in T is empty and the vertex q in S is empty. In this case, we can move a token from p to q in $dist(p, q)$ steps. It is possible that there is another token in the path between p and q. However, when a vertex r contains a token, we can first move a token at r to q and then a token at p to r. We can do this in a similar when even when there are multiple tokens on the path between p and q.

Now, the proof for the odd cycle case. Instead of adding a token to A and B, add a new vertex v with a token and add an edge from v to A with length 0. This way, the problem is similar to the "slide tokens" problem except for the restriction that when a token is moved into A, you must move a token into "B" at the same time. This is trivial when A and B are empty before the operation. If they are not empty, for example, if you want to move a token at v to w through A but A contains a token, you can first move a token at A to w with cost $dist(A, w)$ and empty the vertex A. Do the same thing in case the vertex B is non-empty. Now, we can finally move tokens into vertices A and B, and we get a valid sequence of operations.

AtCoder Grand Contest 003 解説

DEGwer

2016 年 8 月 21 日

A: Wanna go back home

問題概要

二次元の平面上で、原点からスタートし、東西南北のうちの指定された方向に正の距離進む移動を $|S|$ 回行う。この $|S|$ 回の移動のあと、原点に戻ってくるように各移動の距離を設定できるかどうか判定せよ。

解説

北方向に進む移動があり、南方向に進む移動がない場合、最終的な位置は必ず原点より北になるため、原点に戻ってくることはできません。同様に、

- 南方向に進む移動があり、北方向に進む移動がない場合
- 西方向に進む移動があり、東方向に進む移動がない場合
- 東方向に進む移動があり、西方向に進む移動がない場合

も原点に戻ってくることはできません。

それ以外の場合、すなわち、南北方向の移動も、東西方向の移動も、どちらも存在しないかどちらも存在するかのいずれかである場合、原点に戻ってくることができます。具体的には、南北方向の移動どちらもがある場合、北向きの移動と南向きの移動の合計距離を両方 1 にするように移動する (東西方向も同様) 等で原点に戻る手順を構成することができます。

以上より、

- 北方向に進む移動があり、南方向に進む移動がない場合
- 南方向に進む移動があり、北方向に進む移動がない場合
- 西方向に進む移動があり、東方向に進む移動がない場合
- 東方向に進む移動があり、西方向に進む移動がない場合

は NO を出力し、それ以外の場合は YES を出力すればよいです。

B: Simplified mahjong

問題概要

$i(1 \leq i \leq N)$ の書かれたカードが A_i 枚ある。値が連続する、もしくは同じ 2 枚のカードはペアにすることができる。互いに重ならないように最大でペアは何組取れるか。

$N \leq 10^5, A_i \leq 10^9$

解説

まず、$A_i = 0$ となる i がないような場合について考えてみます。このとき、カードの合計枚数を S として、$\lfloor \frac{S}{2} \rfloor$ が答えになることを証明できます。

証明は以下の通りです。

まず、ペアは 2 枚からなる組なので、$\lfloor \frac{S}{2} \rfloor$ 組より多くのペアをとることはできません。逆に、次のようなアルゴリズムで、$\lfloor \frac{S}{2} \rfloor$ 組のペアをとることができます。

以下の手順で、カードをペアにして (ババ抜きのような要領で) 取り除いていくことを考えます。i を 1 から N まで動かし、次の操作をします。

値 i の書かれたカードが B_i 枚残っているとします。値 i のカード同士をペアにして、$\lfloor \frac{B_i}{2} \rfloor$ 組のペアを作り、これらのカードを取り除きます。このとき、値 i の書かれたカードは 0 枚または 1 枚残ります。1 枚が残った場合、

- $i = N$ なら、終了する
- そうでなければ、その 1 枚のカードを、値 $i + 1$ の書かれたカード (これは $A_i \neq 0$ より必ず存在します) とペアにして取り除く

この操作で、高々1 枚の値 N の書かれたカードのみが、ペアにならずに残ります。以上より、$\lfloor \frac{S}{2} \rfloor$ 組のペアが作れることが示されました。

では、問題の、$A_i = 0$ となる i があるかもしれない場合を考えてみます。$A_i = 0$ となるような i の書かれたカードを使うペアはないので、$A_i = 0$ となる i が出てきたところで数列 A を区切り、その区切られた各部分で上の問題を解き、最後にそれらを合計すればいいことが分かります。

以上をまとめると、数列 A_i を 0 が出てくるたびに区切り、その各部分ごとにその部分に属する A_i たちの合計の半分 (小数点以下切り捨て) を求め、すべて足し合わせたものを出力するというアルゴリズムで、この問題を解くことができました。時間計算量は $O(N)$ です。

C: BBuBBBlesort!

問題概要

長さ N の数列があり、全要素は相異なる。これを、連続する 2 要素を反転する操作と、連続する 3 要素を反転する操作でソート列にしたい。前者の操作の回数の最小値を求めよ。

$N \leq 10^5$

解説

連続する 3 要素を反転する操作というのは、1 つ飛ばしの位置にある 2 つの要素を入れ替える操作です。この操作では、数列の要素が全体の何番目の位置にあるかという値の偶奇は、どの要素に対しても変化することはありません。よって、元の列に奇数番目に現れる要素を偶数番目に持っていく操作や、元の列に偶数番目に現れる要素を奇数番目に持っていく操作は、後者の操作のみではできません。

前者の操作では、この偶奇を入れ替えることができます。前者の操作にかかわる要素は 2 つなので、高々2 つの要素に対し、この偶奇を入れ替えることができます。すなわち、その要素が元の列で何番目に現れるかを表す値と、ソート列で何番目に現れるかを表す値の偶奇が異なるような要素の個数の半分の回数の前者の操作は、少なくとも必要となります。

逆に、この回数の前者の操作と、何回かの後者の操作で、元の数列からソート列を得ることができることが証明できます。証明は以下の通りです。

元の列で奇数番目に現れ、ソート列で偶数番目に現れる要素の個数を X とします。明らかに、これは元の列で偶数番目に現れ、ソート列で奇数番目に現れる要素の個数と等しいです。

まず、以下を X 回繰り返します。

元の列で奇数番目に現れ、ソート列で偶数番目に現れる要素を一つとって A、逆に元の列で偶数番目に現れ、ソート列で奇数番目に現れる要素を一つとって B とします。適当な回数の後者の操作で、A を数列の 1 番目に、B を 2 番目に持ってくることができます。ここで前者の操作で A と B を入れ替えます。

そのあと、後者の操作のみを用いて、数列の偶数番目のみからなる数列と、奇数番目からなる数列をそれぞれバブルソートの要領でソートすると、ソート列が得られます。

以上で証明が完了しました。以上をまとめると、元の列で奇数番目に現れ、ソート列で偶数番目に現れる要素の個数を数え、その値を出力すればこの問題を解くことができます。時間計算量は $O(NlogN)$ です。

D: Anticube

問題概要

正整数からなる要素の重複を許す N 要素の集合 $A_1, ..., A_N$ がある。この部分集合で、どの 2 要素の積も立方数にならないようなものの最大のサイズを求めよ。

$N \leq 10^5, 1 \leq$ 集合の要素 $\leq 10^{10}$

解説

各正整数 t に対し、その標準形 $Norm(t)$ を、t が同じ素因数を 3 つ以上含まなくなるまで立方数で割ったものと定めます。例えば、$Norm(144) = 18$ です。

さらに、各正整数 t に対し、その相手 $Pair(t)$ を、t と掛けると立方数になるような整数の標準形 (これは定義より一意) と定めます。

集合の全要素に対して $Norm(t)$ と $Pair(t)$ が求まっていれば、各 $Norm(t) \neq 1$ に対し、標準形が $Norm(t)$ となるような要素数と標準形が $Pair(t)$ となるような要素数を数え、大きいほうをすべて足し合わせたものを答えとすればよいです。(ただし、$Norm(t) = 1$ なる要素が存在する場合、さらに 1 を足します。)

$Norm(t)$ と $Pair(t)$ を各要素に対して求めることを考えます。あらかじめ $(10^{10})^{\frac{1}{3}}$ までの素数を列挙しておけば、$Norm(t)$ に関しては、愚直に試し割りをすることで求めることができます。さらに、この操作で、集合の各要素に含まれる、その要素の 3 乗根以下の大きさの素因数はすべて求めることができます。

さて、$Pair(t)$ を求めることを考えます。集合の各要素に対して、その要素の 3 乗根以下の大きさの素因数はすべて求まっています。そのため、これらの素因数すべてでその要素を割ると、その値は以下の 3 通りに分類できます。

- 素数である
- 素数の 2 乗である
- 相異なる素数 2 つの積である

$Pair(t)$ の値は、2 番目のケースではその値の平方根を、それ以外のケースではその値の 2 乗を求めることで求まります (実装においては、実際に 2 乗を求める必要はありません)。よって、2 番目のケースかどうかを判定すればいいですが、それはあらかじめ $(10^{10})^{\frac{1}{2}}$ までの素数の 2 乗を列挙しておけば判定することができます。

以上をまとめると、

- $(10^{10})^{\frac{1}{2}}$ までの素数を列挙し、
- このデータを使って $Norm(t)$ と $Pair(t)$ を求めて適切に管理し、
- このデータから答えを求める

という操作で、この問題を解くことができます。最初の操作が篩法によって $O(\sqrt{(MAXA_i)}loglog(MAXA_i))$ で、2 番目の操作が (X 以下の素数の個数が $O(\frac{X}{\log X})$ であることを利用すると)$O(N\frac{(MAXA_i)^{\frac{1}{3}}}{\log(MAXA_i)})$ で、最後の操作が適切な実装で $O(NlogN)$ でできるので、十分高速です。

E: Sequential operations on sequence

問題概要

数列 $1, 2, ..., N$ がある。これに、現在の数列を無限回繰り返したものの先頭 A_i 文字をとる操作を Q 回施す。最終的にできる数列に $1, 2, ..., N$ がそれぞれいくつずつ現れるかを求めよ。

$N, Q \leq 10^5, A_i \leq 10^{18}$

解説

まず、$A_i \geq A_{i+1}$ なら、A_i はなかったことにしても問題ありません。なぜなら、先頭 A_i 文字をとった後に先頭 A_{i+1} 文字をとる操作は、単に先頭 A_{i+1} 文字をとる操作と同じだからです。

この事実より、A_i たちを、適切な項の削除により、単調増加となるようにした数列 B を代わりに考えます。B の長さを L とします。この適切な項の削除は、stack 等を用いることで線形時間で行うことができます。

操作を後ろから見ます。すると、最終的にできる数列に現れる各値の個数は、次のようなアルゴリズムで求めることができます。

- はじめに、A_Q 個の 1 を並べる。
- 操作列 B を後ろから見て、以下の操作を順に行う。
  - 操作列の今見ている箇所の値が X だとする。新しい列を、X 項からなり、i 項目は添え字が $mod X$ で i と等しいような元の列の要素の合計とした列とする。
- 最後に現れる数列を出力する。

数列を「のばす」操作でどの項がどこにくるのかを考えれば、このアルゴリズムの正当性は簡単に示すことができます。

このアルゴリズムを高速化することを考えます。まず、各操作を (逆順に) ひとつずつ処理する代わりに、以下のような順番で操作を行います。

- 配列 a, t を用意する。
- $t[L] = 1$ とする。
- 操作列 B を後ろから見て、以下の操作を順に行う。
  - 操作列の今見ている箇所が B_i だとする。
  - $k = B_i$ とする。
  - 操作列 B を $i-1$ から逆順に見て、次の操作を行う。

        * 今見ている箇所が B_j だとする。
        * $k \geq B_j$ なら、$t[j]$ に $t[i] \times \lfloor \frac{k}{B_j} \rfloor$ を加算し、k を $kmodB_j$ とする。
    - $a[1]$ から $a[k]$ までに $t[i]$ を加算する。

- 配列 a の中身を出力する。

このアルゴリズムにおいて、$t[i]$ の値は、i 番目までの操作を行ったときにできる数列が、最終的な数列に何回完全に現れるかを表します。k の値は、その「完全に現れる数列」の部分を除いた残りの長さを表します。

さらにこのアルゴリズムを高速化します。まず、k の値が変化するのは、$k \geq B_j$ のときのみです。この操作で k の値は半分以下となるので、この操作は高々$O(\log A_Q)$ 回しか行われません。よって、$k \geq B_j$ となる (後ろから見て) 最初の要素を二分探索等で求めることで、このシミュレーションをひとつの i に対して $O(\log Q \log A_Q)$ で行うことができます。

さらに、$a[1]$ から $a[k]$ までに $t[i]$ を加算する操作は、累積和のテクニックを用いて全体で線形時間で行うことができます。

以上より、上記のアルゴリズムの高速化で、$O(Q \log Q \log A_Q + N)$ 時間でこの問題を解くことができました。

F: Fraction of fractal

問題概要

$H * W$ のグリッドがあり、各マスは黒か白のいずれか。黒のマスは 4 方向に連結。レベル 0 のフラクタルとは黒いマスひとつで、レベル $k+1$ のフラクタルとは、レベル k のフラクタルをグリッドの黒いマスすべてに置き、それ以外の場所を白で埋めたものとする。レベル K のフラクタルの連結成分の個数を求めよ。

$H, W \leq 1000, K \leq 10^{18}$

解説

一番上の行と一番下の行がともに黒マスであるような列と、一番左の列と一番右の列がともに黒マスであるような行が両方存在したとします。このとき、フラクタルをグリッドに並べていく操作で、すべての黒マスが連結となることが帰納的にわかります。よってこのときは、1 を出力すればよいです。

そうでない場合、一番上の行と一番下の行がともに黒マスであるような列は存在しないとして一般性を失いません。このとき、フラクタルのランクが 1 増えることで、連結成分の個数がどうなるかを考えます。

仮定より、黒マス同士がフラクタルのランクを上げることで新しく縦につながることはありません。もし横にもつながることがなければ、連結成分の個数は、グリッドの黒マスの個数倍になります。つなりこのとき、求める連結成分の個数は (グリッドの黒マスの個数)$^{K-1}$ です。

そこで、それ以外の場合を考えます。もとのフラクタルが横につながることで、いくつの連結成分が減少するかを数えることにします。

この減少は、グリッドで黒マスが横に連続している部分に対応する箇所で起こります。このような箇所 1 箇所につき、連結成分はいくつ減少するでしょうか？この減少は、もとのフラクタルで、同じ行の一番左のマスと一番右のマスが両方黒であるような行を持つような黒マスの連結成分の組の個数分の回数起こります。では、この値はどう計算すればいいでしょうか？

この値は、フラクタルのランクが 1 上がることで、元のグリッドで一番左のマスと一番右のマスがともに黒であるような行の個数倍になります。これは、黒マスが縦につながらないことからわかります。

以上をまとめると、求めたいものは、以下のベクトルの第一成分です。ただし、a はグリッドの黒マスの個数、b はグリッドの黒マス同士が横に連続する場所の個数、c はグリッドの右端のマスと左端のマスが両方黒であるような行の個数とします。

$$\begin{pmatrix} a & -b \\ 0 & c \end{pmatrix}^{K-1} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \tag{1}$$

この値は二分累乗法で求めることができるので、$O(HW + logK)$ でこの問題を解くことができます。

AtCoder Grand Contest 003 Editorial

DEGwer

August 21st, 2016

A: Wanna go back home

Summary of Problem Statement

You start from the origin on a plane and moves $|S|$ times. In each move you move to the given direction (east, west, south, or north) by a positive distance. Determine if you can be at the origin after these moves.

Solution

If you move to the north and you don't move to the south, your position after the $|S|$ movements will be to the north of the origin, so you can't return to the origin. Similarly,

- When you move to the south and you don't move to the north
- When you move to the west and you don't move to the east
- When you move to the east and you don't move to the west

you can't return to the origin.

Otherwise you can return to the origin. When you move to both south and north, you can set the moving distances such that you move to the north by 1 in total and to the south by 1 in total. We can set the moving distance for east/west directions similarly.

Thus, if

- you move to the north and you don't move to the south, or
- you move to the south and you don't move to the north, or
- you move to the west and you don't move to the east, or
- you move to the east and you don't move to the west

print 'NO', and otherwise print 'YES'.

B: Simplified mahjong

Summary of Problem Statement

There are A_i cards with an integer $i (1 \leq i \leq N)$. You can make a pair with two cards if the integers on them are the same or adjacent. Compute the maximum number of disjoint pairs you can make.

$N \leq 10^5, A_i \leq 10^9$

Solution

First, assume that there is no i such that $A_i = 0$. In this case, we can prove that the answer is always $\lfloor \frac{S}{2} \rfloor$, where S is the total number of cards.

The proof is the following.

It is obvious that we can't make more than $\lfloor \frac{S}{2} \rfloor$ pairs. On the other hand, we can construct $\lfloor \frac{S}{2} \rfloor$ pairs in the following algorithm:

Let $x_1, \ldots, x_S$ be all the cards sorted in non-decreasing order. Then, for each i, $x_{i+1} - x_i \leq 1$ (Otherwise, there is no card with the integer $x_i + 1$, which is a contradiction). Thus, we can construct $\lfloor \frac{S}{2} \rfloor$ pairs $(x_1, x_2), (x_3, x_4), \ldots$. This way we can make $\lfloor \frac{S}{2} \rfloor$ pairs.

When $A_i = 0$ for some i, you can't use cards with the integer i, so you can split the sequence at i and for each part you can solve the problem independently (the answer is the sum of these independent problems).

Therefore, you can split the sequence A_i when 0 appears, and for each part compute the half (with the fraction part rounded down) of the sum of A_i, and the answer is the sum of these numbers. The time complexity is $O(N)$.

C: BBuBBBlesort!

Summary of Problem Statement

There is a sequence with N elements (elements are pairwise distinct). You want to sort the sequence using two types of operations: reverse two consecutive elements or three consecutive elements. Minimize the number of former operations.

$N \leq 10^5$

Solution

Reversing the order of 3 consecutive elements is equivalent to swapping two elements at the distance of two. Thus, in this operation, the parity of the position of a particular element never changes. If we want to move an element at an odd position to an even position, we must use the former operation.

In the former operation, we can change the parity of position for two elements. Thus, if we want to change the parity of position for k elements, we need at least $k/2$ operations of the former type.

On the other hand, we can prove that we can sort the sequence using $k/2$ operations of the former type (and some number of perations of the latter type).

Let X be the number of elements that are at odd positions in the original sequence but at even positions in the sorted sequence. This number is the same as the number of elements that are at even positions in the original sequence but at odd positions in the sorted sequence, so $X = k/2$.

First, repeat the following X times:

Let A be an element that is at odd position in the original sequence and at even position in the sorted sequence. Let B be an element that is at even position in the original sequence and at odd position in the sorted sequence. By repeating the latter type of operations, you can move A to the first position and B to the second position. Then you can swap A and B using the former type of operation.

After that, by using only the latter type of the operations, we can sort all elements at even positions like bubble sort. Similarly, we can sort elements at odd positions, and we sort the entire sequence.

In summary, the answer is the number of elements at odd positions that should go to even positions after sorting. The time complexity is $O(NlogN)$.

D: Anticube

Summary of Problem Statement

You are given a multiset $A_1, ..., A_N$. Compute its subset with maximum number of elements such that the product of any two distinct elements is not a cubic number.

$N \leq 10^5, 1 \leq elements \leq 10^{10}$

Solution

For an integer t, define $Norm(t)$ as the small divisor of t such that $t/Norm(t)$ is a cubic number. For example, $Norm(144) = 18$.

Also, define $Pair(t)$ as the unique integer without non-trivial cubic divisor such that $t \times Pair(t)$ is a cubic number. For example, $Pair(18) = 12$.

If we know $Norm(t)$ and $Pair(t)$ for each element t, for each $x \neq 1$ we compute the number of t such that $Norm(t) = x$ and $Pair(t) = x$, and the answer is the sum of maximum of these numbers. (If there is at least one element t such that $Norm(t) = 1$, add 1 to the answer.)

Let's compute $Norm(t)$ and $Pair(t)$ for each element. First, compute all primes smaller than $(10^{10})^{\frac{1}{3}}$. You can compute $Norm(t)$ in a straightforward way. Also, you can compute small divisors of each element.

How can we compute $Pair(t)$? For each element t, we already know all divisors up to $t^{1/3}$. Thus, we can write t as $t = (productofknownprimes) \times s$, where all prime divisors of s are greater than $t^{1/3}$. There are three cases:

- $s = 1$ or s is a prime.
- s is square of a prime.
- s is the product of two distinct primes.

In the first and the third case, $Pair(s) = s^2$, while in the second case $Pair(s) = sqrt(s)$. Thus, we only need to know whether s is a square of a prime. This can be done by pre-compute all primes up to $(10^{10})^{\frac{1}{2}}$ and their squares.

In summary, we can solve the problem in the following way:

- Compute all primes up to $(10^{10})^{\frac{1}{2}}$,
- Compute $Norm(t)$ and $Pair(t)$ for each element t using the precomputed list of small primes, and

- Compute the answer using $Norm(t)$ and $Pair(t)$.

The first step is $O(\sqrt{(MAXA_i)}loglog(MAXA_i))$. The second step is (by prime number theorem: the number of primes up to X is $O(\frac{X}{\log X})$) $O(N\frac{(MAXA_i)^{\frac{1}{3}}}{\log(MAXA_i)})$. The last step is $O(NlogN)$. Each of these steps are sufficiently fast.

E: Sequential operations on sequence

Summary of Problem Statement

There is a sequence $1, 2, ..., N$. You perform Q operations on this sequence. In the i-th operation, you replace the sequence by the first A_i elements of its infinite repetition. Compute the frequencies of $1, 2, ..., N$ in the final sequence.

$N, Q \leq 10^5, A_i \leq 10^{18}$

Solution

If $A_i \geq A_{i+1}$, we can ignore A_i. This is because the sequence of two operations A_i, A_{i+1} is equivalent to a single operation A_{i+1}.

Therefore, by removing appropriate elements, you can convert the sequence A into an increasing sequence B. Let L be the length of B. This sequence can be computed in linear time using a stack.

We process the sequence of operations in the reverse order.

- First construct a sequence with A_Q elements. All elements are 1.
- We process the sequence of operations B in the reverse order, and perform the following operations:
  - Suppose that we currently process X.
  - We replace the sequence by a new sequence with X elements. The i-th element of the new sequence is the sum of elements in the old sequence whose indices are i in $mod X$.
- Print the final sequence.

Let's improve this algorithm. Instead of processing each operation independently (in the reverse order), we process the operations in the following order:

- We use two arrays: a and t.
- Let $t[L] = 1$.
- We process the sequence of operations B in the reverse order, and perform the following operations:
  - Suppose that we currently process B_i.
  - Let $k = B_i$.

- We process the sequence of operations $B_{i-1}, \ldots, B_1$ and perform the following operations:
  - Suppose that we currently process B_j.
  - If $k \geq B_j$, we add $t[i] \times \lfloor \frac{k}{B_j} \rfloor$ to $t[j]$ and replace k with $k mod B_j$.
- Add $t[i]$ to $a[1], \ldots, a[k]$.

- Print the array a.

In this algorithm, $t[i]$ is the number of occurrences of the sequence after the i-th operation in the final sequence. The value of k is the number of elements in the final sequence except for the $t[i]$ occurrences of the sequence.

We will further improve this algorithm. The value of k changes only when $k \geq B_j$. By this operation, k decreases by at least twice, so this operation happens at most $O(\log A_Q)$ times. Thus, if we compute the maximum j such that $k \geq B_j$ using binary search, we can simulate this for each i in $O(\log Q \log A_Q)$ time.

Furthermore, we can add $t[i]$ to $a[1], \ldots, a[k]$ in $O(1)$ using partial sums.

The algorithm works in $O(Q \log Q \log A_Q + N)$ time.

F: Fraction of fractal

Summary of Problem Statement

There is an $H * W$ grid, and each cell is colored black or white. Black cells are 4-connected. The level-0 fractal is a single black cell. The level-$k+1$ fractal is obtained by arranging level-k fractals at black cells in the grid. Compute the number of connected components in the level-K fractal.

$H, W \leq 1000, K \leq 10^{18}$

Solution

We call a grid *vertically connected* if there is a column whose topmost cell and bottommost cell are both black. Similarly, define *horizontally connected*.

When the grid is *vertically connected* and *horizontally connected*, regardless of k, all black cells are connected. This can be proved by a simple induction. Thus, in this case the answer is 1.

When the grid is not *vertically connected* and not *horizontally connected*, each time when the level increase, the number of connected component is multiplied by the number of black cells in the grid. Thus, the answer is $(the number of black cells)^{K-1}$.

Otherwise, without loss of generality, we can assume that the grid is *horizontally connected* but not *vertically connected*.

For each black cell in the level-$k-1$ fractal, we put a given grid, and this increases the number of connected component by one. However, when two black cells are horizontally adjacent in the level-$k-1$ grid, the two grid patterns corresponding to these two cells will be in the same connected components. Thus, the answer is $x - y$, where x is the number of black cells in the level-$k-1$ fractal, and y is the number of horizontally adjacent pairs of black cells in the level-$k-1$ fractal.

In summary, the answer is the first element of the following vector. Here a is the number of black cells in the grid, b is the number of pairs of horizontally adjacent black cells, and c is the number of rows such that both its leftmost cell and rightmost cell are black.

$$\begin{pmatrix} a & -b \\ 0 & c \end{pmatrix}^{K-1} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \tag{1}$$

The time complexity is $O(HW + logK)$.

AtCoder Grand Contest 002 解説

A : Range Product

次のような場合分けをします.

- $0 < A \leq B$ のとき : `Positive`
- $A \leq B < 0$ のとき
  - $B - A + 1$ が偶数のとき : `Positive`
  - $B - A + 1$ が奇数のとき : `Negative`
- $A \leq 0 \leq B$ のとき : `Zero`

B : Box and Ball

次のように問題を言い換えると考えやすいかもしれません.

> N 個のコップがある．最初，1 番目のコップには赤い水が 1 L 入っている．また，2〜N 番目のコップには無色の水が 1 L ずつ入っている．M 回の操作を順に行う．i 回目の操作では，x_i 番目のコップから水を 1 L すくい，それを y_i 番目のコップへ移す．すべての操作を終えた後，赤い水が入っているコップは何個か求めよ.

言い換えた後の問題は次のようにシミュレートできます．まず，必要な配列を

- $num[i] := i$ 番目のコップに入っている水の量
- $red[i] := i$ 番目のコップに赤い水が入っているか

と定義します．その後，次の擬似コードのような処理を行います.

```
for i = 1 to N do
  num[i] ← 1
  red[i] ← false
end for
red[1] ← true

for i = 1 to M do
  if red[x_i] then
    red[y_i] ← true
  end if
  num[x_i] ← num[x_i] − 1
  num[y_i] ← num[y_i] + 1
  if num[x_i] = 0 then
    red[x_i] ← false
  end if
end for
```

シミュレーションが終わった後，配列 red のうち true の個数が答えです．

C : Knot Puzzle

最後に結び目 i をほどくとすると，$a_i + a_{i+1} \geq L$ でなければなりません．条件を満たす i が存在しない場合，明らかに答えは `Impossible` です．逆に，条件を満たす i がひとつでも存在する場合，答えは `Possible` であることが示せます．結び目をほどく順番は次のように構成できます．

まず，$a_i + a_{i+1} \geq L$ を満たす i をひとつ選びます．次に，結び目 $1, 2, \ldots, i-1$ をこの順にほどいた後，結び目 $N-1, N-2, \ldots, i+1$ をこの順にほどきます．最後に，結び目 i をほどきます．こうすると，まだ結び目が残っているひと繋がりのロープには，必ず長さ $a_i + a_{i+1}$ の区間が含まれます．よって，各操作で選ぶひと繋がりのロープは，必ず長さの総和が L 以上となります．

D : Stamp Rally

まずは，「j 組目の兄弟のスコアは i 以下か？」という判定問題を考えてみましょう．この問題は次のように解くことができます．

兄弟が辺 $1, 2, \ldots, i$ のみを通って訪れられる頂点を数え上げ，それが z_i 以上か判定すればよいです．そのために，辺 $1, 2, \ldots, i$ のみからなるグラフを構築します．このグラフの連結成分のうち，頂点 x_i または頂点 y_i を含む連結成分が，兄弟が訪れられる頂点ということになります．例えば Union-Find を用いることで，辺 $1, 2, \ldots, i$ を追加するのを $\mathrm{O}(N + M)$ 時間で，兄弟が訪れ

られる頂点を数え上げるのを $\mathrm{O}(1)$ 時間で行えます．

以上の判定問題において i を二分探索することで，j 組目の兄弟のスコアを $\mathrm{O}((N+M)\log M)$ 時間で求められます．これを Q 組の兄弟について別々に行うことで，すべての兄弟のスコアを $\mathrm{O}(Q(N+M)\log M)$ 時間で求められます．もちろん，この方法では計算時間が掛かりすぎるので，高速化の必要があります．

Q 組の兄弟について別々に二分探索を行う場合，「グラフに辺 1, 2, . . . , M を追加していく」というまったく同じ処理を $\mathrm{O}(Q\log M)$ 回も繰り返すことになり，非効率的です．そこで，Q 組の兄弟について並列に二分探索を行うことを考えます．つまり，「グラフに辺 1, 2, . . . , M を追加していく」というパスを $\mathrm{O}(\log M)$ 回だけ繰り返し，各パスの適切なタイミングで 1, 2, . . . , Q 組目の兄弟が訪れられる頂点を数え上げます．この「適切なタイミング」は過去の二分探索の結果から決まります．この方法の計算時間は $\mathrm{O}((N+M+Q)\log M)$ となり，十分に高速です．

E : Candy Piles

まず，配列 a を a_i の降順にソートしておきます．以降は $a=(7,7,7,6,4,4,4,2,2)$ を例に説明します．

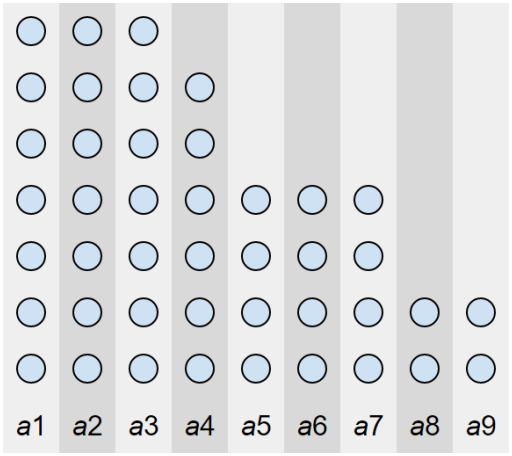


図 1　$a=(7,7,7,6,4,4,4,2,2)$ の例

「キャンディが最も多く残っている山をひとつ選び、その山のキャンディをすべて食べる」という操作は，最も左の列を消すことと等価です．また，「キャンディが残っているすべての山から、1個ずつキャンディを食べる」という操作は，最も下の行を消すことと等価です．これらの操作を交互に行い，すべてのキャンディを消したプレイヤーが負けです．

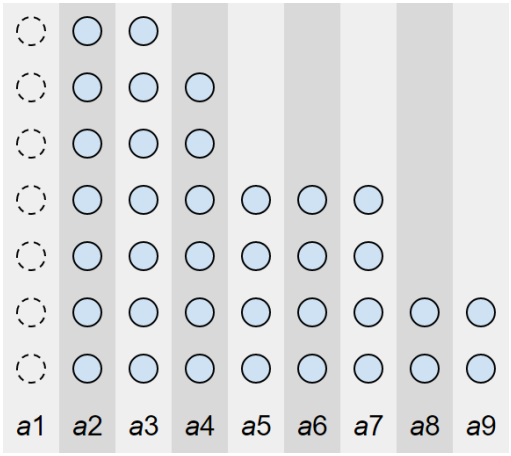


図 2　1 つ目の操作

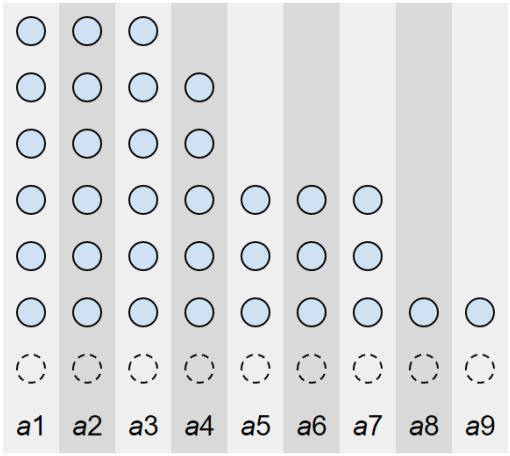


図 3　2 つ目の操作

このゲームは，さらに次のように言い換えられます．配列 a に対応するヤング図形を考えます．最初，原点（左下の角）に駒が置かれています．2 人のプレイヤーは交互に，駒をひとつ上かひとつ右へ動かします．駒をヤング図形の縁へ動かしたプレイヤーが負けです．

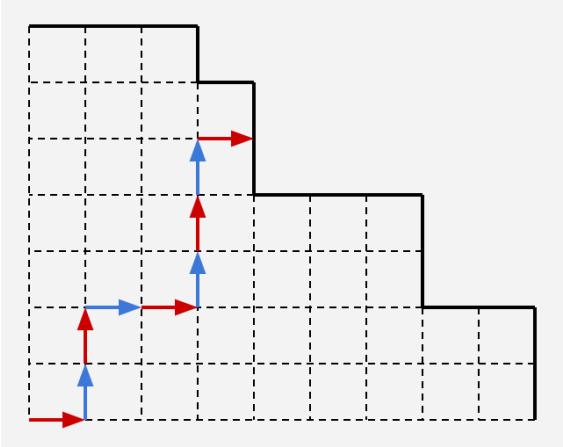


図 4　ヤング図形上のゲーム

ヤング図形上の各座標について，「その座標に駒が置かれた状態でターンが回ってきた場合，自分は勝つか負けるか」を ○ / × で書き込んでみます．まず，縁にはすべて ○ を書き込みます．それ以外の座標については，ひとつ上の座標とひとつ右の座標がともに ○ ならば × を書き込み，そうでなければ ○ を書き込みます．書き込みが終わった後，原点の ○ / × を見れば，答えが分かります．しかし，この方法の計算時間は $\mathrm{O}(\sum a_i)$ で，制限時間に間に合いません．

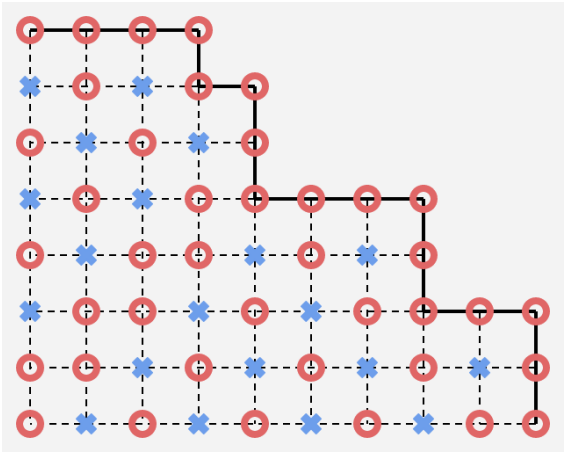


図 5　各座標の ○ / ×

上図を観察すると，縁以外の ○ / × は斜め方向に並んでいることが分かります．この性質を利用すると，原点の ○ / × を次のように求めることができます．まず，原点から右上向きへ，縁に達する直前まで矢印を伸ばします．次に，そこから右向きと上向きへ，縁に達する直前まで矢印を伸ばします．右向きと上向きの矢印の長さがともに偶数ならば，原点は × です．そうでなければ，原点は ○ です．この方法の計算時間は $\mathrm{O}(N)$ で，十分に高速です．

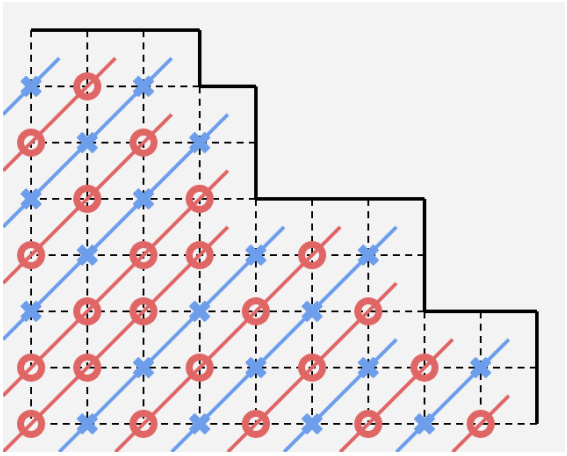


図 6　斜め方向に並ぶ ○ / ×

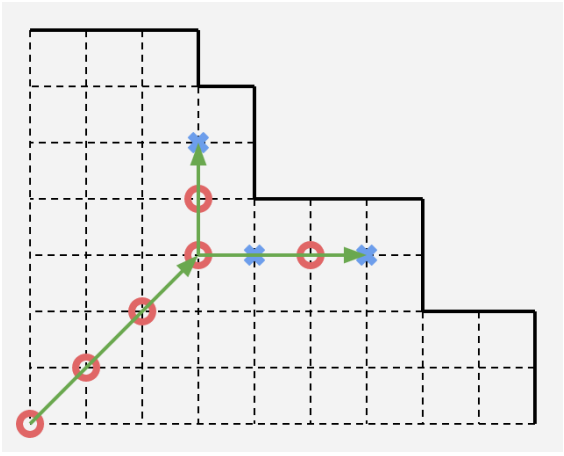


図 7　原点の ○ / × を高速に求める

F : Leftmost Ball

$K = 1$ の場合，答えは 1 通りです．以降は，$K \geq 2$ の場合のみを考えます．さらに，塗り替え後の色の列のうち，色 $1, 2, \ldots, N$ がこの順に現れるようなものだけを数え上げることにします．この個数に $N!$ を掛ければ答えが求まります．

まずは，色の列が与えられたとき，それが塗り替え後の色の列としてあり得るか判定する問題を考えてみましょう．もちろん，色 0 がちょうど N 個，色 1, 2, ..., N がちょうど $K-1$ 個ずつ含まれることが必要条件です．さらに，次のような必要条件も思いつきます．

- 各 $1 \leq i \leq N$ について，(左から i 番目の色 0) $<$ (最も左の色 i) という位置関係が成り立つ．

例えば，$(0, 1, 0, 2, 1, 2, \underline{3}, \underline{0}, 3)$ という色の列は，(最も左の色 3) $<$ (左から 3 番目の色 0) という位置関係なので，塗り替え後の色の列としてあり得ません．実は，これらの必要条件を満たす色の列は，常に塗り替え後の色の列としてあり得ます．というのも，色 0 を左から順に色 1, 2, ..., N へ塗り替えると，塗り替え前の色の列が構成できるからです．

以上より，問題は次のように言い換えられます．

> 色 0 をちょうど N 個，色 1, 2, ..., N をちょうど $K-1$ 個ずつ，左から右へ並べる．ただし，各 $1 \leq i \leq N-1$ について，最初の色 i を並べた直後から色 $i+1$ を並べ始められる．また，各 $1 \leq i \leq N$ について，i 番目の色 0 を並べた直後から色 i を並べ始められる．色の列は何通りか？

この問題の答えは，次のようなグラフをトポロジカルソートする方法の個数として表せます．

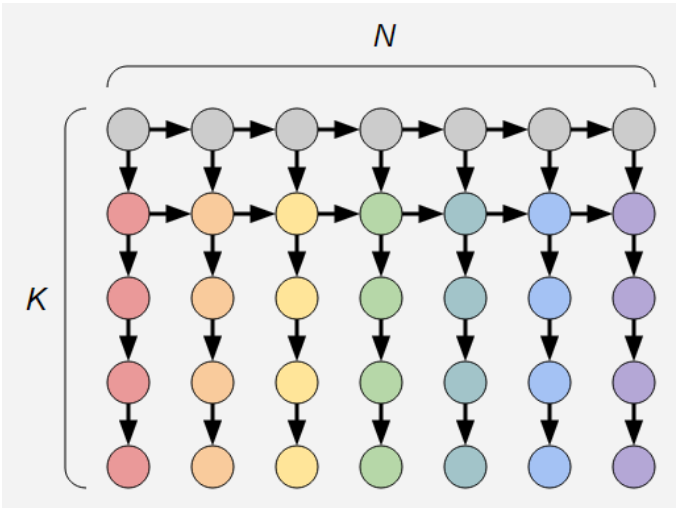


図 8　色の位置関係を表すグラフ

トポロジカルソートする方法の個数は，次のような DP で数え上げられます．下図のようなグラフをトポロジカルソートする方法を $dp[i][j]$ 通りと定義します．下図の場合，先頭に並べられる頂点は 2 通りあります．黒の頂点を先頭に並べる場合，残りの頂点をトポロジカルソートする方法は $dp[i-1][j]$ 通りです．一方，オレンジの頂点を先頭に並べる場合，残りの頂点をトポロジカルソートする方法の個数はどうなるでしょうか？ オレンジの頂点を先頭に並べる場合，残りのオレンジの頂点もあらかじめ位置を決めてしまうことにします．残りのオレンジの頂点は $K-2$ 個で，残りの全色の頂点は $i+j(K-1)-1$ 個なので，位置を決める方法は ${}_{i+j(K-1)-1}C_{K-2}$ 通りです．よって，残りの頂点をトポロジカルソートする方法は ${}_{i+j(K-1)-1}C_{K-2} \cdot dp[i][j-1]$ となります．このような漸化式にしたがって $dp[N][N]$ を計算すればよいです．計算量は $\mathrm{O}(N^2)$ です．

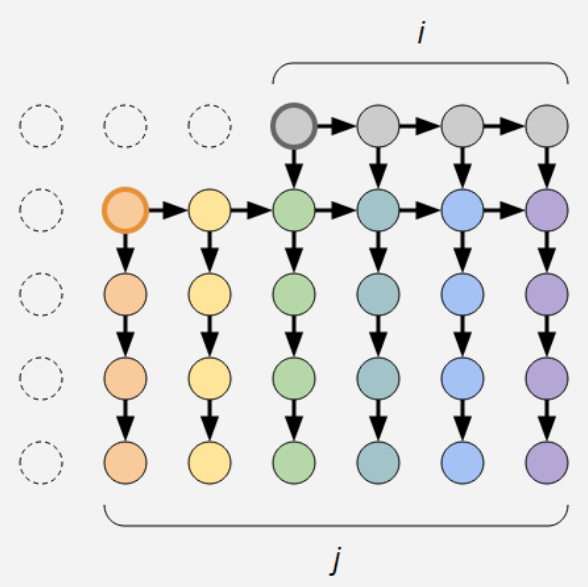


図 9　$dp[i][j]$ に対応するグラフ

AtCoder Grand Contest 002 Editorial

A : Range Product

Consider the following cases:

- $0 < A \leq B$: `Positive`
- $A \leq B < 0$
  - $B - A + 1$ is even : `Positive`
  - $B - A + 1$ is odd : `Negative`
- $A \leq 0 \leq B$: `Zero`

B : Box and Ball

You can restate the problem as follows:

> There are N cups. Initially, the cup 1 contains 1 liter of red water, and each of the other cups contains 1 liter of transparent water. You perform M operations. In the i-th operation, you take 1 liter of water from the cup x_i and pour it into the cup y_i. After you perform all the operations, compute the number of cups that contain red water.

After the restatement, you can solve the problem by the following simulation. First, define two arrays:

- $num[i]$:= The amount of water in the cup i
- $red[i]$:= Whether the cup i contains red water or not

Then, perform the following operations:

```
for i = 1 to N do
  num[i] ← 1
  red[i] ← false
end for
red[1] ← true

for i = 1 to M do
  if red[x_i] then
    red[y_i] ← true
  end if
  num[x_i] ← num[x_i] − 1
  num[y_i] ← num[y_i] + 1
  if num[x_i] = 0 then
    red[x_i] ← false
  end if
end for
```

The answer is the number of "true" in the array red after the simulation.

C : Knot Puzzle

If that the knot i is untied last, $a_i + a_{i+1} \geq L$ must be satisfied. When there is no such i, the answer is obviously `Impossible`. On the other hand, when there exists such i, we can prove that the answer is `Possible`. We can construct the ordering of knotting in the following way:

Choose an i that satisfies $a_i + a_{i+1} \geq L$. First, untie the knots $1, 2, \ldots, i-1$ in this order. Then, untie the knots $N-1$, $N-2$, $\ldots$, $i+1$ in this order. Finally, untie the knot i. This way, the ropes with knots always form a single connected component and this component contains an interval of length $a_i + a_{i+1}$. Thus, in each operation, the length of the chosen rope os at least L.

D : Stamp Rally

Consider the following decision problem: "Is the score of the j-th pair of brothers at most i?" This problem can be solved in the following way.

It is sufficient to count the number of vertices that are reachable by the brothers by using edges $1, 2, \ldots, i$, and check if this is at least z_i. In order to do this, construct a graph with only edges $1, 2, \ldots, i$. The brothers can visit a vertex if this vertex is in the same connected

component of this graph as vertices x_i or y_i. For example, if you use disjoint-set union structure, you can add the edges $1, 2, \ldots, i$ in almost $\mathrm{O}(M)$ time (strictly speaking, there is an extra factor of inverse-ackerman function but we can ignore it) and count the number of vertices that can be visited by the brothers in $\mathrm{O}(1)$ time.

In this decision problem, by using a binary search on i, we can compute the score of the j-th brothers in $\mathrm{O}(M \log M)$ time. If you do this for Q pairs of brothers independently, you can compute the scores for all pairs in $\mathrm{O}(QM \log M)$ time. Of course, we need to improve it.

If you do the binary search for Q pairs of brothers independently, you repeat exactly the same sequence of operations "add edges $1, 2, \ldots, M$ to the graph in this order" $\mathrm{O}(Q \log M)$ times and it looks inefficient. We want to perform the binary search for Q pairs of brothers in parallel. That is, you can repeat the sequence of operations $\mathrm{O}(\log M)$ times and count the number of vertices visited by each pair of brothers at appropriate time. This "appropriate time" can be determined using the results of former binary search.

This way the algorithm works in $\mathrm{O}((M + Q) \log M)$ time.

More detailed description of parallel binary search

We use a queue in this algorithm. This queue contains tuples of the form (L, R, v), where L and R are integers and v is a set of queries. This tuple means that for each query in v, we know that the answer is in the interval $(L, R]$.

- Initialize a disjoint-set union structure.
- Push $(0, M, \{1, ..., Q\})$ into the queue (for each query we know that the answer is in the interval $(0, M]$).
- While the queue is non-empty, pop the front element from the queue. Let (L, R, v) be this element.
  - If $R - L = 1$, we get the answers for queries in v.
  - Otherwise, let $X := (L + R)/2$ (integer division).
  - If the DSU contains more than X edges, initialize it.
  - While it contains less than X edges, add edges to it.
  - Now, by using this DSU, for each query in v we can check if the answer is greater than X or not - separate v into v_1 and v_2.
  - Push (L, X, v_1) and (X, R, v_2) into the queue.

Notice that this way the initialization of DSU can happen at most $O(logM)$ times. Therefore the algorithm works in $O(MlogM)$ time.

E : Candy Piles

First, sort the array a in decreasing order. In the remaining part of this editorial, we use an example $a = (7, 7, 7, 6, 4, 4, 4, 2, 2)$.

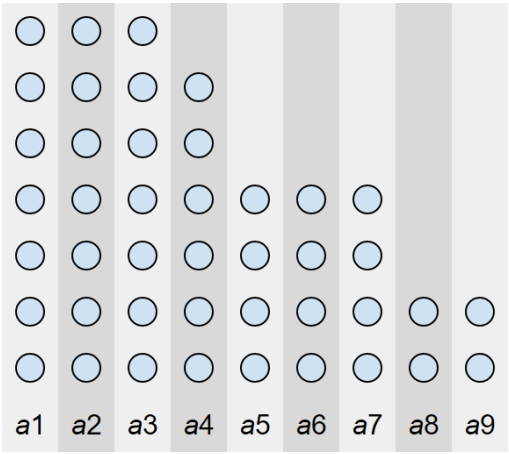


図 1　An example: $a = (7, 7, 7, 6, 4, 4, 4, 2, 2)$

The first type of operation (take all candies from the biggest pile) is equivalent to removing the leftmost column in the diagram. The second type of operation (take a candy from each non-empty pile) is equivalent to removing the bottommost row in the diagram. Two players perform these operations alternately, and the player who removes the last candy loses.

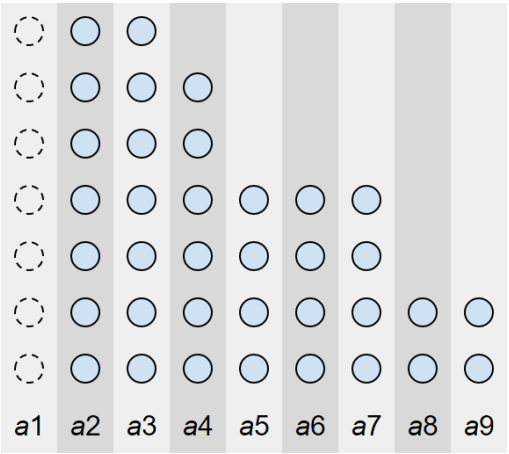


図 2　The first type of operation

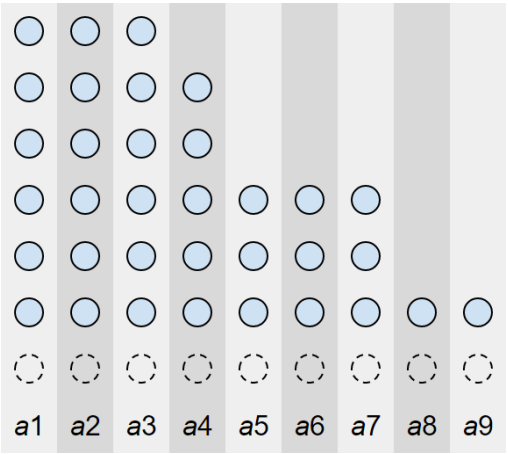


図 3　The second type of operation

We can further restate the statement as follows. Consider a young diagram that corresponds to the array a. Initially, you put a token on the left-bottom corner. Two players move the token by a unit distance to the right or to the up alternately, and the player who moves the token to the edge of the diagram loses.

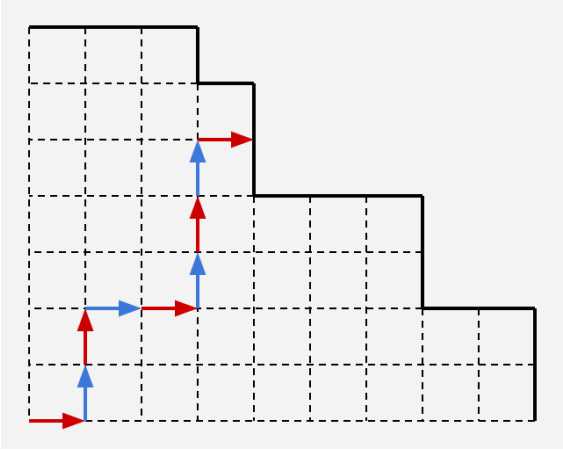


図 4　A game on young diagram

For each lattice point on the young diagram, write a 'o' if it's a winning state and write a 'x' if it's a losing state. First, write 'o' to each lattice point on the top-right boundary. For each other lattice point, write a 'x' if both of top-right adjacent lattice points are 'o', and otherwise write a 'o'. You can check the answer by seeing the bottom-left corner lattice. However, this solution works in $O(\sum a_i)$ and it's too slow.

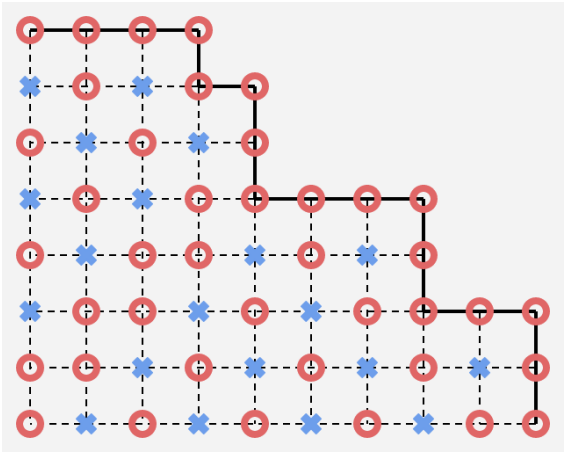


図 5　A diagram with ' ○' and ' × '

From the diagram above, you can see that in each diagonal the same symbol is written (except for the top-right boundary).

Proof: assume that the left-bottom corner is $(0, 0)$ and we introduce a coordinate system. If $(x + 1, y + 1)$ is inside the diagram and this is a losing state, both $(x, y + 1)$ and $(x + 1, y)$ will be winning states and thus (x, y) will be a losing state. Similarly, we can prove that when $(x+2, y+2)$ is inside the diagram and both $(x+2, y+2)$ and $(x+1, y+1)$ are winning states, (x, y) will be a winning state.

You can determine the symbol at the left-bottom corner as follows.

First, you start from the left-bottom corner and go to top-right diagonal one unit before the boundary. If the lengths of both arrows are even, the answer is 'x'. Otherwise the answer is 'o'. This way you can compute the answer in $O(N)$ time.

F : Leftmost Ball

When $K = 1$, the answer is obviously 1. Assume that $K \geq 2$. Also, assume that if we compare the leftmost occurrence of the colors $1, 2, ..., N$, the colors appear in this order. (We can get the answer by multiplying $N!$ to it.)

First, let's discuss how to determine if a given sequence of colors (0 to N) is valid or not. Of course, there must be exactly N occurrences of balls of color 0, and exactly $K - 1$ occurrences

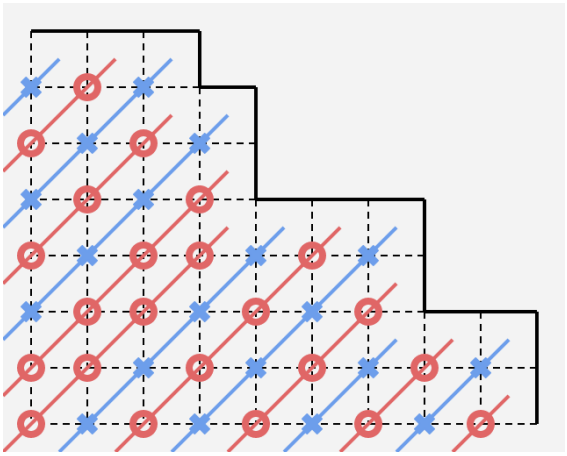


図 6　Diagonal ○ / ×

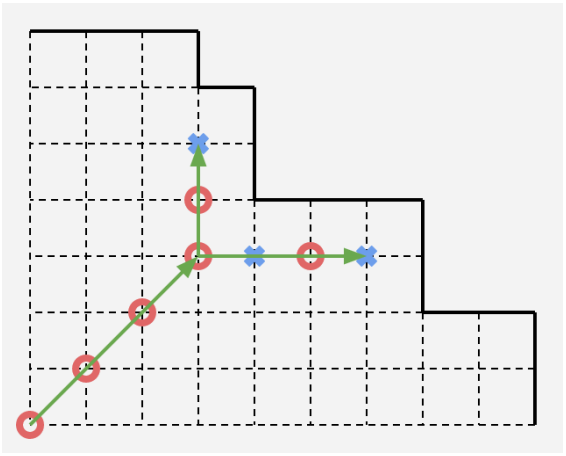


図 7　Compute the symbol at the origin quickly

each of colors $1, 2, ..., N$. Also, the following condition must be satisfied:

- For each $1 \leq i \leq N$, the i-th (from the left) ball of color 0 is to the left of the leftmost ball of color i.

For example, the sequence $(0, 1, 0, 2, 1, 2, \underline{3}, \underline{0}, 3)$ is invalid because the condition above is not satisfied for $i = 3$. On the other hand, we can prove that these conditions are sufficient. If we color the i-th ball of color 0 with color i, we can reconstruct the colors before the repainting to color 0.

Therefore, the problem can be restated as follows:

> You arrange N balls of color 0 and $K - 1$ balls each of colors $1, 2, ..., N$. For each $1 \leq i \leq N - 1$, you can put balls of color $i + 1$ only after you put at least one ball of color i. Also, for each $1 \leq i \leq N$, you can put balls of color i only after you put at least i balls of color 0. How many ways are there to arrange the balls?

The answer is the same as the number of topological orderings of the following graph: (The vertices on the top row corresponds to 1st, 2nd, ..., N-th ball of color 0 from let to right. The red vertices corresponds to 1s5, ..., $K - 1$-th ball of color 1 from top to bottom, and so on.)

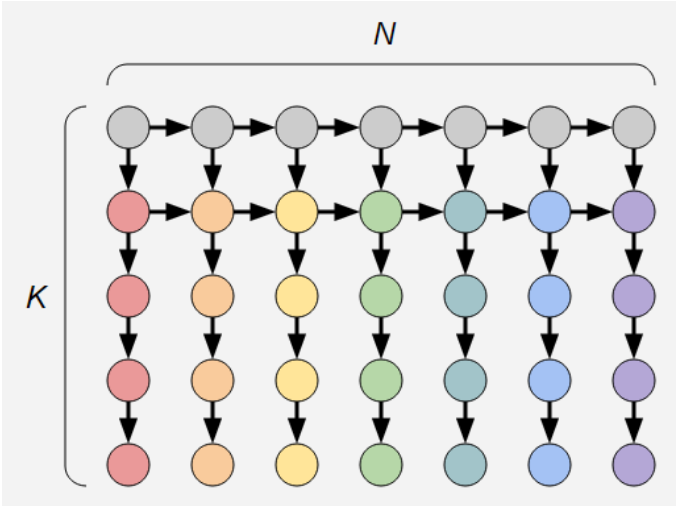


図 8　You count the number of topological orderings of this graph

The number of topolocial orderings can be computed using the following DP. Let $dp[i][j]$ be the number of topological orderings of the following graph.

There are two possible first vertex in a topolocial ordering: the bold black vertex or the bold orange vertex. When the bold black vertex comes first, the number of ways to order the remaining vertices is $dp[i-1][j]$. When the bold orange vertex comes first, the ordering of the remaining vertices can be decided by the topological ordering of non-orange vertices ($dp[i][j-1]$ ways) and the way you interleave orange vertices and non-orange vertices (${}_{i+j(K-1)-1}C_{K-2}$ ways because there are $K-2$ orange vertices and $i+j(K-1)-1$ vertices in total). Thus, the total number of topological orderings is ${}_{i+j(K-1)-1}C_{K-2} \cdot dp[i][j-1]$. (Here C denotes the binomial coefficient)

This way, we can compute the array $dp[][]$, and $dp[N][N]$ is the answer. The time complexity is $\mathrm{O}(N^2)$.

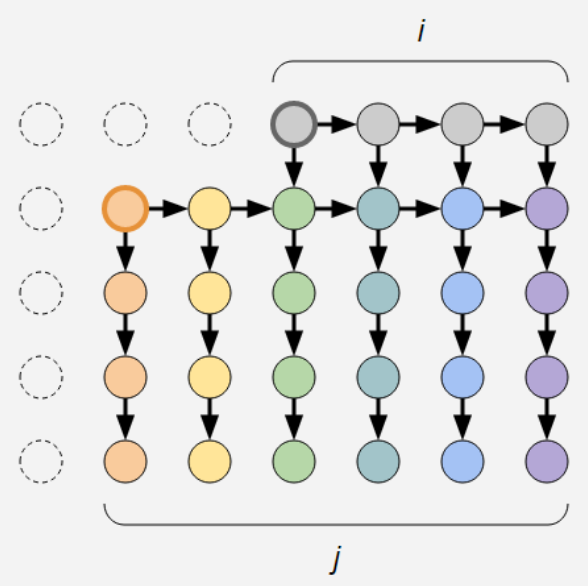


図 9　The graph corresponds to $dp[i][j]$

AtCoder Grand Contest 001 Editorial

July 16, 2016

A. BBQ Easy

N 個のペアを $(a_1, b_1), \ldots, (a_N, b_N)$ とすると，串にさせるものの個数は $min(a_1, b_1) + \cdots + min(a_N, b_N)$ となる．

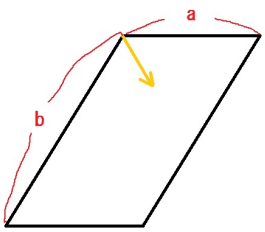
x を入力で与えられた数の最小とし，y が x とペアになっているとする．y の値によらず，この串にさせるものの個数は x となるので，他の串にさせるものの個数を最大化するために y は二番目に小さい数とするのが最適である．同様に，三番目と四番目，五番目と六番目，…をペアにするのが最適となるので，入力をソートして $L_1 \leq \cdots \leq L_{2N}$) とすると，答えは $min(L_1, L_2) + min(L_3, L_4) + \cdots + min(L_{2N-1}, L_{2N})$ となる．

B. Mysterious Light

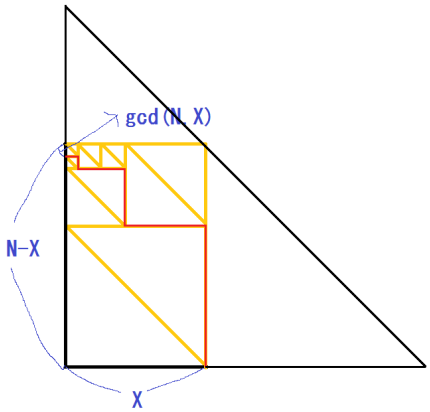
下図で $a * b$ の平行四辺形にこの光を使ったときの長さを $f(a,b)$ とする．

答えは $N + f(N - X, X)$ であり，$a < b$ のとき $f(a,b) = 2a + f(a, b-a)$ であることを使うと $O(N)$ の回答が得られ，300 点を取ることができる．

満点解法では最大公約数を求めるときに使うユークリッドの互助法と似たようなことをする．$a < b$ で b が a の倍数でないとき，$f(a,b) = 2 * floor(b/a) * a + f(a, b\%a)$ となる．これを使うと $O(logN)$ となり満点が得られる．



また，光の軌跡が三角形の集合となり，三角形の辺の長さの和が下図より $N - gcd(N,X)$ となることに気付くと，答えが $3(N - gcd(N,X))$ であることがわかる．



C. Shorten Diameter

木に関する以下の性質を用いる．

木 T の直径を D とする．

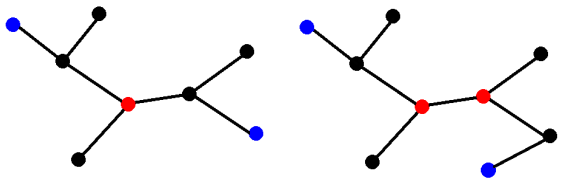
- D が偶数ならば，ある頂点 v が存在して v から他の頂点への距離が $D/2$ 以下となる．
- D が奇数ならば，ある辺 e が存在して e から他の頂点への距離が $(D-1)/2$ 以下となる．

v と e は木の中心とよばれる．

下図で，青い頂点は直径を表している．青い頂点の中点を赤とすると，赤が中心となることが分かる．

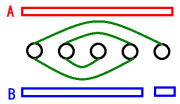
D が偶数のとき，中心の位置を N 通り全て試す．中心から距離 $D/2$ より離れている頂点を全て取り除けばよいので，各頂点から距離 $D/2$ 以上の頂点の個数を数え，その個数の最小値が答えとなる，

同様に，D が奇数のときは全ての辺を中心として試せばよい．この解答は $O(N^2)$ となる．

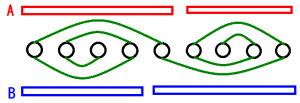


D. Arrays and Palindrome

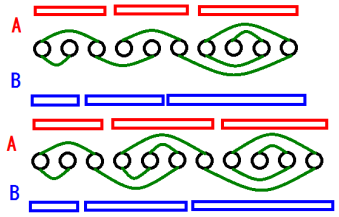
$M = 1$ の場合が最も簡単である．$a = \{x\}$ のとき $b = \{x - 1, 1\}$ とすればよい．



$M = 2$ のときも似たような構成が可能である．$a = \{x, y\}$ のとき $b = \{x - 1, y + 1\}$ とすればよい．



$M \geq 3$ のときも似た構成を試す．つまり，$a = \{x_1, \ldots, x_M\}$ のとき $b = \{x_1 - 1, x_2, \ldots, x_{M-1}, x_M + 1\}$ とする．これは中央の長さの偶奇により，偶数ならうまくいき，奇数ならうまくいかない．



a に奇数が三個以上含まれているとする．a に含まれる奇数の個数を O_a とすると，上図で二箇所を結んでいる辺の個数は $(N - O_a)/2$ となる．同様に，b に含まれる奇数の個数を O_b とすると，上図で二箇所を結んでいる辺の個数は $(N - O_b)/2$ となる．

全ての場所をつなぐためには少なくとも $N - 1$ 本の辺が必要なので，$(N - O_a)/2 + (N - O_b)/2 \geq N - 1$ であり， $O_a + O_b \leq 2$ となるが， これは $O_a > 2$ のとき不可能である．

逆に， a に含まれる個数が二個以下の場合は，奇数が両端になるように並べれば全ての中央の値は偶数となり，上の構成法でよい．

E. BBQ Hard

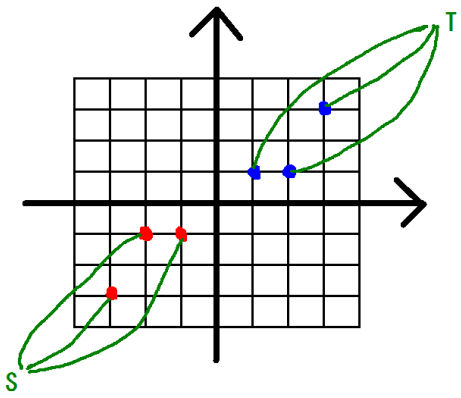
i 番と j 番の集合を選ぶと，串を作る方法は $f(A_i + A_j, B_i + B_j)$ 通りである．ここで $f(x, y) = (x + y)!/x!/y!$ は x 個の同じものと y 個の同じものを並べる方法の個数である．

次の和を計算すればよい:

$$\sum_{1 \le i < j \le N} f(A_i + A_j, B_i + B_j) \tag{1}$$

$f(x, y)$ はグリッド上で $(0, 0)$ から (x, y) への経路数なので，$f(A_i + A_j, B_i + B_j)$ は $(-A_i, -B_i)$ から (A_j, B_j) への経路数となる．

下図で， 点 $(-A_i, -B_i)$ (赤) と点 (A_i, B_i) (青) をプロットしている．赤い点のうちの一つから青い点のうちの一つへの経路の個数を求めるとほとんど答えとなる．



X をその経路数とすると，

$$X = \sum_{1 \le i, j \le N} f(A_i + A_j, B_i + B_j) \tag{2}$$

となるので，答えは

$$\sum_{1\leq i<j\leq N} f(A_i+A_j, B_i+B_j) = (X - \sum_i f(A_i+A_i, B_i+B_i))/2 \tag{3}$$

である．

X は図で S から T への経路数であるので，$O(MAX^2)$ で単純な DP によりもとめられる．f も $O(1)$ でもとめられるので，この解法は $O(MAX^2+N)$ となる．

F. Wide Swap

Q を P の逆の配列とする ($Q_{P_i} = i$ をみたす) と，問題は次のように書き換えられる．

Problem. 順列 Q が与えられる．$|Q_i - Q_j| \geq K$ のとき Q_i と Q_{i+1} をスワップすることができる．1 がなるべく左の場所にあるようにしたい．そのようなな順列のうち，2 がなるべく左にあるようにしたい．そのようなな順列のうち，3 がなるべく左にあるようにしたい…としたときの順列を求めよ．

このとき，Q を別の順列 R に変えることができるためには，任意の $|x-y| < K$ をみたす x, y に対し，Q で x が y の左にあれば，R でも x が y の左になければならない．

逆に，任意の (x, y) に対しこの条件を満たすとき，Q と R の間の転倒数を減らすようなスワップを繰り返すことで Q を R にすることができる．

次の問題を解けばよい：

Problem. 順列 P が与えられる．N 頂点のグラフを作り，$|i-j| < K$ かつ $P_i < P_j$ のとき i から j への辺をはる．このグラフの辞書順最小の topological なラベル付けを求めよ．(s から t への変があるとき，$label(s) < label(t)$ を満たさなければならない．)

一般の DAG に対し，辞書順最小の topological なラベル付けを求めることができる．

ラベル N のつく頂点の出次数は 0 でなければならない．そのような頂点が複数個ある場合は，最も右にある出次数 0 の頂点 (r とする) にラベル N をつければよいことが証明できる．

r のラベルを $x < N$ とすると，r のラベルを N に変えラベル $x+1, \ldots, N$ を $x, \ldots, N-1$ に変えることで辞書順に小さくできるので矛盾する．

よって，次のようにすればよい．

- $k = N$ とする．
- 出次数 0 の頂点のうち最も右にあるものを選び，ラベル k をつける．
- その頂点を取り除き，k を 1 減らして繰り返す．

i 番目の要素を，$P_{i-K+1}, \ldots, P_{i_K-1}$ の中で最大値であるとき極大であると呼ぶことにすると，(出次数が 0 であることに対応する)

$k = N, \ldots, 1$ に対し

- 順列の中で最も右にある極大な要素を見つける．
- その場所に k と書き，その要素を取り除く．

とすればよい．

これは $O(N^2)$ となるので高速化する必要がある．

まず，配列を長さ K ずつのバケットに分割しておく．極大な値はそのバケットの中で必ず最大となっているので，次のような方法でこの問題を解くことができる．

まず，Range Minimum Query (区間の最大値と一点の値の更新を $O(logN)$ でする) を用意しておく．これにより，ある値が極大であるかどうかの判定を $O(logN)$ でできる．

次に，各バケットに対し最大値を求めておき，それが極大である場合は priority queue にいれておく．この priority queue は極大値の集合を含む．

$k = N, \ldots, 1$ に対し，

- priority queue の先頭を取り出しラベル k をつける．
- この要素を削除する．削除によりこのバケットと左右のバケットの三つが影響される可能性がある．
- その三つのバケットに対し，最大値を再び求め，極大である場合は priority queue に追加する．

このアルゴリズムの計算量は $O(NlogN)$ である．

AtCoder Grand Contest 001 Editorial

July 16, 2016

A. BBQ Easy

In this task, you need to divide the given numbers into N pairs. If the pairs are $(a_1, b_1), \ldots, (a_N, b_N)$, the score is $min(a_1, b_1) + \cdots + min(a_N, b_N)$, and your objective is to maximize this score.

Suppose that x is the minimum number among the input, and y is paired with x. Regardless of y, the score of this pair is always x, so y should be the second smallest number in order to maximize the score from other pairs. Thus, the smallest number should be paired with the second smallest number. Similarly, the third smallest number (the minimum after excluding the first pair) should be paired with the fourth, the fifth should be paired with the sixth, ... and so on.

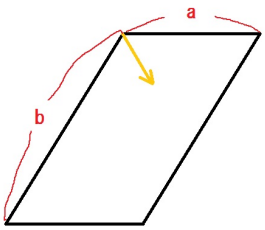
In summary, the solution is as follows: First, let's sort the input numbers in ascending order (i.e., $L_1 \leq \cdots \leq L_{2N}$). Then the answer is $min(L_1, L_2) + min(L_3, L_4) + \cdots + min(L_{2N-1}, L_{2N})$.

B. Mysterious Light

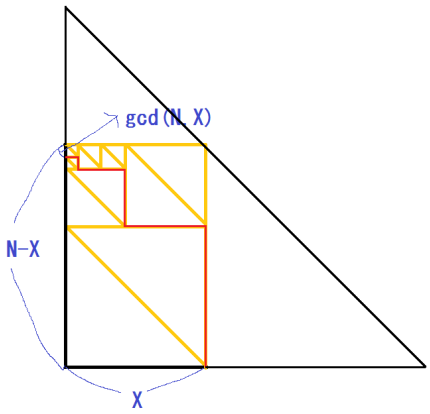
Let $f(a, b)$ be the length of the trajectory when the ray is used in the following parallelogram.

The answer is $N + f(N - X, X)$, and when $a < b$ we can see that $f(a, b) = 2a + f(a, b - a)$. This will lead to an $O(N)$ solution and you can get 300 points.

In order to get the full score, you need to do something similar to Euclid's algorithm of gcd. When $a < b$ and b is not divisible by a, $f(a, b) = 2 * floor(b/a) * a + f(a, b\%a)$. This will lead to an $O(logN)$ solution.



Actually, there is an even simpler solution. The trajectory is a set of triangles, and from the picture below you can see that the sum of lengths of sides is $N - gcd(N, X)$, so the answer is simply $3(N - gcd(N, X))$.



C. Shorten Diameter

We use the following well-known fact about trees.

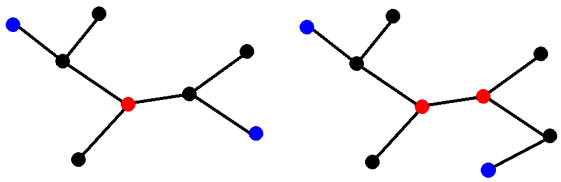
Let T be a tree, and let D be the diameter of the tree.

- If D is even, there exists an vertex v of T such that for each vertex w in T, the distance between w and v is at most $D/2$.
- If D is odd, there exists an edge e of T such that for each vertex w in T, the distance between w and one of the endpoints of e is at most $(D-1)/2$.

Here v and e are called *centers* of the tree.

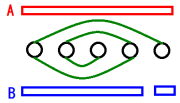
The proof of this fact is not very hard. See the picture below. The blue vertices are the endpoints of the diameters, and the red vertex (or edge) is in the middle of the diameter. This red vertex is the center of the tree; if there is a vertex v such that $dist(v, red) > D/2$, the distance between v and one of blue points will be more than D (because the distance between the red point and each blue point is $D/2$). The proof for odd case is similar.

Now the problem can be solved in the following way (we only describe the solution for the even case, but the odd case is similar). Choose a vertex x in the tree (this will be the center after removal of vertices) and count the number of vertices y such that $dist(x, y) > D/2$. If we remove all such y, the diameter of the remaining graph will be at most D. Thus, we can try all N vertices as x and the answer is the minimum count of such y. The solution works in $O(N^2)$.

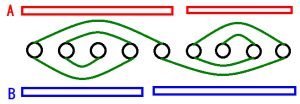


D. Arrays and Palindrome

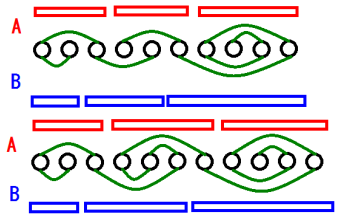
The easiest case is $M = 1$. If $a = \{x\}$ and $b = \{x - 1, 1\}$, the sequences satisfy the condtion as the following picture shows:



When $M = 2$, similar construction works. If $a = \{x, y\}$, choose $b = \{x - 1, y + 1\}$.



What happens when $M \geq 3$ if we construct b in the same manner? I.e., for $a = \{x_1, \ldots, x_M\}$, choose $b = \{x_1 - 1, x_2, \ldots, x_{M-1}, x_M + 1\}$. It depends on the parity of the length of the segment in the middle. When this is even, it works, but when this is odd, it doesn't work.



So, the problem looks difficult when there are many odd elements in a.

When a contains more than two odd numbers, we can prove that there is no solution. Suppose that the sequence a contains O_a odd numbers. Then, the number of arcs that connect two positions is $(N - O_a)/2$. Similarly, if b contains O_b odd numbers, the number of arcs is $(N - O_b)/2$.

In order to satisfy the condition, the arcs must connect all positions. This is the same as connecting N vertices of a graph - and we need at least $N - 1$ arcs.

Thus, $(N - O_a)/2 + (N - O_b)/2 \geq N - 1$, or $O_a + O_b \leq 2$ must be satisfied, but this is impossible when $O_a > 2$.

When a contains at most two odd numbers, we can shuffle the elements of a such that all odd numbers are at the leftmost or the rightmost position. Then the construction we saw above will work.

E. BBQ Hard

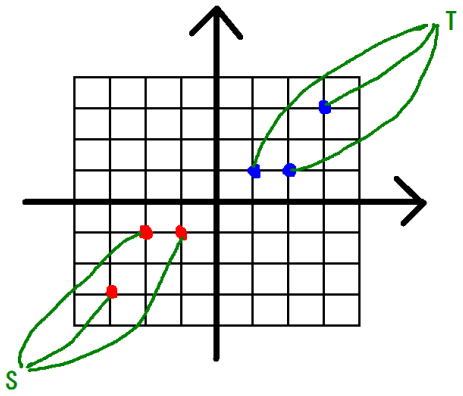
When we choose the i-th pack and the j-th pack, we get $A_i + A_j$ pieces of beef and $B_i + B_j$ pieces of green pepper. There are $f(A_i + A_j, B_i + B_j)$ ways to make a skewer meal using these packs, where $f(x, y) = (x + y)!/x!/y!$ (the number of ways to arrange x identical items and y identical items).

Thus, we want to compute the following sum:

$$\sum_{1 \leq i < j \leq N} f(A_i + A_j, B_i + B_j) \tag{1}$$

The key observation in this task is to find another combinatorial way to define $f(x, y)$. This is the number of ways to go from $(0, 0)$ to (x, y) in a grid. More importantly, $f(A_i + A_j, B_i + B_j)$ is the number of ways to go from $(-A_i, -B_i)$ to (A_j, B_j).

In the following picture, we plot the points $(-A_i, -B_i)$ (red) and the points (A_i, B_i) (blue). If we compute the number of paths from one of red points to one of blue points, we almost get the answer. (In the grid, black edges can be passed only to the right or to the up.)



We need a slight modification. Let X be the number of paths from one of red points to one of blue points, then

$$X = \sum_{1 \le i,j \le N} f(A_i + A_j, B_i + B_j) \tag{2}$$

The answer is

$$\sum_{1 \le i < j \le N} f(A_i + A_j, B_i + B_j) = (X - \sum_i f(A_i + A_i, B_i + B_i))/2 \tag{3}$$

The value of X is the number of paths from S to T in the picture, so it can be computed in $O(MAX^2)$ (where MAX is the maximum coordinate) using a simple DP. The function f can be compute in O(1) with proper pre-calculation. In total, the solution works in $O(MAX^2 + N)$.

F. Wide Swap

Let Q be the inverse of the array P. That is, for each i, $Q_{P_i} = i$. The problem can be restated as follows:

Problem. You are given a permutation Q. The two adjacent elements Q_i and Q_{i+1} can be swapped when $|Q_i - Q_j| \geq K$. You want to move 1 to the leftmost possible position. In case of tie, you want to move 2 to the leftmost possible position. In case of tie, ... and so on. What is the final array?

First, in this version, let's check whether the array Q can be changed into another given array R. Let x and y be two distinct numbers such that $|x-y| < K$. x and y can never be swapped, so if x is to the left of y in Q, x must be to the left of y in R too.

On the other hand, if the relative positions between all such pairs (x, y) are the same in Q and R, we can always change Q into R. This is because when Q and R satisfy this condition and Q and R are not the same, we can always swap two adjacent numbers in Q and reduce the inversion number, and Q will become R after finite number of swaps. (The inversion number is the number of pairs (s, t) such that s is to the left of t in Q but s is to the right of t in R).

Thus, the original problem is equivalent to the following:

Problem. You are given a permutation P. Construct a graph with N vertices, and add an edge from i to j if $|i - j| < K$ and $P_i < P_j$. Find the lexicographically smallest topological labelling of this graph. (In topological labelling, if there is an edge from s to t, $label(s) < label(t)$ must be satisfied.)

There is a general algorithm for finding the smallest topological labelling.

First, determine the vertex that is labelled with N. The out-degree of this vertex must be zero. When there are multiple such vertices, any of those vertices can be labelled with N to find a topological labelling. We can prove that in the lexicographically smallest one, the rightmost vertex among those vertices (let's call it r) should be labelled with N.

Suppose that r is labelled with x. Then, change the label of r to N and change the labels of vertices that are currently labelled as $x + 1, \ldots, N$ to $x, \ldots, N - 1$. This gives a lexicographically smaller topological labelling, so we get a contradiction.

Thus, the following algorithm works:

- Let $k = N$.
- Find the rightmost vertex with out-degree zero, and label it k.
- Remove the vertex.
- Decrement k and go to the second step.

Now, the problem becomes even simpler.

The element at position i is called *local maximum* if P_i is the biggest among $P_{i-K+1}, \ldots, P_{i_K-1}$. (This corresponds to the condition that the out-degree must be zero.)

For each $k = N, \ldots, 1$,

- Find the rightmost local maximum element.
- Label it with k.
- Remove the element.

The straightforward implementation of this algorithm will lead to $O(N^2)$ solution, and we need an efficient data structure to find local maximum elements quickly.

Let's divide the array into buckets. The first bucket contains positions $0, \ldots, K-1$, the second bucket contains positions $K, \ldots, 2K-1$, and so on. Notice that each local maximum element must be the maximum in its bucket.

Thus, the following algorithm works.

First, construct a data structure that supports range minimum query. That is, we can perform the following two types of operations in $O(logN)$:

- For a given position p and a given number x, update the value at position p to x.
- For a given range $[L, R]$, compute the maximum in the indices $[L, R]$.

With this data structure, we can check if a given element is a local maximum or not in $O(logN)$.

Then, for each bucket compute the maximum, and in case it is a local maximum, push it in a priority queue. This priority queue contains the set of local maximum elements.

For each $k = N, \ldots, 1$,

- Pop an element from the priority queue and label this with k.
- Remove this element. Note that the removal of this element affects at most three buckets: the bucket that contains this element and two adjacent buckets.
- For each of those three buckets, recompute the maximum, and in case it is a local maximum, push it into the priority queue.

The time complexity of this solution is $O(NlogN)$.